



Fakultät für Informatik und Automatisierung  
Data-intensive Systems and Visualization Group

# Master Thesis

## Robustness and Adaptability in Multi-Agent Reinforcement Learning: A Unified Taxonomy Toward a Shared Benchmark

**Registration Date:** [FILL IN]

**Submission Date:** [FILL IN]

**Academic Chair:** Prof. Dr.-Ing. Patrick Mäder

**Academic Supervisor:** [FILL IN]

**Submitted by:** Abdullah Alalwani

born [FILL IN]

in [FILL IN]

abdullah.alwani@hotmail.com

**Course of Studies:** [FILL IN]



# Acknowledgments

[FILL IN: acknowledgements text]



# Abstract

This thesis is a taxonomic survey: it builds a single connected account of two MARL literatures and compares them directly, and it reports no new experimental results of its own. A multi-agent reinforcement learning (MARL) policy that performs well in its training simulator is not automatically useful once it leaves that simulator. Two things go wrong that never showed up in training. First, the world is not the simulator: sensors are noisy, a teammate can fail or be attacked, and the assumed dynamics are only an approximation of the real ones. Second, the world does not hold still: traffic patterns shift over the day, teammates keep updating their own policies, and equipment degrades after deployment. The first problem is answered in the literature by *robust* MARL, which trains a policy to survive a bounded, worst-case disturbance fixed in advance. The second is answered by *adaptive* MARL, which tracks a disturbance that keeps changing after deployment, without knowing the change in advance. This thesis argues that both are responses to the same underlying problem, non-stationarity, either imposed by an adversary or arising naturally as agents and environments evolve, and builds a single connected account of the two literatures around that argument. Four research questions organize the work: what sources of uncertainty threaten deployed MARL policies, what mechanisms make policies robust to a bounded disturbance, what mechanisms let policies adapt to an unbounded one, and how the two families relate. The thesis answers them by building a taxonomy from adversarial training and adversarial regularization through distributionally robust value factorization and sim-to-real transfer on the robust side, and from change-point detection and sliding-window estimation through meta-reinforcement learning task inference on the adaptive side, then comparing the two directly on objective, mechanism, and failure mode. The comparison exposes four concrete gaps that block evaluating robust and adaptive MARL methods on common ground: mismatched benchmarks, mismatched metrics, sim-to-real robustness that is never tested against continued post-deployment drift, and an almost complete absence of multi-agent credit assignment inside adaptive mechanisms. Closing these gaps, not surveying further methods, is identified as the concrete next step toward a benchmark that operationalizes robustness and adaptability together in dynamic, simulated environments mapped to real-world task classes.



# Contents

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                         | <b>5</b>  |
| <b>1. Introduction</b>                                                  | <b>1</b>  |
| 1.1. Motivation . . . . .                                               | 1         |
| 1.2. Robustness and Adaptability as One Problem . . . . .               | 1         |
| 1.3. Research Questions . . . . .                                       | 2         |
| 1.4. Thesis Structure . . . . .                                         | 3         |
| 1.5. Scope of Real-World Relevance . . . . .                            | 3         |
| <b>2. Methodology</b>                                                   | <b>5</b>  |
| 2.1. Scope . . . . .                                                    | 5         |
| 2.2. Selection Criteria . . . . .                                       | 6         |
| 2.3. Out of Scope . . . . .                                             | 8         |
| 2.4. How Each Research Question Is Answered . . . . .                   | 8         |
| <b>3. Overview of Deep Reinforcement Learning Algorithms</b>            | <b>9</b>  |
| 3.1. Markov Decision Processes and the Reinforcement Learning Objective | 9         |
| 3.2. Value-Based Methods: Deep Q-Networks . . . . .                     | 10        |
| 3.3. Policy Gradient Methods . . . . .                                  | 11        |
| 3.4. Trust-Region and Clipped Policy Optimization . . . . .             | 12        |
| 3.4.1. Trust Region Policy Optimization (TRPO) . . . . .                | 13        |
| 3.4.2. Proximal Policy Optimization (PPO) . . . . .                     | 13        |
| 3.5. Actor-Critic Methods . . . . .                                     | 14        |
| 3.5.1. Deterministic Policy Gradient (DPG) . . . . .                    | 15        |
| 3.5.2. Deep Deterministic Policy Gradient (DDPG) . . . . .              | 16        |
| 3.5.3. Twin Delayed Deep Deterministic Policy Gradient (TD3) . . . . .  | 17        |
| 3.5.4. Soft Actor-Critic (SAC) . . . . .                                | 18        |
| 3.6. Selection of Algorithms for Multi-Agent Extension . . . . .        | 19        |
| <b>4. Fundamentals of Multi-Agent Reinforcement Learning</b>            | <b>21</b> |
| 4.1. Multi-Agent Reinforcement Learning . . . . .                       | 21        |
| 4.1.1. Stochastic Games . . . . .                                       | 21        |
| 4.1.2. Partial Observability . . . . .                                  | 22        |
| 4.1.3. Joint Policies and Expected Return . . . . .                     | 22        |

|           |                                                                                                                 |           |
|-----------|-----------------------------------------------------------------------------------------------------------------|-----------|
| 4.2.      | Game-Theoretic Foundations . . . . .                                                                            | 23        |
| 4.2.1.    | Normal-Form Games and Classes of Games . . . . .                                                                | 23        |
| 4.2.2.    | Best Response and Minimax Solutions . . . . .                                                                   | 23        |
| 4.2.3.    | Nash Equilibrium . . . . .                                                                                      | 24        |
| 4.2.4.    | Correlated and Coarse Correlated Equilibrium . . . . .                                                          | 24        |
| 4.2.5.    | Pareto Optimality and Social Welfare . . . . .                                                                  | 25        |
| 4.2.6.    | Limitations of Equilibrium Solutions . . . . .                                                                  | 25        |
| 4.3.      | Learning in Multi-Agent Systems . . . . .                                                                       | 25        |
| 4.3.1.    | Independent Learning . . . . .                                                                                  | 25        |
| 4.3.2.    | Non-Stationarity in MARL . . . . .                                                                              | 26        |
| 4.3.3.    | No-Regret Learning . . . . .                                                                                    | 26        |
| 4.4.      | Baseline MARL Algorithms . . . . .                                                                              | 27        |
| 4.4.1.    | Q-Learning-Based MARL and Value Decomposition . . . . .                                                         | 27        |
| 4.4.2.    | Policy-Gradient MARL . . . . .                                                                                  | 29        |
| 4.4.3.    | Actor-Critic MARL . . . . .                                                                                     | 30        |
| 4.4.4.    | Centralized Training with Decentralized Execution . . . . .                                                     | 34        |
| 4.5.      | Selection of Baseline Algorithms . . . . .                                                                      | 35        |
| <b>5.</b> | <b>Robust Multi Agent Reinforcement Learning</b>                                                                | <b>37</b> |
| 5.1.      | Definition of Robustness in Multi-Agent Reinforcement Learning . . . .                                          | 38        |
| 5.2.      | Sources of Uncertainty in Multi-Agent Reinforcement Learning . . . .                                            | 39        |
| 5.2.1.    | Observation and Action Noise . . . . .                                                                          | 39        |
| 5.2.2.    | Non-Stationary Environments . . . . .                                                                           | 40        |
| 5.2.3.    | Communication Failures . . . . .                                                                                | 41        |
| 5.2.4.    | Adversarial Attacks . . . . .                                                                                   | 41        |
| 5.3.      | Taxonomy of Robust Multi-Agent Reinforcement Learning . . . . .                                                 | 44        |
| 5.4.      | Adversarial Training . . . . .                                                                                  | 45        |
| 5.4.1.    | Robust Adversarial Reinforcement Learning (RARL) . . . . .                                                      | 46        |
| 5.4.2.    | Minimax Multi-Agent Deep Deterministic Policy Gradient (M3DDPG) . . . . .                                       | 48        |
| 5.5.      | Adversarial Regularization . . . . .                                                                            | 51        |
| 5.5.1.    | Efficient Robust Networked Multi-Agent Reinforcement Learning with Adversarial Regularization (ERNIE) . . . . . | 52        |
| 5.5.2.    | Robust Multi-Agent Reinforcement Learning by Mutual Information Regularization (MIR3) . . . . .                 | 54        |
| 5.5.3.    | State-Adversarial Policy Regularization (SA-MDP) . . . . .                                                      | 57        |
| 5.5.4.    | Certified Robustness via Interval Bound Propagation (RADIAL-RL) . . . . .                                       | 59        |
| 5.6.      | Distributionally Robust Multi-Agent Reinforcement Learning . . . . .                                            | 61        |
| 5.6.1.    | Distributionally Robust Independent Global Maximization (DrIGM) . . . . .                                       | 62        |
| 5.7.      | Sim-to-Real Robustness . . . . .                                                                                | 66        |
| 5.7.1.    | Adversarial Domain Randomization . . . . .                                                                      | 67        |



|                                                                                               |            |
|-----------------------------------------------------------------------------------------------|------------|
| 5.7.2. Prioritized Experience Replay for Domain Randomization Environments (PERDRE) . . . . . | 69         |
| <b>6. Adaptive Multi-Agent Reinforcement Learning in Non-Stationary Environments</b>          | <b>79</b>  |
| 6.1. Mathematical Foundations of Non-Stationary Reinforcement Learning                        | 80         |
| 6.1.1. Non-Stationary Markov Decision Processes . . . . .                                     | 80         |
| 6.1.2. Bellman Equation in Non-Stationary Environments . . . . .                              | 81         |
| 6.1.3. Variation Budget . . . . .                                                             | 82         |
| 6.1.4. Diameter of an MDP. . . . .                                                            | 82         |
| 6.1.5. Successor Representation and Successor Features . . . . .                              | 83         |
| 6.1.6. Low-Rank Markov Decision Processes. . . . .                                            | 87         |
| 6.2. Types of Non-Stationary Environments . . . . .                                           | 88         |
| 6.2.1. Slowly Varying Environments . . . . .                                                  | 88         |
| 6.2.2. Piecewise Stationary Environments . . . . .                                            | 89         |
| 6.2.3. Switching Non-Stationary Markov Decision Processes . . . . .                           | 90         |
| 6.2.4. Dynamic Parameter Markov Decision Processes . . . . .                                  | 91         |
| 6.2.5. Adversarial Environments . . . . .                                                     | 91         |
| 6.3. Performance Objectives . . . . .                                                         | 92         |
| 6.3.1. Dynamic Regret . . . . .                                                               | 92         |
| 6.3.2. Average Dynamic Suboptimality Gap . . . . .                                            | 93         |
| 6.4. Taxonomy of Adaptive Reinforcement Learning Methods . . . . .                            | 94         |
| 6.4.1. Change-Point Detection . . . . .                                                       | 94         |
| 6.4.2. Sliding Window Methods . . . . .                                                       | 116        |
| 6.4.3. Factored and Causal Adaptation . . . . .                                               | 123        |
| 6.4.4. Factored and Hierarchical Adaptation . . . . .                                         | 126        |
| <b>7. Meta-Reinforcement Learning as an Adaptation Mechanism</b>                              | <b>135</b> |
| 7.1. Learning to Learn . . . . .                                                              | 136        |
| 7.2. Problem Formulation . . . . .                                                            | 137        |
| 7.2.1. From Reinforcement Learning to Meta-Reinforcement Learning                             | 137        |
| 7.3. Meta-Reinforcement Learning as a Partially Observable Decision Process . . . . .         | 140        |
| 7.4. Few-Shot and Many-Shot Meta-Reinforcement Learning . . . . .                             | 141        |
| 7.5. Categories of Meta-Reinforcement Learning . . . . .                                      | 142        |
| 7.6. Policy Parameter Gradient Methods . . . . .                                              | 144        |
| 7.6.1. Learning a Good Initialization . . . . .                                               | 145        |
| 7.7. Black-Box Meta-Reinforcement Learning . . . . .                                          | 146        |
| 7.8. Task Inference Methods . . . . .                                                         | 154        |
| 7.8.1. Theoretical Foundations: Approximate Information State . . .                           | 156        |
| 7.8.2. Variational Bayesian Belief Methods . . . . .                                          | 162        |

|                                                                                                  |                |
|--------------------------------------------------------------------------------------------------|----------------|
| 7.9. Centralized Training with Decentralized Execution for Meta-Reinforcement Learning . . . . . | 173            |
| 7.9.1. CTDE on Top of Policy Parameter Gradient Methods . . . . .                                | 173            |
| 7.9.2. CTDE on Top of Black-Box Methods . . . . .                                                | 174            |
| 7.9.3. CTDE on Top of Task Inference Methods: Meta Global-to-Local (MG2L) . . . . .              | 175            |
| <b>8. Robustness and Adaptability: A Unified Synthesis</b>                                       | <b>183</b>     |
| 8.1. A Shared Root Cause . . . . .                                                               | 183            |
| 8.2. Where the Two Taxonomies Already Touch . . . . .                                            | 184            |
| <b>9. Contributions</b>                                                                          | <b>187</b>     |
| <b>10. Discussion and Future Work</b>                                                            | <b>195</b>     |
| 10.1. No Shared Benchmark . . . . .                                                              | 195            |
| 10.2. No Shared Metric . . . . .                                                                 | 195            |
| 10.3. Sim-to-Real Robustness Is Not Post-Deployment Adaptability . . . . .                       | 196            |
| 10.4. Multi-Agent Credit Assignment Is Missing From Adaptive Mechanisms                          | 196            |
| 10.5. The Concrete Next Step . . . . .                                                           | 196            |
| <b>11. Conclusion</b>                                                                            | <b>197</b>     |
| <b>A. Extended Method Catalogue: Meta-Reinforcement Learning</b>                                 | <b>I</b>       |
| A.1. Additional Policy-Parameter-Gradient Methods . . . . .                                      | I              |
| A.1.1. Learning the Optimization Process . . . . .                                               | V              |
| A.1.2. Learned Optimizers . . . . .                                                              | XVI            |
| A.1.3. Improving Meta-Gradient Estimation . . . . .                                              | XXII           |
| A.1.4. No DICE . . . . .                                                                         | XXXII          |
| A.1.5. Learning Exploration Strategies . . . . .                                                 | XLVI           |
| A.2. Additional Black-Box Methods . . . . .                                                      | LIV            |
| A.2.1. Attention-Based Adaptation Mechanisms . . . . .                                           | LIV            |
| A.2.2. Adaptation via External Memory . . . . .                                                  | LXII           |
| A.2.3. Adaptation via Self-Modifying Weights . . . . .                                           | LXXXV          |
| A.2.4. Training Difficulties and Limitations of Black-Box Methods .                              | CVIII          |
| A.3. Additional Task-Inference Methods . . . . .                                                 | CXI            |
| A.3.1. Decoupled Exploration for Task Inference . . . . .                                        | CXVI           |
| A.3.2. Discriminative and Self-Supervised Representations . . . . .                              | CXXXV          |
| A.3.3. Direct Parameter and Value Regression . . . . .                                           | CXLIII         |
| A.3.4. Exchangeable Models . . . . .                                                             | CLIV           |
| A.3.5. Validating Learned Belief Representations . . . . .                                       | CLX            |
| A.3.6. Synthesis and Comparison . . . . .                                                        | CLXVI          |
| <b>B. Contrastive Log-ratio Upper Bound (CLUB)</b>                                               | <b>CLXXVII</b> |

|                                                                  |                 |
|------------------------------------------------------------------|-----------------|
| <b>C. Total Variation Distance</b>                               | <b>CLXXXI</b>   |
| <b>D. Exponential Family Distributions</b>                       | <b>CLXXXIII</b> |
| D.1. Definition . . . . .                                        | CLXXXIII        |
| D.2. Maximum Likelihood Estimation . . . . .                     | CLXXXIV         |
| D.3. Conjugate Prior . . . . .                                   | CLXXXIV         |
| D.4. Posterior Distribution . . . . .                            | CLXXXV          |
| <b>E. <math>n</math>-Mode Tensor Product</b>                     | <b>CLXXXVII</b> |
| <b>F. Stochastic Computation Graphs</b>                          | <b>CLXXXIX</b>  |
| F.1. Graph Definition . . . . .                                  | CLXXXIX         |
| F.2. Differentiability Requirements . . . . .                    | CXC             |
| F.2.1. Gradient Estimation . . . . .                             | CXC             |
| <b>G. Background: Contrastive Learning and Momentum Contrast</b> | <b>CXCIII</b>   |
| <b>H. Use of Generative AI Tools</b>                             | <b>CXCVII</b>   |
| <b>List of Tables</b>                                            | <b>CXCIX</b>    |
| <b>List of Figures</b>                                           | <b>CCI</b>      |
| <b>Bibliography</b>                                              | <b>CCX</b>      |



---

# 1. Introduction

## 1.1. Motivation

Consider a team of autonomous vehicles driving through a city. Sensors may produce noisy measurements because of rain or fog. Communication between vehicles may be delayed or interrupted. One vehicle may fail, while another encounters an obstacle that changes the traffic flow. Similar challenges appear in robot teams, wireless communication networks, and autonomous drone swarms. Since the agents depend on each other's decisions, even a small disturbance can spread through the system and reduce the overall performance.

Deploy a MARL policy on any of these systems and two problems appear that never showed up in the training simulator. The first is that the world is not the simulator: sensors are noisy, a teammate can be attacked or simply fail, and the dynamics assumed during training are only an approximation of the physical system. The second is that the world does not stay put: traffic patterns shift over the day, teammates keep updating their own policies, and equipment degrades. The first problem is a robustness problem. The second is an adaptability problem. Both are usually studied separately, by separate communities, using separate benchmarks and separate metrics.

This split is a genuine obstacle for anyone trying to deploy MARL on a real task. A policy that resists a fixed perturbation budget but cannot track a slowly drifting environment will eventually be pushed outside that budget and fail anyway. A policy that adapts quickly to benign drift but has no protection against an adversarial teammate or a sensor attack will be exploited the moment an adversary shows up. A system built for real deployment needs both properties, evaluated together, not as two disconnected literatures.

## 1.2. Robustness and Adaptability as One Problem

This thesis treats robustness and adaptability as two responses to one problem: non-stationarity. Even in a MARL system with no adversary and no changing physics, the environment is non-stationary from the point of view of any single learning agent,

because every other agent is also updating its policy at the same time. Robust MARL assumes that whatever pushes the system away from its training conditions is bounded and unknown in advance, and prepares for the worst case inside that bound before deployment. Adaptive MARL assumes the departure can grow without bound but follows some exploitable structure over time, slow drift, abrupt regime changes, recurring conditions, and spends its effort tracking that structure online, after deployment, rather than preparing for its worst instantiation.

Framing the two families this way does more than provide a tidy label. It means a method from one family can be read and evaluated using the vocabulary of the other: a distributionally robust method’s ambiguity set is mechanically close to an adaptive method’s confidence set, and a meta-learned task encoder trained across a distribution of tasks is one gradient step away from being trained across a distribution of adversarially perturbed tasks. Chapter 8 makes this comparison directly. The rest of the thesis builds the two taxonomies that make it possible.

## 1.3. Research Questions

This thesis is organized around four questions.

- **RQ1.** What sources of uncertainty and non-stationarity threaten MARL policies once they leave a fixed training simulator, and how does the literature classify them?
- **RQ2.** What mechanisms have been proposed to make MARL policies robust to a bounded, unknown disturbance, and what do they trade off against each other?
- **RQ3.** What mechanisms have been proposed to let a MARL policy adapt online when the environment or the other agents change after deployment, and what do they trade off against each other?
- **RQ4.** How do robustness and adaptability relate mechanically, and what would it take to evaluate methods from both families in the same dynamic, simulated environment, on the same real-world task class?

RQ1 is answered across the sources-of-uncertainty material woven through Chapters 3 and 4 and the Robust MARL chapter (Chapter 5). RQ2 is answered by the Robust MARL chapter in full. RQ3 is answered jointly by the Adaptive MARL chapter and the meta-reinforcement-learning chapter that follows it, since meta-RL is treated here specifically as an adaptation mechanism rather than as a general-purpose learning paradigm. RQ4 is answered in Chapter 8. Chapters 9 and 10 then state what this synthesis contributes and what it leaves open.

## 1.4. Thesis Structure

Chapter 2 explains how this thesis was built: what counted as in scope, what was deliberately left out, and why. Chapter 3 gives the single-agent reinforcement learning background the rest of the thesis depends on, and Chapter 4 extends it to the multi-agent formalism, game-theoretic solution concepts, and CTDE baselines that later chapters build on directly. Chapter 5 develops robust MARL: sources of uncertainty, adversarial training, adversarial regularization, distributionally robust value factorization, and sim-to-real transfer. Chapter 6 develops adaptive MARL: the non-stationary MDP formalism, change-point detection, and sliding-window estimation. Chapter 7 develops meta-reinforcement learning specifically as a third adaptation mechanism, covering the policy-parameter-gradient, black-box, and task-inference families to the extent that each speaks to adaptability. Chapter 8 places the robust and adaptive taxonomies side by side. Chapter 9 states what this synthesis contributes. Chapter 10 identifies the gaps that block a combined benchmark and the concrete next step. Chapter 11 closes the thesis.

## 1.5. Scope of Real-World Relevance

Traffic signal control, multi-robot coordination, and swarm foraging appear throughout this thesis, starting with the autonomous-vehicle example above. They are used to motivate and organize which disturbances matter: a traffic network motivates slow, structured drift; a multi-robot team motivates both sensor and communication perturbations and a teammate that can fail or be attacked; a foraging swarm motivates both. None of them is a domain in which any method surveyed in this thesis was tested or transferred by this thesis. Every empirical result attributed to a method in the chapters that follow is the one reported in that method’s own source paper, in whatever environment that paper used, not a result produced here. No method covered in this thesis is evaluated on hardware, on a physical-system proxy, or on any simulator run by this thesis. Chapter 10 returns to this distinction directly: sim-to-real robustness, closing the gap between a simulator and one physical deployment, is not the same claim as post-deployment adaptability, and neither is a claim this thesis makes about its own methodology.





---

## 2. Methodology

This thesis is built as a narrative, taxonomy-constructing survey, not an empirical study. No experiments were run to produce any result reported below; every number and every guarantee stated in the following chapters is the one reported by the original authors of the method in question. The contribution of this thesis is the organization of the material and the comparison it enables, not new experimental results. Section 10 in the final chapters turns this limitation into the thesis’s actual conclusion: an empirical benchmark is the natural next step, and this document is the groundwork for it.

This is not the first survey to touch either literature this thesis draws on. Hernandez-Leal, Kaisers, Baarslag, *et al.* [1] survey learning in multiagent environments organized specifically around non-stationarity, and Zhang, Yang, and Başar [2] give the standard broad theoretical overview of MARL algorithms; neither treats robustness to a bounded adversarial disturbance as part of the same problem as adaptability to an unbounded one, and neither builds a taxonomy that a reader can use to select a robust method and an adaptive method to compare directly. Moos, Hansel, Abdulsamad, *et al.* [3] survey robust reinforcement learning in comparable depth to Chapter 5 of this thesis, but for the single-agent case, without the multi-agent credit-assignment question that Chapter 8 identifies as unresolved on both sides of the comparison. What none of these three surveys does, and what this thesis does, is place robust MARL and adaptive MARL under one taxonomy, compare named methods from each side on the same four axes, and use that comparison to state concrete, checkable gaps that block evaluating the two together.

### 2.1. Scope

A method was included if it targets multi-agent reinforcement learning directly, or if it is a single-agent mechanism that multiple MARL methods covered here build on directly, for instance UCRL2’s confidence-set planning, or the actor-critic backbones (DDPG, TD3, SAC) that robust and adaptive MARL algorithms extend. Material was drawn from the algorithm and theory tracks of NeurIPS, ICML, ICLR, and AAMAS, from relevant journal venues, and from a small number of preprints where they are the standard reference for a method, for example M3DDPG and MIR3.

Methods were grouped first by which problem they solve, not by venue, year, or the community that produced them, because the same mechanism, an information bottleneck, a confidence set, a contrastive encoder, recurs across otherwise unrelated papers and literatures.

Concretely, search covered Google Scholar and arXiv listings against the venues above, spanning work from 1993 through 2026. No formal candidate pool was tracked before filtering; consistent with the single-author, non-systematic selection process described in Section 2.2 below, papers were kept as they were found to satisfy the criteria stated there, rather than filtered down from a recorded initial set. This search produced 113 distinct works cited throughout the thesis, of which the exemplar methods discussed in Chapters 5–7 and listed in Table 9.1 are a subset; the remainder are single-agent or game-theoretic prerequisites and background results. Table 2.1 lists, in addition, specific published methods that were identified and considered against the criteria above but ultimately not included.

## 2.2. Selection Criteria

Within the robust MARL literature, a method was included if it explicitly optimizes, bounds, or regularizes against a disturbance to observations, actions, communication, transition dynamics, or reward, and if it reports what happens when its own assumption about that disturbance is violated. Within the adaptive MARL literature, a method was included if it explicitly tracks a non-stationary quantity online, rather than assuming a fixed environment and hoping generalization is enough. Within the meta-reinforcement-learning literature specifically, a method was included only if its adaptation mechanism, not merely its architecture, is directly relevant to tracking task or environment change; this criterion is what separates the meta-RL material kept here from the much larger body of meta-RL work concerned with representation learning and optimizer design for their own sake.

This selection was made by a single author against the three criteria above, not by a systematic multi-reviewer protocol; every category-level exclusion this produced, not just every inclusion, is stated and justified against those same criteria in Section 2.3 below, so the boundary of what was left out is checkable rather than implicit. Table 2.1 makes this concrete at the level of individual methods rather than whole literatures: for each major taxonomy branch, it names at least one method that was considered and excluded, and the criterion above it failed.

**Table 2.1.** – Methods considered but excluded from each major taxonomy branch, against the selection criteria stated above.

| Branch                                      | Method(s) considered but excluded | Reason                                                                                                                                                                                                                          |
|---------------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Adversarial training                        | Action Robust RL / NR-MDP [4]     | Perturbs only the acting agent’s own action (noisy-action robustness), not transition dynamics or teammates; a narrower, single-agent threat model than RARL or M3DDPG                                                          |
| Adversarial regularization                  | CROP [5]                          | Certifies robustness via randomized smoothing over per-state actions and cumulative reward, a different certification mechanism from RADIAL-RL’s interval bound propagation; single-agent, no established multi-agent extension |
| Distributionally robust value factorization | Robust Dynamic Programming [6]    | Foundational single-agent ambiguity-set theory that DrIGM itself builds on, but predates multi-agent value factorization and CTDE entirely; prerequisite theory, not a MARL method                                              |
| Change-point detection                      | CUSUM [7]                         | Classical frequentist change detector; like BOCPD it only detects a change, but lacks the Bayesian run-length posterior that later RL work builds on directly                                                                   |
| Sliding-window estimation                   | Sliding-Window UCB [8]            | Sliding-window analysis developed for the multi-armed bandit setting, not a full sequential MDP; SW-UCRL2 is its direct MDP-level descendant                                                                                    |
| Task-inference meta-RL                      | FOCAL [9]                         | Outside this thesis’s source material rather than found and cut: task inference is fully offline, whereas every task-inference method covered here assumes online interaction at test time                                      |

### 2.3. Out of Scope

To keep this thesis connected to its stated goal, some literatures that surfaced during the search were deliberately excluded or reduced to background material. Single-agent value-based and policy-gradient algorithms are covered only to the depth needed as prerequisites (Chapter 3), not exhaustively. The general game-theoretic taxonomy of solution concepts, tabular joint-action learning algorithms beyond what motivates the non-stationarity problem, communication-protocol design for its own sake, and mean-field MARL are excluded, because none of them bear directly on robustness or adaptability once the core non-stationarity motivation has been established. Inside meta-reinforcement learning, this thesis covers the full method breadth of the policy-parameter-gradient, black-box, and task-inference families, including the black-box family’s external-memory and Hebbian or neuromodulated plasticity mechanisms, since each is a distinct answer to how an agent can adapt with little data at deployment time (Chapter 7). Task-uncertainty methods for Bayesian meta-learning (BMAML, DAMS) remain excluded, since they refine the uncertainty estimate itself rather than the adaptation mechanism. These remaining exclusions are real bodies of work; including them would have added length without adding to the argument this thesis is making.

### 2.4. How Each Research Question Is Answered

RQ1 is answered by cataloguing the disturbance types that robust and adaptive MARL methods explicitly design against, grouped by where they enter the system: the observation channel, the action channel, the communication channel, the transition and reward functions, or the identity and behavior of the other agents. RQ2 and RQ3 are answered by building a mechanism-level taxonomy for each family and describing, for every branch, what assumption it makes, what it costs to train or run, and what happens when its assumption is violated, rather than simply listing methods in publication order. RQ4 is answered by placing the two taxonomies side by side and comparing them on four fixed axes: objective, when the disturbance is handled, what is assumed about it, and the computational pattern used to handle it. That comparison, not a new algorithm, is the technical center of this thesis.

---

## 3. Overview of Deep Reinforcement Learning Algorithms

This chapter introduces the single-agent reinforcement learning algorithms that Chapter 4 extends to multiple agents. It is scoped accordingly: methods that do not resurface as the basis of a later baseline or algorithm, such as distributional value learning or hindsight experience replay, are omitted rather than surveyed for completeness. The remainder of this thesis concentrates on model-free reinforcement learning, so model-based methods are not covered here.

### 3.1. Markov Decision Processes and the Reinforcement Learning Objective

A single agent interacting with an environment is formalized as a Markov Decision Process (MDP), the tuple

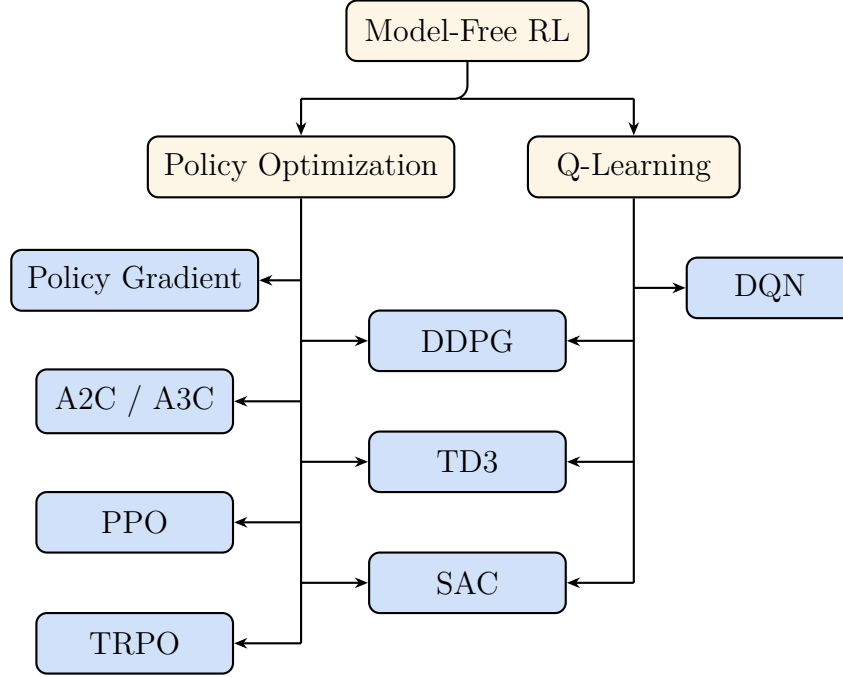
$$(S, A, \mathcal{T}, R, \mu, \gamma), \quad (3.1)$$

where  $S$  is the set of environment states,  $A$  is the set of actions,  $\mathcal{T}(s' | s, a)$  is the state transition function,  $R : S \times A \times S \rightarrow \mathbb{R}$  is the reward function,  $\mu$  is the initial state distribution, and  $\gamma \in [0, 1)$  is the discount factor. At each time step the agent observes the state  $s_t$ , selects an action  $a_t$  according to a policy  $\pi(a_t | s_t)$ , and receives a reward  $r_t = R(s_t, a_t, s_{t+1})$  as the environment transitions to  $s_{t+1} \sim \mathcal{T}(\cdot | s_t, a_t)$ . The objective of reinforcement learning is to find a policy that maximizes the expected discounted return,

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \right], \quad \tau = (s_0, a_0, s_1, a_1, \dots). \quad (3.2)$$

This is precisely the single-agent case of the stochastic game formalized in Chapter 4: setting the number of agents  $n = 1$  in the stochastic game tuple  $(I, S, \{A_i\}, \{R_i\}, \mathcal{T}, \mu)$  recovers the MDP above exactly, with the strategic element introduced by other agents removed. Every algorithm in this chapter is a way of finding a good  $\pi$  for a single such MDP; Chapter 4 is, in large part, an account of what has to change once  $\mathcal{T}$  and  $R$  depend on the actions of more than one agent.

Model-free algorithms are commonly organized by whether they learn a value function, a policy, or both. Figure 3.1 shows this organization restricted to the algorithms actually covered in this chapter and used later in the thesis.



**Figure 3.1.** – Taxonomy of the single-agent model-free algorithms covered in this chapter, restricted to methods that are directly extended to the multi-agent setting in Chapter 4 or reused as a component elsewhere in this thesis.

## 3.2. Value-Based Methods: Deep Q-Networks

Value-based methods learn an action-value function and derive a policy from it, rather than parameterizing the policy directly. The optimal action-value function  $Q^*(s, a)$  satisfies the Bellman optimality equation,

$$Q^*(s, a) = \mathbb{E}_{s'} \left[ r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]. \quad (3.3)$$

The Deep Q-Network (DQN) [10] approximates  $Q^*$  with a neural network  $Q(s, a; \theta)$ , trained by minimizing the temporal-difference loss,

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[ \left( r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right], \quad (3.4)$$

where  $\theta^-$  are the parameters of a periodically updated target network and  $\mathcal{D}$  is a replay buffer of past transitions. The target network prevents the training target from shifting at the same rate as the network being trained, and the replay buffer breaks the temporal correlation between consecutive transitions, both of which are needed for stable convergence with a neural function approximator. This is the exact update rule that Independent Deep Q-Networks (IDQN) in Chapter 4 applies per agent, treating every other agent as part of the environment; DQN is covered here only to the depth needed to make that extension self-contained.

---

**Algorithm 1** Deep Q-Network (DQN)

---

**Require:** Learning rate  $\alpha$ , exploration rate  $\epsilon$ , target update interval

```

1: Initialize Q-network  $\theta$  and target network  $\theta^- \leftarrow \theta$ 
2: Initialize replay buffer  $\mathcal{D}$ 
3: for each episode do
4:   Initialize state  $s_0$ 
5:   for  $t = 0, 1, 2, \dots$  do
6:     With probability  $\epsilon$ , select a random action  $a_t$ ; otherwise  $a_t \leftarrow$ 
        $\arg \max_a Q(s_t, a; \theta)$ 
7:     Execute  $a_t$ , observe reward  $r_t$  and next state  $s_{t+1}$ 
8:     Store  $(s_t, a_t, r_t, s_{t+1})$  in  $\mathcal{D}$ 
9:     Sample a mini-batch  $\mathcal{B}$  from  $\mathcal{D}$ 
10:    for each transition  $(s, a, r, s') \in \mathcal{B}$  do
11:       $y \leftarrow r$  if  $s'$  is terminal, else  $y \leftarrow r + \gamma \max_{a'} Q(s', a'; \theta^-)$ 
12:    end for
13:    Update  $\theta$  by minimizing  $\mathcal{L}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} (y - Q(s, a; \theta))^2$ 
14:    Periodically update target network  $\theta^- \leftarrow \theta$ 
15:  end for
16: end for

```

---

### 3.3. Policy Gradient Methods

Rather than deriving a policy from a value function, policy gradient methods parameterize the policy  $\pi_\theta(a | s)$  directly and adjust  $\theta$  to increase  $J(\pi_\theta)$ . The policy gradient theorem expresses the gradient of the objective as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right], \quad G_t = \sum_{k=t}^T \gamma^{k-t} r_k, \quad (3.5)$$

which increases the probability of actions that led to higher return. REINFORCE [11] is the direct Monte Carlo estimator of this expression, using the full episode return

$G_t$  in place of the expectation. Because  $G_t$  is a high-variance Monte Carlo estimate, it is standard to subtract a state-dependent baseline  $b(s_t)$  from  $G_t$  without introducing bias, and to replace  $G_t - b(s_t)$  with the Generalized Advantage Estimator (GAE) [12],

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad (3.6)$$

which trades bias for variance through  $\lambda \in [0, 1]$  and requires a learned value function  $V(s)$ , the first appearance of the critic that the actor-critic methods of Section 3.5 build on directly. REINFORCE is the basic principle Chapter 4’s independent policy-gradient MARL builds on, and GAE reappears unchanged, applied to a centralized rather than local value function, in MAPPO (Chapter 4) and in several meta-RL methods in Chapter 7.

---

**Algorithm 2** REINFORCE with Baseline

---

**Require:** Learning rate  $\alpha$

- 1: Initialize policy parameters  $\theta$  and baseline parameters  $\psi$
  - 2: **for** each episode **do**
  - 3:   Generate trajectory  $\tau = (s_0, a_0, r_0, \dots, s_T, a_T, r_T)$  by following  $\pi_\theta$
  - 4:   **for**  $t = 0, \dots, T$  **do**
  - 5:     Compute return  $G_t \leftarrow \sum_{k=t}^T \gamma^{k-t} r_k$
  - 6:     Compute baseline  $b(s_t; \psi)$  and advantage  $\hat{A}_t \leftarrow G_t - b(s_t; \psi)$
  - 7:   **end for**
  - 8:   Update baseline:  $\psi \leftarrow \psi - \alpha \nabla_\psi \sum_t (G_t - b(s_t; \psi))^2$
  - 9:   Update policy:  $\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \hat{A}_t$
  - 10: **end for**
- 

## 3.4. Trust-Region and Clipped Policy Optimization

A plain policy gradient step can move the policy arbitrarily far from where it was evaluated, which can degrade performance irrecoverably. Trust-region and clipped methods address this by constraining each update to stay close to the previous policy.



### 3.4.1. Trust Region Policy Optimization (TRPO)

TRPO [13] maximizes a surrogate objective subject to an explicit constraint on the KL-divergence between the new and old policy,

$$\max_{\theta} \mathbb{E}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \quad \text{subject to} \quad \mathbb{E}_t [D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | s_t) \parallel \pi_{\theta}(\cdot | s_t))] \leq \delta, \quad (3.7)$$

solved approximately using a conjugate-gradient step and a line search, which guarantees monotonic policy improvement under mild conditions but is expensive per update. TRPO is used directly, not just as a conceptual ancestor, later in this thesis: the adversarial training objective in RARL (Chapter 5) is optimized with TRPO, and several meta-RL methods in Chapter 7 use it for the outer-loop meta-update.

---

**Algorithm 3** Trust Region Policy Optimization (TRPO)

---

**Require:** KL-divergence bound  $\delta$ , conjugate-gradient iterations  $K$

- 1: Initialize policy parameters  $\theta_0$
  - 2: **for** each iteration  $i = 0, 1, 2, \dots$  **do**
  - 3:   Collect a batch of trajectories using  $\pi_{\theta_i}$
  - 4:   Estimate advantages  $\hat{A}_t$  for every visited state-action pair
  - 5:   Form the surrogate objective  $L(\theta) = \mathbb{E}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_i}(a_t | s_t)} \hat{A}_t \right]$
  - 6:   Compute the policy gradient  $g = \nabla_{\theta} L(\theta)|_{\theta_i}$
  - 7:   Estimate the search direction  $x \approx H^{-1}g$  via  $K$  steps of conjugate gradient, where  $H$  is the Hessian of the KL-divergence at  $\theta_i$
  - 8:   Compute the maximal step size  $\beta = \sqrt{2\delta/(x^{\top} H x)}$
  - 9:   Perform a backtracking line search on  $\theta_i + \beta_j x$  (halving  $\beta_j$  at each step) until the KL constraint is satisfied and  $L(\theta)$  improves
  - 10:   Set  $\theta_{i+1}$  to the accepted step
  - 11: **end for**
- 

### 3.4.2. Proximal Policy Optimization (PPO)

PPO [14] achieves a similar effect to TRPO’s trust region at a fraction of the computational cost, by replacing the KL constraint with a clipped surrogate objective evaluated with ordinary stochastic gradient ascent,

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (3.8)$$

Clipping the probability ratio  $r_t(\theta)$  removes the incentive to push the policy far from  $\pi_{\theta_{\text{old}}}$  within a single batch of updates, which gives most of TRPO’s stability

without solving a constrained optimization problem at every step. PPO does not have a dedicated multi-agent extension of its own in this thesis; instead, it is the algorithm MAPPO (Chapter 4) applies with a centralized critic, and it is the actor-critic algorithm used to train policies throughout the meta-reinforcement-learning methods of Chapter 7, where its clipped objective, together with a value loss and an entropy bonus, is used essentially unchanged.

---

**Algorithm 4** Proximal Policy Optimization (PPO)
 

---

**Require:** Initial policy parameters  $\theta$ , value parameters  $\phi$ , clipping parameter  $\epsilon$ , value-loss coefficient  $c_v$ , entropy coefficient  $c_e$ , epochs  $K$

- 1: **while** not converged **do**
- 2:   Collect trajectories  $\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}$  using  $\pi_{\theta_{\text{old}}}$
- 3:   **for** each timestep  $t$  **do**
- 4:     Compute TD residual:  $\delta_t \leftarrow r_t + \gamma V_{\phi_{\text{old}}}(s_{t+1}) - V_{\phi_{\text{old}}}(s_t)$
- 5:     Compute GAE advantage:  $\hat{A}_t \leftarrow \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}$
- 6:     Compute return target:  $R_t \leftarrow \hat{A}_t + V_{\phi_{\text{old}}}(s_t)$
- 7:   **end for**
- 8:   **for**  $k = 1, \dots, K$  epochs **do**
- 9:     Compute importance ratio:  $r_t(\theta) \leftarrow \pi_{\theta}(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$
- 10:    Compute clipped policy loss:  $L_{\pi}(\theta) = \mathbb{E}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$
- 11:    Compute value loss:  $L_V(\phi) = \mathbb{E}_t [(V_{\phi}(s_t) - R_t)^2]$
- 12:    Compute entropy bonus:  $L_H(\theta) = \mathbb{E}_t [\mathcal{H}(\pi_{\theta}(\cdot | s_t))]$
- 13:    Update  $\theta, \phi$  by ascending  $L_{\pi}(\theta) - c_v L_V(\phi) + c_e L_H(\theta)$
- 14:   **end for**
- 15:    $\theta_{\text{old}} \leftarrow \theta, \phi_{\text{old}} \leftarrow \phi$
- 16: **end while**

---

### 3.5. Actor-Critic Methods

Actor-critic methods combine a policy (the actor) with a learned value function (the critic), using the critic to construct a lower-variance training signal for the actor than the raw episode return. In its simplest synchronous form, Advantage Actor-Critic (A2C) collects a batch of trajectories, computes the advantage  $\hat{A}_t$  from the critic, and updates the actor along  $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t$  and the critic by regressing  $V(s_t)$  toward the observed return; the asynchronous variant, A3C [15], runs many such workers in parallel against a shared set of parameters. A2C is the algorithm Independent Advantage Actor-Critic (IA2C) in Chapter 4 applies per agent; A3C's

specific asynchronous-worker mechanism does not reappear elsewhere in this thesis and is mentioned here only for completeness.

---

**Algorithm 5** Advantage Actor-Critic (A2C)

---

**Require:** Learning rates  $\alpha_\pi, \alpha_V$ , number of parallel environments  $K$

- 1: Initialize actor parameters  $\theta$ , critic parameters  $\phi$
  - 2: **for**  $t = 0, 1, 2, \dots$  **do**
  - 3:     **for**  $k = 1, \dots, K$  (in parallel) **do**
  - 4:         Sample action  $a_t^{(k)} \sim \pi_\theta(\cdot | s_t^{(k)})$ , execute it, observe  $r_t^{(k)}, s_{t+1}^{(k)}$
  - 5:         Compute advantage:  $\hat{A}_t^{(k)} \leftarrow r_t^{(k)} + \gamma V_\phi(s_{t+1}^{(k)}) - V_\phi(s_t^{(k)})$  (or  $\leftarrow -V_\phi(s_t^{(k)})$  if terminal, using  $r_t^{(k)}$  only)
  - 6:     **end for**
  - 7:     Update actor:  $\theta \leftarrow \theta + \alpha_\pi \frac{1}{K} \sum_k \nabla_\theta \log \pi_\theta(a_t^{(k)} | s_t^{(k)}) \hat{A}_t^{(k)}$
  - 8:     Update critic:  $\phi \leftarrow \phi - \alpha_V \frac{1}{K} \sum_k \nabla_\phi \left( \hat{A}_t^{(k)} \right)^2$
  - 9: **end for**
- 

The three algorithms below extend actor-critic methods to continuous action spaces, and are covered in decreasing depth according to how directly they are used later: DPG and DDPG are the direct single-agent ancestors of MADDPG (Chapter 4) and, through it, M3DDPG (Chapter 5); TD3 is mentioned later only as one interchangeable choice of underlying algorithm and is treated briefly; SAC is the underlying algorithm for several meta-RL methods in Chapter 7 and is treated in full.

### 3.5.1. Deterministic Policy Gradient (DPG)

For continuous action spaces, computing  $\arg \max_a Q(s, a)$  at every step, as DQN requires, is intractable. The Deterministic Policy Gradient theorem [16] avoids this by learning a deterministic actor  $a = \mu_\theta(s)$  that approximates the maximizing action directly, so that  $\max_a Q(s, a)$  is replaced by the tractable  $Q(s, \mu_\theta(s))$ . The actor is updated along

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s \sim \mathcal{D}} \left[ \nabla_\theta \mu_\theta(s) \nabla_a Q(s, a) \Big|_{a=\mu_\theta(s)} \right], \quad (3.9)$$

which is the single-agent special case of the MADDPG actor update in Chapter 4, with the centralized critic  $Q_i^\mu(z, a_1, \dots, a_n)$  there collapsing to the local critic  $Q(s, a)$  here once  $n = 1$ .

---

**Algorithm 6** Deterministic Policy Gradient (DPG)

---

**Require:** Learning rates  $\alpha_\mu, \alpha_Q$ , exploration noise process  $\mathcal{N}$

```

1: Initialize actor  $\mu_\theta$  and critic  $Q_\phi$ 
2: for each episode do
3:   for  $t = 0, 1, 2, \dots$  do
4:     Select action  $a_t \leftarrow \mu_\theta(s_t) + \mathcal{N}_t$ 
5:     Execute  $a_t$ , observe reward  $r_t$  and next state  $s_{t+1}$ 
6:     Compute target:  $y \leftarrow r_t + \gamma Q_\phi(s_{t+1}, \mu_\theta(s_{t+1}))$ 
7:     Update critic:  $\phi \leftarrow \phi - \alpha_Q \nabla_\phi (Q_\phi(s_t, a_t) - y)^2$ 
8:     Update actor:  $\theta \leftarrow \theta + \alpha_\mu \nabla_\theta \mu_\theta(s_t) \nabla_a Q_\phi(s_t, a) \big|_{a=\mu_\theta(s_t)}$ 
9:   end for
10: end for

```

---

This on-policy, single-transition form of DPG has no replay buffer or target networks; DDPG below adds both.

### 3.5.2. Deep Deterministic Policy Gradient (DDPG)

DDPG [17] combines the deterministic policy gradient with the deep-network components introduced by DQN: a replay buffer for off-policy learning and target networks for both the actor and critic, updated by a soft update  $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$  rather than DQN’s periodic hard copy. DDPG is off-policy and consists of four networks in total, an actor  $\mu_\theta$ , a critic  $Q_\phi$ , and a target copy of each. This is exactly the architecture MADDPG (Chapter 4) extends by giving every agent’s critic access to the observations and actions of every other agent, while keeping the actor local; DDPG’s own limitations, an overestimation bias in the critic and exploration that relies entirely on injected action noise, are what motivate TD3 and SAC respectively below, and both limitations resurface as motivation for the policy-ensemble mechanism in MADDPG in Chapter 4.

**Algorithm 7** Deep Deterministic Policy Gradient (DDPG)**Require:** Learning rates  $\alpha_\mu, \alpha_Q$ , soft update rate  $\tau$ , exploration noise process  $\mathcal{N}$ 

- 1: Initialize actor  $\mu_\theta$  and critic  $Q_\phi$
- 2: Initialize target networks  $\theta^- \leftarrow \theta, \phi^- \leftarrow \phi$
- 3: Initialize replay buffer  $\mathcal{D}$
- 4: **for** each episode **do**
- 5:     **for**  $t = 0, 1, 2, \dots$  **do**
- 6:         Select action  $a_t \leftarrow \mu_\theta(s_t) + \mathcal{N}_t$ , execute it, observe  $r_t, s_{t+1}$
- 7:         Store  $(s_t, a_t, r_t, s_{t+1})$  in  $\mathcal{D}$
- 8:         Sample a mini-batch  $\mathcal{B}$  from  $\mathcal{D}$
- 9:         **for** each transition  $(s, a, r, s') \in \mathcal{B}$  **do**
- 10:              $y \leftarrow r + \gamma Q_{\phi^-}(s', \mu_{\theta^-}(s'))$
- 11:         **end for**
- 12:         Update critic:  $\phi \leftarrow \phi - \alpha_Q \nabla_\phi \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} (Q_\phi(s, a) - y)^2$
- 13:         Update actor:  $\theta \leftarrow \theta + \alpha_\mu \nabla_\theta \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} \nabla_a Q_\phi(s, a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)$
- 14:         Soft-update targets:  $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-, \phi^- \leftarrow \tau\phi + (1 - \tau)\phi^-$
- 15:     **end for**
- 16: **end for**

**3.5.3. Twin Delayed Deep Deterministic Policy Gradient (TD3)**

TD3 [18] addresses DDPG’s overestimation bias with three modifications: two independent critics, with the Bellman target computed from their minimum rather than a single critic’s estimate; delayed actor updates, so the actor is updated less frequently than the critics; and target policy smoothing, which adds clipped noise to the target action to prevent the critic from exploiting sharp, incorrect peaks in its own value estimate. These modifications substantially improve training stability over DDPG, but TD3 does not have its own single-agent multi-agent extension in this thesis and is named later only as one interchangeable choice of underlying algorithm alongside SAC; it is included here for completeness rather than because a later chapter builds on its specific mechanisms.

---

**Algorithm 8** Twin Delayed DDPG (TD3)
 

---

**Require:** Soft update rate  $\tau$ , policy delay  $d$ , target noise scale  $\sigma$ , noise clip  $c$

- 1: Initialize actor  $\mu_\theta$ , critics  $Q_{\phi_1}, Q_{\phi_2}$ , and their target copies
- 2: Initialize replay buffer  $\mathcal{D}$
- 3: **for**  $t = 0, 1, 2, \dots$  **do**
- 4:     Select action  $a_t \leftarrow \mu_\theta(s_t) + \mathcal{N}_t$ , execute it, store  $(s_t, a_t, r_t, s_{t+1})$  in  $\mathcal{D}$
- 5:     Sample a mini-batch  $\mathcal{B}$ ; for each transition compute the smoothed target action  $\tilde{a}' \leftarrow \mu_{\theta^-}(s') + \text{clip}(\epsilon, -c, c)$ ,  $\epsilon \sim \mathcal{N}(0, \sigma^2)$
- 6:     Compute target:  $y \leftarrow r + \gamma \min_{k=1,2} Q_{\phi_k^-}(s', \tilde{a}')$
- 7:     Update both critics toward  $y$
- 8:     **if**  $t \bmod d = 0$  **then**
- 9:         Update actor using  $Q_{\phi_1}$ :  $\theta \leftarrow \theta + \alpha_\mu \nabla_\theta \mu_\theta(s) \nabla_a Q_{\phi_1}(s, a)|_{a=\mu_\theta(s)}$
- 10:         Soft-update all target networks
- 11:     **end if**
- 12: **end for**

---

### 3.5.4. Soft Actor-Critic (SAC)

SAC [19] departs from DDPG and TD3 by learning a stochastic rather than deterministic policy, and by augmenting the standard return objective with a policy entropy term,

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[ \sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right], \quad (3.10)$$

where  $\mathcal{H}(\pi(\cdot | s_t))$  is the entropy of the policy at  $s_t$  and  $\alpha$  controls the trade-off between reward and exploration. Maximizing entropy alongside return keeps the policy stochastic throughout training, which produces exploration that does not rely on manually injected noise and helps avoid premature convergence to a suboptimal deterministic policy. SAC combines this objective with the same off-policy replay buffer and twin critics used in TD3, giving it both stable, low-variance training and persistent exploration. This combination is why SAC, rather than DDPG or TD3, is the algorithm chosen to train the underlying actor and exploration policies in several of the meta-reinforcement-learning methods of Chapter 7, where the environment itself is deliberately non-stationary across tasks and persistent exploration is therefore especially important.

---

**Algorithm 9** Soft Actor-Critic (SAC)

---

**Require:** Learning rates  $\alpha_\pi, \alpha_Q$ , soft update rate  $\tau$ , entropy coefficient  $\alpha$

- 1: Initialize stochastic actor  $\pi_\theta$ , critics  $Q_{\phi_1}, Q_{\phi_2}$ , and target critics  $Q_{\phi_1^-}, Q_{\phi_2^-}$
- 2: Initialize replay buffer  $\mathcal{D}$
- 3: **for**  $t = 0, 1, 2, \dots$  **do**
- 4:     Sample  $a_t \sim \pi_\theta(\cdot \mid s_t)$ , execute it, store  $(s_t, a_t, r_t, s_{t+1})$  in  $\mathcal{D}$
- 5:     Sample a mini-batch  $\mathcal{B}$  from  $\mathcal{D}$
- 6:     **for** each transition  $(s, a, r, s') \in \mathcal{B}$  **do**
- 7:         Sample  $a' \sim \pi_\theta(\cdot \mid s')$
- 8:          $y \leftarrow r + \gamma \left( \min_{k=1,2} Q_{\phi_k^-}(s', a') - \alpha \log \pi_\theta(a' \mid s') \right)$
- 9:     **end for**
- 10:     Update both critics:  $\phi_k \leftarrow \phi_k - \alpha_Q \nabla_{\phi_k} \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} (Q_{\phi_k}(s, a) - y)^2$ , for  $k = 1, 2$
- 11:     Update actor to match the exponentiated soft Q-function:  $\theta \leftarrow \theta + \alpha_\pi \nabla_\theta \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} (\min_{k=1,2} Q_{\phi_k}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s) \mid s))$
- 12:     Soft-update target critics:  $\phi_k^- \leftarrow \tau \phi_k + (1 - \tau) \phi_k^-$ , for  $k = 1, 2$
- 13: **end for**

---

### 3.6. Selection of Algorithms for Multi-Agent Extension

This chapter covered the single-agent algorithms needed to make Chapter 4’s multi-agent extensions self-contained: DQN underlies IDQN, REINFORCE underlies independent policy-gradient MARL, A2C underlies IA2C, DPG and DDPG underlie MADDPG, and PPO underlies MAPPO. TRPO and SAC are carried forward not through a dedicated multi-agent baseline in Chapter 4, but by direct reuse later in the thesis, TRPO in the adversarial optimization of RARL (Chapter 5) and in several meta-RL methods (Chapter 7), and SAC as the default underlying algorithm for exploration and execution policies throughout Chapter 7. TD3 and A3C are included for completeness but are not built on directly anywhere later in this thesis. Chapter 4 takes each of the extendable algorithms above and asks the same question of all of them: what has to change once the single fixed environment each was designed for is replaced by  $n$  other agents that are learning at the same time.





---

## 4. Fundamentals of Multi-Agent Reinforcement Learning

This chapter introduces the concepts needed to follow the robustness and adaptability methods developed in Chapters 5 to 7. It does not attempt a complete treatment of multi-agent reinforcement learning (MARL) or game theory; concepts that do not resurface later, such as the Folk Theorem or fairness measures beyond social welfare, are omitted or mentioned only in passing. The presentation of the game-theoretic material follows Albrecht et al. [20], condensed to the parts this thesis relies on. Chapter 3 developed the single-agent reinforcement learning algorithms this chapter builds on directly, including the policy gradient theorem, TRPO, DDPG, TD3, and SAC; rather than repeat that material, this chapter treats it as known and focuses on what changes when a single agent becomes  $n$  interacting agents.

### 4.1. Multi-Agent Reinforcement Learning

#### 4.1.1. Stochastic Games

A single agent acting in a changing environment is modeled as a Markov Decision Process (MDP), the formalism underlying every algorithm in Chapter 3. Multi-agent reinforcement learning generalizes this model to  $n$  interacting agents through the stochastic game (also called a Markov game), formally defined as the six-tuple

$$(I, S, \{A_i\}_{i \in I}, \{R_i\}_{i \in I}, \mathcal{T}, \mu), \quad (4.1)$$

where  $I = \{1, \dots, n\}$  is the set of agents,  $S$  is the finite set of environment states,  $A_i$  is the action set of agent  $i$  with joint action space  $A = A_1 \times \dots \times A_n$ ,  $R_i : S \times A \times S \rightarrow \mathbb{R}$  is the reward function of agent  $i$ ,  $\mathcal{T}(s' | s, a)$  is the state transition function, and  $\mu$  is the initial state distribution [20]. Setting  $n = 1$  recovers exactly the single-agent MDP of Chapter 3, and the strategic element introduced by other agents disappears entirely. Everything in this chapter can therefore be read as asking how the single-agent tools of Chapter 3 need to change once  $\mathcal{T}$  and  $R_i$  depend on the actions of agents other than  $i$ .

At each time step, every agent observes the current state and selects an action according to its policy  $\pi_i$ . The resulting joint action determines both the reward received by each agent and the transition to the next state. This formalism underlies every algorithm discussed later in this thesis: robustness (Chapter 5) and adaptability (Chapters 6 and 7) are both framed as problems of learning policies within a stochastic game whose transition dynamics, reward functions, or opponent behavior are uncertain or changing.

### 4.1.2. Partial Observability

Stochastic games assume every agent observes the true state directly. This assumption rarely holds in practice: agents typically sense only part of the environment. The stochastic game is therefore extended to a Partially Observable Stochastic Game (POSG), defined as the eight-tuple

$$(I, S, \{A_i\}_{i \in I}, \{R_i\}_{i \in I}, \mathcal{T}, \mu, \{O_i\}_{i \in I}, \{\mathcal{O}_i\}_{i \in I}), \quad (4.2)$$

where the first six components match the stochastic game,  $O_i$  is the observation space of agent  $i$ , and  $\mathcal{O}_i : A \times S \times O_i \rightarrow [0, 1]$  is the observation function specifying the probability that agent  $i$  receives observation  $o_i$  after the joint action  $a$  leads to state  $s$  [20].

Because the true state is hidden, each agent must act on its history of past observations and actions rather than on the state itself. This is the setting used throughout Chapters 5 to 7: robustness methods must tolerate noisy or adversarially perturbed observations, and adaptability methods must infer changes in the environment from observation histories alone, since the state itself is never directly available.

### 4.1.3. Joint Policies and Expected Return

The behavior of the full system is described by the joint policy

$$\pi = (\pi_1, \pi_2, \dots, \pi_n), \quad (4.3)$$

where  $\pi_i$  is the policy of agent  $i$ , conditioned on its own observation-action history  $h_i$ . Agents are assumed to select actions independently given their local histories, so the joint action probability factorizes as

$$\pi(a \mid \hat{h}) = \prod_{j \in I} \pi_j(a_j \mid h_j). \quad (4.4)$$

Each agent’s objective is to maximize its expected discounted return. For agent  $i$ , the return accumulated along a full history  $\hat{h}_t$  is

$$u_i(\hat{h}_t) = \sum_{\tau=0}^{t-1} \gamma^\tau R_i(s^\tau, a^\tau, s^{\tau+1}), \quad \gamma \in [0, 1), \quad (4.5)$$

and the expected return of agent  $i$  under joint policy  $\pi$  is

$$U_i(\pi) = \mathbb{E}_{\hat{h}_t \sim \pi} [u_i(\hat{h}_t)]. \quad (4.6)$$

$U_i(\pi)$  is the quantity every solution concept in Section 4.2 and every algorithm in Section 4.4 is defined in terms of: equilibrium concepts describe stable joint policies with respect to  $U_i$ , and value-based and policy-gradient algorithms are different ways of optimizing it.

## 4.2. Game-Theoretic Foundations

Section 4.1.1 described how agents interact; this section describes what counts as a good or stable outcome of that interaction, and states which of these solution concepts are actually used later in the thesis.

### 4.2.1. Normal-Form Games and Classes of Games

The simplest strategic setting is the normal-form game, a one-shot interaction defined by the tuple  $(I, \{A_i\}_{i \in I}, \{R_i\}_{i \in I})$  where all agents choose actions simultaneously and each agent’s payoff  $R_i : A \rightarrow \mathbb{R}$  depends on the full joint action [20]. Games are commonly classified by how payoffs relate across agents: in a zero-sum game,  $\sum_i R_i(a) = 0$  for every joint action, so one agent’s gain is exactly another’s loss; in a common-payoff (fully cooperative) game, all agents receive the same reward; general-sum games fall between these extremes. This distinction matters directly for Chapter 5: adversarial robustness methods such as RARL and M3DDPG cast robustness as a two-player zero-sum game between the protagonist agent and an adversary, whereas the value-decomposition baselines in Section 4.4 are built for the common-payoff case.

### 4.2.2. Best Response and Minimax Solutions

Given the policies of all other agents,  $\pi_{-i} = (\pi_1, \dots, \pi_{i-1}, \pi_{i+1}, \dots, \pi_n)$ , the best response of agent  $i$  is the policy that maximizes its own expected return,

$$\text{BR}_i(\pi_{-i}) = \arg \max_{\pi_i} U_i(\langle \pi_i, \pi_{-i} \rangle). \quad (4.7)$$

Best response underlies most of the equilibrium concepts below and is repeatedly estimated, implicitly or explicitly, by learning algorithms that adapt to the current behavior of other agents.

In a two-player zero-sum game, the minimax solution has each player maximize its own return under the assumption that the opponent minimizes it,

$$\pi_i^* = \arg \max_{\pi_i} \min_{\pi_{-i}} U_i(\pi). \quad (4.8)$$

In zero-sum games the minimax solution coincides with the Nash equilibrium defined below. This concept is not part of the user’s proposed structure but is added here because it is the mathematical foundation of the adversarial robustness methods in Chapter 5: RARL, M3DDPG, and distributionally robust MARL are all instances of solving, or approximating, a minimax optimization problem against a perturbation or adversary.

### 4.2.3. Nash Equilibrium

A joint policy  $\pi^* = (\pi_1^*, \dots, \pi_n^*)$  is a Nash equilibrium if no agent can improve its expected return by unilaterally changing its own policy,

$$U_i(\pi_i^*, \pi_{-i}^*) \geq U_i(\pi_i, \pi_{-i}^*), \quad \forall i \in I, \forall \pi_i. \quad (4.9)$$

Equivalently, every agent already plays a best response to the others. At least one Nash equilibrium exists in every finite general-sum normal-form game, though it may not be unique, and exact computation is intractable, which motivates the relaxed  $\epsilon$ -Nash equilibrium and the learning-based alternatives in Section 4.3 [20]–[22]. The Nash equilibrium is the reference point for two of the tabular algorithms this thesis builds on conceptually (Nash-Q [22] and CE-Q [23]), and it is the solution concept the adversarial game in RARL is proven to admit (Chapter 5).

### 4.2.4. Correlated and Coarse Correlated Equilibrium

The Nash equilibrium assumes agents act independently. A correlated equilibrium relaxes this by letting agents condition on a shared recommendation drawn from a correlated policy  $\pi_c$  before deciding whether to follow it; no agent benefits from deviating once it has observed its recommendation [24]. A coarse correlated equilibrium (CCE) is a further relaxation in which an agent must commit to following the recommendation, or not, before observing it, rather than after [25]. This weaker requirement makes CCEs easier to compute and gives them a distinct role in online and no-regret learning (Section 4.3.3), which is the only place these two concepts are used again later in this thesis.

### 4.2.5. Pareto Optimality and Social Welfare

Equilibrium concepts guarantee stability, not efficiency. A joint policy  $\pi$  is Pareto-dominated by  $\pi'$  if every agent does at least as well under  $\pi'$  and at least one agent does strictly better,

$$\forall i \in I : U_i(\pi') \geq U_i(\pi), \quad \exists i \in I : U_i(\pi') > U_i(\pi). \quad (4.10)$$

A joint policy that is not Pareto-dominated by any feasible alternative is Pareto-optimal. This notion of efficiency, rather than stability, is what motivates the Pareto Actor-Critic (PAC) algorithm discussed in Section 4.4, which explicitly biases learning toward the Pareto-optimal equilibrium in cooperative games to avoid the equilibrium-selection problem described next.

### 4.2.6. Limitations of Equilibrium Solutions

Equilibrium concepts are a useful reference point, but three limitations motivate moving beyond them. First, a Nash equilibrium need not be efficient: the Prisoner's Dilemma is the standard example where mutual cooperation dominates the unique Nash equilibrium. Second, many games admit multiple equilibria with very different returns, so a learning algorithm must also solve the equilibrium-selection problem of which equilibrium to converge to. Third, and most relevant to this thesis, equilibrium policies are typically defined only along the trajectories they induce; they offer no guidance for what to do if a disturbance, an execution error, or an opponent's unexpected behavior pushes the system off that trajectory. This last point is the conceptual link between game theory and the rest of the thesis: static equilibrium solutions are, by construction, not built to handle the non-stationary and adversarial conditions that Chapters 5 to 7 study directly.

## 4.3. Learning in Multi-Agent Systems

### 4.3.1. Independent Learning

The simplest way to train multiple agents is independent learning: each agent treats the others as part of the environment and applies a standard single-agent algorithm from Chapter 3, such as Q-learning or a policy gradient method, to its own local observations and rewards. From the perspective of agent  $i$ , the environment has an effective transition function

$$\mathcal{T}_i(s^{t+1} | s^t, a_i) \propto \sum_{a_{-i} \in A_{-i}} \mathcal{T}(s^{t+1} | s^t, \langle a_i, a_{-i} \rangle) \prod_{j \neq i} \pi_j(a_j | s^t), \quad (4.11)$$

which depends explicitly on the policies of the other agents. This is the simplest and most direct route into non-stationarity, described next.

### 4.3.2. Non-Stationarity in MARL

A stochastic process is stationary if  $P(X^{t+\tau}) = P(X^t)$  for all  $\tau \geq 0$ : its statistics do not depend on when they are measured. Single-agent reinforcement learning implicitly relies on this assumption: every convergence guarantee in Chapter 3, from tabular Q-learning to the policy gradient theorem, assumes a fixed policy induces a fixed transition distribution over future states. In MARL, however, every agent updates its policy simultaneously. From the perspective of agent  $i$ , the effective transition function of Equation (4.11) above depends on  $\pi_{-i}$ , and  $\pi_{-i}$  is continually changing as the other agents learn. The environment is therefore non-stationary purely as a consequence of concurrent learning, independent of whether the physical environment itself changes.

This has two direct consequences that recur throughout the thesis. First, whichever single-agent algorithm from Chapter 3 is applied independently by each agent, whether Q-learning or an actor-critic method, relies on exactly the fixed transition dynamics that no longer hold; previously learned value estimates can become stale, learning can become unstable, and agents can cycle without settling on a stable policy. Second, and more importantly for this thesis, non-stationarity from concurrent learning is only one source of non-stationarity among several; Chapter 6 develops a general mathematical treatment of non-stationary environments (including externally changing dynamics, not just other learning agents) and the adaptive methods designed to track them. This section's account of non-stationarity is the special case of that general problem that arises purely from multi-agent training, and it is the reason CTDE and value-decomposition methods (Section 4.4) are structured the way they are: both are, in part, attempts to stabilize learning against exactly this effect.

### 4.3.3. No-Regret Learning

An alternative to targeting a specific equilibrium is to require that a learning algorithm not perform much worse, in hindsight, than the best fixed action or policy would have. The regret of agent  $i$  over the policy space after  $z$  episodes is

$$\text{Regret}_i^z = \max_{\pi_i \in \Pi_i} \sum_{e=1}^z [U_i(\langle \pi_i, \pi_{-i}^e \rangle) - U_i(\pi^e)], \quad (4.12)$$

and a learning algorithm is no-regret if  $\text{Regret}_i^z/z \rightarrow 0$  as  $z \rightarrow \infty$  for every agent. Regret-matching algorithms [26] are the classical example. No-regret learning is included here mainly for its connection back to Section 4.2.4: the empirical distribution of joint actions played by no-regret learners converges to the set of coarse correlated equilibria, which is the only later use of CCE in this thesis. Chapter 6 uses a related but distinct notion, dynamic regret, to evaluate performance under a moving optimum; the two concepts share a name but answer different questions, and this section should not be conflated with that later material.

## 4.4. Baseline MARL Algorithms

The preceding sections introduced the environment model and evaluation criteria. This section reviews the learning paradigms and representative algorithms selected as baselines on top of which the robustness and adaptability mechanisms of this thesis are implemented. They are grouped into three practical categories: Q-learning-based methods, policy-gradient methods, and actor-critic methods. Actor-critic algorithms are, strictly speaking, a subclass of policy-gradient methods, since the actor is still updated by a policy gradient; the three-way grouping is used here only as an organizational convenience for the baselines actually used, not as a claim that the classes are mathematically disjoint.

### 4.4.1. Q-Learning-Based MARL and Value Decomposition

Independent Q-learning extends single-agent Q-learning directly, with each agent learning its own action-value function while treating the others as part of the environment; it inherits the non-stationarity problem of Section 4.3.2 without any mechanism to address it. Centralized Q-learning avoids this by learning a single joint action-value function  $Q : S \times A \rightarrow \mathbb{R}$ , but this does not scale, since  $|A| = \prod_i |A_i|$  grows exponentially in the number of agents, and a centralized function of the joint action cannot be evaluated by an individual agent using only its own observation at execution time.

Value decomposition methods resolve this tension by factorizing a centralized action-value function into per-agent utility functions  $Q_i(h_i, a_i; \theta_i)$  that each agent can evaluate using only its local history. For this factorization to preserve correct decentralized action selection, it must satisfy the Individual-Global-Max (IGM) property,

$$\forall a \in A : a \in A^*(h, z; \theta) \iff \forall i \in I : a_i \in A_i^*(h_i; \theta_i), \quad (4.13)$$

i.e. the joint action that maximizes the centralized value function is exactly the one obtained by each agent independently maximizing its own utility function. Value Decomposition Networks (VDN) [27] enforce IGM by the simplest possible construction, an additive decomposition  $Q(h, z, a; \theta) = \sum_{i \in I} Q_i(h_i, a_i; \theta_i)$ , which is trivially monotonic but cannot represent value functions with genuine interactions between agents. QMIX [28] relaxes this by learning a nonlinear mixing network  $f_{\text{mix}}$  subject only to a monotonicity constraint,

$$\frac{\partial Q(h, z, a; \theta)}{\partial Q_i(h_i, a_i; \theta_i)} > 0 \quad \forall i \in I, \forall a \in A, \quad (4.14)$$

which still guarantees IGM while allowing considerably richer joint value functions than the additive VDN decomposition. VDN and QMIX are the representative Q-learning-based baselines this thesis builds on; the tabular game-theoretic Q-learning variants (Nash-Q [22], CE-Q [23]) are mentioned only to motivate the equilibrium concepts of Section 4.2 and are not themselves used as experimental baselines, since they do not scale beyond small tabular settings.

---

**Algorithm 10** Value Decomposition Networks (VDN)

---

**Require:** Learning rate  $\alpha$ , exploration rate  $\epsilon$ , target update interval

- 1: Initialize utility networks  $\theta_1, \dots, \theta_n$ ; initialize target networks  $\bar{\theta}_i \leftarrow \theta_i$
- 2: Initialize shared replay buffer  $\mathcal{D}$
- 3: **for**  $t = 0, 1, 2, \dots$  **do**
- 4:   Collect local observation histories  $h_1^t, \dots, h_n^t$
- 5:   **for**  $i = 1, \dots, n$  **do**
- 6:     With probability  $\epsilon$ , select a random action  $a_i^t$ ; otherwise  $a_i^t \leftarrow \arg \max_{a_i} Q_i(h_i^t, a_i; \theta_i)$
- 7:   **end for**
- 8:   Execute joint action  $a^t$ , observe shared reward  $r^t$  and next histories  $h^{t+1}$
- 9:   Store  $(h^t, a^t, r^t, h^{t+1})$  in  $\mathcal{D}$ ; sample a mini-batch  $\mathcal{B}$  from  $\mathcal{D}$
- 10:   **for** each transition  $(h, a, r, h') \in \mathcal{B}$  **do**
- 11:      $y \leftarrow r + \gamma \sum_{i \in I} \max_{a'_i} Q_i(h'_i, a'_i; \bar{\theta}_i)$
- 12:   **end for**
- 13:   Update  $\theta_1, \dots, \theta_n$  jointly by minimizing  $\mathcal{L}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} (y - \sum_{i \in I} Q_i(h_i, a_i; \theta_i))^2$
- 14:   Periodically update target networks  $\bar{\theta}_i \leftarrow \theta_i$
- 15: **end for**

---



**Algorithm 11** QMIX**Require:** Learning rate  $\alpha$ , exploration rate  $\epsilon$ , target update interval

- 1: Initialize utility networks  $\theta_1, \dots, \theta_n$  and mixing network  $\theta_{\text{mix}}$
- 2: Initialize target networks  $\bar{\theta}_i \leftarrow \theta_i$ ,  $\bar{\theta}_{\text{mix}} \leftarrow \theta_{\text{mix}}$
- 3: Initialize shared replay buffer  $\mathcal{D}$
- 4: **for**  $t = 0, 1, 2, \dots$  **do**
- 5:     Collect local observation histories  $h_1^t, \dots, h_n^t$  and centralized information  $z^t$
- 6:     **for**  $i = 1, \dots, n$  **do**
- 7:         With probability  $\epsilon$ , select a random action  $a_i^t$ ; otherwise  $a_i^t \leftarrow \arg \max_{a_i} Q_i(h_i^t, a_i; \theta_i)$
- 8:     **end for**
- 9:     Execute joint action  $a^t$ , observe shared reward  $r^t$ , next histories  $h^{t+1}$ , next centralized information  $z^{t+1}$
- 10:     Store  $(h^t, z^t, a^t, r^t, h^{t+1}, z^{t+1})$  in  $\mathcal{D}$ ; sample a mini-batch  $\mathcal{B}$  from  $\mathcal{D}$
- 11:     **for** each transition  $(h, z, a, r, h', z') \in \mathcal{B}$  **do**
- 12:         Compute individual utilities  $Q_i(h'_i, a'_i; \bar{\theta}_i)$  for all  $a'_i \in A_i$  and all  $i \in I$
- 13:         Mix via the hypernetwork-conditioned mixing network:  $Q(h', z', a'; \bar{\theta}) \leftarrow f_{\text{mix}}(Q_1, \dots, Q_n; \bar{\theta}_{\text{mix}}(z'))$
- 14:          $y \leftarrow r + \gamma \max_{a'} Q(h', z', a'; \bar{\theta})$
- 15:     **end for**
- 16:     Update  $\theta_1, \dots, \theta_n, \theta_{\text{mix}}$  jointly by minimizing  $\mathcal{L}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} (y - Q(h, z, a; \theta))^2$
- 17:     Periodically update target networks
- 18: **end for**

**4.4.2. Policy-Gradient MARL**

Policy-gradient methods optimize a parameterized policy  $\pi_\theta(a \mid s)$  directly, rather than deriving actions from a learned value function, using exactly the policy gradient theorem introduced in Chapter 3. The basic principle, REINFORCE, updates parameters in the direction of  $\nabla_\theta \log \pi_\theta(a \mid s) \cdot R$ , increasing the probability of actions that led to higher return. Extended independently to multiple agents, each agent applies this update using only its own trajectory and reward, which avoids the exponential joint action space of centralized value-based methods but suffers from high variance that grows with the number of agents, since each agent’s return depends on the changing policies of every other agent. Chapter 3’s variance-reduction tools, the baseline subtraction and Generalized Advantage Estimation used in single-agent policy gradient methods, still apply per agent here, but they only reduce variance from the agent’s own stochastic returns; they do nothing about the variance introduced by other agents’ policies changing underneath it, which is a distinctly multi-agent

problem. This variance problem, together with the non-stationarity of Section 4.3.2, is what motivates the centralized-critic actor-critic and CTDE methods below.

### 4.4.3. Actor-Critic MARL

Actor-critic methods reduce the variance of the plain policy gradient by learning a critic that estimates a value function, used to construct a lower-variance advantage signal for the actor update, exactly as in the single-agent actor-critic algorithms of Chapter 3. Extended independently to MARL in the same way as REINFORCE, actor-critic methods still suffer from non-stationarity, since each agent’s critic is trained on returns that depend on the other agents’ evolving policies. The two representative algorithms below, MADDPG and MAPPO, both address this by giving the critic access to centralized information during training while keeping the actor strictly local, but they differ in how that centralized critic is built and in what problem that main design choice is aimed at solving; each is also a direct multi-agent descendant of a specific algorithm from Chapter 3, named where it is introduced below.

#### 4.4.3.1. Multi-Agent Deep Deterministic Policy Gradient (MADDPG)

MADDPG [29] extends the deterministic policy gradient, and specifically the DDPG algorithm of Chapter 3, to MARL by pairing a decentralized deterministic actor with a centralized critic for each agent. The actor of agent  $i$ ,  $\mu_i(h_i; \phi_i)$ , plays the same role as the single deterministic actor in DDPG, selecting an action from the agent’s own local observation history alone. The critic,  $Q_i^\mu(z, a_1, \dots, a_n; \theta_i)$ , is where MADDPG departs from DDPG: instead of conditioning only on agent  $i$ ’s own state and action as DDPG’s critic does, it additionally conditions on the centralized information  $z$  (the global state) and the actions of every agent, and is used only during training. The actor is updated along the deterministic policy gradient

$$\nabla_{\phi_i} J(\mu_i) = \mathbb{E} \left[ \nabla_{\phi_i} \mu_i(h_i; \phi_i) \nabla_{a_i} Q_i^\mu(z, a_1, \dots, a_n; \theta_i) \Big|_{a_i = \mu_i(h_i)} \right], \quad (4.15)$$

and the critic is trained by minimizing the temporal-difference loss

$$\mathcal{L}(\theta_i) = \mathbb{E} \left[ (Q_i^\mu(z, a; \theta_i) - y)^2 \right], \quad y = r_i + \gamma Q_i^{\mu'}(z', a'_1, \dots, a'_n; \theta_i^-), \quad (4.16)$$

where  $a'_j = \mu'_j(h'_j; \phi_j^-)$  are the actions of target actor networks and  $\theta_i^-$  denotes a target critic network, following the same soft target-network update used by DDPG and TD3 in Chapter 3. Because agent  $i$ ’s critic requires the actions of every other agent, and these are not always observable in practice, agent  $i$  optionally learns an

approximate model  $\hat{\mu}_i^j(a_j \mid h_j; \psi_i^j)$  of each other agent’s policy from agent  $i$ ’s own local perspective, trained by maximum likelihood with an entropy regularizer,

$$\mathcal{L}(\psi_i^j) = -\mathbb{E} \left[ \log \hat{\mu}_i^j(a_j \mid h_j) + \lambda H(\hat{\mu}_i^j) \right], \quad (4.17)$$

which allows the centralized critic to be evaluated using estimated rather than observed opponent actions.

The main contribution of MADDPG, and the one most relevant to this thesis, is not the centralized critic itself but its policy ensemble training scheme. In competitive settings, a single deterministic policy tends to overfit to the current behavior of its opponents: once agent  $A$  learns to exploit agent  $B$ ’s policy, an update to  $B$  can make  $A$ ’s learned behavior ineffective, and training can cycle without converging to robust behavior. MADDPG addresses this by having each agent maintain  $K$  sub-policies  $\{\mu_i^{(k)}\}_{k=1}^K$ , sampling one uniformly at random at the start of each episode. This exposes every agent to a distribution of opponent behaviors during training rather than a single fixed one, which reduces overfitting to any particular opponent and produces policies that are more robust to changes in opponent behavior at test time. This is the direct conceptual predecessor of the adversarial training methods in Chapter 5: M3DDPG is built directly on the MADDPG critic and replaces the policy ensemble with an explicit minimax adversary, using the minimax solution concept of Section 4.2.2.

---

**Algorithm 12** Multi-Agent Deep Deterministic Policy Gradient (MADDPG)
 

---

**Require:** Learning rates  $\alpha_\mu, \alpha_Q$ , ensemble size  $K$ , soft target update rate  $\tau$

- 1: Initialize  $K$  actor networks  $\mu_i^{(1)}, \dots, \mu_i^{(K)}$  with parameters  $\phi_i^{(1)}, \dots, \phi_i^{(K)}$ , for each agent  $i$
- 2: Initialize critic networks  $\theta_i$ ; initialize target networks  $\theta_i^- \leftarrow \theta_i, \phi_i^{(k),-} \leftarrow \phi_i^{(k)}$
- 3: (Optional) Initialize opponent models  $\psi_i^j$  for  $i \neq j$ , used only if opponent actions are not observed
- 4: Initialize shared replay buffer  $\mathcal{D}$
- 5: **for** each episode **do**
- 6:     Sample an active sub-policy index  $k_i \sim \text{Uniform}\{1, \dots, K\}$  for each agent  $i$
- 7:     **for**  $t = 0, 1, 2, \dots$  **do**
- 8:         Select  $a_i^t \leftarrow \mu_i^{(k_i)}(h_i^t; \phi_i^{(k_i)}) + \mathcal{N}_t$  (exploration noise) for each agent  $i$
- 9:         Execute joint action  $a^t$ , observe rewards  $r^t$ , next centralized information  $z^{t+1}$ , next histories  $h^{t+1}$
- 10:         Store  $(z^t, h^t, a^t, r^t, z^{t+1}, h^{t+1})$  in  $\mathcal{D}$
- 11:         Sample a mini-batch  $\mathcal{B}$  from  $\mathcal{D}$
- 12:         **for** each agent  $i$  **do**
- 13:             **if** opponent actions are not observed **then**
- 14:                 Update opponent model  $\psi_i^j$  for each  $j \neq i$  by minimizing  $\mathcal{L}(\psi_i^j)$
- 15:                 Substitute estimated actions  $\hat{a}_j \sim \hat{\mu}_i^j(\cdot | h_j; \psi_i^j)$  for the unobserved opponent actions
- 16:             **end if**
- 17:             Compute target  $y \leftarrow r_i + \gamma Q_i^{\mu'}(z', a'_1, \dots, a'_n; \theta_i^-)$ , with  $a'_j = \mu_j^{(k_j),-}(h'_j; \phi_j^{(k_j),-})$
- 18:             Update critic:  $\theta_i \leftarrow \theta_i - \alpha_Q \nabla_{\theta_i} \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} (Q_i^\mu(z, a; \theta_i) - y)^2$
- 19:             Update active actor:  $\phi_i^{(k_i)} \leftarrow \phi_i^{(k_i)} + \alpha_\mu \nabla_{\phi_i^{(k_i)}} J(\mu_i^{(k_i)})$
- 20:             Soft-update target networks:  $\theta_i^- \leftarrow \tau \theta_i + (1 - \tau) \theta_i^-$ , and likewise for  $\phi_i^{(k_i),-}$
- 21:         **end for**
- 22:     **end for**
- 23: **end for**

---

#### 4.4.3.2. Multi-Agent Proximal Policy Optimization (MAPPO)

MAPPO [30] applies the same centralized-critic principle to an on-policy, PPO-style actor-critic update instead of MADDPG's off-policy deterministic gradient. PPO itself does not appear as its own algorithm in Chapter 3, but it is the clipped, first-order simplification of the trust-region idea developed there for TRPO: both keep each policy update close to the previous policy to guarantee stable improvement,

but where TRPO enforces this with a KL-divergence constraint solved via conjugate gradients, PPO enforces it with the clipped surrogate objective at a fraction of the computational cost. The environment is treated as a special case of the POSG of Section 4.1.2 with a shared reward, i.e. a decentralized POMDP (Dec-POMDP), and the objective is the expected discounted return of the joint policy already defined in Equation (4.6). Execution is decentralized, with each agent’s actor conditioning only on its own local history,  $\pi_\theta(a_i | h_i)$ , while the critic  $V_\phi(z)$  is centralized and conditions on the global state during training. A further design choice, distinct from the centralized critic itself, is that MAPPO typically shares parameters across all agents: every agent uses the same  $\theta$  and  $\phi$ , rather than maintaining separate networks as in MADDPG. This parameter sharing assumes the agents are homogeneous and trades some representational flexibility for a substantially smaller parameter count, better sample efficiency, and improved generalization across agents, which is part of why MAPPO has been shown to match or exceed off-policy MARL baselines such as QMIX and MADDPG despite the general reputation of on-policy methods as less sample-efficient.

Training proceeds by collecting trajectories under the current policy and computing a generalized advantage estimate, using the same GAE construction introduced for the single-agent case in Chapter 3, now applied to the centralized critic  $V_\phi(z)$  rather than a local one,

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_{\phi_{\text{old}}}(z_{t+1}) - V_{\phi_{\text{old}}}(z_t), \quad (4.18)$$

after which the actor and critic are updated jointly for several epochs on the same batch using the clipped surrogate objective

$$L_\pi(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad r_t(\theta) = \frac{\pi_\theta(a_t | h_t)}{\pi_{\theta_{\text{old}}}(a_t | h_t)}, \quad (4.19)$$

together with a clipped value loss  $L_V(\phi)$  and an entropy bonus  $L_H(\theta)$  that encourages continued exploration; the full training objective is  $L(\theta, \phi) = L_\pi(\theta) - c_v L_V(\phi) + c_e L_H(\theta)$ . The clipping in both the policy and value objectives keeps each update close to the previous policy, which is what allows MAPPO to reuse each batch of trajectories for multiple epochs of updates without the instability that would otherwise result from a large importance-sampling ratio  $r_t(\theta)$ .

---

**Algorithm 13** Multi-Agent Proximal Policy Optimization (MAPPO)
 

---

**Require:** Initial shared policy parameters  $\theta$ , shared value parameters  $\phi$ , clipping parameter  $\epsilon$ , value-loss coefficient  $c_v$ , entropy coefficient  $c_e$ , epochs  $K$

```

1: while not converged do
2:   Collect trajectories  $\mathcal{D} = \{(z_t, h_t, a_t, r_t, z_{t+1})\}$  using  $\pi_{\theta_{\text{old}}}$ , shared across all agents
3:   for each timestep  $t$  do
4:     Compute TD residual:  $\delta_t \leftarrow r_t + \gamma V_{\phi_{\text{old}}}(z_{t+1}) - V_{\phi_{\text{old}}}(z_t)$ 
5:     Compute GAE advantage:  $\hat{A}_t \leftarrow \sum_{l=0}^{T-t-1} (\gamma \lambda)^l \delta_{t+l}$ 
6:     Compute return target:  $R_t \leftarrow \hat{A}_t + V_{\phi_{\text{old}}}(z_t)$ 
7:     Normalize advantages:  $\hat{A}_t \leftarrow (\hat{A}_t - \mu_A) / (\sigma_A + \eta)$ 
8:   end for
9:   for  $k = 1, \dots, K$  epochs do
10:    Compute importance ratio:  $r_t(\theta) \leftarrow \pi_{\theta}(a_t | h_t) / \pi_{\theta_{\text{old}}}(a_t | h_t)$ 
11:    Compute clipped policy objective:  $L_{\pi}(\theta) \leftarrow \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$ 
12:    Compute clipped value estimate:  $V_t^{\text{clip}} \leftarrow V_{\phi_{\text{old}}}(z_t) + \text{clip}(V_{\phi}(z_t) - V_{\phi_{\text{old}}}(z_t), -\epsilon, \epsilon)$ 
13:    Compute value loss:  $L_V(\phi) \leftarrow \mathbb{E}_t [\max((V_{\phi}(z_t) - R_t)^2, (V_t^{\text{clip}} - R_t)^2)]$ 
14:    Compute entropy bonus:  $L_H(\theta) \leftarrow \mathbb{E}_t [H(\pi_{\theta}(\cdot | h_t))]$ 
15:    Compute total objective:  $L(\theta, \phi) \leftarrow L_{\pi}(\theta) - c_v L_V(\phi) + c_e L_H(\theta)$ 
16:    Update shared actor:  $\theta \leftarrow \theta + \alpha_{\pi} \nabla_{\theta} L(\theta, \phi)$ 
17:    Update shared critic:  $\phi \leftarrow \phi - \alpha_V \nabla_{\phi} L_V(\phi)$ 
18:   end for
19:    $\theta_{\text{old}} \leftarrow \theta, \phi_{\text{old}} \leftarrow \phi$ 
20: end while
    
```

---

Both MADDPG and MAPPO are used later in this thesis as actor-critic baselines, and both are specific instances of the CTDE framework introduced next: MADDPG centralizes the critic over the joint action, while MAPPO centralizes it over the global state alone, which is the more common choice when the reward is shared and the critic does not need to reason about individual agents' actions.

#### 4.4.4. Centralized Training with Decentralized Execution

Centralized Training with Decentralized Execution (CTDE) is the training paradigm underlying both the value-decomposition methods of Section 4.4.1 and the actor-critic methods of Section 4.4.3. During training, agents may access additional global

information  $\mathbf{z}$  (the global state, joint observations, or joint actions); during execution, each agent selects actions using only its own local observation history  $\mathbf{h}_i$ . Because the centralized information is used only to train the critic or to build the centralized action-value function, it can be discarded after training without breaking decentralized execution.

For actor-critic methods, the theoretical basis is the multi-agent policy gradient theorem, which extends the single-agent policy gradient to account for the joint behavior of all agents,

$$\nabla_{\phi_i} J(\phi_i) \propto \mathbb{E} \left[ Q_i^\pi(\hat{\mathbf{h}}, \langle \mathbf{a}_i, \mathbf{a}_{-i} \rangle) \nabla_{\phi_i} \log \pi_i(\mathbf{a}_i \mid \mathbf{h}_i; \phi_i) \right], \quad (4.20)$$

where the centralized critic  $Q_i^\pi$  evaluates the full joint action rather than agent  $i$ 's action alone [31]. CTDE is the training paradigm this thesis adopts for its baselines, since it directly addresses the non-stationarity problem of Section 4.3.2 without requiring communication at execution time, and it is the paradigm both robustness (adversarial CTDE critics in Chapter 5) and adaptability (CTDE for meta-RL in Chapter 7) build on directly.

## 4.5. Selection of Baseline Algorithms

This chapter reviewed the representative MARL learning paradigms, Q-learning-based, policy-gradient, and actor-critic, in order to identify suitable baseline algorithms on top of which the robustness and adaptability mechanisms of this thesis are implemented. VDN and QMIX represent the value-decomposition family; MADDPG and MAPPO represent the centralized-critic actor-critic family; both families are trained under the CTDE paradigm, which directly targets the non-stationarity problem central to this thesis. The tabular and purely game-theoretic algorithms discussed in Sections 4.2 and 4.3 (Nash-Q, CE-Q, no-regret learning) are retained only where they clarify why these baselines are structured the way they are; they are not carried forward as experimental baselines themselves, since they do not scale to the deep, high-dimensional settings this thesis targets.

Viewed against Chapter 3, none of these four baselines introduces a new single-agent learning rule: VDN and QMIX still update Q-functions with the same temporal-difference target as DQN, MADDPG still follows the deterministic policy gradient of DDPG, and MAPPO still follows the clipped surrogate objective of PPO with GAE-based advantages. What changes, in every case, is the addition of centralized information during training under CTDE and, for VDN and QMIX, a factorization of the value function into per-agent components. This is the throughline of the whole chapter: the single-agent tools of Chapter 3 remain the computational core of

every baseline used later in this thesis, and the multi-agent machinery introduced here exists specifically to keep those tools usable once non-stationarity, from other learning agents, breaks the assumptions they were built under.



---

## 5. Robust Multi Agent Reinforcement Learning

Multi-agent reinforcement learning has achieved strong results in many cooperative and competitive tasks. Most existing algorithms, however, assume that the environment remains stable during both training and deployment. In practice, this assumption rarely holds. Real-world systems are often affected by uncertainty, unexpected events, and changes that were not observed during training.

Consider a team of autonomous vehicles driving through a city. Sensors may produce noisy measurements because of rain or fog. Communication between vehicles may be delayed or interrupted. One vehicle may fail, while another encounters an obstacle that changes the traffic flow. Similar challenges appear in robot teams, wireless communication networks, and autonomous drone swarms. Since the agents depend on each other's decisions, even a small disturbance can spread through the system and reduce the overall performance.

These challenges motivate the study of robust multi-agent reinforcement learning (Robust MARL). Instead of optimizing only for the average training environment, Robust MARL aims to learn policies that continue to perform well when the environment differs from the one seen during training. A robust policy does not need to be optimal under every possible condition. Instead, it should maintain acceptable performance even when facing uncertainty, disturbances, or adversarial behavior.

Robustness in MARL refers to the ability of a multi-agent system to consistently achieve its objectives despite uncertainties such as observation noise, environmental changes, communication failures, agent failures, model uncertainty, or adversarial attacks. The goal is to limit the performance degradation caused by these disturbances while preserving cooperation among the agents.

Throughout this chapter, we examine the main sources of uncertainty that affect MARL systems, discuss common robustness evaluation metrics, and review the major classes of robust MARL algorithms. These methods include adversarial training, adversarial regularization, distributionally robust optimization, Bayesian approaches, and robust mean-field methods. Together, they provide different strategies for maintaining reliable performance under uncertain and changing conditions.

## 5.1. Definition of Robustness in Multi-Agent Reinforcement Learning

The objective of robust multi-agent reinforcement learning is to learn policies that maintain good performance despite uncertainty. Unlike standard MARL, which maximizes the expected return in a fixed environment, Robust MARL explicitly accounts for disturbances that may occur during training or deployment. These disturbances include noisy observations, uncertain dynamics, communication failures, agent failures, and adversarial attacks.

Consider a Markov game with  $N$  agents. Let  $\pi = (\pi_1, \dots, \pi_N)$  denote the joint policy of all agents. The performance of this policy in the nominal environment is measured by the expected cumulative return

$$J(\pi) = \mathbb{E}_{\mathcal{E}}[R], \quad (5.1)$$

where  $\mathcal{E}$  denotes the nominal environment and  $R$  is the cumulative discounted return obtained by following the joint policy  $\pi$ .

In practice, the environment is rarely known exactly and may differ from the nominal setting because of uncertainty or disturbances. Let  $\Delta$  denote the ambiguity set, which contains all possible environment perturbations that the policy may encounter. Each disturbance  $\delta \in \Delta$  defines a perturbed environment, denoted by  $\mathcal{E}_{\delta}$ .

Instead of maximizing the expected return only in the nominal environment, Robust MARL seeks a policy that performs well under the worst disturbance in the ambiguity set. The robust objective is therefore defined as

$$J_{\text{robust}}(\pi) = \min_{\delta \in \Delta} \mathbb{E}_{\mathcal{E}_{\delta}}[R], \quad (5.2)$$

where the expectation is computed in the perturbed environment  $\mathcal{E}_{\delta}$ . The minimization over  $\Delta$  represents the worst-case disturbance that produces the lowest expected return.

The goal of the learning algorithm is to find the joint policy that maximizes this worst-case performance,

$$\pi^* = \arg \max_{\pi} \min_{\delta \in \Delta} \mathbb{E}_{\mathcal{E}_{\delta}}[R]. \quad (5.3)$$

This minimax objective forms the foundation of many robust reinforcement learning methods. The ambiguity set may represent observation noise, transition uncertainty,

communication failures, adversarial attacks, distribution shift, or any combination of these uncertainties. A policy learned using this objective may sacrifice some nominal performance, but it is less sensitive to unexpected changes and achieves more reliable performance across different operating conditions.

## 5.2. Sources of Uncertainty in Multi-Agent Reinforcement Learning

The performance of a multi-agent reinforcement learning system depends on the assumptions made during training. When these assumptions no longer hold, the learned policy may perform poorly. Robust MARL addresses this problem by preparing agents to operate under different sources of uncertainty rather than only in the nominal environment.

Uncertainty can originate from the environment, the agents themselves, or interactions between the agents and their surroundings. Some disturbances are random, such as sensor noise, while others are intentional, such as adversarial attacks. In many real-world applications, several uncertainty sources occur at the same time. Understanding these sources is essential because each one requires different robustness techniques.

### 5.2.1. Observation and Action Noise

Observation and action noise are common sources of uncertainty in real-world multi-agent systems. They affect different stages of the interaction between an agent and its environment. Observation noise corrupts the information received by the agent, whereas action noise alters the control command executed. Both can significantly reduce the quality of the learned policy and the coordination among agents.

Observation noise occurs when an agent receives inaccurate, incomplete, or corrupted information about its surroundings. Since reinforcement learning policies select actions based on observations, noisy sensory inputs often lead to incorrect decisions. Common sources of observation noise include camera blur, LiDAR measurement errors, GPS inaccuracies, missing sensor readings, and electromagnetic interference. In cooperative tasks, inconsistent observations across agents can also reduce the quality of coordination.

Action noise occurs when the action executed by the environment differs from the action selected by the policy. This discrepancy is usually caused by actuator imperfections, mechanical wear, communication delays, control latency, or external

disturbances. For example, a mobile robot may command a wheel velocity of 1 m/s but execute a lower velocity because of wheel slip. Similarly, an aerial vehicle may deviate from its planned trajectory because of wind or actuator limitations.

Robust MARL methods address observation and action noise by exposing agents to noisy conditions during training, learning robust state representations, estimating hidden states, or optimizing policies that are less sensitive to execution errors. The objective is to maintain reliable performance even when sensing and control are imperfect.

### 5.2.2. Non-Stationary Environments

A non-stationary environment is one whose underlying properties change over time. Unlike a stationary environment, where the transition and reward functions remain fixed, a non-stationary environment may exhibit changes in its dynamics, reward structure, or both. As a result, a policy that performs well during training may become suboptimal after deployment.

Transition uncertainty arises when the probability of moving from one state to another changes over time. These changes may result from varying physical conditions, unknown system dynamics, or modifications to the environment. For example, changes in road friction, payload, weather conditions, or robot dynamics alter the effect of the same action and therefore change the transition function.

Reward uncertainty occurs when the objective of the task changes during execution. The reward function may evolve because of changing mission requirements, delayed rewards, or modifications to the task objective. For example, a warehouse robot may initially prioritize minimizing delivery time but later shift its objective to reducing energy consumption during peak electricity demand.

In multi-agent systems, non-stationarity also arises because agents learn and update their policies simultaneously. From the perspective of an individual agent, the behavior of other agents becomes part of the environment. As teammates improve their policies, the environment continuously changes, making the transition dynamics and expected rewards non-stationary. This phenomenon is one of the main challenges in multi-agent reinforcement learning and often leads to unstable learning.

Robust MARL methods improve performance under moderate environmental changes by optimizing conservative policies that are less sensitive to uncertainty. However, when the environment changes continuously or the task itself evolves, maintaining a fixed robust policy may no longer be sufficient. In these situations, the agents must adapt their behavior to the new conditions rather than simply tolerate them.

The next chapter focuses on *Adaptive Multi-Agent Reinforcement Learning*, which studies algorithms that detect environmental changes and update their policies accordingly. These methods enable agents to learn new behaviors, transfer previously acquired knowledge, and maintain high performance in continuously changing environments.

### 5.2.3. Communication Failures

Many cooperative MARL systems rely on communication to exchange observations, intentions, or learned representations. In practice, communication channels may suffer from packet loss, latency, bandwidth limitations, or temporary disconnections.

Communication failures reduce the quality of shared information and often degrade cooperation among agents. Robust MARL algorithms address this problem by learning decentralized policies, communication-efficient protocols, or strategies that remain effective even when communication is unreliable.

### 5.2.4. Adversarial Attacks

Adversarial attacks intentionally manipulate the observations, actions, rewards, or communication between agents to reduce the performance of a reinforcement learning policy. Unlike random disturbances, adversarial perturbations are carefully optimized to maximize the degradation of the learned policy while remaining as small as possible. In MARL, these attacks are particularly harmful because a successful attack on a single agent can propagate through the team and degrade the overall system performance.

Most adversarial attacks formulate the perturbation as a constrained optimization problem. Let  $X$  denote the original observation and  $\delta$  be an adversarial perturbation. The objective is to find the perturbation that maximizes the learning objective while remaining within a bounded perturbation budget,

$$\delta^* = \arg \max_{\|\delta\| \leq \epsilon} L(\theta, X + \delta), \quad (5.4)$$

where  $L(\theta, \cdot)$  denotes the loss function of the policy network,  $\theta$  represents the network parameters, and  $\epsilon$  controls the maximum magnitude of the perturbation. The constraint ensures that the perturbation remains small enough to be difficult to detect while still causing a significant reduction in policy performance.

One of the earliest and most widely used attack methods is the Fast Gradient Sign Method (FGSM) [32]. FGSM computes a single gradient step in the direction that maximizes the loss,

$$\delta = \epsilon \operatorname{sign}(\nabla_X L(\theta, X)), \quad (5.5)$$

and generates the adversarial observation

$$X_{\text{adv}} = X + \delta. \quad (5.6)$$

Since FGSM requires only one gradient evaluation, it is computationally efficient and is commonly used to evaluate the robustness of reinforcement learning policies.

Projected Gradient Descent (PGD) [33] extends FGSM by performing multiple gradient ascent steps instead of a single update. After each iteration, the perturbation is projected back onto the feasible set defined by the perturbation budget,

$$X^{k+1} = \Pi_{\mathcal{B}_\epsilon(X)} \left( X^k + \alpha \operatorname{sign}(\nabla_X L(\theta, X^k)) \right), \quad (5.7)$$

where  $\alpha$  is the step size and  $\Pi_{\mathcal{B}_\epsilon(X)}$  denotes the projection onto the  $\epsilon$ -ball centered at the original observation. Because PGD performs iterative optimization, it generally produces stronger adversarial examples than FGSM and is often regarded as one of the strongest first-order adversarial attacks.

Although FGSM and PGD are widely used in single-agent reinforcement learning, attacking cooperative MARL systems is considerably more challenging. A wrong action selected by one agent does not necessarily reduce the team reward because the remaining agents may compensate for the mistake. Furthermore, cooperative MARL often relies on a shared team reward, making it difficult to determine which individual action is most harmful. These characteristics reduce the effectiveness of conventional adversarial attacks. [34]

To address this challenge, recent work introduces the concept of *attackability*[34], which measures how easily an agent can be forced to execute a harmful action. Attackability is determined by the difference between the value of the optimal action and the value of a target adversarial action. A large difference makes the attack more difficult because the policy strongly prefers the optimal action. Therefore, the attacker jointly optimizes both the harmfulness of the target action and its attackability using the objective

$$L = (y_{\text{target}} - Q_{\text{target}})^2 + \lambda (Q(a_{\text{target}}) - Q(a_{\text{best}}))^2, \quad (5.8)$$

where the first term encourages the target action to minimize the team reward, while the second term reduces the value difference between the target action and the optimal action, making the attack easier to achieve. The parameter  $\lambda$  controls the trade-off between attack effectiveness and attackability.

The same work proposes Dynamic Jacobian-based Saliency Map Attack (D-JSMA), which extends the classical Jacobian-based Saliency Map Attack (JSMA) [35]. Standard JSMA perturbs only a single input feature, making it ineffective in cooperative MARL because changing one feature often increases the values of multiple actions simultaneously. D-JSMA overcomes this limitation by perturbing pairs of input features according to a saliency score computed from the gradients of the target action and dynamically adjusting the perturbation magnitude until a successful attack is obtained. This strategy produces stronger attacks while requiring only a small number of modified observation features.

The Jacobian-based Saliency Map Attack (JSMA) generates sparse adversarial perturbations by modifying only the most influential input feature. Instead of perturbing all observation dimensions, JSMA computes the Jacobian of the policy or value network with respect to the input observations and constructs a saliency map that identifies the feature most likely to change the selected action. For a target action  $t$ , the saliency score of feature  $i$  is defined as

$$S(X, t)[i] = \begin{cases} 0, & \frac{\partial Q_t(X)}{\partial X_i} < 0 \text{ or } \sum_{k \neq t} \frac{\partial Q_k(X)}{\partial X_i} > 0, \\ \frac{\partial Q_t(X)}{\partial X_i} \left| \sum_{k \neq t} \frac{\partial Q_k(X)}{\partial X_i} \right|, & \text{otherwise,} \end{cases} \quad (5.9)$$

where  $F_t(X)$  denotes the output corresponding to the target action. The attacker selects the feature with the highest saliency score,

$$i^* = \arg \max_i S(X, t)[i], \quad (5.10)$$

and perturbs only this feature. This strategy produces sparse adversarial examples while minimizing the amount of modified information.

Although effective for image classification, standard JSMA is less suitable for cooperative MARL because perturbing a single observation feature often changes the values of multiple actions simultaneously. As a result, modifying only one feature is frequently insufficient to force an agent to execute a harmful action. To overcome this limitation, Dynamic Jacobian-based Saliency Map Attack (D-JSMA) perturbs

two observation features simultaneously and introduces an adaptive perturbation magnitude. The saliency score is computed jointly for a pair of features  $(i, j)$ ,

$$S(X, t)[i, j] = \begin{cases} 0, & \left( \frac{\partial Q_t}{\partial X_i} + \frac{\partial Q_t}{\partial X_j} \right) \left( \sum_{k \neq t} \frac{\partial Q_k}{\partial X_i} + \frac{\partial Q_k}{\partial X_j} \right) > 0, \\ - \left( \frac{\partial Q_t}{\partial X_i} + \frac{\partial Q_t}{\partial X_j} \right) \left( \sum_{k \neq t} \frac{\partial Q_k}{\partial X_i} + \frac{\partial Q_k}{\partial X_j} \right), & \text{otherwise,} \end{cases} \quad (5.11)$$

and the perturbation direction is determined by

$$d_{\text{perturb}} = \text{sign} \left( \frac{\partial Q_t}{\partial X_i} + \frac{\partial Q_t}{\partial X_j} \right). \quad (5.12)$$

By perturbing two features simultaneously and gradually increasing the perturbation magnitude, D-JSMA generates more effective adversarial examples while preserving a small perturbation budget.

In cooperative MARL, attacking every agent is often unnecessary and computationally expensive. Instead, the attacker may first identify the most influential or vulnerable agent and generate adversarial perturbations only for that agent's observations. Since the behavior of one agent can influence the decisions of its teammates, successfully compromising a single critical agent may be sufficient to substantially reduce the overall team reward. This targeted attack strategy improves computational efficiency while maintaining a high attack success rate.

### 5.3. Taxonomy of Robust Multi-Agent Reinforcement Learning

Robust MARL methods have been developed to address different sources of uncertainty. Since the underlying assumptions and types of disturbances vary across applications, no single method is suitable for every scenario. As a result, existing approaches differ in how they model uncertainty and improve policy robustness.

Most robust MARL algorithms can be grouped according to the strategy they use to handle uncertainty. Some methods expose agents to difficult situations during training, allowing them to learn policies that perform well under disturbances. Others modify the learning objective by introducing regularization terms that encourage stable representations and reduce sensitivity to perturbations. A different class of



methods model uncertainty explicitly through robust optimization or probabilistic inference, providing stronger theoretical guarantees under uncertain environments.

Based on the robustness mechanism employed, the methods discussed in this chapter are classified into the following categories:

- **Adversarial Training:** Trains agents against an adversary that generates worst-case disturbances during learning. The objective is to improve robustness by repeatedly solving a minimax optimization problem. as in M3DDPG [36] and RARL [37]
- **Adversarial Regularization:** Introduces regularization terms into the optimization objective to reduce the sensitivity of learned policies or value functions to perturbations without explicitly training against an adversary. as in MIR3 [38], ERNIE [39], SA-MDP [40], and RADIAL-RL [41]
- **Distributionally Robust MARL:** Optimizes the policy against the worst-case probability distribution within an ambiguity set rather than a single nominal environment. This approach provides robustness against distribution shift and model uncertainty as in DrIGM[42].
- **Bayesian Robust MARL:** Represents uncertainty using probability distributions over model parameters, value functions, or policies. Bayesian inference enables agents to reason about uncertainty and make more reliable decisions under limited or noisy information.
- **Robust Mean-Field MARL:** Approximates interactions among a large number of agents using mean-field theory while incorporating robustness against perturbations and adversarial disturbances. This approach improves scalability without sacrificing robustness.

Each category addresses uncertainty from a different perspective and is suitable for different application scenarios. The following sections examine these approaches in detail, beginning with adversarial training methods and continuing through the remaining categories.

## 5.4. Adversarial Training

Adversarial training is one of the most widely used approaches for improving robustness in reinforcement learning. The main idea is to expose agents to challenging or worst-case situations during training so that they learn policies capable of maintaining good performance under uncertainty. Rather than training only in the nominal

environment, an adversary deliberately introduces disturbances that make the learning task more difficult.

The adversary may perturb observations, modify actions, alter environment dynamics, or generate difficult initial conditions. By repeatedly interacting with these disturbances, the learning agent becomes less sensitive to uncertainty and develops policies that generalize better to unseen environments. This idea follows the principle that training under harder conditions prepares the agent for a wider range of situations during deployment.

Most adversarial training methods formulate learning as a two-player zero-sum game. One player is the learning agent, whose objective is to maximize the expected return, while the other player is an adversary that seeks to minimize the agent’s performance by generating worst-case perturbations. The interaction between the two players can be expressed as the following minimax optimization problem,

$$\pi^* = \arg \max_{\pi} \min_{\psi} \mathbb{E} [R(\pi, \psi)], \quad (5.13)$$

where  $\pi$  denotes the policy of the learning agent,  $\psi$  represents the adversarial policy, and  $R(\pi, \psi)$  is the cumulative return obtained when both policies interact. The adversary searches for perturbations that minimize the return, while the learning agent continuously updates its policy to maximize its performance against these worst-case disturbances.

The exact form of the adversary depends on the application. In some methods, the adversary injects observation or action perturbations. In others, it modifies the environment dynamics, applies external forces, changes initial states, or controls competing agents. Although the implementation differs across algorithms, the objective remains the same: to learn a policy that performs reliably under difficult operating conditions.

Adversarial training has become a fundamental framework for robust reinforcement learning because it directly optimizes worst-case performance. Many robust MARL algorithms build upon this formulation, including Robust Adversarial Reinforcement Learning (RARL), M3DDPG, and several recent adversarial regularization methods.

#### 5.4.1. Robust Adversarial Reinforcement Learning (RARL)

Robust Adversarial Reinforcement Learning (RARL), proposed by Pinto *et al.* [37], is one of the earliest methods designed to improve the robustness of reinforcement learning policies against environmental uncertainty. Instead of exposing the learning

agent to randomly generated disturbances, RARL introduces an intelligent adversary that actively searches for perturbations that reduce the agent's performance. The protagonist is therefore trained under increasingly difficult conditions, leading to policies that generalize better to unseen environments.

The main motivation behind RARL is that random disturbances are often too weak to expose the weaknesses of a policy. In contrast, an adversarial agent continuously adapts its behavior to generate challenging situations that maximize the protagonist's difficulty. As the protagonist learns to overcome these disturbances, the adversary also becomes stronger, creating a competitive learning process that progressively improves robustness.

RARL formulates this interaction as a two-player zero-sum game consisting of a protagonist and an adversary. The protagonist seeks to maximize the cumulative reward, whereas the adversary attempts to minimize the protagonist's reward by applying disturbances to the environment. Let  $\mu$  denote the protagonist policy and  $\nu$  the adversarial policy. Their optimization objectives are given by

$$\max_{\mu} R(\mu, \nu), \quad (5.14)$$

and

$$\min_{\nu} R(\mu, \nu), \quad (5.15)$$

which together form the minimax optimization problem

$$\mu^* = \arg \max_{\mu} \min_{\nu} R(\mu, \nu). \quad (5.16)$$

The adversary does not directly control the protagonist. Instead, it modifies the environment dynamics by injecting disturbances into the system. Consequently, the transition function depends on the actions of both agents,

$$s_{t+1} = P(s_t, a_t^P, a_t^A), \quad (5.17)$$

where  $a_t^P$  denotes the protagonist's action and  $a_t^A$  represents the disturbance generated by the adversary. The protagonist receives the cumulative reward

$$R_P = \sum_t r_t, \quad (5.18)$$

while the adversary receives the opposite reward,

$$R_A = - \sum_t r_t. \quad (5.19)$$

This reward structure ensures that both agents have completely opposing objectives, encouraging the adversary to discover increasingly difficult disturbances while forcing the protagonist to learn policies that remain effective under these conditions.

The training procedure is summarized in Algorithm 14. First, the protagonist and adversary policies are initialized. During each training iteration, the adversary is frozen while the protagonist interacts with the environment and updates its policy using trajectories collected under the current adversarial disturbances. Once the protagonist has completed its updates, its policy is fixed, and the adversary is trained to generate stronger perturbations that further reduce the protagonist’s return. These two optimization phases alternate until convergence. In the original implementation, both policies are represented by independent neural networks and optimized using Trust Region Policy Optimization (TRPO) [13].

The alternating optimization between the protagonist and the adversary gradually increases the difficulty of the training environment. As the adversary discovers stronger perturbations, the protagonist learns policies that are less sensitive to environmental uncertainty and unexpected disturbances. Compared with conventional training under nominal conditions, this strategy often produces policies with better robustness and improved generalization to unseen environments.

Despite these advantages, RARL has several limitations. Training two policies simultaneously increases the computational cost and may lead to unstable optimization if one player improves much faster than the other. Furthermore, the learned policy is only robust against the types of disturbances generated by the adversary during training. If the deployment environment contains significantly different disturbances, the robustness gained from adversarial training may be limited.

#### 5.4.2. Minimax Multi-Agent Deep Deterministic Policy Gradient (M3DDPG)

Although Robust Adversarial Reinforcement Learning (RARL) improves the robustness of single-agent reinforcement learning, it cannot be directly applied to multi-agent environments. In MARL, each agent interacts with several learning agents whose policies continuously change during training. As a result, uncertainty arises not only from the environment but also from the behavior of other agents. Standard MADDPG addresses the non-stationarity problem by employing a centralized

**Algorithm 14** Robust Adversarial Reinforcement Learning (RARL)

---

**Require:** Environment  $\mathcal{E}$ , protagonist policy  $\mu_{\theta_\mu}$ , adversary policy  $\nu_{\theta_\nu}$ , training iterations  $N$ , protagonist updates  $N_\mu$ , adversary updates  $N_\nu$ **Ensure:** Robust protagonist policy  $\mu_{\theta_\mu}$ 

```

1: Initialize protagonist parameters  $\theta_\mu$ 
2: Initialize adversary parameters  $\theta_\nu$ 
3: for  $i = 1$  to  $N$  do
4:   Freeze adversary policy  $\nu_{\theta_\nu}$ 
5:   for  $j = 1$  to  $N_\mu$  do
6:     Collect trajectories using protagonist  $\mu_{\theta_\mu}$  and fixed adversary  $\nu_{\theta_\nu}$ 
7:     Compute protagonist returns
8:     Update  $\theta_\mu$  using TRPO
9:   end for
10:  Freeze protagonist policy  $\mu_{\theta_\mu}$ 
11:  for  $j = 1$  to  $N_\nu$  do
12:    Collect trajectories using fixed protagonist  $\mu_{\theta_\mu}$  and adversary  $\nu_{\theta_\nu}$ 
13:    Compute adversary returns as  $-R$ 
14:    Update  $\theta_\nu$  using TRPO
15:  end for
16: end for
17: return Robust protagonist policy  $\mu_{\theta_\mu}$ 

```

---

critic, but it does not explicitly optimize for robustness. Consequently, agents often become specialized to the teammates or opponents encountered during training and may perform poorly when their behaviors change during deployment. [43]

To overcome this limitation, Li *et al.* proposed the Minimax Multi-Agent Deep Deterministic Policy Gradient (M3DDPG) algorithm [36]. The central idea is to train each agent against the worst possible actions of the other agents instead of their nominal actions. Rather than assuming that teammates or opponents always behave as expected, M3DDPG optimizes each policy for the most unfavorable joint action, thereby improving robustness against changes in the behavior of other agents.

For agent  $i$ , the robust objective is formulated as

$$J_M(\theta_i) = \min_{a_{-i}} \mathbb{E}[R_i], \quad (5.20)$$

where  $a_{-i}$  denotes the actions of all agents except agent  $i$ . This objective encourages the policy to maximize its expected return under the worst possible behavior of the remaining agents.

To incorporate this objective into actor-critic learning, M3DDPG modifies the Bellman equation of the centralized critic. Instead of evaluating the expected value of the next joint action, the critic estimates the worst-case value,

$$Q_{M,i}(s, a) = r + \gamma \mathbb{E} \left[ \min_{a'} Q_{M,i}^{u'}(s', a') \mid a'_i = \mu_i(s') \right]. \quad (5.21)$$

This formulation requires solving an optimization problem over the actions of the other agents at every update step. Since solving this optimization exactly is computationally expensive in continuous action spaces, M3DDPG introduces the Multi-Agent Adversarial Learning (MAAL) approximation. Instead of computing the exact worst-case action, the algorithm performs a single gradient step that moves each opponent's action toward the direction that minimizes the critic value,

$$\hat{\epsilon}_j = -\alpha \nabla_{a_j} Q_{M,i}(s, a), \quad (5.22)$$

where  $\alpha$  controls the magnitude of the perturbation. The perturbed action is then computed as

$$a_j^* = a_j + \hat{\epsilon}_j. \quad (5.23)$$

This approximation is conceptually similar to the Fast Gradient Sign Method (FGSM) [32], used in adversarial machine learning. However, instead of perturbing observations, M3DDPG perturbs the actions of the other agents to generate stronger adversarial behaviors during training.

The overall training procedure is summarized in Algorithm 15. At each interaction step, all agents execute their actions and store the resulting transition in a replay buffer. During learning, the target actors first generate the next actions. Adversarial perturbations are then computed for the actions of the other agents using the gradient of the centralized critic. These perturbed actions are used to compute the worst-case target value for updating the critic. After updating the critic, the actor is optimized using the adversarially perturbed joint action, followed by a soft update of the target networks. By repeatedly optimizing against these worst-case opponent actions, each agent learns a policy that is more robust to changes in the behavior of other agents.

Compared with MADDPG, M3DDPG significantly improves robustness to opponent policy changes and adversarial behaviors while preserving the advantages of centralized training with decentralized execution. However, the method increases the computational cost because adversarial perturbations must be computed during both the actor and critic updates. Moreover, the one-step gradient approximation

provides only a local estimate of the worst-case action and may not always identify the true adversarial strategy.

## 5.5. Adversarial Regularization

Adversarial training improves robustness by exposing agents to worst-case disturbances generated by an adversary during training. Although this approach has been successful in many reinforcement learning tasks, jointly training an adversarial policy and a learning policy substantially increases the computational cost and may introduce instability during optimization. In addition, the robustness achieved by adversarial training is often limited to the types of disturbances produced by the adversary.

Adversarial regularization addresses these limitations by improving robustness through modifications to the learning objective rather than introducing an explicit adversarial agent. Instead of repeatedly solving a minimax game, these methods add regularization terms that encourage policies or value functions to become less sensitive to perturbations. As a result, the learned representations become more stable and generalize better to unseen environments.

The central idea is to optimize both the task performance and a robustness objective simultaneously. Let  $J(\pi)$  denote the original reinforcement learning objective and let  $\mathcal{R}(\pi)$  represent a robustness regularization term. The optimization problem can be written as

$$J_{\text{reg}}(\pi) = J(\pi) - \lambda \mathcal{R}(\pi), \quad (5.24)$$

where  $\lambda$  is a non-negative hyperparameter that controls the trade-off between maximizing the expected return and improving robustness. The form of the regularization term depends on the specific algorithm and may encourage invariant feature representations, smooth value functions, robust communication, or consistent policies under adversarial perturbations.

Compared with adversarial training, adversarial regularization generally requires fewer computational resources because it avoids training a separate adversarial policy. These methods can also be integrated into existing reinforcement learning algorithms with relatively small modifications to the optimization objective. However, their robustness depends heavily on the design of the regularization term and may not provide the same level of worst-case guarantees as minimax-based approaches.

Several recent robust MARL algorithms adopt this strategy. In the following sections, we examine four representative methods: *Efficient Robust Networked Multi-Agent Reinforcement Learning with Adversarial Regularization (ERNIE)*, *Mutual Information Regularized Robust Multi-Agent Reinforcement Learning (MIR3)*, and multi-agent extensions of two single-agent regularizers, *State-Adversarial Policy Regularization (SA-MDP)* and *Certified Robustness via Interval Bound Propagation (RADIAL-RL)*. These methods improve robustness through different regularization mechanisms while preserving the efficiency of standard multi-agent reinforcement learning algorithms.

### 5.5.1. Efficient Robust Networked Multi-Agent Reinforcement Learning with Adversarial Regularization (ERNIE)

Unlike adversarial training methods, which improve robustness by exposing agents to worst-case disturbances, ERNIE improves robustness by directly regularizing the learning objective. The central idea is that a robust policy should respond smoothly to small changes in its observations. If small perturbations produce only small changes in the selected actions, the learned policy is less sensitive to observation noise and environmental uncertainty. Based on this observation, ERNIE encourages smooth policies through adversarial regularization rather than explicit adversarial training. [39]

ERNIE assumes that the environment is inherently smooth. More precisely, small changes in the state should not produce large variations in either the reward function or the transition dynamics. This assumption is expressed through the Lipschitz continuity of the reward function,

$$|r(s, a) - r(s', a)| \leq L_r \|s - s'\|, \quad (5.25)$$

and the transition function,

$$\|P(\cdot|s, a) - P(\cdot|s', a)\|_1 \leq L_p \|s - s'\|, \quad (5.26)$$

where  $L_r$  and  $L_p$  are the Lipschitz constants of the reward and transition functions, respectively. :

Based on these assumptions, the authors establish several theoretical results. First, they prove that a smooth environment leads to a smooth action-value function. Specifically,



$$|Q(s, a) - Q(s', a)| \leq L_Q \|s - s'\|, \quad (5.27)$$

where

$$L_Q = L_r + \frac{\gamma L_P}{1 - \gamma}. \quad (5.28)$$

This result shows that if the environment changes gradually, the optimal value function also changes gradually.

The authors further prove that for any desired approximation accuracy, there exists a near-optimal policy that is also Lipschitz continuous. Finally, they show that smooth policies are naturally robust against bounded observation perturbations. If the observed state is corrupted by

$$\tilde{s}_t = s_t + \delta_t, \quad \|\delta_t\| \leq \epsilon, \quad (5.29)$$

then the difference between the value functions of the original and perturbed policies satisfies

$$|V^\pi(s) - V^{\tilde{\pi}}(s)| \leq \frac{2L_\pi \epsilon}{(1 - \gamma)^2}, \quad (5.30)$$

and similarly,

$$|Q^\pi(s, a) - Q^{\tilde{\pi}}(s, a)| \leq \frac{2L_\pi \epsilon}{(1 - \gamma)^2}, \quad (5.31)$$

where  $L_\pi$  denotes the Lipschitz constant of the policy. These theoretical guarantees establish a direct connection between policy smoothness and robustness.

To encourage smooth policies during learning, ERNIE introduces an adversarial regularization term that measures the maximum change in the policy caused by bounded observation perturbations,

$$R^\pi(o_k, \theta_k) = \max_{\|\delta\| \leq \epsilon} D(\pi_{\theta_k}(o_k + \delta), \pi_{\theta_k}(o_k)), \quad (5.32)$$

where  $D(\cdot, \cdot)$  measures the difference between the perturbed and original policy outputs. The adversarial perturbation is chosen to maximize this difference, while the

learning algorithm minimizes it. In practice, solving the inner maximization problem in  $R^\pi(o_k, \theta_k)$  exactly is computationally expensive. Therefore, ERNIE approximates the adversarial perturbation using first-order attack methods such as the Fast Gradient Sign Method (FGSM) or Projected Gradient Descent (PGD), which were introduced in Section 5.2.4. These methods efficiently estimate the perturbation that maximizes the discrepancy between the policy outputs under the original and perturbed observations. Consequently, the policy becomes less sensitive to small changes in its observations. The regularizer is incorporated into the original MARL objective as

$$F(\theta) = L(\theta) + \lambda \sum_{n=1}^N \mathbb{E} [R^\pi(o_n, \theta_n)], \quad (5.33)$$

where  $L(\theta)$  is the original optimization objective,  $\lambda$  controls the strength of the regularization, and  $N$  is the number of agents. Larger values of  $\lambda$  improve robustness by enforcing smoother policies but may reduce the final task performance because the optimization places greater emphasis on robustness than exploitation.

Besides observation perturbations, ERNIE also considers malicious agents that intentionally execute harmful actions. To improve robustness against such behaviors, the authors introduce an additional value-function regularizer,

$$R_A = \max_{D(a, a') \leq K} \|Q(s, a) - Q(s, a')\|^2, \quad (5.34)$$

which limits the sensitivity of the value function to bounded changes in the joint action. This regularizer reduces the influence of malicious teammates by encouraging the critic to produce similar value estimates for nearby joint actions.

### 5.5.2. Robust Multi-Agent Reinforcement Learning by Mutual Information Regularization (MIR3)

Most robust MARL algorithms improve robustness by explicitly exposing agents to adversarial perturbations during training. Although effective, these approaches require modeling an adversary and repeatedly solving a minimax optimization problem, which increases both the computational cost and the complexity of training. MIR3 takes a fundamentally different approach by asking a simple question: *Can robustness be achieved without training against adversaries?* [38]. Rather than learning to defeat every possible attacker, MIR3 learns policies that are inherently less

dependent on other agents, making them naturally more robust to unexpected teammate failures and adversarial behaviors. The motivation behind MIR3 comes from a common failure mode in cooperative MARL. During training, agents often become highly dependent on one another. If one agent is compromised or behaves unexpectedly, the remaining agents may continue to follow its behavior, causing the error to propagate throughout the team. Such cascading failures become increasingly severe as the number of agents grows because the number of possible attack combinations increases exponentially. Instead of attempting to defend against every possible attack scenario, MIR3 reduces unnecessary dependencies among agents so that each agent can make more reliable decisions using primarily its own observations.

This idea is inspired by human decision making. Humans do not memorize every possible accident that may occur while driving. Instead, they develop general driving habits, such as maintaining a safe distance and paying attention to their surroundings. These habits improve robustness across many different situations without explicitly preparing for every possible failure. MIR3 applies the same principle by encouraging agents to retain only the information necessary for solving the task while discarding unnecessary correlations with their teammates.

To achieve this objective, MIR3 introduces an Information Bottleneck (IB)[44] between the agent’s history and its policy. Let  $h_t$  denote the observation history of an agent and  $a_t$  its action. Instead of maximizing only the expected return, MIR3 minimizes the mutual information between the history and the selected action,

$$I(h_t; a_t), \tag{5.35}$$

thereby encouraging the policy to use only the information that is essential for decision making. Reducing this mutual information removes redundant dependencies between agents and prevents policies from overfitting to the specific teammate behaviors observed during training.

The resulting optimization objective becomes

$$J(\pi) = \mathbb{E}[R] - \lambda I(h; a), \tag{5.36}$$

where  $\lambda$  controls the trade-off between task performance and robustness. A larger value of  $\lambda$  forces the policy to rely on less information, leading to stronger robustness at the possible expense of the nominal return.

The overall architecture is illustrated by the following information flow:

History  $\rightarrow$  Information Bottleneck  $\rightarrow$  Policy  $\rightarrow$  Action

The information bottleneck removes unnecessary information before the policy generates an action. Consequently, the learned policy focuses on task-relevant information while reducing spurious correlations between agents. From another perspective, MIR3 can also be interpreted as learning a robust action prior. Instead of allowing arbitrary actions, the policy remains close to a learned prior distribution over actions, providing a stable reference when observations are corrupted or teammates behave unexpectedly.

The theoretical foundation of MIR3 is based on the control-as-inference framework. Instead of directly maximizing the expected return, the reinforcement learning problem is reformulated as probabilistic inference over trajectories. The objective is to maximize the probability that a trajectory is optimal,

$$p(\tau \mid O_{1:T}), \quad (5.37)$$

where  $O_t$  is a binary optimality variable indicating whether the action at time step  $t$  is optimal. Under this formulation, the probability of optimality is defined as

$$P(O_t = 1 \mid s_t, a_t) = \exp(r_t), \quad (5.38)$$

which leads to

$$p(\tau \mid O_{1:T}) \propto p(\tau) \exp\left(\sum_t r_t\right). \quad (5.39)$$

Using this probabilistic interpretation, the authors prove that maximizing the mutual-information-regularized objective maximizes a lower bound on the robust control objective,

$$J(\pi) \geq \mathbb{E}[r - \lambda I(h; a)]. \quad (5.40)$$

This result provides a theoretical guarantee that reducing the mutual information improves robustness against a broad class of attack scenarios rather than only against specific adversarial perturbations.

The complete training procedure is summarized in Algorithm 17. During each training iteration, the agents first interact with the environment to collect trajectories, which are stored in a replay buffer. The CLUB estimator is then used to estimate the mutual information between the observation histories and the corresponding actions. This estimate is used to construct the mutual-information-regularized reward, which replaces the original reward when updating the backbone MARL algorithm. Finally,

the actor and critic networks are optimized using the modified reward, allowing the agents to learn policies that maximize task performance while reducing unnecessary dependence on teammate behaviors. Since MIR3 modifies only the learning objective, it can be integrated into existing MARL algorithms such as MADDPG, QMIX, and MAPPO without changing their network architectures. A detailed description of the CLUB estimator and its derivation is provided in Appendix B.

### 5.5.3. State-Adversarial Policy Regularization (SA-MDP)

Zhang *et al.* [40] address a narrower but common source of uncertainty than ERNIE’s general Lipschitz-smoothness assumption: the observation channel specifically, rather than the reward or transition function. They formalize this as a State-Adversarial MDP, in which the policy still acts in the true state  $s$ , but perceives a perturbed observation  $\hat{s}$  drawn from a bounded neighborhood of  $s$ ,

$$B(s) = \{\hat{s} : \|\hat{s} - s\|_\infty \leq \epsilon\}, \quad (5.41)$$

matching the bounded observation-perturbation model introduced in Section 5.2.4. The guiding principle is the same one ERNIE arrives at from a different assumption: a policy that is not sensitive to which point of  $B(s)$  it happens to observe is, by construction, robust to any attack confined to that neighborhood. Zhang *et al.* originally develop this regularizer for single-agent PPO, DDPG, and DQN; the remainder of this subsection presents their regularizer and then extends it to the multi-agent case directly, since nothing in its construction is specific to a single learner.

**Policy-consistency regularization.** For a stochastic policy such as PPO’s, the regularizer penalizes the largest divergence between the action distribution at the true observation and at any perturbed observation inside  $B(s)$ ,

$$R_{\text{PPO}} = \sum_s \max_{\hat{s} \in B(s)} D_{\text{KL}}(\pi(a | s) \parallel \pi(a | \hat{s})). \quad (5.42)$$

For a deterministic policy such as DDPG’s, there is no distribution to compare, so the regularizer instead penalizes the largest change in the selected action itself,

$$R_{\text{DDPG}} = \sum_s \max_{\hat{s} \in B(s)} \|\mu(s) - \mu(\hat{s})\|_2. \quad (5.43)$$

Either regularizer is added directly to the policy’s own training objective, so no adversary is trained and no additional network is introduced,

$$\mathcal{L}_{\text{PPO}}^{\text{robust}} = \mathcal{L}_{\text{PPO}} + \lambda R_{\text{Reg}}, \quad (5.44)$$

where  $\lambda$  trades task performance for observation robustness, in the same way ERNIE’s and MIR3’s regularization weights do.

**Extending the regularizer to MARL.** Nothing in  $R_{\text{Reg}}$  depends on there being only one policy being regularized. For agent  $i$  with observation  $o^i$ , define its own bounded perturbation neighborhood  $\Delta_i(o^i) = \{\hat{o}^i : \|\hat{o}^i - o^i\|_\infty \leq \epsilon\}$ , exactly as  $B(s)$  is defined for the single-agent case, and apply the same consistency penalty to each agent’s own policy independently,

$$L_{\text{MARL}}^{\text{robust}} = L_{\text{MARL}} + \lambda \sum_{i=1}^N \max_{\hat{o}^i \in \Delta_i(o^i)} R_{\text{Reg}}(o^i, \hat{o}^i), \quad (5.45)$$

where  $L_{\text{MARL}}$  is the unmodified loss of the underlying MARL algorithm. Because each agent’s regularization term depends only on that agent’s own observation and policy, this extension requires no additional inter-agent communication or centralized adversary, and it is compatible with any backbone MARL algorithm in the same way MIR3 is: the base loss is untouched, and only a per-agent penalty term is added on top of it. The complete training procedure is summarized in Algorithm 18. At every environment step, each agent generates a bounded perturbation of its own observation, computes how far its policy’s output moves under that perturbation, and adds the resulting consistency loss, averaged across agents, to the ordinary MARL loss before updating.

Because this regularizer only ever compares a policy against itself under a bounded perturbation, its guarantee is exactly as strong as the perturbation bound  $\epsilon$  built into  $\Delta_i(o^i)$ : an observation attack that stays within  $\epsilon$  is penalized during training, but nothing about the construction bounds what happens once the true disturbance exceeds it, the same failure mode ERNIE’s smoothness guarantee has once its own Lipschitz assumption is violated. Unlike ERNIE and MIR3, however, this regularizer targets only the observation channel; it says nothing about robustness to a compromised transition model, a malicious teammate’s actions, or the communication channel, all of which ERNIE’s and MIR3’s regularizers touch in some form.

#### 5.5.4. Certified Robustness via Interval Bound Propagation (RADIAL-RL)

ERNIE and SA-MDP both regularize a policy toward being insensitive to bounded perturbations, but neither certifies that insensitivity: their regularizers penalize a sampled or gradient-estimated worst-case perturbation, not the entire perturbation region at once. Oikarinen *et al.* [41] target discrete-action, value-based agents specifically, DQN in particular, and ask a sharper question than either: not just whether the perturbed Q-values move, but whether they can move enough to make a suboptimal action rank above the nominal best one. A large but uniform drop in every action’s value under perturbation is harmless, since the agent still selects the same action; the actual failure mode is two actions’ values becoming competitive, or swapping order, under a perturbation the network never explicitly considered.

**Certified Q-value bounds.** For a perturbation budget  $\|\delta\|_p \leq \epsilon$ , RADIAL-RL does not sample perturbations to estimate their effect. Instead, it computes a certified interval  $[\underline{Q}_i(s), \overline{Q}_i(s)]$  that provably contains  $Q_i(s + \delta)$  for every  $\delta$  inside the budget, using Interval Bound Propagation (IBP) through the Q-network’s layers. Writing the perturbed state’s coordinate-wise bounds as  $\bar{s} = s + \epsilon$  and  $\underline{s} = s - \epsilon$ , and splitting each layer’s weight matrix into its non-negative and non-positive parts,

$$W^{l,+} = \max(W^l, 0), \quad W^{l,-} = \min(W^l, 0), \quad (5.46)$$

the bounds propagate forward one layer at a time. At the first layer,

$$\underline{z}^1 = \text{ReLU}(W^{1,+}\underline{s} + W^{1,-}\bar{s} + b^1), \quad \bar{z}^1 = \text{ReLU}(W^{1,+}\bar{s} + W^{1,-}\underline{s} + b^1), \quad (5.47)$$

and at every subsequent layer  $l + 1$ ,

$$\underline{z}^{l+1} = \text{ReLU}(W^{l,+}\underline{z}^l + W^{l,-}\bar{z}^l + b^l), \quad \bar{z}^{l+1} = \text{ReLU}(W^{l,+}\bar{z}^l + W^{l,-}\underline{z}^l + b^l). \quad (5.48)$$

The lower bound always pairs the non-negative weights with the input’s lower bound and the non-positive weights with its upper bound, and symmetrically for the upper bound, so that each propagated interval remains a valid enclosure regardless of each weight’s sign. At the final layer  $L$ , this yields the certified Q-value interval directly,

$$\underline{Q} = W^{L,+}\underline{z}^L + W^{L,-}\bar{z}^L + b^L, \quad \overline{Q} = W^{L,+}\bar{z}^L + W^{L,-}\underline{z}^L + b^L. \quad (5.49)$$

Every  $Q_i(s + \delta)$  for  $\|\delta\|_p \leq \epsilon$  is guaranteed to lie in  $[\underline{Q}_i(s), \overline{Q}_i(s)]$ ; nothing about this bound depends on which particular perturbation is realized.

**Robust action selection and the margin regularizer.** Let  $a^* = \arg \max_a Q(s, a)$  be the nominal best action under the unperturbed state. The action selection is certifiably robust to any perturbation inside the budget exactly when the nominal best action’s worst case still beats every other action’s best case,

$$\underline{Q}(s, a^*) > \bar{Q}(s, a), \quad \forall a \neq a^*. \quad (5.50)$$

This condition is what a sampled adversarial perturbation cannot guarantee, since it only checks the specific perturbations tried during training, not the full region at once. RADIAL-RL turns the condition directly into a training penalty, the robust margin regularizer,

$$L_{\text{robust}} = \sum_{a \neq a^*} \max\left(0, \bar{Q}(s, a) - \underline{Q}(s, a^*)\right), \quad (5.51)$$

which is zero exactly when the certified robustness condition already holds, and grows linearly in how far any competing action’s upper bound intrudes past the best action’s lower bound otherwise. The total training loss combines this with the ordinary reinforcement learning loss,

$$L = L_{\text{RL}} + \lambda L_{\text{robust}}, \quad (5.52)$$

so the network is trained simultaneously to solve the task, maximize expected return, and keep the decision boundary between the best action and its runners-up certifiably wide rather than merely empirically wide.

**Extending the regularizer to MARL.** As stated, this construction is single-agent and specific to a value-based backbone. It extends directly to QMIX, since QMIX already maintains one individual action-value function  $Q_i(o_i, a_i)$  per agent before mixing them into  $Q_{\text{tot}}$ . Applying interval bound propagation to each agent’s own network gives per-agent certified bounds  $\underline{Q}_i, \bar{Q}_i$  from that agent’s own observation perturbation budget, and the margin regularizer is applied per agent against that agent’s own nominal best action  $a_i^* = \arg \max_a Q_i(o_i, a)$ ,

$$L_{\text{margin}}^i = \sum_{a \neq a_i^*} \max\left(0, \bar{Q}_i(o_i, a) - \underline{Q}_i(o_i, a_i^*)\right). \quad (5.53)$$

The resulting robust QMIX objective adds the summed per-agent margin penalty on top of the unmodified QMIX temporal-difference loss,

$$L_{\text{QMIX-R}} = L_{\text{QMIX}} + \lambda_M \sum_{i=1}^N L_{\text{margin}}^i, \quad L_{\text{QMIX}} = \frac{1}{B} \sum_{t=1}^B (y_t - Q_{\text{tot},t})^2, \quad (5.54)$$



where  $B$  is the batch size and  $y_t$  is the ordinary QMIX target. Because each agent’s certified bounds and margin penalty depend only on that agent’s own network and its own observation perturbation, this extension changes nothing about QMIX’s mixing network, its monotonicity constraint, or its centralized-training-decentralized-execution structure; it only adds a per-agent penalty computed alongside the existing TD loss.

Because the margin regularizer is derived from a certified bound rather than a sampled or gradient-estimated one, RADIAL-RL’s robustness guarantee holds for every perturbation inside the trained budget  $\epsilon$ , not only the ones a first-order attack such as FGSM or PGD happens to find, which is a strictly stronger guarantee than ERNIE’s or SA-MDP’s within that budget. The guarantee is, however, exactly as wide as  $\epsilon$  and no wider: a disturbance that exceeds the trained perturbation budget, or one that targets the transition function, a teammate’s policy, or the communication channel rather than the observation, is outside what interval bound propagation certifies anything about. Computing certified bounds through every layer at every training step is also more expensive than a single sampled or one-step gradient perturbation, so this stronger guarantee is bought at a higher computational cost than ERNIE’s or SA-MDP’s regularizers.

## 5.6. Distributionally Robust Multi-Agent Reinforcement Learning

Distributionally Robust Multi-Agent Reinforcement Learning (DR-MARL) aims to learn policies that remain effective when the deployment environment differs from the one used during training. Instead of assuming that the transition dynamics are known exactly, DR-MARL considers a family of plausible environment models and optimizes the policy for the worst-case model within this family. This approach improves robustness against model mismatch, observation uncertainty, simulator-to-real-world transfer, and other distributional shifts.

The uncertainty in the environment is represented by an ambiguity set, also called a confidence set, which contains all probability distributions that are considered plausible. The learning objective is formulated as a minimax optimization problem,

$$\pi^* = \arg \max_{\pi} \min_{P \in \mathcal{P}} V^{\pi}(P), \quad (5.55)$$

where  $V^{\pi}(P)$  denotes the expected return of policy  $\pi$  under transition model  $P$ , and  $\mathcal{P}$  is the ambiguity set containing all admissible transition distributions. Unlike conventional reinforcement learning, which optimizes the expected return under a single

estimated model, distributionally robust methods explicitly optimize the worst-case performance over all models in  $\mathcal{P}$ .

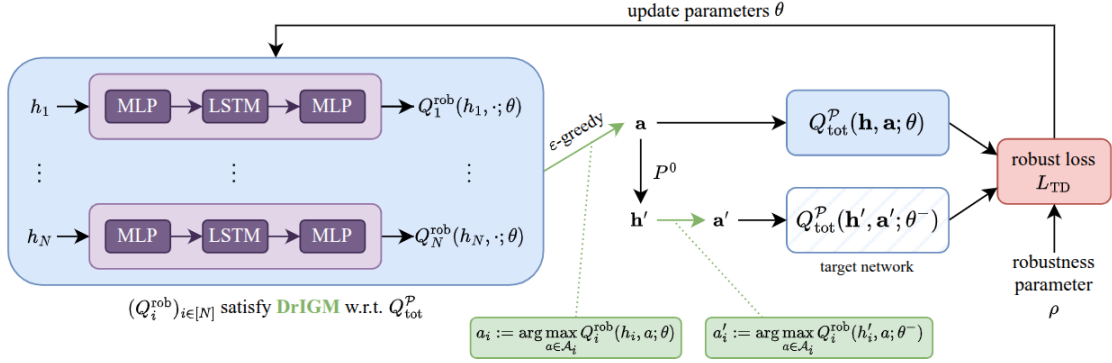
The ambiguity set is commonly constructed using statistical distance measures such as the Wasserstein distance, the Kullback-Leibler divergence, or the Total Variation Distance (TVD). Among these measures, TVD is frequently used because it directly quantifies the largest possible difference between two probability distributions. A detailed definition of the Total Variation Distance is provided in Appendix C.

In cooperative MARL, distributional robustness introduces an additional challenge because individual agents must coordinate under the same uncertain environment. If each agent independently optimizes against its own worst-case model, the resulting joint policy may no longer satisfy the Independent Global Maximization (IGM) property introduced in Section 4.4.1. Consequently, recent methods perform robust optimization directly on the joint value function before decomposing it into individual value functions, thereby preserving decentralized execution while improving robustness.

This section presents Distributionally Robust Independent Global Maximization (DrIGM), a framework that extends value factorization methods such as VDN, QMIX, and QTRAN to uncertain environments. Instead of optimizing each agent independently against different worst-case models, DrIGM performs robust optimization on the joint value function and then derives individual action-value functions that preserve the Independent Global Maximization (IGM) property introduced in Section 4.4.1. This formulation enables cooperative MARL algorithms to maintain decentralized execution while improving robustness to uncertainty and distributional shifts.

### 5.6.1. Distributionally Robust Independent Global Maximization (DrIGM)

Value factorization methods such as Value Decomposition Networks (VDN), QMIX, and QTRAN have achieved strong performance in cooperative multi-agent reinforcement learning by decomposing a joint action-value function into individual value functions while preserving the Independent Global Maximization (IGM) property discussed in Section 4.4.1. However, these methods assume that the environment encountered during deployment follows the same transition dynamics observed during training. In real-world applications, this assumption rarely holds because sensor noise, modeling errors, hardware degradation, and simulator-to-real-world discrepancies may alter the underlying environment dynamics. As a result, policies learned under the nominal model often experience a significant performance degradation after deployment[42].



**Figure 5.1.** – Overview of the DrIGM training framework. Each agent learns a robust local action-value function, which is combined through a value factorization network to construct the robust joint action-value function. Robust Bellman targets are computed using the target network, and the network parameters are updated by minimizing the robust temporal-difference loss under the selected uncertainty model. Adapted from [42].

Distributionally Robust Independent Global Maximization (DrIGM) extends value factorization methods to uncertain environments by optimizing the joint value function under the worst-case transition model. Instead of learning policies for a single transition model, DrIGM assumes that the true environment belongs to an ambiguity set containing multiple plausible transition distributions. The objective is to maximize the worst-case expected return over this uncertainty set,

$$\max_{\pi} \min_{P \in \mathcal{P}} V^{\pi}(P), \quad (5.56)$$

where  $\mathcal{P}$  denotes the ambiguity set introduced in the previous section.

The framework is formulated using a Decentralized Partially Observable Markov Decision Process (Dec-POMDP). Instead of considering a single transition model, DrIGM defines an uncertainty set for every joint history-action pair,

$$\mathcal{P} = \prod_{(h,a) \in \mathcal{H} \times \mathcal{A}} \mathcal{P}_{h,a}, \quad \mathcal{P}_{h,a} \subseteq \Delta(\mathcal{H}), \quad (5.57)$$

where  $\Delta(\mathcal{H})$  denotes the probability simplex over the joint history space. Each element of the ambiguity set represents a possible transition model consistent with the available uncertainty information.

Based on this uncertainty model, the standard Bellman operator is replaced by the robust Bellman operator,

$$(\mathbf{T}Q)(\mathbf{h}, \mathbf{a}) = r(s, \mathbf{a}) + \gamma \inf_{P_{\mathbf{h}, \mathbf{a}} \in \mathcal{P}_{\mathbf{h}, \mathbf{a}}} \mathbb{E}_{\mathbf{h}' \sim P_{\mathbf{h}, \mathbf{a}}} \left[ \max_{\mathbf{a}'} Q(\mathbf{h}', \mathbf{a}') \right], \quad (5.58)$$

which evaluates the worst-case expected return over all transition models contained in the ambiguity set. Consequently, the robust joint action-value function becomes

$$Q_{\text{tot}}^{\mathcal{P}}(\mathbf{h}, \mathbf{a}) = \inf_{P \in \mathcal{P}} Q_{\text{tot}}^P(\mathbf{h}, \mathbf{a}). \quad (5.59)$$

Unlike conventional value factorization, the joint value function is therefore optimized against the worst possible transition model rather than the nominal environment.

A straightforward solution would be to allow each agent to compute its own worst-case value function independently. However, this approach generally produces different worst-case transition models for different agents, causing the decentralized policies to violate the IGM property. Consequently, the individual greedy actions no longer correspond to the optimal joint action.

To address this problem, the authors introduce the Distributionally Robust Independent Global Maximization (DrIGM) principle. Instead of constructing independent robust value functions, all agents share the same worst-case transition model derived from the robust joint value function. The robust IGM condition is defined as

$$\left( \arg \max_{a_1} Q_1^{\text{rob}}(h_1, a_1), \dots, \arg \max_{a_N} Q_N^{\text{rob}}(h_N, a_N) \right) \subseteq \arg \max_{\mathbf{a}} Q_{\text{tot}}^{\mathcal{P}}(\mathbf{h}, \mathbf{a}), \quad (5.60)$$

where each local robust action-value function is computed under the same worst-case transition model,

$$Q_i^{\text{rob}}(h_i, a_i) = Q_i^{P^{\text{worst}}(\mathbf{h}, \mathbf{a})}(h_i, a_i), \quad (5.61)$$

with

$$P^{\text{worst}}(\mathbf{h}, \mathbf{a}) \in \arg \inf_{P \in \mathcal{P}} Q_{\text{tot}}^P(\mathbf{h}, \mathbf{a}). \quad (5.62)$$

By anchoring every agent to the same worst-case environment, DrIGM preserves decentralized execution while maintaining the robust joint policy. Furthermore, the

authors prove that any value factorization method satisfying the original IGM property also satisfies the DrIGM property after replacing the nominal joint value function with its robust counterpart. Consequently, existing algorithms such as VDN, QMIX, and QTRAN can be extended to distributionally robust learning without modifying their network architectures, mixing networks, or reward functions.

To compute the robust Bellman target, the paper considers two uncertainty models. The first is the  $\rho$ -contamination model,

$$P_{\mathbf{h},\mathbf{a}} = (1 - \rho)P_{\mathbf{h},\mathbf{a}}^0 + \rho v_{\mathbf{h},\mathbf{a}}, \quad (5.63)$$

where  $v_{\mathbf{h},\mathbf{a}}$  is an arbitrary probability distribution. Under this model, the robust target becomes

$$y = r + \gamma(1 - \rho)Q_{\text{tot}}(s', a'), \quad (5.64)$$

where larger values of  $\rho$  produce more conservative policies. In general, the worst case over the contamination term  $v_{\mathbf{h},\mathbf{a}}$  contributes an explicit  $\rho \cdot \min_{s'} V_{\text{tot}}^{\mathcal{P}}(s')$  addition to this target; it does not appear above because the derivation assumes a fail state, an absorbing state with zero reward that the adversary can always route into, together with a normalization under which the robust value function's minimum over all states is zero, and the two together force that term to vanish [42]. The second uncertainty model constrains the transition distribution using the Total Variation Distance,

$$\mathcal{P}_{\mathbf{h},\mathbf{a}} = \left\{ P \in \Delta(\mathcal{H}) \mid d_{\text{TV}}(P, P_{\mathbf{h},\mathbf{a}}^0) \leq \rho \right\}, \quad (5.65)$$

where  $d_{\text{TV}}(\cdot, \cdot)$  denotes the Total Variation Distance defined in Appendix C. Compared with the  $\rho$ -contamination model, TVD uncertainty provides a stronger theoretical guarantee but generally requires higher computational cost.

The robust temporal-difference objective is

$$L_{\text{TD}} = \left( Q_{\text{tot}}^{\mathcal{P}}(\mathbf{h}, \mathbf{a}; \theta) - (\mathcal{T} Q_{\text{tot}}^{\mathcal{P}}(\cdot, \cdot; \theta^-))(\mathbf{h}, \mathbf{a}) \right)^2. \quad (5.66)$$

The complete training procedure is summarized in Algorithm 20 and figure 5.1. Similar to conventional value factorization methods, DrIGM maintains individual action-value networks together with a value factorization network. During training, agents interact with the environment using an  $\epsilon$ -greedy exploration policy and store the collected transitions in a replay buffer. Mini-batches are sampled to compute

robust Bellman targets using the uncertainty model described earlier, and the network parameters are updated by minimizing the robust temporal-difference loss. Finally, the target network is periodically synchronized with the online network. Since DrIGM modifies only the Bellman target while preserving the original value factorization architecture, it can be applied directly to algorithms such as VDN, QMIX, and QTRAN without changing their network structures or decentralized execution mechanism [42].

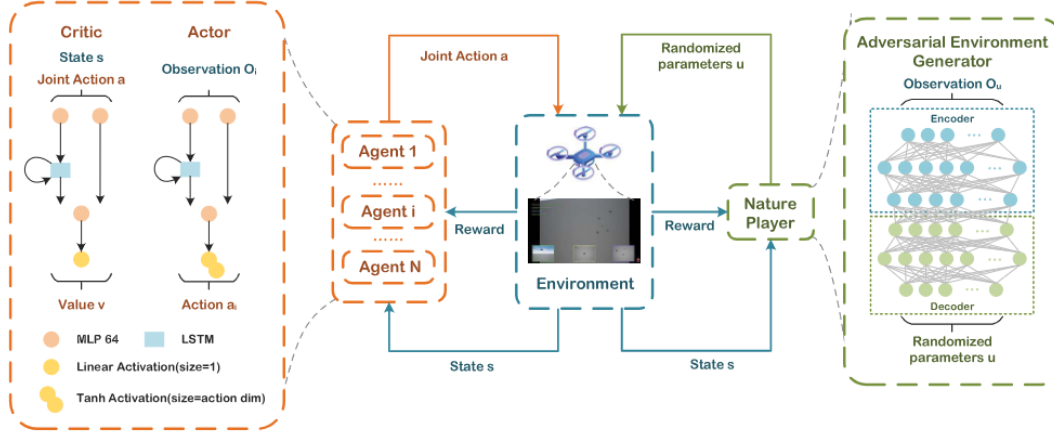
## 5.7. Sim-to-Real Robustness

Many multi-agent reinforcement learning algorithms are trained entirely in simulation because collecting data in the real world is expensive, time-consuming, and may expose physical systems to unnecessary risks. Although modern simulators can accurately model many aspects of the environment, they cannot perfectly reproduce real-world conditions. Differences in sensor characteristics, actuator dynamics, environmental disturbances, and physical parameters create a discrepancy between the simulated and real environments, commonly referred to as the *simulation-to-reality (Sim-to-Real) gap*. This gap often causes policies that perform well in simulation to experience a significant loss of performance after deployment[45].

A common approach for reducing the Sim-to-Real gap is Domain Randomization (DR). Instead of training the policy in a single simulated environment, DR randomly varies physical parameters such as mass, friction, wind speed, sensor noise, and other environmental properties. As a result, the policy is exposed to many different environments during training and becomes less dependent on a particular simulator configuration. At deployment, the real environment is expected to resemble one of the randomized environments encountered during training. [45]

Despite its success, conventional domain randomization samples environment parameters independently from predefined probability distributions. Consequently, many generated environments are either too easy or provide little useful information for improving robustness. Furthermore, random sampling does not explicitly consider the weaknesses of the current policy, resulting in inefficient training and limited robustness under severe domain shifts. [45]

To address these limitations, recent work introduces Adversarial Domain Randomization (ADR), where the environment parameters are generated by an adversarial policy rather than random sampling. Instead of selecting parameters uniformly, the adversary continuously adapts the environment according to the current learning progress and generates challenging but realistic scenarios that maximize the difficulty of the task. The reinforcement learning agent is therefore trained against



**Figure 5.2.** – Overview of the Adversarial Domain Randomization (ADR) framework. The Nature Player generates randomized environment parameters using an adversarial environment generator, while the MARL agents interact with the resulting environment. The actor-critic networks and the Nature Player are trained jointly in a minimax optimization framework, enabling the agents to learn policies that transfer more effectively from simulation to real-world environments. Adapted from [45].

increasingly difficult environments, improving its ability to transfer from simulation to real-world deployment. One representative method based on this idea is the robust MARL framework proposed by Chen *et al.* [45], which combines adversarial domain randomization with prioritized experience replay to improve the robustness of cooperative dual-UAV transportation tasks.

### 5.7.1. Adversarial Domain Randomization

Conventional Domain Randomization (DR) improves the transferability of reinforcement learning policies by randomly varying the parameters of the simulated environment during training. Although this strategy exposes the agent to many different environments, the generated scenarios are independent of the current learning progress. As a result, many sampled environments are either too simple or contribute little to improving the robustness of the learned policy. Consequently, a large number of training episodes may be required before the policy encounters sufficiently challenging scenarios. [45]

Adversarial Domain Randomization (ADR) addresses this limitation by replacing random parameter sampling with an adversarial environment generator, referred to as the *Nature Player*. Instead of drawing the domain randomization parameters from a predefined distribution, the Nature Player continuously observes the current environment and generates parameters that maximize the difficulty of the task while remaining physically plausible. The MARL policy and the Nature Player therefore play a minimax game in which the policy seeks to maximize its expected return, whereas the Nature Player attempts to construct increasingly challenging environments. The resulting optimization problem is

$$\max_{\pi} \min_{\mu} J(\pi, \mu), \quad (5.67)$$

where  $\pi$  denotes the MARL policy and  $\mu$  represents the Nature Player. Unlike adversarial training methods that directly perturb observations or actions, ADR modifies the environment itself by changing its physical parameters before the agents interact with it. [45]

The interaction between the agents and the Nature Player is formulated as an Adversarial Stationary Markov Game,

$$\mathcal{G} = \langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, \{o_i\}_{i \in \mathcal{N}}, \{\mathcal{R}_i\}_{i \in \mathcal{N}}, o_{\mu}, T, \mathcal{U}, \gamma \rangle, \quad (5.68)$$

where  $\mathcal{U}$  denotes the space of domain randomization parameters. The transition function

$$T : \mathcal{S} \times \mathcal{A}_1 \times \cdots \times \mathcal{A}_N \times \mathcal{U} \rightarrow \mathcal{S} \quad (5.69)$$

and the reward function

$$\mathcal{R}_i : \mathcal{S} \times \mathcal{A}_1 \times \cdots \times \mathcal{A}_N \times \mathcal{U} \rightarrow \mathbb{R} \quad (5.70)$$

both depend on the generated environment parameters. Under finite state, action, and domain-randomization spaces, the authors prove that the adversarial game admits at least one perfect Nash equilibrium. [45]

The architecture of the Nature Player consists of an encoder-decoder network. The encoder first maps the observed environment state into a latent representation  $\mathbf{z}$ , and the decoder then generates the domain randomization parameters  $\mathbf{u}$ . Because the generated parameters depend on the current environment, the adversary continuously adapts the training scenarios as the policy improves. This adaptive mechanism



produces more informative training environments than conventional random sampling. [45]

To prevent the Nature Player from generating unrealistic environments, the paper introduces the following objective function,

$$L_\mu = \alpha_0 r + \beta_0 D_{\text{KL}}(P_{\text{generated}}(u) \| P_{\text{normal}}(u)) - \omega_0 \text{MSE}, \quad (5.71)$$

where  $r$  is the average reward of the MARL agents, the Kullback-Leibler divergence constrains the generated parameter distribution to remain close to a predefined Gaussian prior, and the mean squared error regularizes the generated parameters to remain close to the observed environment. Together, these terms encourage the Nature Player to generate environments that are both challenging and physically realistic. [45] The complete training procedure is illustrated in Figure 5.2 and summarized in Algorithm 21. During each training iteration, the Nature Player first generates a new set of environment parameters, which are used to update the simulator. The MARL agents then interact with the randomized environment, and both the Nature Player and the actor-critic networks are updated using the collected experiences. By repeatedly training against increasingly difficult environments, the learned policy becomes more robust to the discrepancies encountered during real-world deployment. [45]

### 5.7.2. Prioritized Experience Replay for Domain Randomization Environments (PERDRE)

Training under adversarial domain randomization generates experiences from a wide range of environment configurations. However, treating all experiences equally during replay is inefficient because many transitions are either too easy or repeatedly sampled, contributing little to policy improvement. Conventional experience replay samples transitions uniformly from the replay buffer, ignoring both their learning value and the frequency with which similar environments have already been revisited. Consequently, valuable experiences collected from challenging environments may be underutilized. [45]

To address this issue, Chen *et al.* propose Prioritized Experience Replay for Domain Randomization Environments (PERDRE). Instead of sampling transitions uniformly, PERDRE assigns a priority to each experience according to both its temporal-difference error and the replay frequency of the corresponding domain-randomization subset. This strategy encourages the learning algorithm to revisit informative experiences while maintaining diversity among different randomized environments.

Each experience stored in the replay buffer is represented as

$$e = (o_{i,t}, a_{i,t}, o_{i,t+1}, r_{i,t}, c_{i,t}^p, c_{i,t+1}^p, c_{i,t}^q, c_{i,t+1}^q), \quad (5.72)$$

where  $o_{i,t}$  and  $a_{i,t}$  denote the observation and action of agent  $i$ ,  $r_{i,t}$  is the received reward, and  $c^p$  and  $c^q$  represent the internal memory states of the actor and critic networks, respectively. The experiences are further divided into subsets according to the generated domain-randomization parameters so that environments with similar characteristics are grouped together.

For every sampled experience, PERDRE computes an importance weight that combines the critic loss and the replay frequency of the corresponding subset,

$$I_w = \text{Rank}(L) + \text{Rank}'(C), \quad (5.73)$$

where  $L$  is the critic loss associated with the experience,  $C$  denotes the number of times that the corresponding subset has already been replayed,  $\text{Rank}(\cdot)$  ranks the critic losses in ascending order, and  $\text{Rank}'(\cdot)$  assigns higher scores to subsets that have been sampled less frequently. Consequently, experiences with large temporal-difference errors and underrepresented environments receive higher priorities. The sampling probability is then computed as

$$P(e) = \left( \frac{\text{Rank}(I_w)}{\sum_e \text{Rank}(I_w)} \right)^{\alpha_s} + \beta_s, \quad (5.74)$$

where  $\alpha_s$  controls the influence of the importance ranking and  $\beta_s$  guarantees that every experience retains a nonzero probability of being sampled. This sampling strategy balances exploitation of informative experiences with sufficient exploration of different randomized environments.

The complete sampling procedure is summarized in Algorithm 22. During each update, a small set of candidate experiences is first selected from the replay buffer. Their priorities are then computed according to the critic loss and replay frequency, after which the experiences are probabilistically inserted into the update batch. By emphasizing difficult and underrepresented environments, PERDRE improves sample efficiency and accelerates policy convergence under adversarial domain randomization. [45]

---

**Algorithm 15** Minimax Multi-Agent Deep Deterministic Policy Gradient (M3DDPG)

---

**Require:** Number of agents  $N$ , actor parameters  $\theta$ , critic parameters  $\phi$ , target networks  $\theta', \phi'$ , replay buffer  $\mathcal{D}$ , perturbation coefficient  $\alpha$

```
1: Initialize actor  $\mu_i$ , critic  $Q_i$ , target actor  $\mu'_i$ , and target critic  $Q'_i$  for each agent
2: for each episode do
3:   Initialize exploration noise
4:   Receive initial observations  $o = (o_1, \dots, o_N)$ 
5:   for each environment step do
6:     for each agent  $i$  do
7:       Select action  $a_i = \mu_i(o_i) + \mathcal{N}$ 
8:     end for
9:     Execute joint action  $a = (a_1, \dots, a_N)$ 
10:    Observe rewards  $r = (r_1, \dots, r_N)$  and next observations  $o'$ 
11:    Store transition  $(o, a, r, o')$  in replay buffer  $\mathcal{D}$ 
12:    for each agent  $i$  do
13:      Sample a minibatch from  $\mathcal{D}$ 
14:      Generate target actions using target actors
15:      for each opponent  $j \neq i$  do
16:        Compute adversarial perturbation
17:         $\hat{\epsilon}_j = -\alpha \nabla_{a_j} Q'_i(o', a')$ 
18:        Set  $a_j^* = a'_j + \hat{\epsilon}_j$ 
19:      end for
20:      Compute target value
21:       $y_i = r_i + \gamma Q'_i(o', a_1^*, \dots, a_i', \dots, a_N^*)$ 
22:      Update critic by minimizing
23:       $L(\phi_i) = (Q_i(o, a) - y_i)^2$ 
24:      for each opponent  $j \neq i$  do
25:         $\hat{\epsilon}_j = -\alpha \nabla_{a_j} Q_i(o, a)$ 
26:         $a_j^* = a_j + \hat{\epsilon}_j$ 
27:      end for
28:      Update actor using the minimax policy gradient
29:      Soft update target actor and critic networks
30:    end for
31:  end for
32: end for
```

---

**Algorithm 16** ERNIE Training

---

**Require:** Actor parameters  $\{\theta_i\}_{i=1}^N$ , critic parameters  $\{\phi_i\}_{i=1}^N$ , target networks  $\{\theta'_i, \phi'_i\}_{i=1}^N$ , adversarial regularization weight  $\lambda$ , replay buffer  $\mathcal{D}$

- 1: Initialize actor networks  $\pi_{\theta_i}$  and critic networks  $Q_{\phi_i}$  for all agents
- 2: Initialize target networks  $\theta'_i \leftarrow \theta_i, \phi'_i \leftarrow \phi_i$
- 3: Initialize replay buffer  $\mathcal{D}$
- 4: **for** each training episode **do**
- 5:   Reset the environment and receive initial observations  $\mathbf{o}$
- 6:   **for** each environment step **do**
- 7:     Each agent selects action  $\mathbf{a}_i = \pi_{\theta_i}(\mathbf{o}_i) + \mathcal{N}$
- 8:     Execute the joint action  $\mathbf{a}$  in the environment
- 9:     Receive reward vector  $\mathbf{r}$  and next observations  $\mathbf{o}'$
- 10:    Store  $(\mathbf{o}, \mathbf{a}, \mathbf{r}, \mathbf{o}')$  in  $\mathcal{D}$
- 11:    Update  $\mathbf{o} \leftarrow \mathbf{o}'$
- 12:   **end for**
- 13:   **for** each gradient update **do**
- 14:     Sample a minibatch  $(\mathbf{o}, \mathbf{a}, \mathbf{r}, \mathbf{o}')$  from  $\mathcal{D}$
- 15:     Compute target actions  $\mathbf{a}'_i = \pi_{\theta'_i}(\mathbf{o}'_i)$
- 16:     Compute TD target
- 17:      $y_i = r_i + \gamma Q_{\phi'_i}(\mathbf{o}', \mathbf{a}')$
- 18:     Update critic by minimizing
- 19:      $L_Q = \mathbb{E} [(Q_{\phi_i}(\mathbf{o}, \mathbf{a}) - y_i)^2]$
- 20:     Generate adversarial perturbation
- 21:      $\delta_i = \epsilon \cdot \text{sign}(\nabla_{\mathbf{o}_i} L_Q)$
- 22:     Construct perturbed observation
- 23:      $\tilde{\mathbf{o}}_i = \mathbf{o}_i + \delta_i$
- 24:     Compute adversarial actor loss
- 25:      $L_{\text{adv}} = -Q_{\phi_i}(\tilde{\mathbf{o}}, \pi_{\theta}(\tilde{\mathbf{o}}))$
- 26:     Compute policy gradient loss
- 27:      $L_{\pi} = -Q_{\phi_i}(\mathbf{o}, \pi_{\theta}(\mathbf{o}))$
- 28:     Compute ERNIE objective
- 29:      $L_{\text{actor}} = L_{\pi} + \lambda L_{\text{adv}}$
- 30:     Update actor parameters using  $\nabla_{\theta_i} L_{\text{actor}}$
- 31:     Soft update target networks
- 32:      $\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$
- 33:      $\phi'_i \leftarrow \tau \phi_i + (1 - \tau) \phi'_i$
- 34:    **end for**
- 35: **end for**
- 36: **return** Trained robust policies  $\{\pi_{\theta_i}\}_{i=1}^N$

---

---

**Algorithm 17** MIR3 with a MADDPG Backbone

---

**Require:** Policy networks  $\{\pi_1, \dots, \pi_N\}$ , critic networks  $\{Q_1, \dots, Q_N\}$ , replay buffer  $\mathcal{D}$ , CLUB estimator  $\text{CLUB}(\mathbf{h}, \mathbf{a})$ , regularization coefficient  $\lambda$

**Ensure:** Robust policy networks  $\{\pi_1, \dots, \pi_N\}$

```

1: for episode = 1 to  $K$  do
2:   Execute the current joint policy
3:   Collect trajectories and store them in replay buffer  $\mathcal{D}$ 
4:   Sample a minibatch from  $\mathcal{D}$ 
5:   Update the CLUB estimator
6:   Estimate the mutual information
7:    $I(\mathbf{h}, \mathbf{a}) = \sum_{i=1}^N \text{CLUB}(\mathbf{h}_i, \mathbf{a}_i)$ 
8:   Compute the mutual-information-regularized reward
9:    $r_t^{MI} = r_t - \lambda I(\mathbf{h}, \mathbf{a})$ 
10:  for each agent  $i$  do
11:    Update critic  $Q_i$  using the modified reward  $r_t^{MI}$ 
12:    Update actor policy  $\pi_i$  using the backbone MARL algorithm
13:  end for
14: end for
15: return  $\{\pi_1, \pi_2, \dots, \pi_N\}$ 

```

---

---

**Algorithm 18** Observation-Robust MARL with Policy Consistency Regularization
 

---

**Require:** Number of agents  $N$ , regularization coefficient  $\lambda$ , perturbation bound  $\epsilon$ , learning rate  $\alpha$

**Ensure:** Policy parameters  $\theta_1, \dots, \theta_N$

- 1: Initialize policy parameters  $\theta_1, \dots, \theta_N$
  - 2: **for**  $t = 1, \dots, T$  **do**
  - 3:   Collect observations  $o_t^i$  for all agents  $i = 1, \dots, N$
  - 4:   Select actions  $a_t^i \sim \pi_i(\cdot \mid o_t^i; \theta_i)$
  - 5:   Execute joint action  $\mathbf{a}_t = (a_t^1, \dots, a_t^N)$
  - 6:   Observe rewards  $\mathbf{r}_t$  and next observations  $\mathbf{o}_{t+1}$
  - 7:   **for**  $i = 1, \dots, N$  **do**
  - 8:     Generate perturbation  $\delta_t^i$  such that  $\|\delta_t^i\|_\infty \leq \epsilon$
  - 9:     Construct perturbed observation  $\tilde{o}_t^i = o_t^i + \delta_t^i$
  - 10:    Compute original policy distribution  $p_t^i = \pi_i(\cdot \mid o_t^i; \theta_i)$
  - 11:    Compute perturbed policy distribution  $\tilde{p}_t^i = \pi_i(\cdot \mid \tilde{o}_t^i; \theta_i)$
  - 12:    Compute consistency loss  $L_{\text{reg}}^i = D_{\text{KL}}(p_t^i \parallel \tilde{p}_t^i)$
  - 13:   **end for**
  - 14:   Compute regularization loss  $L_{\text{reg}} = \frac{1}{N} \sum_{i=1}^N L_{\text{reg}}^i$
  - 15:   Compute base MARL loss  $L_{\text{MARL}}$
  - 16:   Compute total loss  $L = L_{\text{MARL}} + \lambda L_{\text{reg}}$
  - 17:   Update policy parameters  $\theta_i \leftarrow \theta_i - \alpha \nabla_{\theta_i} L$  for all  $i$
  - 18: **end for**
  - 19: **return**  $\theta_1, \dots, \theta_N$
-

---

**Algorithm 19** Robust Value Factorization via Interval Bound Propagation

---

**Require:** Number of agents  $N$ , perturbation budget  $\epsilon$ , margin weight  $\lambda_M$ , learning rate  $\alpha$

**Ensure:** Individual action-value networks  $\{Q_i\}_{i=1}^N$  and mixing network

- 1: Initialize  $\{Q_i\}_{i=1}^N$ , mixing network, and target networks
  - 2: **for** each training iteration **do**
  - 3:     Sample a minibatch of transitions  $(\mathbf{o}, \mathbf{a}, \mathbf{r}, \mathbf{o}')$  from the replay buffer
  - 4:     **for**  $i = 1, \dots, N$  **do**
  - 5:         Propagate  $\underline{Q}_i = Q_i - \epsilon$ ,  $\bar{Q}_i = Q_i + \epsilon$  through  $Q_i$  via Interval Bound Propagation
  - 6:         Obtain certified bounds  $\underline{Q}_i(o_i, \cdot)$ ,  $\bar{Q}_i(o_i, \cdot)$
  - 7:         Identify nominal best action  $a_i^* = \arg \max_a Q_i(o_i, a)$
  - 8:         Compute margin loss  $L_{\text{margin}}^i = \sum_{a \neq a_i^*} \max \left( 0, \bar{Q}_i(o_i, a) - \underline{Q}_i(o_i, a_i^*) \right)$
  - 9:     **end for**
  - 10:     Compute mixed value  $Q_{\text{tot}}$  and TD target  $y$ ; compute  $L_{\text{QMIX}} = (y - Q_{\text{tot}})^2$
  - 11:     Compute total loss  $L = L_{\text{QMIX}} + \lambda_M \sum_{i=1}^N L_{\text{margin}}^i$
  - 12:     Update all networks using  $\nabla L$  with learning rate  $\alpha$
  - 13:     Periodically synchronize target networks
  - 14: **end for**
  - 15: **return** Trained individual action-value networks and mixing network
-

---

**Algorithm 20** Distributionally Robust Value Factorization (DrIGM)
 

---

**Require:** Robustness parameter  $\rho$ , target network update frequency  $f$ , exploration rate  $\epsilon$

**Ensure:** Robust individual action-value networks  $\{Q_i^{\text{rob}}\}_{i=1}^N$

```

1: Initialize replay buffer  $\mathcal{D}$ 
2: Initialize robust individual action-value networks  $\{Q_i^{\text{rob}}\}_{i=1}^N$  with parameters  $\theta$ 
3: Initialize the value factorization network producing  $Q_{\text{tot}}^{\mathcal{P}}$ 
4: Initialize target network parameters  $\theta^- \leftarrow \theta$ 
5: for episode  $e = 1, \dots, E$  do
6:   Observe the initial state  $s^0$ 
7:   for each agent  $i$  do
8:     Receive the initial observation  $o_i^0 = \sigma_i(s^0)$ 
9:   end for
10:  for time step  $t = 1, \dots, T$  do
11:    for each agent  $i$  do
12:      Select action  $a_i^t$  using an  $\epsilon$ -greedy policy based on  $Q_i^{\text{rob}}$ 
13:    end for
14:    Execute the joint action  $a^t = (a_1^t, \dots, a_N^t)$ 
15:    Observe reward  $r^t$ , next state  $s^{t+1}$ , and next observations  $o_i^{t+1} = \sigma_i(s^{t+1})$ 
16:    Store  $(h^t, a^t, r^t, h^{t+1})$  in replay buffer  $\mathcal{D}$ 
17:    Sample a mini-batch  $(h, a, r, h')$  from  $\mathcal{D}$ 
18:    Compute the robust Bellman target according to the selected uncertainty
    model
19:    Compute the temporal-difference loss
20:     $L_{\text{TD}} = (Q_{\text{tot}}^{\mathcal{P}}(h, a; \theta) - (\mathcal{T}Q_{\text{tot}}^{\mathcal{P}}(\cdot, \cdot; \theta^-))(h, a))^2$ 
21:    Update the network parameters  $\theta$  by minimizing  $L_{\text{TD}}$ 
22:    if target network update then
23:       $\theta^- \leftarrow \theta$ 
24:    end if
25:  end for
26: end for
27: return  $\{Q_i^{\text{rob}}\}_{i=1}^N$ 
    
```

---



**Algorithm 21** MARL with Adversarial Domain Randomization (ADR)

**Require:** Critic networks  $Q_\sigma$ , actor networks  $\pi_{\theta_i}$ , target critics  $Q_{\sigma'}$ , target actors  $\pi_{\theta'_i}$ , Nature Player  $\mu_\delta$ , replay buffers  $\{\mathcal{D}_i\}_{i=1}^N$

**Ensure:** Robust MARL policy  $\{\pi_{\theta_i}\}_{i=1}^N$

1: Initialize the MARL networks, target networks, Nature Player, and replay buffers

2: **for** training iteration  $t = 1, \dots, T$  **do**

3:   Reset the environment

4:   Observe the environment state  $o_\mu$

5:   Generate the domain randomization parameter

$$u \leftarrow \mu_\delta(o_\mu)$$

6:   Update the simulator using the generated parameter  $u$

7:   **for** each agent  $i = 1, \dots, N$  **do**

8:     Observe the current observation  $o_i$

9:     Select the action

$$a_i \leftarrow \pi_{\theta_i}(o_i) + \epsilon$$

10:   **end for**

11:   Execute the joint action

$$a = (a_1, a_2, \dots, a_N)$$

12:   Observe the reward  $r$  and the next observations  $o'$

13:   Compute the Nature Player loss

$$L_\mu = \alpha_0 r + \beta_0 D_{KL}(P_{\text{generated}} || P_{\text{normal}}) - \omega_0 \text{MSE}$$

14:   Update the Nature Player parameters  $\delta$

15:   **for** each agent  $i = 1, \dots, N$  **do**

16:     Store the transition

$$(o_i, a_i, r, o'_i, u, c_i^p, c_i^q, c_i^{p'}, c_i^{q'})$$

in replay buffer  $\mathcal{D}_i$

17:     Sample a mini-batch from  $\mathcal{D}_i$

18:     Update the critic network  $Q_\sigma$

19:     Update the actor network  $\pi_{\theta_i}$

20:     Soft update the target critic

$$Q_{\sigma'} \leftarrow Q_\sigma$$

21:     Soft update the target actor

$$\pi_{\theta'_i} \leftarrow \pi_{\theta_i}$$

22:   **end for**

23: **end for**

24: **return** Robust actor policies  $\{\pi_{\theta_i}\}_{i=1}^N$

---

**Algorithm 22** Prioritized Experience Replay for Domain Randomization Environments (PERDRE)

---

**Require:** Replay buffer  $\mathcal{D}$ , update batch  $B$ , random sample size  $M$ , subset replay counts  $K_{\text{list}}$

- 1: **while**  $B$  is not full **do**
- 2:     Randomly sample  $M$  experiences from  $\mathcal{D}$
- 3:     **for** each sampled experience  $e$  **do**
- 4:         Determine the subset  $\mathcal{E}$  containing  $e$
- 5:         Retrieve the replay count  $C$  of subset  $\mathcal{E}$
- 6:         Compute the critic loss  $L(e)$
- 7:         Compute the importance weight

$$I_w = \text{Rank}(L) + \text{Rank}'(C)$$

- 8:     Compute the sampling probability

$$P(e) = \left( \frac{\text{Rank}(I_w)}{\sum_e \text{Rank}(I_w)} \right)^{\alpha_s} + \beta_s$$

- 9:     Sample a random number  $r \sim U(0, 1)$
  - 10:    **if**  $r < P(e)$  **then**
  - 11:         Insert  $e$  into batch  $B$
  - 12:         Update the replay count of subset  $\mathcal{E}$
  - 13:    **end if**
  - 14:    **if**  $B$  is full **then**
  - 15:         **break**
  - 16:    **end if**
  - 17:    **end for**
  - 18: **end while**
  - 19: **return** Update batch  $B$
-

---

## 6. Adaptive Multi-Agent Reinforcement Learning in Non-Stationary Environments

The previous chapter focused on robustness, where agents are trained to maintain their performance under disturbances and uncertainty. Robust policies reduce performance degradation, but they do not necessarily improve when the environment changes over time. In many real-world applications, the environment is not fixed. Traffic patterns change throughout the day, wireless networks experience varying channel conditions, robots encounter new terrains, and teammates continuously update their policies. A policy that performs well initially may therefore become suboptimal as these changes accumulate.

Adaptive reinforcement learning addresses this limitation by enabling agents to modify their behavior in response to changing environments. Instead of assuming fixed transition dynamics and reward functions, adaptive methods continuously estimate environmental changes and update the learned policy accordingly. The objective is not only to maintain performance but also to recover quickly after changes occur while minimizing additional interaction with the environment.

Adaptation becomes even more challenging in multi-agent reinforcement learning. Since multiple agents learn simultaneously, the environment perceived by each agent changes even if the underlying physical system remains unchanged. From the perspective of an individual agent, changes in the policies of other agents appear as changes in the transition dynamics, violating the stationarity assumption required by many classical reinforcement learning algorithms. This phenomenon, commonly referred to as *agent-induced non-stationarity*, is one of the fundamental challenges in MARL.

This chapter introduces the mathematical foundations of non-stationary reinforcement learning and reviews the principal approaches developed to address it. First, the mathematical formulation of non-stationary Markov decision processes is presented together with common measures that quantify environmental changes. Next, different types of non-stationary environments are discussed, including slowly varying, piecewise stationary, switching, dynamic parameter, and adversarial settings. The chapter then surveys the main families of adaptive reinforcement learning methods, including change-point detection, sliding-window methods, representation

learning, hierarchical adaptation, meta-learning, transfer learning, and Bayesian approaches.

## 6.1. Mathematical Foundations of Non-Stationary Reinforcement Learning

Classical reinforcement learning assumes that the environment can be modeled as a stationary Markov Decision Process (MDP). Under this assumption, the transition probabilities and reward function remain constant throughout learning. Consequently, experiences collected early in training remain valid at later stages, allowing value functions and policies to converge.

Many practical applications violate this assumption. Environmental conditions may change continuously, abrupt events may modify the system dynamics, or latent operating modes may switch unexpectedly. As a result, the probability distribution governing future states evolves over time, and previously collected experiences may no longer represent the current environment.

To model these scenarios, the stationary MDP is extended to a *Non-Stationary Markov Decision Process (NS-MDP)* whose dynamics explicitly depend on time.

### 6.1.1. Non-Stationary Markov Decision Processes

A non-stationary Markov Decision Process is defined as

$$\mathcal{M}_t = (\mathcal{S}, \mathcal{A}, P_t, R_t, \gamma_t), \quad (6.1)$$

where

- $\mathcal{S}$  denotes the state space,
- $\mathcal{A}$  denotes the action space,
- $P_t(s'|s, a)$  is the time-dependent transition function,
- $R_t(s, a)$  is the time-dependent reward function,
- $\gamma_t$  is the discount factor at time  $t$ .

Unlike a stationary MDP, both the transition dynamics and the reward function may vary throughout learning. Formally,

$$P_t(s'|s, a) \neq P_{t+1}(s'|s, a), \quad (6.2)$$

and

$$R_t(s, a) \neq R_{t+1}(s, a). \quad (6.3)$$

These changes may occur gradually, abruptly, periodically, or according to an underlying latent process. As a result, the optimal policy also becomes time dependent,

$$\pi_t^* = \arg \max_{\pi} V_t^{\pi}, \quad (6.4)$$

requiring the learning algorithm to continuously adapt instead of converging to a single stationary policy.

### 6.1.2. Bellman Equation in Non-Stationary Environments

Since both the dynamics and rewards evolve over time, the Bellman equation must also incorporate temporal dependence. The value function of policy  $\pi$  at time step  $t$  is therefore written as

$$V_t^{\pi}(s) = \mathbb{E} \left[ r_t(s, a) + \gamma V_{t+1}^{\pi}(s') \right]. \quad (6.5)$$

Unlike the stationary Bellman equation, the value function now depends explicitly on time. Consequently, classical convergence guarantees no longer hold because the target value changes continuously during learning. Adaptive reinforcement learning algorithms therefore aim to track the moving optimum rather than converge to a fixed solution.

### 6.1.3. Variation Budget

The degree of non-stationarity can vary significantly across environments. In some applications, the dynamics change slowly over time, while in others abrupt changes occur frequently. To quantify the magnitude of these changes, the reinforcement learning literature commonly uses the *variation budget*[46]. It measures the cumulative amount of change in the environment throughout the learning horizon and provides a convenient way to characterize the difficulty of adapting to a non-stationary process.

Consider a sequence of environments parameterized by  $\{\theta_t\}_{t=1}^T$ , where  $\theta_t$  represents the environment parameters at time step  $t$ . The variation budget is defined as

$$V_T = \sum_{t=1}^{T-1} \|\theta_{t+1} - \theta_t\|, \quad (6.6)$$

where  $T$  denotes the time horizon and  $\|\cdot\|$  is an appropriate norm. A small variation budget indicates that the environment changes gradually, whereas a large variation budget corresponds to rapidly changing environments.

In reinforcement learning, the variation budget is often decomposed into separate measures for the transition dynamics and the reward function. The transition variation budget is defined as

### 6.1.4. Diameter of an MDP.

The diameter of an MDP measures how difficult it is to move between states. It is defined as

$$D(M) = \max_{s, s' \in S} \min_{\pi} \mathbb{E} [T(s' \mid M, \pi, s)], \quad (6.7)$$

where  $T(s' \mid M, \pi, s)$  denotes the first hitting time of state  $s'$  when following policy  $\pi$  from state  $s$ .

The diameter represents the worst-case expected number of steps required to reach any target state under the best possible policy. A small diameter indicates that all states are easy to reach, which makes exploration more efficient. In contrast, a large diameter means that some states are difficult to access. If the agent explores an unfavorable region of the state space, it may require many interactions before returning to a more rewarding region.

The diameter also determines the cost of exploration. When exploratory actions move the agent away from high-value states, recovering may require up to  $O(D)$  additional steps. As a result, the regret incurred during learning naturally scales with the diameter of the MDP.

Unlike quantities such as the mixing time, return time, or assumptions based on ergodicity, the diameter directly measures the effort needed to recover from poor exploratory decisions. For this reason, regret analyses in model-based reinforcement learning often use the MDP diameter as the primary measure of exploration difficulty.

### 6.1.5. Successor Representation and Successor Features

One of the main goals in reinforcement learning is to learn knowledge that can be reused across different tasks. Traditional value functions are tightly coupled with a specific reward function. As a result, even a small change in the reward often requires relearning the entire value function. Successor Representation (SR) addresses this limitation by separating the environment dynamics from the reward function. This separation allows an agent to reuse previously learned knowledge whenever the dynamics remain unchanged.

The key idea behind SR is to predict *where* the agent is likely to visit in the future instead of directly predicting future rewards. Once these future state occupancies are known, the value of any reward function can be computed without relearning the dynamics.

Unlike the Bellman equation, which recursively discounts future rewards, SR discounts the expected future state occupancies. The value function can therefore be written as

$$V(s) = \sum_{s'} \phi(s, s') R(s'), \quad (6.8)$$

where  $R(s')$  is the immediate reward associated with state  $s'$ , and  $\phi(s, s')$  is the *successor representation*, defined as

$$\phi(s, s') = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t \mathbb{1}(s_t = s') \mid s_0 = s \right]. \quad (6.9)$$

The successor representation measures the expected discounted number of future visits to state  $s'$  when starting from state  $s$ . Since it depends only on the transition dynamics and the policy, it can be reused whenever the reward function changes.

For a policy  $\pi$ , the successor representation can also be expressed as

$$\phi^\pi(s, s') = \sum_{t=1}^{\infty} \gamma^{t-1} (P(s'|s, \pi(s)))^t. \quad (6.10)$$

#### 6.1.5.1. Learning the Successor Representation

The successor representation can be learned efficiently using temporal-difference (TD) learning. Instead of updating a value estimate, the agent updates the successor matrix after every interaction with the environment. The TD error is given by

$$\delta_t(s') = \mathbb{I}(s_t = s') + \gamma \hat{M}(s_{t+1}, s') - \hat{M}(s_t, s'), \quad (6.11)$$

where  $\hat{M}$  denotes the current estimate of the successor matrix.

This update resembles the standard TD update used in value-based reinforcement learning, making SR computationally efficient while preserving the ability to generalize across different reward functions.

#### 6.1.5.2. Comparison with Other Learning Strategies

Successor Representation occupies a middle ground between model-free and model-based reinforcement learning. Model-based methods explicitly learn the environment dynamics and are therefore highly flexible, but they often require significant computational resources. Model-free methods are computationally efficient but usually struggle to adapt when the reward function changes.

SR combines advantages from both approaches. It captures the long-term dynamics of the environment without requiring a complete transition model. Consequently, it provides a good balance between computational efficiency and transfer capability.

**Table 6.1.** – Comparison of different reinforcement learning paradigms.

| Method                   | Flexibility | Efficiency |
|--------------------------|-------------|------------|
| Model-based RL           | High        | Low        |
| Episodic Learning        | High        | Low–Medium |
| Successor Representation | Medium      | Medium     |
| Model-free RL            | Low         | High       |
| Pavlovian Learning       | Low         | High       |



### 6.1.5.3. Transfer Learning with Successor Representations

One of the main advantages of successor representations is their ability to transfer knowledge between related tasks. Consider several tasks that share the same environment dynamics but differ in their reward functions. Instead of learning a new policy from scratch, the agent can reuse previously learned successor representations and only update the reward component.

Suppose that several policies  $\pi_j$  have already been learned. For a new reward function  $R$ , the best policy can be selected by evaluating all previously learned policies under the new reward,

$$\tilde{\pi}_R(s) = \arg \max_a \max_j Q_R^{\pi_j}(s, a). \quad (6.12)$$

This approach works because the action-value function naturally separates the environment dynamics from the reward function,

$$Q_{R_j}^{\pi_j}(s_t, a_t) = \mathbb{E}^{\pi_j} [r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots]. \quad (6.13)$$

Since the dynamics remain unchanged, only the reward needs to be re-evaluated. This greatly improves sample efficiency and reduces the amount of interaction required for learning new tasks.

### 6.1.5.4. Successor Features

Although successor representations provide an elegant decomposition of the value function, they operate directly on states. In many practical applications, however, rewards depend on high-level features rather than individual states. Successor Features (SF) extend SR by representing rewards as a function of feature vectors.

The immediate reward is assumed to be a linear combination of state features,

$$r_t = \phi_t^\top \mathbf{w}, \quad (6.14)$$

where  $\phi_t$  denotes the feature vector describing the current state and  $\mathbf{w}$  contains the task-specific reward weights.

For example, in a navigation problem,  $\phi_t$  may indicate which objects have been collected, while  $\mathbf{w}$  specifies the reward associated with each object.

Substituting this reward model into the action-value function gives

$$\begin{aligned}
 Q^\pi(s_t, a_t) &= \mathbb{E} [r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots] \\
 &= \mathbb{E} \left[ \sum_{k=0}^{\infty} \gamma^k \phi_{t+k} \right]^\top \mathbf{w} \\
 &= \psi^\pi(s_t, a_t)^\top \mathbf{w},
 \end{aligned} \tag{6.15}$$

where

$$\psi^\pi(s, a) = \mathbb{E} \left[ \sum_{k=0}^{\infty} \gamma^k \phi_{t+k} \right] \tag{6.16}$$

is known as the *successor feature*. Similar to SR, successor features capture long-term information about the environment while separating it from the reward function.

#### 6.1.5.5. Limitations of Successor Features

The main limitation of successor features is the assumption that the reward function is linear,

$$r_t = \phi_t^\top \mathbf{w}. \tag{6.17}$$

Although this assumption simplifies transfer learning, it is often too restrictive for real-world problems. Many tasks involve complex and highly nonlinear reward functions that cannot be expressed as a simple weighted sum of features.

As a result, the performance of successor features depends heavily on the quality of the feature representation and the validity of the linear reward assumption.

#### 6.1.5.6. Successor Feature Representation

To overcome the limitations of successor features, Successor Feature Representation (SFR) removes the linear reward assumption. Instead of representing rewards as a weighted sum of features, SFR allows rewards to be modeled as an arbitrary nonlinear function,

$$r_t = R(\phi_t). \tag{6.18}$$

The action-value function can then be written as

$$\begin{aligned}
 Q^\pi(s_t, a_t) &= \sum_{k=0}^{\infty} \gamma^k \mathbb{E} [R(\phi_{t+k})] \\
 &= \sum_{k=0}^{\infty} \gamma^k \int_{\Phi} p(\phi_{t+k} = \phi | s_t, a_t; \pi) R(\phi) d\phi \\
 &= \int_{\Phi} R(\phi) \sum_{k=0}^{\infty} \gamma^k p(\phi_{t+k} = \phi | s_t, a_t; \pi) d\phi \\
 &= \int_{\Phi} R(\phi) \xi^\pi(s_t, a_t, \phi) d\phi,
 \end{aligned} \tag{6.19}$$

where

$$\xi^\pi(s, a, \phi) = \sum_{k=0}^{\infty} \gamma^k p(\phi_{t+k} = \phi | s, a; \pi) \tag{6.20}$$

is called the *Successor Feature Representation*.

Rather than predicting future states or expected feature vectors, SFR estimates the discounted distribution of future features. This formulation makes it possible to evaluate arbitrary reward functions while preserving the transferability of learned environment dynamics. Consequently, SFR extends the original successor feature framework to a much broader class of reinforcement learning problems.

### 6.1.6. Low-Rank Markov Decision Processes.

Successor Feature Representation assumes that the environment can be described using a compact feature representation. A similar idea appears in low-rank Markov decision processes (Low-Rank MDPs), where the transition dynamics are also represented in a low-dimensional latent space. Instead of learning a separate transition probability for every state-action pair, the transition model is factorized as

$$P(s' | s, a) = \sum_{i=1}^d \phi_i(s, a) \mu_i(s') = \phi(s, a)^\top \mu(s'), \tag{6.21}$$

where  $d \ll |\mathcal{S}||\mathcal{A}|$ . Here,  $\phi(s, a) \in \mathbb{R}^d$  is a latent feature vector associated with the state-action pair, while  $\mu(s') \in \mathbb{R}^d$  contains a set of basis transition distributions over the next states.

Unlike classical tabular MDPs, which estimate a transition probability for every  $(s, a, s')$  tuple, Low-Rank MDPs assume that all transition dynamics lie in a low-dimensional subspace. The latent feature vector  $\phi(s, a)$  determines how strongly each basis transition contributes to the next-state distribution. This factorization substantially reduces the number of parameters that must be learned.

The low-rank assumption also improves sample efficiency. Instead of estimating  $O(|\mathcal{S}||\mathcal{A}|)$  independent transition models, the learner only needs to estimate  $d$  latent components. As a result, the sample complexity scales with the intrinsic dimension of the environment rather than the size of the state-action space.

This factorization closely resembles Principal Component Analysis (PCA). Rather than modeling the full transition matrix directly, the transition dynamics are approximated by a small number of latent basis components that capture the dominant patterns of the environment.

## 6.2. Types of Non-Stationary Environments

Non-stationary environments differ not only in the magnitude of change but also in how the changes occur over time. Some environments evolve gradually, while others undergo sudden transitions or switch between several operating modes. In other cases, the environment changes continuously because its underlying parameters evolve, or it may change in an adversarial manner with the objective of degrading the agent's performance. These different forms of non-stationarity require different adaptation strategies and are commonly modeled using distinct variants of the Markov decision process.

Table 6.2 summarizes the main categories of non-stationary environments discussed in the literature.

Each category is discussed in more detail below.

### 6.2.1. Slowly Varying Environments

In slowly varying environments, the transition dynamics and reward function change gradually over time. Consecutive environments differ only slightly, allowing the optimal policy to evolve smoothly rather than changing abruptly. This assumption is appropriate for applications where the underlying system evolves continuously, such as weather conditions, traffic density, or battery degradation.

A common assumption is that the transition function satisfies

**Table 6.2.** – Common types of non-stationary environments.

| Type                 |      | Characteristics                                    | Typical Model            | Core Challenge                                        |
|----------------------|------|----------------------------------------------------|--------------------------|-------------------------------------------------------|
| Slowly Varying       |      | Continuous and gradual changes over time           | Non-Stationary MDP       | Tracking a slowly moving optimum                      |
| Piecewise Stationary | Sta- | Abrupt changes at unknown time instants            | Piecewise Stationary MDP | Rapid detection and adaptation after changes          |
| Switching            |      | Transitions between several operating modes        | Switching NS-MDP         | Inferring the current regime and adapting accordingly |
| Dynamic Parameter    | Pa-  | Environment parameters evolve continuously         | Dynamic Parameter MDP    | Estimating continuously changing parameters           |
| Adversarial          |      | Worst-case changes without statistical assumptions | Adversarial MDP          | Maintaining performance under malicious changes       |

$$\|P_{t+1} - P_t\| \leq \varepsilon, \quad (6.22)$$

where  $\varepsilon$  is a small positive constant that bounds the amount of change between consecutive time steps.

Since the environment evolves continuously, adaptive algorithms aim to track the moving optimum instead of relearning the policy from scratch. Online learning methods and adaptive policy updates are commonly used in this setting because they can gradually adjust the policy as the environment changes.

### 6.2.2. Piecewise Stationary Environments

Piecewise stationary environments remain unchanged for a period of time before undergoing abrupt transitions at unknown change points. Between two consecutive change points, the environment behaves as a stationary MDP, but after a change occurs the transition dynamics or reward function are replaced by a different model.

The transition dynamics can be written as

$$P_t = \begin{cases} P^{(1)}, & t < \tau_1, \\ P^{(2)}, & \tau_1 \leq t < \tau_2, \\ \vdots & \end{cases} \quad (6.23)$$

where  $\tau_i$  denotes the unknown change points.

This model captures situations such as equipment failures, traffic incidents, or sudden environmental events. The primary challenge is to detect changes quickly while avoiding false alarms, since delayed adaptation leads to significant performance degradation.

### 6.2.3. Switching Non-Stationary Markov Decision Processes

Switching non-stationary environments assume that the system alternates between a finite set of operating regimes. Each regime corresponds to a different stationary MDP, while the active regime changes over time according to a stochastic process.

Let

$$\{\mathcal{M}^1, \mathcal{M}^2, \dots, \mathcal{M}^K\}$$

denote a collection of stationary MDPs. At time  $t$ , the environment is described by one of these models,

$$\mathcal{M}_t \in \{\mathcal{M}^1, \dots, \mathcal{M}^K\}.$$

The switching process is often modeled as a Markov chain,

$$P(\mathcal{M}_t | \mathcal{M}_{t-1}), \quad (6.24)$$

where the current regime depends only on the previous one and The switches between these MDPs at unknown change points.

In many applications the active regime is not directly observable. The learning agent must therefore infer the current operating mode from observations while simultaneously learning an appropriate control policy.

#### 6.2.4. Dynamic Parameter Markov Decision Processes

Dynamic Parameter MDPs assume that the environment is governed by a parametric model whose parameters evolve over time. Instead of switching between several discrete environments, the transition dynamics and reward function change continuously as their underlying parameters vary.

The transition and reward models can be expressed as

$$P_{\theta_t}(s'|s, a), \quad (6.25)$$

and

$$R_{\phi_t}(s, a), \quad (6.26)$$

where  $\theta_t$  and  $\phi_t$  denote time-dependent model parameters.

This formulation is suitable for systems whose dynamics evolve continuously, such as robotic platforms with changing payloads, vehicles experiencing gradual wear, or communication networks with varying channel conditions. The main challenge is to estimate the changing parameters accurately while maintaining a high-quality policy.

#### 6.2.5. Adversarial Environments

Unlike the previous settings, adversarial environments make no assumptions about how the environment changes. Instead, the transition dynamics, rewards, or observations may be modified in a worst-case manner to reduce the agent's performance. These changes may result from malicious attacks or from highly unpredictable operating conditions.

The learning objective is therefore no longer to optimize the expected return under a fixed environment but to maintain satisfactory performance under the worst possible sequence of changes. This setting is commonly formulated as a minimax optimization problem, where the learning agent maximizes its return while an adversary attempts to minimize it.

Adversarial models provide strong robustness guarantees but are generally more difficult to solve because they assume little or no prior knowledge about the evolution of the environment. They are widely used to evaluate the robustness and adaptability of reinforcement learning algorithms under extreme conditions. Adversarial environments were covered in the previous chapter on robust MARL. Therefore, this

chapter focuses on non-stationary environments that arise from natural changes in the environment rather than adversarial interventions.

### 6.3. Performance Objectives

The objective of reinforcement learning changes fundamentally in non-stationary environments. In a stationary MDP, the environment remains fixed throughout learning, and the goal is to learn a policy that converges to the optimal solution. Consequently, cumulative reward or expected return provides an appropriate measure of performance.

In a non-stationary environment, however, the optimal policy evolves over time as the transition dynamics or reward function change. Under these conditions, convergence to a single stationary policy is neither possible nor desirable. Instead, the objective is to continuously adapt the policy so that its performance remains close to the time-varying optimum. As a result, performance evaluation shifts from measuring convergence to measuring how effectively the learning algorithm tracks environmental changes.

Several performance objectives have been proposed to evaluate adaptive reinforcement learning algorithms. Unlike stationary reinforcement learning, these objectives quantify not only the quality of the learned policy but also its ability to respond to changing environments. Among the most widely used measures are dynamic regret and the average dynamic suboptimality gap, which compare the learner against the optimal policy at each point in time rather than against a single fixed policy.

#### 6.3.1. Dynamic Regret

Dynamic regret is one of the standard performance measures for non-stationary reinforcement learning. Unlike static regret, which compares the learner with a single optimal policy over the entire learning horizon, dynamic regret compares the learner with the optimal policy at every time step. It therefore measures how well the learning algorithm tracks a continuously changing optimum.

For a learning horizon of length  $T$ , the dynamic regret is defined as

$$\text{DynReg}_T(\pi) = \sum_{t=1}^T (\rho_t^* - \mathbb{E}_{P_t, R_t, \pi} [r_t(s_t, a_t)]), \quad (6.27)$$



where  $\rho_t^*$  denotes the optimal long-term average reward at time step  $t$ , while the second term represents the expected reward obtained by the learning policy under the current environment.

The expected reward can be written as

$$\mathbb{E} [r_t(s_t, a_t)] = \sum_s d_t^\pi(s) \sum_a \pi(a|s) r_t(s, a), \quad (6.28)$$

where  $d_t^\pi(s)$  denotes the state visitation distribution induced by policy  $\pi$  at time step  $t$ .

A small dynamic regret indicates that the learned policy successfully follows the changing optimal policy, whereas a large regret implies that the learner adapts too slowly to environmental changes.

#### 6.3.2. Average Dynamic Suboptimality Gap

While dynamic regret evaluates the cumulative performance loss over time, it does not directly measure how closely the learned policy follows the optimal policy after each environmental change. For this purpose, many adaptive reinforcement learning methods report the *Average Dynamic Suboptimality Gap*, which measures the average difference between the value of the learned policy and the value of the optimal policy across a sequence of tasks or environments.

Consider a sequence of  $K$  tasks or environments. The average dynamic suboptimality gap is defined as

$$\text{Gap}_{\text{Ave}}(K) = \frac{1}{K} \sum_{k=1}^K (V_k^* - V_k^{\pi_k}), \quad (6.29)$$

where  $V_k^*$  denotes the value of the optimal policy for the  $k$ -th task, and  $V_k^{\pi_k}$  is the value achieved by the policy learned for the same task.

A small suboptimality gap indicates that the learning algorithm rapidly adapts to environmental changes and consistently produces policies that remain close to the optimal solution. Conversely, a large gap suggests that the adaptation process is slow or ineffective, causing the learned policy to lag behind the changing optimum.

Unlike dynamic regret, which accumulates errors throughout the learning process, the average dynamic suboptimality gap focuses on the quality of the learned policy

after adaptation. Consequently, it is widely used to evaluate meta-learning, continual learning, and transfer learning algorithms that operate across multiple non-stationary tasks.

## 6.4. Taxonomy of Adaptive Reinforcement Learning Methods

Adaptive reinforcement learning methods have been developed to address different forms of non-stationarity. Although these methods share the common objective of maintaining high performance in changing environments, they differ in how they detect environmental changes, represent task information, and update the learned policy.

Broadly, existing approaches can be grouped into several categories. Change-point detection methods explicitly identify when the environment changes before adapting the policy. Sliding-window methods continuously discard outdated experiences to focus on recent observations. Representation learning approaches identify the components of the environment that have changed and update only the affected parts of the model. Hierarchical adaptation separates long-term adaptation from low-level decision making, while meta-learning and transfer learning exploit prior experience to enable rapid adaptation to new tasks. Bayesian approaches model uncertainty explicitly and continuously update the agent’s belief about the current environment.

The following sections review these families of adaptive reinforcement learning methods together with representative algorithms proposed in the literature.

### 6.4.1. Change-Point Detection

Change-point detection methods assume that the environment remains stationary until an unknown change occurs. Their primary objective is therefore to detect these changes as quickly and accurately as possible. Once a change is detected, the learning algorithm updates or restarts the policy using data collected under the new environment.

This approach is particularly effective for piecewise stationary environments, where the transition dynamics or reward function remain unchanged for a period of time before changing abruptly. Rather than continuously adapting the policy, change-point detection methods first determine whether a significant change has occurred

and then trigger an adaptation mechanism only when necessary. This strategy reduces unnecessary policy updates while allowing the agent to respond rapidly after environmental changes.

Most change-point detection algorithms consist of two stages. The first stage continuously monitors observations, rewards, or transition dynamics to determine whether the statistical properties of the environment have changed. The second stage performs policy adaptation after a change has been detected. Depending on the algorithm, this adaptation may involve restarting the learning process, updating the value function, maintaining multiple candidate models, or transferring knowledge from previously encountered environments.

The change-point detection methods presented in this section address different forms of non-stationary environments introduced in Section 6.2. Bayesian Online Change-Point Detection (BOCPD), UCRL2, and their combination in Restarted BOCPD are designed for Piecewise Stationary MDPs, where abrupt changes divide the environment into stationary segments. In contrast, Reinforcement Learning with Context Detection (RL-CD) targets Switching Non-Stationary MDPs by identifying recurring operating contexts and reusing previously learned policies.

#### 6.4.1.1. Bayesian Online Change-Point Detection (BOCPD)

Bayesian Online Change-Point Detection (BOCPD), proposed by Adams and MacKay [47], is a probabilistic framework for detecting abrupt changes in sequential data. Unlike offline change-point detection methods, which require the complete observation sequence before identifying change points, BOCPD processes observations sequentially and updates its belief after every newly observed sample. This online formulation enables the algorithm to detect changes immediately as data become available, making it particularly suitable for adaptive decision-making problems.

BOCPD assumes that the observed data are generated by a sequence of stationary regimes. Within each regime, observations are independently generated from the same probability distribution. When a change point occurs, the parameters of the distribution are replaced by a new set of parameters, indicating that a different regime has become active. Formally, the observations are modeled as

$$o_t \sim p(o \mid \theta^{(k)}), \quad (6.30)$$

where  $\theta^{(k)}$  denotes the parameters associated with the  $k$ -th regime. Throughout this thesis, the observation model is assumed to belong to the exponential family introduced in Appendix D. This assumption allows the posterior distribution to be updated recursively using the corresponding sufficient statistics.

**Run Length** The central concept of BOCPD is the *run length*, denoted by  $\ell_t$ , which represents the number of consecutive observations generated since the most recent change point. Instead of directly estimating whether a change has occurred, BOCPD maintains a probability distribution over all possible run lengths. Consequently, change-point detection becomes equivalent to estimating the posterior distribution of the run length.

At every time step, two outcomes are possible. If the current regime continues, the run length increases by one. Otherwise, a change point occurs and the run length is reset to zero. Formally,

$$\ell_t = \begin{cases} \ell_{t-1} + 1, & \text{if no change occurs,} \\ 0, & \text{if a change point occurs.} \end{cases} \quad (6.31)$$

Rather than selecting a single run-length estimate, BOCPD computes the posterior probability of every possible run length. A high posterior probability at  $\ell_t = 0$  indicates that a new regime has likely started, whereas larger values of  $\ell_t$  suggest that the current regime is continuing.

**Hazard Function** The probability that a new regime begins at the current time step is modeled through the *hazard function*. The hazard function specifies the prior probability that a change point occurs given the current run length. Formally,

$$H(\tau) = \frac{P_{\text{gap}}(g = \tau)}{\sum_{t=\tau}^{\infty} P_{\text{gap}}(g = t)}, \quad (6.32)$$

where  $P_{\text{gap}}(g)$  denotes the probability distribution of the interval between two consecutive change points.

Using the hazard function, the transition model of the run length is defined as

$$P(\ell_t \mid \ell_{t-1}) = \begin{cases} H(\ell_{t-1} + 1), & \ell_t = 0, \\ 1 - H(\ell_{t-1} + 1), & \ell_t = \ell_{t-1} + 1, \\ 0, & \text{otherwise.} \end{cases} \quad (6.33)$$

The first case corresponds to the occurrence of a change point, whereas the second case indicates that the current regime continues.

A common assumption is that the intervals between change points follow a geometric distribution. Under this assumption, the hazard function becomes constant,

$$H(\tau) = \frac{1}{\lambda}, \quad (6.34)$$

where  $\lambda$  denotes the expected duration of a stationary regime.

**Recursive Bayesian Inference** The objective of BOCPD is to recursively estimate the posterior distribution of the run length as new observations become available. Rather than maintaining a single estimate of the current regime, the algorithm computes the probability of every possible run length conditioned on all observations collected so far. This probabilistic formulation allows BOCPD to quantify the uncertainty associated with the location of the most recent change point.

Suppose that observations  $o_{1:t}$  have been collected up to time step  $t$ . The recursive update is based on the joint distribution of the current run length and the observation sequence,

$$P(\ell_t, o_{1:t}) = \sum_{\ell_{t-1}} P(\ell_t | \ell_{t-1}) P(o_t | \ell_{t-1}, o_t^{(\ell)}) P(\ell_{t-1}, o_{1:t-1}), \quad (6.35)$$

where  $P(\ell_t | \ell_{t-1})$  is the run-length transition model defined by the hazard function,  $P(o_t | \ell_{t-1}, o_t^{(\ell)})$  is the predictive likelihood of the newly observed sample, and  $P(\ell_{t-1}, o_{1:t-1})$  denotes the joint distribution from the previous time step.

Equation (6.35) evaluates every possible run-length hypothesis. For each hypothesis, the algorithm considers both possible outcomes: the current regime continues, causing the run length to increase by one, or a change point occurs, resetting the run length to zero. Each hypothesis is weighted by the probability of observing the new sample under the corresponding predictive model.

The posterior distribution of the run length is obtained by normalizing the joint distribution,

$$P(\ell_t | o_{1:t}) = \frac{P(\ell_t, o_{1:t})}{P(o_{1:t})}, \quad (6.36)$$

where

$$P(o_{1:t}) = \sum_{\ell_t} P(\ell_t, o_{1:t}) \quad (6.37)$$

is the marginal likelihood of the observations. The posterior distribution represents the belief over all possible run lengths after incorporating the latest observation. A

high posterior probability at  $\ell_t = 0$  indicates that a change point has likely occurred, whereas larger run lengths suggest that the current regime is continuing.

An important feature of BOCPD is that prediction and inference are performed jointly. Instead of relying on a single run-length estimate, the predictive distribution of the next observation is obtained by averaging over all possible run lengths,

$$P(o_{t+1}|o_{1:t}) = \sum_{\ell_t} P(o_{t+1}|\ell_t, o_{1:t})P(\ell_t|o_{1:t}). \quad (6.38)$$

This Bayesian model averaging naturally accounts for uncertainty in the current regime and produces more robust predictions than selecting only the most likely run length.

To compute the predictive likelihood efficiently, BOCPD maintains a set of sufficient statistics for every possible run length. Assuming an exponential-family D observation model with a conjugate prior, the sufficient statistics are updated recursively. When a change point occurs, the statistics are reset to their prior values,

$$v_{t+1}^{(0)} = v_{\text{prior}}, \quad \chi_{t+1}^{(0)} = \chi_{\text{prior}}. \quad (6.39)$$

For continuing run-length hypotheses, the sufficient statistics are updated as

$$v_{t+1}^{(\ell+1)} = v_t^{(\ell)} + 1, \quad (6.40)$$

$$\chi_{t+1}^{(\ell+1)} = \chi_t^{(\ell)} + u(o_t), \quad (6.41)$$

where  $u(o_t)$  denotes the sufficient statistic of the newly observed sample. These recursive updates eliminate the need to store the complete observation history and enable BOCPD to perform exact online Bayesian inference with constant memory per run-length hypothesis.

**Advantages** BOCPD offers several advantages for online change-point detection. First, it performs exact Bayesian inference, allowing the algorithm to maintain a probabilistic belief over all possible change-point locations instead of making binary decisions. Second, the recursive formulation enables online processing, making the algorithm suitable for sequential data where observations arrive continuously. Third, the use of conjugate priors and sufficient statistics allows the posterior distribution to be updated efficiently without storing the complete observation history. Finally, BOCPD naturally quantifies uncertainty, providing a confidence measure for every possible run length rather than only identifying the most likely change point [47].

**Limitations** Despite its advantages, BOCPD has several limitations when applied to reinforcement learning. The algorithm detects changes in the observation distribution but does not learn a control policy or determine how the agent should adapt after a change has been detected. In addition, the computational complexity increases with the number of possible run lengths because the posterior distribution must be maintained for every hypothesis. The performance of BOCPD also depends on the choice of the hazard function and the observation model. An inaccurate hazard model or an inappropriate likelihood function may delay change detection or increase the false alarm rate. Consequently, BOCPD is commonly combined with reinforcement learning algorithms that provide the adaptation mechanism after a change has been detected [47].

The complete Bayesian Online Change-Point Detection procedure is summarized in Algorithm 23. At each time step, BOCPD receives a new observation, updates the posterior distribution over all possible run lengths, recursively updates the sufficient statistics for every run-length hypothesis, and predicts the next observation. This recursive Bayesian update enables the algorithm to detect changes online while maintaining a probabilistic belief over the location of the most recent change point.

### 6.4.1.2. Near-optimal Regret Bounds for Reinforcement Learning (UCRL2)

Unlike BOCPD, which detects abrupt changes in sequential observations, UCRL2 addresses the problem of learning an optimal policy in an unknown Markov decision process (MDP). Proposed by Jaksch *et al.* [48], UCRL2 is a model-based reinforcement learning algorithm built on the principle of optimism in the face of uncertainty (OFU). Rather than committing to a single estimated model of the environment, the algorithm maintains a set of statistically plausible MDPs and plans optimistically within this set. By selecting the policy that performs best under the most favorable plausible model, UCRL2 naturally balances exploration and exploitation while achieving near-optimal regret guarantees.

Although UCRL2 was originally developed for stationary environments, it has become one of the fundamental algorithms underlying many adaptive reinforcement learning methods. In particular, its optimistic planning framework serves as the learning component of Restarted BOCPD, which combines change-point detection with model-based reinforcement learning.

**Regret Minimization** The objective of UCRL2 is to minimize the cumulative regret incurred while interacting with an unknown MDP. Regret measures the performance difference between the learning algorithm and an optimal policy that has

complete knowledge of the environment. Formally, the regret after  $T$  time steps is defined as

$$\Delta(M, \pi, T) = T\rho^*(M) - R(M, \pi, T), \quad (6.42)$$

where  $\rho^*(M) = \max_{\pi} \lim_{T \rightarrow \infty} E_{\pi, p, r} [R(M, A, s, T)]$  denotes the optimal average reward of the true MDP and  $R(M, \pi, T)$  is the cumulative reward obtained by policy  $\pi$ . An algorithm is considered efficient if its regret grows sublinearly with time. In this case,

$$\frac{\Delta(M, \pi, T)}{T} \rightarrow 0, \quad T \rightarrow \infty, \quad (6.43)$$

which implies that the learned policy eventually achieves the same long-term performance as the optimal policy.

**Optimism in the Face of Uncertainty** The central idea of UCRL2 is the principle of optimism in the face of uncertainty. Since the transition probabilities and reward function of the environment are initially unknown, the algorithm does not rely on a single estimated model. Instead, it constructs a set of plausible MDPs that are statistically consistent with the observed data. Among these candidates, UCRL2 selects the model with the highest expected average reward and computes the corresponding optimal policy.

This optimistic strategy encourages exploration without requiring an explicit exploration mechanism. State-action pairs with limited observations are associated with greater uncertainty and therefore remain attractive because they may correspond to more rewarding models. As additional experience is collected, the uncertainty decreases, allowing the optimistic model to gradually converge toward the true environment.

**Empirical Estimation** At the beginning of each learning episode, UCRL2 estimates the reward function and transition probabilities using all observations collected since the start of learning. For every state-action pair  $(s, a)$ , the algorithm maintains the visitation count  $N_k(s, a)$  together with the cumulative rewards and observed transitions.

The empirical reward estimate is computed as

$$\hat{r}_k(s, a) = \frac{R_k(s, a)}{\max\{1, N_k(s, a)\}}, \quad (6.44)$$



where  $R_k(s, a)$  denotes the cumulative reward collected after executing action  $a$  in state  $s$  before episode  $k$ .

Similarly, the transition probabilities are estimated as

$$\hat{p}_k(s'|s, a) = \frac{P_k(s, a, s')}{\max\{1, N_k(s, a)\}}, \quad (6.45)$$

where  $P_k(s, a, s')$  denotes the number of observed transitions from state  $s$  to state  $s'$  after taking action  $a$ . These empirical estimates provide the current approximation of the unknown MDP and are continuously refined as more interactions with the environment are collected.

**Confidence Sets** Because the empirical reward and transition estimates are obtained from a finite number of observations, they are subject to statistical uncertainty. Rather than assuming that these estimates exactly match the true environment, UCRL2 constructs a confidence set containing all MDPs that are statistically consistent with the collected data. With high probability, the true MDP belongs to this set.

For the reward function, the confidence interval is defined as

$$|\tilde{r}(s, a) - \hat{r}_k(s, a)| \leq \sqrt{\frac{7 \log\left(\frac{2SA t_k}{\delta}\right)}{2 \max\{1, N_k(s, a)\}}}, \quad (6.46)$$

where  $\delta$  is the confidence parameter.

Similarly, the transition probabilities satisfy

$$\|\tilde{p}(\cdot|s, a) - \hat{p}_k(\cdot|s, a)\|_1 \leq \sqrt{\frac{14S \log\left(\frac{2A t_k}{\delta}\right)}{\max\{1, N_k(s, a)\}}}. \quad (6.47)$$

Together, these confidence bounds define the set of plausible MDPs,

$$\mathcal{M}_k = \{(\tilde{P}, \tilde{R}) : \tilde{P}, \tilde{R} \text{ satisfy the confidence bounds}\}. \quad (6.48)$$

As the number of visits to each state-action pair increases, the confidence intervals become narrower because the empirical estimates become more accurate. Consequently, the confidence set gradually shrinks toward the true MDP while still accounting for the uncertainty arising from limited observations.

**Optimistic Planning** Once the confidence set has been constructed, UCRL2 follows the principle of optimism in the face of uncertainty by planning over all plausible MDPs rather than relying solely on the empirical estimates. Specifically, the algorithm searches for the MDP within the confidence set that yields the highest optimal average reward. Formally, the optimistic MDP is obtained by

$$\tilde{M}_k = \arg \max_{M \in \mathcal{M}_k} \rho^*(M), \quad (6.49)$$

where  $\rho^*(M)$  denotes the optimal average reward achievable in MDP  $M$ . The corresponding optimal policy is then executed during the current learning episode. This optimistic strategy naturally balances exploration and exploitation. State-action pairs with high uncertainty remain attractive because they may belong to a more rewarding environment, while well-explored regions are exploited according to the current empirical estimates.

**Extended Value Iteration** To compute the optimistic policy, UCRL2 employs Extended Value Iteration (EVI), which extends the conventional value iteration algorithm to account for uncertainty in both the reward function and transition probabilities. Unlike standard value iteration, which assumes that the environment model is known, EVI simultaneously optimizes over all MDPs contained in the confidence set.

At iteration  $i$ , the value function is updated according to

$$u_{i+1}(s) = \max_{a \in \mathcal{A}} \left[ \tilde{r}(s, a) + \max_{\tilde{p}(\cdot|s, a) \in \mathcal{M}_k} \sum_{s' \in \mathcal{S}} \tilde{p}(s'|s, a) u_i(s') \right], \quad (6.50)$$

where  $\tilde{r}(s, a)$  and  $\tilde{p}(s'|s, a)$  denote the optimistic reward and transition model selected from the confidence set. The first maximization selects the action with the highest expected return, while the second searches for the transition model that maximizes the future value within the confidence region.

The iteration continues until the value function converges according to the span criterion

$$\text{span}(u_{i+1} - u_i) \leq \varepsilon, \quad (6.51)$$

where

$$\text{span}(u) = \max_s u(s) - \min_s u(s), \quad (6.52)$$

and  $\varepsilon$  is a predefined convergence threshold. Once convergence is achieved, the resulting optimistic value function is used to derive the policy executed during the current episode.

**Episode Updates** UCRL2 divides the interaction process into a sequence of learning episodes. At the beginning of each episode, the algorithm computes the empirical reward and transition estimates, constructs the confidence set of plausible MDPs, and determines an optimistic policy using Extended Value Iteration.

During an episode, the policy remains fixed while the agent interacts with the environment and collects new observations. The episode terminates when the number of visits to at least one state-action pair doubles relative to its count at the beginning of the episode. Formally, the stopping criterion is

$$v_k(s, a) \geq \max\{1, N_k(s, a)\}, \quad (6.53)$$

where  $N_k(s, a)$  denotes the number of visits to state-action pair  $(s, a)$  before episode  $k$ , and  $v_k(s, a)$  is the number of visits accumulated during the current episode. Once this condition is satisfied, the empirical estimates, confidence sets, and optimistic policy are recomputed using all observations collected so far. This episodic update strategy ensures that planning is repeated only after sufficient new information has been gathered, reducing computational cost while continuously improving the model estimates.

The complete UCRL2 procedure is summarized in Algorithm 24. At the beginning of each episode, the algorithm estimates the reward function and transition probabilities from the accumulated observations, constructs a confidence set of plausible MDPs, and computes an optimistic policy using Extended Value Iteration. The policy is then executed until the episode termination criterion is satisfied, after which the empirical estimates and confidence sets are updated before the next episode begins.

#### 6.4.1.3. Restarted Bayesian Online Change-Point Detection for Non-Stationary MDPs

Although BOCPD provides an effective mechanism for detecting abrupt changes and UCRL2 achieves near-optimal regret in stationary environments, neither method alone is sufficient for reinforcement learning in piecewise stationary MDPs. BOCPD

identifies change points but does not specify how the agent should adapt its policy after a change is detected. Conversely, UCRL2 assumes that the environment remains stationary throughout learning, causing its empirical transition model and confidence sets to become invalid after an environmental change.

Restarted Bayesian Online Change-Point Detection combines these two methods into a unified adaptive reinforcement learning framework. BOCPD continuously monitors the observed state transitions and detects abrupt changes in the environment, while UCRL2 learns an optimistic policy within each stationary segment. Whenever BOCPD detects a new change point, the accumulated statistics maintained by UCRL2 are discarded and the learning process is restarted using only observations collected after the detected change point [49].

**Main Idea** The central idea of the proposed framework is to separate change detection from policy learning. BOCPD is responsible for determining whether the transition dynamics have changed, whereas UCRL2 focuses on learning an optimistic policy for the current stationary segment. By coupling these two components, the algorithm prevents outdated observations from influencing future policy updates while preserving the exploration guarantees provided by UCRL2.

Unlike the original BOCPD algorithm, which was developed for general sequential observations, the restarted framework adapts the Bayesian change-point detector to reinforcement learning by modeling state transitions directly. Several modifications are introduced to make BOCPD compatible with Markov decision processes, including a multinomial observation model, a Laplace predictive estimator, a forecaster-based detection mechanism, and a restart strategy that reinitializes UCRL2 whenever a change is detected [49].

**Framework Overview** During interaction with the environment, every observed state transition is processed by the BOCPD module. The detector continuously evaluates whether the transition distribution remains consistent with the current stationary segment. If no change is detected, UCRL2 continues collecting observations, updating its empirical transition model, and executing the optimistic policy computed by Extended Value Iteration.

When the detector identifies a change point, all empirical statistics maintained by UCRL2, including visitation counts, transition estimates, and confidence sets, are discarded. A new learning episode is then initialized using only observations collected after the detected change point. This restart mechanism prevents obsolete information from degrading policy performance while allowing UCRL2 to efficiently learn the dynamics of the new stationary segment [49].

**Multinomial Observation Model** The original BOCPD framework was developed for generic sequential observations and does not assume a particular data source. In reinforcement learning, however, the observations correspond to state transitions generated by the interaction between the agent and the environment. To apply BOCPD in this setting, the observation model must therefore be adapted to represent transition probabilities instead of arbitrary data samples [49].

Consider a state-action pair  $(s, a)$ . Executing action  $a$  in state  $s$  results in a transition to one of the successor states in the state space  $\mathcal{S}$ . Since exactly one successor state is observed after each interaction, the transition outcome naturally follows a multinomial distribution. Let

$$\mathbf{p}_{s,a} = (p_1, p_2, \dots, p_{|\mathcal{S}|}) \quad (6.54)$$

denote the transition probability vector, where

$$\sum_{i=1}^{|\mathcal{S}|} p_i = 1. \quad (6.55)$$

Given the current state-action pair  $(s, a)$ , the observed successor state is modeled as

$$s' \sim \text{Multinomial}(\mathbf{p}_{s,a}). \quad (6.56)$$

Consequently, each state-action pair defines an independent multinomial observation model whose parameters correspond to the transition probabilities of the underlying MDP.

To perform Bayesian inference, the multinomial likelihood is combined with a Dirichlet prior,

$$\mathbf{p}_{s,a} \sim \text{Dir}(\boldsymbol{\alpha}), \quad (6.57)$$

where

$$\boldsymbol{\alpha} = (\alpha_1, \alpha_2, \dots, \alpha_{|\mathcal{S}|}) \quad (6.58)$$

contains the pseudo-counts associated with each successor state. Because the Dirichlet distribution is conjugate to the multinomial distribution, the posterior distribution can be updated efficiently after every observed transition without storing

the complete interaction history. This property makes the multinomial observation model well suited for online Bayesian change-point detection in reinforcement learning [49].

**Laplace Predictor** After modeling state transitions using a multinomial distribution, the next step is to estimate the probability of observing the next transition. In the original BOCPD framework, this probability is obtained from the predictive distribution of the underlying probabilistic model. For reinforcement learning, however, transition probabilities are initially unknown and many successor states may not have been observed. Estimating the predictive distribution directly from empirical frequencies therefore assigns zero probability to unseen transitions, making the detector overly sensitive during the early stages of learning.

To overcome this problem, the restarted BOCPD framework adopts a Laplace predictor. The predictor estimates the probability of the next observation by augmenting the empirical transition counts with one pseudo-count for every possible successor state. This smoothing technique guarantees that every transition remains possible, even if it has not yet been observed, resulting in more stable Bayesian updates during online learning [49].

Assume that a forecaster is initialized at time step  $s$  and has observed the sequence

$$o_{t'}, o_{t'+1}, \dots, o_t.$$

Let  $n_{t':t}$  denote the number of observations collected by the forecaster and let  $O$  denote the number of possible observations. The Laplace predictor estimates the probability of the next observation as

$$\text{PRED}(o_{t+1} \mid o_{s:t}) = \frac{\sum_{i=t'}^t \mathbf{1}(o_i = o_{t+1}) + 1}{n_{t':t} + O}, \quad (6.59)$$

where  $\mathbf{1}(\cdot)$  is the indicator function that equals one when its argument is true and zero otherwise.

Equation (6.59) estimates the predictive probability from the observed transition frequencies while assigning one pseudo-count to every possible outcome. Consequently, successor states that have not yet been observed still receive a small, non-zero probability. As more observations are collected, the influence of the pseudo-counts gradually decreases and the predictor converges to the empirical transition distribution.

The Laplace predictor forms the probabilistic model used by each forecaster to evaluate incoming transitions. These predictive probabilities are subsequently used

to update the forecaster weights and determine whether the observed transitions are more consistent with the current stationary segment or with a newly emerging one.

**Forecaster Framework** The restarted BOCPD framework maintains a collection of forecasters, where each forecaster represents a different hypothesis regarding the location of the most recent change point. A forecaster initialized at time step  $s$  assumes that the current stationary segment began at time  $s$  and uses all observations collected since then to predict future transitions. As new observations become available, every forecaster independently evaluates how well its predictive model explains the observed data [49].

Suppose a forecaster starts at time step  $s$ . After observing the sequence

$$o_{t'}, o_{t'+1}, \dots, o_t,$$

its predictive performance is evaluated using the logarithmic loss

$$\ell_{t':t} = -\log(\text{PRED}(o_t|o_{t':t-1})), \quad (6.60)$$

where  $\text{PRED}(o_t|o_{t':t-1})$  is the predictive probability computed using the Laplace predictor in Equation (6.59). A small loss indicates that the observed transition is consistent with the forecaster's prediction, whereas a large loss suggests that the forecaster no longer accurately represents the current environment.

Instead of using the loss directly, the algorithm assigns a weight to every forecaster. The weight is updated recursively according to

$$\omega_{t':t} = \omega_{t':t-1} \text{PRED}(o_t|o_{t':t-1}), \quad (6.61)$$

where  $\omega_{t':t}$  denotes the weight of the forecaster initialized at time step  $s$ . Forecasters that consistently predict the observed transitions receive larger weights, whereas inaccurate forecasters gradually lose weight.

As the environment remains stationary, the forecaster corresponding to the beginning of the current stationary segment accumulates the largest weight because its predictive model remains consistent with the observed transitions. After an abrupt environmental change, however, its predictions become inaccurate and its weight decreases rapidly. At the same time, newly created forecasters initialized after the change begin to explain the observations more accurately, causing their weights to increase.

The collection of forecaster weights therefore provides an online measure of how well different change-point hypotheses explain the current observations. Rather than committing to a single estimate immediately, the algorithm continuously compares these competing hypotheses and uses their relative weights to determine whether a change point has occurred.

**Restart Criterion** The forecaster weights provide a quantitative measure of how well each candidate stationary segment explains the observed transitions. During a stationary period, the forecaster corresponding to the beginning of the current segment consistently assigns high predictive probabilities to the observed transitions and therefore maintains the largest weight. After an abrupt environmental change, however, its predictions become increasingly inaccurate, causing its weight to decrease. At the same time, forecasters initialized after the change rapidly accumulate larger weights because their predictive models better match the new transition distribution [49].

To determine whether a change has occurred, the algorithm compares the cumulative weights of the competing forecasters. Let  $\omega_{r,s:t}$  denote the cumulative weight of the forecaster initialized at time step  $r$  after processing the observations from time step  $s$  to  $t$ . Similarly, let  $\omega_{r,r:t}$  denote the weight obtained by the forecaster that started at time step  $r$  and has processed all observations since its initialization.

A restart is triggered whenever

$$\omega_{r,t':t} > \omega_{r,r:t}, \tag{6.62}$$

indicating that the observations are explained more accurately by a recently initialized forecaster than by the one representing the current stationary segment.

Intuitively, this comparison measures whether the statistical properties of the observed transitions have changed. If the older forecaster continues to provide the best explanation of the observations, the environment is assumed to remain stationary and learning proceeds without interruption. Conversely, if a newer forecaster consistently achieves a larger cumulative weight, the algorithm concludes that the transition dynamics have changed and identifies the beginning of a new stationary segment.

Once the restart criterion in Equation (6.62) is satisfied, the change-point detector reports the estimated change point to the reinforcement learning module. UCRL2 then discards all empirical statistics collected before the detected change point and begins learning a new transition model using only observations from the current



stationary segment. This mechanism prevents outdated observations from biasing the transition estimates while preserving the optimistic exploration strategy of UCRL2.

**Integration with UCRL2** The restart criterion provides the interface between the Bayesian change-point detector and the reinforcement learning algorithm. BOCPD continuously monitors the observed state transitions and determines whether the current observations remain consistent with the existing stationary segment. As long as no change point is detected, UCRL2 continues learning using the accumulated transition counts, reward estimates, and confidence sets. Once the restart criterion is satisfied, the detector signals that the underlying MDP has changed and initiates a new learning episode [49].

Following a restart, all empirical statistics collected before the detected change point are discarded. In particular, the visitation counts, empirical transition probabilities, and reward estimates maintained by UCRL2 are reinitialized so that only observations collected within the current stationary segment contribute to model estimation. New confidence sets are then constructed from the updated statistics, and Extended Value Iteration is executed to compute a new optimistic policy for the current environment.

Formally, if the detector estimates a change point at time step  $\hat{t}$ , the empirical statistics are reconstructed using only the observations collected after  $\hat{t}$ ,

$$\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1}) \mid t \geq \hat{t}\}, \quad (6.63)$$

where  $\mathcal{D}$  denotes the dataset used to estimate the transition probabilities and reward function after the restart. Consequently, the confidence set

$$\mathcal{M}_k = \{(\tilde{P}, \tilde{R}) : \|\tilde{P} - \hat{P}\| \leq \beta_P, \|\tilde{R} - \hat{R}\| \leq \beta_R\} \quad (6.64)$$

is reconstructed using only the new observations. The optimistic MDP selected from this confidence set therefore reflects the current stationary segment rather than the previous environment.

This integration combines the complementary strengths of both algorithms. BOCPD provides a statistically principled mechanism for detecting abrupt changes, while UCRL2 efficiently learns an optimistic policy once a new stationary segment has been identified. By resetting the empirical model after every detected change, the framework prevents obsolete observations from biasing future policy updates and preserves the near-optimal exploration properties of UCRL2 within each stationary segment [49].

The complete Restarted BOCPD framework is summarized in Algorithm 25. At each interaction step, the algorithm monitors the observed state transitions using BOCPD while simultaneously learning an optimistic policy using UCRL2. When the change detector identifies a new stationary segment, the empirical statistics maintained by UCRL2 are discarded and the learning process is restarted using only observations collected after the detected change point.

Algorithm 25 illustrates how BOCPD and UCRL2 operate together within a single adaptive reinforcement learning framework. During normal operation, UCRL2 interacts with the environment and updates its empirical model using the observed transitions. At the same time, BOCPD continuously evaluates whether the transition distribution remains consistent with the current stationary segment. When a change point is detected, the observations collected before the estimated change point are discarded, the empirical model is reconstructed using only the current stationary segment, and UCRL2 computes a new optimistic policy. If no change is detected, learning proceeds without interruption. This integration enables the agent to rapidly adapt to abrupt environmental changes while retaining the near-optimal exploration strategy of UCRL2.

**Advantages** Restarted BOCPD combines Bayesian change-point detection with optimistic reinforcement learning to address piecewise stationary environments. The framework detects abrupt changes online and automatically restarts the learning process when the current model no longer explains the observed transitions. By discarding outdated observations, the algorithm avoids negative transfer from previous stationary segments and rapidly adapts to the new environment. Furthermore, integrating UCRL2 preserves efficient exploration within each stationary segment while maintaining the theoretical guarantees of optimistic reinforcement learning [49].

**Limitations** The framework assumes that environmental changes are abrupt and that the environment remains stationary between consecutive change points. Its performance therefore depends on the accuracy of the Bayesian change detector and the choice of the hazard function. Delayed detections increase the amount of outdated experience used by UCRL2, whereas false alarms may trigger unnecessary restarts and increase the learning cost. In addition, the algorithm is designed for tabular MDPs and requires maintaining multiple forecasters, making it difficult to scale to environments with large or continuous state spaces [49].

The methods discussed so far assume that the environment changes through a sequence of independent stationary segments. After each detected change, the learning process is restarted because the previous model is no longer considered valid. In many real-world applications, however, environments repeatedly switch among a

finite number of previously observed operating modes. Restarting the learning algorithm after every change becomes inefficient because an effective policy may already exist for the current operating context. Reinforcement Learning with Context Detection (RL-CD) addresses this setting by identifying the active context and reusing previously learned knowledge instead of learning a new policy from scratch.

#### 6.4.1.4. Reinforcement Learning with Context Detection (RL-CD)

Reinforcement Learning with Context Detection (RL-CD) was proposed by Da Silva *et al.* [50] to address environments that repeatedly switch between a finite number of stationary Markov decision processes. Rather than treating every environmental change as an entirely new task, RL-CD assumes that previously encountered environments may reappear over time. The objective is therefore to recognize the current operating context and reuse the policy learned for that context instead of learning a new policy from [50]. Unlike adaptive methods designed for continuously changing environments, RL-CD is specifically intended for *Switching Non-Stationary MDPs*. The environment is assumed to switch among a known or gradually expanding set of contexts,

$$\mathcal{C} = \{C_1, C_2, \dots, C_K\}, \quad (6.65)$$

where each context corresponds to an individual stationary MDP,

$$\mathcal{M}^{(i)} = (\mathcal{S}, \mathcal{A}, P^{(i)}, R^{(i)}, \gamma). \quad (6.66)$$

At every time step, the environment is governed by one context,

$$C_t \in \mathcal{C}, \quad (6.67)$$

and the active context may change during execution. Consequently, the learning problem consists of jointly identifying the current context and selecting the policy associated with that context.

The central idea of RL-CD is that environments often revisit previously observed operating modes. For example, an autonomous vehicle may repeatedly experience morning traffic, afternoon traffic, evening congestion, and nighttime traffic. Although these contexts differ, they recur over time. Instead of relearning a policy whenever the traffic pattern changes, RL-CD identifies the active context and immediately reuses the corresponding policy. This substantially reduces the amount of interaction required after each environmental change. [50]

To achieve this objective, RL-CD maintains a collection of context-specific models. Each model stores estimates of the transition probabilities, reward function, and policy for one context. During interaction, every model predicts the current transition and reward. The model whose predictions best match the observed experience is selected as the active context. If none of the existing models provides sufficiently accurate predictions, a new context model is created and learned incrementally. Consequently, RL-CD continuously expands its library of contexts while reusing previously learned knowledge whenever a familiar environment [50]

**Assumptions** RL-CD assumes that each context corresponds to a stationary MDP with fixed transition probabilities and reward function. The environment therefore changes only when switching from one context to another. Furthermore, the algorithm assumes that the number of contexts is finite and that previously encountered contexts may reappear during execution. These assumptions distinguish RL-CD from methods designed for continuously drifting environments, where the dynamics evolve gradually and previously learned models may no longer be valid. RL-CD is independent of the underlying reinforcement learning algorithm and can be combined with any model-based reinforcement learning method for policy learning. [50]

**Architecture** Each context is represented by an independent model containing estimates of the transition probabilities, reward function, visitation counts, and the associated policy. The reward model is updated using an incremental moving average,

$$R_m(s, a) \leftarrow R_m(s, a) + \frac{r - R_m(s, a)}{N_m(s, a) + 1}, \quad (6.68)$$

where  $r$  is the observed reward and  $N_m(s, a)$  denotes the number of times state-action pair  $(s, a)$  has been visited under model  $m$ . This incremental update enables the reward estimates to improve continuously as additional experience is collected. [50]

**Context Detection** The key challenge in switching non-stationary environments is determining when the active context changes. Unlike many adaptive reinforcement learning algorithms that monitor the agent’s return or the temporal-difference error, RL-CD detects context switches by evaluating the prediction accuracy of every context model. The underlying intuition is that a model learned for the correct context should accurately predict both the observed reward and the next-state transition.

If these predictions become inaccurate, the current context model is no longer consistent with the environment, indicating that a context switch has likely occurred. [50]

To evaluate the quality of each model, RL-CD combines three quantities: a confidence measure, the reward prediction quality, and the transition prediction quality.

The confidence of model  $m$  is defined as

$$c_m(s, a) = \frac{N_m(s, a)}{M}, \quad (6.69)$$

where  $N_m(s, a)$  denotes the visitation count of state-action pair  $(s, a)$  for model  $m$ , and  $M$  is the maximum memory size. Models that have accumulated more experience are assigned higher confidence because their transition and reward estimates are expected to be more reliable.

The reward prediction quality is computed from the reward prediction error,

$$\Delta R_m = \frac{r - R_m(s, a)}{N_m(s, a) + 1}, \quad (6.70)$$

where  $r$  is the observed reward and  $R_m(s, a)$  is the reward predicted by model  $m$ . The corresponding quality measure is

$$e_m^R = 1 - 2 \left( Z_R (\Delta R_m)^2 \right), \quad (6.71)$$

where

$$Z_R = \frac{1}{R_{\max} - R_{\min}} \quad (6.72)$$

is a normalization constant that scales the prediction error. Large reward prediction errors reduce the quality score, indicating that the model no longer accurately represents the current environment. [50]

The transition prediction quality is computed in a similar manner. First, the transition probability update for successor state  $\kappa$  is calculated as

$$\Delta T_m(\kappa) = \begin{cases} \frac{1 - T_m(s, a, \kappa)}{N_m(s, a) + 1}, & \kappa = s', \\ \frac{-T_m(s, a, \kappa)}{N_m(s, a) + 1}, & \kappa \neq s', \end{cases} \quad (6.73)$$

where  $T_m(s, a, \kappa)$  denotes the estimated transition probability to successor state  $\kappa$ . The transition quality is then given by

$$e_m^T = 1 - 2 \left( Z_T \sum_{\kappa \in \mathcal{S}} (\Delta T_m(\kappa))^2 \right), \quad (6.74)$$

where

$$Z_T = \frac{1}{2} (N_m(s, a) + 1)^2 \quad (6.75)$$

is a normalization factor. Similar to the reward quality, poor transition predictions reduce the quality score of the corresponding model.

The reward and transition quality measures are combined into a single instantaneous quality score,

$$e_m = c_m(s, a) \left( \Omega e_m^R + (1 - \Omega) e_m^T \right), \quad (6.76)$$

where  $\Omega \in [0, 1]$  controls the relative importance of reward and transition prediction. The confidence term gives greater weight to models that have accumulated sufficient experience while reducing the influence of newly created models.

Because prediction errors may fluctuate due to stochastic transitions, RL-CD maintains a quality trace using an exponential moving average,

$$E_m \leftarrow E_m + \rho(e_m - E_m), \quad (6.77)$$

where  $\rho$  is the smoothing factor. The quality trace provides a more stable estimate of model performance by filtering out short-term fluctuations.

At every interaction step, the quality trace is computed for all context models. The model with the highest quality score is selected as the active context. If the highest quality falls below a predefined threshold, the algorithm concludes that none of the existing models adequately explains the current observations and a new context model is created.

**Context Selection** After computing the quality trace for every context model, RL-CD selects the model that best explains the current observations. Rather than estimating posterior probabilities or likelihoods, the algorithm compares the quality traces of all available context models and activates the model with the highest quality score. Formally, the selected context is

$$m_{\text{cur}} = \arg \max_{m \in \mathcal{M}} E_m, \quad (6.78)$$

where  $E_m$  denotes the quality trace of model  $m$ , and  $\mathcal{M}$  is the set of all learned context models.

The quality trace combines the reward prediction quality, transition prediction quality, and model confidence into a single measure that reflects how accurately a model predicts the current interaction. Consequently, the selected context is the one whose transition and reward models are most consistent with the observed experience.

If the quality trace of the selected model falls below a predefined threshold,

$$E_{m_{\text{cur}}} < E_{\min}, \quad (6.79)$$

the algorithm assumes that none of the existing context models adequately represents the current environment. A new context model is then created and initialized to learn the dynamics of the new operating mode. Otherwise, the selected context is reused, allowing the agent to immediately exploit the policy previously learned for that environment. This context selection mechanism enables RL-CD to recognize recurring environments while avoiding unnecessary relearning.

**Policy Reuse** Once the current context has been identified, RL-CD activates the policy associated with the selected context. If the detected context has been encountered previously, the corresponding transition model, reward model, and policy are immediately reused without requiring additional training. Consequently, the agent can resume near-optimal behavior after only a short detection period.

If no existing context adequately explains the observed experience, RL-CD initializes a new context model and begins learning a policy for the new environment using a model-based reinforcement learning algorithm. As additional experience is collected, the transition probabilities, reward estimates, and policy are updated incrementally until the new context becomes sufficiently accurate.

This context reuse mechanism is the principal advantage of RL-CD over conventional reinforcement learning algorithms. Instead of relearning policies whenever the

environment revisits a previous operating mode, RL-CD retrieves the corresponding model from memory, significantly reducing both the adaptation time and the amount of interaction required to recover after a context switch. This capability makes RL-CD particularly well suited to Switching Non-Stationary MDPs, where environments repeatedly alternate among a finite set of stationary contexts. The complete RL-CD procedure is summarized in Algorithm 26. During each interaction step, the algorithm first executes the current policy, evaluates the prediction quality of every context model, selects the model with the highest quality score, and creates a new context if none of the existing models adequately explain the observed transition. Only the selected context model is updated, while all other models remain unchanged for future reuse.

### 6.4.2. Sliding Window Methods

Sliding window methods address non-stationary environments by continuously updating the learning model using only the most recent observations. Unlike change-point detection methods, they do not explicitly determine when the environment changes. Instead, they assume that recent interactions provide a more accurate representation of the current environment than older experiences. As a result, outdated observations are gradually discarded while newly collected data are incorporated into the learning process.

This approach is particularly suitable for environments whose transition dynamics or reward functions evolve continuously over time. Since no explicit change detector is required, sliding window methods naturally adapt to both gradual and abrupt changes without restarting the learning algorithm. However, their performance depends strongly on the choice of the window size, which determines how quickly outdated information is forgotten.

**Dynamic Regret** Since the environment changes over time, the objective is no longer to compete against a single optimal policy. Instead, performance is evaluated using dynamic regret, which compares the accumulated reward with the sequence of optimal policies corresponding to the changing environments. For environments with bounded variation, SW-UCRL achieves a dynamic regret bound that depends on both the learning horizon and the variation budget. Consequently, the algorithm adapts more effectively as long as the amount of environmental change remains limited



**Advantages** Sliding window methods are simple to implement and require no explicit change detection mechanism. By continuously replacing outdated observations with recent ones, they naturally adapt to changing environments while avoiding unnecessary policy restarts. They are particularly effective in environments with gradual changes, where the distinction between consecutive regimes is often difficult to identify.

**Limitations** The performance of sliding window methods depends critically on the window size. A small window produces noisy transition and reward estimates because only a limited number of observations are available, whereas a large window slows adaptation by retaining outdated information for longer periods. Moreover, these methods treat all observations within the window equally, regardless of how informative they are, which may reduce adaptation efficiency in environments with frequent or abrupt changes.

Although SW-UCRL significantly improves the adaptability of UCRL2 by discarding outdated observations, it still assumes that the empirical MDP estimated from the sliding window accurately represents the current environment. This assumption does not always hold in environments with continuously changing transition dynamics. Since the sliding window may contain observations collected under different environment configurations, the resulting empirical transition model can become inconsistent with any true MDP. In particular, averaging transitions from multiple environments may produce an empirical MDP with an artificially large diameter, which degrades optimistic planning and increases regret. This limitation motivated the development of Sliding-Window UCRL with Confidence Widening (SWUCRL2-CW), which extends SW-UCRL by enlarging the confidence sets to account for the additional uncertainty introduced by non-stationarity.

##### 6.4.2.1. Sliding-Window UCRL2 (SW-UCRL2)

SW-UCRL2 was proposed by Gajane, Ortner, and Auer [51] as a direct extension of UCRL2 to non-stationary environments. It keeps the same optimism-in-the-face-of-uncertainty framework: estimate the MDP, build a confidence set of plausible models, plan optimistically inside that set. The one change is where the estimates come from.

UCRL2 computes its empirical reward and transition estimates from every observation collected since the start of learning. This works when the MDP is fixed. When rewards or transitions change over time, old observations describe an environment that no longer exists, and including them biases the estimates toward the past.

SW-UCRL2 fixes this by using a sliding window. At episode  $k$ , the algorithm computes  $\hat{r}_k(s, a)$  and  $\hat{p}_k(s'|s, a)$  only from the last  $W$  time steps,  $[t_k - W, t_k)$ , instead of the full history. Everything older than  $W$  steps is discarded. The confidence sets, the optimistic MDP selection, and the extended value iteration step are unchanged from UCRL2; only the data feeding into them is restricted to the window.

This turns non-stationarity into a bias–variance trade-off controlled by a single parameter,  $W$ . A short window forgets outdated data fast but produces noisy estimates from few samples. A long window gives stable estimates but reacts slowly to real changes. This trade-off is what the regret analysis below formalizes, and it is also what motivates the confidence-widening extension discussed in the Limitations paragraph above.

**Empirical Estimation** SW-UCRL2 estimates the reward function and transition probabilities using only observations from the current window. For every state-action pair  $(s, a)$ , the algorithm tracks how many times that pair was visited inside the window, together with the rewards and transitions observed during those visits.

The windowed visitation count is

$$N_k(s, a) = \#\{t_k - W \leq \tau < t_k : s_\tau = s, a_\tau = a\}, \quad (6.80)$$

and the windowed reward estimate is

$$\hat{r}_k(s, a) = \frac{R_k(s, a)}{\max\{1, N_k(s, a)\}}, \quad (6.81)$$

where  $R_k(s, a) = \sum_{\tau=t_k-W}^{t_k-1} r_\tau \mathbf{1}_{s_\tau=s, a_\tau=a}$  is the cumulative reward collected from  $(s, a)$  inside the window.

Similarly, the windowed transition estimate is

$$\hat{p}_k(s' | s, a) = \frac{P_k(s, a, s')}{\max\{1, N_k(s, a)\}}, \quad (6.82)$$

where  $P_k(s, a, s')$  counts transitions to  $s'$  from  $(s, a)$  observed inside the window. As soon as an observation falls outside  $[t_k - W, t_k)$ , it stops contributing to these estimates, regardless of how much data was collected before the window.

**Confidence Sets** SW-UCRL2 builds its confidence set the same way UCRL2 does, using Hoeffding-style bounds around the windowed estimates. The reward confidence interval is

$$|\tilde{r}(s, a) - \hat{r}_k(s, a)| \leq \sqrt{\frac{7 \log(2SA t_k / \delta)}{2 \max\{1, N_k(s, a)\}}}, \quad (6.83)$$

and the transition confidence interval is

$$\|\tilde{p}(\cdot | s, a) - \hat{p}_k(\cdot | s, a)\|_1 \leq \sqrt{\frac{14S \log(2At_k/\delta)}{\max\{1, N_k(s, a)\}}}. \quad (6.84)$$

These bounds define the set of plausible MDPs  $\mathcal{M}_k$ , exactly as in UCRL2. The only difference is that  $N_k(s, a)$ ,  $\hat{r}_k(s, a)$ , and  $\hat{p}_k(\cdot | s, a)$  are now computed from the window instead of the full history. Because the window has bounded size, these confidence intervals do not shrink indefinitely as  $t \rightarrow \infty$ ; they stabilize once the window is full, which keeps the algorithm reactive to change instead of growing increasingly confident in a possibly outdated model.

**Optimistic Planning** Given  $\mathcal{M}_k$ , SW-UCRL2 selects the optimistic MDP

$$\tilde{M}_k = \arg \max_{M' \in \mathcal{M}_k} \rho^*(M'), \quad (6.85)$$

and computes the corresponding policy  $\tilde{\pi}_k$  using extended value iteration, in the same way as UCRL2. Formally, the policy satisfies

$$\rho^*(\tilde{M}_k, \tilde{\pi}_k) \geq \max_{M' \in \mathcal{M}_k, \pi, s'} \rho(M', \pi, s') - \frac{1}{\sqrt{t_k}}. \quad (6.86)$$

An episode ends once some state-action pair has been visited as many times during the episode as it was visited in the window before the episode started:

$$v_k(s, a) \geq \max\{1, N_k(s, a)\}. \quad (6.87)$$

At that point the window, the estimates, the confidence set, and the policy are all recomputed. Algorithm 27 summarizes the full procedure.

**Regret Analysis** Because the environment can change, SW-UCRL2 is evaluated using dynamic regret rather than the static regret used for UCRL2. Within any interval where the MDP stays fixed, the regret bound takes the form

$$\text{Regret} = \tilde{O}\left(\frac{T}{\sqrt{W}} DS \sqrt{A}\right). \quad (6.88)$$

Larger windows improve the statistical accuracy of the estimates, which lowers this term. Combining this with the cost of adapting to each change gives the full regret bound over the horizon:

$$\Delta(T) \leq 2lW + 66.12 \left\lceil \frac{T}{\sqrt{W}} \right\rceil DS \sqrt{A \log(T/\delta)}, \quad (6.89)$$

where  $l$  is the number of changes in the environment. The bound has two parts. The term  $2lW$  is the adaptation cost: every time the environment changes, up to  $W$  steps of stale data remain in the window until they age out. The second term is the estimation cost, which shrinks as  $W$  grows. These two costs pull in opposite directions, which is exactly the trade-off described in the overview above.

**Optimal Window Size** Minimizing (6.89) over  $W$  gives the optimal window size

$$W^* = \left( 16.53 \frac{TDS\sqrt{A\log(T/\delta)}}{l} \right)^{2/3}. \quad (6.90)$$

Substituting  $W^*$  back into the regret bound gives the final result:

$$\Delta(T) = \tilde{O}\left(l^{1/3}T^{2/3}D^{2/3}S^{2/3}A^{1/3}\right). \quad (6.91)$$

This is worse than the  $\tilde{O}(\sqrt{T})$  regret achievable in a stationary MDP, and that gap is expected: the algorithm must keep re-learning as the environment shifts, so it cannot converge the way a stationary learner can. Equation (6.91) shows that the degradation is graceful. Regret grows with the number of changes  $l$  and the environment’s complexity  $(D, S, A)$ , but not explosively. Doubling the number of changes only raises regret by a factor of  $2^{1/3} \approx 1.26$ .

**Limitations Recap** As already noted,  $W^*$  requires knowing  $T$ ,  $l$ , and the variation budget in advance, which is rarely realistic in practice. This is precisely the gap that SWUCRL2-CW closes by widening the confidence sets instead of tuning  $W$  against unknown quantities.

#### 6.4.2.2. Sliding-Window UCRL2 with Confidence Widening (SWUCRL2-CW)

SWUCRL2-CW was proposed by Cheung, Simchi-Levi, and Zhu [52] to fix a specific failure mode of plain SW-UCRL2. The fix is not a smarter window rule; it is extra optimism.

**Why the Naive Sliding Window Fails** Cheung et al. show that the empirical MDP built from a window of size  $W$  can have a diameter as large as  $\Theta(W)$ , far larger than the true maximum diameter  $D_{\max}$  of any individual MDP the learner actually faces. This happens because a window spanning a period of change mixes transitions from different underlying MDPs. The averaged transition matrix that results does not

correspond to any single, well-behaved MDP: it can describe a process that is far harder to navigate than either of the real MDPs it was blended from. Extended value iteration, run on a confidence set built this way, can then return a policy with an uncontrollably large bias span, and the resulting regret bound degrades accordingly. Widening the window does not fix this, since a larger window only mixes more environments together.

**Confidence Widening** SWUCRL2-CW addresses this by adding a second tuning parameter, the confidence widening parameter  $\eta \geq 0$ , on top of the window size  $W$ . Where SW-UCRL2 tightens its confidence region as more windowed data accumulates, SWUCRL2-CW deliberately keeps the transition confidence region wider than the data alone would justify. The reward confidence region is unchanged from the sliding-window construction:

$$H_{r,t}(s, a) = \{ \dot{r} \in [0, 1] : |\dot{r} - \hat{r}_t(s, a)| \leq \text{rad}_{r,t}(s, a) \}, \quad \text{rad}_{r,t}(s, a) = 2\sqrt{\frac{2 \log(\text{SAT}/\delta)}{N_t^+(s, a)}}, \quad (6.92)$$

where  $N_t^+(s, a) = \max\{1, N_t(s, a)\}$  and  $N_t(s, a)$  counts visits within the current window, exactly as in Equations (6.80)–(6.81). The transition confidence region, however, gains the widening term:

$$H_{p,t}(s, a)(\eta) = \{ \dot{p}(\cdot | s, a) \in \Delta_{\mathcal{S}} : \|\dot{p}(\cdot | s, a) - \hat{p}_t(\cdot | s, a)\|_1 \leq \text{rad}_{p,t}(s, a) + \eta \}, \quad (6.93)$$

where  $\Delta_{\mathcal{S}}$  is the probability simplex over  $\mathcal{S}$  and  $\text{rad}_{p,t}(s, a)$  is the same windowed Hoeffding-style radius used in SW-UCRL2. The extra  $+\eta$  slack is what keeps the diameter of the optimistic MDP under control: it guarantees the confidence region contains transition kernels close enough to a well-behaved MDP, even when the raw windowed estimate does not.

**Policy Construction** SWUCRL2-CW runs in episodes exactly like SW-UCRL2, with one difference in the stopping rule. Episode  $m$  starts at time  $\tau(m)$  and ends at the earliest of two events: the time index becomes a multiple of  $W$ , which caps every episode at  $W$  steps, or some state-action pair is visited during the episode at least as often as it was visited in the preceding window,

$$v_m(s, a) \geq N_{\tau(m)}(s, a). \quad (6.94)$$

At the start of each episode, the algorithm computes the widened confidence sets  $H_{r,\tau(m)}$  and  $H_{p,\tau(m)}(\eta)$  from the preceding  $W$  steps of data, and runs extended value iteration over them to obtain a policy  $\tilde{\pi}_m$ , which is then held fixed for the rest of the episode. Algorithm 28 summarizes the complete procedure. It follows the same

episodic template as Algorithm 27, with two changes: the transition confidence set is widened by  $\eta$ , and the episode stopping rule uses the doubling-style criterion above instead of the visitation-based rule in SW-UCRL2.

**Regret Analysis** The confidence widening technique introduces a direct trade-off: too little widening leaves the diameter problem unresolved, while too much widening injects excess optimism and inflates regret on its own. Cheung et al. formalize this trade-off in the following bound.

**Theorem 1.** [52] *Assume  $S > 1$ . With probability at least  $1 - O(\delta)$ , the SWUCRL2-CW algorithm with window size  $W$  and confidence widening parameter  $\eta > 0$  satisfies the dynamic regret bound*

$$\tilde{O}\left(B_p \frac{W}{\eta} + B_r W + \frac{\sqrt{SAT}}{\sqrt{W}} + D_{\max} \left[ B_p W + \frac{S\sqrt{AT}}{\sqrt{W}} + T\eta + \frac{SAT}{W} + \sqrt{T} \right]\right), \quad (6.95)$$

where  $B_r$  and  $B_p$  are the variation budgets of the reward and transition distributions over the horizon  $T$ , and  $D_{\max}$  is the maximum diameter across all MDPs encountered.

Setting

$$W^* = 3S^{2/3}A^{1/2}T^{1/2}(B_r + B_p + 1)^{-1/2}, \quad \eta^* = \sqrt{(B_p + 1)W^*/T}, \quad (6.96)$$

specializes the bound to

$$\tilde{O}\left(D_{\max}(B_r + B_p + 1)^{1/4}S^{2/3}A^{1/2}T^{3/4}\right). \quad (6.97)$$

**Comparison with SW-UCRL2** Two differences from the SW-UCRL2 bound in Equation (6.91) stand out. First, SWUCRL2-CW’s regret is stated directly in terms of the variation budgets  $B_r$  and  $B_p$  rather than a count of discrete change points  $l$ ; this makes it apply to environments that drift continuously, not only ones that switch between stationary regimes. Second, the horizon dependence is  $T^{3/4}$  rather than  $T^{2/3}$ , which is a weaker guarantee in  $T$  alone. The two bounds are not directly comparable because they are stated under different non-stationarity models, but the comparison illustrates a recurring theme in this line of work: guarding against the diameter blow-up of the naive sliding window costs something, and confidence widening is one way to pay that cost while still controlling regret in closed form.

### 6.4.3. Factored and Causal Adaptation

Change-point detection and sliding window methods both treat the state as a single, undifferentiated block. When something in the environment changes, the entire model is re-estimated, whether from a fresh window of data or after a detected change point. This is wasteful whenever only part of the environment has actually changed. Factored and causal adaptation methods address this by decomposing the environment into separate pieces and updating only the pieces responsible for the change.

#### 6.4.3.1. Factored State Representations

The starting point is to stop treating the state as an atomic vector  $s = \mathbf{x}$  and instead represent it as a collection of factors,

$$s = (x_1, x_2, \dots, x_n), \quad (6.98)$$

where each factor  $x_i$  corresponds to a distinct, interpretable aspect of the environment (for instance, an agent's position, a resource level, or an opponent's status). The premise behind factorization is that real-world non-stationarity is rarely global: a change in one factor, such as a shift in wind speed, does not usually imply that every other factor, such as the agent's own physical dynamics, has changed as well. A monolithic state representation cannot distinguish between these cases, so it forces the learner to re-adapt everything even when only one factor has drifted.

#### 6.4.3.2. Latent Factorization and Disentanglement

When the true factors of the environment are not directly observed, the same idea can be applied in latent space. Instead of modeling the state distribution  $p(s)$  directly, the learner models a factorized latent distribution

$$p(s) \approx p(z_1, z_2, \dots, z_n), \quad (6.99)$$

where the latent variables  $z_1, \dots, z_n$  are encouraged to be approximately independent, each capturing a different underlying source of variation. This is the disentanglement problem, and it has a well-developed toolkit in the representation learning literature. Three methods are particularly relevant here.

**$\beta$ -VAE** The  $\beta$ -VAE modifies the standard variational autoencoder objective by up-weighting the KL divergence term:

$$\mathcal{L}_{\beta\text{-VAE}} = \mathbb{E}_{q(z|x)} [\log p(x | z)] - \beta D_{\text{KL}}(q(z | x) \| p(z)). \quad (6.100)$$

Larger values of  $\beta$  push the approximate posterior  $q(z | x)$  closer to a factorized prior  $p(z)$ , which in practice encourages the individual dimensions of  $z$  to become more independent. The trade-off is reconstruction quality: excessive  $\beta$  can destroy information needed to reconstruct  $x$ , since the pressure toward independence competes directly with the reconstruction term in Equation (6.100).

**FactorVAE** FactorVAE targets independence more directly by penalizing the Total Correlation (TC) of the latent code, rather than relying on the KL term alone. Total correlation measures how far the joint distribution of the latents is from the product of its marginals:

$$TC(z) = D_{\text{KL}}\left(q(z) \| \prod_i q(z_i)\right). \quad (6.101)$$

A small  $TC(z)$  indicates that the latent dimensions are close to independent. FactorVAE’s objective adds this penalty directly to the standard evidence lower bound (ELBO):

$$\mathcal{L}_{\text{FactorVAE}} = \text{ELBO} - \gamma TC(z). \quad (6.102)$$

Because the TC penalty targets the aggregate posterior  $q(z)$  rather than the per-sample posterior  $q(z | x)$ , FactorVAE can push toward independence without paying as steep a reconstruction cost as  $\beta$ -VAE.

**$\beta$ -TCVAE**  $\beta$ -TCVAE makes the connection between the two previous methods explicit by decomposing the KL term in the standard VAE objective into three interpretable parts:

$$D_{\text{KL}}(q(z | x) \| p(z)) = \underbrace{I_q(x; z)}_{\text{mutual information}} + \underbrace{TC(z)}_{\text{total correlation}} + \underbrace{\sum_i D_{\text{KL}}(q(z_i) \| p(z_i))}_{\text{dimension-wise KL}}. \quad (6.103)$$

This decomposition shows that the plain  $\beta$ -VAE objective in Equation (6.100) penalizes all three terms equally, even though only the total correlation term is directly responsible for disentanglement. The mutual information term  $I_q(x; z)$  measures how much information the latent code carries about the input, and penalizing it too, as  $\beta$ -VAE implicitly does, unnecessarily degrades reconstruction.  $\beta$ -TCVAE instead reweights the three terms separately, applying the disentanglement pressure specifically to  $TC(z)$  while leaving mutual information largely untouched.



### 6.4.3.3. Causal Adaptation

Factorization on its own only says that the environment can be decomposed; it does not say which factor is responsible when performance drops. Causal adaptation adds this missing piece by asking a more specific question: of all the latent factors, which one actually caused the change in reward or transition behavior? Answering this correctly means the learner can update only the model of the affected factor and reuse everything else unchanged, rather than re-adapting the full model as change-point and sliding window methods do.

The procedure has five steps.

1. **Observe a performance drop.** The learner detects that its accumulated reward or value estimates have degraded, using the same kind of signal that triggers change-point detection in Section 6.4.1.
2. **Estimate which latent factor changed.** Using the factorized latent representation from Section 6.4.3.2, the learner compares the latent encoding of new observations against the latent encoding under the old model, for example via

$$\Delta_i = \|z_{\text{old},i} - z_{\text{new},i}\|, \quad (6.104)$$

computed separately for each factor  $i$ . A large  $\Delta_i$  signals that factor  $i$  is a candidate source of the change.

3. **Use a causal graph to identify affected variables.** Because factors can influence one another, a raw latent shift in one factor may simply be a downstream effect of a change elsewhere. The learner uses a causal graph over the factors to trace shifts back to their root cause, rather than treating every factor with an elevated  $\Delta_i$  as independently responsible.
4. **Update only the corresponding model.** Once the causally responsible factor is identified, the learner re-estimates the sub-model associated with that factor only, whether that is a transition component, a reward component, or both.
5. **Reuse everything else.** All sub-models associated with factors that were not implicated by the causal graph are left untouched, since re-adapting them would waste samples on parts of the environment that have not actually changed.

This procedure is what distinguishes causal adaptation from the coarser signal used by sliding window and change-point methods. Where SW-UCRL2 (Section 6.4.2.1) and its variants respond to any detected non-stationarity by discarding or re-weighting

all past data uniformly, causal adaptation attempts to localize the change and confine the cost of re-learning to the factor actually responsible for it. The trade-off is that this requires two additional assumptions that sliding window methods do not: a factorized representation of the state that is accurate enough to be meaningful, and a causal graph over the factors that is either known in advance or estimated reliably online. When these assumptions hold, causal adaptation can in principle recover from a change using far less data than a method that re-learns the entire model; when they do not hold, misattributing the change to the wrong factor can be worse than not localizing it at all.

#### 6.4.4. Factored and Hierarchical Adaptation

The causal adaptation methods in Section 6.4.3.3 localize change along the state’s factors: which component of the environment changed. Hierarchical adaptation localizes change along a different axis: at which timescale it operates. The idea is to split the learner into two levels. A high-level module reasons about slow, long-term shifts in the environment and decides when and how the agent’s behavior should change in response. A low-level module handles the fast, moment-to-moment problem of acting efficiently given the agent’s current understanding of the situation. Separating these two concerns means the low-level module does not need to relearn from scratch every time the environment shifts, and the high-level module does not need to reason about every low-level action in order to detect that a shift has occurred.

##### 6.4.4.1. Hierarchical Policies

Formally, the overall policy is expressed as a mixture over a latent variable  $\mathbf{z}$ , which stands in for the high-level module’s belief about the current situation:

$$\pi(a \mid s) = \sum_{\mathbf{z}} \pi(a \mid s, \mathbf{z}) P(\mathbf{z} \mid s). \quad (6.105)$$

This decomposes naturally into two policies operating at two different levels of abstraction. The high-level policy  $\pi_H(g \mid s)$  observes the state and produces a goal  $g$ , which represents a decision about what kind of behavior is currently appropriate rather than a specific action. The low-level policy  $\pi_L(a \mid s, g)$  then takes both the state and the goal and produces the primitive action actually executed in the environment:

$$\pi_H(g \mid s), \quad \pi_L(a \mid s, g). \quad (6.106)$$

In the non-stationary setting, the goal  $g$  (or, in the more general form of Equation (6.105), the latent  $\mathbf{z}$ ) plays the same role that the change point played in

Section 6.4.1 and that the window played in Section 6.4.2.1: it is the signal that tells the rest of the system that something about the environment has changed and that behavior should adjust. The difference is that here this signal is produced by a dedicated policy rather than a statistical detector or a fixed data-retention rule, and it can be updated on a slower timescale than the low-level policy that consumes it.

Three adaptation strategies build on this two-level structure, each addressing a different failure mode of naive hierarchical learning under non-stationarity.

### 6.4.4.2. Hierarchical Adaptation with Selective Learning Activation

The most direct failure mode of a two-level system is wasted computation: if both levels update continuously, the low-level policy spends effort re-learning behavior that has not actually become suboptimal, and the high-level policy risks reacting to noise rather than genuine change. Selective learning activation addresses this by using the high-level module purely as a gate. Rather than producing a goal that reshapes low-level behavior at every step, the high-level module monitors a signal from the low-level policy, such as its recent temporal-difference error or return, and uses it to decide whether the low-level policy should currently be learning at all. When the low-level policy's performance is stable, learning is switched off and the current policy is simply executed. When the signal indicates that performance has degraded, indicating a possible change in the environment, learning is switched back on until performance stabilizes again. This keeps the low-level policy's updates concentrated exactly on the periods where they are needed, and avoids the instability that comes from continuously updating a policy against a moving target, which is a known source of difficulty in hierarchical reinforcement learning even in the absence of external non-stationarity.

### 6.4.4.3. Decoupled Adaptation: The STAR Framework

A second failure mode is coupling: if the high-level and low-level modules are trained jointly, a change that only affects one of them can still destabilize both, since each treats the other as part of its environment. Decoupled adaptation addresses this by training the state representation used by the high-level module separately from the task-execution policy used by the low-level module, so that a change detected at one level does not automatically propagate into unnecessary updates at the other. Under this framework, the high-level module's job is restricted to maintaining an up-to-date representation of the current situation, while the low-level module's job is restricted to acting well given whatever representation it is handed. Because the two are decoupled, the high-level representation can be updated in response to a

detected shift without forcing every low-level skill to be relearned, and conversely the low-level policy can continue executing a skill unchanged as long as the high-level representation says the situation calling for that skill has not changed.

#### 6.4.4.4. Lifelong Non-Stationary Adaptation with Latent Representations

The first two strategies both assume the agent is dealing with one environment that occasionally shifts. A lifelong learning setting is harder: the agent faces an open-ended sequence of environments over its lifetime, and needs to both adapt to the current one and retain what it learned about earlier ones, since a previously seen situation may recur. This strategy extends the hierarchical structure with a persistent latent representation  $z$  that acts as a memory of the situations the agent has encountered so far, rather than a single belief about the present moment. As the agent moves through its lifetime, new observations update this latent representation, and the high-level policy  $\pi_H(g \mid s, z)$  conditions its goal selection on it. When the agent re-encounters a previously seen situation, the corresponding region of latent space is already populated, allowing the low-level policy to recover its earlier behavior quickly rather than relearning it, which is the central goal shared with the causal adaptation approach in Section 6.4.3.3: minimize how much must be relearned when only part of the problem has actually changed. The distinguishing feature here is that the "part" being tracked is a point in a continuous, learned latent space rather than a discrete factor of the state, which makes this approach more naturally suited to environments that drift smoothly or cycle through recurring regimes rather than jumping between discrete factor values.

---

**Algorithm 23** Bayesian Online Change-Point Detection (BOCPD)

---

**Require:** Hazard function  $H(\cdot)$

**Require:** Prior hyperparameters  $\nu_{\text{prior}}, \chi_{\text{prior}}$

```

1: Initialize run-length posterior:
2:  $P(\ell_0 = 0) \leftarrow 1$ 
3: Initialize sufficient statistics:
4:  $\nu^{(0)} \leftarrow \nu_{\text{prior}}$ 
5:  $\chi^{(0)} \leftarrow \chi_{\text{prior}}$ 
6: for  $t = 1, 2, \dots$  do
7:   Observe new sample  $o_t$ 
8:   Initialize change-point probability:
9:    $P(\ell_t = 0, o_{1:t}) \leftarrow 0$ 
10:  for all possible run lengths  $\ell_{t-1}$  do
11:    Compute predictive likelihood
12:     $\pi_t^{(\ell)} \leftarrow P(o_t \mid \nu^{(\ell)}, \chi^{(\ell)})$ 
13:    Compute growth probability
14:     $P(\ell_t = \ell_{t-1} + 1, o_{1:t}) \leftarrow P(\ell_{t-1}, o_{1:t-1})\pi_t^{(\ell)}(1 - H(\ell_{t-1}))$ 
15:    Accumulate change-point probability
16:     $P(\ell_t = 0, o_{1:t}) \leftarrow P(\ell_t = 0, o_{1:t-1}) + P(\ell_{t-1}, o_{1:t-1})\pi_t^{(\ell)}H(\ell_{t-1})$ 
17:  end for
18:  Compute evidence
19:   $P(o_{1:t}) \leftarrow \sum_{\ell_t} P(\ell_t, o_{1:t})$ 
20:  Update posterior run-length distribution
21:   $P(\ell_t \mid o_{1:t}) \leftarrow \frac{P(\ell_t, o_{1:t})}{P(o_{1:t})}$ 
22:  Reset sufficient statistics for a new regime
23:   $\nu^{(0)} \leftarrow \nu_{\text{prior}}$ 
24:   $\chi^{(0)} \leftarrow \chi_{\text{prior}}$ 
25:  for all possible run lengths  $\ell_t$  do
26:    Update sufficient statistics
27:     $\nu^{(\ell+1)} \leftarrow \nu^{(\ell)} + 1$ 
28:     $\chi^{(\ell+1)} \leftarrow \chi^{(\ell)} + u(o_t)$ 
29:  end for
30:  Predict next observation
31:   $P(o_{t+1} \mid o_{1:t}) \leftarrow \sum_{\ell_t} P(o_{t+1} \mid \ell_t, o_{1:t})P(\ell_t \mid o_{1:t})$ 
32: end for

```

---

---

**Algorithm 24** UCRL2 Algorithm [48]

---

**Require:** Confidence parameter  $\delta \in (0, 1)$

- 1: Initialize time step  $t \leftarrow 1$
- 2: Observe the initial state  $s_1$
- 3: **for** episode  $k = 1, 2, \dots$  **do**
- 4:   Set the episode start time  $t_k \leftarrow t$
- 5:   **for all**  $(s, a) \in \mathcal{S} \times \mathcal{A}$  **do**
- 6:      $v_k(s, a) \leftarrow 0$
- 7:     Compute visitation counts  $N_k(s, a)$
- 8:   **end for**
- 9:   Compute empirical reward estimates  $\hat{r}_k(s, a)$
- 10:   Compute empirical transition estimates  $\hat{p}_k(s'|s, a)$
- 11:   Construct the confidence set  $\mathcal{M}_k$
- 12:   Compute the optimistic MDP

$$\tilde{M}_k = \arg \max_{M \in \mathcal{M}_k} \rho^*(M)$$

- 13:   Compute the optimistic policy  $\tilde{\pi}_k$  using Extended Value Iteration
- 14:   **while**  $v_k(s_t, \tilde{\pi}_k(s_t)) < \max\{1, N_k(s_t, \tilde{\pi}_k(s_t))\}$  **do**
- 15:     Execute action
- 16:      $a_t \leftarrow \tilde{\pi}_k(s_t)$
- 17:     Observe reward  $r_t$  and next state  $s_{t+1}$
- 18:     Update visitation count

$$v_k(s_t, a_t) \leftarrow v_k(s_t, a_t) + 1$$

- 19:   **end while**
  - 20: **end for**
-

---

**Algorithm 25** Restarted Bayesian Online Change-Point Detection

---

**Require:** Initial state  $s_0$

- 1: Initialize BOCPD (Algorithm 23)
  - 2: Initialize UCRL2 (Algorithm 24)
  - 3: Initialize dataset  $\mathcal{D} \leftarrow \emptyset$
  - 4: **while** learning continues **do**
  - 5:     Observe current state  $s_t$
  - 6:     Select action  $a_t$  using the current UCRL2 policy
  - 7:     Execute  $a_t$  and observe reward  $r_t$  and next state  $s_{t+1}$
  - 8:     Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $\mathcal{D}$
  - 9:     Update BOCPD using the observed transition
  - 10:    **if** BOCPD detects a change point **then**
  - 11:       Estimate the change point  $\hat{\tau}$
  - 12:       Discard observations collected before  $\hat{\tau}$
  - 13:       Reinitialize UCRL2 statistics
  - 14:       Reconstruct empirical transition and reward models
  - 15:       Recompute confidence sets
  - 16:       Compute a new optimistic policy using Extended Value Iteration
  - 17:    **else**
  - 18:       Update empirical transition probabilities
  - 19:       Update empirical rewards
  - 20:       Update visitation counts
  - 21:    **end if**
  - 22: **end while**
-

---

**Algorithm 26** Reinforcement Learning with Context Detection (RL-CD)
 

---

**Require:** Initial state  $s_0$ , maximum memory size  $M$ , quality threshold  $E_{\min}$ , smoothing factor  $\rho$ , weighting factor  $\Omega$

```

1: Create a new context model  $m_{\text{cur}}$ 
2:  $\mathcal{M} \leftarrow \{m_{\text{cur}}\}$ 
3:  $s \leftarrow s_0$ 
4: while interaction continues do
5:   Select action  $a \sim \pi_{m_{\text{cur}}}(s)$ 
6:   Execute  $a$  and observe reward  $r$  and successor state  $s'$ 
7:   for all  $m \in \mathcal{M}$  do
8:     Compute reward prediction error
9:      $\Delta R_m \leftarrow \frac{r - R_m(s, a)}{N_m(s, a) + 1}$ 
10:    Compute reward prediction quality
11:     $e_m^R \leftarrow 1 - 2(Z_R(\Delta R_m)^2)$ 
12:    for all  $\kappa \in \mathcal{S}$  do
13:      if  $\kappa = s'$  then
14:         $\Delta T_m(\kappa) \leftarrow \frac{1 - T_m(s, a, \kappa)}{N_m(s, a) + 1}$ 
15:      else
16:         $\Delta T_m(\kappa) \leftarrow \frac{-T_m(s, a, \kappa)}{N_m(s, a) + 1}$ 
17:      end if
18:    end for
19:    Compute transition prediction quality
20:     $e_m^T \leftarrow 1 - 2(Z_T \sum_{\kappa \in \mathcal{S}} (\Delta T_m(\kappa))^2)$ 
21:    Compute confidence
22:     $c_m(s, a) \leftarrow \frac{N_m(s, a)}{M}$ 
23:    Compute instantaneous quality
24:     $e_m \leftarrow c_m(s, a) (\Omega e_m^R + (1 - \Omega) e_m^T)$ 
25:    Update quality trace
26:     $E_m \leftarrow E_m + \rho(e_m - E_m)$ 
27:  end for
28:  Select the highest-quality context
29:   $m_{\text{cur}} \leftarrow \arg \max_{m \in \mathcal{M}} E_m$ 
30:  if  $E_{m_{\text{cur}}} < E_{\min}$  then
31:    Create a new context model  $m_{\text{new}}$ 
32:     $\mathcal{M} \leftarrow \mathcal{M} \cup \{m_{\text{new}}\}$ 
33:     $m_{\text{cur}} \leftarrow m_{\text{new}}$ 
34:  end if
35:  for all  $\kappa \in \mathcal{S}$  do
36:    if  $\kappa = s'$  then
37:       $\Delta T_{m_{\text{cur}}}(\kappa) \leftarrow \frac{1 - T_{m_{\text{cur}}}(s, a, \kappa)}{N_{m_{\text{cur}}}(s, a) + 1}$ 
38:    else
39:       $\Delta T_{m_{\text{cur}}}(\kappa) \leftarrow \frac{-T_{m_{\text{cur}}}(s, a, \kappa)}{N_{m_{\text{cur}}}(s, a) + 1}$ 
40:    end if
41:     $T_{m_{\text{cur}}}(s, a, \kappa) \leftarrow T_{m_{\text{cur}}}(s, a, \kappa) + \Delta T_{m_{\text{cur}}}(\kappa)$ 
42:  end for
43:   $\Delta R_{m_{\text{cur}}} \leftarrow \frac{r - R_{m_{\text{cur}}}(s, a)}{N_{m_{\text{cur}}}(s, a) + 1}$ 
    
```



---

**Algorithm 27** Sliding-Window UCRL2 (SW-UCRL2)
 

---

**Require:** Confidence parameter  $\delta \in (0, 1)$

**Require:** State space  $\mathcal{S}$ , action space  $\mathcal{A}$

**Require:** Window size  $W$

```

1: Initialize  $t \leftarrow 1$ 
2: Observe initial state  $s_1$ 
3: for episode  $k = 1, 2, \dots$  do
4:    $t_k \leftarrow t$ 
5:   for all  $(s, a) \in \mathcal{S} \times \mathcal{A}$  do
6:      $v_k(s, a) \leftarrow 0$ 
7:      $N_k(s, a) \leftarrow \#\{t_k - W \leq \tau < t_k : s_\tau = s, a_\tau = a\}$ 
8:   end for
9:   for all  $s, s' \in \mathcal{S}, a \in \mathcal{A}$  do
10:     $R_k(s, a) \leftarrow \sum_{\tau=t_k-W}^{t_k-1} r_\tau \mathbf{1}_{s_\tau=s, a_\tau=a}$ 
11:     $P_k(s, a, s') \leftarrow \#\{t_k - W \leq \tau < t_k : s_\tau = s, a_\tau = a, s_{\tau+1} = s'\}$ 
12:   end for
13:   Compute estimates:
14:      $\hat{r}_k(s, a) \leftarrow \frac{R_k(s, a)}{\max\{1, N_k(s, a)\}}$ 
15:      $\hat{p}_k(s' | s, a) \leftarrow \frac{P_k(s, a, s')}{\max\{1, N_k(s, a)\}}$ 
16:   Construct confidence set  $\mathcal{M}_k$  such that
17:      $|\tilde{r}(s, a) - \hat{r}_k(s, a)| \leq \sqrt{\frac{7 \log(2SA t_k / \delta)}{2 \max\{1, N_k(s, a)\}}}$ 
18:      $\|\tilde{p}(\cdot | s, a) - \hat{p}_k(\cdot | s, a)\|_1 \leq \sqrt{\frac{14S \log(2At_k / \delta)}{\max\{1, N_k(s, a)\}}}$ 
19:   Compute optimistic policy  $\tilde{\pi}_k$  and optimistic MDP  $\tilde{M}_k \in \mathcal{M}_k$  via extended
   value iteration such that
20:      $\rho^* \leftarrow \min_s \rho(\tilde{M}_k, \tilde{\pi}_k, s) \geq \max_{M' \in \mathcal{M}_k, \pi, s'} \rho(M', \pi, s') - \frac{1}{\sqrt{t_k}}$ 
21:   while  $v_k(s_t, \tilde{\pi}_k(s_t)) < \max\{1, N_k(s_t, \tilde{\pi}_k(s_t))\}$  do
22:      $a_t \leftarrow \tilde{\pi}_k(s_t)$ 
23:     Observe reward  $r_t$  and next state  $s_{t+1}$ 
24:      $v_k(s_t, a_t) \leftarrow v_k(s_t, a_t) + 1$ 
25:      $t \leftarrow t + 1$ 
26:   end while
27: end for

```

---

---

**Algorithm 28** Sliding-Window UCRL2 with Confidence Widening (SWUCRL2-CW)
 

---

**Require:** Confidence parameter  $\delta \in (0, 1)$

**Require:** State space  $\mathcal{S}$ , action space  $\mathcal{A}$

**Require:** Window size  $W$ , confidence widening parameter  $\eta \geq 0$

```

1: Initialize  $t \leftarrow 1$ , episode index  $m \leftarrow 1$ ,  $\tau(1) \leftarrow 1$ 
2: Observe initial state  $s_1$ 
3: for episode  $m = 1, 2, \dots$  do
4:   for all  $(s, a) \in \mathcal{S} \times \mathcal{A}$  do
5:      $N_t(s, a) \leftarrow \#\{(\tau(m) - W) \vee 1 \leq q < \tau(m) : s_q = s, a_q = a\}$ 
6:      $N_t^+(s, a) \leftarrow \max\{1, N_t(s, a)\}$ 
7:      $v_m(s, a) \leftarrow 0$ 
8:   end for
9:   for all  $s, s' \in \mathcal{S}, a \in \mathcal{A}$  do
10:     $\hat{r}_t(s, a) \leftarrow \frac{1}{N_t^+(s, a)} \sum_{q=(\tau(m)-W) \vee 1}^{t-1} R_q(s, a) \mathbf{1}(s_q = s, a_q = a)$ 
11:     $\hat{p}_t(s' | s, a) \leftarrow \frac{1}{N_t^+(s, a)} \sum_{q=(\tau(m)-W) \vee 1}^{t-1} \mathbf{1}(s_q = s, a_q = a, s_{q+1} = s')$ 
12:   end for
13:   Construct reward confidence region:
14:    $H_{r,t}(s, a) \leftarrow \{\dot{r} \in [0, 1] : |\dot{r} - \hat{r}_t(s, a)| \leq \text{rad}_{r,t}(s, a)\}, \quad \text{rad}_{r,t}(s, a) =$ 

$$2\sqrt{\frac{2 \log(\text{SAT}/\delta)}{N_t^+(s, a)}}$$

15:   Construct widened transition confidence region:
16:    $H_{p,t}(s, a)(\eta) \leftarrow \{\dot{p}(\cdot | s, a) \in \Delta_{\mathcal{S}} : \|\dot{p}(\cdot | s, a) - \hat{p}_t(\cdot | s, a)\|_1 \leq \text{rad}_{p,t}(s, a) + \eta\}$ 
17:   Run extended value iteration on  $H_{r,t}, H_{p,t}(\eta)$  to obtain optimistic policy  $\tilde{\pi}_m$ 
   and optimistic MDP  $\tilde{M}_m$ 
18:   repeat
19:      $a_t \leftarrow \tilde{\pi}_m(s_t)$ 
20:     Observe reward  $R_t(s_t, a_t)$  and next state  $s_{t+1}$ 
21:      $v_m(s_t, a_t) \leftarrow v_m(s_t, a_t) + 1$ 
22:      $t \leftarrow t + 1$ 
23:   until  $t$  is a multiple of  $W$ , or  $v_m(s_t, a_t) \geq N_t(s_t, a_t)$  for the just-visited pair
24:    $\tau(m+1) \leftarrow t$ 
25: end for
    
```

---

---

## 7. Meta-Reinforcement Learning as an Adaptation Mechanism

The previous chapter treated adaptation as a property added to an otherwise ordinary reinforcement learning loop: detect a change, then re-estimate; or keep a sliding window, then re-estimate. Meta-reinforcement learning takes a different route to the same goal. Instead of adapting an explicit model of the environment online, it learns the adaptation procedure itself, offline, across a distribution of related tasks, so that a previously unseen task, or a previously unseen drift in the current one, can be handled with very little new data at deployment time. This chapter develops one curated exemplar per family, matching the depth given to robust MARL and adaptive MARL in Chapters 5 and 6 and the exemplars Chapter 8 compares directly: MAML for the policy-parameter-gradient family,  $RL^2$  for the black-box family, and VariBAD and PEARL for the task-inference family. Each family’s own literature is considerably larger than one exemplar; Appendix A develops the rest of it in full, including the black-box family’s attention-based, external-memory, and weight-space (Hebbian and self-referential) adaptation mechanisms, and the task-inference family’s discriminative, direct-regression, and exchangeable-process representations, so that breadth is preserved without letting any one adaptation mechanism dominate this chapter’s comparison with Chapters 5 and 6. Only the tabular game-theoretic machinery and single-agent algorithmic detail already covered in Chapters 3 and 4 are treated as known rather than repeated here.

Meta-reinforcement learning (Meta-RL) extends reinforcement learning by enabling an agent to adapt quickly to new tasks instead of learning every task from scratch. Traditional reinforcement learning optimizes a policy for a single task and often requires a large number of interactions whenever the environment changes. While this approach performs well in stationary environments, it becomes inefficient when the dynamics, reward function, or task objective vary over time. Meta-RL addresses this limitation by learning an adaptation strategy from a distribution of related tasks. During meta-training, the agent acquires prior knowledge that allows it to solve previously unseen tasks using only a small amount of new experience.

This chapter introduces the fundamental concepts of Meta-RL and develops the theoretical framework used throughout the remainder of this thesis. It begins by motivating the idea of learning to learn and extends the standard reinforcement learning formulation from a single task to a distribution of tasks. The chapter then

presents the general Meta-RL framework, including the adaptation function, the inner and outer optimization loops, and the objective of maximizing the expected return after adaptation. Since the underlying task is unknown during deployment, the chapter also explains how Meta-RL can be formulated as a partially observable decision-making problem in which the agent must infer the hidden task while simultaneously learning an effective control policy.

Next, the chapter introduces the main adaptation mechanisms proposed in the literature. These include gradient-based methods that adapt policy parameters through optimization, memory-based methods that store and exploit previous experience, latent task inference methods that estimate a hidden task representation, and learned optimization approaches that directly learn the adaptation process. The chapter also discusses the role of exploration, highlighting that exploration in Meta-RL aims not only to maximize reward but also to gather information that improves future adaptation.

To provide a unified view of existing methods, the chapter presents several complementary taxonomies. Meta-RL algorithms are categorized according to the amount of adaptation data available, such as few-shot and many-shot learning, the use of environment models in model-free and model-based approaches, and the adaptation mechanism employed through explicit or implicit adaptation. Additional comparisons are made between Meta-RL and related paradigms, including multitask reinforcement learning and meta-supervised learning, to clarify their objectives and assumptions.

Finally, the chapter extends the Meta-RL framework to the multi-agent setting. It introduces the Centralized Training with Decentralized Execution (CTDE) paradigm and explains how meta-learning can be incorporated into this framework to enable multiple agents to adapt efficiently while coordinating with one another. This discussion establishes the conceptual foundation for the meta-multi-agent reinforcement learning methods presented in the following chapters.

## 7.1. Learning to Learn

The objective of reinforcement learning is to learn a policy that maximizes the expected return for a single task. Meta-reinforcement learning changes this objective. Instead of learning only a policy, it learns an adaptation mechanism that enables the policy to improve rapidly when facing a new task. The agent acquires this ability by training over a distribution of related tasks rather than a single environment.

The central idea is often summarized as *learning to learn*. During meta-training, the agent extracts knowledge that is common across tasks. This knowledge serves as a

prior that guides future learning. When a previously unseen task is encountered, the agent uses only a small amount of interaction data to adapt its behavior instead of learning from scratch, so that what is learned is no longer task-specific but a general learning strategy reusable across related tasks. Section 7.2.1 below states this distinction formally, alongside the task distribution, the inner- and outer-loop optimization it induces, and the resulting meta-objective.

## 7.2. Problem Formulation

This section extends the reinforcement learning formulation presented in Chapter 3.1. Instead of optimizing a policy for a single Markov Decision Process (MDP), Meta-RL considers a distribution of related tasks and aims to learn an adaptation strategy that can rapidly specialize to a previously unseen task. Unless otherwise stated, the notation introduced in the reinforcement learning chapter is used throughout this chapter.

### 7.2.1. From Reinforcement Learning to Meta-Reinforcement Learning

Conventional reinforcement learning optimizes a policy for a single task. During training, the agent repeatedly interacts with one environment and updates its policy to maximize the expected return. Once training is complete, the learned policy is specialized to that particular task.

Meta-reinforcement learning extends this setting by considering a distribution of related tasks rather than a single task. During meta-training, tasks are sampled from a task distribution,

$$\mathcal{T} \sim p(\mathcal{T}),$$

where each task corresponds to a different MDP. Depending on the application, tasks may differ in their transition dynamics, reward function, initial state distribution, goals, or other environment properties.

For each sampled task  $\mathcal{T}_i$ , the agent interacts with the environment and collects a task-specific dataset,

$$D_{\mathcal{T}_i} = \{\tau_1, \tau_2, \dots, \tau_K\},$$

where each trajectory  $\tau_k$  corresponds to one episode. During meta-training, the learning algorithm receives a collection of task-specific datasets,

$$\mathcal{D} = \{D_{\mathcal{T}_1}, D_{\mathcal{T}_2}, \dots, D_{\mathcal{T}_N}\},$$

instead of learning from a single dataset.

It is useful to distinguish between an *episode* and a *trial*. An episode denotes a single interaction with the environment, whereas a trial consists of multiple episodes collected from the same task. The first episodes are typically used for adaptation, while the remaining episodes evaluate the adapted policy. After completing a trial, a new task is sampled from the task distribution.

The key difference between reinforcement learning and Meta-RL lies in what is learned. Reinforcement learning directly learns a policy,

$$\pi(a|s),$$

whereas Meta-RL learns an adaptation function,

$$f_{\theta} : D_{\mathcal{T}} \rightarrow \phi,$$

which maps the interaction data collected from a task to the parameters of an adapted policy,

$$\phi = f_{\theta}(D_{\mathcal{T}}),$$

where  $\theta$  denotes the meta-parameters and  $\phi$  represents the task-specific parameters after adaptation. The adapted policy is then given by

$$\pi_{\phi}(a|s).$$

The optimization process consists of two nested learning loops. The *inner loop* performs task-specific adaptation by computing the adapted parameters  $\phi$  from the interaction data collected on the current task. The *outer loop* updates the meta-parameters  $\theta$  using the performance achieved after adaptation across multiple tasks. Consequently, the inner loop learns a task-specific policy, whereas the outer loop learns an adaptation strategy that generalizes across tasks.

Since each task represents a different optimization problem, every task has its own objective,

---

**Algorithm 29** General Meta-Reinforcement Learning Procedure

---

**Require:** Task distribution  $p(\mathcal{T})$

**Require:** Meta-parameters  $\theta$

```

1: while training do
2:   Sample task  $\mathcal{T} \sim p(\mathcal{T})$ 
3:   Collect interaction data  $D$ 
4:   Adapt  $\phi = f_\theta(D)$ 
5:   Evaluate adapted policy  $J_{\mathcal{T}}(\pi_\phi)$ 
6:   Update  $\theta$  using the meta-objective
7: end while
8: return  $\theta$ 

```

---

$$J_{\mathcal{T}_i}(\pi),$$

or equivalently, its own loss function,

$$\mathcal{L}_{\mathcal{T}_i}(\theta).$$

The goal of meta-training is therefore not to optimize the performance of a single task, but to learn meta-parameters that perform well over the entire task distribution. This leads to the following meta-objective,

$$\max_{\theta} \mathbb{E}_{\mathcal{T} \sim p(\mathcal{T})} [J_{\mathcal{T}}(\pi_{f_\theta(D_{\mathcal{T}})})].$$

Equivalently, the optimization can be written as a minimization problem,

$$\min_{\theta} \mathbb{E}_{\mathcal{T} \sim p(\mathcal{T})} [\mathcal{L}_{\mathcal{T}}(f_\theta(D_{\mathcal{T}}))].$$

Algorithm 29 summarizes the general Meta-RL framework. Most existing methods follow this procedure and mainly differ in how the adaptation function  $f_\theta$  is implemented and how the meta-parameters are updated.

### 7.3. Meta-Reinforcement Learning as a Partially Observable Decision Process

In conventional reinforcement learning, the environment is represented by a fixed Markov Decision Process (MDP), and the agent is trained to solve that specific task. During both training and evaluation, the transition dynamics and reward function remain unchanged. Consequently, the current state contains sufficient information for selecting an action according to the Markov property.

Meta-reinforcement learning considers a different setting. At the beginning of each trial, a task is sampled from a distribution,

$$\mathcal{T} \sim p(\mathcal{T}),$$

but the identity of the sampled task is not directly observed by the agent. Although the agent can observe states, actions, and rewards, it must infer the underlying task from its interaction history.

The interaction history up to time step  $t$  is given by

$$\tau_{0:t} = (s_0, a_0, r_0, \dots, s_t),$$

which contains information about the hidden task. As additional observations are collected, the agent gradually refines its estimate of the current task.

This uncertainty transforms the original MDP into a partially observable decision process. The complete environment state can be written as

$$x_t = (s_t, \mathcal{T}),$$

where  $s_t$  is the observable environment state and  $\mathcal{T}$  is the hidden task variable. Since the task cannot be observed directly, the agent maintains a belief distribution,

$$b_t(\mathcal{T}) = P(\mathcal{T} \mid \tau_{0:t}),$$

which represents the probability that the current interaction corresponds to each possible task.

The policy therefore depends not only on the current state but also on the inferred task,



$$\pi(a_t \mid s_t, b_t).$$

Consequently, Meta-RL consists of two coupled problems. The first is *task inference*, which estimates the hidden task from previous interactions. The second is *control*, which selects actions that maximize the expected return given the inferred task. Different Meta-RL algorithms mainly differ in how they perform task inference. For example, memory-based methods encode the interaction history using recurrent or external memory, while latent-variable approaches explicitly estimate a task representation. Gradient-based methods implicitly infer the task through the parameter adaptation process.

## 7.4. Few-Shot and Many-Shot Meta-Reinforcement Learning

The primary objective of Meta-RL is to enable an agent to adapt efficiently to new tasks. The amount of interaction data available during adaptation plays a central role in determining the learning setting. Depending on the adaptation budget, Meta-RL methods are generally categorized into few-shot and many-shot learning. The term *shot* refers to the number of exploration episodes collected from a new task before evaluating the adapted policy.

**Few-Shot Meta-Reinforcement Learning** Few-shot Meta-RL assumes that only a small number of episodes are available for adaptation, often ranging from one to a few episodes. Under this setting, the adaptation mechanism must extract as much information as possible from limited interaction data. Consequently, few-shot methods emphasize rapid adaptation and strong generalization rather than extensive optimization.

This setting is particularly suitable for applications where collecting additional experience is expensive or unsafe, such as robotics, autonomous driving, and real-world control systems. Representative approaches include gradient-based methods such as MAML and CAVIA, as well as latent task inference methods such as PEARL.

**Many-Shot Meta-Reinforcement Learning** Many-shot Meta-RL relaxes the restriction on the adaptation budget by allowing the agent to collect a larger number of episodes from a new task. Instead of adapting immediately, the agent gradually improves its performance over multiple interactions. This additional experience

allows the adaptation process to optimize more complex policies but reduces the emphasis on rapid generalization.

Many-shot methods are appropriate when interaction with the environment is relatively inexpensive or when long-term performance is more important than immediate adaptation. Representative methods include Meta-Gradient Reinforcement Learning, FRODO, and STAC.

Table 7.1 summarizes the main differences between the two settings.

**Table 7.1.** – Comparison between few-shot and many-shot Meta-RL.

| Aspect            | Few-Shot              | Many-Shot                |
|-------------------|-----------------------|--------------------------|
| Adaptation budget | One or a few episodes | Many episodes            |
| Primary objective | Fast adaptation       | Progressive optimization |
| Interaction cost  | High                  | Low                      |
| Sample efficiency | Critical              | Less restrictive         |
| Generalization    | Strong emphasis       | Moderate emphasis        |
| Learning strategy | Rapid specialization  | Continuous improvement   |

Few-shot and many-shot Meta-RL describe different adaptation settings rather than different classes of algorithms. Many Meta-RL methods can operate in either setting, depending on the number of episodes allocated for adaptation during evaluation.

## 7.5. Categories of Meta-Reinforcement Learning

Although Meta-RL algorithms share the same objective of enabling rapid adaptation to new tasks, they differ in how the adaptation function

$$f_{\theta} : D\tau \rightarrow \phi$$

is implemented. The adaptation function maps the interaction data collected from a task to an adapted policy or policy parameters. Existing methods can be broadly categorized into three families: policy parameter gradient methods, black-box methods, and task inference methods. Each family represents a different realization of the adaptation function.

Policy parameter gradient (PPG) methods explicitly parameterize the adaptation function as an optimization procedure. The interaction data collected from a new

task are used to compute one or more gradient updates that transform the meta-parameters into task-specific parameters. The adaptation function therefore takes the form

$$\phi = f_{\theta}(D_{\mathcal{T}}) = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}}(\theta; D_{\mathcal{T}}),$$

where  $\theta$  denotes the shared meta-parameters and  $\phi$  represents the adapted parameters for the current task.

Black-box methods learn the adaptation function implicitly without explicitly defining an optimization algorithm. Instead, the interaction history is processed by a recurrent network or an external memory module whose internal state evolves during interaction. In this case, the adaptation function can be expressed as

$$h_t = f_{\theta}(h_{t-1}, o_t, a_{t-1}, r_{t-1}),$$

where the hidden state  $h_t$  serves as the adapted representation used by the policy,

$$a_t \sim \pi_{\theta}(a_t | o_t, h_t).$$

Task inference methods formulate adaptation as a latent variable estimation problem. Rather than adapting the policy parameters directly, they infer a latent task representation from the interaction data and condition the policy on this representation. The adaptation function is therefore written as

$$z = f_{\theta}(D_{\mathcal{T}}),$$

where  $z$  is a latent embedding that summarizes the current task. The adapted policy becomes

$$a \sim \pi_{\theta}(a | s, z).$$

These three categories are not mutually exclusive, and recent methods often combine ideas from multiple families. Nevertheless, they provide a useful taxonomy for organizing the Meta-RL literature and highlight the different design choices used to achieve fast adaptation.

## 7.6. Policy Parameter Gradient Methods

Policy parameter gradient (PPG) methods formulate meta-learning as an optimization problem in the policy parameter space. Their central objective is to learn meta-parameters that enable a policy to adapt rapidly to a new task through gradient-based optimization. Given the interaction data collected from a task, the adaptation function computes task-specific policy parameters by performing one or more gradient updates,

$$\phi = f_{\theta}(D_{\mathcal{T}}) = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}}(\theta; D_{\mathcal{T}}),$$

where  $\theta$  denotes the shared meta-parameters and  $\phi$  represents the adapted policy parameters. PPG methods explicitly define the adaptation process as a sequence of optimization steps.

Most existing methods share the same optimization framework but differ in the design of the adaptation function. Early approaches focus on learning a good initialization of the policy parameters such that only a few gradient updates are required to solve a new task. Later work extends this idea by adapting only a subset of the policy parameters [53], learning task-specific context variables [54], learning the optimization hyperparameters [55], or improving the geometry of the parameter updates through learned curvature or preconditioning matrices [56], [57]. Bayesian approaches further generalize the initialization by learning a distribution over policy parameters instead of a single initialization [58], [59].

As the field has matured, research has expanded beyond the basic adaptation mechanism. One direction focuses on improving the optimization process itself by learning update rules or optimization hyperparameters. Another direction studies the statistical properties of the meta-gradient estimator, proposing methods to reduce variance, correct bias [60], or balance the bias-variance trade-off in few-shot learning [61], [62]. Other work addresses efficient exploration by encouraging the agent to collect informative trajectories before adaptation [63], [64] [65], while recent methods extend gradient-based Meta-RL to settings such as offline learning, continuous adaptation in non-stationary environments [66], cross-platform policy transfer [67], and meta-learned reward shaping [68]. A distinctive characteristic of policy parameter gradient methods is that the trajectories collected during adaptation are generated by a dynamically changing policy [69], [70]. After each inner-loop gradient update, the policy parameters are modified, causing subsequent trajectories to be sampled from a different policy. Consequently, the adaptation dataset depends on the sequence of policy updates rather than a fixed policy. This dependency couples the trajectory distribution with the meta-parameters, making meta-gradient estimation substantially more challenging than in conventional reinforcement learning. As a

result, recent research has focused on developing estimators that correctly account for the changing trajectory distribution while reducing bias, variance, and computational cost. Although these methods differ in their implementation, they all optimize the same objective: learning policy parameters that can be efficiently adapted to previously unseen tasks using only a small amount of interaction data. The remainder of this section develops MAML, the foundational instance of learning a good initialization, in full; the further directions this overview surveys, adapting only part of the network, learning the optimization process itself, correcting meta-gradient estimator bias, and learning the exploration behavior used to collect adaptation data, are developed in full in Appendix A.

### 7.6.1. Learning a Good Initialization

The earliest policy parameter gradient methods focus on learning a parameter initialization that enables rapid adaptation to new tasks. During meta-training, the initialization is optimized such that a small number of gradient updates is sufficient to achieve high performance on an unseen task. The key idea is that the meta-learned initialization should lie close to the task-specific optima across the task distribution, thereby reducing the amount of data and optimization required during adaptation.

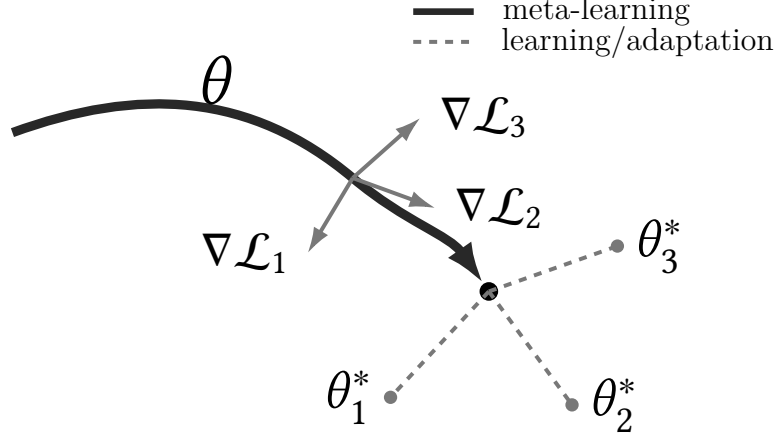
This idea forms the basis of several influential methods. MAML learns a shared initialization for all policy parameters, whereas later approaches improve its efficiency by adapting only selected parameters or low-dimensional task-specific variables. The following sections review these methods and discuss the trade-offs between adaptation speed, computational cost, and parameter efficiency.

#### 7.6.1.1. Model-Agnostic Meta-Learning (MAML)

Model-Agnostic Meta-Learning (MAML) is the first gradient-based meta-learning algorithm and serves as the foundation for many subsequent policy parameter gradient methods. Instead of learning a policy that performs well on a single task, MAML learns a shared parameter initialization that can be rapidly adapted to a new task using only a few gradient updates [71].

As illustrated in Figure 7.1, MAML separates learning into an inner loop and an outer loop. During the inner loop, the current policy is adapted to each sampled task using task-specific trajectories. During the outer loop, the initialization is updated according to the performance of the adapted policies across all sampled tasks.

**Figure 7.1.** – Overview of the MAML framework, which learns an initialization  $\theta$  for rapid adaptation to new tasks.[71]



This optimization encourages the learned initialization to lie close to the optimal parameters of every task in the task distribution.

For a sampled task  $\mathcal{T}_i$ , the policy parameters are adapted using one or more gradient updates,

$$\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(\theta),$$

where  $\alpha$  denotes the inner-loop learning rate. After adaptation, new trajectories are collected using the adapted policy and used to optimize the meta-objective. The meta-update is given by

$$\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_i \mathcal{L}_{\mathcal{T}_i}(\theta'_i),$$

where  $\beta$  is the meta learning rate. The complete meta-training procedure is summarized in Algorithm 30.

## 7.7. Black-Box Meta-Reinforcement Learning

As introduced in Section 5.5, black-box methods realize the adaptation function  $f_{\theta}$  implicitly. Rather than specifying an explicit optimization procedure, as in policy parameter gradient methods, or an explicit latent-variable inference model, as in

**Algorithm 30** MAML Meta-Training for Reinforcement Learning [71]**Require:** Task distribution  $p(\mathcal{T})$ , inner learning rate  $\alpha$ , outer learning rate  $\beta$ 


---

```

1: Initialize policy parameters  $\theta$ 
2: while not converged do
3:   Sample a batch of tasks  $\{\mathcal{T}_i\} \sim p(\mathcal{T})$ 
4:   for all tasks  $\mathcal{T}_i$  do
5:     Collect adaptation trajectories  $D_i \sim \pi_\theta$ 
6:     Adapt the policy  $\theta'_i = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(\theta)$ 
7:     Collect evaluation trajectories  $D'_i \sim \pi_{\theta'_i}$ 
8:   end for
9:   Compute the meta-objective  $J(\theta) = \sum_i \mathcal{L}_{\mathcal{T}_i}(\theta'_i)$ 
10:  Update the initialization  $\theta \leftarrow \theta - \beta \nabla_\theta J(\theta)$ 
11: end while
12: return Meta-learned initialization  $\theta^*$ 

```

---

task inference methods, black-box methods delegate the entire adaptation process to a single learned model. The interaction history collected from a task,

$$D_{\mathcal{T}} = (o_0, a_0, r_0, o_1, a_1, r_1, \dots, o_t, a_t, r_t),$$

is processed sequentially, and the model maintains an internal adaptation state that is updated at every timestep. In its most direct form, this state is the hidden state of a recurrent network,

$$h_t = f_\theta(h_{t-1}, o_t, a_{t-1}, r_{t-1}), \quad a_t \sim \pi_\theta(a_t \mid o_t, h_t).$$

No explicit gradient update, plasticity rule, or belief update is defined at this level of description. Adaptation is instead an emergent property of the forward pass: as the internal state evolves with incoming experience, the policy’s behavior changes accordingly, without any modification of  $\theta$  during the interaction itself. The parameters  $\theta$  are trained once, across a distribution of tasks  $p(\mathcal{T})$ , and are held fixed at test time. In this sense, the entire learning algorithm executed at test time is encoded implicitly inside the weights  $\theta$  that define  $f_\theta$ , rather than inside an explicit update rule applied during interaction.

This formulation places the burden of adaptation entirely on the expressiveness of  $f_\theta$  and on the capacity of its internal state to summarize everything relevant about the task from experience alone. This internal state must simultaneously encode an estimate of the task identity, a record of what has already been explored, and a representation of the agent’s current beliefs about the environment dynamics and reward

structure. Because these quantities are never separated into distinct variables, black-box methods are often described as fully amortized: the cost of task inference and policy optimization, performed explicitly and iteratively in other Meta-RL families, is absorbed once during meta-training into the weights of a single forward model.

The internal adaptation state need not take the form of a recurrent hidden vector. A separate line of work implements the same black-box principle in weight space, where the adaptation state is the network’s own weight matrix  $W_t$  rather than a hidden state  $h_t$ . In these methods,  $W_t$  is updated at every timestep by a local, differentiable rule that is itself learned during meta-training,

$$W_t = W_{t-1} + \Delta W_t, \quad a_t \sim \pi_\theta(a_t \mid o_t, W_t),$$

so that the network’s own parameters, rather than a separate recurrent state, carry the information accumulated during interaction. This is a direct generalization of the recurrent formulation above:  $W_t$  plays exactly the role that  $h_t$  plays in the recurrent case, and no gradient descent on  $\theta$  is performed during interaction in either case. What differs is only where the adaptation state lives and how it is written to.

The choice of architecture for  $f_\theta$ , and the choice of where its adaptation state lives, therefore largely define the subcategories of this family. Early approaches implement  $f_\theta$  as a recurrent neural network, most commonly an LSTM, whose hidden state plays the role of  $h_t$  directly. Later work augments or replaces the recurrent backbone with temporal convolutions, self-attention, or explicit external memory modules, motivated by the difficulty of encoding long task histories, exploration trajectories, and control information into a single fixed-size recurrent state. A further line of work equips the model with structured memory, separating short-term working memory from long-term episodic memory, in order to support recall of previously encountered tasks and more precise credit assignment. Finally, a distinct line of work moves the adaptation state out of the hidden state entirely and into the weights themselves, drawing on Hebbian and neuromodulated plasticity to define how  $W_t$  is written to at each timestep.

The remainder of this section presents RL<sup>2</sup>, the foundational formulation of Meta-RL as a black-box sequential prediction problem, in full. Two further developments extend the same principle along different mechanisms and are covered in full in Appendix A: attention-based and external-memory architectures that replace the recurrent backbone, and weight-space methods that move the adaptation state into  $W_t$  directly using Hebbian and neuromodulated plasticity. Training any of these formulations shares a common set of practical difficulties, a memory bottleneck in whatever single object carries the adaptation state, gradient or plasticity-rule instability over long horizons, and sensitivity to how well the test-time task distribution



matches the one seen during meta-training; Appendix A works through these difficulties against the specific architectures that motivate each one, and closes by contrasting this entire black-box family against MAML’s gradient-based alternative from Section 7.6 above.

**Why “black-box.”** The name reflects what is deliberately left unspecified. Nowhere in this formulation does the designer state how the agent should remember relevant information, how it should explore an unfamiliar task, how it should infer which task it is currently solving, or how any of this should translate into adaptation. All of it is learned end to end, purely from the requirement that the resulting behavior maximize reward across the task distribution. This is what separates black-box methods from the policy parameter gradient methods of Section 5.6, where the adaptation procedure itself, gradient descent, is fixed by the designer, and from the task inference methods of Section 5.9, where an explicit belief-update mechanism is specified in advance. In a recurrent black-box model, for instance, nothing in the model’s specification says that the hidden state should encode task identity, or the reward function, or the environment’s transition dynamics, or a record of what has already been explored, or the agent’s uncertainty about any of these. Empirically, a successfully trained model ends up encoding some combination of exactly this information, but this is a discovered consequence of training on the meta-RL objective, not a property anyone engineered into the architecture.

#### 7.7.0.1. History-Conditioned Policies as Implicit Reinforcement Learning Algorithms (RL<sup>2</sup> and L2RL)

##### **Two mechanisms of adaptation: conditioning versus parameter generation.**

Before presenting a specific method, it is useful to distinguish two fundamentally different ways a policy can depend on accumulated experience. The first is *adaptation via input*: the policy weights  $\theta$  remain fixed and shared across all tasks, and the policy instead receives an evolving memory state as an additional input,

$$\pi_{\theta}(a \mid s, h), \quad h_t \leftarrow f_{\theta}(h_{t-1}, o_t, a_{t-1}, r_{t-1}, d_{t-1}).$$

Here, task-specific behavior emerges purely because  $h$  changes as experience accumulates, while the function  $\pi_{\theta}$  itself never changes across tasks. The second mechanism is *adaptation via parameters*: a task embedding  $e$  is computed from experience, and this embedding is passed through a hypernetwork  $h_{\theta}$  that outputs an entire set of task-specific policy weights,

$$\pi_\phi(a \mid s), \quad \phi = h_\theta(e).$$

In this second case, the base policy network receives no memory as input at all. Instead, its own weights  $\phi$  differ from task to task, generated in a single forward pass by  $h_\theta$ . Table 7.2 summarizes the distinction. The recurrent methods presented in this subsection, RL<sup>2</sup> and L2RL, fall entirely into the first category: the hidden state  $h_t$  is memory that a fixed policy reads from, never a set of generated weights. The second category, adaptation via parameter generation, is taken up separately in Section 5.9.

**Table 7.2.** – Adaptation via memory versus adaptation via generated parameters.

|                           | Adaptation via Input<br>(RL <sup>2</sup> /L2RL) | Adaptation via Pa-<br>rameters (Hypernet-<br>work)     |
|---------------------------|-------------------------------------------------|--------------------------------------------------------|
| Policy form               | $\pi_\theta(a \mid s, h)$                       | $\pi_\phi(a \mid s), \phi = h_\theta(e)$               |
| What changes across tasks | Memory state $h$                                | Policy weights $\phi$                                  |
| Weight sharing            | Fully shared $\theta$                           | Shared hypernetwork $\theta$ ,<br>task-specific $\phi$ |
| Mechanism                 | Recurrent state update,<br>every timestep       | Single forward pass<br>through $h_\theta$              |

**Origins.** The adaptation-via-input mechanism was introduced independently and almost simultaneously by Duan et al. [72], under the name RL<sup>2</sup>, and by Wang et al. [73], under the name Learn to Reinforcement Learn (L2RL). Both methods realize the adaptation function  $f_\theta$  of Section 5.7 as the hidden state of a recurrent network, and both frame meta-learning as training a single recurrent policy to act as an implicit reinforcement learning algorithm. They are presented here as a single unified method, since they share an identical adaptation mechanism and differ only in the outer-loop procedure used to train  $\theta$ , a detail that is incidental to the mechanism itself and addressed briefly at the end of this subsection.

**Problem setup.** Consider a distribution over Markov decision processes  $p(\mathcal{M})$ . Rather than learning a policy for a single MDP  $\mathcal{M}$ , the goal is to learn a policy that performs well, on average, across the entire distribution  $p(\mathcal{M})$ , after interacting with each sampled MDP for a limited number of episodes. At every timestep the agent receives a transition tuple

$$(o_t, a_{t-1}, r_{t-1}, d_{t-1}),$$

where  $o_t$  is the current observation,  $a_{t-1}$  and  $r_{t-1}$  are the previous action and reward, and  $d_{t-1}$  is a termination flag indicating whether the previous episode has ended. Crucially, the identity of the underlying task  $\mathcal{M}$  is never observed directly. It can only be inferred from the sequence of past transitions. The full history available to the agent at time  $t$  is therefore

$$H_t = \{x_0, a_0, r_0, \dots, x_{t-1}, a_{t-1}, r_{t-1}, x_t\},$$

and the policy is conditioned on this entire history rather than on the current observation alone,

$$\pi_\theta(a_t \mid H_t).$$

**Recurrent state as sufficient statistic.** Explicitly conditioning on the full history  $H_t$  is intractable, since  $H_t$  grows without bound. Both RL<sup>2</sup> and L2RL address this by compressing  $H_t$  into a fixed-size recurrent state  $h_t$ , updated at every timestep by an LSTM,

$$h_t \leftarrow f_\theta(h_{t-1}, o_t, a_{t-1}, r_{t-1}, d_{t-1}), \quad \pi_\theta(a_t \mid h_t).$$

The hidden state  $h_t$  is interpreted as an implicitly learned sufficient statistic of the episode history: rather than being hand-designed, as in a Bayesian filter or a belief-state estimator, the mapping from history to sufficient statistic is learned end to end, jointly with the policy itself. No component of the model is reset or updated by gradient descent during interaction with a task. Adaptation to a new task is entirely a consequence of  $h_t$  evolving as new transitions are observed, while  $\theta$ , and therefore  $\pi_\theta$  itself, remains fixed throughout the interaction. This is precisely the adaptation-via-input mechanism defined above: the policy never changes, only what it is fed changes.

**Two-level optimization.** Training proceeds as a two-level procedure. The inner loop is not a learning procedure in the conventional sense: it is simply the forward evaluation of  $f_\theta$  as the agent interacts with a sampled task, with the hidden state  $h_t$  preserved across all episodes belonging to that task. The outer loop treats the entire multi-episode interaction as a single trial and optimizes  $\theta$  to maximize the expected return obtained across this trial, averaged over the task distribution,

$$J(\theta) = \mathbb{E}_{\mathcal{M} \sim p(\mathcal{M})} [R(\mathcal{M})].$$

RL<sup>2</sup> optimizes this objective with Trust Region Policy Optimization,

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[ \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | h_t) G_t \right], \quad \theta \leftarrow \text{TRPO}(\theta, \nabla_{\theta} J(\theta)),$$

while L2RL instead trains  $\theta$  with an advantage actor-critic objective augmented with entropy regularization,

$$\mathcal{L}(\theta) = \underbrace{\mathcal{L}_{\text{policy}}(\theta)}_{\text{actor}} + \underbrace{\beta_v \cdot \mathcal{L}_{\text{value}}(\theta)}_{\text{critic}} + \underbrace{\beta_e \cdot \mathcal{H}[\pi_{\theta}]}_{\text{entropy regularization}}.$$

This choice of outer-loop algorithm is incidental to the method and does not affect the underlying adaptation mechanism, which in both cases is the evolving hidden state  $h_t$  read by a fixed policy  $\pi_{\theta}$ . What matters structurally is not which RL algorithm sits in the outer loop, but that the inner loop performs no optimization at all: it is a pure forward pass through a recurrent state.

**Meta-training and meta-testing.** Algorithm 31 and Algorithm 32 summarize the resulting procedure, following the RL<sup>2</sup> formulation of Duan et al. [72]. During meta-training, a task is sampled from  $p(\mathcal{M})$ , the hidden state is reset once, and the agent interacts with the task for  $K$  episodes without resetting  $h_t$  between them, allowing information to accumulate across episode boundaries. The resulting trajectories are used to compute the outer-loop policy gradient and update  $\theta$ . During meta-testing,  $\theta$  is frozen entirely, and the agent is evaluated on a previously unseen task  $\mathcal{M}_{\text{test}}$ . Adaptation to  $\mathcal{M}_{\text{test}}$  takes place purely through the evolution of  $h_t$  across the  $K$  evaluation episodes, with no parameter updates of any kind.

---

**Algorithm 31**  $RL^2$  / L2RL Meta-Training

---

**Require:** Task distribution  $p(\mathcal{M})$ , recurrent policy  $\pi_\theta$ , number of meta-iterations  $N$ , episodes per task  $K$   
Initialize policy parameters  $\theta$   
**for**  $i = 1$  to  $N$  **do**  
    Sample a task:  $\mathcal{M} \sim p(\mathcal{M})$   
    Initialize hidden state:  $h_0 \leftarrow \mathbf{0}$   
    **for**  $k = 1$  to  $K$  **do**  
        Reset environment for task  $\mathcal{M}$   
        **while** episode not terminated **do**  
            Observe observation  $o_t$   
            Update hidden state:  $h_t \leftarrow f_\theta(h_{t-1}, o_t, a_{t-1}, r_{t-1}, d_{t-1})$   
            Sample action:  $a_t \sim \pi_\theta(a_t \mid h_t)$   
            Execute action  $a_t$   
            Receive  $(r_t, o_{t+1}, d_t)$   
            Store transition  $(o_t, a_t, r_t, o_{t+1}, d_t)$   
        **end while**  
    **end for**  
    Compute outer-loop objective  $J(\theta) = \mathbb{E}_{\mathcal{M} \sim p(\mathcal{M})}[R(\mathcal{M})]$   
    Compute policy gradient  $\nabla_\theta J(\theta) = \mathbb{E}[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid h_t) G_t]$   
    Update  $\theta$  using TRPO ( $RL^2$ ) or actor-critic with entropy regularization (L2RL)  
**end for****return**  $\pi_\theta$

---

---

**Algorithm 32**  $RL^2$  / L2RL Meta-Testing

---

**Require:** Trained recurrent policy  $\pi_\theta$ , unseen task  $\mathcal{M}_{\text{test}}$ , number of episodes  $K$   
Freeze policy parameters  $\theta$   
Sample an unseen task  $\mathcal{M}_{\text{test}} \sim p(\mathcal{M})$   
Initialize hidden state:  $h_0 \leftarrow \mathbf{0}$   
**for**  $k = 1$  to  $K$  **do**  
    Reset environment for task  $\mathcal{M}_{\text{test}}$   
    **while** episode not terminated **do**  
        Observe observation  $o_t$   
        Update hidden state:  $h_t \leftarrow f_\theta(h_{t-1}, o_t, a_{t-1}, r_{t-1}, d_{t-1})$   
        Select action:  $a_t \sim \pi_\theta(a_t \mid h_t)$   
        Execute action  $a_t$   
        Receive reward, next observation, and termination flag  $(r_t, o_{t+1}, d_t)$   
    **end while**  
**end for****return** Adapted policy behavior on  $\mathcal{M}_{\text{test}}$

---

**Advantages and limitations.** This formulation offers several practical advantages. No exploration strategy is hand-designed; instead, exploration behavior emerges directly from the meta-training objective. The resulting agent can adapt to previously unseen tasks without any weight updates at test time, training is fully end to end, and the framework is agnostic to the outer-loop RL algorithm used to train  $\theta$ . These advantages come with corresponding limitations. Because adaptation happens entirely through  $h_t$  as an input to a fixed policy, all information relevant to the task, including task identity, exploration history, and control state, must be compressed into that single vector, creating a memory bottleneck. Effective meta-learning typically requires a large number of training tasks before it emerges reliably. Performance at test time depends heavily on the similarity between the test task and the training task distribution, and learning long-range dependencies across multiple episodes remains difficult for recurrent architectures, since credit for an early exploratory action may only be realized many timesteps, or even an entire episode, later.

**Limitation motivating later sections.** The compression of task identity, exploration history, and control state into a single input vector  $h_t$  is the central weakness of the adaptation-via-input mechanism. Beck et al. [74] argue that this compression is avoidable, and propose replacing it with adaptation via parameters: rather than feeding memory into a fixed policy, a hypernetwork produces an entirely new policy for each task from a task embedding,  $\phi = h_\theta(e)$ . This alternative mechanism is developed fully in Section 5.9. Section 5.7.3, by contrast, keeps the adaptation-via-input mechanism but strengthens it with structured external memory in place of a single recurrent vector.

Section 5.7.1.2 extends the adaptation-via-input mechanism from the discrete, on-policy setting considered above to continuous control, via a deterministic and a stochastic actor-critic instantiation of the hidden-state-as-belief principle.

## 7.8. Task Inference Methods

As introduced in Section 5.5, task inference methods realize the adaptation function  $f_\theta$  by making the identity of the current task an explicit object of computation, rather than leaving it implicit inside a hidden state, as in Section 5.7, or inside a set of adapted parameters, as in Section 5.6. The underlying assumption is that a reinforcement learning task with unknown reward and dynamics can be modeled as a Partially Observable Markov Decision Process (POMDP) in which the unobserved variable is not a hidden state of the environment in the usual sense, but the identity of the task itself. Writing  $s$  for the observable state and  $w$  for this hidden task variable, the agent’s problem is to act well under a belief

$$b_t(\mathbf{w}) = p(\mathbf{w} \mid \tau_{0:t}),$$

maintained and updated as new experience  $\tau_{0:t}$  is observed. Unlike the recurrent hidden state of Section 5.7.1, which is trained end to end with no explicit target and no guarantee that it corresponds to anything resembling a belief,  $b_t(\mathbf{w})$  here is an explicit quantity that the model is trained to approximate directly, most often through some form of Bayesian or variational inference, though, as this section shows, not exclusively.

**Why an explicit belief.** Section 5.7.1.1 already previewed the central motivation for this family: in a purely black-box, recurrent formulation, the hidden state  $\mathbf{h}_t$  must simultaneously encode task identity, exploration history, and control-relevant state, with nothing in the architecture separating these roles from one another. Task inference methods respond to this entanglement directly, by dedicating a component of the model, typically an encoder  $q_\phi(\mathbf{z} \mid \tau)$  or  $q_\phi(\mathbf{w} \mid \tau)$ , specifically to the problem of estimating task identity, and by conditioning the policy on the resulting quantity rather than on an undifferentiated hidden state. This separation is not free: computing or approximating  $b_t(\mathbf{w})$  exactly is generally intractable, since it requires integrating over every possible task consistent with the observed history, and the methods in this section differ substantially in how they approximate this intractable quantity and what mechanism they use to do so.

**Two largely independent design questions.** Surveying this literature reveals that the choices made by any one method largely factor into two separate questions, addressed largely independently of one another. The first, and primary, question is mechanistic: given a batch of experience from the current task, how is a representation of task identity actually computed? VariBAD and PEARL below both posit an explicit generative model over trajectories conditioned on a latent task variable and fit an approximate posterior over that variable variationally, by maximizing an evidence lower bound; this is the most direct instantiation of  $b_t(\mathbf{w})$ , and the one this chapter develops in full. Appendix A covers three structurally different answers to the same question found elsewhere in this literature: shaping a representation directly through a discriminative or self-supervised objective instead of a generative model, regressing directly from experience to task-specific parameters or a value estimate rather than to any probabilistic belief, and grounding a representation in the formal statistical property of exchangeability instead of an assumed encoder architecture.

The second question is largely orthogonal to the first: given a task representation, however it is obtained, should the same policy both explore to gather informative

experience and exploit that experience to act well, or should these two objectives be optimized separately? VariBAD and PEARL both take the former approach, exploration emerges implicitly as a side effect of optimizing return, exactly as in Section 7.6’s black-box discussion. A separate line of work, covered in Appendix A, argues explicitly that this coupling creates a chicken-and-egg optimization problem and trains a dedicated exploration policy on an information-theoretic objective instead.

**Grounding the approximation.** Every mechanism surveyed in this section produces, at best, an approximate substitute for the true belief  $b_t(\mathbf{w})$ . This section opens with a theoretical framework, the Approximate Information State, that addresses a question logically prior to any specific architecture: what properties must an approximate representation of history satisfy for planning or control performed on top of it to remain close to optimal, and how does representation error propagate into control error. This framework does not prescribe any particular encoder architecture, and is not itself a meta-reinforcement learning method, but it provides the justification for treating any of the approximate mechanisms surveyed afterward, in this chapter and in Appendix A alike, as reasonable things to build in the first place.

**Organization.** The remainder of this section presents the Approximate Information State framework, followed by the variational Bayesian belief methods VariBAD, the foundational Bayes-Adaptive formulation of this family, and PEARL, its efficient off-policy counterpart. A closing section then examines how each of the three meta-RL families in this chapter extends to the multi-agent setting under centralized training with decentralized execution. The remaining task-inference methods, decoupled exploration, discriminative and self-supervised representations, direct parameter and value regression, exchangeable models, belief-representation validation, and a closing comparison across every task-inference method covered in this thesis, are developed in full in Appendix A.

### 7.8.1. Theoretical Foundations: Approximate Information State

Subramanian et al. [75] study reinforcement learning under partial observability from a different angle than any method surveyed so far in this chapter. Where Section 5.7 builds specific architectures, recurrent, attention-based, or memory-augmented, that implicitly compress history into a representation, and Sections 5.9.2 onward build specific encoders that explicitly estimate a task belief, this work asks a question



logically prior to any of these architectural choices: what properties must *any* compressed representation of history satisfy for planning or control performed on top of it to remain close to optimal? The resulting framework, the Approximate Information State (AIS), is not itself a meta-reinforcement learning method or a specific encoder architecture. It is a set of conditions against which any of the representations built elsewhere in this chapter, an RNN hidden state, a variational task posterior, a contrastive embedding, can be evaluated and justified.

**The belief state and why it is avoided.** The classical solution to partial observability maintains a belief over the true, unobserved state,  $b_t = P(s_t \mid h_t)$ , where  $h_t = (o_1, a_1, \dots, o_t)$  is the full history of observations and actions. This belief state is sufficient for optimal control, since it contains everything needed to act optimally going forward, but computing it exactly requires knowing the transition and observation models, performing exact Bayesian filtering, and representing a distribution over a continuous simplex, all of which make exact planning computationally intractable in most settings of interest. A large body of work, including the recurrent and memory-based architectures of Section 5.7 and the encoders introduced later in this section, sidesteps this intractability by learning some latent representation instead, an RNN hidden state, a Predictive State Representation, or a learned embedding of history. Almost none of these methods, however, explain why the resulting latent representation should be sufficient for control in the first place. This is the gap AIS is designed to fill.

**From exact to approximate information states.** Rather than asking whether the exact belief state can be computed, the authors ask a more modest question: what properties must a compressed representation of history satisfy to behave like a state, whether or not it equals the true belief exactly? Write  $H_t = (O_1, A_1, \dots, O_t)$  for the full history and suppose it can be replaced by a compression  $Z_t = \sigma(H_t)$  without losing the information relevant to control. If this is possible,  $Z_t$  is called an *information state*, and it is defined by two equivalent conditions. The first, more convenient for theoretical analysis since it works directly with latent-space transitions, requires that  $Z_t$  preserve everything needed to compute the expected immediate reward and to predict the distribution of the next information state,

$$\mathbb{E}[R_t \mid H_t, A_t] = \mathbb{E}[R_t \mid Z_t, A_t], \quad P(Z_{t+1} \mid H_t, A_t) = P(Z_{t+1} \mid Z_t, A_t).$$

The second, closer to practical implementation, since an agent naturally receives observations sequentially and updates an internal memory, requires reward prediction as above together with a recursive update rule,

$$Z_{t+1} = f(Z_t, A_t, O_{t+1}),$$

so that the agent no longer needs to retain the complete history, only the current compressed state and the incoming observation. The two definitions are mathematically equivalent under suitable assumptions, and the central theorem of this framework establishes that if  $Z_t$  satisfies either one, dynamic programming can be performed entirely within the compressed space  $Z$ , yielding a policy exactly as good as one computed on the full history.

**Approximate Information State.** An exact information state of this kind is, in general, no easier to construct than the exact belief state itself: any neural network encoder, whether an RNN, a Transformer, or a learned latent-state model, carries some inevitable approximation error, and the resulting latent state may not capture every property of history relevant to control. The paper’s main contribution is to show that this does not matter: if the approximation error is sufficiently small, the resulting policy remains nearly optimal. Formally, an approximate encoder  $\hat{\sigma}$  produces an approximate information state  $\hat{Z}_t = \hat{\sigma}(H_t)$ , and the two exact conditions above are relaxed to two approximate properties. Approximate Property 1 (AP1), reward prediction, requires

$$|\mathbb{E}[R_t \mid H_t, A_t] - \mathbb{E}[R_t \mid \hat{Z}_t, A_t]| \leq \epsilon_r,$$

and Approximate Property 2 (AP2), transition prediction, requires the distribution of the next approximate information state to be close to what the compressed dynamics predict,

$$P(\hat{Z}_{t+1} \mid H_t, A_t) \approx P(\hat{Z}_{t+1} \mid \hat{Z}_t, A_t).$$

**Measuring distributional closeness.** Formalizing AP2 requires a notion of distance between distributions. Rather than committing to a single fixed choice, such as KL divergence, Wasserstein distance, or total variation, the framework defines closeness through a general Integral Probability Metric (IPM),

$$d_F(\mu, \nu) = \sup_{f \in F} |\mathbb{E}_\mu[f] - \mathbb{E}_\nu[f]|,$$

where  $\mu$  is the true distribution,  $\nu$  is the approximating distribution, and  $F$  is a class of test functions. Many familiar distances arise as special cases of this construction for particular choices of  $F$ , summarized in Table 7.3.

**Table 7.3.** – Integral Probability Metrics arising from different function classes  $F$ .

| Function class              | Metric                         | Form                                                                                     | Advantage                                                                                              | Disadvantage                                   |
|-----------------------------|--------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Bounded functions           | Total Variation                | $f \in \{f : \ f\ _\infty \leq 1\}$                                                      | Very accurate                                                                                          | Expensive to compute repeatedly                |
| 1-Lipschitz functions       | Wasserstein                    | $f \in \{f : \ f\ _{\text{Lip}} \leq 1\}$                                                | Excellent theoretical properties                                                                       | Requires solving an optimization problem, slow |
| RKHS functions              | Maximum Mean Discrepancy (MMD) | $F = \{f \in \mathcal{H} : \ f\ _{\mathcal{H}} \leq 1\}$ , $\mathcal{H}$ a Hilbert space | Estimated directly from samples; no optimization, linear programming, or adversarial training required | —                                              |
| Bounded Lipschitz functions | Bounded-Lipschitz distance     | $f \in \{f : \ f\ _\infty + \ f\ _{\text{Lip}} \leq 1\}$                                 | —                                                                                                      | —                                              |

MMD is the practically preferred choice among these: unlike Wasserstein distance, it requires no adversarial training or optimization subroutine and can be estimated directly from samples, which is what makes it usable inside a standard reinforcement learning training loop rather than requiring a separately optimized critic or discriminator at every step. With this metric fixed, AP2 becomes  $d_F(\mu, \nu) \leq \epsilon_p$ , giving both approximate properties a single, precisely bounded form of error,  $\epsilon_r$  for reward and  $\epsilon_p$  for transition prediction.

**Contraction and error propagation.** The central theoretical result of this framework is that these two one-step error bounds are enough to bound the total degradation in control performance, with no dependence on quantities that might otherwise be expected to matter. Define the approximate Bellman backup operator on the latent space as  $\hat{B}V(\hat{z}) = \mathbb{E}[\hat{r} + \gamma V(\hat{z}')]$ , using the approximate reward and transition models in place of the true ones. Since the ordinary Bellman operator is a contraction,  $\|BV_1 - BV_2\| \leq \gamma \|V_1 - V_2\|$  for  $0 < \gamma < 1$ , the paper proves that the AIS Bellman operator remains a contraction under the same discounting assumption, so that value iteration performed entirely in the approximate latent space still converges to a unique fixed point. Because errors are propagated through a contracting operator, they do not grow exponentially with the planning horizon, as might be

feared; instead, the discount factor causes errors further in the future to be weighted less, and in the worst case the total planning error is bounded by

$$\text{Planning Error} \approx \frac{\epsilon_r + \epsilon_p}{1 - \gamma}.$$

This bound has no dependence on the length of the history being compressed, the dimensionality of the observation space, or the size of the neural network used to compute  $\hat{Z}_t$ : only the one-step reward and transition approximation errors, and the discount factor, appear. This is the theoretical property that ultimately justifies building learned, imperfect representations of history at all, throughout this chapter, provided that whatever architecture is used is trained to keep  $\epsilon_r$  and  $\epsilon_p$  small. For the infinite-horizon case, where backward induction is unavailable since there is no terminal time, the same guarantee is obtained by working with a time-homogeneous Bellman operator defined directly on the latent information state,

$$[B\bar{V}](z) = \max_{a \in A} \mathbb{E}[R_t + \gamma \bar{V}(Z_{t+1}) \mid Z_t = z, A_t = a].$$

**Learning an AIS.** A natural question is what training objective should be used to learn the encoder  $\hat{\sigma}$  that produces  $\hat{Z}_t$ . Most representation learning objectives used elsewhere in this thesis, reconstruction, contrastive prediction as in Section 5.7.3.2’s CPC objective, or general next-observation prediction, are not directly aligned with AP1 and AP2. The paper argues that the encoder should instead be trained explicitly to satisfy these two properties: the resulting latent state should support accurate reward prediction and accurate transition prediction in the compressed space, and nothing else is required of it. This motivates the *AIS Generator*, an architecture that learns not only the latent state  $Z_t$  but every component needed to verify AP1 and AP2 directly: a reward model and a transition model defined on the latent space, trained jointly with the encoder itself, rather than learning the latent state in isolation and hoping it happens to support downstream reward and transition prediction.

**Multi-timescale optimization.** Training proceeds using different learning rates for different components, rather than updating everything at a single shared rate. The AIS generator, encoder, reward model, and transition model, is updated fastest, the critic at a medium rate, and the actor slowest. The reasoning mirrors a chicken-and-egg concern seen elsewhere in this chapter: if the policy changes at every update, the latent representation can never catch up to a moving target, so the representation is learned first, followed by the critic, followed by the actor, each on a slower timescale

than the one before it. The authors draw an explicit analogy to the Expectation-Maximization algorithm, which alternates between estimating hidden variables and optimizing model parameters; AIS training alternates similarly between improving the latent representation, the generator, and improving the value function and policy built on top of it.

**The complete algorithm.** Figure notwithstanding, the training loop proceeds as follows. Given an observation  $\mathbf{o}_t$ , the agent forms the history  $H_t$ , computes the AIS encoding  $\hat{Z}_t = \hat{\sigma}(H_t)$ , and uses this latent state to drive a reward model, a transition model, and the policy. The reward model is trained by regression against observed reward,

$$\hat{r}_t = R_\phi(\mathbf{z}_t, \mathbf{a}_t), \quad L_r = (r_t - \hat{r}_t)^2,$$

so that an accurate reward model is evidence that the representation contains reward-relevant information. The transition model is trained by minimizing the chosen IPM between the true and predicted next-latent distributions,  $L_{\text{transition}} = d_F(\mu, \nu)$ , and the two losses are combined into the AIS encoder loss,

$$L_{\text{AIS}} = \frac{1}{T} \sum_{t=1}^T \left[ \lambda |R_t - \hat{r}(\hat{Z}_t, A_t)|^2 + (1 - \lambda) \log(\nu_t(\hat{Z}_{t+1})) \right],$$

using the MMD-based transition term in practice, with  $\lambda$  trading off reward accuracy against transition accuracy. Policy learning proceeds on top of this fixed or slowly-updating latent state, either with a purely actor-based vanilla policy gradient,

$$J(\theta) = \mathbb{E} \left[ \sum_t \gamma^t R_t \right], \quad \nabla_\theta J(\bar{\xi}, \theta) = \mathbb{E} \left[ \sum_{t=1}^{\infty} \left( \sum_{\tau=1}^t \nabla_\theta \log \pi_\theta(A_\tau | \hat{Z}_\tau) \right) \gamma^{t-1} R_t \right],$$

where  $\bar{\xi}$  denotes the encoder parameters, or with an actor-critic architecture using  $Q(\mathbf{z}, \mathbf{a})$  or an advantage function  $A(\mathbf{z}, \mathbf{a})$  built on the same latent representation.

Algorithm 33 summarizes the resulting procedure. At every meta-iteration, one episode is collected using the current policy and AIS generator, the AIS loss is computed from the resulting reward and transition prediction errors, the policy gradient is computed using the same latent representations, and both the AIS generator parameters  $\bar{\xi}$  and the policy parameters  $\theta$  are updated, at their respective learning rates  $\mathbf{a}_k$  and  $\mathbf{b}_k$ , reflecting the multi-timescale structure described above.

---

**Algorithm 33** AIS-based Partially Observed Reinforcement Learning

---

**Require:** Initial AIS generator  $(\hat{\sigma}, \hat{P}, \hat{r})_{\xi_0}$ , initial policy  $\pi_0$ , discount factor  $\gamma$ , reward weight  $\lambda$ , number of episodes  $K$ , AIS learning rates  $\{a_k\}_{k=1}^K$ , policy learning rates  $\{b_k\}_{k=1}^K$

**Ensure:** Learned policy  $\pi_K$ , learned AIS generator  $(\hat{\sigma}, \hat{P}, \hat{r})_{\xi_K}$

**for**  $k = 1, \dots, K$  **do**

Reset the environment and perform one episode using  $\pi_{k-1}$  and  $(\hat{\sigma}, \hat{P}, \hat{r})_{\xi_{k-1}}$

Collect  $A_{1:T}$ ,  $Y_{1:T}$ , and  $R_{1:T}$

Compute the AIS loss  $L_{\text{AIS}}$  using  $A_{1:T}$ ,  $Y_{1:T}$ ,  $R_{1:T}$ ,  $\lambda$ , and  $(\hat{\sigma}, \hat{P}, \hat{r})_{\xi_{k-1}}$

Compute the policy gradient  $\nabla_{\theta} J(\tilde{\xi}, \theta)$  using  $A_{1:T}$ ,  $Y_{1:T}$ ,  $R_{1:T}$ ,  $\gamma$ ,  $\pi_{k-1}$ , and  $(\hat{\sigma})_{\xi_{k-1}}$

Update  $\tilde{\xi}_{k+1} = \tilde{\xi}_k - a_k \nabla_{\tilde{\xi}} \mathcal{L}(\tilde{\xi}_k)$  and  $\theta_{k+1} = \theta_k + b_k \widehat{\nabla}_{\theta} J(\tilde{\xi}_k, \theta_k)$

**end for**

---

**Role in this chapter.** AIS does not compete with any of the encoders surveyed in the remainder of this section, it grounds them. Sections 5.9.2 through 5.9.6 each build a specific mechanism for compressing experience into a task representation, variational posteriors, contrastive embeddings, direct parameter regression, or exchangeability-based inference, and each could, in principle, be analyzed through this same lens: does the resulting representation approximately satisfy AP1 and AP2, and if so, the error-propagation bound above guarantees that control built on top of it is approximately optimal, regardless of the specific architecture used to produce it. Where a later method explicitly optimizes a reward-prediction or transition-prediction objective as part of its representation learning loss, as several do, this framework provides the theoretical reason such objectives are well founded, rather than an arbitrary design choice among reconstruction, contrastive, or predictive alternatives.

### 7.8.2. Variational Bayesian Belief Methods

The methods in this subsection all follow the same high-level template: posit an explicit generative model of the task, in which trajectories are generated conditioned on some latent task variable, and fit an approximate posterior over that variable using variational inference, typically by maximizing an evidence lower bound. This is the most direct instantiation of the belief  $b_t(\mathbf{w})$  introduced in the section overview: rather than leaving task identity implicit inside a hidden state, as in Section 5.7.1, or approaching it through a discriminative objective, as in Section 5.9.4, these methods build a genuine probabilistic model and train an inference network to approximate its posterior.

### 7.8.2.1. VariBAD: A Very Good Method for Bayes-Adaptive Deep RL via Meta-Learning

Zintgraf et al. [76] give this family its foundational formulation, by taking the classical Bayesian reinforcement learning framework literally and making it tractable through amortized variational inference. Bayesian RL prescribes maintaining a belief, or posterior, over the unknown parameters of the MDP, its reward and transition functions, and acting optimally with respect to the resulting Bayes-Adaptive MDP (BAMDP), in which the state is augmented with this belief.

**The Bayes-Adaptive MDP.** In a BAMDP, the augmented state is a pair of the ordinary environment state and the current belief,  $s^+ = (s_t, b_t)$ , where  $b_t = p(R, T \mid \tau_{:t})$  is the posterior over the reward function  $R$  and transition function  $T$  given the trajectory observed so far. The BAMDP transition function integrates over this belief,

$$T^+(s_{t+1}^+ \mid s_t^+, a_t, r_t) = \mathbb{E}_{b_t} [T(s_{t+1} \mid s_t, a_t)] \delta(b_{t+1} = p(R, T \mid \tau_{t+1})),$$

and the posterior expected reward is  $R^+(s_t^+, a_t, s_{t+1}^+) = \mathbb{E}_{b_{t+1}} [R(s_t, a_t, s_{t+1})]$ , giving an objective

$$J^+(\pi) = \mathbb{E} \left[ \sum_{t=0}^{H^+-1} \gamma^t R^+(s_t^+, a_t, s_{t+1}^+) \right]$$

. A policy that is optimal with respect to this augmented objective is, by construction, Bayes-optimal: it automatically balances exploration and exploitation, since acting to reduce uncertainty in  $b_t$  is itself rewarded whenever doing so leads to better future reward, with no separate exploration bonus or mechanism required. This is the same Bayes-optimality argument implicit in the belief formalization of Section 5.9’s opening, made concrete here as an explicit augmented MDP. Solving the BAMDP exactly, however, is intractable in essentially every setting of practical interest: the belief space is enormous, exact posterior updates are expensive to compute, and planning directly in belief space is generally impossible, exactly the difficulty the AIS framework of Section 5.9.1 addresses at a more general level.

**Approximating the belief with a learned latent task variable.** Rather than attempting to maintain the full posterior over  $R$  and  $T$  directly, VariBAD introduces a low-dimensional latent variable  $m \sim p(m)$  representing the task, and models the reward and transition functions as conditioned on it,  $T(s' \mid s, a, m)$  and  $R(r \mid s, a, s', m)$ . Because  $m$  is low-dimensional, inference over it is far cheaper than inference over the full space of reward and transition functions, giving exactly the kind of tractable

substitute for the true belief that Section 5.9.1’s framework justifies, provided the resulting representation preserves enough reward- and transition-relevant information. Figure 7.2 shows the resulting architecture, built from three components. The encoder  $q_\phi$  is a recurrent network that takes the current state, previous action, and current reward, together with its own previous hidden state  $h_{t-1}$ , and outputs the parameters of an approximate posterior over the task variable,

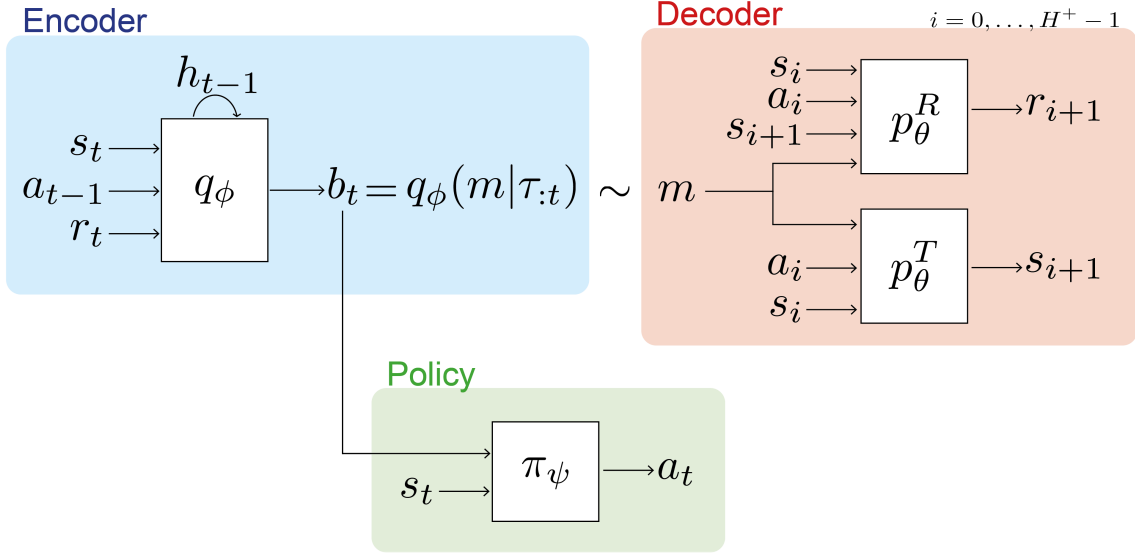
$$b_t = q_\phi(m \mid \tau_t),$$

exactly the structure of a Variational Autoencoder, with  $b_t$  playing the role of a learned belief rather than the raw hidden state used directly in Section 5.7.1. The decoder, shown on the right of Figure 7.2, consists of two separate models conditioned on a sample  $m$  from this posterior: a reward model  $p_\theta^R$ , which takes  $(s_i, a_i, s_{i+1})$  and predicts  $r_{i+1}$ , and a transition model  $p_\theta^T$ , which takes  $(s_i, a_i)$  and predicts  $s_{i+1}$ , both applied across every timestep  $i = 0, \dots, H^+ - 1$  of the trajectory. The policy, shown at the bottom of the figure, takes the current state  $s_t$  together with the belief  $b_t$  produced by the encoder, rather than a single sampled value of  $m$ ,

$$\pi_\psi(a_t \mid s_t, b_t),$$

so that the agent’s behavior can depend on how uncertain it currently is about the task, acting more exploratively when the posterior is diffuse and more exploitatively once it has concentrated, which is precisely the mechanism that gives rise to Bayes-optimal exploration in practice.





**Figure 7.2.** – VariBAD architecture. The encoder  $q_\phi$  recurrently processes the current state, previous action, and reward, together with its own previous hidden state  $h_{t-1}$ , to produce the belief  $b_t = q_\phi(m | \tau_{1:t})$  over the task variable  $m$ . The decoder reconstructs the trajectory from a sampled  $m$ , via a separate reward model  $p_\theta^R$  and transition model  $p_\theta^T$ , applied at every timestep  $i$ . The policy  $\pi_\psi$  is conditioned on the belief  $b_t$  rather than on a single posterior sample. Adapted from Zintgraf et al. [76].

**Training objective.** The encoder and decoder are trained jointly by maximizing the log-likelihood of the observed trajectory under the generative model,  $\log p_\theta(\tau_{1:H})$ , which decomposes as

$$\log p(\tau_H | m) = \log p(s_0 | m) + \sum_{i=0}^{H-1} \left[ \log p(s_{i+1} | s_i, a_i, m) + \log p(r_{i+1} | s_i, a_i, s_{i+1}, m) \right],$$

so that the decoder pair shown in Figure 7.2 is directly responsible for learning the task’s dynamics and reward structure, conditioned on the inferred latent variable. Since the true posterior over  $m$  is intractable, training proceeds by maximizing an evidence lower bound at every timestep,

$$\text{ELBO}_t = \mathbb{E}_{q_\phi(m | \tau_{1:t})} \left[ \log p_\theta(\tau_{1:H} | m) \right] - \text{KL}(q_\phi(m | \tau_{1:t}) \| p(m)),$$

with the reconstruction term expanding to the trajectory log-likelihood given above, evaluated under the current approximate posterior  $b_t$ . The full training objective combines this variational term with the ordinary reinforcement learning objective, averaged over the task distribution,

$$L(\phi, \theta, \psi) = \mathbb{E}_{p(M)} \left[ \underbrace{J(\psi, \phi)}_{\text{RL objective}} + \lambda \sum_{t=0}^{H^+} \text{ELBO}_t \right],$$

so that the encoder, decoder, and policy in Figure 7.2 are all trained end to end from a single combined loss, with the KL term acting as a regularizer keeping the approximate posterior close to the prior  $p(m)$  except where the evidence in  $\tau_t$  genuinely warrants deviating from it, the same information-bottleneck role the KL term plays in every other variational method surveyed in this subsection.

**Training and testing.** Algorithmically, meta-training samples a task  $M_i \sim p(M)$ , runs the current policy to collect a trajectory, computes the belief  $b_t = q_\phi(m \mid \tau)$  from that trajectory using the encoder in Figure 7.2, uses the decoder to predict transitions and rewards from the resulting posterior samples, and jointly optimizes the RL objective and the ELBO loss. At test time, no gradient updates of any kind are performed: the agent interacts with a new task, the encoder updates its belief  $b_t$  over  $m$  from the accumulating trajectory exactly as during training, and the policy conditions on this evolving belief throughout, so that adaptation happens entirely through online Bayesian-style inference, in the same spirit as the adaptation-via-input mechanism of Section 5.7.1, but here the evolving representation the policy reads from is an explicit approximate posterior with a well-defined generative-model interpretation, rather than an unconstrained recurrent hidden state.

**Relation to later methods.** VariBAD establishes the template that PEARL (Section 5.9.2.2) and ELUE (Section 5.9.2.3) both follow and modify: a variational encoder producing an approximate posterior over a latent task variable, trained with an ELBO-style objective against a fixed prior, with the policy conditioned on the result. What VariBAD does not address is sample efficiency at meta-training time: like  $\text{RL}^2$  in Section 5.7.1.1, it is trained on-policy, requiring fresh trajectories from the current policy at every training step. Section 5.9.2.2 addresses this directly, replacing VariBAD’s recurrent trajectory encoder, shown in Figure 7.2, with a permutation-invariant one and enabling fully off-policy training.

### 7.8.2.2. PEARL: Efficient Off-Policy Meta-Reinforcement Learning via Probabilistic Context Variables

Rakelly et al. [77] address the two limitations left open by VariBAD, and by RL<sup>2</sup> before it: reliance on on-policy data throughout meta-training, and the lack of a formal mechanism for reasoning about uncertainty over task identity, a gap that hurts performance most in sparse-reward settings where informative reward signal is scarce. Rather than modifying the belief representation itself, PEARL’s central contribution is architectural, a context encoder that is both permutation-invariant and structured to be updated incrementally, combined with a training procedure that disentangles task inference from policy optimization enough to be trained fully off-policy.

**Task distribution and context.** A task is drawn from a distribution over MDPs,  $\mathcal{T} \sim p(\mathcal{T})$ , where each task is fully specified by an initial state distribution, a transition function, and a reward function,

$$\mathcal{T} = \{p(s_0), p(s_{t+1} \mid s_t, a_t), r(s_t, a_t)\}.$$

The agent’s experience of a given task is summarized by its context: individual transitions  $C_{1:N}^{\mathcal{T}} = \{(s_i, a_i, s'_i, r_i)\}_{i=1}^N$ , collected into a context set  $c_{1:N}^{\mathcal{T}} = \{c_1^{\mathcal{T}}, \dots, c_N^{\mathcal{T}}\}$ . Unlike the trajectory  $\tau_t$  used by VariBAD in Section 5.9.2.1, which is treated as an ordered sequence, PEARL treats context explicitly as an unordered collection of transitions, which is what motivates the permutation-invariant encoder introduced below.

**Latent context variable.** PEARL introduces a random variable  $z$  intended to encode whatever information about the current task is relevant to acting well, and conditions the policy on both the state and this latent context,

$$\pi_{\theta}(a \mid s, z).$$

At every point during interaction with a task,  $z$  is inferred online from the accumulated context through a variational inference network,  $z \sim q_{\phi}(z \mid c)$ , evaluated against a fixed Gaussian prior,  $p(z) = \mathcal{N}(0, I)$ .

**Training objective.** The encoder, policy, and value function are trained jointly by optimizing

$$\mathbb{E}_{\mathcal{T}} \left[ \mathbb{E}_{z \sim q_{\phi}(z | c^{\mathcal{T}})} \left[ R(\mathcal{T}, z) + \beta D_{\text{KL}}(q_{\phi}(z | c^{\mathcal{T}}) \parallel p(z)) \right] \right],$$

where  $R(\mathcal{T}, z)$  is a training signal reflecting task performance under the sampled context variable, implemented in practice through the critic, that is, a Bellman-style value estimate, and  $\beta$  controls the strength of the information bottleneck, trading off how much  $z$  is compressed toward the prior against how much task-relevant information it is allowed to retain. The two terms play complementary roles:  $R(\mathcal{T}, z)$  pushes  $z$  toward encoding whatever information improves performance, while the KL term discourages  $z$  from encoding more than that, penalizing any deviation from the prior that is not earning its keep in terms of performance. This is the same information-bottleneck structure used by VariBAD’s ELBO in Section 5.9.2.1, but here with no explicit reconstruction term at all:  $R(\mathcal{T}, z)$  is task performance itself, evaluated through the actor-critic objective, rather than a log-likelihood of reconstructing observed transitions under a generative model.

At meta-training time, this objective is used to learn two components jointly: a variational inference network  $q_{\phi}(z | c)$ , which, given a context of past transitions, infers a posterior over  $z$ , and a policy together with a value function, both conditioned on  $z$  as part of their input.

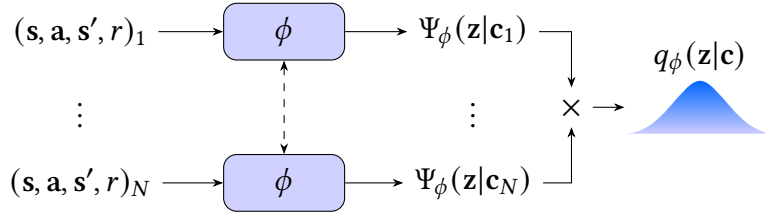
**Permutation-invariant context encoder.** Because context is treated as an unordered set of transitions, the encoder must be invariant to the order in which those transitions are presented. PEARL achieves this through a Gaussian factorization of the posterior,

$$q_{\phi}(\mathbf{z} | \mathbf{c}_{1:N}) \propto \prod_{n=1}^N \Psi_{\phi}(\mathbf{z} | \mathbf{c}_n), \quad \Psi_{\phi}(\mathbf{z} | \mathbf{c}_n) = \mathcal{N}(f_{\phi}^{\mu}(\mathbf{c}_n), f_{\phi}^{\sigma}(\mathbf{c}_n)),$$

so that every individual transition  $\mathbf{c}_n$  is passed independently through the same network  $\phi$  to produce its own Gaussian factor, shown in Figure 7.3: each transition  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)_n$  is mapped by a shared network  $\phi$  to a per-transition Gaussian  $\Psi_{\phi}(\mathbf{z} | \mathbf{c}_n)$ , and these  $N$  Gaussians are multiplied together to produce the final posterior  $q_{\phi}(\mathbf{z} | \mathbf{c})$ . Because a product is invariant to the order of its factors, this construction is permutation-invariant by design, with no architectural bias toward any particular ordering of the context, unlike VariBAD’s recurrent encoder in Section 5.9.2.1, which processes context sequentially. The product of Gaussians has a closed form,

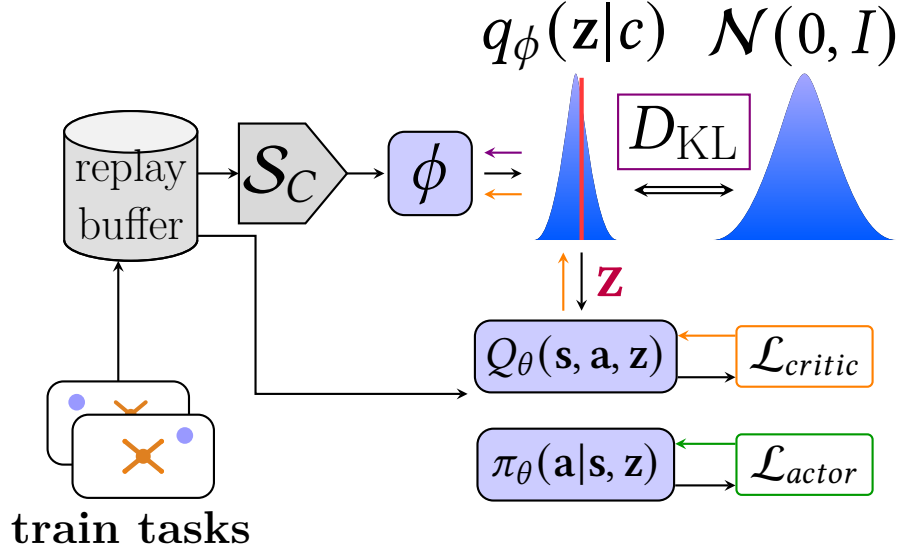
$$\Sigma^{-1} = \sum_{n=1}^N \Sigma_n^{-1}, \quad \mu = \Sigma \left( \sum_{n=1}^N \Sigma_n^{-1} \mu_n \right),$$

which additionally makes the posterior update incremental: a new transition simply contributes its own precision  $\Sigma_n^{-1}$  and precision-weighted mean to the running sums, without requiring the entire context to be reprocessed from scratch, giving both the permutation invariance and the efficient online updating that a sequential encoder does not provide directly.



**Figure 7.3.** – PEARL’s permutation-invariant context encoder. Each transition in the context is passed independently through the same network  $\phi$  to produce a per-transition Gaussian factor  $\Psi_\phi(\mathbf{z} | \mathbf{c}_n)$ . The factors are multiplied together to form the posterior  $q_\phi(\mathbf{z} | \mathbf{c})$ , making the encoder invariant to the order in which transitions are presented. Adapted from Rakelly et al. [77].

**Disentangling task inference from off-policy control.** Figure 7.4 shows the full training architecture. Context is not drawn from the same batch used for policy optimization; instead, it is sampled from the replay buffer through a separate sampling strategy  $\mathcal{S}_C$ , decoupling what data the encoder  $\phi$  sees from what data the critic  $Q_\theta$  and actor  $\pi_\theta$  are trained on at that step. The resulting posterior  $q_\phi(\mathbf{z} | \mathbf{c})$  is regularized toward the prior  $\mathcal{N}(0, I)$  via the KL term, and a sample  $\mathbf{z}$  from this posterior conditions both the critic,  $Q_\theta(\mathbf{s}, \mathbf{a}, \mathbf{z})$ , and the actor,  $\pi_\theta(\mathbf{a} | \mathbf{s}, \mathbf{z})$ , which are trained with standard off-policy actor-critic losses on transitions drawn from the same replay buffer. Because the encoder does not depend on the context and the RL batch being the same data, and because both can be drawn from a growing replay buffer rather than requiring fresh on-policy rollouts, this training procedure is off-policy in the same sense as any standard off-policy actor-critic method, in contrast to VariBAD and  $\text{RL}^2$ , both of which require on-policy data at every training step.



**Figure 7.4.** – Full PEARL training architecture. Context is sampled from the replay buffer via a separate strategy  $\mathcal{S}_C$  and passed through the encoder  $\phi$  to produce the posterior  $q_\phi(\mathbf{z} | c)$ , regularized toward the prior  $\mathcal{N}(0, I)$  via  $D_{\text{KL}}$ . A sample  $\mathbf{z}$  conditions both the critic  $Q_\theta$  and the actor  $\pi_\theta$ , trained with off-policy losses  $\mathcal{L}_{\text{critic}}$  and  $\mathcal{L}_{\text{actor}}$  on transitions drawn from the same buffer. Adapted from Rakelly et al. [77].

**Actor-critic losses.** The critic and actor are trained with standard soft actor-critic objectives, conditioned on the sampled context variable. The critic loss is a Bellman residual conditioned on  $\mathbf{z}$ ,

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_{\substack{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}') \sim \mathcal{B} \\ \mathbf{z} \sim q_\phi(\mathbf{z} | c)}} \left[ Q_\theta(\mathbf{s}, \mathbf{a}, \mathbf{z}) - (r + \bar{V}(\mathbf{s}', \bar{\mathbf{z}})) \right]^2,$$

and the actor loss follows the SAC formulation of matching the policy to the exponentiated soft Q-function,

$$\mathcal{L}_{\text{actor}} = \mathbb{E}_{\substack{\mathbf{z} \sim q_\phi(\mathbf{z} | c) \\ \mathbf{s} \sim \mathcal{B}, \mathbf{a} \sim \pi_\theta}} \left[ D_{\text{KL}} \left( \pi_\theta(\mathbf{a} | \mathbf{s}, \bar{\mathbf{z}}) \parallel \frac{\exp(Q_\theta(\mathbf{s}, \mathbf{a}, \bar{\mathbf{z}}))}{\mathcal{Z}_\theta(\mathbf{s})} \right) \right],$$

with  $\bar{\mathbf{z}}$  denoting a context variable treated as fixed with respect to the actor’s gradient, so that gradients from the actor loss do not flow back into the encoder. Only the critic and KL losses update the encoder parameters  $\phi$ , keeping the division of labor between task inference and control explicit at the level of the gradient flow, not just

the architecture. This is precisely the coupling that Section 5.9.2.3 later identifies as an incomplete decoupling: because  $\phi$  is updated via  $\mathcal{L}_{\text{critic}}$ , the encoder’s training target shifts every time the critic does, even though the encoder never receives gradients from the actor loss directly.

**Meta-training and meta-testing.** Algorithm 34 summarizes meta-training. For each sampled batch of training tasks, the agent first gathers data for each task by repeatedly sampling  $\mathbf{z}$  from the current posterior, rolling out the resulting policy, and accumulating the resulting transitions into a per-task context. Training then proceeds by, for each task, separately sampling a context batch (for the encoder) and an RL batch (for the actor and critic), computing the three losses above, and updating  $\phi$ ,  $\theta_\pi$ , and  $\theta_Q$  at their own learning rates. Algorithm 35 shows meta-testing: the agent repeatedly samples  $\mathbf{z}$  from the posterior given the context accumulated so far, rolls out the resulting policy to collect more data, and accumulates that data into the context for the next round, with no gradient updates performed at any point, so that all adaptation occurs through the encoder’s online posterior updates.

---

**Algorithm 34** PEARL Meta-Training

---

**Require:** Batch of training tasks  $\{\mathcal{T}_i\}_{i=1}^T \sim p(\mathcal{T})$ , learning rates  $\alpha_1, \alpha_2, \alpha_3$   
Initialize replay buffers  $\mathcal{B}^i$  for each training task  
**while** not done **do**  
    **for** each task  $\mathcal{T}_i$  **do**  
        Initialize context  $c^i \leftarrow \{\}$   
        **for**  $k = 1, \dots, K$  **do**  
            Sample  $z \sim q_\phi(z \mid c^i)$   
            Gather data using  $\pi_\theta(a \mid s, z)$  and add it to  $\mathcal{B}^i$   
            Update  $c^i \leftarrow \{(s_j, a_j, s'_j, r_j)\}_{j=1}^N \sim \mathcal{B}^i$   
        **end for**  
    **end for**  
    **for** each training step **do**  
        **for** each task  $\mathcal{T}_i$  **do**  
            Sample context  $c^i \sim S_c(\mathcal{B}^i)$  and RL batch  $b^i \sim \mathcal{B}^i$   
            Sample  $z \sim q_\phi(z \mid c^i)$   
             $\mathcal{L}_{\text{actor}}^i \leftarrow \mathcal{L}_{\text{actor}}(b^i, z)$   
             $\mathcal{L}_{\text{critic}}^i \leftarrow \mathcal{L}_{\text{critic}}(b^i, z)$   
             $\mathcal{L}_{\text{KL}}^i \leftarrow \beta D_{\text{KL}}(q_\phi(z \mid c^i) \parallel r(z))$   
        **end for**  
         $\phi \leftarrow \phi - \alpha_1 \nabla_\phi \sum_i (\mathcal{L}_{\text{critic}}^i + \mathcal{L}_{\text{KL}}^i)$   
         $\theta_\pi \leftarrow \theta_\pi - \alpha_2 \nabla_{\theta_\pi} \sum_i \mathcal{L}_{\text{actor}}^i$   
         $\theta_Q \leftarrow \theta_Q - \alpha_3 \nabla_{\theta_Q} \sum_i \mathcal{L}_{\text{critic}}^i$   
    **end for**  
**end while**

---



---

**Algorithm 35** PEARL Meta-Testing

---

**Require:** Test task  $\mathcal{T} \sim p(\mathcal{T})$   
Initialize context  $c^\mathcal{T} \leftarrow \{\}$   
**for**  $k = 1, \dots, K$  **do**  
    Sample  $z \sim q_\phi(z \mid c^\mathcal{T})$   
    Roll out policy  $\pi_\theta(a \mid s, z)$  to collect data  $\mathcal{D}_k^\mathcal{T} \leftarrow \{(s_j, a_j, s'_j, r_j)\}_{j=1}^N$   
    Accumulate context  $c^\mathcal{T} \leftarrow c^\mathcal{T} \cup \mathcal{D}_k^\mathcal{T}$   
**end for**

---

**Relation to earlier and later methods.** Relative to VariBAD in Section 5.9.2.1, PEARL trades the sequential, reconstruction-based encoder for a permutation-invariant, discriminatively-trained one, and trades on-policy training for off-policy training, at the cost of no longer having an explicit generative model of the task’s dynamics and



reward function to fall back on for interpretability or auxiliary supervision. Two separate lines of later work directly target weaknesses in this design. Section 5.9.2.3 argues that PEARL is not, in fact, fully off-policy in the strictest sense, since the encoder’s training signal still flows indirectly through the critic, exactly as made explicit above, and that conditioning the policy on a single sample  $z$  rather than the full belief  $q_\phi(z \mid c)$  discards exactly the uncertainty information the belief was meant to carry. Sections 5.9.3 and 5.9.4 separately argue that an encoder trained only to support value prediction, as PEARL’s is here, can absorb shared, task-irrelevant information rather than discriminating cleanly between tasks, motivating the contrastive and variance-based alternatives developed there.

A further method, ELUE, fixes a specific off-policy limitation left open by PEARL; it is covered in full in Appendix A.

## 7.9. Centralized Training with Decentralized Execution for Meta-Reinforcement Learning

Every method surveyed so far in this chapter, across Section 5.6, Section 5.7, and Section 5.9, is presented in the single-agent setting: one agent, one policy, one stream of experience from which to adapt. Extending any of these families to the multi-agent setting is not automatic. The standard paradigm for multi-agent reinforcement learning, Centralized Training with Decentralized Execution (CTDE), allows a critic or other training-time component to access global information unavailable to any individual agent, while each agent’s actor must still act using only its own local observations at execution time. This section examines how CTDE can be layered on top of each of the three meta-RL families surveyed in this chapter, and treats one instantiation, Meta Global-to-Local, in full detail, since it is the only one of the three that introduces a genuinely new architectural problem specific to combining CTDE with task inference.

### 7.9.1. CTDE on Top of Policy Parameter Gradient Methods

Extending a method from Section 5.6 to CTDE raises a design question specific to gradient-based inner-loop adaptation: which components of the multi-agent system should be updated by the inner loop at all. Three choices are available, summarized in Table 7.4.

**Table 7.4.** – Choices for what the inner loop adapts when combining CTDE with policy parameter gradient methods.

| Inner-loop adaptation | Advantage                                                   | Recommendation                   |
|-----------------------|-------------------------------------------------------------|----------------------------------|
| Actor only            | Simple, directly improves adaptation of behavior            | Good baseline                    |
| Critic only           | Helps value estimation but does not directly adapt behavior | Not preferred                    |
| Actor + critic        | Adapts both policy and value estimation                     | Best choice for a primary method |

Adapting the actor only keeps the inner loop close to the single-agent MAML formulation of Section 5.6.1.1, treating value estimation as a fixed, slowly-changing quantity that does not itself need to specialize to a new task within the few inner-loop steps available. Adapting the critic only is generally weaker, since improving value estimation does not translate directly into improved behavior unless the actor is also updated to exploit that improved estimate. Adapting both actor and critic gives the most complete inner loop, allowing both behavior and value estimation to specialize to the task at hand, at the cost of a larger inner-loop parameter set to adapt with the limited data available during meta-testing, the same capacity-versus-data trade-off already discussed for single-agent gradient-based methods in Section 5.6.

### 7.9.2. CTDE on Top of Black-Box Methods

Extending a black-box method from Section 5.7 to CTDE is comparatively direct, since the adaptation-via-input mechanism of Section 5.7.1 already generalizes naturally to a multi-agent setting. Each agent  $i$  maintains its own recurrent hidden state, updated from its own local observation, action, and reward stream,

$$h_i^t = f_\theta(h_i^{t-1}, o_i^t, a_i^{t-1}, r^{t-1}),$$

with the agent’s decentralized policy depending only on this local hidden state and local observation,  $a \sim \pi_{\theta_i}^i(a \mid o_i^t, h_i^t)$ , exactly the single-agent recurrent formulation of Section 5.7.1.1, applied independently per agent. The centralized component required for CTDE enters only through the critic, or through an aggregation of the per-agent hidden states,  $h = f_{\text{agg}}((h^i)_{i=1}^N)$ , which the centralized critic can then take

as input alongside the global state,  $V_\psi(s, h)$  for an MAPPO-style value function, or  $Q_\psi(s, a, h)$  for an actor-critic formulation. Because each agent’s hidden state already functions as an implicit belief over its own local task and history, exactly as in Section 5.7.1, this aggregation gives the centralized critic access to a summary of every agent’s implicit adaptation state during training, while each agent’s actor continues to condition only on its own hidden state at execution time, preserving the decentralized execution constraint.

### 7.9.3. CTDE on Top of Task Inference Methods: Meta Global-to-Local (MG2L)

Extending a task inference method from Section 5.9 to CTDE introduces a problem that neither of the two extensions above needs to confront: task identity itself may not be locally inferable. Zhao et al. [78] identify this problem directly and build a method, Meta Global-to-Local, on top of PEARL’s context-based inference mechanism (Section 5.9.2.2) to address it.

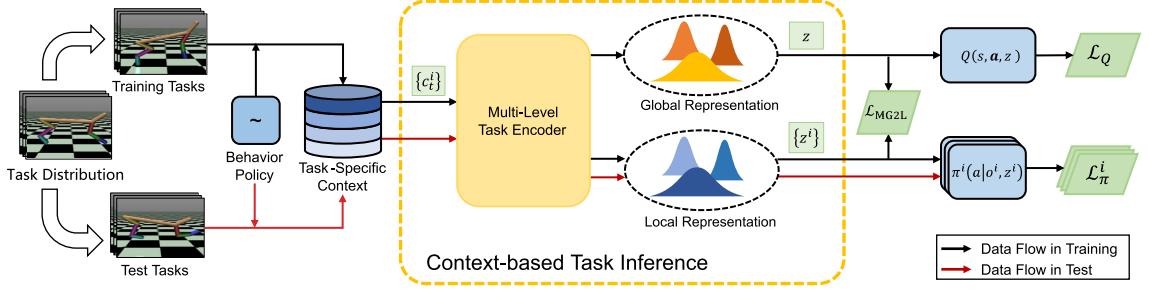
**The global-to-local gap.** In single-agent meta-RL, an agent can in principle observe enough of the environment to infer the task correctly on its own, exactly the assumption underlying every method in Section 5.9.2. In a multi-agent system, the complete context is distributed across agents,  $c = \{c^1, c^2, \dots, c^n\}$ , while agent  $i$  has access only to its own local context  $c^i$ . Because the global context aggregates strictly more information than any single agent’s local context, the mutual information between a task representation and the full context necessarily exceeds the mutual information achievable from any local context alone,

$$I(z; c) > I(z_i; c^i),$$

a gap the authors term the global-to-local gap. A representation trained only on local context, as a naive per-agent extension of PEARL would produce, is fundamentally limited by this inequality, no matter how well trained the local encoder is.

**Meta-CTDE.** The proposed solution follows the CTDE template directly: a centralized critic conditions on global information,  $Q(o, a, z)$  with  $z \sim q(z | c)$  inferred from the full, distributed context, while each decentralized actor conditions only on a local task representation,  $\pi^i(a^i | o^i, z^i)$  with  $z^i \sim q^i(z^i | c^i)$  inferred from local context alone. Figure 7.5 shows the resulting pipeline. Trajectories from the training and test task distributions, together with task-specific context collected by a behavior policy, feed a Multi-Level Task Encoder, which produces both a global representation

$z$ , used to train the centralized critic via  $\mathcal{L}_Q$ , and a per-agent local representation  $z^i$ , used to condition the decentralized actor  $\pi^i$  via  $\mathcal{L}_\pi^i$ . Data flow during training, shown by black arrows, includes both global and local representations; data flow at test time, shown by red arrows, includes only the locally inferable  $z^i$ , since the global context is unavailable once execution is decentralized. The loss  $\mathcal{L}_{\text{MG2L}}$  connecting the global and local representations is what closes the global-to-local gap, and is the central technical contribution of this method, developed below.

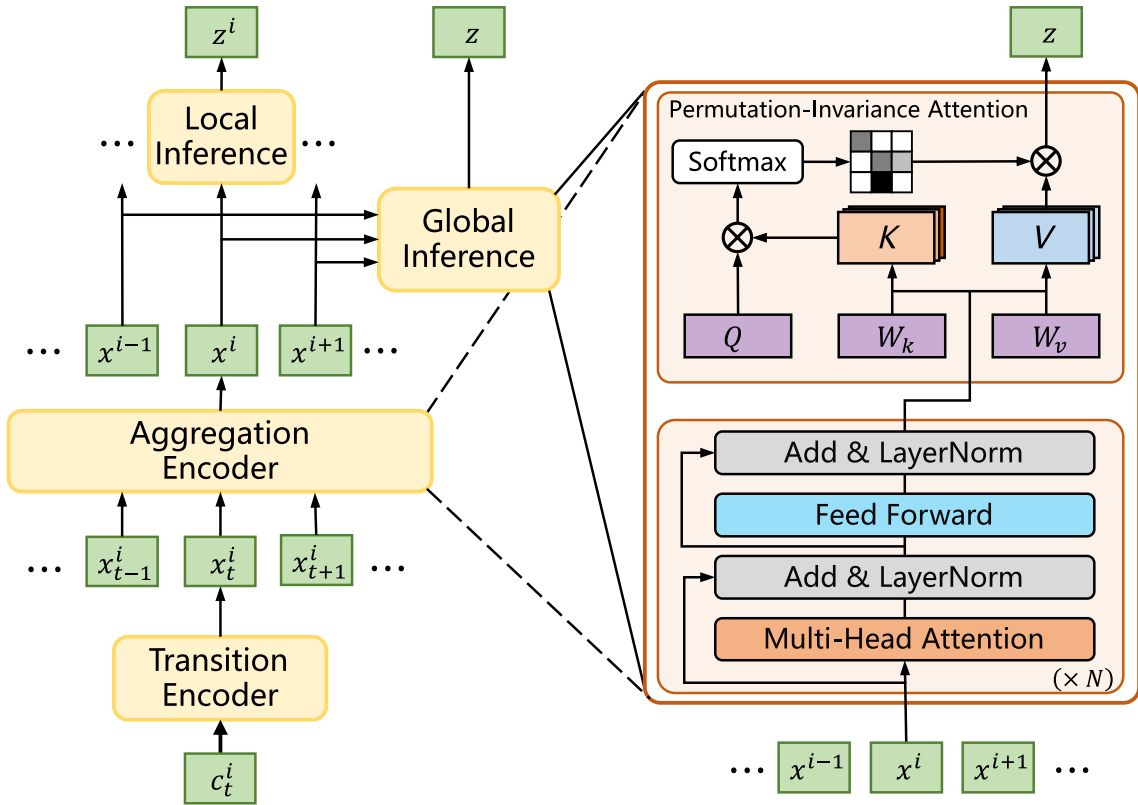


**Figure 7.5.** – Meta-CTDE overview. Training and test task trajectories, together with task-specific context gathered by a behavior policy, feed a Multi-Level Task Encoder producing a global representation  $z$  and per-agent local representations  $z^i$ . The global representation trains the centralized critic  $Q(s, a, z)$  via  $\mathcal{L}_Q$ ; each local representation conditions a decentralized actor  $\pi^i(a | o^i, z^i)$  via  $\mathcal{L}_\pi^i$ ; and  $\mathcal{L}_{\text{MG2L}}$  links the two representations. Black arrows show training-time data flow (both representations available); red arrows show test-time data flow (only the local representation is available). Adapted from Zhao et al. [78].

**Multilevel Task Encoder.** The encoder that produces both  $z$  and  $z^i$  is built in three stages, shown in Figure 7.6. At the transition level, each transition  $c_t^i = (o_t^i, a_t^i, r_t, o_{t+1}^i)$  is passed through a transition encoder, an MLP shared across all agents and all timesteps, producing  $x_t^i = E_{\text{tran}}(c_t^i)$ . At the aggregation level, the sequence of transition-level representations for a given agent is aggregated across time using Permutation-Invariance Attention (PIA), designed so that  $f_E(P(X)) = f_E(X)$  for any permutation  $P$  of the input sequence, matching the exchangeability principle already discussed in Section 5.9.6, but here applied to a sequence of transitions from a single agent rather than to inter-task context. PIA removes positional encoding and uses attention with a shared query, so that transitions are combined with a weighted sum rather than treated equally, reflecting that not every transition is equally informative about the task,

$$x^i = E_{\text{agg}}(x_0^i, \dots, x_T^i) = \sum_{t=0}^T \sigma \left( \frac{Q_{\text{agg}}(x_t^i W_{\text{agg}}^k)^\top}{\sqrt{d}} \right) x_t^i W_{\text{agg}}^v.$$

Figure 7.6 shows this aggregation encoder as  $N$  stacked blocks of multi-head attention followed by a feed-forward layer, each with residual connections and layer normalization, with the PIA mechanism itself detailed in the inset: queries  $Q$ , keys  $K$ , and values  $V$  are computed from the transition-level representations  $x_t^i$ , combined via softmax attention, with the aggregation producing  $x^i$  fed forward. At the multi-agent level, a global encoder combines the aggregated representations from every agent,  $z = E_G(x^1, \dots, x^n)$ , while a separate local encoder produces each agent’s own representation,  $z^i = E_L^i(x^i)$ , from only that agent’s aggregated representation.



**Figure 7.6.** – Multi-Level Task Encoder detail. The transition encoder  $E_{\text{tran}}$  processes each transition  $c_t^i$  into  $x_t^i$ . The aggregation encoder combines the sequence  $x_{t-1}^i, x_t^i, x_{t+1}^i, \dots$  across time using stacked Permutation-Invariance Attention blocks (inset), producing  $x^i$ . The global inference module combines every agent’s  $x^i$  into  $z$ ; the local inference module produces each agent’s own  $z^i$  from  $x^i$  alone. Adapted from Zhao et al. [78].

**Why not the standard auxiliary losses.** Training this encoder using the RL loss alone is insufficient, since RL losses are stochastic, high-variance, and can carry task-irrelevant information, echoing the same diagnosis CCM made of PEARL’s value-function-trained encoder in Section 5.9.4.1. The standard fix used elsewhere in this chapter, an auxiliary next-state or next-reward prediction loss such as the CPC objectives of Section 5.7.3.2 and Section 5.9.7, is not available here either: from any individual agent’s perspective, the environment is non-stationary, since the other agents’ behavior is also changing during training, making next-observation prediction an unreliable training signal in the multi-agent setting.

**Global representation loss.** The global encoder is instead trained by maximizing a mutual information lower bound between the global representation and the task, using an InfoNCE-style contrastive construction directly analogous to CCM’s information-gain bound in Section 5.9.3.3,

$$I(z; \mathcal{T}) \geq \mathbb{E}_{\mathcal{T}, z, c} \left[ \log \frac{h(c; z)}{\sum_{\mathcal{T}} h(c^*; z)} \right] + \log(N),$$

where  $h(c; z) \triangleq p(z | c) / p(z) \approx \exp(g(E_{\phi_g}(c^+); E_{\phi_g}(c)))$  for a similarity function  $g$ ,  $c^+$  a positive context drawn from the same task, and  $c^* = c^+ \cup c^-$  the union of positive and negative contexts. The resulting training objective combines this InfoNCE-style term with a KL regularizer against a prior, exactly the information-bottleneck structure used throughout Section 5.9.2 and Section 5.9.3,

$$L_G = L_{\text{InfoNCE}} + \alpha D_{\text{KL}}(q(z | c) \| p(z)) \approx -\frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} \left[ \log \frac{h(c; z)}{\sum_{\mathcal{M}} h(c^*; z)} \right] + \alpha D_{\text{KL}}(q(z | c) \| p(z)).$$

**From global to local: naive consistency and why it fails.** The harder question is how to train the local encoder  $z^i$  so that it carries as much task-relevant information as possible, given that agent  $i$  cannot observe the global context  $c$  directly. A naive approach would minimize a KL consistency loss,  $D_{\text{KL}}(q(z^i | c^i) \| q(z | c))$ , directly forcing the local distribution to match the global one. This does not work well in practice: because the local context genuinely does not contain all the information present in the global context, forcing  $z^i \approx z$  is an unachievable target, and the authors report that this consistency loss initially decreases during training but later fails to converge, as the local encoder is pushed toward a target it structurally cannot reach.

**Global-to-local distillation via conditional mutual information.** The fix reframes the objective: rather than making  $z^i$  match  $z$ , the local representation should be trained to provide additional task information *given* the global representation, maximizing the conditional mutual information

$$I(z^i; c \mid z) = I(z, z^i; c) - I(z; c),$$

equivalently minimizing  $I_r(z; z^i \mid c) = I(z; c) - I(z, z^i; c)$ , which is upper bounded using the same InfoNCE machinery as the global loss,

$$I_r(z; z^i \mid c) \leq I_{\text{Llo}}(z; c) - I_{\text{NCE}}(z, z^i; c),$$

with  $I_{\text{Llo}}(z; c) = \mathbb{E}_{z,c} [\log(h(c; z)) / (\sum_{\mathcal{M}^-} h(c^-; z))] + \log N$  and  $I_{\text{NCE}}(z, z^i; c) = \mathbb{E}_{z^i, c} [\log(h(c; z, z^i)) / (\sum_{\mathcal{M}} h(c^*; z, z^i))] + \log N$ . Minimizing the resulting upper bound,

$$L_{\text{upper}} = \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} \left\{ \log \frac{h(c; z)}{h(c; z, z^i)} + \log \frac{\sum_{\mathcal{M}} h(c^*; z, z^i)}{\sum_{\mathcal{M}^-} h(c^-; z)} \right\},$$

with  $h(c; z, z^i) \approx \exp(g([z^+, z^{i+}], [z, z^i]))$ , gives the local training objective, combined with a KL regularizer as before,

$$\mathcal{L}_L(\phi_g, \phi_l^i) = \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} \left\{ \log \frac{h(c; z)}{h(c; z, z^i)} + \log \frac{\sum_{\mathcal{M}} h(c^*; z, z^i)}{\sum_{\mathcal{M}^-} h(c^-; z)} \right\} + \alpha D_{\text{KL}}.$$

The intuition is that  $z^i$  is not asked to reconstruct or approximate the full global representation  $z$ , which it structurally cannot do; instead, it is trained to preserve as much useful task information as can actually be extracted from its own local context  $c^i$ , while being shaped in the direction indicated by the richer global representation, distillation toward a direction rather than toward an unreachable target.

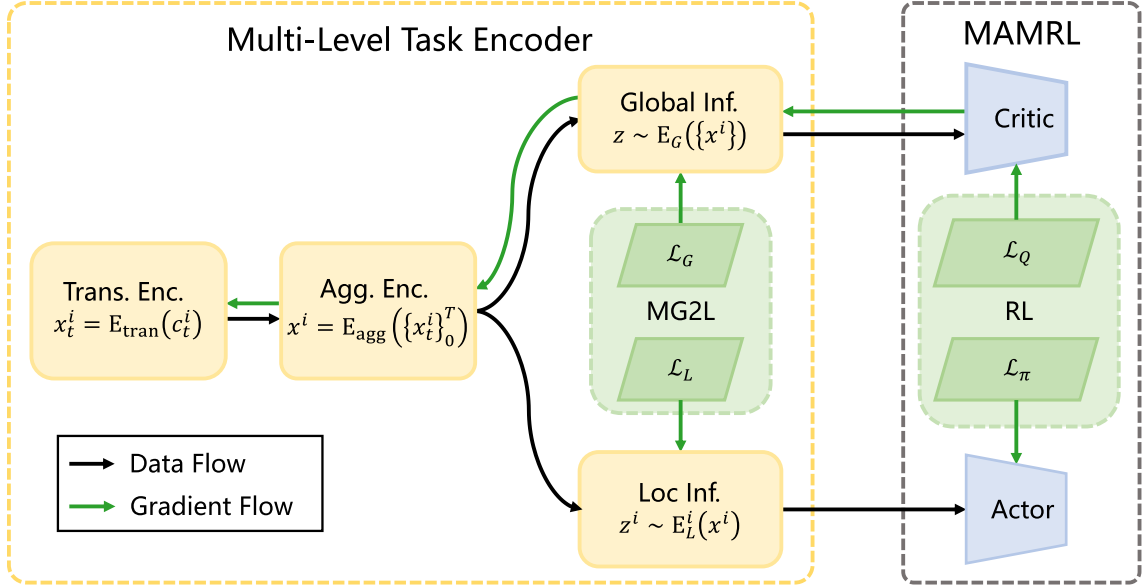
**Critic and actor.** With both representations available, the centralized critic is trained with a standard temporal-difference loss conditioned on the global representation,

$$\mathcal{L}_Q(\phi_g, \theta_c) = \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} [\hat{R}_t - Q(s_t, \mathbf{a}_t, z)]^2,$$

while each decentralized actor is trained with an MAPPO-style clipped objective conditioned on its own local representation, with the representation itself treated as fixed with respect to the actor’s own gradient via a stop-gradient operator  $\text{sg}(\cdot)$ ,

$$\mathcal{L}_\pi(\theta_a^i) = -\frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} \left[ \mu_t^i A_t^i + \epsilon S[\pi^i(o_t^i, \text{sg}(z^i))] \right], \quad A_t^i = \sum_{l=0}^{\infty} (\gamma \lambda)^l [r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l})].$$

Figure 7.7 summarizes how every one of these components connects, distinguishing data flow (black arrows) from gradient flow (green arrows). The transition encoder feeds the aggregation encoder, which feeds both the global and local inference modules; the global inference module trains via  $\mathcal{L}_G$  and additionally supplies the gradient direction for  $\mathcal{L}_L$ , the module linking global and local representations described above; the global representation  $z$  feeds the centralized critic, trained via  $\mathcal{L}_Q$ ; and the local representation  $z^i$  feeds the decentralized actor, trained via  $\mathcal{L}_\pi$ .



**Figure 7.7.** – Full MG2L architecture, distinguishing data flow (black) from gradient flow (green). The Multi-Level Task Encoder (left) produces global and local task inferences, linked by the MG2L losses  $\mathcal{L}_G$  and  $\mathcal{L}_L$ . The resulting representations feed the centralized critic and decentralized actor (right), trained with the standard MAMRL losses  $\mathcal{L}_Q$  and  $\mathcal{L}_\pi$ . Adapted from Zhao et al. [78].



**Training procedure.** Meta-training, summarized in Algorithm 36, alternates between collecting context and RL data and updating every network. At every iteration, transitions are first collected for context using a task representation sampled from the prior, then a fresh round of transitions is collected for RL training using the representation inferred from the accumulated context. The centralized critic and global encoder are updated jointly by minimizing  $\mathcal{L}_Q + \mathcal{L}_G$ , each decentralized actor is updated by minimizing  $\mathcal{L}_\pi^i$ , and each local encoder is updated by minimizing  $\mathcal{L}_L$ .

---

**Algorithm 36** Centralized Meta-Training (MG2L)

---

**Require:** Batch of training tasks  $\{M_k\}_{k=1}^T \sim p(\mathcal{M})$ , context buffer  $\mathcal{C}_k$ , RL buffer  $\mathcal{B}_k$   
 per task, networks  $\theta_a^i, \theta_c, \phi_g, \phi_l^i$   
**while** not done **do**  
   **Collect transitions for context**  
     **for** each task  $M_k$ , timestep  $t$  **do**  
       Sample prior representation  $z_t^i \sim \mathcal{N}(0, I)$   
       Roll out  $\pi^i(a_t^i | o_t^i, z_t^i)$  on  $M_k$ ; store transition into  $\mathcal{C}_k$   
     **end for**  
   **Collect transitions for RL**  
     **for** each task  $M_k$ , timestep  $t$  **do**  
       Sample context  $c^i$  from  $\mathcal{C}_k$ ; compute  $z_t^i = E_{\phi_g, \phi_l^i}(c^i)$   
       Roll out  $\pi^i(a_t^i | o_t^i, z_t^i)$  on  $M_k$ ; store transition into  $\mathcal{B}_k$   
     **end for**  
   **Update networks**  
     **for** each task  $M_k$  **do**  
       Sample RL batch from  $\mathcal{B}_k$   
       Update  $\theta_c, \phi_g$  to minimize  $\mathcal{L}_Q(\theta_c, \phi_g) + \mathcal{L}_G(\phi_g)$   
       Update  $\theta_a^i$  to minimize  $\sum_i \mathcal{L}_\pi^i(\theta_a^i)$   
       Update  $\phi_l^i$  to minimize  $\mathcal{L}_L(\phi_g, \phi_l^i)$   
     **end for**  
**end while**

---

At test time, Algorithm 37 uses only the decentralized components, consistent with the CTDE constraint that execution cannot depend on global information. An exploration phase samples task representations from the prior, rolls out the local policy, and stores transitions into a priority buffer, tracking a transition priority weight  $\omega_t^i$ . An evaluation phase then samples context weighted by this priority, computes the local representation from the local encoder alone,  $z_t^i = E_{\phi_l^i}(c^i)$ , and rolls out the resulting policy, with no global encoder and no centralized critic involved anywhere in this procedure.

---

**Algorithm 37** Decentralized Meta-Test (MG2L)

---

**Require:** Trained models  $E_{\phi_g, \phi_l^i}$ ,  $\pi^i$ , test task  $M$ , priority buffer  $\mathcal{C}$

**Exploration**

**for** episode =  $1, \dots, K$ , timestep  $t$  **do**

    Sample prior representation  $z_t^i \sim \mathcal{N}(0, I)$

    Roll out  $\pi^i(a_t^i \mid o_t^i, z_t^i)$  on  $M$ ; store transition into  $\mathcal{C}$ ; update  $\omega_t^i$

**end for**

**Evaluation**

**for** episode =  $K + 1, \dots, N$ , timestep  $t$  **do**

    Sample context  $c^i$  from  $\mathcal{C}$ , weighted by  $\omega_t^i$ ; compute  $z_t^i = E_{\phi_l^i}(c^i)$

    Roll out  $\pi^i(a_t^i \mid o_t^i, z_t^i)$  on  $M$ ; observe  $r_t, o_{t+1}^i$

**end for**

---

**Relation to the rest of this chapter.** MG2L is a direct multi-agent extension of PEARL (Section 5.9.2.2), inheriting its permutation-invariant, context-based inference mechanism and its InfoNCE-based training signal from CCM (Section 5.9.3.3 and Section 5.9.4.1), but introduces a genuinely new problem, and a genuinely new solution, that has no single-agent counterpart anywhere else in this chapter: the global-to-local gap cannot be closed by any representation-learning mechanism surveyed in Section 5.9 alone, since every one of those mechanisms implicitly assumes the full task-relevant context is available to the encoder being trained. The conditional mutual information objective introduced here, distilling a local representation toward a global one without forcing exact equality, is the specific innovation required once that assumption is dropped, and is a candidate template for extending any of the other task inference mechanisms in Section 5.9, VariBAD’s reconstruction-based belief, ESCP’s variance-and-determinant embedding, or BRUNO’s exchangeable posterior, to the same distributed-context, multi-agent setting.

---

## 8. Robustness and Adaptability: A Unified Synthesis

RQ4 asked how robust MARL and adaptive MARL relate. Table 8.1 lays the two families out on four axes built directly from the taxonomies developed in the preceding chapters.

### 8.1. A Shared Root Cause

The two families share a root cause and diverge only on what they assume about it. Both exist because a MARL policy’s environment, including the other agents inside it, does not stay fixed. Robust MARL assumes that departure is bounded and treats every point inside that bound as equally likely, including the worst one; this is why its central objective, developed in Chapter 5, is a minimax over an ambiguity set  $\mathcal{P}$  rather than an expectation over a single nominal environment. Adaptive MARL assumes the departure can grow without bound but follows some exploitable structure over time, and spends its effort tracking that structure, whether through a confidence set that shrinks as evidence accumulates, as in UCRL2, or through a task representation learned once across many tasks and inferred online at test time, as in PEARL, rather than preparing for the worst instantiation of that structure.

This is not a connection unique to this thesis. Treating a robust objective as an online-learning problem, and reading its ambiguity set the way a confidence set is read, is exactly what Wang and Zou [79] do for single-agent robust RL, estimating the uncertainty set from streaming data rather than fixing it in advance, and what Farhat, Ghosh, Atia, *et al.* [80] do for the multi-agent case, solving a distributionally robust Markov game through online interaction rather than a fixed offline ambiguity set. Both results confirm that the boundary between the robust and adaptive objectives is not structural, only a modeling choice about whether the ambiguity set is estimated once or updated as data arrives. What is missing from that literature, and what this section adds, is the same reduction stated at the level of the taxonomy built in this thesis, so it applies to the exemplar methods surveyed here rather than to one algorithm at a time.

## 8.2. Where the Two Taxonomies Already Touch

Holding the two taxonomies up against each other, rather than reading them as separate literatures, makes three points of contact visible that are not stated as such by any single paper covered in this thesis.

**Distributionally robust methods are adaptive methods with a frozen, non-shrinking confidence set.** The claim can be stated precisely using the two methods' own formulations from Chapters 5 and 6. DrIGM optimizes against the worst transition model inside a per-history-action ambiguity set built once around a fixed reference model  $P_{\mathbf{h},a}^0$  at a fixed total-variation radius  $\rho$ ,

$$\mathcal{P}_{\mathbf{h},a} = \left\{ P \in \Delta(\mathcal{H}) : d_{\text{TV}}(P, P_{\mathbf{h},a}^0) \leq \rho \right\},$$

and never updated as new transitions are observed. SWUCRL2-CW's transition confidence region at time  $t$  is

$$H_{p,t}(s, a)(\eta) = \left\{ \hat{p}(\cdot \mid s, a) \in \Delta_S : \|\hat{p}(\cdot \mid s, a) - \hat{p}_t(\cdot \mid s, a)\|_1 \leq \text{rad}_{p,t}(s, a) + \eta \right\},$$

re-centered on the running windowed estimate  $\hat{p}_t$  and shrinking with visitation count through  $\text{rad}_{p,t}(s, a) \rightarrow 0$ , with a fixed floor  $\eta$  added on top for exactly the reason  $\mathcal{P}_{\mathbf{h},a}$  is fixed in DrIGM: protection against a model that has moved since it was last estimated. Freezing the re-centering step ( $\hat{p}_t \equiv \hat{p}_0$ , never updated) and disabling the shrinking term ( $\text{rad}_{p,t}(s, a) \equiv 0$ , so only the constant floor remains) turns  $H_{p,t}(s, a)(\eta)$  into an  $L^1$  ball of fixed radius  $\eta$  around a fixed center, the same construction as  $\mathcal{P}_{\mathbf{h},a}$ , whose radius  $\rho$  is stated in total-variation distance rather than  $L^1$ . By Appendix C's identity  $d_{\text{TV}} = \frac{1}{2}\|\cdot\|_1$ , a total-variation ball of radius  $\rho$  is an  $L^1$  ball of radius  $2\rho$ , so the matching radius is  $\rho = \eta/2$ , not  $\rho = \eta$ . The two sets also range over different spaces:  $\mathcal{P}_{\mathbf{h},a} \subseteq \Delta(\mathcal{H})$  is a set of distributions over joint histories, as DrIGM is built for a Dec-POMDP, while  $H_{p,t}(s, a)(\eta) \subseteq \Delta_S$  is a set of distributions over next states, as SWUCRL2-CW is built for a fully observable MDP; they coincide as sets, not only as constructions, when the Dec-POMDP is fully observable and a history reduces to the current state, so  $\mathcal{H} = S$ . Outside that case, DrIGM's frozen set and SWUCRL2-CW's frozen limit are the same mechanism, an  $L^1$ /total-variation ball of fixed radius around a fixed center, applied to different underlying spaces, not literally the same set. DrIGM is that special case mechanically; it is that special case set-theoretically only under full observability. Neither DrIGM nor SWUCRL2-CW is described this way by its own authors; the connection becomes visible only once both are read against the same four-axis comparison.

**Task-inference encoders can be meta-trained against adversarial tasks.** PEARL and VariBAD, developed in Chapter 7, and ELUE, developed in Appendix A, all meta-train a context encoder across a distribution of tasks  $p(\mathcal{T})$  so that a new task can be inferred from a handful of transitions at test time. Nothing about this training procedure requires  $p(\mathcal{T})$  to contain only benign variation in dynamics or reward. A task distribution that includes adversarially perturbed variants, generated the way ADR’s Nature Player generates difficult but physically plausible domain-randomization parameters, would push a task-inference method toward the robust objective of Chapter 5 without changing its underlying mechanism at all. This is a straightforward recombination of two existing pieces, not a new algorithm, and it is exactly the kind of gap that stays invisible until the two taxonomies are compared directly.

**Multi-agent credit assignment is the missing piece on both sides.** M3DDPG trains each agent against the worst-case actions of its teammates, treating a shift in teammate behavior as an adversarial disturbance to defend against. Change-point detectors and meta-RL task encoders, on the adaptive side, generally treat “the environment changed” as a single undifferentiated event, without asking whether the physical environment changed or a specific teammate’s policy changed. Meta Global-to-Local is the one method surveyed in Chapter 7 that engages with multi-agent structure directly, and it does so for task inference, not for attributing an observed change to a specific cause. Neither side of the table has a general answer to who, or what, caused a given disturbance.

These three points of contact are gaps, not results; nothing in this thesis tests any of them. Chapter 10 returns to them as concrete directions rather than as claims.

**Table 8.1.** – Robust MARL and adaptive MARL compared on four axes.

| Axis                      | Robust<br>(Chapter 5)                                                                                         | MARL<br>Adaptive<br>(Chapters 6 and 7)                                                                                                                   |
|---------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Objective                 | Maximize worst-case return over a bounded, unknown disturbance                                                | Minimize dynamic regret against a moving optimum, or adapt in a handful of episodes to a new task                                                        |
| When handled              | Before deployment, during training                                                                            | After deployment, on-line, or amortized across meta-training and applied at test time                                                                    |
| Assumption on disturbance | Bounded, adversarial or not, fixed for the duration of training                                               | Unbounded over time, but structured: piecewise stationary, slowly varying, switching, or drawn from a task distribution                                  |
| Mechanism                 | Minimax training (RARL, M3DDPG), regularization penalty (ERNIE, MIR3), or worst-case transition model (DrIGM) | Detection and restart (BOCPD), windowed re-estimation (SW-UCRL2, SWUCRL2-CW), or a learned adaptation procedure (MAML, RL <sup>2</sup> , VariBAD, PEARL) |
| Failure mode              | The real disturbance exceeds the bound trained against                                                        | The change is faster than the detector’s reaction time, or unlike anything in the meta-training task distribution                                        |

---

## 9. Contributions

This thesis contributes four things beyond a list of methods.

1. **A single taxonomy for two literatures.** Robust MARL and adaptive MARL are placed under one framing, non-stationarity handled before deployment versus after, rather than treated as two subfields that happen to both use the word MARL. Chapter 5 and Chapter 6, together with Chapter 7, are written to be read against each other, not independently.
2. **A four-axis comparison.** Table 8.1 lets a specific method from either family be located and compared against a specific method from the other, on objective, timing, assumption, mechanism, and failure mode, rather than comparing entire subfields in the abstract. This is the mechanism that makes Chapter 8's three points of contact visible in the first place.
3. **A curated, depth-first set of exemplar methods.** Rather than an exhaustive catalogue, each taxonomy branch, adversarial training, adversarial regularization, distributionally robust value factorization, sim-to-real transfer, change-point detection, sliding-window estimation, and each meta-RL family, is represented by a small number of methods, most branches one to three, adversarial regularization four, selected specifically for what they assume, what they cost, and how they fail. This is a shortlist suitable for selecting baselines in an empirical study, not a survey aiming for completeness. Table 9.1 lists this shortlist directly, method by method, on the same five axes as Table 8.1.
4. **A concrete gap analysis.** Chapter 10 identifies four specific, checkable gaps, not a vague call for more research, that block evaluating robust and adaptive MARL methods on common ground: no shared benchmark, no shared metric, no method tested against both a bounded attack and unbounded drift together, and no general multi-agent credit assignment mechanism for what caused an observed change.

**Table 9.1.** – The curated exemplar methods referenced in Chapters 5–7 and Appendix A, one row per method, on the same five axes as Table 8.1. Every cell restates what that method’s own subsection, or Chapter 8’s comparison of it, already says; none is a new characterization.

| Method                                        | Objective                                                             | Timing                             | Assumption                                                          | Mechanism                                                                                  | Failure mode                                                                                          |
|-----------------------------------------------|-----------------------------------------------------------------------|------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <i>Adversarial training (Chapter 5)</i>       |                                                                       |                                    |                                                                     |                                                                                            |                                                                                                       |
| RARL                                          | Max-min return, protagonist vs. adversary                             | Before deployment, during training | Adversary can inject disturbances matching test-time perturbations  | Alternating zero-sum mini-max training                                                     | Robust only to disturbance types its own adversary generated                                          |
| M3DDPG                                        | Max return under worst-case joint action of other agents              | Before deployment, during training | Worst case is well approximated by a one-step opponent perturbation | MAAL: one-step gradient approximation of adversarial opponent actions in the critic target | One-step approximation only estimates the worst case locally; added compute cost                      |
| <i>Adversarial regularization (Chapter 5)</i> |                                                                       |                                    |                                                                     |                                                                                            |                                                                                                       |
| ERNIE                                         | Task return plus a Lipschitz-smoothness penalty on action sensitivity | Before deployment, during training | Reward and transition functions are Lipschitz continuous            | Adversarial regularization term, no adversary trained                                      | Robustness/performance trade-off set by $\lambda$ ; guarantee holds only under the assumed smoothness |



Table 9.1 – continued from previous page

| Method    | Objective                                                                                          | Timing                             | Assumption                                                                                                     | Mechanism                                                                                                    | Failure mode                                                                                                                                                    |
|-----------|----------------------------------------------------------------------------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MIR3      | Task return plus an information-bottleneck penalty between history and action                      | Before deployment, during training | Robustness follows from reduced dependence on teammate behavior, not defending one worst case                  | CLUB-estimated mutual-information-regularized reward, plugged into an existing backbone algorithm            | Only maximizes a lower bound on the true robust objective; $\lambda$ trades return for robustness                                                               |
| SA-MDP    | Task return plus a per-agent penalty on policy divergence under a bounded observation perturbation | Before deployment, during training | The true disturbance stays within the same bounded $\ell_\infty$ neighborhood used during training             | Per-agent policy-consistency regularizer, extended from a single-agent regularizer with no adversary trained | Guarantee holds only within the trained perturbation bound; addresses the observation channel only, not transitions, teammates, or communication                |
| RADIAL-RL | Task return plus a per-agent margin penalty certifying the best action’s rank under perturbation   | Before deployment, during training | Certified Q-value bounds from interval bound propagation hold for every perturbation inside the trained budget | Per-agent interval bound propagation through each QMIX agent network, margin loss added to the TD loss       | Guarantee is exactly as wide as the trained budget; certifies the observation channel only, and bound computation is costlier than a sampled or one-step attack |

Table 9.1 – continued from previous page

| Method                                                         | Objective                                                                              | Timing                                                       | Assumption                                                                        | Mechanism                                                                    | Failure mode                                                                                   |
|----------------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <i>Distributionally robust value factorization (Chapter 5)</i> |                                                                                        |                                                              |                                                                                   |                                                                              |                                                                                                |
| DrIGM                                                          | Minimax return over an ambiguity set of transition models                              | Before deployment; ambiguity set fixed once, during training | True environment lies inside a fixed, pre-specified ambiguity set                 | DrIGM-compliant robust Bellman target, preserves IGM for VDN/QMIX/QTRAN      | A disturbance exceeding the fixed set’s radius is not covered; the set is never updated online |
| <i>Sim-to-real transfer (Chapter 5)</i>                        |                                                                                        |                                                              |                                                                                   |                                                                              |                                                                                                |
| ADR                                                            | Max return against an adversarial, physically-plausible domain-randomization generator | Before deployment, during training in simulation             | Sim-to-real gap is captured by physical-parameter shifts within a plausible range | Minimax game against a Nature Player, KL/MSE-constrained parameter generator | Says nothing about continued drift after deployment begins (Chapter 10)                        |
| PERDRE                                                         | Same as ADR, plus efficient use of domain-randomized experience                        | Before deployment, during training in simulation             | Uniform replay sampling wastes informative transitions from hard environments     | Prioritized replay by TD-error and domain-subset replay frequency            | Same as ADR: no treatment of post-deployment drift (Chapter 10)                                |
| <i>Change-point detection (Chapter 6)</i>                      |                                                                                        |                                                              |                                                                                   |                                                                              |                                                                                                |

Table 9.1 – continued from previous page

| Method                                       | Objective                                                                     | Timing                               | Assumption                                                                                      | Mechanism                                                                                         | Failure mode                                                                                                                                                       |
|----------------------------------------------|-------------------------------------------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BOCPD                                        | Recursively estimate the posterior over run length to localize a change point | After deployment, online             | Piecewise-stationary regimes, exponential-family observations, geometric inter-change intervals | Bayesian recursive run-length posterior update                                                    | Detects a change but has no adaptation mechanism of its own; a wrong hazard or likelihood model delays detection or raises false alarms                            |
| RL-CD                                        | Recognize the current context and reuse the policy already learned for it     | After deployment, online             | Environment switches among a finite, possibly recurring set of stationary contexts              | Library of context-specific models; best-predicting model activated, new one created if none fits | Assumes discrete switching among stationary contexts; continuously drifting dynamics fall outside this design                                                      |
| <i>Sliding-window estimation (Chapter 6)</i> |                                                                               |                                      |                                                                                                 |                                                                                                   |                                                                                                                                                                    |
| SW-UCRL2                                     | Minimize dynamic regret against the sequence of changing optimal policies     | After deployment, online, continuous | A bounded variation budget; the windowed data represents the current environment                | UCRL2-style optimism restricted to a sliding window of recent data                                | A window spanning a change can inflate the empirical MDP's diameter, degrading planning; the optimal window size needs the horizon and variation budget in advance |

Table 9.1 – continued from previous page

| Method                                               | Objective                                                                                               | Timing                                                       | Assumption                                                                                        | Mechanism                                                                           | Failure mode                                                                                                                                           |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| SWUCRL2-CW                                           | Same dynamic-regret objective, corrected for SW-UCRL2’s diameter blow-up                                | After deployment, online, episodic                           | Variation budgets exist; a widening parameter $\eta$ can be chosen to control the extra pessimism | Confidence widening: a fixed floor $\eta$ added to the transition confidence region | Too little widening leaves the diameter problem unresolved; too much inflates regret on its own; the optimal $\eta$ needs the budgets known in advance |
| <i>Policy-parameter-gradient meta-RL (Chapter 7)</i> |                                                                                                         |                                                              |                                                                                                   |                                                                                     |                                                                                                                                                        |
| MAML                                                 | Learn a shared initialization that reaches near-optimal performance after a few gradient steps per task | Amortized meta-training; a few gradient updates at test time | All parameters need inner-loop adaptation; the task distribution is representative                | Bi-level optimization: inner-loop gradient adaptation, outer-loop meta-update       | Full-parameter, second-order adaptation is computationally costly; tasks far from the training distribution are not covered                            |
| <i>Black-box meta-RL (Chapter 7)</i>                 |                                                                                                         |                                                              |                                                                                                   |                                                                                     |                                                                                                                                                        |

Table 9.1 – continued from previous page

| Method                                    | Objective                                                                                            | Timing                                                                                | Assumption                                                                                        | Mechanism                                                                                    | Failure mode                                                                                                                                                                        |
|-------------------------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RL <sup>2</sup>                           | Max expected return across a task distribution, trained with an outer-loop policy-gradient objective | Amortized meta-training; test-time adaptation via activations only, no weight updates | Task-relevant information can be compressed into a single recurrent hidden state                  | Recurrent policy conditioned on the full interaction history (adaptation via input)          | Memory bottleneck in the hidden state; performance depends on similarity to the training task distribution; long-range credit assignment across episodes is hard for recurrent nets |
| <i>Task-inference meta-RL (Chapter 7)</i> |                                                                                                      |                                                                                       |                                                                                                   |                                                                                              |                                                                                                                                                                                     |
| VariBAD                                   | Act optimally with respect to a belief-augmented BAMDP objective (Bayes-optimal exploration)         | Amortized meta-training; online belief update at test time, no gradient steps         | Task variation is captured by unknown reward/transition parameters drawn from a task distribution | RNN encoder produces an approximate posterior belief, trained via ELBO plus the RL objective | Exact BAMDP is intractable and only approximated; relies on on-policy data and a sequential, reconstruction-based encoder                                                           |

Table 9.1 – continued from previous page

| Method | Objective                                                                      | Timing                                                                                               | Assumption                                                                                                      | Mechanism                                                                                                | Failure mode                                                                                                                                      |
|--------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| PEARL  | Same task-inference objective, trained fully off-policy                        | Amortized meta-training (off-policy); online context inference at test time                          | Task is fully specified by its initial-state, transition, and reward components; context is an exchangeable set | Permutation-invariant context encoder producing a latent variable, disentangled from policy optimization | Encoder training signal still flows through the critic; conditioning on one sampled latent discards the uncertainty the belief was meant to carry |
| ELUE   | Same task-inference objective, with encoder training decoupled from the policy | Amortized meta-training (fully off-policy embedding learning); online context inference at test time | Task representation can be learned directly from replay-buffer transitions, independent of the current policy   | Conditional VAE trained on raw transitions, decoupled from the actor/critic loop                         | Shares the family-level limit in Table 8.1: task inference is only as good as the training task distribution’s coverage                           |

---

## 10. Discussion and Future Work

Building the two taxonomies side by side, and comparing them directly in Chapter 8, surfaces gaps that block evaluating robust and adaptive MARL methods on common ground. This is the goal RQ4 sets out to answer, and four gaps stand out as the concrete obstacles between the taxonomy built here and an actual benchmark.

### 10.1. No Shared Benchmark

Robust MARL methods, RARL, M3DDPG, ERNIE, MIR3, DrIGM, are almost universally tested against a fixed, bounded perturbation applied at test time: a fixed attack budget, a fixed domain-randomization range, a fixed adversarial opponent. Adaptive MARL methods, BOCPD, RL-CD, SW-UCRL2, SWUCRL2-CW, are almost universally tested against a schedule of changes that is never adversarial, only assumed piecewise stationary or slowly varying. No environment covered in this thesis applies both at once, a bounded adversarial perturbation layered on top of an ongoing, unbounded drift, in the same multi-agent setting. A method that has only ever been evaluated against one type of disturbance has not actually been tested against the other, no matter how strong its published guarantee looks in isolation.

### 10.2. No Shared Metric

Robust MARL reports worst-case return degradation under a specific, named attack. Adaptive MARL reports dynamic regret or an average dynamic suboptimality gap across a sequence of tasks or environments. These measure genuinely different things, and neither can be read off the other. Comparing a robust method against an adaptive one currently requires re-running both under a metric neither was designed to optimize, which is a large part of why no such comparison exists in the literature surveyed here.

### **10.3. Sim-to-Real Robustness Is Not Post-Deployment Adaptability**

ADR and PERDRE, covered in Chapter 5, close the gap between a simulator and one physical deployment, fixed once training ends. They say nothing about what happens if the physical environment keeps changing after that point, which is exactly the setting the change-point and sliding-window methods of Chapter 6 target. Those methods, in turn, are validated almost entirely in tabular or low-dimensional settings, not in the continuous-control, deep-MARL simulators that sim-to-real work actually targets. No method surveyed in this thesis has been trained with adversarial domain randomization for a real-robot-style deployment and then evaluated for how well it tracks continued drift after that deployment begins.

### **10.4. Multi-Agent Credit Assignment Is Missing From Adaptive Mechanisms**

Change-point detectors and meta-RL task encoders alike generally treat “the environment changed” as a single event, without asking whether the physical environment changed or a specific teammate’s or opponent’s policy update did. Meta Global-to-Local, covered in Chapter 7, is the one method here that engages with multi-agent structure directly, and it does so for task inference, not for attributing a change once training is already over.

### **10.5. The Concrete Next Step**

None of these four gaps are missing algorithms. They are missing evaluation infrastructure. Closing them means building a dynamic, simulated multi-agent environment family that supports both a bounded adversarial perturbation and an unbounded structured drift at once, mapped to a real-world task class such as multi-robot coordination, traffic signal control, or swarm foraging; evaluating with a metric that captures both worst-case degradation and tracking speed in the same number, or the same reported pair of numbers; and reporting results for methods drawn from both sides of Table 8.1, for example DrIGM against SWUCRL2-CW, or M3DDPG against a PEARL-style task encoder meta-trained across an adversarially augmented task distribution, rather than reporting only one side at a time as the current literature does. That is the empirical work this thesis is written to set up, and it is the direction the remainder of this research program takes.



---

## 11. Conclusion

Robust MARL and adaptive MARL answer the same question, how does a policy survive an environment that will not hold still, with two different assumptions about how that environment moves. Reading them as one taxonomy instead of two, as Chapters 3 through 7 of this thesis do, makes the four-axis comparison of Table 8.1 possible, and that comparison is what exposes the gaps in Chapter 10: no shared benchmark, no shared metric, no method tested against a bounded attack and unbounded drift together, and almost no multi-agent credit assignment for what actually caused a change. None of these four gaps require a new algorithm to close. They require a dynamic, simulated, multi-agent benchmark built specifically to test both properties at once, evaluated with a metric that reports both, on methods drawn deliberately from both sides of the comparison built here. That benchmark, not another taxonomy, is the concrete next step, and it is the one this research program takes on next.



---

## A. Extended Method Catalogue: Meta-Reinforcement Learning

Chapter 7 develops meta-reinforcement learning through one curated exemplar per family: MAML for policy-parameter-gradient methods,  $RL^2$  for black-box methods, and VariBAD and PEARL for task-inference methods, the same exemplars Chapter 8 compares directly against the robust MARL taxonomy. This appendix holds the remaining methods each family’s own source material covers, kept in full rather than cut, so that the breadth of the underlying literature stays part of this thesis without letting any one adaptation mechanism dominate the taxonomy’s main narrative. Nothing here is required to follow Chapters 7 through 11; each section below is referenced from the corresponding point in Chapter 7 for a reader who wants the fuller picture.

### A.1. Additional Policy-Parameter-Gradient Methods

Chapter 7 covers MAML as the foundational instance of learning a good initialization. The methods below extend that idea along three further directions: adapting only part of the network instead of all of it, learning the optimization process itself rather than assuming plain gradient descent, correcting known biases in the meta-gradient estimator, and learning the exploration behavior used to collect adaptation data in the first place.

#### A.1.0.1. Almost No Inner Loop (ANIL)

MAML assumes that all policy parameters require adaptation during the inner loop. Raghu et al. [53] questioned this assumption by investigating which network layers actually change during adaptation. Their analysis showed that the feature extraction layers undergo only minor changes, whereas most of the adaptation occurs in the task-specific output layer.

Based on this observation, they proposed Almost No Inner Loop (ANIL), which separates the network parameters into a feature extractor and an output layer,

---

**Algorithm 38** Almost No Inner Loop (ANIL) [53]

---

**Require:** Task distribution  $p(\mathcal{T})$ , inner learning rate  $\alpha$ , outer learning rate  $\beta$

```

1: Initialize  $\theta = (\theta_{\text{body}}, \theta_{\text{head}})$ 
2: while not converged do
3:   Sample a batch of tasks  $\{\mathcal{T}_i\} \sim p(\mathcal{T})$ 
4:   for all tasks  $\mathcal{T}_i$  do
5:     Collect adaptation trajectories using  $\pi_\theta$ 
6:     Update only the output layer  $\theta'_{\text{head}} \leftarrow \theta_{\text{head}} - \alpha \nabla_{\theta_{\text{head}}} \mathcal{L}_{\mathcal{T}_i}$ 
7:     Keep the feature extractor fixed  $\theta'_{\text{body}} \leftarrow \theta_{\text{body}}$ 
8:     Collect evaluation trajectories using  $(\theta'_{\text{body}}, \theta'_{\text{head}})$ 
9:   end for
10:  Update both  $(\theta_{\text{body}}, \theta_{\text{head}})$ 
    using the meta-objective.
11: end while

```

---

$$\theta = (\theta_{\text{body}}, \theta_{\text{head}}).$$

During the inner loop, only the output layer is updated,

$$\theta'_{\text{head}} = \theta_{\text{head}} - \alpha \nabla_{\theta_{\text{head}}} \mathcal{L}_{\mathcal{T}}(\theta_{\text{body}}, \theta_{\text{head}}),$$

while the feature extractor remains fixed,

$$\theta'_{\text{body}} = \theta_{\text{body}}.$$

During the outer loop, both the feature extractor and the output layer are updated using the meta-objective. The complete procedure is summarized in Algorithm 38.

Experiments showed that freezing the feature extractor during adaptation has little impact on performance, indicating that MAML primarily learns transferable representations rather than rapidly adapting all network parameters. These findings suggest that the effectiveness of MAML stems largely from feature reuse, while task-specific adaptation is mainly performed by the output layer.

#### A.1.0.2. Body Only Update (BOIL)

ANIL demonstrated that adapting only the output layer achieves performance comparable to MAML, suggesting that meta-learning primarily benefits from feature reuse. BOIL challenges this interpretation by arguing that rapid adaptation should

instead arise from *representation change*. Rather than reusing a fixed representation, the feature extractor should adapt to the current task while the output layer remains unchanged. [81]

Accordingly, BOIL decomposes the policy parameters into a feature extractor and a policy head,

$$\theta = (\theta_{\text{body}}, \theta_{\text{head}}).$$

During the inner loop, only the feature extractor is updated,

$$\theta'_{\text{body}} = \theta_{\text{body}} - \alpha \nabla_{\theta_{\text{body}}} \mathcal{L}_{\mathcal{T}}(\theta_{\text{body}}, \theta_{\text{head}}),$$

while the policy head remains fixed,

$$\theta'_{\text{head}} = \theta_{\text{head}}.$$

After adaptation, the outer loop updates both the feature extractor and the policy head using the post-adaptation objective across the sampled tasks. Consequently, the feature extractor learns representations that can rapidly move toward task-specific features, whereas the policy head captures knowledge that is shared across tasks. BOIL provides a complementary perspective to ANIL. While ANIL attributes MAML’s success to feature reuse, BOIL shows that adapting the feature extractor can substantially improve generalization, particularly when tasks exhibit significant domain shifts. These findings suggest that the effectiveness of parameter adaptation depends on the similarity between tasks. When tasks share a common representation, reusing features may be sufficient. However, when tasks differ substantially, adapting the representation itself becomes more beneficial.

Algorithm 39 summarizes the BOIL adaptation procedure.

#### A.1.0.3. Fast Context Adaptation via Meta-Learning (CAVIA)

MAML, ANIL, and BOIL differ in which network parameters are adapted during the inner loop. CAVIA takes a different approach by introducing a small set of task-specific context parameters while keeping the policy parameters fixed during adaptation. This substantially reduces the number of trainable parameters and improves scalability, particularly for large neural networks [82]. Specifically, the model parameters are divided into shared parameters  $\theta$  and context parameters  $\phi$ ,

---

**Algorithm 39** BOIL Meta-Training for Reinforcement Learning

---

**Require:** Task distribution  $p(\mathcal{T})$ , inner learning rate  $\alpha$ , outer learning rate  $\beta$

```

1: Initialize  $\theta = (\theta_{\text{body}}, \theta_{\text{head}})$ 
2: while not converged do
3:   Sample a batch of tasks  $\{\mathcal{T}_i\} \sim p(\mathcal{T})$ 
4:   for all tasks  $\mathcal{T}_i$  do
5:     Collect adaptation trajectories  $D_i \sim \pi_\theta$ 
6:     Update only the feature extractor  $\theta'_{\text{body}} \leftarrow \theta_{\text{body}} - \alpha \nabla_{\theta_{\text{body}}} \mathcal{L}_{\mathcal{T}_i}$ 
7:     Keep the policy head fixed  $\theta'_{\text{head}} \leftarrow \theta_{\text{head}}$ 
8:     Collect evaluation trajectories  $D'_i \sim \pi_{(\theta'_{\text{body}}, \theta'_{\text{head}})}$ 
9:   end for
10:  Update  $(\theta_{\text{body}}, \theta_{\text{head}})$  using the meta-objective.
11: end while

```

---

$$(\theta, \phi),$$

where  $\theta$  is shared across all tasks and  $\phi$  is a low-dimensional task-specific context vector. During adaptation, only the context parameters are updated,

$$\phi_i = \phi_0 - \alpha \nabla_{\phi} \mathcal{L}_{\mathcal{T}_i}(f_{\phi_0, \theta}),$$

with the initialization

$$\phi_0 = \mathbf{0}.$$

The shared parameters remain fixed during the inner loop,

$$\theta' = \theta,$$

and are updated only during the outer loop using the post-adaptation objective,

$$\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_i \mathcal{L}_{\mathcal{T}_i}(f_{\phi_i, \theta}).$$

As illustrated in Figure A.1, the context vector is injected into every hidden layer of the policy network, allowing it to modulate the shared representation according to the current task. Consequently, the context vector acts as a task embedding that is learned through gradient descent rather than inferred by a separate encoder. Since

---

**Algorithm 40** CAVIA Meta-Training for Reinforcement Learning [82]

---

**Require:** Task distribution  $p(\mathcal{T})$ , learning rates  $\alpha$  and  $\beta$

- 1: Initialize shared parameters  $\theta$
- 2: Initialize context vector  $\phi_0 = \mathbf{0}$
- 3: **while** not converged **do**
- 4:     Sample a batch of tasks  $\{\mathcal{T}_i\}_{i=1}^N \sim p(\mathcal{T})$
- 5:     **for all** tasks  $\mathcal{T}_i$  **do**
- 6:         Collect adaptation trajectories  $D_i^{\text{train}} \sim \pi_{\phi_0, \theta}$
- 7:         Update only the context parameters  $\phi_i = \phi_0 - \alpha \nabla_{\phi} \mathcal{L}_{\mathcal{T}_i}(\phi_0, \theta)$
- 8:         Collect evaluation trajectories  $D_i^{\text{test}} \sim \pi_{\phi_i, \theta}$
- 9:     **end for**
- 10:     Update the shared parameters  $\theta \leftarrow \theta - \beta \nabla_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{\mathcal{T}_i}(\phi_i, \theta)$
- 11: **end while**
- 12: **return** Shared parameters  $\theta$

---

only a small number of parameters are adapted, CAVIA reduces computational cost, mitigates meta-overfitting, and scales more effectively than methods that update the entire network.

### A.1.1. Learning the Optimization Process

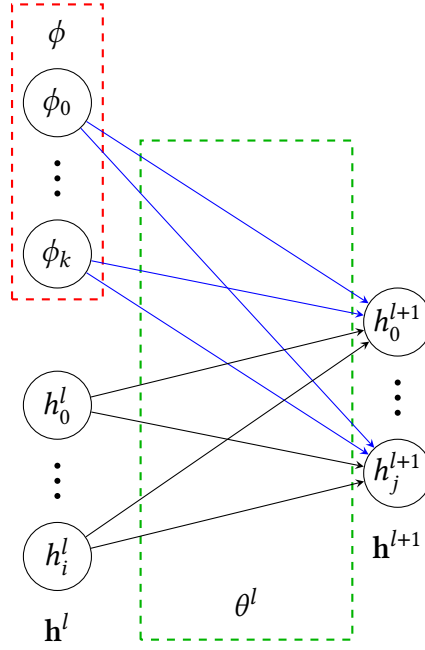
The methods discussed so far assume that the adaptation rule is predefined, typically using gradient descent with a fixed learning rate. Although learning a good initialization significantly improves adaptation, the optimization process itself remains hand-designed. The following methods extend this framework by meta-learning different components of the optimization procedure, including the learning rate, update direction, parameter geometry, and, ultimately, the optimizer itself.

#### A.1.1.1. Meta-SGD

MAML uses a fixed gradient descent update during adaptation, requiring the inner-loop learning rate to be manually selected. Meta-SGD replaces this hand-designed update with a learnable optimization rule by introducing a parameter-specific update vector  $\alpha$  [83].

The adaptation step becomes

$$\theta' = \theta - \alpha \odot \nabla_{\theta} \mathcal{L}_{\mathcal{T}}(\theta),$$



**Figure A.1.** – Overview of the CAVIA architecture. A low-dimensional context vector  $\phi$  is adapted for each task, while the shared parameters  $\theta$  remain fixed during the inner loop. Adapted from [82]



---

**Algorithm 41** Meta-SGD for Reinforcement Learning [83]

---

**Require:** Task distribution  $p(\mathcal{T})$ , outer learning rate  $\beta$

```

1: Initialize policy parameters  $\theta$ 
2: Initialize learnable update vector  $\alpha$ 
3: while not converged do
4:   Sample a batch of tasks  $\{\mathcal{T}_i\} \sim p(\mathcal{T})$ 
5:   for all tasks  $\mathcal{T}_i$  do
6:     Collect adaptation trajectories  $D_i^{\text{train}} \sim \pi_\theta$ 
7:     Compute the policy gradient  $g_i = \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(\theta)$ 
8:     Adapt the policy  $\theta'_i = \theta - \alpha \odot g_i$ 
9:     Collect evaluation trajectories  $D_i^{\text{test}} \sim \pi_{\theta'_i}$ 
10:   end for
11:   Update the meta-parameters  $(\theta, \alpha) \leftarrow (\theta, \alpha) - \beta \nabla_{(\theta, \alpha)} \sum_i \mathcal{L}_{\mathcal{T}_i}(\theta'_i)$ 
12: end while
13: return  $\theta, \alpha$ 

```

---

where  $\alpha$  has the same dimensionality as  $\theta$  and  $\odot$  denotes element-wise multiplication. Compared with MAML, Meta-SGD removes the need to manually tune the inner-loop learning rate while providing a more expressive adaptation rule. However, it still relies on higher-order differentiation and introduces an additional learnable vector with the same dimensionality as the policy parameters.

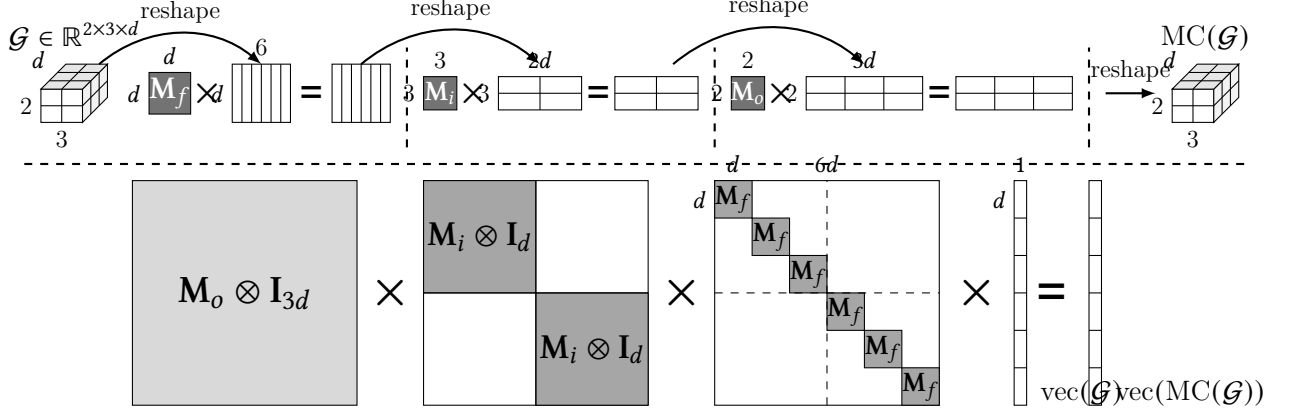
During the outer loop, both the initialization  $\theta$  and the learnable update vector  $\alpha$  are optimized through the meta-objective. During adaptation, the learned vector  $\alpha$  is used to update the policy parameters according to

$$\min_{\theta, \alpha} \sum_{\mathcal{T}_i} L_{\mathcal{T}_i}(\theta - \alpha \nabla_\theta L_{\mathcal{T}_i}(\theta, D_{\mathcal{T}_i}^{\text{train}}), D_{\mathcal{T}_i}^{\text{test}})$$

. As a result, Meta-SGD learns both the initial policy parameters and the optimization rule used for fast adaptation. The complete optimization procedure is summarized in Algorithm 41.

### A.1.1.2. Meta-Curvature

Meta-SGD improves MAML by learning an independent update for each policy parameter. However, it assumes that every parameter is updated independently and therefore ignores correlations between parameters. Consequently, the learned update cannot exploit the underlying geometry of the optimization landscape. Meta-Curvature (MC) addresses this limitation by learning a transformation of the policy gradient before each adaptation step [56].



**Figure A.2.** – Tensor decomposition of the meta-curvature transformation. Instead of learning a full curvature matrix, Meta-Curvature factorizes the transformation into three learnable matrices,  $M_o$ ,  $M_i$ , and  $M_f$ , which model dependencies between output channels, input channels, and filter parameters, respectively. This decomposition enables an efficient approximation of the full gradient transformation while keeping the number of learnable parameters manageable. Adapted from [56].

In MAML, the inner-loop adaptation is performed using standard gradient descent,

$$\theta' = \theta - \alpha \nabla_{\theta} \mathcal{L},$$

which applies the raw gradient directly to the policy parameters. Meta-Curvature instead learns a linear transformation of the gradient,

$$\theta' = \theta - \alpha \underbrace{M \nabla_{\theta} \mathcal{L}}_{\text{transformed gradient}},$$

where  $M$  is the learnable meta-curvature matrix. Rather than following the raw gradient, the policy is updated using a transformed gradient that captures more informative update directions across tasks.

The transformed gradient is defined as

$$MC(g) = Mg,$$

where

$$g = \nabla_{\theta} \mathcal{L}.$$

The matrix  $M$  captures dependencies between policy parameters and learns how gradients should be modified during adaptation. Intuitively, it learns that a particular gradient pattern should be transformed into a more effective update direction. Unlike second-order optimization methods such as Newton’s method or the Natural Gradient, which compute curvature from the local optimization landscape, Meta-Curvature directly learns this transformation from the meta-training tasks to maximize generalization. As a result, the learned transformation can be interpreted as a data-driven approximation of curvature information. Table A.1 summarizes the relationship between these methods.

**Table A.1.** – Comparison of gradient transformations.

| Method           | Update Direction  |
|------------------|-------------------|
| Newton Method    | $H^{-1} \nabla L$ |
| Natural Gradient | $F^{-1} \nabla L$ |
| Meta-Curvature   | $M \nabla L$      |

During meta-training, both the policy initialization  $\theta$  and the meta-curvature matrix  $M$  are optimized jointly,

$$\min_{\theta, M} \sum_i \mathcal{L}_{\text{val}}^{\tau_i}(\theta'_i).$$

The gradient of the validation loss with respect to the meta-curvature matrix is

$$\nabla_M \mathcal{L}_{\text{val}}^{\tau_i}(\theta') = -\alpha \nabla_{\theta'_i} \mathcal{L}_{\text{val}}^{\tau_i}(\theta') \nabla_{\theta} \mathcal{L}_{\text{tr}}^{\tau_i}(\theta)^T,$$

where  $\theta'$  denotes the adapted policy parameters after the inner-loop update. Consequently, the meta-curvature matrix is updated according to

$$M_{\mathcal{T}} = M - \beta \sum_{i=1}^{|\mathcal{T}|} \nabla_M \mathcal{L}_{\text{val}}^{\tau_i}(\theta'_i),$$

which can be rewritten as

$$M_{\mathcal{T}} = M + \alpha\beta \sum_{i=1}^{|\mathcal{T}|} \underbrace{\nabla_{\theta} \mathcal{L}_{\text{val}}^{\tau_i}(\theta'_i)}_{g_{\text{val}}^{(i)}} \underbrace{\nabla_{\theta} \mathcal{L}_{\text{tr}}^{\tau_i}(\theta)}_{(g_{\text{tr}}^{(i)})^T}.$$

This update is the outer product of the validation and training gradients, allowing the learned transformation to capture interactions between policy parameters across multiple tasks.

Although learning a full matrix  $M$  is conceptually simple, it is computationally infeasible for modern neural networks because the number of parameters grows quadratically with the model size. To overcome this limitation, Meta-Curvature factorizes the transformation into three smaller matrices,

$$\text{MC}(G) = G \times_3 M_f \times_2 M_i \times_1 M_o,$$

where  $G \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times d}$  denotes the gradient tensor,  $M_o \in \mathbb{R}^{C_{\text{out}} \times C_{\text{out}}}$  models dependencies between output channels,  $M_i \in \mathbb{R}^{C_{\text{in}} \times C_{\text{in}}}$  models dependencies between input channels, and  $M_f \in \mathbb{R}^{d \times d}$  models dependencies between parameters within each filter. The operator  $\times_n$  denotes the  $n$ -mode tensor product, which is described in AppendixE

The tensor decomposition can be expressed equivalently using Kronecker products,

$$\begin{aligned} \widehat{M}_o &= M_o \otimes I_{C_{\text{in}}} \otimes I_d, \\ \widehat{M}_i &= I_{C_{\text{out}}} \otimes M_i \otimes I_d, \\ \widehat{M}_f &= I_{C_{\text{out}}} \otimes I_{C_{\text{in}}} \otimes M_f, \end{aligned}$$

where

$$\widehat{M}_o, \widehat{M}_i, \widehat{M}_f \in \mathbb{R}^{C_{\text{out}} C_{\text{in}} d \times C_{\text{out}} C_{\text{in}} d}.$$

The resulting meta-curvature matrix is

$$M_{mc} = \widehat{M}_o \widehat{M}_i \widehat{M}_f,$$

which transforms the vectorized gradient according to

$$\text{vec}(\text{MC}(G)) = M_{mc} \text{vec}(G).$$

---

**Algorithm 42** Training MAML with Meta-Curvature for Reinforcement Learning [56]

---

**Require:** Task distribution  $p(\mathcal{T})$

**Require:** Inner learning rate  $\alpha$

**Require:** Outer learning rate  $\beta$

```

1: Initialize policy parameters  $\theta$ 
2: Initialize  $M_o \leftarrow I$ ,  $M_i \leftarrow I$ ,  $M_f \leftarrow I$ 
3: while not converged do
4:   Sample a batch of tasks  $\{\mathcal{T}_i\} \sim p(\mathcal{T})$ 
5:   for all tasks  $\mathcal{T}_i$  do
6:     Compute the training gradient  $g_i \leftarrow \nabla_{\theta} \mathcal{L}_{\text{tr}}^{\mathcal{T}_i}(\theta)$ 
7:     Transform the gradient  $g_i^{\text{MC}} \leftarrow M_{\text{mc}} g_i$ 
8:     Perform one inner-loop update  $\theta'_i \leftarrow \theta - \alpha g_i^{\text{MC}}$ 
9:   end for
10:  Update the policy parameters  $\theta \leftarrow \text{Adam}(\theta, \beta, \nabla_{\theta} \sum_i \mathcal{L}_{\text{val}}^{\mathcal{T}_i}(\theta'_i))$ 
11:  Update the output-channel curvature  $M_o \leftarrow \text{Adam}(M_o, \beta, \nabla_{M_o} \sum_i \mathcal{L}_{\text{val}}^{\mathcal{T}_i}(\theta'_i))$ 
12:  Update the input-channel curvature  $M_i \leftarrow \text{Adam}(M_i, \beta, \nabla_{M_i} \sum_i \mathcal{L}_{\text{val}}^{\mathcal{T}_i}(\theta'_i))$ 
13:  Update the filter curvature  $M_f \leftarrow \text{Adam}(M_f, \beta, \nabla_{M_f} \sum_i \mathcal{L}_{\text{val}}^{\mathcal{T}_i}(\theta'_i))$ 
14:  calculate  $M_{\text{mc}}$ 

$$\widehat{M}_o = M_o \otimes I_{\text{Cin}} \otimes I_d, \quad \widehat{M}_i = I_{\text{Cout}} \otimes M_i \otimes I_d, \quad \widehat{M}_f = I_{\text{Cout}} \otimes I_{\text{Cin}} \otimes M_f,$$

15:  
$$M_{\text{mc}} = \widehat{M}_o \widehat{M}_i \widehat{M}_f,$$

16: end while
17: return  $\theta, M_{\text{mc}}$ 

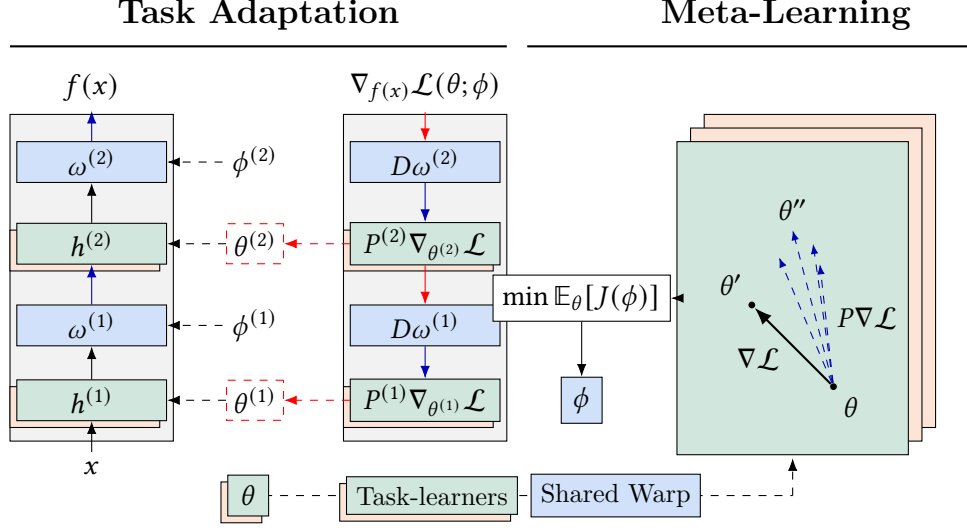
```

---

As illustrated in Figure A.2, this decomposition provides an efficient approximation of the full curvature transformation while maintaining a tractable number of learnable parameters. During the outer loop, the three matrices are optimized independently using automatic differentiation,

$$\begin{aligned}
M_o &\leftarrow \text{Adam}(M_o, \nabla_{M_o} \mathcal{L}_{\text{val}}), \\
M_i &\leftarrow \text{Adam}(M_i, \nabla_{M_i} \mathcal{L}_{\text{val}}), \\
M_f &\leftarrow \text{Adam}(M_f, \nabla_{M_f} \mathcal{L}_{\text{val}}).
\end{aligned}$$

The complete optimization procedure is summarized in Algorithm 42.



**Figure A.3.** – WarpGrad architecture. Trainable warp layers are inserted between the original network layers to learn a shared parameter transformation. During adaptation, task-specific learners update the model parameters, while the shared warp layers transform the gradients through their Jacobians, producing a learned preconditioning matrix. Adapted from [57].

### A.1.1.3. Meta-Learning with Warped Gradient Descent

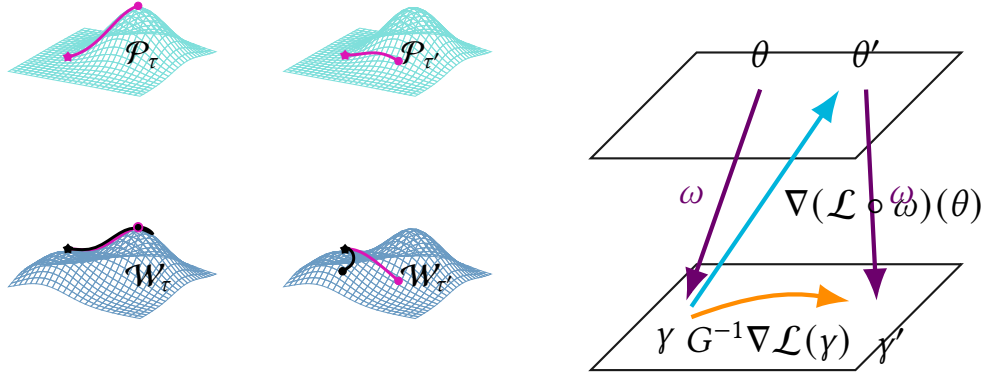
Meta-SGD and Meta-Curvature improve adaptation by learning how the policy gradient should be updated. However, both methods still perform optimization directly in the original parameter space. Consequently, the effectiveness of the update remains constrained by the geometry of that space. Meta-Learning with Warped Gradient Descent (WarpGrad) addresses this limitation by learning a parameter transformation that changes the geometry in which optimization is performed rather than directly modifying the optimization rule [57].

Gradient-based meta-learning methods typically perform task adaptation using

$$\theta \leftarrow \theta - \alpha \nabla \mathcal{L}(\theta),$$

while meta-learning focuses on learning a suitable initialization or optimization hyperparameters. WarpGrad instead learns a parameterized preconditioning matrix,

$$\theta \leftarrow \theta - \alpha P(\theta; \phi) \nabla \mathcal{L}(\theta),$$



**Figure A.4.** – Geometric interpretation of WarpGrad. The warp transformation maps the original parameter space  $\theta$  to a warped space  $\gamma$ , where gradient descent follows the geometry induced by the learned transformation. The update is then projected back to the original parameter space through the Jacobian of the warp transformation. Adapted from [57].

where  $P(\theta; \phi)$  is a learnable transformation parameterized by the meta-parameters  $\phi$ . Instead of learning the initialization, WarpGrad learns how gradients should behave during optimization by modifying the geometry of the parameter space. Standard gradient descent performs optimization in Euclidean space, whereas WarpGrad learns a geometry that better supports rapid adaptation across tasks.

To construct this transformation, WarpGrad inserts trainable warp layers between the layers of the original network,

$$\hat{f} = \mathbf{w}^{(L)} \circ \mathbf{h}^{(L)} \circ \dots \circ \mathbf{w}^{(1)} \circ \mathbf{h}^{(1)},$$

where  $\mathbf{h}^{(l)}$  denotes the original network layer with parameters  $\theta^{(l)}$ , and  $\mathbf{w}^{(l)}$  represents the corresponding warp layer. As illustrated in Figure A.3, the warp layers remain shared across all tasks, while the task-specific learner updates only the model parameters. During backpropagation, the warp layers transform the gradients through their Jacobians, yielding an implicit preconditioning matrix,

$$P(\theta) \nabla \mathcal{L}.$$

When the warp transformation is linear, the resulting preconditioning matrix is block diagonal. Nonlinear warp layers produce a data-dependent transformation that captures richer optimization geometry.

From a geometric perspective, WarpGrad defines a reparameterization of each network layer,

$$\gamma^{(l)} = \Omega^{(l)}(\theta^{(l)}; \phi^{(l)}),$$

where  $\Omega^{(l)}$  denotes the learnable warp transformation. Instead of minimizing the loss directly in the original parameter space, optimization is performed over the warped parameters,

$$\mathcal{L}(\Omega(\theta)).$$

Applying the chain rule gives

$$\Delta\theta^{(l)} = \nabla_{\theta^{(l)}}(L \circ \Omega) = \underbrace{J_{\Omega^{(l)}}(\theta^{(l)})^T}_{\text{Jacobian}} \nabla_{\gamma^{(l)}} \mathcal{L}_{\text{task}}^{\tau}(\gamma^{(l)}),$$

where

$$J_{\Omega^{(l)}} = \frac{\partial \gamma^{(l)}}{\partial \theta^{(l)}}$$

is the Jacobian of the warp transformation. The optimization is performed in the warped parameter space,

$$\Delta\gamma = G^{-1} \nabla_{\gamma} \mathcal{L}(\gamma),$$

where

$$G^{-1} = J_{\Omega}(\theta) J_{\Omega}(\theta)^T$$

defines the induced geometry of the warped space. Figure A.4 illustrates this interpretation. Rather than following the Euclidean gradient, optimization proceeds along directions determined by the learned geometry.

Unlike MAML, WarpGrad does not backpropagate through the entire adaptation trajectory. MAML optimizes

$$L(\theta_K),$$

where



$$\theta_K = \underbrace{U(U(\cdots U(\theta_0)))}_{K \text{ updates}}, \quad \theta_0 \sim p(\theta|\phi).$$

Instead of differentiating through every optimization step, WarpGrad treats every parameter visited during training as a useful meta-training sample and independently optimizes the warp parameters at each sampled point. The resulting objective is

$$\hat{L}(\phi) = \sum_{\tau \sim p(\tau)} \sum_{\theta^\tau \sim p(\theta|\tau)} \mathcal{L}_{\text{meta}}^\tau (\perp [\theta^\tau - \alpha \nabla \mathcal{L}_{\text{task}}^\tau(\theta^\tau; \phi)]; \phi),$$

where  $\perp$  denotes the stop-gradient operator. The complete meta-objective combines optimization geometry and initialization learning,

$$J(\phi, \theta_0) = \hat{L}(\phi) + \lambda C(\theta_0),$$

where

$$C(\theta_0) = \sum_{\tau} \mathcal{L}_{\text{meta}}(\theta_K^\tau).$$

Consequently, the warp parameters  $\phi$  learn the optimization geometry, whereas  $\theta_0$  learns a favorable initialization. Since WarpGrad does not require backpropagation through the full adaptation trajectory, it scales to substantially longer adaptation horizons while retaining the convergence properties of gradient descent. The online meta-training procedure is summarized in Algorithm 43: warp and initialization gradients are accumulated across a batch of tasks and applied after every batch. An offline variant follows the identical inner-loop adaptation step but, instead of updating immediately, appends each visited  $(\tau, \theta_k^\tau)$  pair to a replay buffer  $\mathcal{B}$  and accumulates  $g_\phi$  and  $g_{\theta_0}$  over  $\eta$  buffered pairs before applying an update; this trades immediacy for the ability to revisit and re-weight past adaptation trajectories, at the cost of the extra buffer-management bookkeeping. Both variants use the same gradient definitions and the same update rule; only when the update is applied, and against how large a batch, differs.

---

**Algorithm 43** WarpGrad Online Meta-Training [57]

---

**Require:** Task distribution  $p(\tau)$

**Require:** Learning rates  $\alpha, \beta, \lambda$

```

1: Initialize warp parameters  $\phi$ 
2: Initialize policy initialization  $\theta_0$ 
3: while not converged do
4:   Sample a mini-batch of tasks  $\mathcal{T} \sim p(\tau)$ 
5:    $g_\phi \leftarrow 0, \quad g_{\theta_0} \leftarrow 0$ 
6:   for all  $\tau \in \mathcal{T}$  do
7:     Sample task initialization  $\theta_0^\tau \sim p(\theta_0|\tau)$ 
8:     for  $k = 0, \dots, K_\tau - 1$  do
9:       Perform one task adaptation step  $\theta_{k+1}^\tau \leftarrow \theta_k^\tau - \alpha \nabla \mathcal{L}_{\text{task}}^\tau(\theta_k^\tau; \phi)$ 
10:      Accumulate warp gradient  $g_\phi \leftarrow g_\phi + \nabla \mathcal{L}(\phi; \theta_k^\tau)$ 
11:      Accumulate initialization gradient  $g_{\theta_0} \leftarrow g_{\theta_0} + \nabla C(\theta_0^\tau; \theta_{0:k}^\tau)$ 
12:    end for
13:  end for
14:  Update warp parameters  $\phi \leftarrow \phi - \beta g_\phi$ 
15:  Update initialization  $\theta_0 \leftarrow \theta_0 - \lambda g_{\theta_0}$ 
16: end while
17: return  $\phi, \theta_0$ 

```

---

### A.1.2. Learned Optimizers

The methods discussed so far improve gradient-based adaptation by learning the initialization, learning rates, or optimization geometry. However, they still rely on a handcrafted optimization rule such as gradient descent or Adam. Learned optimizers take a different approach by replacing the predefined update rule with a trainable model that learns how the policy parameters should be updated.

Most optimization algorithms follow the iterative update

$$\theta_{t+1} = \theta_t + \Delta\theta_t,$$

where the update  $\Delta\theta_t$  is determined by a predefined optimization rule. For example, stochastic gradient descent computes

$$\Delta\theta_t = -\alpha \nabla_{\theta} \mathcal{L}(\theta_t),$$

while momentum and Adam additionally exploit information from previous optimization steps. Learned optimizers instead parameterize the update as

$$\Delta\theta_t = f_\psi(s_t),$$

where  $f_\psi$  is a neural network with parameters  $\psi$  and  $s_t$  represents the current optimization state. Depending on the method, the optimization state may include the current gradient, previous updates, loss values, or other optimization statistics. Rather than following manually designed rules, the optimizer learns an update strategy from experience across a distribution of optimization tasks.

Although these methods share the same objective, they differ in how the optimizer is trained. Early work learns the update rule directly through gradient-based meta-learning. Later methods formulate optimization as a sequential decision-making problem and train the optimizer using reinforcement learning. More recent approaches improve generalization by learning scale-invariant optimization policies that transfer across different models, datasets, and tasks. The following sections describe these developments in chronological order.

#### A.1.2.1. Learning to Learn by Gradient Descent

The first learned optimizer was proposed by Andrychowicz et al. [84], who replaced the handcrafted optimization rule with a recurrent neural network. Instead of defining the parameter update explicitly, the optimizer learns an update function directly from optimization experience. This allows the optimizer to automatically discover behaviors such as adaptive learning rates, momentum, and damping without manually specifying these mechanisms.

Rather than using the standard gradient descent update,

$$\Delta\theta_t = -\alpha g_t,$$

where

$$g_t = \nabla_{\theta} \mathcal{L}(\theta_t),$$

the parameter update is predicted by an LSTM,

$$\Delta\theta_t = f_\psi(g_t, h_{t-1}),$$

where  $f_\psi$  denotes the learned optimizer,  $\psi$  are its parameters, and  $h_{t-1}$  is the hidden state that summarizes the optimization history. The model parameters are then updated according to

---

**Algorithm 44** Learning to Learn by Gradient Descent [84]

---

**Require:** Task distribution  $p(\mathcal{T})$

**Require:** Outer learning rate  $\beta$

```

1: Initialize optimizer parameters  $\psi$ 
2: Initialize policy parameters  $\theta_0$ 
3: repeat
4:   Sample a mini-batch of tasks  $\mathcal{T} \sim p(\mathcal{T})$ 
5:   for all tasks  $\mathcal{T}_i$  do
6:     Initialize hidden state  $h_0 \leftarrow 0$ 
7:     for  $k = 0, \dots, K - 1$  do
8:       Compute the policy gradient  $g_k = \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(\theta_k)$ 
9:       Predict the parameter update  $\Delta\theta_k, h_k = f_{\psi}(g_k, h_{k-1})$ 
10:      Update the policy parameters  $\theta_{k+1} = \theta_k + \Delta\theta_k$ 
11:    end for
12:  end for
13:  Update the optimizer parameters  $\psi, \theta_0 \leftarrow \psi, \theta_0 - \beta \nabla_{\psi, \theta_0} \sum_{\mathcal{T}_i} \mathcal{L}_{\mathcal{T}_i}(\theta_K^{\mathcal{T}_i})$ 
14: until convergence
15: return  $\psi, \theta_0$ 

```

---

$$\theta_{t+1} = \theta_t + \Delta\theta_t.$$

By maintaining an internal memory, the LSTM can exploit information from previous optimization steps, enabling it to learn optimization strategies that depend on the history of the training process rather than only the current gradient. As a result, behaviors such as adaptive step sizes and momentum emerge naturally from the learned update rule instead of being explicitly designed. The complete training procedure is summarized in Algorithm 44.

#### A.1.2.2. Learn to Optimize

Although Learning to Learn by Gradient Descent demonstrates that optimization rules can be learned automatically, the optimizer is trained through backpropagation over complete optimization trajectories. Learn to Optimize addresses this limitation by formulating optimization as a sequential decision-making problem, allowing the optimizer to be learned using reinforcement learning [85].

The optimization process is modeled as a Markov decision process, where each optimization step corresponds to one interaction with the environment. The current

model parameters define the optimization state, the optimizer predicts the parameter update, and the objective function determines the optimization cost. The correspondence between optimization and reinforcement learning is summarized in Table A.2.

**Table A.2.** – Optimization as a reinforcement learning problem.

| Optimization                          | Reinforcement Learning |
|---------------------------------------|------------------------|
| Model parameters $\mathbf{x}_t$       | State $s_t$            |
| Parameter update $\Delta\mathbf{x}_t$ | Action $\mathbf{a}_t$  |
| Objective value $f(\mathbf{x}_t)$     | Cost $c(s_t)$          |
| Optimization trajectory               | RL trajectory          |

The optimizer is represented as a policy

$$\pi_\psi(s_t) = \Delta\mathbf{x}_t,$$

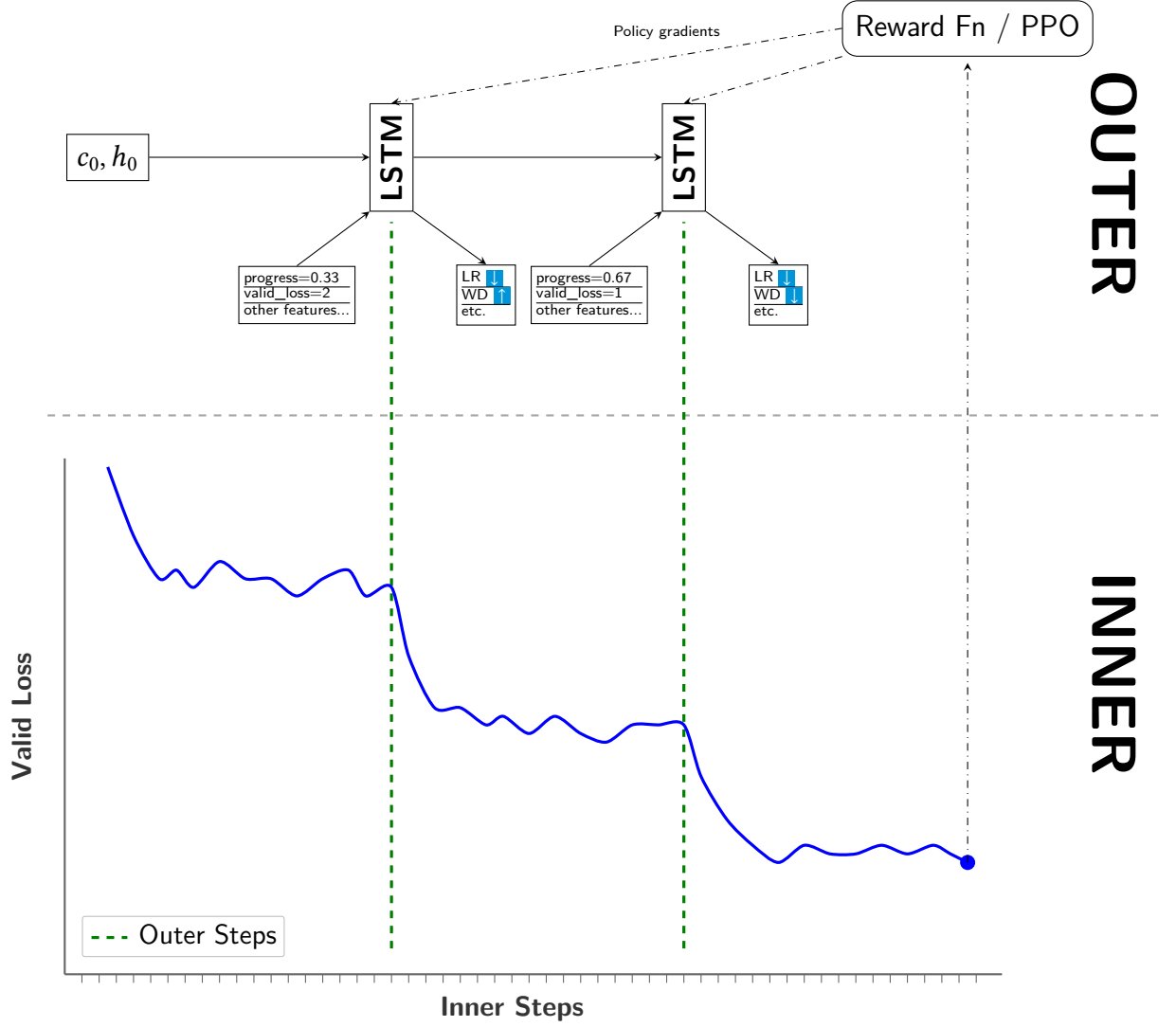
where the state consists of optimization information such as the current gradient, changes in the objective value, and a history of previous optimization steps. Instead of following a predefined update rule, the policy predicts the parameter update that minimizes the cumulative optimization cost,

$$\min_{\pi} \mathbb{E} \left[ \sum_{t=0}^T f(\mathbf{x}_t) \right].$$

To train the optimizer, the authors employ Guided Policy Search (GPS). First, trajectory-centric controllers generate effective optimization trajectories. The policy network is then trained to imitate these trajectories, and both the controllers and the policy are refined iteratively. This approach enables the learned optimizer to discover optimization strategies such as adaptive step sizes, overshooting, recovery, and implicit handling of curvature without explicitly encoding these behaviors. The complete training procedure is summarized in Algorithm 45.

### A.1.2.3. A Generalizable Approach to Learning Optimizers

Although Learn to Optimize formulates optimization as a reinforcement learning problem, the learned optimizer remains specialized to optimization tasks that closely resemble those encountered during training. As a result, it often fails to generalize across different neural network architectures, datasets, and optimization problems [55].



**Figure A.5.** – Training framework of the generalizable learned optimizer. During the inner loop, the policy observes optimization statistics and predicts relative modifications to the optimizer hyperparameters. The outer loop updates the policy using PPO according to the optimization reward. Adapted from [55].

---

**Algorithm 45** Learn to Optimize using Guided Policy Search [85]

---

**Require:** Distribution of optimization tasks

```

1: Initialize optimizer policy  $\pi_\psi$ 
2: repeat
3:   Sample a batch of optimization tasks
4:   for all tasks do
5:     Initialize model parameters  $x_0$ 
6:     for  $t = 0, \dots, T - 1$  do
7:       Observe optimization state  $s_t$ 
8:       Predict parameter update  $\Delta x_t = \pi_\psi(s_t)$ 
9:       Update model parameters  $x_{t+1} = x_t + \Delta x_t$ 
10:    end for
11:    Generate trajectory-centric demonstrations
12:  end for
13:  Update the optimizer policy using Guided Policy Search
14: until convergence
15: return Learned optimizer  $\pi_\psi$ 

```

---

There are three factors that limit the transferability of previous learned optimizers. First, raw gradients are highly sensitive to parameter scale, making them poor representations across different models. Second, predicting absolute parameter updates tightly couples the learned policy to the optimization problems seen during training. Finally, previous methods optimize the final loss after relatively short optimization trajectories, encouraging specialization rather than general optimization behavior.

To overcome these limitations, the optimizer retains the same reinforcement learning framework introduced in the previous section but introduces three key innovations.

**Scale-invariant state representation.** Instead of providing raw gradients to the policy, the optimizer constructs unitless features from normalized gradients, moving averages, variance estimates, and relative ratios. These features remove the dependence on parameter scale and improve transfer across different optimization problems.

**Relative actions.** Rather than predicting absolute optimizer hyperparameters, the policy predicts relative modifications. For example, the learning rate is updated according to

$$\eta_{t+1} = \eta_t \exp(a_t),$$

**Algorithm 46** Training a Generalizable Learned Optimizer [55]

---

**Require:** Distribution of optimization tasks  $p(\mathcal{T})$   
**Require:** Reinforcement learning policy  $\pi_\phi$

- 1: Initialize policy parameters  $\phi$
- 2: **while** not converged **do**
- 3:   Sample an optimization task  $\mathcal{T} \sim p(\mathcal{T})$
- 4:   Initialize the target neural network
- 5:   **for**  $t = 0, \dots, T - 1$  **do**
- 6:     Compute optimization statistics  $s_t$
- 7:     Construct unitless state features from  $s_t$
- 8:     Predict a relative action  $a_t \sim \pi_\phi(s_t)$
- 9:     Update the optimizer hyperparameters  $h_{t+1} \leftarrow h_t \exp(a_t)$
- 10:    Update the target network using  $\theta_{t+1} = \text{Adam}(\theta_t, h_{t+1})$
- 11:    Observe optimization reward  $r_t$
- 12:   **end for**
- 13:   Update the policy parameters  $\phi \leftarrow \text{RLUpdate}(\phi, \{s_t, a_t, r_t\}_{t=0}^T)$
- 14: **end while**
- 15: **return** Learned optimizer policy  $\pi_\phi$

---

where  $a_t$  is produced by the policy network. Consequently, the optimizer learns whether a hyperparameter should increase or decrease instead of predicting its absolute value, making the learned policy less dependent on the scale of the optimization problem.

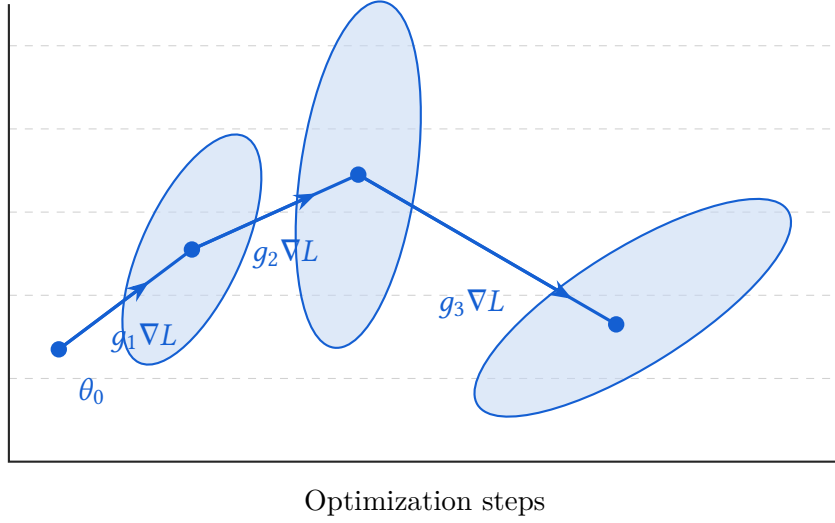
**Long-horizon reinforcement learning objective.** Instead of optimizing only the final loss after a short optimization trajectory, the policy is trained using a reward that encourages sustained optimization improvement over long horizons. This objective enables the optimizer to learn optimization strategies that generalize better across unseen tasks.

Figure A.5 illustrates the resulting framework. During the inner loop, the optimizer trains the target network while collecting optimization statistics. During the outer loop, the reinforcement learning policy is updated using the observed optimization performance. The complete training procedure is summarized in Algorithm 46.

### A.1.3. Improving Meta-Gradient Estimation

Policy parameter gradient methods optimize a meta-objective defined as an expectation over trajectories generated by the current policy. Since the trajectory dis-





**Figure A.6.** – figure illustrates how uncertainty increases with each adaptation step

tribution depends on the policy parameters, estimating the meta-gradient requires differentiating an expectation whose underlying distribution changes throughout the adaptation process. As a result, meta-gradient estimation faces three major challenges: high variance, higher-order differentiation, and estimation bias.

Consider the empirical meta-objective

$$\mathcal{T}_N^{\text{meta}}(\theta, \mathcal{T}) = \mathbb{E}_{\mathcal{T}} [V_{\mathcal{T}}(\theta'_N)],$$

where  $\mathcal{T}$  is a Task and

$$\theta'_N = \theta + \eta \frac{1}{N} \sum_{i=1}^N R(\tau_i, \mathcal{T}) \nabla_{\theta} \log p_{\theta, \mathcal{T}}(\tau_i).$$

Applying the log-derivative trick gives[62]

$$\nabla_{\theta} \mathcal{L}_N^{\text{meta}}(\theta, \mathcal{T}) = \underbrace{\mathbb{E} \left[ V_{\mathcal{T}}(\theta'_N) \sum_{i=1}^N \nabla_{\theta} \log p_{\theta}(\tau_i) \right]}_{\text{Term (i)}} + \underbrace{\mathbb{E} \left[ \left( I + \eta \frac{1}{N} \sum_{i=1}^N R(\tau_i) \nabla_{\theta}^2 \log p_{\theta}(\tau_i) \right) \nabla V_{\mathcal{T}}(\theta'_N) \right]}_{\text{Term (ii)}}.$$

The first term corresponds to the score-function estimator. Although it provides an unbiased estimate of the policy gradient, it accumulates contributions from every adaptation step through

$$\sum_{i=1}^N \nabla_{\theta} \log p_{\theta}(\tau_i),$$

causing the variance of the meta-gradient to grow approximately linearly with the number of inner-loop updates,

$$\text{Var} = \Theta(N).$$

Figure A.6 illustrates this accumulation of sampling noise.

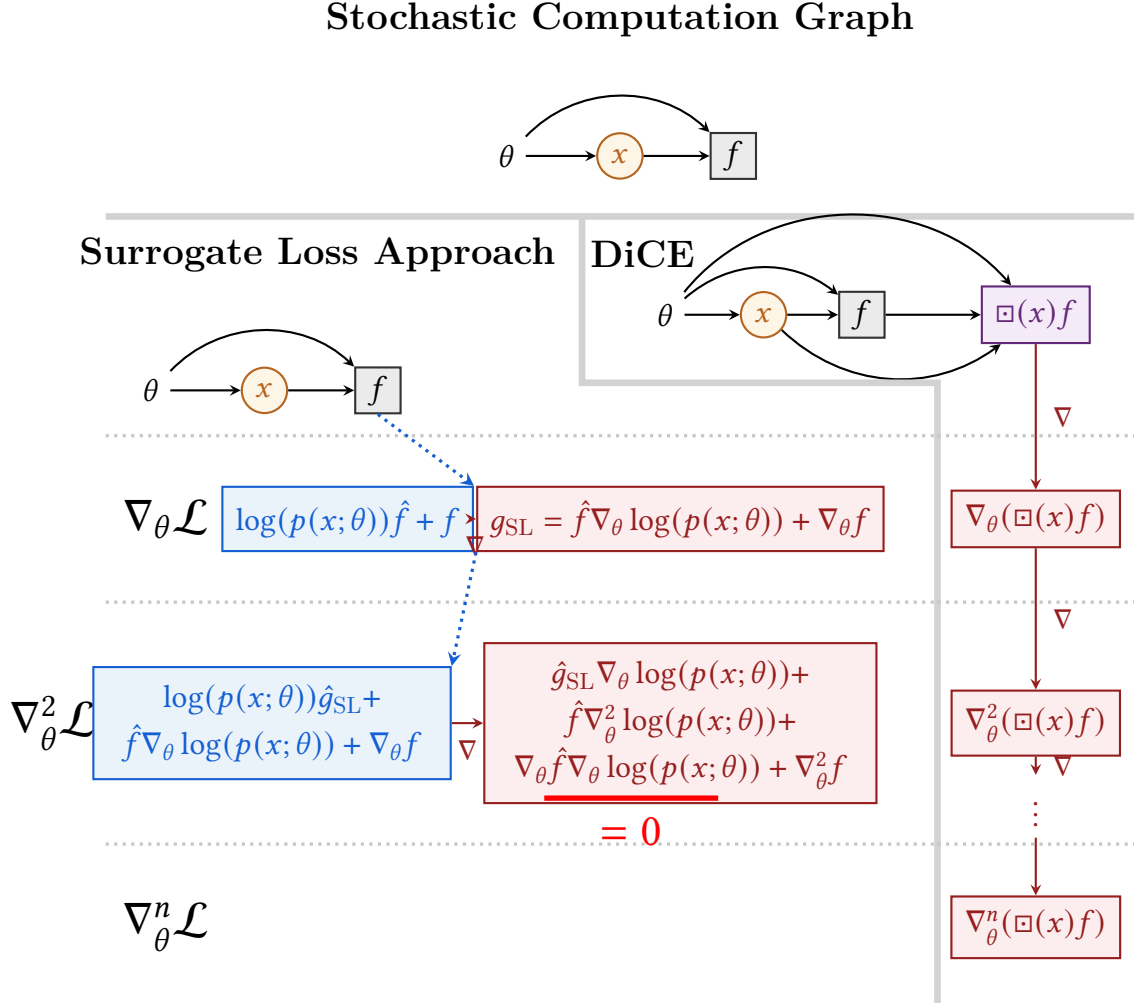
The second term contains the Hessian

$$\nabla_{\theta}^2 \log p_{\theta}(\tau),$$

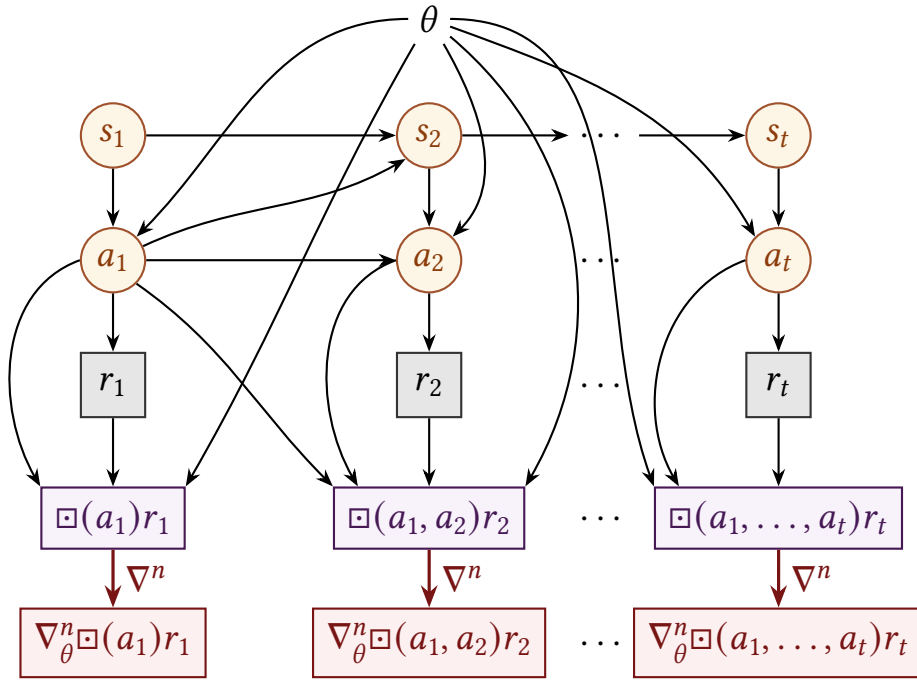
which captures how the adaptation update changes with respect to the meta-parameters. Computing these higher-order derivatives requires differentiating through stochastic sampling operations, which cannot be handled correctly by conventional automatic differentiation. This limitation motivates methods based on stochastic computation graphs, discussed in Appendix F.

In practice, many meta-learning algorithms further simplify the computation of higher-order derivatives or reuse trajectories collected under previous policies to improve computational efficiency. Although these approximations reduce the computational cost and often lower the estimator variance, they no longer optimize the exact meta-objective and therefore introduce bias into the estimated meta-gradient. Consequently, practical meta-gradient estimators must balance three competing objectives: maintaining unbiasedness, reducing variance, and achieving computational efficiency.

The following sections review representative methods that address these challenges. DiCE introduces an unbiased estimator for higher-order gradients in stochastic computation graphs. Loaded DiCE reduces the estimator variance, while NoDICE avoids differentiating through obsolete trajectories. Later methods further investigate the trade-off between bias, variance, and sample efficiency through biased, debiased, and off-policy meta-gradient estimators.



**Figure A.7.** – Comparison between the conventional surrogate loss and the DiCE objective. The surrogate loss computes the correct first-order policy gradient but loses the dependency of the sampled objective on the policy parameters during automatic differentiation, resulting in incorrect higher-order derivatives. DiCE preserves these dependencies by augmenting the computation graph with the MagicBox operator, enabling unbiased differentiation of arbitrary order. Adapted from [86].



**Figure A.8.** – DiCE objective for reinforcement learning. Each reward is weighted by a MagicBox operator that depends on all preceding stochastic actions influencing that reward. This construction preserves the correct higher-order dependencies, allowing automatic differentiation to compute unbiased higher-order policy gradients. Adapted from [86].

### A.1.3.1. DiCE

The previous section showed that higher-order meta-gradients require differentiating through stochastic computation graphs. A common solution approximates the expectation using Monte Carlo sampling and optimizes a surrogate loss (SL) whose first-order gradient matches the desired policy gradient. While this produces a correct first-order estimator, it is incompatible with repeated differentiation using automatic differentiation.

Specifically, the surrogate loss internally treats the Monte Carlo estimate of the objective as a detached quantity,

$$\nabla_{\theta} \hat{f}(x) = 0,$$

where  $\hat{f}(x)$  denotes the sampled objective. Consequently, the dependency of  $f(x, \theta)$  on the policy parameters is removed during backpropagation. As illustrated in Figure A.7, the first-order gradient is computed correctly, but when differentiating a second time the mixed term

$$\nabla_{\theta} f(x; \theta) \nabla_{\theta} \log p(x; \theta)$$

is omitted because  $\hat{f}(x)$  is treated as a constant. As a result, the surrogate loss produces an incorrect Hessian and, more generally, incorrect higher-order derivatives, making it unsuitable for gradient-based meta-learning algorithms that require repeated differentiation through stochastic computation graphs.

DiCE addresses this limitation by constructing a single objective whose expectation matches the original stochastic objective while remaining infinitely differentiable. Unlike the surrogate loss, DiCE preserves the complete dependency structure between stochastic nodes and the objective, allowing automatic differentiation to correctly compute derivatives of arbitrary order. Figure A.7 compares the computation graph of the conventional surrogate loss with the DiCE objective. The reinforcement learning formulation of DiCE is illustrated in Figure A.8, where each reward is multiplied by a MagicBox operator that accumulates the score-function terms of all preceding stochastic actions. To preserve the missing higher-order dependencies, DiCE introduces a special operator, called the *MagicBox* operator, denoted by  $\boxtimes(\mathcal{W})$ . The operator is designed to satisfy two complementary properties. During the forward pass, it behaves as a constant,

$$\boxtimes(\mathcal{W}) = 1,$$

so it does not alter the value of the original objective. During backpropagation, however, it restores the missing score-function terms by satisfying

$$\nabla_{\theta} \boxminus (\mathcal{W}) = \boxminus (\mathcal{W}) \sum_{w \in \mathcal{W}} \nabla_{\theta} \log p(w; \theta),$$

where  $\mathcal{W}$  denotes the set of stochastic nodes that influence the corresponding cost node.

The MagicBox operator is implemented using the stop-gradient operator as

$$\boxminus (\mathcal{W}) = \exp(\tau - \perp (\tau)),$$

where

$$\tau = \sum_{w \in \mathcal{W}} \log p(w; \theta),$$

and  $\perp (\cdot)$  denotes the stop-gradient operation. Since the stop-gradient operator has identical values during the forward pass,

$$\tau - \perp (\tau) = 0,$$

which implies

$$\boxminus (\mathcal{W}) = 1.$$

During backpropagation, however, gradients are propagated only through  $\tau$ , while  $\perp (\tau)$  is treated as a constant. Consequently, the derivative of the MagicBox operator becomes

$$\nabla_{\theta} \boxminus (\mathcal{W}) = \boxminus (\mathcal{W}) \nabla_{\theta} \tau = \boxminus (\mathcal{W}) \sum_{w \in \mathcal{W}} \nabla_{\theta} \log p(w; \theta),$$

thereby recovering the score-function terms that are lost by the conventional surrogate loss.

Using the MagicBox operator, the DiCE objective is constructed as

$$\mathcal{L}_{\text{DiCE}} = \sum_{c \in \mathcal{C}} \boxminus (\mathcal{W}_c) c,$$

where  $\mathcal{W}_c$  denotes the set of stochastic nodes that influence the cost node  $c$ . Unlike the conventional surrogate loss, this objective preserves all stochastic dependencies while leaving the forward computation unchanged. Foerster et al. proved that the resulting objective satisfies

$$\mathbb{E} [\nabla_{\theta}^n \mathcal{L}_{\text{DiCE}}] = \nabla_{\theta}^n \mathbb{E} [\mathcal{L}], \quad \forall n,$$

meaning that every derivative computed using automatic differentiation is an unbiased estimator of the corresponding derivative of the original stochastic objective.

The general DiCE objective can be specialized to reinforcement learning by treating each sampled action as a stochastic node and each reward as a cost node in the stochastic computation graph. Since the reward at time step  $t$  depends only on the actions that precede it, the corresponding MagicBox operator includes exactly those actions,

$$J_{\text{DiCE}} = \sum_t \boxplus(\{a_1, \dots, a_t\}) r_t.$$

Consequently, each reward is associated with the score-function terms of all actions that influence it, preserving the complete dependency structure of the stochastic computation graph. Figure A.8 illustrates this construction. Every reward is multiplied by a MagicBox operator whose stochastic ancestors correspond to all preceding actions, enabling automatic differentiation to compute unbiased higher-order policy gradients.

Similar to REINFORCE, DiCE can incorporate a baseline to reduce the variance of the score-function estimator without changing the expected objective. The variance-reduced objective is defined as

$$\mathcal{L}_{\text{DiCE}} = \sum_{c \in \mathcal{C}} \boxplus(\mathcal{W}_c) c + \sum_w (1 - \boxplus(\{w\})) b_w,$$

where  $b_w$  is any baseline that does not depend on the corresponding stochastic node  $w$ . During the forward pass, the baseline term evaluates to zero because  $\boxplus(\{w\}) = 1$ , leaving the objective unchanged. During backpropagation, however, it contributes the correction term

$$-b_w \nabla_{\theta} \log p(w; \theta),$$

which reduces the variance of the gradient estimator while preserving unbiasedness. Common choices for the baseline include the state-value function, the average return, and the critic in actor–critic methods. Algorithm 47 summarizes the recursive construction of the DiCE objective and the computation of higher-order gradients. By building a single infinitely differentiable objective, DiCE enables automatic differentiation to compute unbiased derivatives of arbitrary order without modifying the underlying optimization algorithm. Although DiCE provides correct higher-order gradient estimates, it inherits the high variance of score-function estimators. Loaded DiCE addresses this limitation by constructing an equivalent estimator with substantially lower variance while preserving the correctness of higher-order derivatives.

### A.1.3.2. Loaded DiCE

Although DiCE computes unbiased higher-order policy gradients, it inherits the high variance of score-function estimators because it directly weights the MagicBox operator by Monte Carlo returns. Furthermore, the original DiCE objective cannot directly exploit common variance reduction techniques based on value functions or generalized advantage estimation (GAE), since introducing these baselines generally breaks the higher-order dependency structure preserved by DiCE [87]. Figure A.9 illustrates several stochastic computation graphs in which latent variables or partial observability introduce additional stochastic dependencies, making higher-order gradient estimation more challenging.

Loaded DiCE addresses these limitations by reformulating the DiCE objective so that value functions and advantage estimators can be incorporated without sacrificing the correctness of higher-order derivatives. The derivation begins with the original DiCE objective,

$$J = \sum_{t=0}^T \gamma^t \boxdot (a_{\leq t}) r_t,$$

where  $\boxdot(a_{\leq t})$  denotes the MagicBox operator applied to all actions preceding reward  $r_t$ .

Instead of weighting each reward independently, Loaded DiCE rewrites the objective using the discounted return

$$R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'},$$



which yields the equivalent telescoping form

$$J = R_0 + \sum_{t=0}^T \gamma^t (\mathbb{Q}(a_{\leq t}) - \mathbb{Q}(a_{< t})) R_t.$$

This reformulation changes the objective from a backward reward decomposition to a forward return formulation while preserving the expected gradient. More importantly, it separates the stochastic dependency introduced at each time step, making it possible to replace Monte Carlo returns with lower-variance value estimates.

Using the Markov property,

$$Q(s_t, a_t) = \mathbb{E} [R_t \mid s_t, a_t],$$

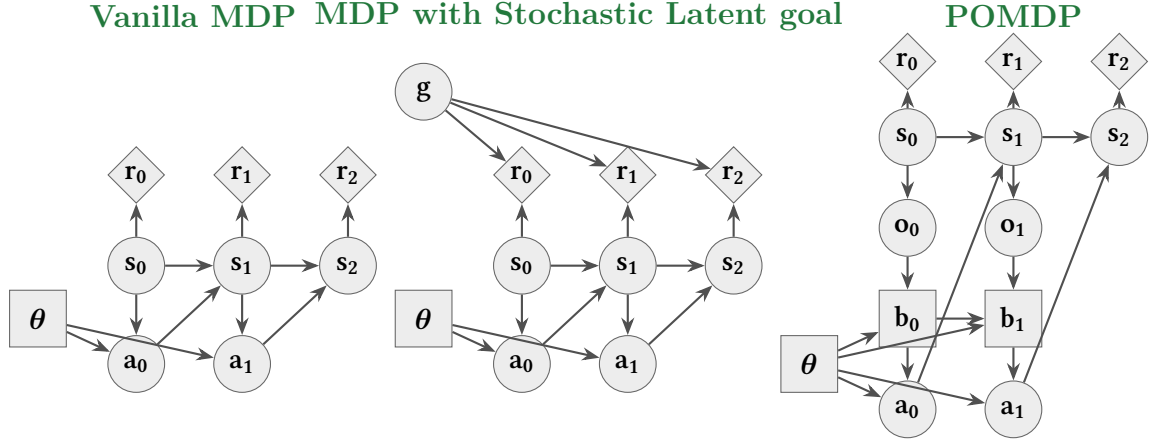
the return can be replaced by the action-value function, giving

$$\mathbb{E}[J] = \mathbb{E} \left[ \sum_{t=0}^T \gamma^t (\mathbb{Q}(a_{\leq t}) - \mathbb{Q}(a_{< t})) Q(s_t, a_t) \right].$$

Finally, Loaded DiCE replaces the action-value function with an arbitrary advantage estimator,

$$J^\diamond = \sum_{t=0}^T \gamma^t (\mathbb{Q}(a_{\leq t}) - \mathbb{Q}(a_{< t})) A(s_t, a_t),$$

allowing any advantage function, including TD( $\lambda$ ) and generalized advantage estimation (GAE), to be incorporated into higher-order policy gradient estimation. Consequently, Loaded DiCE substantially reduces the variance of DiCE while preserving correct higher-order derivatives. Algorithm 48 summarizes the recursive construction of the Loaded DiCE objective. At each time step, the weighted stochastic dependencies are accumulated through the MagicBox operator and multiplied by an arbitrary advantage estimator, enabling critic-based variance reduction while preserving the higher-order gradient properties of DiCE.



**Figure A.9.** – Examples of stochastic computation graphs encountered in reinforcement learning. Besides standard MDPs, latent variables and partial observability introduce additional stochastic dependencies that complicate higher-order gradient estimation. Loaded DiCE extends DiCE to support variance reduction techniques while preserving these dependencies. Adapted from [87].

#### A.1.4. No DICE

Loaded DiCE significantly reduces the variance of higher-order policy gradients by incorporating critic-based estimators while preserving correct higher-order differentiation. However, reducing variance alone does not guarantee an unbiased meta-gradient. During adaptation, each policy update changes the distribution of trajectories collected in subsequent steps, creating dependencies between the meta-parameters, the sampled data, and the policy updates. Existing higher-order estimators do not fully account for these dependencies, introducing bias into the estimated meta-gradient [70].

##### A.1.4.1. Problem Formulation

Let  $\eta$  denote the meta-parameters and  $\theta^k$  denote the policy parameters after the  $k$ -th adaptation step. The meta-parameters are updated according to

$$\eta_{t+1} = \eta_t + \nabla_{\eta} J_K(\eta),$$

where the meta-objective is defined as

$$J_K(\eta) = \sum_{k=0}^K \mathbb{E}_{\{\mathcal{D}^i \sim p(\mathcal{D}^i | \theta^i)\}_{i=0}^{k-1}} \left[ \mathbb{E}_{\tau \sim p(\tau | \theta^k)} [\mathcal{R}(\tau)] \right].$$

Here,

- $K$  is the number of adaptation steps;
- $\eta$  denotes the meta-parameters;
- $\theta^k$  represents the policy parameters after the  $k$ -th adaptation step;
- $\mathcal{D}^i$  denotes the dataset (or batch of trajectories) collected at adaptation step  $i$ ;
- $\tau$  is an evaluation trajectory sampled from the adapted policy;
- $\mathcal{R}(\tau)$  denotes the return obtained from trajectory  $\tau$ .

During adaptation, the policy evolves according to

$$\theta^{i+1} = \Psi(\eta, \theta^i, \mathcal{D}^i),$$

where  $\Psi(\cdot)$  denotes the policy update operator. At each adaptation step, the data are sampled using the current policy,

$$\mathcal{D}^i \sim p(\mathcal{D}^i | \theta^i).$$

Consequently, the data distribution depends implicitly on the meta-parameters through the sequence of policy updates. A change in  $\eta$  modifies the policy parameters  $\theta^i$ , which changes the distribution of the collected data  $\mathcal{D}^i$ . These data are then used to compute the next policy update, creating a chain of dependencies throughout the adaptation process.

Figure A.10 illustrates the corresponding stochastic computation graph. The policy parameters  $\theta_j$  determine the distribution used to collect the dataset  $\mathcal{D}_j$ . The sampled data are then used by the policy update operator  $\Psi$  to produce the next policy parameters  $\theta_{j+1}$ . Repeating this process generates a sequence of policies whose final evaluation produces the return optimized by the meta-objective.

The dependency shown in Figure A.10 creates a long-range credit assignment problem across the adaptation process. Trajectories collected at an early adaptation step influence all subsequent policy updates and therefore affect the final return. An unbiased meta-gradient should account for the contribution of these intermediate trajectories.

The authors identify three major sources of bias in existing meta-gradient estimators:

1. **Missing sampling correction.** Previous methods ignore the dependence of the trajectory distribution on the meta-parameters, producing biased meta-gradients.
2. **Biased Hessian estimation.** Existing approaches differentiate the expected policy objective, whereas the inner-loop optimization is performed using sampled policy gradients, leading to inconsistent Hessian estimates.
3. **Truncation bias.** Long adaptation horizons are commonly handled by truncating the computation graph to reduce computational cost. However, truncation removes dependencies between early adaptation steps and the final objective, introducing additional bias.

#### A.1.4.2. Unbiased Meta-Gradient

To explicitly account for the dependence of the sampled data on the meta-parameters, the meta-objective can be written as

$$J_K(\eta) = \sum_{k=0}^K \int R(\tau) p(\tau | \theta_k) \prod_{i=0}^{k-1} p(D_i | \theta_i) dD_0 \cdots dD_{k-1} d\tau.$$

This formulation makes the complete sampling process explicit. The final return depends not only on the evaluation trajectory  $\tau$ , but also on every intermediate dataset collected during adaptation.

Differentiating the above objective with respect to the meta-parameters yields the unbiased meta-gradient

$$\nabla_{\eta} J_K(\eta) = \sum_{k=0}^K \mathbb{E}_{\substack{\{D^i\}_{i=0}^{k-1} \\ \tau \sim p(\tau | \theta^k)}} \left[ \left( \underbrace{\sum_{j=0}^{k-1} \nabla_{\eta} \theta_j \nabla_{\theta_j} \log p(D_j | \theta_j)}_{\text{Sampling Correction}} + \underbrace{\nabla_{\eta} \theta_k \nabla_{\theta_k} \log p(\tau | \theta_k)}_{\text{Direct Term}} \right) R(\tau) \right].$$

The meta-gradient consists of two complementary components.

The first component,

$$\sum_{j=0}^{k-1} \nabla_{\eta} \theta_j \nabla_{\theta_j} \log p(D_j | \theta_j),$$

is referred to as the *sampling correction*. It accounts for the influence of the meta-parameters on the trajectory distributions encountered during previous adaptation steps. Consequently, it assigns credit to the intermediate datasets that contributed to the final adapted policy.

The second component,

$$\nabla_{\eta} \theta_k \nabla_{\theta_k} \log p(\tau | \theta_k),$$

is the *direct term*. This is the conventional policy-gradient contribution that measures how the meta-parameters influence the final adapted policy through the evaluation trajectory.

The Jacobian

$$\nabla_{\eta} \theta^k \in \mathbb{R}^{d_{\eta} \times d_{\theta}}$$

describes how the policy parameters after the  $k$ -th adaptation step change with respect to the meta-parameters, where  $d_{\eta}$  and  $d_{\theta}$  denote the dimensions of the meta-parameter vector and the policy parameter vector, respectively.

Because each policy update depends on both the previous policy parameters and the sampled dataset, the Jacobian is computed recursively as

$$\nabla_{\eta} \theta^k = \nabla_{\eta} \theta^0 + \sum_{i=0}^{k-1} (\nabla_{\eta} \Psi(\eta, \theta^i, D^i) + \nabla_{\eta} \theta^i \nabla_{\theta^i} \Psi(\eta, \theta^i, D^i)).$$

The first term,

$$\nabla_{\eta} \Psi(\eta, \theta^i, D^i),$$

captures the direct dependence of the update rule on the meta-parameters. The second term,

$$\nabla_{\eta} \theta^i \nabla_{\theta^i} \Psi(\eta, \theta^i, D^i),$$

propagates the influence of the meta-parameters through all previous adaptation steps via the chain rule. Consequently, the Jacobian accumulates the effect of every policy update throughout the adaptation process.

The key insight of NoDICE is that an unbiased meta-gradient must account for both how the meta-parameters modify the policy updates and how they influence the trajectory distributions used to construct those updates. Previous estimators neglect the latter dependency, resulting in biased meta-gradient estimates.

### A.1.4.3. Hessian Estimation

Besides the missing sampling correction, NoDICE identifies another source of bias arising from the estimation of second-order derivatives. During the inner-loop adaptation, the policy is updated using a *sampled* policy gradient computed from the collected trajectories. However, existing higher-order estimators, such as DiCE, differentiate the *expected* policy gradient. Since these two quantities are generally not equivalent, the resulting Hessian is biased.

The expected Hessian of the policy objective is given by

$$\nabla_{\theta}^2 J(\theta) = \mathbb{E} \left[ (\nabla_{\theta} \log \pi(\tau; \theta)) (\nabla_{\theta} \log \pi(\tau; \theta))^T + \nabla_{\theta}^2 \log \pi(\tau; \theta) \right] R(\tau).$$

In contrast, the policy update during adaptation is computed from a finite dataset  $D = \{\tau_1, \dots, \tau_{|D|}\}$ , yielding the sampled Hessian

$$\nabla_{\theta}^2 J(\theta, D) = \frac{1}{|D|} \sum_{\tau \in D} \nabla_{\theta}^2 \log \pi(\tau; \theta) R(\tau).$$

The two Hessians correspond to different optimization procedures. The first differentiates the expected policy objective, whereas the second differentiates the objective evaluated on the sampled trajectories actually used to update the policy.

Consequently, DiCE differentiates the expected policy gradient even though the inner-loop optimization is performed using sampled policy gradients. This mismatch introduces bias into the estimated meta-gradient because the differentiated objective no longer corresponds to the optimization trajectory followed during adaptation.

NoDICE addresses this inconsistency by deriving the meta-gradient directly from the sampled optimization process. Instead of replacing sampled quantities with their expectations, the estimator remains consistent with the trajectory data used during adaptation, producing an unbiased meta-gradient estimator.

Algorithm 49 summarizes the complete NoDICE procedure. The algorithm combines the direct gradient term with the sampling-correction term while remaining consistent with the sampled policy updates used throughout the adaptation process.

#### A.1.4.4. Meta-Gradient Estimation via Off-Policy Evaluation

The previous section showed that existing higher-order meta-gradient estimators suffer from several sources of bias, including missing sampling corrections and inconsistencies between the sampled optimization process and the estimated Hessian. While these methods differ considerably in their derivations, Tang *et al.* [69] demonstrated that they can be understood within a common theoretical framework based on off-policy evaluation (OPE). Rather than differentiating policy-gradient estimators directly, the proposed framework first estimates the value function using an off-policy estimator and then differentiates this estimator to obtain higher-order policy gradients. This perspective unifies existing meta-gradient estimators and provides a principled way to analyze their bias-variance trade-off.

#### A.1.4.5. Problem Formulation

Consider the meta-objective

$$F(\theta) = \mathbb{E}_{\mathcal{T} \sim p(\mathcal{T})} [V^{\pi_{\theta'}}(x_0, g)],$$

where  $\mathcal{T}$  denotes a task sampled from the task distribution  $p(\mathcal{T})$ ,  $x_0$  is the initial state, and

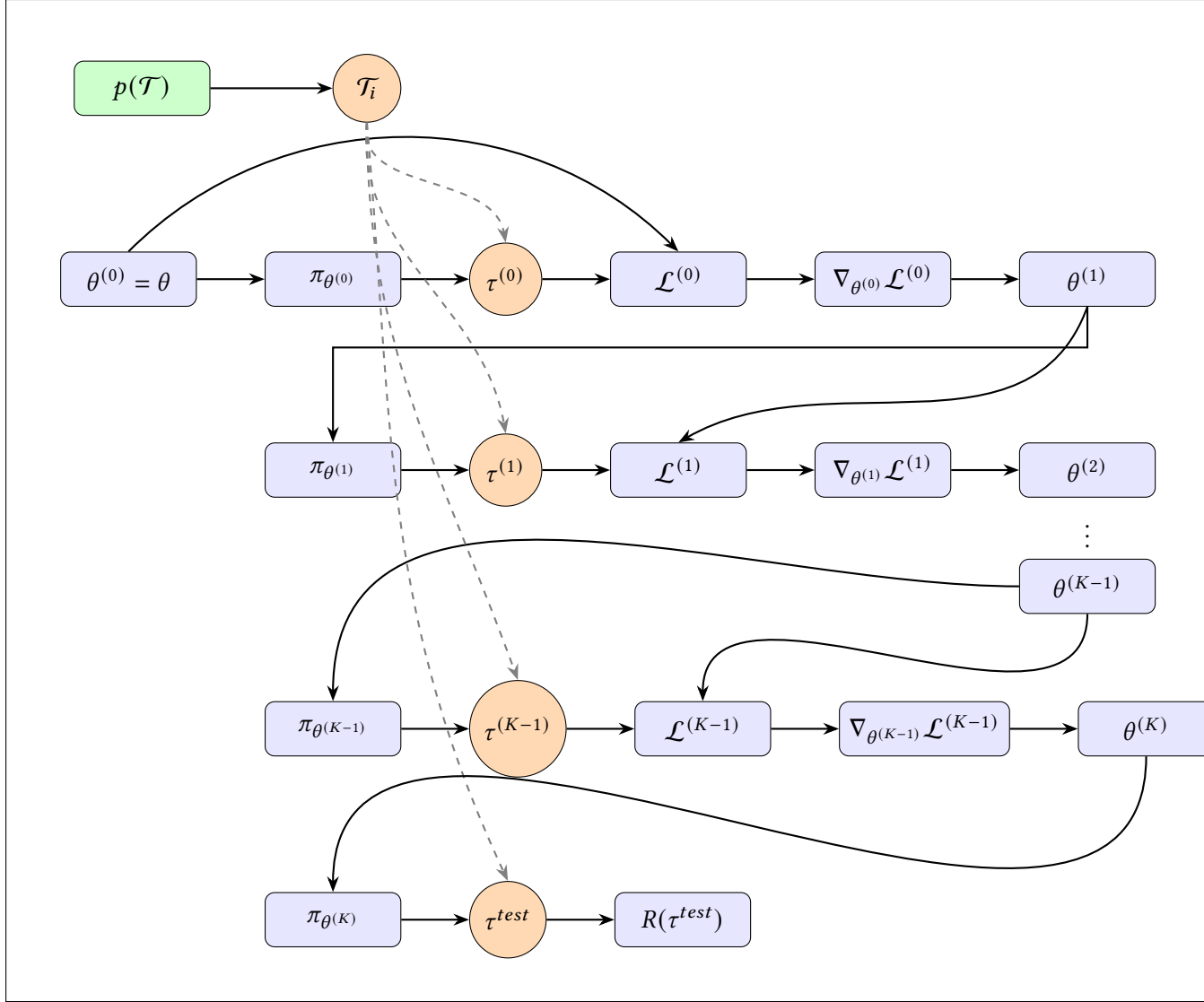
$$\theta' = \theta + U(\theta, \mathcal{T})$$

are the adapted policy parameters after one adaptation step. The update function is defined as

$$U(\theta, \mathcal{T}) = \eta \nabla_{\theta} V^{\pi_{\theta}}(x_0, \mathcal{T}),$$

where  $\eta$  is the inner-loop learning rate.

Differentiating the meta-objective with respect to the policy parameters yields



**Figure A.10.** – Stochastic computation graph for gradient-based meta-reinforcement learning. Starting from the initial policy  $\theta^{(0)}$ , each adaptation step collects a trajectory  $\tau^{(k)}$ , computes the corresponding policy gradient, and updates the policy parameters to  $\theta^{(k+1)}$ . Consequently, the distribution of every trajectory depends on all previous policy updates, creating long-range stochastic dependencies between the meta-parameters, intermediate trajectories, and the final evaluation return  $R(\tau^{\text{test}})$ .



$$\nabla_{\theta} F(\theta) = \mathbb{E}_{\mathcal{T} \sim p(\mathcal{T})} \left[ \underbrace{\nabla_{\theta} \theta'}_{\text{Adaptation Jacobian}} \quad \underbrace{\nabla_{\theta'} V^{\pi_{\theta'}}(x_0, \mathcal{T})}_{\text{Policy Gradient}} \right].$$

The first term describes how the adapted parameters depend on the initial policy. Using the definition of the adaptation operator,

$$\nabla_{\theta} \theta' = I + \eta \nabla_{\theta}^2 V^{\pi_{\theta}}(x_0, \mathcal{T}),$$

where

$$H_{\theta} = \nabla_{\theta}^2 V^{\pi_{\theta}}(x_0, \mathcal{T})$$

is the Hessian of the value function.

Consequently, estimating the meta-gradient requires computing both the policy gradient and the Hessian of the value function.

Unlike supervised learning, however, the value function is defined as

$$V^{\pi_{\theta}}(x_0, \mathcal{T}) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\mathcal{R}(\tau)],$$

where the trajectory distribution

$$p_{\theta}(\tau)$$

depends on the policy parameters. Therefore, differentiating the value function requires differentiating the trajectory distribution itself, making higher-order gradient estimation considerably more challenging than in supervised learning.

A straightforward estimator computes

$$\widehat{\nabla_{\theta} F} = (I + \eta \widehat{H}) \widehat{g},$$

where

$$\widehat{H} \approx \nabla_{\theta}^2 V^{\pi_{\theta}}, \quad \widehat{g} \approx \nabla_{\theta} V^{\pi_{\theta}}.$$

Although both estimators may individually be unbiased,

$$\mathbb{E}[\widehat{H}] = H, \quad \mathbb{E}[\widehat{g}] = g,$$

their product is generally not,

$$\mathbb{E}[\widehat{H}\widehat{g}] \neq Hg.$$

This observation reveals an important limitation of existing higher-order meta-gradient estimators. Even if the Hessian and policy-gradient estimators are individually unbiased, multiplying two stochastic estimators introduces additional bias into the resulting meta-gradient. Consequently, differentiating policy-gradient estimators directly does not necessarily produce an unbiased estimate of the desired higher-order derivative.

#### A.1.4.6. Off-Policy Evaluation Framework

Instead of differentiating a policy-gradient estimator, Tang *et al.* propose differentiating an off-policy estimator of the value function. The framework consists of two steps.

First, construct an off-policy estimator

$$\widehat{V}^{\pi_\theta}(x) \approx V^{\pi_\theta}(x),$$

using trajectories generated by a behavior policy.

Second, differentiate the value estimator to obtain higher-order derivatives,

$$\nabla_\theta^m \widehat{V}^{\pi_\theta}(x) \approx \nabla_\theta^m V^{\pi_\theta}(x).$$

The central theoretical result states that if the value estimator is unbiased,

$$\mathbb{E}_\mu \left[ \widehat{V}^{\pi_\theta}(x) \right] = V^{\pi_\theta}(x),$$

then every derivative is also unbiased,

$$\boxed{\mathbb{E}_\mu \left[ \nabla_\theta^m \widehat{V}^{\pi_\theta}(x) \right] = \nabla_\theta^m V^{\pi_\theta}(x).}$$

Here,

XL

- $\mu$  denotes the fixed behavior policy used to collect trajectories;
- $\pi_\theta$  denotes the target policy being optimized;
- $m$  is the derivative order.

This theorem shifts the problem of higher-order meta-gradient estimation from differentiating stochastic policy-gradient estimators to constructing accurate off-policy value estimators. Different choices of off-policy estimators lead to different bias–variance trade-offs while remaining within the same theoretical framework.

#### A.1.4.7. Off-Policy Value Estimation

The unified framework relies on constructing an off-policy estimator of the value function before computing higher-order derivatives. The simplest unbiased estimator is obtained using importance sampling, which reweights trajectories collected under a behavior policy so that they represent the target policy.

**Importance Sampling** Let  $\mu$  denote the behavior policy used to collect trajectories and let  $\pi_\theta$  denote the target policy. Their discrepancy is corrected using the importance ratio

$$\rho_t^\theta = \frac{\pi_\theta(a_t|x_t)}{\mu(a_t|x_t)}.$$

Using these importance weights, the off-policy value estimator is

$$\widehat{V}_{\text{IS}}^{\pi_\theta}(x_0) = \sum_{t=0}^{\infty} \gamma^t \left( \prod_{s=0}^t \rho_s^\theta \right) r_t.$$

Since the trajectory probability factorizes over time, the trajectory importance ratio becomes

$$\frac{p_{\pi_\theta}(\tau)}{p_\mu(\tau)} = \prod_{t=0}^T \frac{\pi_\theta(a_t|x_t)}{\mu(a_t|x_t)} = \prod_{t=0}^T \rho_t^\theta.$$

Importance sampling therefore provides an unbiased estimate of the value function using trajectories generated by a different policy.

However, the product of importance ratios can grow or shrink exponentially with the trajectory length. When some importance ratios become significantly larger than one,

$$\rho_t^\theta \gg 1,$$

the trajectory weight becomes highly unstable, resulting in high-variance estimates of the value function. Since higher-order policy gradients require differentiating these importance weights, the variance is further amplified when estimating gradients and Hessians.

Consequently, although importance sampling is unbiased, it is often impractical for higher-order meta-gradient estimation because of its poor variance properties.

**Doubly Robust Estimator** To reduce the variance of importance sampling while preserving unbiasedness, the framework adopts a doubly robust (DR) estimator. The key idea is to introduce a learned action-value function that explains most of the return, while importance sampling is used only to correct the approximation error.

The doubly robust estimator is defined recursively as

$$\widehat{V}_{\text{DR}}^{\pi_\theta}(x_t) = Q(x_t, \pi_\theta(x_t)) + \rho_t^\theta \left( r_t + \gamma \widehat{V}_{\text{DR}}(x_{t+1}) - Q(x_t, a_t) \right),$$

where

- $Q(x_t, a_t)$  is the critic,
- $\rho_t^\theta$  is the importance ratio,
- $r_t + \gamma \widehat{V}_{\text{DR}}(x_{t+1}) - Q(x_t, a_t)$  is the temporal-difference residual.

Unlike importance sampling, the critic explains a large portion of the return, while the importance-weighted correction compensates only for the critic’s prediction error. Consequently, the estimator preserves unbiasedness while substantially reducing variance.

Differentiating the doubly robust estimator yields

$$\nabla_\theta \widehat{V}_{\text{DR}}(x_t) = \nabla_\theta Q(x_t, \pi_\theta(x_t)) + \rho_t^\theta \delta_t \nabla_\theta \log \pi_\theta(a_t | x_t) + \gamma \rho_t^\theta \nabla_\theta \widehat{V}_{\text{DR}}(x_{t+1}),$$

where

$$\delta_t = r_t + \gamma \widehat{V}_{\text{DR}}(x_{t+1}) - Q(x_t, a_t)$$

denotes the temporal-difference error.

Applying automatic differentiation again gives the recursive Hessian

$$\begin{aligned} \nabla_{\theta}^2 \widehat{V}_{\text{DR}}^{\pi_{\theta}}(x_t) &= \rho_t^{\theta} \delta_t \left( \nabla_{\theta}^2 \log \pi_t + \nabla_{\theta} \log \pi_t \nabla_{\theta} \log \pi_t^T \right) \\ &\quad + \gamma \rho_t^{\theta} \nabla_{\theta} \widehat{V}_{\text{DR}}(x_t) \nabla_{\theta} \log \pi_t^T \\ &\quad + \gamma \rho_t^{\theta} \nabla_{\theta} \log \pi_t \nabla_{\theta} \widehat{V}_{\text{DR}}(x_t)^T \\ &\quad + \nabla_{\theta}^2 Q(x_t, \pi_{\theta}(x_t)) + \gamma \rho_t^{\theta} \nabla_{\theta}^2 \widehat{V}_{\text{DR}}(x_{t+1}). \end{aligned}$$

Unlike DiCE, which differentiates policy-gradient estimators directly, the proposed framework differentiates the off-policy value estimator itself. As a result, the same estimator naturally provides both the policy gradient and higher-order derivatives required for meta-gradient optimization. Furthermore, the inclusion of the critic substantially reduces the estimator variance, making the resulting gradients considerably more stable than pure importance sampling. The off-policy evaluation perspective also provides a unified interpretation of several existing higher-order meta-gradient estimators. In particular, DiCE, Loaded DiCE, low-variance curvature (LVC), and Taylor-based policy optimization (TayPO) can all be viewed as different approximations of the same off-policy value estimation framework, differing primarily in how they balance estimation bias and variance.

#### A.1.4.8. A Unified View of Meta-Gradient Estimators

The off-policy evaluation framework provides a unified interpretation of several higher-order meta-gradient estimators. Although DiCE, Loaded DiCE, Low Variance Curvature (LVC), and Taylor Policy Optimization (TayPO) appear to employ different optimization strategies, they can all be viewed as differentiating different approximations of the same off-policy value estimator.

Their main differences lie in how the value function is approximated and, consequently, how they balance estimation bias and variance. Table A.3 summarizes this interpretation.

This perspective reveals that existing methods differ primarily in how they approximate the value function before differentiation. Exact importance sampling produces unbiased estimates but often suffers from prohibitively high variance. Conversely, low-order approximations reduce variance at the cost of introducing bias. Therefore, higher-order meta-gradient estimation can be interpreted as selecting an appropriate

**Table A.3.** – Unified interpretation of higher-order meta-gradient estimators based on off-policy value estimation.

| Method      | Off-policy interpretation                        |
|-------------|--------------------------------------------------|
| DiCE        | Differentiated importance sampling estimator     |
| Loaded DiCE | Importance sampling with variance reduction      |
| LVC         | Low-order Taylor approximation                   |
| TayPO       | Truncated Taylor expansion of the value function |

approximation of the off-policy value function according to the desired bias–variance trade-off.

#### A.1.4.9. Taylor Policy Optimization

The previous sections presented two extremes. Importance sampling provides an unbiased estimate of the value function but exhibits high variance, whereas low-order approximations reduce variance by introducing bias. TayPO bridges these two extremes by expressing the off-policy value function as a Taylor expansion around the behavior policy and truncating the expansion at a desired order.

The value function is decomposed as

$$V^{\pi_\theta}(x_0) = \sum_{k=0}^{\infty} U_k^{\pi_\theta}(x_0),$$

where each term represents a higher-order correction that accounts for the discrepancy between the target policy and the behavior policy.

The  $K$ -th order component is estimated by

$$U_K^{\pi_\theta}(x_0) = \mathbb{E}_\mu \left[ \widehat{U}_K^{\pi_\theta}(x_0) \right],$$

with

$$\widehat{U}_K^{\pi_\theta}(x_0) = \sum_{t_1=0}^{\infty} \sum_{t_2=t_1+1}^{\infty} \cdots \sum_{t_K=t_{K-1}+1}^{\infty} \gamma^{t_K} \left( \prod_{i=1}^K (\rho_{t_i}^\theta - 1) \right) Q^\mu(x_{t_K}, a_{t_K}).$$

Instead of using the complete expansion, TayPO truncates the series after the  $K$ -th term,

**Table A.4.** – Bias–variance trade-off controlled by the Taylor expansion order.

| Taylor order $K$ | Bias | Variance |
|------------------|------|----------|
| Small            | High | Low      |
| Large            | Low  | High     |

$$\widehat{V}_K^{\pi_\theta} = \sum_{k=0}^K \widehat{U}_k.$$

This approximation is referred to as TayPO- $K$ , where the truncation order directly controls the bias–variance trade-off.

The approximation error satisfies

$$V^{\pi_\theta} = V_K^{\pi_\theta} + \mathcal{O}\left(\|\pi_\theta - \mu\|^{K+1}\right),$$

showing that the approximation becomes exact as the truncation order increases. Consequently, the bias of the estimated derivatives is bounded by

$$\left\| \mathbb{E} \left[ \nabla^m \widehat{V}_K \right] - \nabla^m V \right\| \leq \sum_{k=K+1}^{\infty} \|\nabla^m U_k\|.$$

The truncation order therefore provides an explicit mechanism for controlling the bias–variance trade-off. Small values of  $K$  discard many higher-order terms, resulting in larger approximation bias but significantly lower variance. Conversely, increasing  $K$  reduces the approximation bias at the expense of higher estimator variance. Table A.4 summarizes this relationship.

The complete off-policy meta-gradient estimation procedure is summarized in Algorithm 50. The algorithm first estimates the value function using an off-policy estimator, such as importance sampling, the doubly robust estimator, or TayPO, and then differentiates the resulting estimator to obtain the policy gradient and higher-order derivatives required for meta-learning. Algorithm 50 summarizes the unified framework for higher-order meta-gradient estimation. The algorithm first estimates the value function using an off-policy estimator and then obtains higher-order derivatives through automatic differentiation. Different meta-gradient estimators arise from different choices of the off-policy value estimator. For example, importance sampling provides an unbiased but high-variance estimator, the doubly

robust estimator reduces variance by incorporating a critic, and TayPO controls the bias–variance trade-off by truncating the Taylor expansion of the value function. Consequently, seemingly different meta-gradient algorithms can be interpreted as instances of the same off-policy evaluation framework. The off-policy evaluation framework unifies many existing higher-order meta-gradient estimators under a common theoretical perspective. Rather than designing new differentiation rules for each estimator, the framework shifts the focus to constructing accurate value estimators whose derivatives provide the desired policy gradients and Hessians. This perspective not only explains the relationship between DiCE, Loaded DiCE, LVC, and TayPO, but also clarifies how different approximation strategies trade estimation bias for variance while remaining within the same optimization framework.

Although the off-policy evaluation framework provides a unified interpretation of higher-order meta-gradient estimators, it also highlights an inherent bias–variance trade-off. Exact estimators based on importance sampling remain unbiased but often exhibit prohibitively high variance, whereas low-order approximations substantially reduce variance by introducing bias. This observation motivates methods that intentionally accept a controlled amount of bias to obtain more stable and sample-efficient meta-gradient estimates.

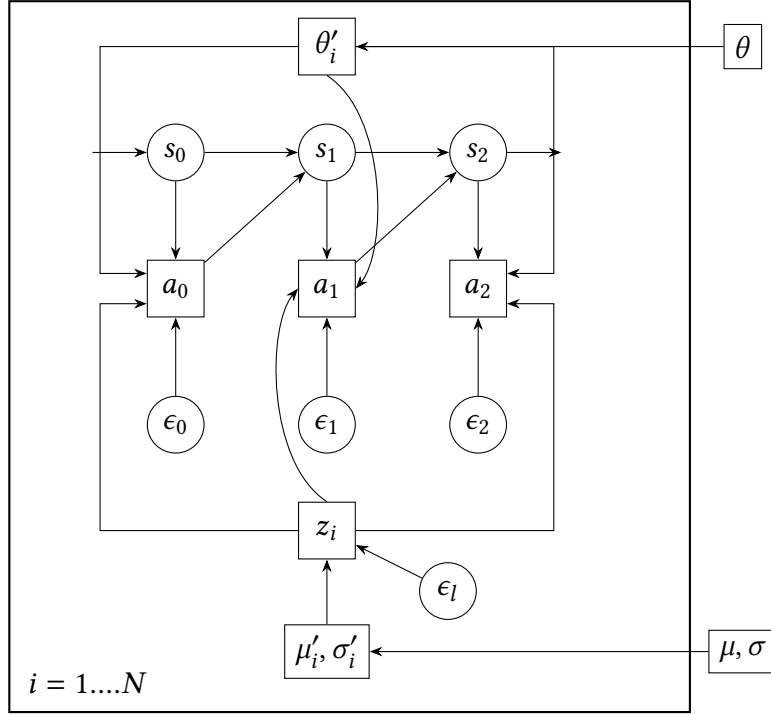
### **A.1.5. Learning Exploration Strategies**

The meta-learning methods discussed so far assume that the adaptation trajectories contain sufficient information to identify the current task. In practice, however, collecting informative trajectories is itself a challenging exploration problem. Standard exploration techniques, such as  $\epsilon$ -greedy policies, Gaussian action noise, or task-agnostic intrinsic rewards, are designed for conventional reinforcement learning and fail to exploit the task distribution available during meta-training. As a result, they often generate random or temporally inconsistent behaviors that provide limited information for rapid adaptation.

Meta-reinforcement learning addresses this limitation by learning exploration as part of the meta-learning process. Rather than optimizing only the post-adaptation policy, these methods also optimize the behavior used before adaptation so that the collected trajectories maximize the effectiveness of subsequent policy updates. Consequently, exploration becomes structured, task-aware, and explicitly optimized to accelerate adaptation on previously unseen tasks.

This section reviews three representative approaches. Meta-Reinforcement Learning of Structured Exploration Strategies (MAESN) introduces latent variables that generate temporally coherent exploration behaviors. E-MAML extends gradient-based





**Figure A.11.** – Overview of MAESN, showing task-specific latent variables  $z_i$  used to drive structured exploration during adaptation[65]

meta-learning by explicitly assigning credit to exploration trajectories according to their contribution to post-adaptation performance.

#### A.1.5.1. Meta-Reinforcement Learning of Structured Exploration Strategies (MAESN)

Meta-Reinforcement Learning of Structured Exploration Strategies (MAESN) addresses the exploration problem by learning a latent space of exploration behaviors. Instead of relying on action-space noise, MAESN conditions the policy on a latent variable that induces temporally coherent exploration strategies. During meta-training, the latent space is optimized jointly with the policy so that sampling different latent variables produces distinct exploration behaviors that can be rapidly adapted to new tasks.

**Structured Exploration** Conventional exploration methods, such as Gaussian action noise, perturb each action independently, resulting in temporally incoherent behavior. For example, an agent may move left at one time step and immediately move

right at the next, preventing systematic exploration of the environment. MAESN instead introduces a latent variable

$$z \sim \mathcal{N}(\mu, \sigma),$$

and conditions the policy on this latent representation,

$$\pi_{\theta}(a|s, z).$$

Rather than directly selecting individual actions, the latent variable determines a consistent exploration mode throughout an episode. Different latent samples therefore correspond to different high-level behaviors, allowing the agent to perform coherent exploration before adapting its policy.

**Latent Space Learning** During meta-training, each training task maintains its own latent distribution,

$$z_i \sim \mathcal{N}(\mu_i, \sigma_i),$$

while all tasks share the policy parameters  $\theta$ . The latent distributions are optimized jointly with the policy by maximizing the expected post-adaptation return while regularizing the latent variables toward a standard Gaussian prior,

$$\max_{\theta, \{\mu_i, \sigma_i\}} \sum_i \mathbb{E} \left[ \sum_t R_i(s_t) \right] - \sum_i D_{\text{KL}} (\mathcal{N}(\mu_i, \sigma_i) \| \mathcal{N}(0, I)).$$

The KL regularization encourages the task-specific latent distributions to remain close to the shared prior, allowing unseen tasks to initialize exploration from

$$z \sim \mathcal{N}(0, I).$$

**Fast Adaptation** For each task, the latent distribution is adapted using policy gradients,

$$\mu'_i = \mu_i + \alpha_\mu \nabla_{\mu_i} J,$$

$$\sigma'_i = \sigma_i + \alpha_\sigma \nabla_{\sigma_i} J,$$

where  $J$  denotes the expected return. Optionally, the policy parameters can also be updated,

$$\theta' = \theta + \alpha_\theta \nabla_\theta J.$$

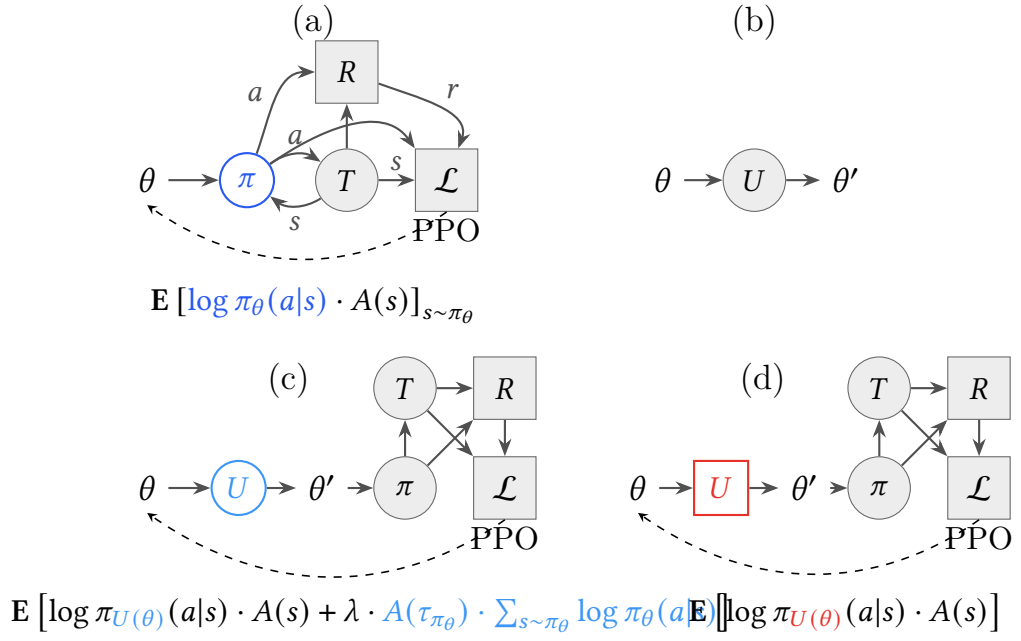
During meta-training, both the policy parameters and the latent distributions are optimized jointly using Trust Region Policy Optimization (TRPO), as summarized in Algorithm 51. At meta-test time, exploration begins from the shared prior

$$z \sim \mathcal{N}(0, I),$$

and the latent distribution is adapted using the observed rewards, enabling progressively more informative exploration on previously unseen tasks.

#### A.1.5.2. Exploration-MAML (E-MAML)

Although MAESN learns structured exploration through latent variables, it does not explicitly optimize exploration according to its contribution to adaptation. Exploration-MAML (E-MAML) addresses this limitation by modifying the meta-objective to assign credit to the trajectories collected before adaptation. Instead of rewarding exploratory actions only by their immediate return, E-MAML evaluates them according to how much they improve the performance of the adapted policy. Consequently, the initial policy is encouraged to collect trajectories that are maximally informative for subsequent learning rather than immediately rewarding.



**Figure A.12.** – Computation graphs for PPO, MAML, and E-MAML. The E-MAML graph preserves the dependence between exploration trajectories and the post-adaptation objective, allowing the meta-gradient to optimize exploration behavior.[63]

**Exploration-aware Meta-objective** Consider a task sampled from the task distribution. During adaptation, the agent first collects an exploration trajectory

$$\bar{\tau} \sim \pi_{\theta},$$

which is used to update the policy,

$$\theta' = U(\theta, \bar{\tau}).$$

The adapted policy then collects a second trajectory

$$\tau \sim \pi_{\theta'},$$

whose return defines the meta-objective

$$\mathcal{L}_{\text{meta}} = \mathbb{E}_{\tau, \bar{\tau}} [R(\tau)] = \iint R(\tau) \pi_{U(\theta, \bar{\tau})}(\tau) \pi_{\theta}(\bar{\tau}) d\bar{\tau} d\tau,$$

where  $\bar{\tau}$  denotes the pre-adaptation trajectory and  $\tau$  denotes the post-adaptation trajectory. Unlike conventional MAML, the objective explicitly models the dependence of the final return on the exploration data collected before adaptation.

**Exploration Credit Assignment** Differentiating the meta-objective yields

$$\nabla_{\theta} \mathcal{L}_{\text{meta}} \approx \frac{1}{T} \sum_{i=1}^T R(\tau^i) \nabla_{\theta} \log \pi_{U(\theta, \bar{\tau})}(\tau^i) + \frac{1}{T} \sum_{i=1}^T R(\tau^i) \nabla_{\theta} \log \pi_{\theta}(\bar{\tau}^i).$$

The first term is the conventional MAML meta-gradient, which optimizes the adapted policy after the inner update. The second term is introduced by E-MAML and assigns credit to the exploration trajectory according to the return achieved after adaptation. As a result, trajectories that lead to more effective policy updates receive higher probability during future exploration.

**Stochastic Computation Graph** Figure A.12 illustrates the computation graph of E-MAML. Standard MAML treats the inner update as a deterministic operation, ignoring the stochastic dependence between the sampled exploration trajectories and the policy update. Consequently, the initial policy receives no gradient encouraging it to collect informative experiences. E-MAML preserves this dependency by treating the sampling process as part of the computation graph. The additional exploration gradient propagates through the inner update, enabling the meta-learner to optimize not only how the policy adapts but also how it explores before adaptation.

Algorithm 52 summarizes the E-MAML training procedure. For each task, the agent first gathers exploration trajectories using the current policy and performs an inner policy update. The adapted policy then interacts with the environment to collect meta trajectories. During the meta-update, E-MAML combines the conventional adaptation gradient with an additional exploration gradient that rewards exploration according to its contribution to post-adaptation performance, enabling the policy to learn how to gather informative experiences for future adaptation.

#### A.1.5.3. Extending to Continuous Control: Memory-Based Control (RDPG and RSVG)

The recurrent formulation introduced in Section 5.7.1.1 is not restricted to discrete action spaces or on-policy training. Heess et al. [88] apply the same adaptation-via-input mechanism to continuous control, pairing the recurrent hidden state with deterministic and stochastic actor-critic algorithms. The underlying mechanism is unchanged: a fixed policy  $\theta$  reads from an evolving hidden state  $h_t$ , and no parameter update of any kind occurs during interaction with a task. What changes is the interpretation of  $h_t$  and the class of actor-critic algorithm built on top of it.

**Hidden state as belief.** Rather than treating  $h_t$  purely as a sufficient statistic of history, as in Section 5.7.1.1, this formulation interprets  $h_t$  explicitly as a belief over the unobserved state of the environment. The recurrent network compresses the sequence of past observations, actions, and rewards into  $h_t$ , and both the actor and the critic are conditioned on this belief rather than on a raw observation,

$$h_t \leftarrow f_\theta(h_{t-1}, o_t, a_{t-1}, r_{t-1}).$$

This is the same update rule used throughout Section 5.7.1, with the interpretation of  $h_t$  made explicit rather than left implicit.

**Recurrent Deterministic Policy Gradient (RDPG).** The first instantiation pairs this recurrent belief state with a deterministic actor-critic architecture, extending the Deterministic Policy Gradient algorithm to the partially observed, memory-based setting. The actor is a deterministic policy  $\mu(h)$  conditioned on the belief state, and its parameters are updated using the deterministic policy gradient,

$$\nabla_{\theta} J = \mathbb{E}[\nabla_a Q(h, a) \nabla_{\theta} \mu(h)].$$

The critic estimates the action-value function  $Q(h, \mu(h))$  and is trained with a temporal-difference objective,

$$L = \mathbb{E}_{\tau}[(\hat{Q}(h, a) - y)^2], \quad y_t^i = r_t^i + \gamma Q^{\omega'}(h_{t+1}^i, \mu^{\theta'}(h_{t+1}^i)),$$

where  $\omega'$  and  $\theta'$  denote target network parameters, following the standard target-network stabilization used in deterministic actor-critic methods.

**Episode-level replay.** Standard DPG-style methods sample individual transitions from a replay buffer, since the critic and actor updates only require single transition tuples. This is no longer valid once the policy and critic are conditioned on a recurrent hidden state  $h_t$ , since  $h_t$  depends on the entire trajectory that preceded it, not on any single transition in isolation. Training on a transition sampled out of order, or without its preceding history, would provide an incorrect or undefined hidden state. RDPG addresses this by storing and sampling entire episodes in the replay buffer, rather than individual transitions, so that the recurrent state can always be recomputed consistently from the start of an episode forward.

**Recurrent Stochastic Value Gradients (RSVG(0)).** The second instantiation replaces the deterministic actor with a stochastic policy, extending Stochastic Value Gradients to the same recurrent, belief-conditioned setting. The action is sampled as a function of the belief state and an independent noise variable  $v$ ,

$$a = \pi(h, v),$$

which in its simplified, reparameterized form reduces to a Gaussian policy with mean produced by the belief state,

$$a = \mu(h) + \sigma v.$$

This reparameterization allows gradients to be propagated through the sampling operation in the same way as in standard stochastic value gradient methods, while retaining the recurrent belief state  $h_t$  as the sole carrier of task-relevant information across time.

**Relation to Section 5.7.1.1.** RDPG and RSVG(0) demonstrate that the adaptation-via-input mechanism introduced in Section 5.7.1.1 is not tied to a specific policy class or a specific outer-loop algorithm. The same recurrent hidden state, updated identically at every timestep, can support a discrete policy trained with TRPO or actor-critic with entropy regularization, as in RL<sup>2</sup> and L2RL, or a continuous, deterministic or stochastic policy trained with DPG- or SVG-style actor-critic updates, as shown here. In every case, the underlying principle is unchanged: adaptation occurs because the input to a fixed policy evolves, not because the policy’s own parameters change during interaction. The one implementation detail that does not generalize automatically is the replay buffer: any recurrent, belief-conditioned method of this kind must sample and train on whole episodes rather than isolated transitions, a requirement that reappears in later sections whenever a recurrent or memory-conditioned critic is involved.

This concludes the purely recurrent formulations of black-box Meta-RL. Section 5.7.2 turns to architectural alternatives to the RNN backbone itself, examining how temporal convolution with attention, and later the Transformer architecture, address the long-history and parallelization limitations of the recurrent hidden state while preserving the same adaptation-via-input principle.

## A.2. Additional Black-Box Methods

Chapter 7 covers RL<sup>2</sup> as the foundational black-box formulation, in which the adaptation state is a recurrent hidden vector updated at every timestep. The methods below replace or extend that recurrent backbone: attention-based and external-memory architectures that address the same adaptation-via-input principle with a different mechanism for building and reading the adaptation state, and weight-space methods that move the adaptation state out of a hidden vector entirely and into the network’s own self-modifying weights.

### A.2.1. Attention-Based Adaptation Mechanisms

The methods presented in Section 5.7.1 rely entirely on a single LSTM to implement the adaptation function  $f_\theta$ . While effective, a plain recurrent backbone has



two structural weaknesses that become increasingly limiting as task horizons grow. First, RNNs process history sequentially, one timestep at a time, which prevents parallelization during training and makes optimization over long horizons slow. Second, information from early in an episode must survive many sequential updates of the hidden state to remain accessible later, which makes it difficult for the network to access long-range history reliably, even before considering the vanishing-gradient problems discussed in Section 5.7.4. This subsection examines two architectures that address these weaknesses while preserving the adaptation-via-input principle established in Section 5.7.1: the policy still reads from an evolving representation of history rather than having its own parameters modified, but the mechanism used to build and read that representation is no longer a plain recurrent state.

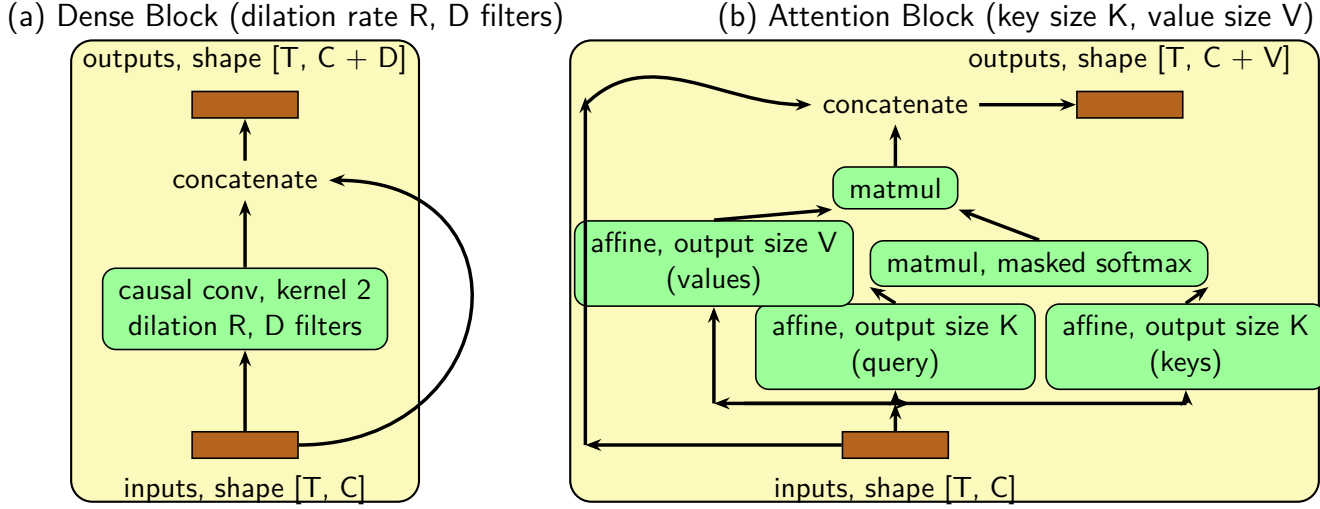
#### A.2.1.1. A Simple Neural Attentive Meta-Learner (SNAIL)

Mishra et al. [89] observe that existing meta-learning approaches fall into two unsatisfying categories. Methods such as MAML (Section 5.6.1.1) are hand-designed, in the sense that the structure of the adaptation procedure, a small number of gradient steps, is specified by the designer rather than learned. Purely recurrent black-box methods, as in Section 5.7.1, avoid this hand-design but are limited by the RNN itself: they struggle to access long-range history and cannot be parallelized across time during training. Pure attention, on the other hand, has no inherent notion of temporal order and is therefore poorly suited on its own to sequential decision-making, where the order of past events carries meaning.

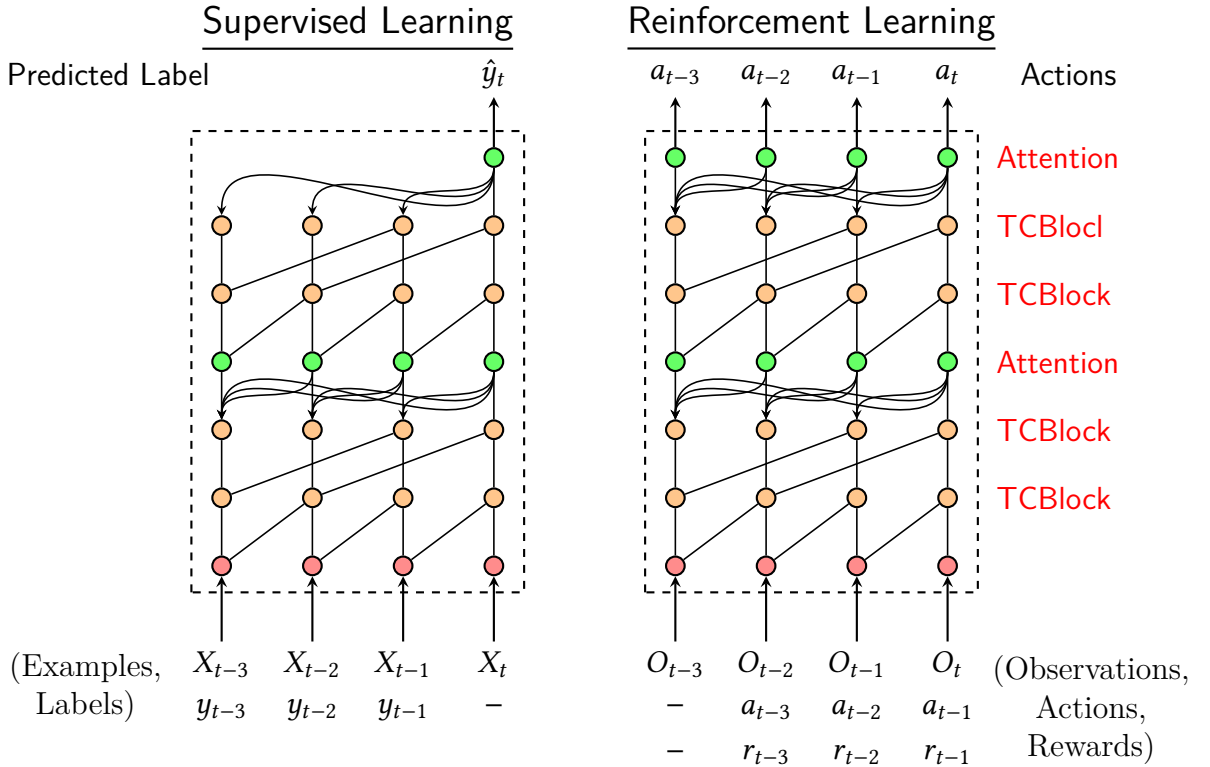
**Key insight.** SNAIL is motivated by the observation that an effective meta-learner needs two distinct capabilities that are individually well served by two different architectural components. It must store past experience efficiently, which favors an architecture that can process long sequences without the sequential bottleneck of an RNN, and it must retrieve relevant information selectively, which favors an architecture that can attend directly to specific past timesteps regardless of their distance from the present. SNAIL combines temporal convolution, which processes history and provides local temporal structure, with causal attention, which selects relevant information from anywhere in the past.

**Architecture.** The input to the network is the same transition sequence used throughout Section 5.7,  $(o_t, a_{t-1}, r_{t-1})$ . SNAIL alternates temporal convolution (TC) blocks and attention blocks,

$$\text{input} \rightarrow \text{TC} \rightarrow \text{Attention} \rightarrow \text{TC} \rightarrow \text{Attention} \rightarrow \text{action}.$$



**Figure A.13.** – The two main components of the SNAIL architecture: (a) the Temporal Convolution (TC) block captures long-range temporal dependencies through dilated causal convolutions, while (b) the attention block selectively retrieves relevant information from past observations using causal self-attention.



**Figure A.14.** – SNAIL architecture applied to supervised learning (left) and reinforcement learning (right). Orange nodes denote temporal convolution (TC) blocks, green nodes denote attention blocks, and red nodes denote the raw input sequence. Adapted from Mishra et al. [89].

Figure A.13 illustrates this structure and its correspondence to the supervised meta-learning setting SNAIL was originally designed for. In the supervised case, the network receives a sequence of example-label pairs  $(\mathbf{x}_{t-i}, \mathbf{y}_{t-i})$  and must predict the label  $\hat{\mathbf{y}}_t$  of the current example without having observed  $\mathbf{y}_t$ . In the reinforcement learning case, the same architecture receives the transition sequence  $(\mathbf{o}_{t-i}, \mathbf{a}_{t-i-1}, \mathbf{r}_{t-i-1})$  and outputs an action  $\mathbf{a}_t$ . In both cases, the alternating TC and attention blocks play the same role: TC blocks aggregate local temporal context, and attention blocks selectively pull in relevant information from earlier in the sequence, with the final attention block’s output used to produce the prediction or action.

**Temporal convolution blocks.** Each TC block is built by stacking dilated causal convolutions, so that the receptive field grows exponentially with depth while the output at every timestep depends only on inputs at or before that timestep, preserving the autoregressive structure required for sequential decision-making. A TC block of sequence length  $T$  and  $D$  filters applies a sequence of dense blocks with exponentially increasing dilation rates, as shown in Algorithm 53.

---

**Algorithm 53** TCBlock

---

```

function TCBlock(inputs, sequence length  $T$ , number of filters  $D$ )
  for  $i$  in  $1, \dots, \lceil \log_2 T \rceil$  do
    inputs = DenseBlock(inputs,  $2^i$ ,  $D$ )
  end for
  return inputs
end function

```

---

Each call to DenseBlock applies a pair of causal convolutions at a given dilation rate  $R$ , combines them with a gated activation, and concatenates the result with the block’s input, following the gated PixelCNN-style activation used by Mishra et al. [89], as shown in Algorithm 54.

---

**Algorithm 54** DenseBlock

---

```

function DenseBlock(inputs, dilation rate  $R$ , number of filters  $D$ )
   $\mathbf{x}f, \mathbf{x}g$  = CausalConv(inputs,  $R, D$ ), CausalConv(inputs,  $R, D$ )
  activations =  $\tanh(\mathbf{x}f) * \text{sigmoid}(\mathbf{x}g)$ 
  return concat(inputs, activations)
end function

```

---

The gated activation  $\tanh(\mathbf{x}f) * \text{sigmoid}(\mathbf{x}g)$  allows the network to learn which components of the convolved signal to pass through, and the concatenation of activations

with the original input, rather than a replacement of it, is what gives the block its dense connectivity, allowing information from every earlier layer to remain directly accessible to later layers.

**Attention blocks.** Each attention block applies causal self-attention over the sequence of TC-processed representations, allowing the network to selectively retrieve specific past timesteps that are most relevant to the current decision, rather than relying on everything being compressed uniformly into a single fixed-size state as in Section 5.7.1. Given an input sequence of shape  $[T, C]$ , the attention block computes query and key projections of size  $K$  and a value projection of size  $V$ , as shown in Algorithm 55.

---

**Algorithm 55** AttentionBlock

---

```

function AttentionBlock(inputs, key size  $K$ , value size  $V$ )
  keys, query = affine(inputs,  $K$ ), affine(inputs,  $K$ )
  logits = matmul(query, transpose(keys))
  probs = CausallyMaskedSoftmax(logits/ $\sqrt{K}$ )
  values = affine(inputs,  $V$ )
  read = matmul(probs, values)
  return concat(inputs, read)
end function

```

---

The causal mask applied to the softmax ensures that the query at timestep  $t$  can only attend to keys at timesteps  $\leq t$ , preserving the same autoregressive constraint enforced by the causal convolutions in the TC blocks. Scaling the logits by  $1/\sqrt{K}$  follows the standard scaled dot-product attention formulation and prevents the softmax from saturating as the key dimension  $K$  grows. As with the TC block, the retrieved information is concatenated with, rather than substituted for, the block’s input, so that the identity of the original representation at each timestep is preserved alongside whatever is retrieved from the past.

**Relation to the adaptation-via-input mechanism.** SNAIL does not depart from the adaptation-via-input principle established in Section 5.7.1: the network parameters  $\theta$  are trained once, across the task distribution, and held fixed at test time, while the sequence of TC and attention representations plays the same role that the hidden state  $h_t$  plays in RL<sup>2</sup> and L2RL, an evolving representation of history that a fixed policy reads from. What changes is only how that representation is built and accessed: convolution and attention replace the single recurrent update, giving the network parallelizable training and more direct access to distant past timesteps, at the cost of a more complex architecture than a single LSTM.

Section 5.7.2.2 continues in this direction, examining Transformer-based architectures that rely on attention alone and make an explicit case for why context-dependent attention weights are, by themselves, sufficient to

#### A.2.1.2. Transformers as Meta-Reinforcement Learners (TrMRL)

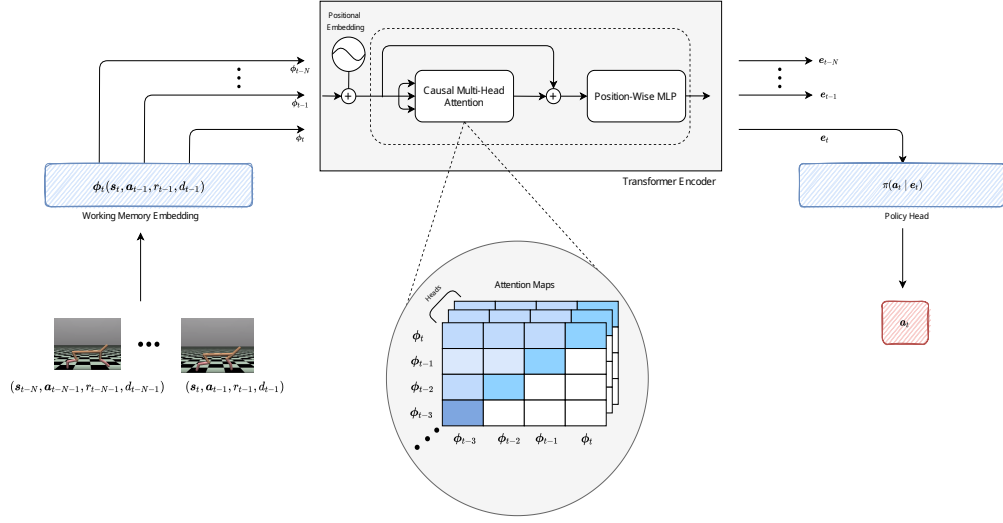
While SNAIL (Section 5.7.2.1) combines temporal convolution with attention, Melo [90] argues that attention alone is sufficient to implement fast adaptation, without any recurrent or convolutional component at all. The resulting architecture, TrMRL, applies a standard Transformer encoder directly to a window of past transitions and treats the resulting context-dependent representation as the adaptation state read by the policy.

**Claim of the paper.** TrMRL is motivated by two properties of the Transformer architecture that map directly onto the requirements of Meta-RL. Long-sequence modeling allows the network to infer the identity of the current task from an extended window of trajectory history, addressing the long-range access limitation of a plain RNN discussed in Section 5.7.2. Context-dependent attention weights allow the representation the policy reads from to change immediately as new transitions arrive, without requiring any weight update, giving online fast adaptation as a direct consequence of the attention mechanism itself.

**Working memory embedding.** As in the memory-based methods surveyed later in Section 5.7.3, TrMRL builds its representation on top of a working memory embedding that summarizes each transition. At timestep  $t$ , the working memory embedding is

$$\phi_t = \phi(s_t, a_{t-1}, r_{t-1}, d_{t-1}),$$

where  $d_{t-1}$  is a termination flag indicating whether the previous episode ended at that timestep. Figure A.15 shows this embedding step on the left: raw transitions  $(s_{t-N}, a_{t-N-1}, r_{t-N-1}, d_{t-N-1})$  through  $(s_t, a_{t-1}, r_{t-1}, d_{t-1})$  are each mapped to a working memory embedding  $\phi_{t-N}, \dots, \phi_t$ , forming a queue of the  $N$  most recent memory slots.



**Figure A.15.** – TrMRL architecture. Raw transitions are embedded into working memory vectors  $\phi_t$ , processed by  $L$  layers of a causal Transformer encoder with positional embedding, causal multi-head attention, and a position-wise MLP, producing refined representations  $e_t$ . The policy head is conditioned on the final representation  $e_t$  to produce the action  $a_t$ . The attention maps below illustrate how each memory slot attends only to itself and earlier slots, with attention weight distributed unevenly across the window. Adapted from Melo [90].

**Task representation.** A task is treated as a distribution over working memory vectors,  $T(\phi) : \Phi \rightarrow [0, \infty)$ , and the representation of the task at time  $t$  is defined as a weighted combination of the projected memories in the current window,

$$\mu_T = \sum_{t=0}^N \alpha_t W(\phi_t), \quad \sum_t \alpha_t = 1,$$

where  $W$  is a learned projection and  $\alpha_t$  determines the importance assigned to each memory slot. This weighted combination is not computed by a hand-specified rule; it is precisely what the Transformer’s attention mechanism computes internally, as described next.

**Transformer encoder over memory.** TrMRL applies a Transformer encoder with  $L$  layers to the working memory queue  $\phi_{t-N}, \dots, \phi_t$ , as shown in the central block of Figure A.15. Each memory vector is combined with a positional embedding, so

that the network retains a notion of the order in which transitions occurred, and then passed through  $L$  stacked blocks of causal multi-head attention followed by a position-wise MLP, with residual connections around each. Formally, letting  $e_t^0 = \phi_t$ ,

$$e_t^l = f(e_0^{l-1}, \dots, e_t^{l-1}), \quad l = 1, \dots, L,$$

where  $f$  denotes one Transformer layer. Each successive layer produces a more refined representation of the memory window, and the interpretation of this process is the same at every layer: the agent asks which of the previous memories are most relevant to the current situation, and attention determines how much each one should count. The inset in Figure A.15 illustrates the resulting attention maps: because attention is causal, memory slot  $\phi_t$  can attend to  $\phi_{t-3}$ ,  $\phi_{t-2}$ ,  $\phi_{t-1}$ , and itself, but not to any future slot, and the weight assigned to each is learned and generally uneven, concentrating on the memory slots most relevant to the current decision rather than treating the entire window uniformly.

**Consensus representation.** The output of the final Transformer layer,  $e_t$ , is interpreted by the authors as a consensus representation: self-attention aggregates the memory window into a representation that minimizes Bayes risk with respect to the memories it draws from. Intuitively, memories that are inconsistent with the rest of the window are suppressed by low attention weight, while consistent, recurring patterns across memories dominate the resulting representation. This is analogous to Bayesian averaging or belief updating, where a posterior estimate is formed by weighting multiple pieces of evidence according to their reliability, except that here the weighting is computed directly by the learned attention mechanism rather than by an explicit probabilistic model, in contrast to the explicit belief network used by the task inference methods of Section 5.9.

**Policy head.** The final representation  $e_t$  is passed to a policy head that outputs the action distribution,  $\pi(a_t | e_t)$ , as shown on the right of Figure A.15. No component of the Transformer’s weights is updated during interaction with a task. As in every method presented in this section, the parameters are trained once during meta-training and held fixed at test time; adaptation is entirely a consequence of  $e_t$  changing as the memory window slides forward and new transitions enter it.

**Architectural improvements over naive Transformer-RL.** Two design choices distinguish TrMRL from a direct application of the standard Transformer to RL

trajectories. Causal attention, already reflected in Figure A.15, restricts each memory slot to attend only to itself and earlier slots, removing attention to variables that could not causally have influenced the current decision. A memory queue mechanism bounds the window to the  $N$  most recent memory slots, functioning as a sliding window rather than requiring attention over an unbounded and growing history, which keeps computation tractable and focuses the consensus representation on recent, task-relevant experience.

**Why earlier Transformer-RL methods failed.** The authors attribute the difficulty of applying Transformers to RL prior to this work to three compounding factors. RL gradients are inherently noisy compared to the supervised setting Transformers were originally developed for. Exploration in RL depends on stable behavior early in training, before the policy has converged. Combined, an unstable Transformer, one that has not yet learned sensible attention weights, produces erratic early behavior that destroys exploration before the agent has had a chance to gather the experience needed to learn at all. TrMRL’s causal attention and memory queue mechanism are presented as mitigating this instability by constraining what the attention mechanism can attend to and bounding the size of the problem it must solve at any given time.

**Relation to the adaptation-via-input mechanism.** TrMRL is the clearest embodiment of the adaptation-via-input principle introduced in Section 5.7.1: the policy  $\pi_\theta$  never changes during interaction, and the entire adaptation process consists of the representation  $e_t$  evolving as the memory queue slides forward. What SNAIL achieves through a combination of dilated convolution and attention, TrMRL achieves through attention alone, with the consensus-representation interpretation offering the most direct connection between this family of methods and the belief-based framing developed independently in Section 5.9.

This concludes Section 5.7.2. Section 5.7.3 turns from architectural alternatives to the RNN backbone toward methods that introduce explicit external memory in addition to, or in place of, the sliding attention window used here, beginning with the earliest key-value memory architectures for reinforcement learning.

## A.2.2. Adaptation via External Memory

Every method presented so far in this section, whether recurrent (Section 5.7.1) or attention-based (Section 5.7.2), represents the adaptation state as a single evolving vector: a hidden state  $h_t$ , or a Transformer’s context-refined representation  $e_t$ . This



vector must simultaneously carry everything the agent has learned about the current task, including information observed many timesteps in the past. When a task requires recalling a specific piece of information encountered early in an episode and using it much later, after many intervening observations, this single-vector representation becomes a genuine bottleneck: nothing forces the network to preserve that specific piece of information rather than overwriting or diluting it as new observations arrive.

The methods in this subsection address this by introducing an explicit external memory module, separate from the main recurrent or attention state, whose sole function is to store past observations and make them individually retrievable rather than folding them irreversibly into a single evolving vector. This subsection traces the development of this idea from its earliest form, a fixed-size key-value memory read by simple attention, through successive refinements that address specific weaknesses of that original mechanism.

#### A.2.2.1. Control of Memory, Active Perception, and Action in Minecraft (MQN, RMQN, FRMQN)

Oh et al. [91] introduce the first key-value external memory architecture for reinforcement learning, motivated by tasks in which standard RL struggles for three related reasons: observations are only partially informative of the true state (POMDP), rewards are delayed, and, most importantly for this subsection, information relevant to a decision can appear early in an episode but only become necessary much later.

**Motivating example.** Consider an agent that observes an indicator color, yellow or green, early in an episode. The indicator then disappears from view. Later, the agent must choose between a red block and a blue block, where the correct choice depends on the indicator color observed earlier. Because the indicator is no longer visible at decision time, the agent must have retained it in memory, and because many observations may occur between seeing the indicator and making the choice, a single recurrent hidden state risks overwriting this information before it is needed. This is precisely the failure mode that external memory is designed to prevent.

**Encoding.** Each observation  $x_t$  is first passed through an encoder to produce a fixed-size embedding,

$$e_t = \phi_{\text{enc}}(x_t).$$

**Memory writing.** Rather than compressing observations into a single evolving state, the model maintains an explicit memory of the  $M$  most recent observation embeddings,  $E_t = [e_{t-1}, \dots, e_{t-M}]$ , and projects them into separate key and value representations,

$$M_t^{\text{key}} = W_{\text{key}} E_t, \quad M_t^{\text{val}} = W_{\text{val}} E_t.$$

The key representation is used for matching against a query at read time, while the value representation is what is actually retrieved and returned to the policy. Separating these two roles allows the network to learn a matching representation optimized for retrieval accuracy, independent of the representation it needs for downstream decision-making. Figure A.16(a) shows this write operation: the current input  $x_t$  is projected by two separate weight matrices,  $W^{\text{key}}$  and  $W^{\text{val}}$ , into the key and value memory blocks  $M_t^{\text{key}}$  and  $M_t^{\text{val}}$ , each holding  $M$  slots.

**Memory reading.** Given a context vector  $h_t$ , the model computes attention weights over the stored keys and retrieves a weighted combination of the corresponding values,

$$p_{t,i} = \frac{\exp(h_t^\top M_t^{\text{key}}[i])}{\sum_j \exp(h_t^\top M_t^{\text{key}}[j])}, \quad o_t = M_t^{\text{val}} p_t.$$

Figure A.16(b) shows the corresponding read operation: the context vector  $h_t$  is compared against every slot of  $M_t^{\text{key}}$ , producing attention weights  $p_t$  via softmax, which are then used to combine the slots of  $M_t^{\text{val}}$  into the single retrieved vector. This is a direct precursor to the attention mechanisms used in Sections 5.7.2.1 and 5.7.2.2, restricted here to a fixed window of  $M$  past observations rather than the full sequence processed by a Transformer. The interpretation is the same throughout this section: the context vector is compared against past memory keys, and the resulting attention weights determine how much each past observation contributes to the retrieved memory  $o_t$ .

**Context vector variants.** The three architectures introduced by Oh et al. [91] differ only in how the context vector  $h_t$ , used as the query for memory reading, is computed. Table A.5 summarizes the three variants in order of increasing complexity, and Figure A.17 shows them alongside two non-memory baselines for comparison.

**Table A.5.** – Context vector computation in the MQN family.

| Architecture | Context vector                                               |
|--------------|--------------------------------------------------------------|
| MQN          | $h_t = W_c e_t$                                              |
| RMQN         | $(h_t, c_t) = \text{LSTM}(e_t, h_{t-1}, c_{t-1})$            |
| FRMQN        | $(h_t, c_t) = \text{LSTM}([e_t, o_{t-1}], h_{t-1}, c_{t-1})$ |

MQN computes the context vector directly from the current observation embedding, with no recurrence at all: memory retrieval depends entirely on what the agent currently sees, not on any accumulated history. RMQN adds an LSTM over the sequence of observation embeddings, so that the query used for memory retrieval is itself informed by the agent’s trajectory rather than the current observation alone. FRMQN goes one step further and feeds the previously retrieved memory  $o_{t-1}$  back into the LSTM as an additional input alongside  $e_t$ , so that what was retrieved at the last timestep influences both the recurrent state and, through it, what is retrieved next. This closes a feedback loop between reading and remembering that is absent in the other two variants, visible in Figure A.17(e) as the additional arrow connecting the Memory block back into Context.

As Figure A.17 makes clear, this progression is also a progression away from two simpler baselines: DQN (a), which has no separate memory module at all and instead stacks the last  $M$  raw frames as input, and DRQN (b), which replaces frame-stacking with a single recurrent context but still has no explicit, individually addressable memory. MQN, RMQN, and FRMQN each add an explicit memory module on top of this recurrent baseline, with the three variants differing only in how richly the context that queries that memory is computed. In every variant, the final Q-value prediction is a function of both the context and the retrieved memory,

$$q_t = \phi_q(h_t, o_t).$$

**Where each variant is most effective.** The three variants are not simply successive improvements; each is best suited to a different failure mode, summarized in Table A.6.

**Table A.6.** – Best-suited scenarios for each variant of the MQN family.

| Architecture | Best use case                                                                                                                                 | Reasoning                                                                                                                                                                                                         |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MQN          | Simple, fixed environments where the current view is enough to know what to look for in memory.                                               | Without recurrence, it can overfit to training patterns and fails at complex visual matching.                                                                                                                     |
| RMQN         | Randomly generated maps or navigation tasks where tracking the history of movement is crucial.                                                | The recurrent connection over embeddings is vital for handling changing topologies that MQN’s memoryless query cannot capture.                                                                                    |
| FRMQN        | Complex reasoning tasks, such as comparing two patterns seen at different times, and tasks requiring generalization to unseen or larger maps. | The feedback loop between retrieved memory and the recurrent state allows the model to compare and refine information across multiple reads, making it the most robust of the three on difficult cognitive tasks. |

This progression illustrates a pattern that recurs throughout Section 5.7.3: adding recurrence over the query (RMQN) and then feeding retrieved memory back into that recurrence (FRMQN) each address a specific limitation of the simpler variant before it, rather than being arbitrary architectural variations.

**Meta-training and meta-testing.** Algorithms 56 and 57 summarize the resulting training and evaluation procedure. The structure mirrors the recurrent methods of Section 5.7.1, with one addition: at every timestep, the current observation embedding is written into the external memory before the context vector is computed and used to read from it, so that memory writing and memory reading occur once per timestep, in that order.

---

**Algorithm 56** Meta-Training with External Memory

---

**Require:** Task distribution  $p(\mathcal{T})$ , meta-parameters  $\theta$

**while** not converged **do**

    Sample a mini-batch of tasks  $\{\mathcal{T}_i\}_{i=1}^N \sim p(\mathcal{T})$

**for** each task  $\mathcal{T}_i$  **do**

        Reset environment

        Initialize external memory  $M = \emptyset$

        Initialize recurrent state  $h_0$

**for**  $t = 1, \dots, T$  **do**

            Observe  $o_t$

            Encode observation  $e_t = f_\theta(o_t)$

            Store memory  $M \leftarrow M \cup \{e_t\}$

            Compute query  $q_t = g_\theta(h_{t-1}, e_t)$

            Compute attention weights  $w_i = \frac{\exp(q_t^\top M_i)}{\sum_j \exp(q_t^\top M_j)}$

            Read memory  $r_t = \sum_i w_i M_i$

            Update hidden state  $h_t = \text{RNN}(e_t, r_t, h_{t-1})$

            Compute policy  $\pi_\theta(a_t \mid o_t, r_t, h_t)$

            Sample action  $a_t \sim \pi_\theta$

            Execute  $a_t$  and observe  $(r_t, o_{t+1})$

**end for**

**end for**

    Compute meta-objective  $J(\theta) = \frac{1}{N} \sum_{i=1}^N G_i$

    Update meta-parameters  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

**end while**

---

---

**Algorithm 57** Meta-Testing with External Memory

---

**Require:** Learned meta-parameters  $\theta$

**for** each unseen task  $\mathcal{T}$  **do**

    Reset environment

    Initialize external memory  $M = \emptyset$

    Initialize recurrent state  $h_0$

**for**  $t = 1, \dots, T$  **do**

        Observe  $o_t$

        Encode observation  $e_t = f_\theta(o_t)$

        Store memory  $M \leftarrow M \cup \{e_t\}$

        Compute query  $q_t = g_\theta(h_{t-1}, e_t)$

        Retrieve memory  $r_t = \sum_i w_i M_i$

        Update hidden state  $h_t = \text{RNN}(e_t, r_t, h_{t-1})$

        Select action  $a_t = \arg \max_a \pi_\theta(a \mid o_t, r_t, h_t)$

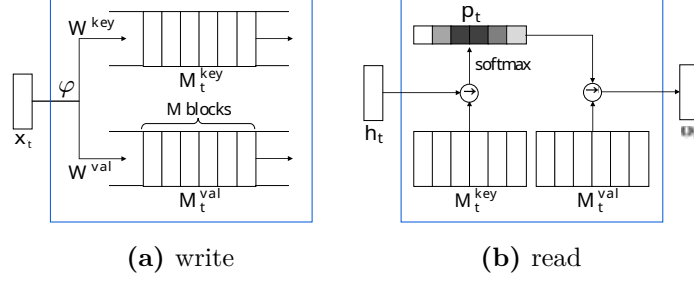
        Execute action and receive next observation

**end for**

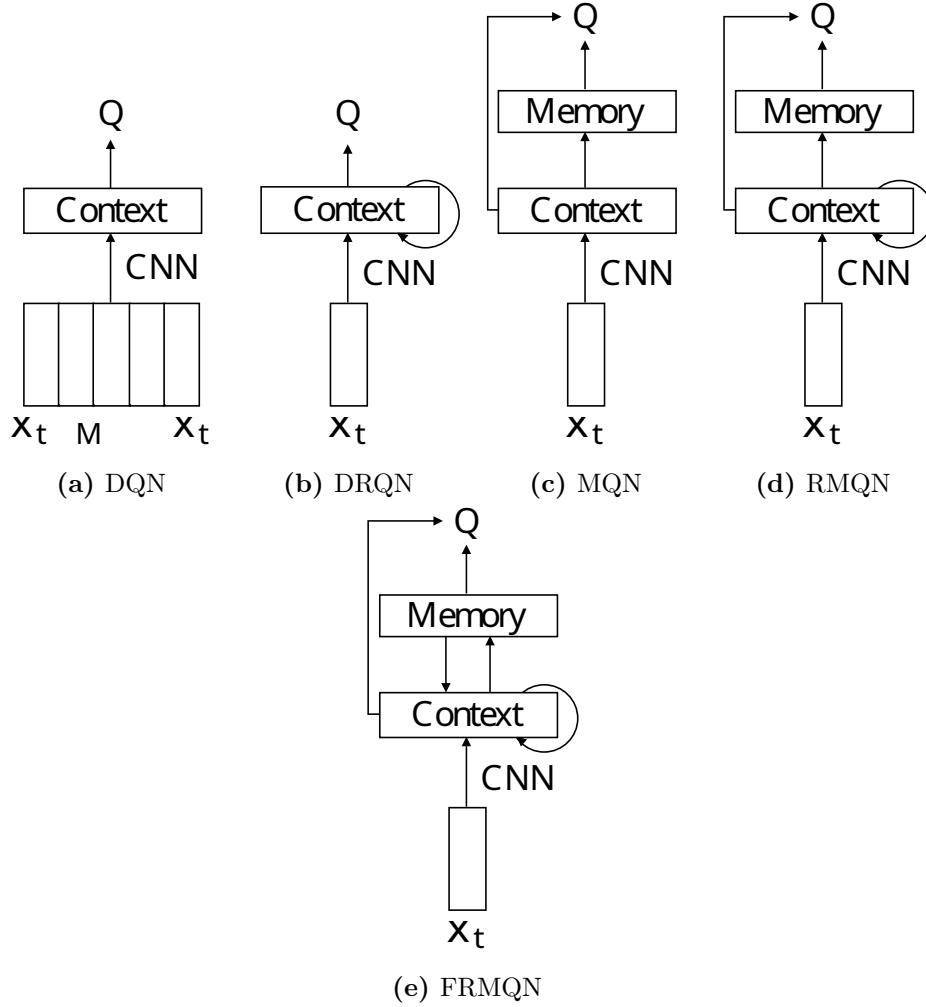
**end for**

---

**Relation to later methods.** This is the first architecture in Section 5.7 to separate memory storage from the main adaptation state entirely, rather than folding all history into a single hidden state or attention context. The remaining subsections of Section 5.7.3 build directly on this key-value read/write mechanism. Section 5.7.3.2 formalizes the distinction between this kind of short-window episodic memory and a separate working memory, and addresses the credit-assignment difficulty of back-propagating through memory writes. Section 5.7.3.3 extends the retrieval mechanism to recognize when a task itself has recurred, rather than only retrieving within a single episode. Section 5.7.3.4 addresses a structural limitation shared by the read mechanism introduced here: retrieving memory as a weighted average, as in  $o_t$  above, discards the relationships between individual memory slots, which becomes a problem once retrieval needs to support multi-step planning rather than single-step recall.

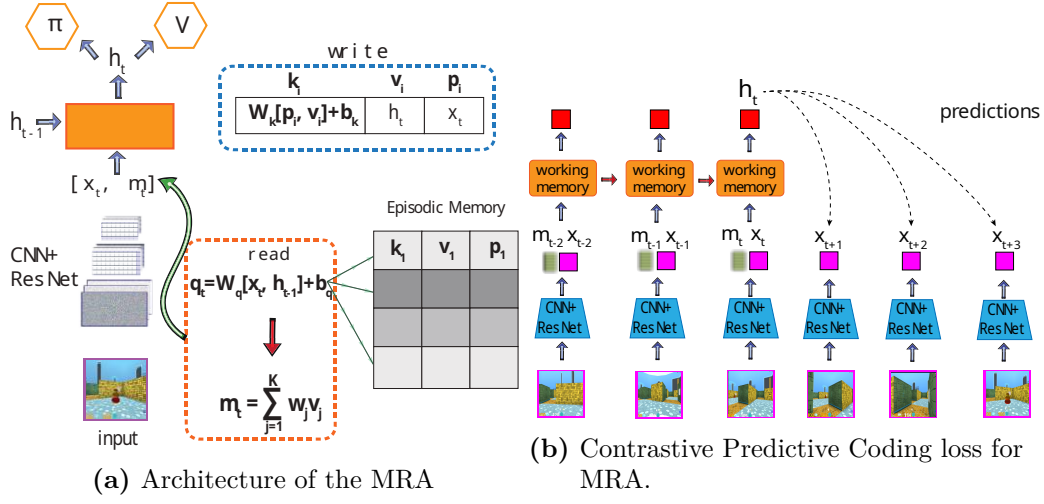


**Figure A.16.** – Memory operations of the MQN family. (a) Write: the current input  $x_t$  is projected into key and value memory blocks via  $W^{\text{key}}$  and  $W^{\text{val}}$ . (b) Read: the context vector  $h_t$  is compared against the stored keys  $M_t^{\text{key}}$  via softmax attention to produce weights  $p_t$ , which are used to combine the stored values  $M_t^{\text{val}}$  into the retrieved memory. Adapted from Oh et al. [91].



**Figure A.17.** – Architectural comparison of DQN, DRQN, MQN, RMQN, and FRMQN. DQN (a) stacks the last  $M$  frames as input with no separate memory module. DRQN (b) replaces frame-stacking with a recurrent context. MQN (c) adds an explicit memory module read by a memoryless context. RMQN (d) reads the same memory using a recurrent context. FRMQN (e) additionally feeds the retrieved memory back into the recurrent context, closing the feedback loop between reading and remembering. Adapted from Oh et al. [91].





**Figure A.18.** – The MRA agent. (a) Working memory (an LSTM) and episodic memory (a key-value store) interact through explicit write and read operations: writing stores the current working memory state and input without gradient, and reading retrieves a nearest-neighbor-weighted combination of stored values, fed back into the working memory at the next timestep. (b) The CPC auxiliary loss: the working memory state  $h_t$  at a given timestep is trained to predict the CNN+ResNet embeddings of future observations  $x_{t+1}$ ,  $x_{t+2}$ ,  $x_{t+3}$ , shown by the dashed arrows, providing a reward-independent training signal. Adapted from Fortunato et al. [92].

### A.2.2.2. Generalization of Reinforcement Learners with Working and Episodic Memory (MRA)

Fortunato et al. [92] formalize the distinction between two kinds of memory that the Minecraft architectures of Section 5.7.3.1 leave implicit, and combine them with an auxiliary representation learning objective, producing an architecture referred to as the Memory, RL, and Auxiliary losses (MRA) agent. The central claim of this work is that memory alone is not sufficient for generalization in reinforcement learning: the agent additionally needs representations that are useful independently of the reward signal, since reward alone is too sparse and too delayed to shape rich state representations on its own.

**Two types of memory.** MRA separates the agent’s memory into two explicitly distinct components, shown in Figure A.18(a). Working memory is the hidden state of an LSTM, responsible for short-term, within-episode dependencies,

$$h_t = \text{LSTM}(x_t, m_t),$$

where  $m_t$  is the memory retrieved from the second component. Episodic memory is an explicit key-value store, in the same spirit as the memory introduced in Section 5.7.3.1, but here holding three quantities per slot rather than two: a key  $k_i$ , a value  $v_i = h_i$ , equal to the working memory state at the time the slot was written, and the input embedding  $p_i = x_i$  itself. Working memory handles short-term dependencies within the current sequence of observations, while episodic memory is responsible for long-term recall of specific past experiences, a division of labor that the MQN family of Section 5.7.3.1 does not make explicit, since its single memory module is queried directly by the context vector without a separate short-term state feeding into it.

**Writing to episodic memory.** At every timestep, the current working memory state and input are written into episodic memory as a new key,

$$k_i = W_k[p_i, v_i] + b_k.$$

This write operation is not differentiated through: the values written to memory are stored without gradient, functioning as a growing replay buffer of past experiences rather than as a component that is optimized directly. Figure A.18(a) shows this write step at the top: the previous working memory state  $h_{t-1}$  and current input  $x_t$  together produce the working memory update, and the resulting  $(k_i, v_i, p_i)$  triple, with  $v_i = h_t$  and  $p_i = x_t$ , is appended to episodic memory.

**Reading from episodic memory.** At read time, a query is formed from the current input and the previous working memory state,

$$q_t = W_q[x_t, h_{t-1}] + b_q,$$

and this query is compared against the stored keys using a nearest-neighbor attention rule, rather than the softmax-over-dot-product rule used in Section 5.7.3.1,

$$m_t = \sum_{j=1}^K w_j v_j, \quad w_j \propto \frac{1}{\epsilon + \|q_t - k_j\|^2}.$$

Slots whose keys lie closer to the query in Euclidean distance receive proportionally larger weight, with  $\epsilon$  preventing division by zero when a stored key coincides exactly with the query. Figure A.18(a) shows this read step on the left: the query  $q_t$  is compared against every key  $k_i$  in episodic memory, and the resulting retrieval  $m_t$  is fed back into the working memory LSTM at the next timestep, closing the loop between the two memory systems.

**Credit assignment through memory.** Writing to episodic memory at every timestep creates a genuine optimization difficulty: since every past write could in principle influence a much later read, full backpropagation through time across the entire memory-writing history is intractable. MRA resolves this by applying a stop-gradient at the point of memory writing,

$$k_i = W_k[\perp(p_i), \perp(v_i)] + b_k,$$

where  $\perp(\cdot)$  denotes the stop-gradient operator. Rather than gradients flowing backward through the entire chain of intermediate timesteps between a memory write and a later read, this allows gradients computed at read time to jump directly to the specific memory entries that were retrieved, without needing to be propagated through every timestep in between. This trades exact credit assignment through the full write history for a tractable, if approximate, form of long-range credit assignment, and is one of the more significant departures from the read/write mechanism of Section 5.7.3.1, which does not require this trick because it operates over a small fixed window of  $M$  recent observations rather than an unbounded growing memory.

**Auxiliary representation learning with CPC.** MRA’s second contribution addresses a separate problem: the RL reward signal alone is sparse and delayed, which is insufficient to shape a representation rich enough for either memory system to operate on effectively. The agent’s representation must be good independently of whether reward happens to be available at a given timestep. MRA addresses this with an unsupervised auxiliary objective based on Contrastive Predictive Coding (CPC), which does not depend on reward at all and instead trains the working memory state to predict future observation embeddings,

$$\sum_{\tau=1}^N \text{CPCLoss}[h_t; x_{t+1}, x_{t+2}, \dots, x_{t+\tau}],$$

where  $N$  is the prediction horizon. Each term in this sum is a contrastive loss comparing the true future embedding against a set of negative samples,

$$\text{CPCLoss}(h_t, x_{t+\tau}) = -\log \frac{\exp(f(h_t, x_{t+\tau}))}{\sum_{x' \in \mathcal{N}} \exp(f(h_t, x'))},$$

where  $x_{t+\tau}$  is the true future embedding, treated as the positive sample,  $\mathcal{N}$  is a set of negative samples drawn from other embeddings, and  $f(\cdot, \cdot)$  is a similarity function, typically a dot product or bilinear form. This loss is maximized when  $h_t$  is highly similar to the true future embedding  $x_{t+\tau}$  and dissimilar to the negative samples, which forces the working memory state to encode information that is predictive of what will be observed next, rather than only information that happens to be useful for the current reward. Figure A.18(b) illustrates this: the working memory state  $h_t$ , computed after processing a sequence of past inputs, is trained to predict the CNN+ResNet embeddings of several future frames  $x_{t+1}, x_{t+2}, x_{t+3}$ , shown by the dashed arrows fanning out from  $h_t$  to each future embedding. The total training objective combines this auxiliary loss with the standard reinforcement learning loss,

$$\mathcal{L} = \mathcal{L}_{\text{RL}} + \lambda \mathcal{L}_{\text{CPC}},$$

so that the representation is shaped both by task reward and by the requirement that it be predictive of future observations, forcing it to encode temporal structure that reward alone would not necessarily select for.

**Findings.** Table A.7 summarizes the main empirical findings reported for MRA.

**Table A.7.** – Main findings reported for the MRA agent.

| Finding                        | Explanation                                                                                                | Implication                                                                           |
|--------------------------------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Memory type matters            | Working memory handles short-term dependencies; episodic memory handles long-term recall.                  | Different memory systems are needed for different temporal structures.                |
| Episodic memory is crucial     | Improves performance on many tasks, especially those requiring recognition and inference.                  | External memory is essential for generalization in RL.                                |
| Synergy between memory and CPC | The combination of external memory and the CPC auxiliary loss performs better than either component alone. | Representation learning and memory are complementary, not substitutable.              |
| Generalization remains limited | Performance drops on larger-scale and harder tasks.                                                        | Current models still struggle with genuine generalization beyond the training regime. |

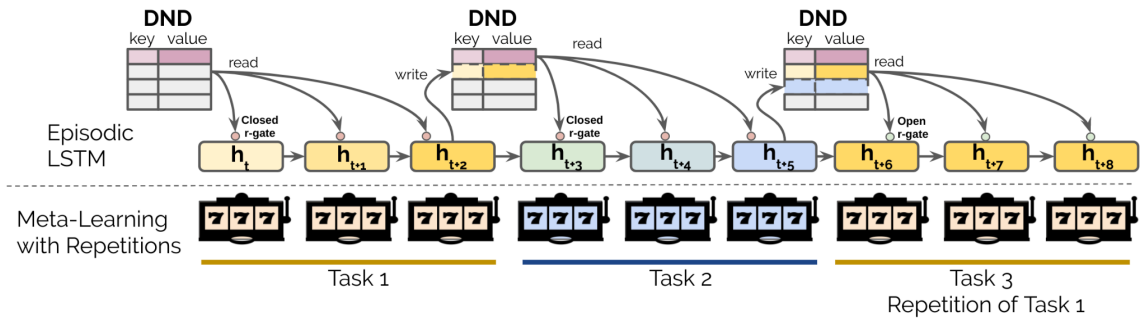
**Limitations.** Despite these gains, MRA leaves several problems open, which fall into a small number of recurring categories. On generalization, the agent extrapolates poorly and tends to overfit to the training regime, with no explicit uncertainty model or latent task variable to support adaptation to conditions unlike those seen during meta-training, in contrast to the task inference methods discussed in Section 5.9. On memory itself, storage remains unstructured and therefore difficult to interpret, retrieval offers no guarantees and is sensitive to the quality of the learned embeddings, the memory buffer is of fixed size, and retrieval time grows with the size of the memory, none of which is addressed by the architecture itself. On credit assignment, the stop-gradient trick described above enables tractable training but at the cost of jumpy, approximate gradients rather than exact backpropagation through the full write history, and the underlying LSTM working memory is still subject to the vanishing-gradient behavior of recurrent networks more generally. On representation learning, the CPC objective is not sufficient on its own to prevent the agent from learning spurious correlations, and the architecture has no explicit relational bias to support compositional reasoning across memory entries. Table A.8 organizes these limitations by category.

**Table A.8.** – Categories of limitations of the MRA agent.

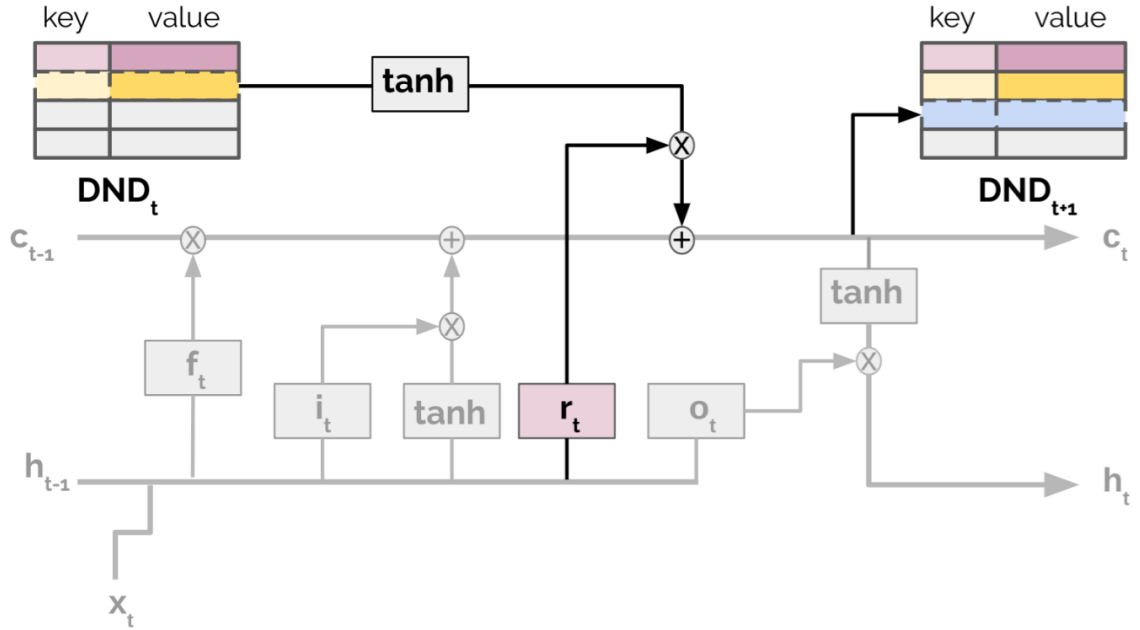
| Category          | Problem                                   | Impact                                           |
|-------------------|-------------------------------------------|--------------------------------------------------|
| Generalization    | Poor extrapolation                        | Fails under distribution shift                   |
| Task inference    | No latent task variable                   | Weak adaptation to novel tasks                   |
| Memory            | Unstructured storage                      | No interpretability of what is stored            |
| Credit assignment | Jumpy gradients from stop-gradient writes | Suboptimal long-range credit assignment          |
| Representation    | CPC alone is insufficient                 | Susceptible to spurious features                 |
| Architecture      | Fixed-size buffer                         | Memory inefficiency at scale                     |
| Recurrence        | Vanishing gradients in the LSTM           | Limited long-term learning within working memory |
| Structure         | No relational bias                        | Poor compositionality across memory entries      |

**Relation to earlier and later methods.** MRA can be summarized concisely relative to its two components: the working memory LSTM represents what the agent is thinking about right now, the episodic memory represents what the agent has experienced before, and the CPC objective is what allows the agent to understand patterns over time independently of reward. Together, these let the agent recall specific past experiences and generalize them to new situations, addressing the entanglement problem of Section 5.7.1 more directly than the single memory module of Section 5.7.3.1, at the cost of the credit-assignment and interpretability issues summarized above. One limitation in particular is not addressed by MRA: its episodic memory stores experience within a single trajectory, but offers no explicit mechanism for recognizing that the current task itself has been encountered before, in an earlier episode entirely. Section 5.7.3.3 addresses this gap directly.

### A.2.2.3. Been There, Done That: Meta-Learning with Episodic Recall



**Figure A.19.** – Episodic recall across a sequence of tasks with repetition. As the agent moves through Task 1, Task 2, and Task 3, the reinstatement gate remains closed (no relevant memory to recall), and a new entry is written to the DND at the end of each task. When Task 1 recurs, the reinstatement gate opens, and the LSTM cell state is reinstated from the corresponding DND entry rather than being rebuilt from scratch. Adapted from Ritter et al.



**Figure A.20.** – The reinstatement mechanism inside a single LSTM cell. The standard forget-gate and input-gate pathway (gray) updates the cell state as usual. In parallel, the memory retrieved from  $\text{DND}_t$  is passed through a  $\tanh$  nonlinearity and gated by the reinstatement gate  $r_t$  (pink), before being added into the cell state  $c_t$ . The updated context key and cell state are written to the dictionary, producing  $\text{DND}_{t+1}$ . Adapted from Ritter et al.

Section 5.7.3.2 closed by noting a gap left open by MRA: its episodic memory stores experience within a single trajectory, but offers no mechanism for recognizing that the current task itself has been encountered before, in an earlier episode entirely. Ritter et al. address this gap directly, targeting a specific failure of the recurrent meta-learners of Section 5.7.1: an LSTM-based agent such as  $\text{RL}^2$  learns how to learn new tasks quickly, but forgets everything about a task the moment its hidden state is reset at an episode boundary. If the same task reappears later, whether after a handful of episodes or after the agent has been through many other tasks in between, the agent has no way to recognize it, retrieve the solution it already found, or avoid re-exploring territory it has already explored. This is a direct consequence of the adaptation-via-input mechanism established in Section 5.7.1: the hidden state  $h_t$  is the entire adaptation state, and resetting it at episode boundaries discards everything, useful or not.



**Problem reformulation.** The standard meta-learning setup treats each task as an independent draw,  $m \sim \mathcal{D}$ . This paper replaces that assumption with a task sequence exhibiting recurrence,

$$(m_{n+1}, c_{n+1}) \mid (m_1, c_1), \dots, (m_n, c_n) \sim \Omega(\theta, \mathcal{D}),$$

where  $m$  denotes an MDP, or task,  $c$  denotes a context identifier associated with that task, and  $\Omega$  is a stochastic process controlling how and when tasks repeat across the sequence. This is a genuinely different problem formulation from everything presented so far in Section 5.7: rather than tasks being drawn i.i.d. from a fixed distribution, tasks are structured to reappear, and the agent is expected to exploit that structure. Figure A.19 illustrates this setting directly: Task 1 and Task 2 are each encountered once, but Task 3 is followed by a repetition of Task 1, and a well-designed agent should be able to exploit this repetition rather than treating it as an entirely new problem.

**Base architecture.** The starting point is the same recurrent formulation as Section 5.7.1: a sequence of states, actions, and rewards is fed into an LSTM, which learns an internal, implicit learning algorithm. As established there, this base architecture’s memory exists only within a single episode; nothing about a completed episode survives once its hidden state is reset.

**Adding episodic memory.** To address this, the agent is equipped with a Differentiable Neural Dictionary (DND), a key-value memory of the form  $\text{Memory} = \{(\text{key}_i, \text{value}_i)\}$ , shown as the DND table at the top of Figure A.19. The key stored for each entry is an embedding of the task context  $c$ , and, critically, the value stored is not an observation embedding, as in Sections 5.7.3.1 and 5.7.3.2, but the LSTM’s own *cell state*  $c_t$  at the point the entry was written. Retrieval proceeds by  $k$ -nearest-neighbor search over the stored keys, with the retrieved memory formed as a distance-weighted combination of the corresponding values,

$$c^{\text{ep}} = \sum_i w_i v_i, \quad w_i = \frac{1}{\text{distance}(c_{\text{query}}, k_i)}.$$

Entries whose stored context is closer to the current query context contribute proportionally more to the retrieved cell state  $c^{\text{ep}}$ , in the same spirit as the distance-weighted retrieval used by MRA in Section 5.7.3.2, but applied here to whole internal LSTM states rather than to observation-level values.

**Reinstatement.** The architectures of Sections 5.7.3.1 and 5.7.3.2 both retrieve memory and feed it into the network as an additional input, alongside the current observation. This paper takes a structurally different approach: rather than feeding  $c^{\text{ep}}$  in as an input, it is injected directly into the LSTM’s internal cell state through a modified update rule,

$$c_t = i_t \odot c_{\text{in}} + f_t \odot c_{t-1} + r_t \odot c^{\text{ep}},$$

where  $i_t$  and  $f_t$  are the standard LSTM input and forget gates, and  $r_t$  is a new *reinstatement gate*,

$$r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r).$$

Figure A.20 shows this mechanism in detail. The standard LSTM cell state pathway,  $c_{t-1}$  gated by  $f_t$ , combined with the candidate update  $i_t \odot \tanh(\cdot)$ , is shown running along the gray horizontal line. The reinstatement pathway runs in parallel: the memory retrieved from  $\text{DND}_t$  is passed through a  $\tanh$  nonlinearity, gated by  $r_t$ , and added into the cell state alongside the standard LSTM update. The reinstatement gate  $r_t$  therefore determines, at every timestep, how much of the retrieved past cell state should be blended into the current one. At the end of the timestep, the current context key and the resulting cell state are written into the dictionary, producing  $\text{DND}_{t+1}$ , shown on the right of Figure A.20.

**Interpretation.** This architecture assigns two clearly separated roles to its two memory systems, in the same spirit as the working/episodic split introduced by MRA in Section 5.7.3.2, but applied here across episodes rather than within one. The LSTM functions as working memory, responsible for the moment-to-moment computation of the current episode. The DND functions as long-term episodic memory, responsible for retaining a compressed record, the cell state itself, of every task the agent has previously encountered. The reinstatement gate  $r_t$  is what decides when to recall: when its value is near zero, as in the closed r-gate segments of Figure A.19, the LSTM behaves as an ordinary recurrent network, building up its state from scratch, exactly as in Section 5.7.1. When the gate opens, as in the rightmost segment of Figure A.19 corresponding to the repetition of Task 1, the agent effectively jumps back to the internal computational state it had reached the last time it solved this task, rather than merely being told, via an input, that it has seen this context before.

**Full mechanism.** At every timestep, the agent observes the current context  $c$ , queries the DND to retrieve  $c^{\text{ep}}$ , computes the LSTM update with reinstatement as described above, and acts using the resulting hidden state  $h_t$  to compute the policy, value function, or other actor-critic quantities, following the same downstream usage as in Section 5.7.1. At the end of each episode, the context and the final cell state,  $(c, c_T)$ , are written into the DND as a new entry, making the solution the agent reached available for reinstatement the next time a similar context is observed.

**Relation to earlier and later methods.** This method resolves the specific gap left open by MRA in Section 5.7.3.2 by moving what is stored in episodic memory from observation-level embeddings to whole internal states of the working memory itself, and by moving how that memory is used from feeding it in as an input to injecting it directly into the recurrent state. Both changes are motivated by the same underlying goal: allowing the agent to recognize and exploit task recurrence, something none of the architectures in Sections 5.7.1–5.7.3.2 are designed to do. What this method does not address, and shares with every architecture presented so far in Section 5.7.3, is how memory is combined once retrieved: both the distance-weighted sum used here and the attention-weighted sums used in Sections 5.7.3.1 and 5.7.3.2 collapse multiple memory entries into a single weighted average, discarding any relational structure between individual entries. Section 5.7.3.4 addresses this limitation directly, in a setting where preserving those relationships is essential for multi-step planning rather than single-step recall.

#### A.2.2.4. Rapid Task-Solving in Novel Environments (Episodic Planning Network)

Every method presented so far in Section 5.7.3 retrieves memory as a weighted average of stored entries: the softmax-weighted sum in Section 5.7.3.1, the nearest-neighbor-weighted sum in Section 5.7.3.2, and the distance-weighted sum in Section 5.7.3.3. Ritter et al. [93] argue that this shared retrieval mechanism has a fundamental limitation: collapsing multiple memory entries into a single combined vector,  $\sum_i \alpha_i m_i$ , destroys the graph structure connecting them. Retrieval and planning are not the same operation, and a memory system built only for retrieval cannot, on its own, support planning.

**Motivating example.** Suppose the agent has stored three transitions during exploration:  $m_1$  representing  $A \rightarrow B$ ,  $m_2$  representing  $B \rightarrow C$ , and  $m_3$  representing  $C \rightarrow \text{Goal}$ . Solving a task that requires reaching the goal from  $A$  requires composing these three transitions into a path,  $A \rightarrow B \rightarrow C \rightarrow \text{Goal}$ . This composition requires knowing that  $m_1$  connects to  $m_2$ , and that  $m_2$  connects to  $m_3$ . A weighted average

of  $m_1$ ,  $m_2$ , and  $m_3$  retains no such information: the connectivity between individual memories, and therefore the path itself, disappears entirely once they are combined. Iterative, multi-step reasoning over the structure of the environment is impossible if the memory system never represents that structure in the first place.

**Problem formulation.** This paper poses a more demanding question than the recurring-task setting of Section 5.7.3.3: can an agent enter a completely new environment and rapidly solve many different tasks inside it using only a single episode of experience? To do this well, the agent must explore the environment efficiently, remember what it discovers, build an internal model of the environment’s structure, and plan shortest paths to solve new tasks as they are posed. Formally, an environment is defined as  $e = (S, A, P, R)$ , where  $P$  denotes the environment dynamics and  $R$  is a distribution over reward functions. A task is then  $t = (S, A, P, r)$  with  $r \sim R$ : the state space, action space, and dynamics are fixed for a given environment, while the reward function, and therefore the goal, varies from task to task within it. Training optimizes the expected performance across both environments and the tasks sampled within each,

$$\mathbb{E}_{e \sim E} \left[ \mathbb{E}_{r \sim R_e} [J_{e,r}(\theta)] \right].$$

This formulation differs from the task-distribution setups used throughout the rest of Section 5.7: rather than each task corresponding to an entirely separate MDP, many tasks share the same environment and dynamics, and differ only in their reward function, so that a single episode of exploration can, in principle, support solving many of them.

**High-level idea.** Rather than compressing memories into a single representation once, as every retrieval-based method in this section does, the Episodic Planning Network (EPN) repeatedly reasons over the full set of stored memories using iterative self-attention. This is conceptually close to Value Iteration Networks and to message passing in graph neural networks: information about the goal is propagated backward through the graph of stored transitions, one step at a time, rather than being retrieved in a single lookup.

**Episodic memory as a graph.** EPN’s memory stores individual transitions rather than observation embeddings or internal states. At every timestep, the agent stores

$$m_t = [o_t, a_{t-1}, o_{t-1}],$$

recording the transition  $\mathbf{o}_{t-1} \xrightarrow{a_{t-1}} \mathbf{o}_t$ . Because each memory entry explicitly encodes which observation led to which other observation via which action, the resulting memory is naturally a graph, with observations as nodes and stored transitions as edges, rather than a flat, unordered buffer as in Sections 5.7.3.1–5.7.3.3.

**Goal-conditioned iterative planning.** The planner module operates on this graph in two steps. First, since planning must be directed toward a specific objective, the current goal embedding is appended to every stored memory. Second, the planner repeatedly updates its internal representation of the memories through iterative self-attention, following a residual update rule,

$$B_{i+1} = f(B_i + \phi(B_i)), \quad \phi(x) = \text{MHA}(\text{LayerNorm}(x)),$$

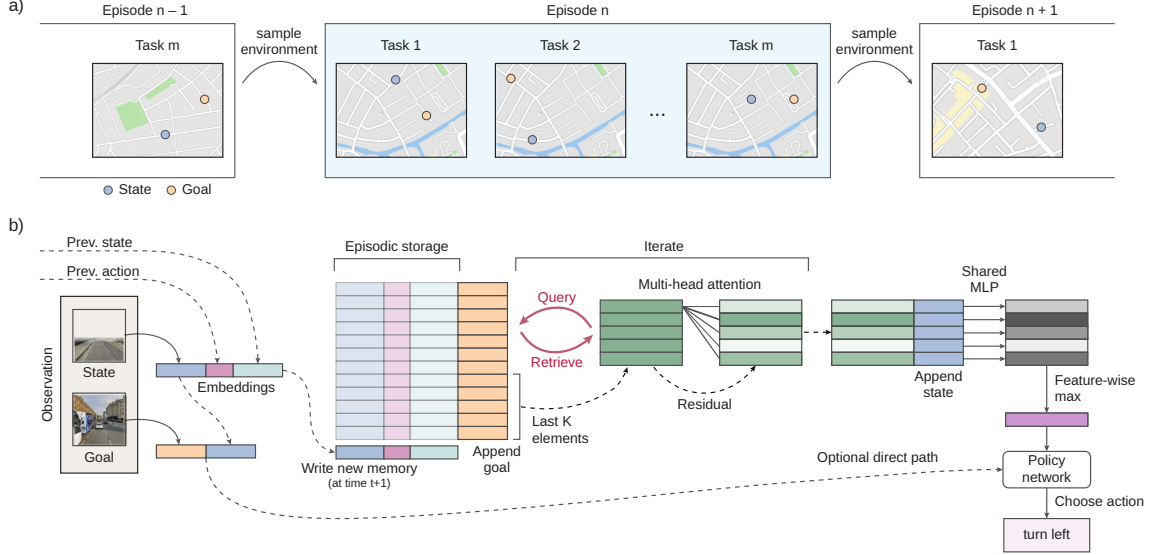
where MHA denotes multi-head attention,  $f$  is an MLP with a ReLU nonlinearity, and the addition of  $B_i$  and  $\phi(B_i)$  before applying  $f$  follows a standard residual, ResNet-style connection. Here  $B_i \in \mathbb{R}^{K \times d}$  is the planner’s internal belief representation after  $i$  iterations, with  $K$  the number of stored memories and  $d$  the embedding dimension. Crucially, the attention in  $\phi(B_i)$  is computed among the memories themselves, each memory attending to every other memory, rather than between a single query vector and a set of memories, as in every read operation in Sections 5.7.3.1 through 5.7.3.3. Each iteration allows information to propagate one step further across the transition graph: since the goal has been appended to every memory, this propagation functions like a learned form of value iteration, with information about the goal flowing backward through the graph, one edge at a time, at every application of  $\phi$ . Table A.9 summarizes what successive iterations come to represent.

**Table A.9.** – Interpretation of successive planning iterations in EPN.

| Iteration | Meaning                                                |
|-----------|--------------------------------------------------------|
| $B_0$     | Raw transition memories, no propagation yet            |
| $B_1$     | Local connectivity knowledge (immediate neighbors)     |
| $B_2$     | Short paths discovered                                 |
| $B_3$     | Longer paths inferred                                  |
| $B_k$     | Approximate global value map over the transition graph |

As this table indicates, the number of attention iterations directly determines the planning horizon: after one iteration, only memories adjacent to the goal have propagated information about it; after several iterations, that information has reached progressively more distant parts of the graph, eventually approximating a global

value map over all stored transitions. This is the property that a single weighted-average retrieval, used throughout Sections 5.7.3.1–5.7.3.3, cannot provide, since it performs no analogous multi-step propagation across memory entries.



**Figure A.21.** – Episodic Planning Network architecture. Stored transitions form a graph over observations, the goal embedding is appended to every memory, and the planner module iteratively updates its belief representation  $B_t$  through repeated self-attention among memories, propagating goal-relevant information across the graph before the policy network selects an action. Adapted from Ritter et al. [93].

**Policy network.** After the planning iterations complete, the resulting representation is used by a policy network conditioned on both the current observation and the goal,

$$\pi(a \mid o, G),$$

where  $G$  denotes the goal embedding threaded through the planning process described above.

**Drawbacks.** The iterative self-attention mechanism that gives EPN its planning capability also limits its scalability: because every planning iteration attends over the full set of stored memories, the architecture is not well suited to long episodes, where the number of stored transitions, and therefore the cost of each planning iteration, grows accordingly.

**Synthesis of Section 5.7.3.** The four architectures presented in this subsection can be read as a single line of development, each addressing a specific limitation of the one before it. The Minecraft architectures of Section 5.7.3.1 first separate memory storage from the main recurrent state, introducing key-value read and write operations in place of a single evolving hidden vector. MRA, in Section 5.7.3.2, formalizes this into two explicit memory systems, adds a mechanism for tractable credit assignment through memory writes, and shows that memory alone is insufficient without an auxiliary, reward-independent representation learning objective. Been There, Done That, in Section 5.7.3.3, shows that what is stored matters as much as how it is retrieved: storing whole internal states rather than observation embeddings, and injecting them directly into the recurrent state through reinstatement, allows an agent to recognize and exploit task recurrence across episode boundaries, something none of the earlier architectures support. Finally, EPN, in this subsection, shows that how retrieved memories are combined matters just as much: replacing weighted-average retrieval with iterative self-attention among memories preserves the relational structure needed for multi-step planning, at the cost of scalability to long episodes.

Across all four methods, the underlying adaptation-via-input principle from Section 5.7.1 is preserved: network parameters are trained once and held fixed at test time, and adaptation occurs entirely through the evolving content of external memory and the representations computed from it. Section 5.7.4 turns to a different family of black-box methods entirely, in which the adaptation state is no longer memory read by a fixed policy, but the network’s own weights, modified directly through Hebbian and neuromodulated plasticity.

### A.2.3. Adaptation via Self-Modifying Weights

Sections 5.7.1 through 5.7.3 all place the adaptation state outside the network’s own weights, whether in a recurrent hidden state, an attention context, or an explicit external memory module. In every case,  $\theta$  itself never changes during interaction with a task; only what the policy reads changes. This subsection turns to a structurally different family of black-box methods, in which the adaptation state *is* the network’s own weight matrix. As introduced in the overview to Section 5.7, this is a direct generalization of the recurrent formulation: rather than

$$h_t = f_\theta(h_{t-1}, o_t, a_{t-1}, r_{t-1}),$$

these methods maintain an evolving weight matrix  $W_t$ , updated at every timestep by a local, differentiable rule that is itself learned during meta-training,

$$W_t = W_{t-1} + \Delta W_t, \quad a_t \sim \pi_\theta(a_t \mid o_t, W_t).$$

No gradient descent on  $\theta$  occurs during interaction in either formulation; what differs is only where the adaptation state lives, and how it is written to. This subsection traces this idea from its origin as a biologically inspired local learning rule, through successive refinements that make the rule modulated, fully self-referential, and, in one case, trained without backpropagation at all.

### A.2.3.1. Differentiable Hebbian Plasticity and Neuromodulation

Miconi et al. introduce the foundational mechanism of this subsection in two stages, published as two papers that share the same core architecture and are presented here as a single method. The first, [94], establishes differentiable Hebbian plasticity: a network that modifies its own weights during its lifetime, using a biologically inspired local learning rule made differentiable so that it can itself be optimized by backpropagation. The second, [95], extends this mechanism with neuromodulation, letting the network decide, at every timestep, when and how strongly its own plasticity should occur, rather than applying it at a fixed, uniform rate. Both directly target the same limitation that motivates the recurrent formulation of Section 5.7.1: once training finishes, the weights of a standard neural network are fixed, and the network cannot continue to adapt during deployment. Rather than relying on an external recurrent state to carry adaptation, as in Section 5.7.1, these methods modify the network’s own connection weights during interaction with a task.

**Key idea.** Rather than learning only a fixed set of weights  $\mathbf{w}$ , the network additionally learns how much each individual connection should change in response to experience. Plasticity itself, not just the weights, becomes a learned, differentiable quantity.

**Model structure.** Each connection in the network is decomposed into two parts: a fixed weight  $\mathbf{w}_{ij}^\ell$ , identical in role to a standard neural network weight, and a plastic component,  $\alpha_{i,j} \cdot \text{Hebb}_{i,j}^\ell(t)$ , which evolves over the course of an episode. The effective weight used in the forward pass at time  $t$  is the sum of the two,

$$\mathbf{w}_{i,j}^\ell + \alpha_{i,j} \cdot \text{Hebb}_{i,j}^\ell(t),$$

and neural activations are computed using this effective weight rather than the fixed weight alone,



$$x_j(t) = \sigma \left( \sum_i \left[ w_{i,j} x_i(t-1) + \alpha_{i,j} \text{Hebb}_{i,j}(t) x_i(t-1) \right] \right).$$

**Hebbian trace and clipping.** In its base form, the Hebbian trace is updated at every timestep according to the classical Hebbian principle, correlating pre- and post-synaptic activity, with exponential decay toward the new value,

$$\text{Hebb}_{i,j}(t+1) = \eta x_i(t-1) x_j(t) + (1 - \eta) \text{Hebb}_{i,j}(t).$$

This trace stores a running estimate of the correlation between neurons  $i$  and  $j$ : it acts as a fast, decaying form of memory, distinct in kind from the persistent external memories of Section 5.7.3, since it is written into the weights themselves rather than into a separate module, and it decays automatically over time rather than persisting until explicitly overwritten or evicted. Backpropamine reformulates this update with an explicit clipping operation, to prevent unbounded growth of the trace over a long episode,

$$\text{Hebb}_{ij}(t+1) = \text{Clip}(\text{Hebb}_{ij}(t) + \eta x_i(t-1) x_j(t)), \quad \text{Clip}(x) = \begin{cases} 1 & \text{if } x > 1 \\ -1 & \text{if } x < -1 \\ x & \text{otherwise.} \end{cases}$$

**From a fixed rate to a learned, network-generated signal.** In the base formulation, the trace is updated by every pre- and post-synaptic co-activation at a fixed rate  $\eta$ , regardless of whether that particular co-activation was actually useful for solving the task. Backpropamine replaces this fixed rate with a dynamic signal  $M(t)$ , produced by the network itself as a function of its own current activity,

$$\text{Hebb}_{ij}(t+1) = \text{Clip}(\text{Hebb}_{ij}(t) + M(t) x_i(t-1) x_j(t)).$$

The signal  $M(t)$  controls not only when plasticity occurs and how strongly, but also its sign: a negative value of  $M(t)$  can weaken a connection in response to the same co-activation pattern that a positive value would strengthen. This is what gives the resulting network its self-modifying character: the network does not merely have plasticity built into it by design, it actively regulates its own plasticity in response to what it is currently experiencing, drawing an explicit analogy to dopamine-modulated synaptic plasticity in biological neural circuits, from which the name

Backpropamine, combining backpropagation with this dopamine-like modulation, is derived.

**Retroactive neuromodulation.** The simple form above still updates the Hebbian trace using only the immediately preceding activations. This is a poor match for many reinforcement learning settings, where a reward signal, and therefore the modulatory signal it should produce, may only become available some time after the activity that deserves credit for it. Retroactive neuromodulation, inspired by the timing of dopaminergic signaling in biology, addresses this with an eligibility trace that accumulates recent activity independently of the modulatory signal,

$$E_{ij}(t+1) = (1 - \eta)E_{ij}(t) + \eta x_i(t-1) x_j(t),$$

with the Hebbian update then driven by the modulatory signal acting on this accumulated trace, rather than on the instantaneous activation product,

$$\text{Hebb}_{ij}(t+1) = \text{Clip}(\text{Hebb}_{ij}(t) + M(t) E_{ij}(t)).$$

Because  $E_{ij}(t)$  already summarizes recent co-activation by the time  $M(t)$  arrives, this allows learning to happen later, when a modulatory or reward signal finally becomes available, rather than only at the exact moment the relevant activity occurred. This directly targets one of the recurring limitations noted throughout Section 5.7: credit assignment across time, addressed in Section 5.7.1 only implicitly through the recurrent state’s own gradient flow, and in Section 5.7.3.2 through a stop-gradient trick at memory writes, is addressed here by decoupling when activity is recorded from when it is consolidated into a lasting weight change.

**Two-level optimization.** Training operates on two clearly separated timescales in both papers, matching the structure used throughout Section 5.7. Within an episode, only the Hebbian trace, and where applicable the eligibility trace, update at every timestep, constituting fast, within-lifetime learning; the base weights  $\mathbf{w}$ , the plasticity coefficients  $\alpha$ , the Hebbian rate  $\eta$ , and, in Backpropamine, the modulator network parameters  $\theta_M$ , all remain fixed throughout the episode. Across episodes, these outer-loop parameters are optimized by backpropagation through the entire episode, using the accumulated task loss, framed by Backpropamine explicitly as evolution constituting the outer loop of meta-learning and plasticity constituting the inner loop of fast adaptation. The meta-learned outer-loop parameters here are not the weights that solve the task directly, as in Section 5.6, but the plasticity mechanism itself: what should be plastic, by how much, how quickly plastic traces should decay, and, in the modulated case, when plasticity should occur at all.

**Algorithm.** Algorithm 58 presents the general procedure, parameterized so that setting the modulatory signal  $M_t \equiv \eta$  for all  $t$  recovers the base differentiable plasticity formulation of [94] exactly, while a learned modulator network  $M_t = \text{modulator}_{\theta_M}(x_t)$  recovers the full Backpropamine formulation of [95].

---

**Algorithm 58** Meta-Reinforcement Learning with (Neuromodulated) Differentiable Hebbian Plasticity

---

**Require:** Task distribution  $p(\tau)$ , parameters  $\theta = \{w, \alpha, \eta, \theta_M\}$  ( $\theta_M$  omitted for the unmodulated variant)

**for** each meta-iteration **do**

Sample a batch of episodes

**for** each episode **do**

Hebb  $\leftarrow 0$ ; (optional)  $E \leftarrow 0$ ;  $h \leftarrow$  initial state

**for**  $t = 1, \dots, T$  **do**

$w_{\text{eff}} = w + \alpha \cdot \text{Hebb}$

$x_t = \text{NN}(x_{t-1}, w_{\text{eff}})$

$M_t \leftarrow \eta$  **or**  $M_t = \text{modulator}_{\theta_M}(x_t)$   $\triangleright$  unmodulated vs. neuromodulated

**if** simple update **then**

Hebb  $+= M_t \cdot \text{outer}(x_{t-1}, x_t)$

**else if** retroactive update **then**

$E = (1 - \eta) \cdot E + \eta \cdot \text{outer}(x_{t-1}, x_t)$

Hebb  $+= M_t \cdot E$

**end if**

Hebb = clip(Hebb, -1, 1)

action/output = policy( $x_t$ ); observe next input, reward

**end for**

Compute loss  $L$

**end for**

Update  $\theta$  using Backpropagation Through Time (BPTT)

**end for**

---

**Limitations.** Several questions are left open by both formulations. The stability of the Hebbian update is not guaranteed and depends on the learned coefficients remaining well behaved, even with clipping in place. Scalability to very deep architectures is untested, since every plastic layer adds a full matrix of Hebbian traces that must be maintained and updated at every timestep. The interaction between this mechanism and modern architectures such as Transformers is not explored. Neither method performs explicit task inference: the Hebbian trace, modulated or not, adapts to whatever correlations are present in the current episode’s activity, with no explicit representation of which task is currently being solved, in contrast to the task inference methods of Section 5.9.

**Relation to later methods.** Together, these two papers establish the structural elements that every subsequent method in Section 5.7.4 builds on: an effective weight that is the sum of a fixed and a plastic component, a Hebbian trace updated by pre- and post-synaptic correlation, and, once modulated, a learned signal governing when that trace should actually be written. What remains fixed across both papers is the underlying mechanism by which weights are modified: a Hebbian outer-product update added into an existing weight matrix, computed by a separate slow network acting on a separate fast one. Section 5.7.4.2 removes this separation entirely, introducing an architecture in which every quantity, including the update rule’s own inputs, is generated by the same self-referential weight matrix that is being modified.

#### A.2.3.2. From Fast Weight Programmers to a Fully Self-Referential Weight Matrix

Section 5.7.4.1 modifies a network’s weights through a Hebbian trace added on top of a separately maintained fixed weight, with the update itself computed from the network’s own current activations. This subsection presents a different route to the same goal, self-modifying weights, taken by Irie et al. [96], arising from a separate line of work on Fast Weight Programmers, and shows how it can be pushed to its logical conclusion: a network with no separation between the part that computes updates and the part that is updated, at all.

**Motivation.** In a standard neural network, once training finishes, the weight matrix is fixed. Since the weight matrix of a network can be understood as encoding a program, in the sense that it determines the function the network computes, a network whose weights never change after training is a network whose program can never be revised at runtime. The goal of this line of work is to build a network whose own weight matrix updates itself during deployment.

**Fast Weight Programmers.** The original formulation of this idea uses two networks: a slow network, which generates rapid weight changes, and a fast network, which receives those changes and is updated by them.

Process input (slow net)  $\rightarrow$  Generate update rule for fast net  $\rightarrow$  Fast net produces output with

Auto-regressive Transformers can themselves be understood as a form of fast-weight memory. Writing the query, key, and value projections at timestep  $i$  as  $\mathbf{a}^{(i)}, \mathbf{b}^{(i)} =$

$\mathbf{W}_a \mathbf{x}^{(i)}, \mathbf{W}_b \mathbf{x}^{(i)}$ , the fast weight matrix accumulates an outer product of these projections at every step,

$$\mathbf{W}^{(i)} = \sigma(\mathbf{W}^{(i-1)} + \mathbf{a}^{(i)} \otimes \mathbf{b}^{(i)}), \quad \mathbf{y}^{(i)} = \mathbf{W}^{(i)} \mathbf{x}^{(i)},$$

where  $\otimes$  denotes the outer product,  $\mathbf{u} \otimes \mathbf{v} = \mathbf{u} \mathbf{v}^\top$ . The second equation stores a key-value pair from the current input into the fast weight matrix at every timestep. If  $\mathbf{w}_t = \sum_{i=0}^t k_i \otimes v_i$ , then querying with a key  $k_j$  recovers  $k_j \cdot \mathbf{w}_t = \sum_{i=0}^t (k_j \cdot k_i) \otimes v_i$ , which reduces to  $v_j$  whenever the keys are orthonormal, since  $k_j \cdot k_i = 0$  for  $i \neq j$  and 1 for  $i = j$ . Multiplying the accumulated weight matrix  $\mathbf{W}$  by a key  $\mathbf{K}$  therefore behaves like autoregressive attention, giving a direct structural link between the weight-modification methods of this subsection and the attention-based methods of Section 5.7.2: both implement a form of associative recall, one by writing into a weight matrix, the other by attending over a sequence of stored vectors. Table A.10 summarizes the resulting trade-off between the two.

**Table A.10.** – Fast Weight Programmers compared to standard Transformer attention.

| Property            | Transformer | Fast Weight Programmer |
|---------------------|-------------|------------------------|
| Complexity          | Quadratic   | Linear                 |
| Context length      | Bounded     | Unbounded              |
| Retrieval precision | High        | Low                    |
| Expressivity        | Low         | High (with delta rule) |

Retrieval precision here refers to the fraction of retrieved items that are actually relevant,  $\text{Retrieval Precision} = \text{Relevant Retrieved Items} / \text{Total Retrieved Items}$ , and expressivity refers to the network’s ability to model complex patterns. The plain outer-product update above already gives Fast Weight Programmers linear complexity and unbounded context, at the cost of lower retrieval precision than full attention, since interference between stored key-value pairs grows as more of them are written into the same fixed-size matrix. A more expressive mechanism, referred to as the delta rule, closes much of this gap.

**The delta-rule update.** At every timestep  $t$ , the slow network generates four quantities from the current input: a key  $k_t$ , a value  $v_t$ , a query  $q_t$ , and a learning rate  $\beta_t$ , all as functions of the same slow weight matrix,  $k_t, v_t, q_t, \beta_t = \mathbf{W}_{\text{slow}} \cdot \mathbf{X}$ . The fast weight matrix is then updated not by simply adding a new outer product, but by first reading out the value currently associated with the key, and writing only the correction needed to move toward the new target value,

$$W_t = W_{t-1} + \sigma(\beta_t)(v_t - \bar{v}_t) \otimes \phi(k_t), \quad \bar{v}_t = W_{t-1}\phi(k_t),$$

where  $\phi(k)$  normalizes the key vector, for example via softmax, so that its entries sum to one. The correction term  $v_t - \bar{v}_t$  is a form of Widrow-Hoff, or delta-rule, learning: rather than blindly accumulating outer products, as in the plain Fast Weight Programmer above, the update is proportional to the error between the value currently stored at key  $k_t$  and the new value the network wants to store there, which is what gives the delta rule its higher expressivity relative to the plain outer-product accumulation. Reading from the fast weight matrix uses the same normalized-query mechanism,

$$y_t = W_t \phi(q_t).$$

Interpreting these three quantities gives a direct correspondence to the read/write memories of Section 5.7.3: the key specifies where to write, the value specifies what to write, and the query specifies what to read, exactly the roles played by keys and values in the external memory architectures of that subsection, except that here reading and writing occur through matrix-vector products against the network’s own weight matrix, rather than through attention over a separate memory module.

**Limitation of Fast Weight Programmers.** Despite this delta-rule improvement, the two-network architecture above has a structural limitation: the slow network’s own weights,  $W_{\text{slow}}$ , are learned offline during meta-training and remain fixed thereafter. Only the fast weights change during interaction with a task. The mechanism is therefore not fully self-referential: the network cannot access, modify, or update its own parameters using itself, since the parameters doing the modifying,  $W_{\text{slow}}$ , are never themselves the target of any modification.

**The Self-Referential Weight Matrix (SRWM).** Irie et al. [96] remove this separation entirely. Rather than a slow network generating updates for a separate fast network, there is a single weight matrix, and every quantity involved in modifying that matrix, including the matrix’s own inputs, is generated by the matrix acting on itself. At every timestep,

$$y_t, k_t, q_t, \beta_t = W_{t-1} \phi(x_t),$$

the same weight matrix that is about to be updated produces the current output, the key, the query, and the learning rate, all from a single forward pass over the current

input. The read, generate, and update steps that follow use exactly the delta-rule mechanism introduced above, but applied entirely within this single matrix,

$$\bar{v}_t = W_{t-1}\phi(k_t), \quad v_t = W_{t-1}\phi(q_t), \quad W_t = W_{t-1} + \sigma(\beta_t)(v_t - \bar{v}_t) \otimes \phi(k_t).$$

Table A.11 summarizes the interpretation of each quantity.

**Table A.11.** – Interpretation of the components of a Self-Referential Weight Matrix update.

| Component   | Meaning                      |
|-------------|------------------------------|
| $k_t$       | Where to modify              |
| $q_t$       | How to compute the new value |
| $v_t$       | New value                    |
| $\bar{v}_t$ | Old value                    |
| $\beta_t$   | Learning rate                |

Because  $W_{t-1}$  is used both to produce  $k_t, q_t, \beta_t$  and to compute  $\bar{v}_t$  and  $v_t$ , the matrix is genuinely modifying itself using its own current state, with no external slow network supplying any of the quantities involved. This is the sense in which SRWM is fully self-referential, and it is a direct, more extreme realization of the same principle introduced at the start of Section 5.7.4: the adaptation state is the weight matrix itself, and here the mechanism that decides how that matrix changes is not a separately trained component, but a computation performed by the matrix on itself. Irie et al. [96] evaluate this mechanism in both supervised few-shot learning and multi-task reinforcement learning on procedurally generated environments, directly relevant to the sim-to-real and multi-task settings considered elsewhere in this thesis.

**Relation to earlier and later methods.** SRWM and Section 5.7.4.1’s Hebbian mechanisms both write into a weight matrix using an outer product at every timestep, and both remove any explicit external memory module, placing the entire adaptation state inside the weights rather than in a structure like those of Section 5.7.3. They differ in what computes the update: Section 5.7.4.1 computes  $\text{Hebb}_{ij}(t)$  from raw pre- and post-synaptic activations, optionally gated by a separately parameterized modulator network, while SRWM computes every one of  $k_t, q_t, v_t, \beta_t$  from the same matrix being modified, with no separate modulator or slow network involved at all. Section 5.7.4.3 takes this outer-product self-modification mechanism and extends it with multi-step recursive updates and an explicit optimal-transport framing of

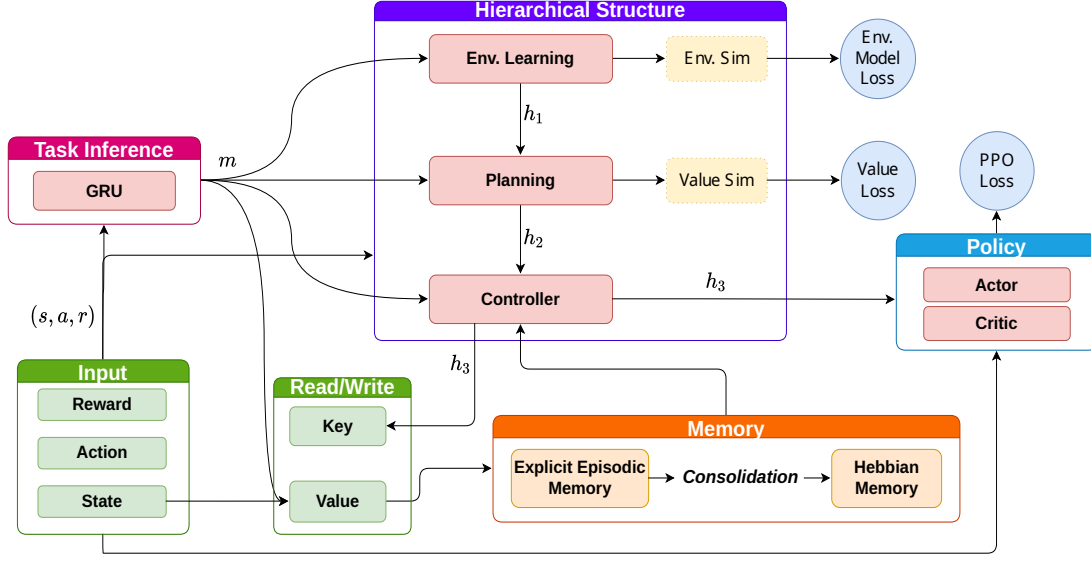
meta-reinforcement learning, applying the same underlying principle in a reinforcement learning setting rather than the general sequence-modeling setting considered here.

#### A.2.3.3. BIMRL: Brain-Inspired Meta-Reinforcement Learning

The methods presented so far in Section 5.7.4 isolate a single mechanism, a Hebbian trace, a neuromodulatory signal, or a fully self-referential weight matrix, and study it largely in isolation. Rohani et al. [97] take a different approach, asking a broader question: how does the human brain adapt to a new task after only a handful of experiences? Their answer is that biological adaptation is not the product of a single mechanism but of several interacting systems operating together: a system that infers which task is currently being solved, a system that predicts how the environment behaves, a system that plans using those predictions, both long-term and short-term memory, and a mechanism for directed exploration. BIMRL is an architecture that instantiates each of these systems as a distinct module, one of which is a Hebbian associative memory closely related to the plasticity mechanisms of Section 5.7.4.1, embedded within a much larger system. Because it combines mechanisms from several families surveyed in this chapter, rather than extending any one of them in isolation, it is presented here as a hybrid architecture that closes Section 5.7.4.

**Architecture overview.** Figure A.22 shows the full architecture. The raw input at every timestep is the transition tuple  $(s, a, r)$ , shown at the bottom left. This input feeds a task inference module, a GRU that summarizes the trajectory so far into a task embedding  $m$ , playing the same role as the task-embedding mechanisms discussed elsewhere in this thesis, but here feeding into a much larger hierarchical structure rather than directly conditioning the policy. The task embedding  $m$ , together with the raw input, feeds three hierarchically arranged components: an environment learning module, a planning module, and a controller, connected in sequence via intermediate hidden representations  $h_1$ ,  $h_2$ , and  $h_3$ . The controller additionally reads from and writes to an external memory system, and its output feeds an actor-critic policy head trained with PPO.





**Figure A.22.** – BIMRL architecture. The task inference module (GRU) produces a task embedding  $m$  from the transition  $(s, a, r)$ , feeding a hierarchical structure of environment learning, planning, and control modules, connected via  $h_1$ ,  $h_2$ , and  $h_3$ . The controller reads from and writes to a memory module combining explicit episodic memory and Hebbian memory via a consolidation process, and its output drives an actor-critic policy trained with PPO. Adapted from Rohani et al. [97].

**World model.** Rather than optimizing reward directly, the first hierarchical module, environment learning, is trained to predict the environment’s dynamics: the next state given the current state and action, the reward given the current state and action, and the inverse dynamics, which action caused a given transition between two states. This module maximizes the likelihood of the observed trajectory given the task embedding,  $P(\tau | m)$ , and additionally predicts a sequence of future value estimates  $V_{t+1}, \dots, V_{t+n}$ . Its training objective is a negative evidence lower bound,

$$\mathcal{L}_{\text{env}} = -\mathbb{E}_{q_\phi(m|\tau_x)} [\log p_\theta(\tau_{H^+} | m)] - \text{KL}(q_\phi(m | \tau_x) \| p_\theta(m)),$$

where the reconstruction term decomposes across the predicted trajectory as

$$\begin{aligned}
\log p(\tau_{:H^+} \mid m, a_{0:n}) &= \log p(s_0 \mid m) \\
&+ \sum_{i=0}^{H^+-1} \sum_{j=0}^{\min(n,i)} \log p(s_{i+1} \mid s_{i-j}, a_{i-j:i}, m) \\
&+ \sum_{i=0}^{H^+-1} \sum_{j=0}^{\min(n,i)} \log p(r_{i+1} \mid s_{i-j}, a_{i-j:i}, m) \\
&+ \sum_{i=0}^{H^+-1} \sum_{j=0}^{\min(n,i)} \log p(a_{i+1} \mid s_{i-j:i+1}, m).
\end{aligned}$$

This gives the environment learning module a training signal entirely independent of task reward, in the same spirit as the CPC auxiliary objective used by MRA in Section 5.7.3.2, though here the predictive objective covers states, rewards, and inverse dynamics jointly rather than only future latent observations.

**Planning layer.** The second hierarchical module takes the world model’s output  $h_1$  and asks a forward-looking question: if a given sequence of actions were executed from here, how much reward would result? Given the world model features  $h_1$ , the current state, and a sequence of candidate future actions, this module predicts the resulting future return, trained with a multi-step value regression loss,

$$\mathcal{L}_{\text{value}} = \sum_{i=0}^{H^+-1} \sum_{j=0}^n \left\| V_\psi(s_i, a_i, \dots, a_{i+j}, h^{(2)}) - G_{i+j+1:i+j+1+k} \right\|^2, \quad G_{t:t+k} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{k-1} R_{t+k} + \gamma^k V_\theta(s_{t+k})$$

**Controller.** The third hierarchical module combines the task embedding  $m$ , the planning module’s output  $h_2$ , the world model’s features, and information retrieved from memory, producing a controller hidden state  $h_{3,t} = f(h_{2,t}, o_t, \text{mem}_t)$ , where  $\text{mem}_t$  is supplied by the memory module described next. The controller’s output drives an actor-critic policy trained with the standard clipped PPO objective,

$$\mathcal{L}_{\text{PPO}} = \mathcal{L}_{\text{actor}} + c_v \mathcal{L}_{\text{critic}} - c_e \mathcal{L}_{\text{entropy}},$$

with

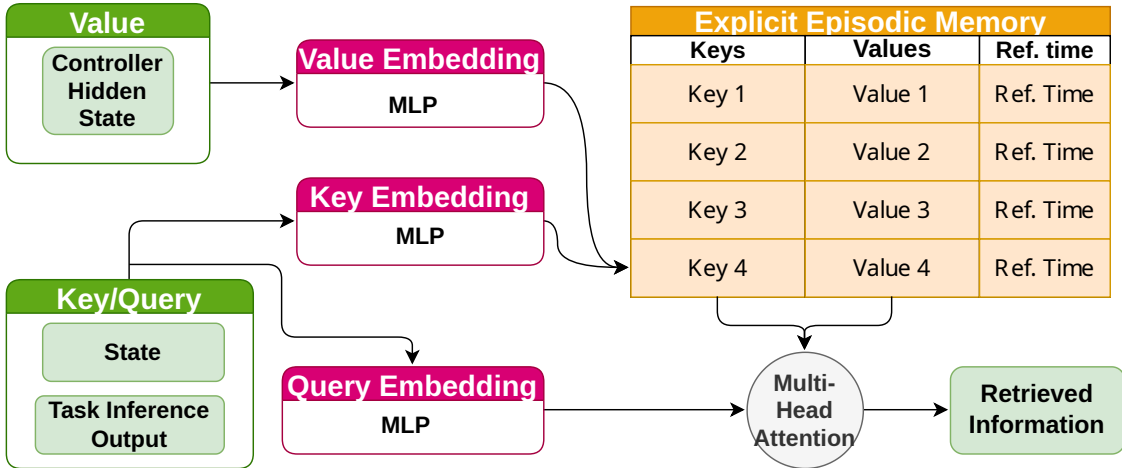
$$\mathcal{L}_{\text{actor}} = -\mathbb{E} \left[ \min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t) \right], \quad r_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)},$$

$$\mathcal{L}_{\text{critic}} = (V_{\theta}(s_t) - G_t)^2, \quad \mathcal{L}_{\text{entropy}} = \mathbb{E}[\mathcal{H}(\pi_{\theta}(\cdot | s_t))].$$

The three modules are trained jointly with a single combined objective,

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{env}} + \lambda_1 \mathcal{L}_{\text{value}} + \lambda_2 \mathcal{L}_{\text{PPO}}.$$

**Memory module.** BIMRL’s memory system consists of two components, shown at the bottom right of Figure A.22: an explicit episodic memory, specific to the current task, and a Hebbian memory that persists and accumulates information across tasks. This split mirrors the working/episodic distinction introduced by MRA in Section 5.7.3.2, but adds a second, longer-lived Hebbian component on top of it, and the read and write operations for the episodic component are themselves shown in Figure A.23.



**Figure A.23.** – Read and write operations for BIMRL’s explicit episodic memory. The controller hidden state produces a value embedding, while the current state and task inference output jointly produce a key embedding for writing, and a query embedding for reading. Retrieval uses multi-head attention over the stored keys and values, together with each slot’s reference time, producing the retrieved information used by the controller. Adapted from Rohani et al. [97].

As shown in Figure A.23, each episodic memory slot stores a key, a value, and a reference time. The controller’s hidden state is embedded via an MLP to produce the value written into memory, while the current state together with the task inference

output is embedded to produce both the key used for writing and the query used for reading, with retrieval performed via multi-head attention over the stored key-value pairs, in the same spirit as the attention-based reads of Section 5.7.3, but restricted here to selecting which memories should participate in the Hebbian consolidation step described next, rather than being used directly as the retrieved output.

**Memory consolidation.** Rather than retaining every episodic memory slot indefinitely, BIMRL selects only the most important slots for long-term consolidation, in a process explicitly framed as analogous to the hippocampus in biological memory systems. Each memory slot  $i$  is assigned a reference score based on how much attention it received over the course of the episode,

$$r_i = \frac{\sum_{t=s_i}^H W_i^t}{H - s_i}, \quad W^t = \text{MultiHeadAttention}(Q, K, V) = [W_i^t]_{i=1}^N,$$

where  $s_i$  is the timestep at which slot  $i$  was written and  $H$  is the episode length. The top- $K$  slots by this score,  $\mathcal{I} = \text{TopK}(r_1, \dots, r_N)$ , are selected for consolidation into the long-term Hebbian memory.

**Hebbian consolidation.** The selected slots are then used to update a persistent Hebbian associative memory matrix, using an update rule closely related to the differentiable plasticity mechanism of Section 5.7.4.1, but here including an explicit potentiation-decay structure rather than a simple decaying trace,

$$W^{\text{assoc}} \leftarrow W^{\text{assoc}} + \Delta W^{\text{assoc}}, \quad \Delta W_{kl}^{\text{assoc}} = \gamma^+ (w_{\max} - W_{kl}^{\text{assoc}}) v_{t,k}^s k_{t,l}^s - \gamma^- W_{kl}^{\text{assoc}} (k_{t,l}^s)^2,$$

where  $k_t^s$  and  $v_t^s$  are the key and value of a selected event,  $\gamma^+$  is a potentiation coefficient,  $\gamma^-$  is a decay, or regularization, coefficient, and  $w_{\max}$  bounds the maximum Hebbian weight, playing a role analogous to the clipping operation used in Backpropamine in Section 5.7.4.1. Retrieval from this Hebbian memory is a simple linear readout,  $\hat{v}_t = W^{\text{assoc}} k_t$ , and the final memory signal fed to the controller blends the two memory systems,

$$ep_t = \sum_{i \in \mathcal{I}} W_i v_i, \quad mem_t = \beta_t ep_t + (1 - \beta_t) \hat{v}_t,$$

giving the controller access to both the current episode’s most salient specific experiences, through  $ep_t$ , and a long-term, gradually consolidated associative memory that persists and accumulates across tasks, through  $\hat{v}_t$ .

**Intrinsic reward.** BIMRL augments the extrinsic task reward with an intrinsic exploration signal,  $r_t = r_t^{\text{ext}} + \beta r_t^{\text{int}}$ , combining two components: a novelty term, measuring the distance between the current latent state and previously visited latent states within the same episode,  $d_i = \|z_t - z_i\|^2$ , with a larger distance indicating higher novelty, and a prediction-error term measuring how poorly the world model anticipated the current transition,  $e_t = \|s_{t+1} - \hat{s}_{t+1}\|^2 + \|r_{t+1} - \hat{r}_{t+1}\|^2$ . The two are combined multiplicatively,  $r_t^{\text{int}} = \text{Novelty} \times \text{Prediction Error}$ , so that exploration is rewarded most where the agent is both in unfamiliar territory and unable to predict what will happen next.

**Strengths and limitations.** BIMRL’s explicit separation of task inference, environment modeling, planning, and control, each as its own trained module, gives it a clearer functional decomposition than any single end-to-end recurrent or attention-based architecture in this chapter, and its memory system, combining episodic recall with longer-term Hebbian consolidation, goes beyond what a single RNN hidden state or a single external memory module can offer. Local Hebbian plasticity allows rapid online adaptation without requiring a gradient update at test time, and the built-in curiosity-driven intrinsic reward gives the agent a principled exploration mechanism rather than relying on the exploration behavior that emerges implicitly from the meta-training objective, as in Section 5.7.1. Against this, the architecture is considerably more complex than any other method in this chapter, with several interacting modules and a combined loss with multiple weighted terms that must be balanced during training. Its empirical demonstration is primarily on MiniGrid, a comparatively simple grid-world benchmark, and its scalability to large-scale continuous control or large-scale multi-agent settings, both directly relevant to the benchmarking objectives of this thesis, has not been established.

**Relation to the rest of Section 5.7.4.** BIMRL illustrates that the mechanisms surveyed individually in this subsection are not mutually exclusive. Its Hebbian associative memory is a direct descendant of the plasticity mechanism in Section 5.7.4.1, applied not to the network’s main task-solving weights, as in Sections 5.7.4.1 through 5.7.4.3, but to a dedicated associative memory matrix operating alongside a separately trained controller. This closes Section 5.7.4: rather than adding a further refinement to the outer-product self-modification thread of Sections 5.7.4.2 and 5.7.4.3, BIMRL demonstrates that Hebbian plasticity can equally be deployed as one component within a larger, modular architecture, combined with task inference, world modeling, and explicit episodic memory of the kind surveyed in Section 5.7.3, rather than standing as a self-contained adaptation mechanism on its own.

#### A.2.3.4. Meta Optimized Dynamical Synapses (MetODS)

Chalvidal et al. [98] take the outer-product self-modification mechanism developed by SRWM in Section 5.7.4.2 and place it inside an explicit reinforcement learning formulation, framing meta-RL itself as an optimal transport problem, and extending the single-step outer-product write into a multi-step recursive update.

**Meta-RL as a transport problem.** For every task, there exists some optimal policy. Meta-RL can therefore be viewed as the problem of constructing a mapping from tasks to their corresponding optimal policies,  $\tau \in \mathbb{T} \rightarrow \pi_\tau^*$ . Chalvidal et al. formalize this as a transport problem: let  $\mu_{\mathbb{T}}$  be a distribution over tasks and  $\mu_{\pi^*}$  a distribution over optimal policies. A transport plan is a coupling  $\gamma \in \Gamma(\mu_{\mathbb{T}}, \mu_{\pi^*})$ , where  $\gamma(\tau, \pi)$  specifies how much probability mass of task  $\tau$  is assigned to policy  $\pi$ . The goal is to find the transport plan that maximizes expected reward under this coupling,

$$\max_{\gamma \in \Gamma(\mu_{\mathbb{T}}, \mu_{\pi^*})} \mathbb{E}_{(\tau, \pi) \sim \gamma} [R(\tau, \pi)].$$

Meta-RL, in this framing, is the problem of transporting probability mass from a distribution over tasks to a distribution over high-performing policies. Rather than learning a mechanism that maps a task directly to a fixed policy, as the task inference methods of Section 5.9 do, the goal here is to learn a mechanism that maps tasks to an evolving *policy update process*,

$$\max_{\theta} \mathbb{E}_{\tau \sim \mu_{\mathbb{T}}} \left[ \mathbb{E}_{\pi \sim \mu_{\pi}^{\theta, \tau, t}} R(\tau, \pi) \right],$$

where  $\theta$  denotes the meta-parameters of the network responsible for updating the policy weights, and  $\mu_{\pi}^{\theta, \tau, t}$  is the distribution over policies induced by that update process at time  $t$ , for task  $\tau$ .

**Metrics of a good transport plan.** This framing suggests three criteria for evaluating a meta-RL method, independent of any particular architecture. Efficiency measures optimization speed: how quickly the distribution  $\mu_{\pi}^{\theta, \tau, t}$  is transported toward a distribution of high-performing policies as  $t$  increases. Capacity measures model expressivity: how well the system can represent and exploit the structure of a task, how much it can understand and remember about the task from experience. Generality measures robustness and transfer: how well the learned update process performs on tasks different from, or entirely unseen during, meta-training. As in many learning systems, these three criteria trade off against one another:

high capacity risks overfitting and therefore reduced generality, high efficiency can come at the expense of exploration, and high generality can come at the expense of task-specific specialization. A well-designed meta-RL method must balance all three rather than maximizing any single one in isolation, a lens the authors apply not only to their own method but to meta-RL methods generally, including the recurrent and memory-based methods surveyed earlier in Section 5.7.

**Core concept.** MetODS instantiates this transport process directly in weight space, following the same adaptation-in-weight-space principle introduced at the start of Section 5.7.4. The policy depends on a set of weights that evolve over time,

$$\pi(a \mid s, W_t), \quad W_{t+1} = W_t + \Delta(W_t), \quad \Delta(W_t) = F_\theta(W_t),$$

where the update function  $F_\theta$  is itself learned during meta-training. This is presented explicitly as an extension of Hebbian learning: where the plasticity mechanisms of Section 5.7.4.1 update weights based on local pre- and post-synaptic correlation alone, MetODS extends this with reward-driven learning, multi-step recursion, and a fully learnable update function, rather than a fixed-form Hebbian trace.

**Read and write operations.** The dynamic weight matrix  $W_t$  is intended to compress the agent’s experience of the current task  $\tau$  over time, so that  $\pi(a \mid s, W_t) \sim \mu_\pi^{\theta, \tau, t}$  for every  $t \leq T$ . Two operations govern how  $W_t$  is used and updated. The read operation produces an updated activation from the current weights and an experience embedding  $v$ ,

$$\phi(W, v) = \sigma(Wv),$$

while the write operation stores the self-correlation of the current embedding, scaled by a learnable plasticity strength,

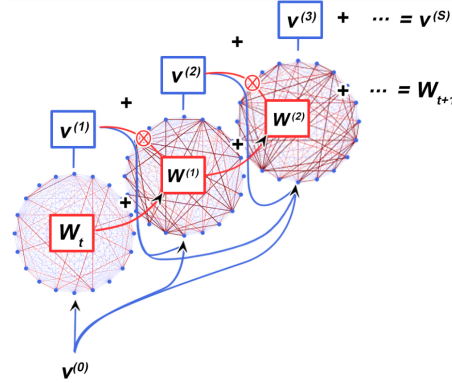
$$\psi(v) = \alpha \odot (v \otimes v),$$

where  $\alpha$  determines where and how strongly each synapse is updated, directly analogous to the plasticity coefficient  $\alpha_{i,j}$  of Section 5.7.4.1, but here applied to the self-correlation of a single embedding vector rather than to the correlation between two distinct pre- and post-synaptic activations.

**Multi-step recursion.** What distinguishes MetODS from the single-step updates of Sections 5.7.4.1 and 5.7.4.2 is that the read and write operations are not applied once per timestep, but iterated over  $S$  internal recursion steps within a single timestep, with each step blending information from every previous step rather than only the immediately preceding one,

$$v^{(s)} = \sum_{l=0}^{s-1} \kappa_s^{(l)} v^{(l)} + \kappa_s^{(s)} \phi(W^{(s-1)}, v^{(s-1)}), \quad W^{(s)} = \sum_{l=0}^s \beta_s^{(l)} W^{(l)} + \beta_s^{(s)} \psi(v^{(s-1)}),$$

for  $s \in [1, S]$ , where  $\kappa$  and  $\beta$  are learnable coefficients controlling how much weight each previous recursion step contributes to the current one. Figure A.24 visualizes this recursive structure: starting from the initial state  $v^{(0)}$  and the weight matrix  $W_t$  carried over from the previous timestep, each recursion step reads from the current weight matrix to produce a new activation  $v^{(s)}$ , writes the self-correlation of that activation back into an updated weight matrix, shown as the increasingly dense graphs  $W^{(1)}, W^{(2)}, \dots$ , and accumulates across steps to produce the final activation  $v^{(S)}$  and updated weight matrix  $W_{t+1}$  used at the next timestep.



**Figure A.24.** – Multi-step recursive synaptic update in MetODS. Starting from  $v^{(0)}$  and the previous weight matrix  $W_t$ , each recursion step  $s$  reads from the current weights to produce  $v^{(s)}$  and writes a new self-correlation term into the weight matrix, shown as the graph structures  $W^{(1)}, W^{(2)}, \dots$ , growing denser with each recursion step. The accumulated activations and weights across all  $S$  steps produce  $v^{(S)}$  and  $W_{t+1}$ . Adapted from Chalvidal et al. [98].

**Entire pipeline.** At every timestep, an embedding network first encodes the current transition,  $v^{(0)} = f_\phi(s_t, a_{t-1}, r_{t-1})$ . This embedding is then passed through the



recursive plasticity module described above, producing a sequence of intermediate activations and weight matrices,  $(v^{(0)}, W_{t-1}) \rightarrow (v^{(1)}, W^{(1)}) \rightarrow \dots \rightarrow (v^{(S)}, W^{(S)})$ . The final activation is read out by a policy readout module,  $(a_t, \hat{V}_t) = g_\psi(v^{(S)})$ , representing an actor and a critic that can, in principle, be instantiated by any underlying actor-critic algorithm, such as TD3 or SAC. All components,  $\theta = \{f_\phi, g_\psi, \alpha, \kappa, \beta\}$ , are trained jointly by gradient descent, with the roles of each summarized in Table A.12.

**Table A.12.** – Learned components of MetODS and their role during adaptation.

| Parameter        | What it learns       | Role during adaptation                                                           |
|------------------|----------------------|----------------------------------------------------------------------------------|
| $f_\phi$         | State representation | Encodes observations, previous action, and reward into $v^{(0)}$ .               |
| $g_\psi$         | Policy and critic    | Produces the action and value estimate from the final recursion step $v^{(S)}$ . |
| $\alpha$         | Synaptic plasticity  | Determines where, and by how much, each synapse in $W$ changes.                  |
| $\kappa$         | Activation recursion | Determines how activations evolve across recursion steps.                        |
| $\beta$          | Weight recursion     | Determines how old and newly written synaptic information are combined.          |
| $W_0$ (optional) | Initial memory       | Sets the initial synaptic state before any adaptation occurs.                    |

**Algorithm.** Algorithm 59 summarizes meta-training. For every sampled task, a trajectory is collected using the current policy, and at every timestep the recursive read-write procedure above is applied for  $S$  steps before the policy readout is computed and the weight matrix is carried forward to the next timestep. All meta-parameters, including  $W_0$ , are updated by maximizing the same transport-problem objective introduced at the start of this subsection.

---

**Algorithm 59** MetODS Synaptic Learning

---

**Require:**  $\theta = [f_\phi, g_\psi, \alpha, \kappa, \beta]$  and  $W_0$

**repeat**

    Sample a task  $\mathcal{T} \sim \mu_{\mathbb{T}}$

    Collect trajectory using  $\pi_\theta$

**for**  $1 \leq t \leq T$  **do**

$v^{(0)} = f_\phi(s_t, a_{t-1}, r_{t-1})$

$W^{(0)} = W_{t-1}$

**for**  $1 \leq s \leq S$  **do**

$v^{(s)} \leftarrow \sum_{l=0}^{s-1} \kappa_s^{(l)} v^{(l)} + \kappa_s^{(s)} \sigma(W^{(s-1)} \cdot v^{(s-1)})$

$W^{(s)} \leftarrow \sum_{l=0}^{s-1} \beta_s^{(l)} W^{(l)} + \beta_s^{(s)} (\alpha \odot v^{(s-1)} \otimes v^{(s-1)})$

**end for**

$a_t, V \leftarrow g_\psi(v^{(S)})$

$W_t \leftarrow W^{(S)}$

**end for**

    Update  $\theta = [f_\phi, g_\psi, \alpha, \kappa, \beta]$  and  $W_0$  using  $\max_\theta \mathbb{E}_{\tau \sim \mu_{\mathbb{T}}} [\mathbb{E}_{\pi \sim \mu_\pi^{\theta, \tau, t}} R(\tau, \pi)]$

**until** convergence

---

**Advantages and limitations.** MetODS shares the core advantages of the outer-product self-modification family: no gradient updates are performed during adaptation itself, adaptation occurs online at every timestep, and the adaptation state lives in the weights  $W_t$  rather than in a separate hidden state, supporting one-shot learning and applying equally to discrete and continuous control. Synaptic memory persists naturally over time without requiring an explicit external memory module, and the mechanism is explicitly motivated by fast synaptic plasticity in biological neural circuits. Against this, the published formulation applies dynamic weights to only a single layer, leaving the extension of plasticity to every layer of the network as future work; the multi-step recursion increases computational cost relative to a standard forward pass, since every timestep now requires  $S$  internal read-write iterations rather than one; maintaining  $W_t$  requires an additional state matrix beyond the network’s ordinary parameters; and performance on highly diverse benchmarks, such as MetaWorld ML10, remains limited relative to methods with narrower task distributions.

**Relation to earlier and later methods.** MetODS and SRWM in Section 5.7.4.2 share the same underlying write mechanism, an outer product of a vector with itself, added into an evolving weight matrix, but differ in two respects. SRWM computes every quantity involved in its update, the key, query, value, and learning rate, from

a single self-referential matrix acting on itself, with no explicit multi-step recursion within a timestep. MetODS instead introduces  $S$  explicit recursion steps per timestep, each blending contributions from every previous step via learnable coefficients  $\kappa$  and  $\beta$ , and frames the entire problem within an explicit reinforcement learning and optimal-transport formulation, rather than the general sequence-modeling setting SRWM is evaluated in. Section 5.7.4.4 turns to a substantially different training paradigm within the same family of weight-space adaptation methods: rather than learning the update rule by backpropagation, as in every method presented so far in Section 5.7.4, it removes backpropagation from the outer loop entirely, using Evolution Strategies to optimize a Hebbian update rule applied to an otherwise randomly initialized network.

#### A.2.3.5. Meta-Learning through Hebbian Plasticity in Random Networks

Every method presented so far in Section 5.7.4 is trained by backpropagation: the outer loop, whether it optimizes a Hebbian rate as in Section 5.7.4.1, a self-referential weight matrix as in Section 5.7.4.2, or a multi-step recursive update as in Section 5.7.4.3, always computes gradients of a task loss with respect to its meta-parameters and updates them accordingly. Najarro and Risi [99] remove this assumption entirely, asking a more extreme version of the question that motivates this whole subsection: can an agent learn during its lifetime without backpropagation, and, further, without any reward signal at all during that lifetime? Their answer departs from every other method in this chapter in what it optimizes: rather than learning the network’s weights, directly or through a plasticity mechanism applied to them, this method learns only the local update rule itself, and lets the weights, initialized entirely at random, organize themselves through that rule alone.

**Conceptual shift.** Table A.13 summarizes the shift in perspective relative to standard reinforcement learning, and relative to every other method in Section 5.7.4. Standard RL, along with the Hebbian and self-referential methods above, learns a set of weights  $w$  that solve a task, producing a network that is static once training ends, with learning stopping at test time and requiring a reward signal throughout training. This method instead learns an update rule  $h$ , producing a network that remains dynamic indefinitely, continuing to adapt its own weights at test time exactly as it did during training, and requiring no reward signal during the lifetime in which that adaptation occurs.

**Table A.13.** – Conceptual shift from standard reinforcement learning to learning a Hebbian update rule.

| Traditional RL (and Sections 5.7.4.1–5.7.4.3) | This method                          |
|-----------------------------------------------|--------------------------------------|
| Learn weights $\mathbf{w}$                    | Learn the update rule $\mathbf{h}$   |
| Static network after training                 | Dynamic network, indefinitely        |
| Learning stops at test time                   | Learning continues at test time      |
| Needs a reward signal throughout              | No reward signal during the lifetime |

**What is learned.** The network’s weights are initialized uniformly at random,  $\mathbf{w} \sim U[-0.1, 0.1]$ , and never directly optimized by any gradient-based procedure. What is optimized instead is a set of Hebbian rule parameters,  $\mathbf{h} \sim U[0, 1]$  at initialization, which govern how every connection updates itself, purely as a function of local pre- and post-synaptic activity, throughout the agent’s lifetime.

**Hebbian learning rule.** Each synapse updates according to a general four-term Hebbian rule,

$$\Delta \mathbf{w}_{ij} = \eta \cdot (A \mathbf{o}_i \mathbf{o}_j + B \mathbf{o}_i + C \mathbf{o}_j + D),$$

where  $\mathbf{o}_i$  and  $\mathbf{o}_j$  are the pre- and post-synaptic activations. Table A.14 interprets each term. The correlation term  $A \mathbf{o}_i \mathbf{o}_j$  is the classical Hebbian principle used throughout this subsection, cells that fire together wire together, matching the correlation term at the core of the plasticity mechanisms in Section 5.7.4.1. The remaining three terms extend this beyond pure correlation: a presynaptic-only term  $B \mathbf{o}_i$ , a postsynaptic-only term  $C \mathbf{o}_j$ , and a constant bias term  $D$  representing baseline drift independent of either neuron’s activity.

**Table A.14.** – Terms of the general Hebbian update rule.

| Term                          | Meaning                                                                           |
|-------------------------------|-----------------------------------------------------------------------------------|
| $A \mathbf{o}_i \mathbf{o}_j$ | Correlation (Hebbian): cells that fire together, wire together.                   |
| $B \mathbf{o}_i$              | Presynaptic-only influence, independent of the postsynaptic neuron’s activity.    |
| $C \mathbf{o}_j$              | Postsynaptic-only influence, independent of the presynaptic neuron’s activity.    |
| $D$                           | Constant bias, representing baseline drift unrelated to either neuron’s activity. |

Crucially, each synapse in the network has its own independent set of coefficients  $\{A, B, C, D, \eta\}$ , rather than a single shared rule applied uniformly across the whole

network. This gives the resulting plasticity mechanism substantial expressive power: since each connection can, in principle, implement a different local learning strategy, the network’s self-organizing behavior is far richer than a single global Hebbian rule would allow. Because every term in the update depends only on locally available activations, no gradient needs to be backpropagated through the network to compute it, giving both local learning and continuous online adaptation as direct consequences of the rule’s structure, rather than as separately engineered properties.

**Algorithm.** The resulting procedure separates cleanly into an inner loop of lifetime learning and an outer loop of evolutionary optimization. In the inner loop, at every timestep the agent observes the current state  $s_t$ , computes the resulting neuron activations  $o_i$ , updates every weight in the network using the Hebbian rule above, and acts, all without ever receiving or using a reward signal during this process. In the outer loop, the Hebbian rule parameters  $h = \{A, B, C, D, \eta\}$ , shared meta-parameters governing every synapse’s individual coefficients, are optimized using Evolution Strategies,

$$h_{t+1} = h_t + \frac{\alpha}{n\sigma} \sum_i F_i \cdot (h_t + \sigma \epsilon_i),$$

where  $F_i$  is a fitness function evaluating the performance achieved after a full lifetime of weight evolution under a perturbed candidate rule, for instance  $F(h) = \mathbb{E}_{\text{episode}} \left[ \sum_{t=1}^T r_t \right]$ ,  $\alpha$  controls how strongly the parameters move at each generation,  $\sigma$  controls the magnitude of the perturbation noise, and  $\epsilon_i \sim \mathcal{N}(0, 1)$  is standard normal noise applied to generate each candidate. This is the point at which reward re-enters the picture: reward is never observed or used during the inner-loop lifetime itself, but it is used entirely outside that lifetime, as the fitness signal that shapes which Hebbian rules survive across generations.

**Relation to the two-loop structure of this section.** This still fits the same inner-loop/outer-loop structure used throughout Section 5.7, but with both loops changed relative to every other method presented so far. The inner loop here performs no reward-driven learning at all, unlike RL<sup>2</sup>’s implicit hidden-state adaptation in Section 5.7.1 or the reward-shaped Hebbian traces of Sections 5.7.4.1 through 5.7.4.3; it performs purely unsupervised, correlation-driven self-organization. The outer loop replaces backpropagation with a population-based, gradient-free search, meaning this method does not strictly satisfy the black-box definition of Section 5.7 as originally stated, which assumes  $f_\theta$  is trained via the standard reinforcement learning objective through backpropagation. It is included in this subsection because it shares every other defining property of the family: no explicit optimization algorithm and

no explicit task-inference model are specified for the inner loop, adaptation is entirely a property of the forward dynamics of the network, here written into weight-space through a learned local rule rather than into a hidden state, and the parameters governing that adaptation, the Hebbian coefficients  $\mathbf{h}$ , are fixed once outer-loop training (in this case, evolution rather than gradient descent) concludes.

**Limitations.** The search space over Hebbian coefficients is large, with millions of independent per-synapse parameters to optimize, and Evolution Strategies is comparatively sample-inefficient relative to gradient-based optimization, making outer-loop training slower than in any backpropagation-based method in this subsection. The method performs no explicit task inference, in common with the plasticity-based methods of Sections 5.7.4.1 through 5.7.4.3 and in contrast with the task inference methods of Section 5.9. Because weights self-organize purely from local activity with no global objective guiding them moment to moment, a poorly evolved rule carries a real risk of instability, with no mechanism analogous to the clipping operation used in Backpropamine in Section 5.7.4.1 to bound the resulting weight changes.

**Relation to earlier and later methods.** This method demonstrates that the outer-product and Hebbian mechanisms surveyed throughout Section 5.7.4 do not strictly require backpropagation to be learned, only a fitness signal evaluated after the fact. Where Section 5.7.4.1 learns a single scalar or network-generated modulation of a shared Hebbian rate, and Sections 5.7.4.2 and 5.7.4.3 learn fully self-referential or recursively structured weight-update mechanisms, this method learns a compact, four-term local rule applied independently at every synapse, trading architectural expressiveness for a rule simple enough to be searched over by evolution rather than requiring differentiability throughout. Section 5.7.4.5 returns to backpropagation-trained methods, but combines Hebbian plasticity, in the form of a persistent associative memory, with several additional modules, task inference, world modeling, and explicit episodic memory, drawn from other sections of this chapter, closing Section 5.7.4 with an architecture that treats plasticity as one component among several rather than as the sole adaptation mechanism.

#### A.2.4. Training Difficulties and Limitations of Black-Box Methods

Section 5.7 has surveyed two ways of realizing the adaptation function  $f_\theta$  with no explicit optimization procedure: an evolving representation read by a fixed policy, whether a recurrent hidden state (Section 5.7.1), an attention-refined context (Section 5.7.2), or explicit external memory (Section 5.7.3), and an evolving weight

matrix modified directly by a learned local rule (Section 5.7.4). Both place the entire burden of adapting on a single end-to-end trained system, with no explicit optimizer or task variable constraining how adaptation happens.

**Advantages.** This structure has real benefits. No gradient computation is needed during adaptation itself, since adaptation is a forward pass rather than an optimization step. Adaptation occurs at every timestep rather than only after a batch of data is collected, as in Section 5.6. The same mechanism that adapts to a new task also handles partial observability naturally, since both are solved by maintaining a representation, hidden state, attention context, or weight matrix, that summarizes relevant unobserved information from history. This suits online decision-making, and, since no exploration strategy is hand-designed anywhere in Sections 5.7.1–5.7.4, these methods can learn exploration strategies more sophisticated than a human might specify explicitly, provided training succeeds.

**Difficulties.** These same properties make training hard. Optimization is difficult: training a recurrent or memory-augmented policy with RL is already hard in the single-task setting, and meta-RL compounds this with long multi-episode trials, delayed rewards, and an adaptation strategy that must emerge implicitly rather than being specified. Recurrent backbones (Section 5.7.1, and parts of Section 5.7.3) suffer vanishing and exploding gradients over long horizons; Transformer backbones (Section 5.7.2) avoid this but are unstable to train without care, and an unstable model can destroy exploration before the agent has gathered enough experience to learn at all, precisely why TrMRL (Section 5.7.2.2) restricts attention causally and bounds its memory window. This instability is one face of a deeper problem: exploration and exploitation form a chicken-and-egg loop, since the agent cannot learn to exploit information until it knows how to explore, and cannot learn to explore effectively until it can use what exploration reveals, with both required to emerge from a single training signal. Weight-space methods (Section 5.7.4) sidestep the unrolled-gradient problem but introduce their own instability: none of their update rules are guaranteed stable, a risk most acute in the evolution-trained rule of Section 5.7.4.4, which has no gradient signal to discourage unstable solutions during search. Memory and compute requirements also grow with interaction length throughout, whether from backpropagating through a multi-episode recurrent state (Section 5.7.1) or from an accumulating external memory buffer (Section 5.7.3).

**Generalization and interpretability.** Two further limitations follow from the same lack of structure. Generalization is bounded by the training distribution: a method such as  $RL^2$  learns a policy, and an adaptation strategy, specialized to the tasks it was meta-trained on, and performance can degrade when test tasks differ from

that distribution in ways never encountered, a concern raised explicitly for MQN (Section 5.7.3.1), MRA (Section 5.7.3.2), and, in a different form, as the entire motivation for EPN’s departure from weighted-average retrieval (Section 5.7.3.4). Interpretability is limited throughout Section 5.7: none of these methods offer a clear account of what information their adaptation state actually holds, how task inference implicitly occurs within it, or why the resulting policy adapts as it does to a given sequence of experience, since the whole process is embedded in a hidden state, attention context, or weight matrix with no explicit variable to inspect. None of the weight-space methods of Section 5.7.4 perform explicit task inference either: their update rules adapt to whatever local activity or correlation they observe, with no explicit representation of task identity. Only BIMRL (Section 5.7.4.5) avoids relying on plasticity as the sole adaptation mechanism, at the cost of the architectural and training complexity of coordinating several modules under one combined loss.

**Section 5.7 in relation to the rest of this chapter.** Table A.15 makes this trade-off concrete by contrasting MAML (Section 5.6.1.1) with  $\text{RL}^2$  (Section 5.7.1.1).

**Table A.15.** – Policy parameter gradient adaptation (MAML) compared to black-box adaptation ( $\text{RL}^2$ ).

| Aspect                          | MAML                                           | $\text{RL}^2$                                                       |
|---------------------------------|------------------------------------------------|---------------------------------------------------------------------|
| Adaptation mechanism            | Gradient descent                               | Hidden state updates                                                |
| Explicit test-time optimization | Yes                                            | No                                                                  |
| Memory                          | Optional                                       | Essential                                                           |
| Adaptation frequency            | After collecting a batch of data               | Every timestep                                                      |
| Inner loop                      | Explicit optimizer                             | RNN or Transformer                                                  |
| Generalization                  | Often better outside the training distribution | Strong within the training distribution, weaker out-of-distribution |

Black-box methods place minimal structural assumptions on how adaptation should occur, which is what lets them discover adaptation strategies a hand-designed algorithm might not, and what gives them the every-timestep adaptation frequency and reliance on memory shown above. The same lack of structure is what makes them hard to train and hard to interpret. Section 5.6 imposes the structure black-box methods lack, a fixed, explicit optimization procedure with only its initialization, rate, or curvature meta-learned, trading flexibility for interpretability and, per the table, often better out-of-distribution generalization. Section 5.9 imposes a different



structure, an explicit latent task variable and belief update, shown in Section 5.7.1.1 to directly address the entanglement problem that motivates much of Section 5.7’s own internal development, from external memory (Section 5.7.3) to weight-space adaptation (Section 5.7.4). No family dominates the others on every axis; the choice is where to place the trade-off between adaptation flexibility, training stability, interpretability, and sample efficiency, one this thesis returns to empirically in later chapters.

## A.3. Additional Task-Inference Methods

Chapter 7 covers the Approximate Information State framework and two variational Bayesian belief methods, VariBAD and PEARL, in full. The methods below cover the remaining ways the literature approximates the task belief  $b_t(\mathbf{w})$ : a further variational method that fixes a specific limitation of PEARL, methods that decouple exploration from exploitation, methods that replace variational inference with a discriminative or self-supervised objective, methods that regress directly to task-specific parameters or values instead of a probabilistic belief, a method that grounds its representation in the theory of exchangeable stochastic processes, a methodology for validating what a learned belief representation actually encodes, and a closing comparison across every task-inference method covered in this thesis.

### A.3.0.1. ELUE: Off-Policy Meta-Reinforcement Learning with Belief-Based Task Inference

Imagawa et al. [100] target a specific gap left open by PEARL in Section 5.9.2.2: is PEARL’s training procedure genuinely off-policy, and does its policy actually condition on the uncertainty its own encoder produces? The authors answer both questions negatively, and propose an architecture, ELUE (Embedding Learning and Uncertainty Evaluation), built directly to fix each.

**PEARL is not fully off-policy.** Although PEARL trains its actor and critic off-policy from a replay buffer, its encoder is not trained independently of the policy. As shown in Section 5.9.2.2, the encoder parameters  $\phi$  are updated using the critic loss  $\mathcal{L}_{\text{critic}}$  together with the KL term, so the encoder’s training signal depends on the current critic. As the policy changes over training, the critic changes with it, and since the encoder’s gradient flows through the critic, the encoder’s training target changes as well. This means data stored earlier in the replay buffer, though still perfectly valid for retraining the critic itself, becomes progressively less useful for training the encoder, since it was collected and evaluated under a critic that no

longer matches the current one. ELUE’s central design decision is to remove this dependency entirely: embed learning should not touch the actor or critic at all, and should instead learn a task representation directly from raw transitions in the replay buffer, so that old data remains just as useful for encoder training regardless of how much the policy has since changed. The authors term the result fully off-policy embedding learning, achieved through a workflow of replay buffer  $\rightarrow$  Conditional VAE  $\rightarrow$  encoder, entirely decoupled from the actor and critic training loop.

**Meta-RL as a POMDP.** As in the section overview, the task is modeled as a hidden variable governing transition and reward, here written  $\phi$  to match the source notation, with transition  $P(s' | s, a, \phi)$  and reward  $R(s, a, \phi)$ , and the agent observing only the history  $h_t = (s_0, a_0, r_0, \dots, s_t)$ . The belief over this hidden variable is  $b_t(\phi) = P(\phi | h_t)$ . To keep this belief tractable, ELUE replaces exact Bayesian updating with a learned latent variable  $z$ , inferred by an encoder  $q(z | c)$  from a set of transition tuples  $c = (s, a, r, s')$ , following the same substitution VariBAD and PEARL both make in Sections 5.9.2.1 and 5.9.2.2.

**Permutation-invariant encoder.** ELUE’s encoder is permutation-invariant by a different construction than PEARL’s product-of-Gaussians in Section 5.9.2.2. Given a set of  $k$  context transitions, a shared network  $f$  maps each transition independently to a vector, these vectors are summed, and a second network  $g$  maps the sum to the parameters of a Gaussian distribution over  $z$ ,

$$q(z | c_1, \dots, c_k) = g \left( \sum_i f(c_i) \right),$$

with  $g$  producing the mean and variance of a Gaussian and being, by construction, invariant to the order in which the  $c_i$  are summed. This is the Deep Sets construction for permutation-invariant functions, and achieves the same goal as PEARL’s per-transition Gaussian factorization, invariance to context ordering, through summation before decoding into distributional parameters, rather than through multiplying per-transition Gaussian factors together.

**Decoder and training objective.** A decoder predicts the reward and next state directly from the current state, action, and sampled latent,  $p(r, s' | s, a, z)$ , and the encoder-decoder pair is trained with a standard ELBO,

$$\mathcal{L} = -\mathbb{E}_q \left[ \log p(r, s' | s, a, z) \right] + D_{\text{KL}}(q(z) \| p(z)).$$

Because this loss involves only the encoder, the decoder, and transitions sampled from the replay buffer, with no actor or critic term anywhere in it, its gradient is entirely independent of the current policy, which is precisely what makes ELUE’s embedding learning fully off-policy in the sense argued above, unlike PEARL’s encoder loss in Section 5.9.2.2, which includes the critic loss directly.

**Conditioning the policy on the belief, not a sample.** ELUE’s second departure from PEARL concerns what the policy is actually conditioned on. The authors argue that PEARL does not truly condition its policy on the belief at all: PEARL samples a single  $z \sim q(z \mid c)$  and conditions the policy and critic on that one sample,  $\pi_\theta(a \mid s, z)$ ,  $Q_\theta(s, a, z)$ , which discards the shape of the posterior entirely, its mean and its variance, keeping only one draw from it. A policy conditioned this way cannot represent how confident or uncertain the agent currently is about the task, since a single sample looks the same whether it came from a sharply peaked posterior or a broad, uncertain one. Following VariBAD’s approach in Section 5.9.2.1, ELUE instead conditions the policy and value functions on the belief distribution itself, represented directly by its parameters,

$$\pi(a \mid s, b), \quad Q(s, a, b),$$

where  $b = (\mu, \sigma)$  is the belief produced by the encoder, giving the policy explicit access to its own uncertainty over the task at every decision.

**Information bottleneck across every network.** Where PEARL applies a single KL penalty to the latent context variable  $z$  and VariBAD applies a single KL penalty inside its ELBO, ELUE applies an information bottleneck penalty separately to an intermediate latent feature inside each of its three trained networks: the actor, the critic, and the value function each has its own internal latent  $\mathbf{w}$ , penalized by an information bottleneck objective,

$$J = \mathbb{E} \left[ \sum_t \gamma^t (r_t - \beta I(\mathbf{w}_t; b_t \mid s_t)) \right],$$

where  $I(\mathbf{w}_t; b_t \mid s_t)$  is the mutual information between the network’s internal latent and the belief, conditioned on the state. Computing this mutual information exactly requires the marginal  $p(\mathbf{w} \mid s) = \int_b p(\mathbf{w} \mid s, b)p(b) db$ , which is intractable, so ELUE uses the standard variational approximation to the information bottleneck, reducing the penalized objective to  $r - \beta \log p(\mathbf{w} \mid s, b)$ . Applying this penalty consistently to the actor’s, critic’s, and value function’s own internal representations, rather than

only to the shared task variable, is intended to encourage each network to retain only the information from the belief that it specifically needs, rather than propagating the full belief representation unfiltered through every downstream computation.

**SAC training.** With the belief-conditioned representations in place, training follows soft actor-critic: the actor loss is  $\mathcal{L}_{\text{actor}} = \mathbb{E}[\beta \log \pi - Q]$ , the critic target is  $\hat{Q} = r + \gamma V(s', b')$ , and the value target is  $\hat{V} = Q - \beta \log \pi$ , each computed using the belief-conditioned  $Q(s, a, b)$ ,  $V(s, b)$ , and  $\pi(a \mid s, b)$  introduced above, rather than the sample-conditioned versions used in PEARL.

**Meta-training.** Algorithm 60 summarizes meta-training. For each sampled batch of tasks, a belief is initialized, and data is collected using the belief-conditioned policy while the belief is updated online; this data is stored in a per-task replay buffer. Training then samples a task batch and, for each, computes the embedding loss described above and updates the encoder, then computes and minimizes the actor and critic losses.

---

**Algorithm 60** ELUE Meta-Training

---

**Require:** Meta-training task set  $\mathcal{T}$

**while** not converged **do**

    Sample a batch of tasks  $\mathcal{T}' \sim \mathcal{T}$

    Initialize belief for each sampled task

**for** each task  $\tau_i \in \mathcal{T}'$  **do**

**for** each data collection step **do**

            Collect transition using policy  $\pi(\cdot \mid s^i, b^i)$

            Update belief  $b^i$

            Store transition in replay buffer  $D^i$

**end for**

**end for**

**for** each training step **do**

        Sample a task batch  $\mathcal{T}'$  uniformly from  $\mathcal{T}$

        Compute embedding loss  $L_{\text{embed}}$  and update encoder parameters by minimizing it

        Compute actor loss  $L_{\text{actor}}$  and critic loss  $L_{\text{critic}}$

        Update actor parameters by minimizing  $L_{\text{actor}}$

        Update critic parameters by minimizing  $L_{\text{critic}}$

**end for**

**end while**

---

**Meta-testing and Belief Gradient (BelGrad).** At meta-test time, ELUE follow the Algorithm 61: the meta-trained model collects transitions from the new task, uses them to update its belief, acts using the belief-conditioned policy, and periodically fine-tunes the actor and critic, repeating this cycle for a fixed number of semi-amortized update rounds. Notably, the encoder and decoder parameters themselves are not updated at test time. The authors motivate this by two failure modes that would otherwise occur: continuing to train the encoder on the small amount of data available from a single test task risks catastrophic forgetting of what it learned across the full meta-training task distribution, and risks the encoder collapsing its posterior onto a single, narrow mode corresponding only to the current task, losing the general-purpose belief-estimation capability it was meta-trained to have.

---

**Algorithm 61** ELUE Meta-Testing

---

**Require:** A meta-test task

```

while not done do
  for step in data collection steps do
    Gather data from the meta-test task using policy  $\pi(a \mid s, b)$ 
    Update belief  $b$  and replay buffer  $\mathcal{D}$ 
  end for
  for step in training steps do
    Calculate  $\mathcal{L}_{\text{actor}}$  and  $\mathcal{L}_{\text{critic}}$ 
    Update the network parameters by minimizing the modified  $\mathcal{L}_{\text{actor}}$  and
     $\mathcal{L}_{\text{critic}}$ 
  end for
end while

```

---

Freezing the encoder does not mean the belief itself is frozen, however. Rather than treating the belief  $b = (\mu, \sigma)$  produced by a single forward pass through the encoder as fixed, ELUE introduces Belief Gradient (BelGrad): the belief parameters are copied out and optimized jointly with the policy via gradient descent, following a Semi-Amortized Variational Autoencoder approach, in which an initial amortized estimate from the encoder is refined by a small number of additional gradient steps rather than being accepted as final. This gives a workflow of encoder  $\rightarrow$  belief  $\rightarrow$  gradient descent  $\rightarrow$  better belief  $\rightarrow$  policy, allowing the belief itself to keep improving at test time using the same reinforcement learning signal used to fine-tune the policy, without requiring the encoder or decoder to be touched at all.

**Relation to earlier methods.** ELUE combines a specific criticism of each of the two methods preceding it in this subsection into a single architecture: from PEARL, it inherits the goal of off-policy training but corrects what the authors argue is

an incomplete decoupling of task inference from control, by removing the actor and critic from the embedding-learning loss entirely, and replacing PEARL’s product-of-Gaussians encoder with a Deep Sets-style alternative that is permutation-invariant in a structurally different way. From VariBAD, it inherits belief-based policy conditioning, feeding the full distribution rather than a single sample, but drops the explicit reconstruction-based generative model. It also introduces a mechanism with no counterpart in either prior method: Belief Gradient, a test-time refinement of the belief itself through direct gradient optimization, going beyond the purely feed-forward, gradient-free adaptation used by VariBAD, PEARL, and every black-box method surveyed in Section 5.7.

### A.3.1. Decoupled Exploration for Task Inference

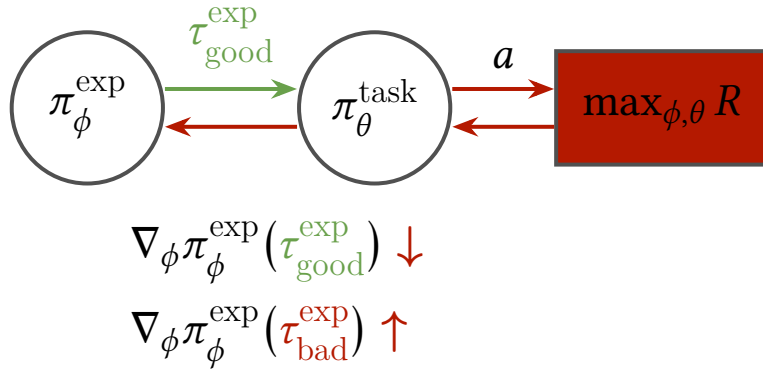
Every method surveyed in Section 5.9.2 trains its context or belief encoder end to end with the policy that uses it: the same gradient signal that shapes the policy also shapes what the encoder learns to represent, and the trajectories the encoder is trained on are the ones the current policy happens to produce. This subsection surveys methods that argue this coupling is itself a source of difficulty, giving rise to a chicken-and-egg optimization problem, and propose resolving it by training a dedicated exploration policy on a separate, information-theoretic objective, decoupled from the policy that ultimately exploits the resulting task representation.

#### A.3.1.1. DREAM: Explore then Execute

Liu et al. [101] give this idea its clearest formulation and its only formal treatment among the methods surveyed in this section, with both a consistency argument and a sample-complexity result supporting the decoupled design.

**The chicken-and-egg problem.** In an end-to-end architecture such as  $RL^2$  or the belief-based methods of Section 5.9.2, the execution policy and the exploration behavior that precedes it are entangled through a single training signal. Figure A.25 illustrates the resulting failure mode. The exploration policy  $\pi_\phi^{\text{exp}}$  produces a trajectory that is fed to the task policy  $\pi_\theta^{\text{task}}$ , which acts and receives reward; that reward is then the only signal used to update both policies jointly, via  $\max_{\phi, \theta} R$ . Suppose  $\pi_\phi^{\text{exp}}$  happens to produce a genuinely informative trajectory,  $\tau_{\text{good}}^{\text{exp}}$ , but  $\pi_\theta^{\text{task}}$  is still suboptimal and fails to act well even given this good information. The resulting reward is low, and the gradient with respect to  $\phi$  decreases the probability of producing  $\tau_{\text{good}}^{\text{exp}}$  again,  $\nabla_\phi \pi_\phi^{\text{exp}}(\tau_{\text{good}}^{\text{exp}}) \downarrow$ , even though the trajectory itself was informative and the failure was entirely the execution policy’s fault. Symmetrically, a bad,

uninformative trajectory that happens to be followed by a lucky high reward can have its probability increased,  $\nabla_{\phi} \pi_{\phi}^{\text{exp}}(\tau_{\text{bad}}^{\text{exp}}) \uparrow$ . The execution policy depends on the exploration policy to supply informative trajectories, while the exploration policy depends on gradient signal routed back through the execution policy to know which trajectories were valuable, and each can corrupt the learning signal for the other. This produces three concrete failure modes: local optima, sample inefficiency, and, because the entire learning signal collapses when reward is rarely observed, failure in sparse-reward settings. The underlying dynamic is self-reinforcing: bad exploration leads to bad execution, which produces a bad gradient signal for the exploration policy, which keeps exploration bad. Formally, when an exploration trajectory carries no information about the task,  $I(\tau^{\text{exp}}; \mathcal{T}) = 0$ , there is no useful signal available to correct this cycle at all.



**Figure A.25.** – The chicken-and-egg problem in end-to-end exploration-execution training. A good exploration trajectory  $\tau_{\text{good}}^{\text{exp}}$  can be penalized if the execution policy  $\pi_{\theta}^{\text{task}}$  fails to exploit it, while a bad trajectory  $\tau_{\text{bad}}^{\text{exp}}$  can be reinforced if it is followed by a lucky reward, since both  $\pi_{\phi}^{\text{exp}}$  and  $\pi_{\theta}^{\text{task}}$  are updated from the same reward signal  $\max_{\phi, \theta} R$ . Adapted from Liu et al. [101].

**Problem setup.** A trial consists of one exploration episode followed by  $N$  execution episodes, with the overall objective

$$J(\pi^{\text{exp}}, \pi^{\text{exe}}) = \mathbb{E}_{\mathcal{T} \sim p(\mathcal{T}), \tau^{\text{exp}} \sim \pi^{\text{exp}}} \left[ V^{\pi^{\text{exe}}}(\tau^{\text{exp}}; \mathcal{T}) \right],$$

the expected value achieved by the execution policy, given the trajectory produced by the exploration policy, over the task distribution. RL<sup>2</sup>-style approaches optimize this objective end to end and on-policy for both exploration and execution, which is precisely what creates the coupling described above.

**End-to-end versus decoupled approaches.** Table A.16 summarizes the trade-off DREAM is designed to resolve. End-to-end approaches can, in principle, represent the optimal joint policy, since nothing constrains the exploration and execution policies to be trained independently, but they suffer the chicken-and-egg problem above, leading to local optima and poor sample efficiency. Decoupled approaches avoid this problem by construction, but risk failing to learn the optimal policy even given unlimited meta-training data, because an exploration objective defined independently of the execution policy’s actual needs may explore task-irrelevant information, or may simply be unable to explore everything relevant within a single, finite-length exploration episode.

**Table A.16.** – Trade-off between end-to-end and decoupled exploration-execution training.

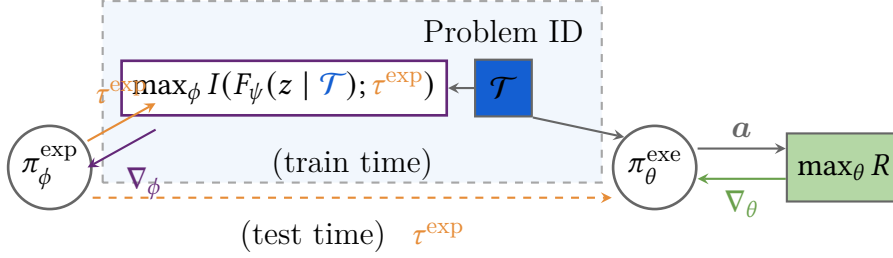
| Approach   | Pros                                                 | Cons                                                                                                |
|------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| End-to-end | Can represent the optimal joint policy (consistency) | Chicken-and-egg problem leads to local optima and sample inefficiency                               |
| Decoupled  | Avoids the chicken-and-egg problem                   | Does not necessarily learn the optimal policy even with infinite meta-training data (inconsistency) |

DREAM’s goal is to construct a decoupled exploration objective specifically designed to recover only task-relevant information, avoiding the inconsistency that naively decoupled approaches would otherwise suffer.

**Architecture.** Figure A.26 shows the resulting architecture, which trains two components on two entirely separate objectives, connected only through the latent variable  $z$  and, at test time, through the trajectory  $\tau^{\text{exp}}$  itself, with no gradient flowing between them in either direction.



(b) **DREAM: Decoupled Reward-free ExplorAction and execution in Meta-RL**



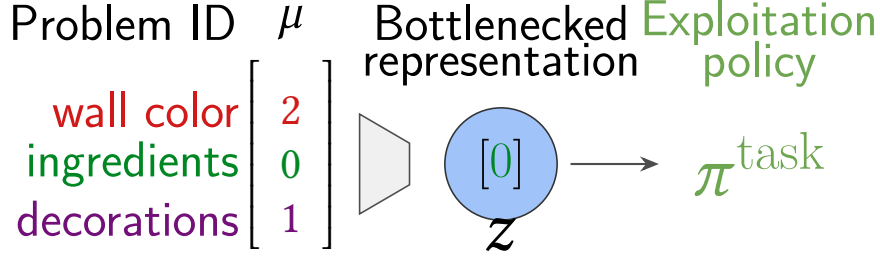
**Figure A.26.** – DREAM’s decoupled architecture. At train time, the problem ID module maximizes  $I(F_\psi(z | \mathcal{T}); \tau^{\text{exp}})$ , updating the exploration policy  $\pi_\phi^{\text{exp}}$  via  $\nabla_\phi$  without any gradient from the execution side. The execution policy  $\pi_\theta^{\text{exe}}$  is updated via  $\nabla_\theta$  to maximize return  $R$ , conditioned on  $z$ . At test time, only the exploration trajectory  $\tau^{\text{exp}}$ , not any gradient, passes from the exploration side to the execution side. Adapted from Liu et al. [101].

**Learning exploitation: encoder and execution policy.** The first component, corresponding to the “egg” half of the problem, learns an encoder  $F_\psi(z | \mu)$ , which maps the true problem identity  $\mu$ , not a trajectory, directly to a compressed latent  $z$ , together with an exploitation policy  $\pi_\theta^{\text{Task}}(a | s, z)$  conditioned on that latent. Figure A.27 illustrates this compression with a concrete example: the problem identity  $\mu$  might specify several attributes of the task, such as wall color, ingredients, and decorations, each with its own value, and the encoder is trained to compress this into a much lower-dimensional, or even scalar, bottlenecked representation  $z$ , retaining only what the exploitation policy actually needs. This pair is trained by maximizing

$$\max_{\psi, \theta} \underbrace{\mathbb{E}_{\mu \sim p(\mu), z \sim F_\psi(z | \mu)} \left[ V^{\pi_\theta^{\text{Task}}}(z; \mu) \right]}_{\text{exploration reward}} - \underbrace{\lambda I(z; \mu)}_{\text{information bottleneck}},$$

where  $V^{\pi_\theta^{\text{Task}}}$  is the expected return achieved by the exploitation policy given  $z$ , and the information bottleneck term penalizes  $z$  for retaining more information about  $\mu$  than is needed, using the same KL-based bound seen in every variational method surveyed in Section 5.9.2,  $I(z; \mu) \leq \mathbb{E}_\mu [D_{\text{KL}}(F_\psi(z | \mu) \| r(z))]$  against a reference distribution  $r(z)$ . Crucially, this objective is trained using the true problem identity  $\mu$ , available at meta-training time, rather than from an exploration trajectory, so it

can be optimized entirely independently of how well the exploration policy currently performs.



**Figure A.27.** – The information bottleneck compresses the full problem identity  $\mu$ , here specifying wall color, ingredients, and decorations, into a much smaller bottlenecked representation  $z$ , which the exploitation policy  $\pi^{\text{task}}$  is conditioned on. Adapted from Liu et al. [101].

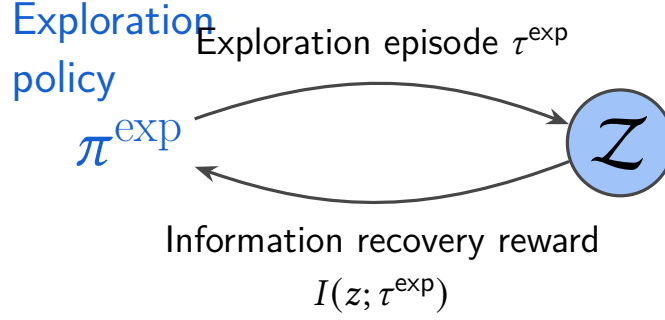
**Learning exploration: decoder and exploration policy.** The second component, the “chicken” half, learns a decoder  $q_\omega(z | \tau^{\text{exp}})$ , predicting the task latent from an exploration trajectory, together with an exploration policy  $\pi_\phi^{\text{exp}}(a_t | s_t, \tau_{:t}^{\text{exp}})$ . As Figure A.28 shows, this policy is rewarded purely for how much its resulting trajectory reveals about  $z$ , an information recovery reward  $I(z; \tau^{\text{exp}})$ , with no dependence on task reward at all. The training goal is to maximize this mutual information,

$$I(z; \tau^{\text{exp}}) = H(z) - H(z | \tau^{\text{exp}}) \geq H(z) + \mathbb{E}_{\mu, z \sim F_\psi, \tau^{\text{exp}} \sim \pi_\phi^{\text{exp}}} [\log q_\omega(z | \tau^{\text{exp}})],$$

a standard variational lower bound obtained by replacing the intractable true posterior  $p(z | \tau^{\text{exp}})$  with the learned decoder  $q_\omega$ . This bound decomposes elegantly across timesteps,

$$\log q_\omega(z | \tau^{\text{exp}}) = \log q_\omega(z | \tau_0^{\text{exp}}) + \underbrace{\sum_{t=0}^{T-1} \log q_\omega(z | \tau_{:t+1}^{\text{exp}}) - \log q_\omega(z | \tau_{:t}^{\text{exp}})}_{\text{per-timestep reward (information gain)}},$$

giving a dense, per-timestep exploration reward with a direct interpretation: how much more does the transition  $(s_t, a_t, s_{t+1})$  tell the decoder about  $z$  than it already knew from the trajectory so far. This exploration objective has a decisive practical advantage over the end-to-end signal in Figure A.25: it is entirely independent of how well the exploitation policy currently performs, and it is dense at every timestep rather than sparse and delayed until an entire trial concludes.



**Figure A.28.** – The exploration policy  $\pi^{\text{exp}}$  produces an exploration episode  $\tau^{\text{exp}}$ , and is rewarded by the information recovery reward  $I(z; \tau^{\text{exp}})$ , measuring how well  $z$  can be recovered from the resulting trajectory, entirely independent of task reward. Adapted from Liu et al. [101].

**Meta-testing.** At test time, the two policies are composed sequentially with no further training,

$$\underbrace{\text{Roll out } \pi_{\phi}^{\text{exp}}}_{\tau^{\text{exp}} \sim \pi_{\phi}^{\text{exp}}} \xrightarrow{\tau^{\text{exp}}} \underbrace{\text{Apply encoder}}_{z \sim q_{\omega}(z|\tau^{\text{exp}})} \xrightarrow{z} \underbrace{\text{Roll out } \pi_{\theta}^{\text{task}}}_{\pi_{\theta}^{\text{task}}(a|s,z)},$$

as also shown by the dashed test-time path in Figure A.26: the exploration policy produces a trajectory, the decoder  $q_{\omega}$ , not the training-time encoder  $F_{\psi}$ , infers  $z$  from that trajectory since the true problem identity  $\mu$  is unavailable at test time, and the exploitation policy acts using this inferred latent.

**Consistency.** Despite the general concern that decoupled approaches sacrifice consistency, the authors argue informally that DREAM remains consistent: given enough meta-training data, the encoder-decoder pair can learn to represent  $z$  as exactly the task-relevant information, and provided the exploration episode is long enough, the exploration policy can learn to recover this same  $z$  during exploration. The specific construction of the exploration objective, maximizing information about the same  $z$  the exploitation policy actually needs, rather than an unrelated proxy objective, is what allows DREAM to sidestep the general inconsistency risk of naively decoupled approaches noted in Table A.16.

**Sample complexity.** The benefit of decoupling can be made concrete in a toy tabular setting. Consider a family of tabular MDPs in which knowing the task identity  $\mu$  is both necessary and sufficient for optimal return, so that  $z = \mu$ , and for each  $\mu$  there exists a unique exploration trajectory  $\tau_\mu^{\text{exp}}$  that fully reveals it,  $p(\mu \mid \tau_\mu^{\text{exp}}) = 1$ . An end-to-end tabular Q-learning approach updates the value of obtaining this trajectory using the expected exploitation return that follows it,

$$Q^{\text{exp}}(\tau_\mu^{\text{exp}}) \leftarrow V^{\text{Task}}(\tau_\mu^{\text{exp}}),$$

which requires observing  $\tau_\mu^{\text{exp}}$  many times before its value is learned accurately, since it is learned indirectly through downstream returns. DREAM’s decoupled update instead sets

$$Q^{\text{exp}}(\tau_\mu^{\text{exp}}) \leftarrow \log q_\omega(\mu \mid \tau_\mu^{\text{exp}}),$$

which only requires observing  $\tau_\mu^{\text{exp}}$  once, since  $q_\omega(\mu \mid \tau_\mu^{\text{exp}}) = p(\mu \mid \tau_\mu^{\text{exp}}) = 1$  can be learned directly as a supervised prediction target, without waiting for it to propagate through downstream value estimates. This distinction underlies the formal sample-complexity comparison: RL<sup>2</sup>-style end-to-end learning requires

$$\Omega(|A|^{2H}|M| \log |A|^H)$$

samples, where  $M$  is the number of tasks,  $H$  is the horizon, and  $A$  is the action space, since it requires encountering a full successful trial, both an informative  $\tau^{\text{exp}}$  and a correct exploitation of it, together, whereas DREAM requires only

$$O(|A|^H \log |A|^H),$$

since its learning signal requires only correct exploration to be observed, not a jointly successful trial. The gap between these two bounds is the formal counterpart of the informal chicken-and-egg argument given above: DREAM learns exploration and exploitation in  $\Omega(|A|^H|M|)$  fewer samples than the end-to-end approach precisely because it removes the requirement that exploration and exploitation succeed simultaneously in order for either to receive a useful learning signal.

**Instruction-based extension.** DREAM’s exploration objective can be made reward-free entirely under Instruction-Based Meta-RL (IMRL), in which the goal is specified directly as an instruction  $i \in \mathcal{I}$ , represented as a one-hot vector, for example “cook pizza” or “go to the red object,” rather than needing to be inferred from reward. The state is augmented with the instruction,  $s \rightarrow (s, i)$ , what remains to be inferred is the environment itself, for instance where the required ingredients are located, and the reward becomes instruction-conditioned,  $R_{\mathcal{T}}(s, i)$ . Because the goal is already specified by the instruction, the exploration episode no longer needs any reward signal at all to be useful, it only needs to reveal the task-relevant environment structure, allowing DREAM’s exploration policy to operate in an entirely reward-free regime both during meta-training and at test time.

**Algorithm.** Algorithm 62 summarizes a Double DQN instantiation of DREAM, combining the instruction-based extension with the exploration and exploitation objectives above. At every trial, a problem  $\mathcal{T}$  is sampled and the exploration policy is rolled out to produce  $\tau^{\text{exp}}$ , stored in a dedicated exploration replay buffer. An instruction  $i$  is sampled,  $z$  is drawn either from the problem-ID encoder  $f_{\psi}(\mathcal{T})$  or from the trajectory decoder  $g_{\omega}(\tau^{\text{exp}})$ , and the execution policy is rolled out conditioned on  $(i, z)$ , with its transitions stored in a separate execution replay buffer. The exploration policy is updated using the information-gain reward computed from the difference in embedding distance across consecutive trajectory prefixes, while the execution policy and both embedding networks are updated using standard DDQN updates on their respective replay buffers, with target networks updated periodically as usual.

---

**Algorithm 62** DREAM (DDQN instantiation)

---

```

Initialize execution replay buffer  $\mathcal{B}_{\text{exe}} \leftarrow \{\}$  and exploration replay buffer  $\mathcal{B}_{\text{exp}} \leftarrow \{\}$ 
Initialize execution Q-value  $\hat{Q}^{\text{exe}}$  parameters  $\theta, \theta'$ 
Initialize exploration Q-value  $\hat{Q}^{\text{exp}}$  parameters  $\phi, \phi'$ 
Initialize problem ID embedder  $f_\psi$  parameters  $\psi, \psi'$ 
Initialize trajectory embedder  $g_\omega$  parameters  $\omega, \omega'$ 
for trial = 1 to max trials do
    Sample problem  $\mathcal{T} \sim p(\mathcal{T})$ , defining MDP  $\langle \mathcal{S}, \mathcal{A}, R_{\mathcal{T}}, T_{\mathcal{T}} \rangle$ 
    Roll out  $\epsilon$ -greedy exploration policy  $\hat{Q}^{\text{exp}}(s_t, \tau_t^{\text{exp}}, a_t; \phi)$ , producing  $\tau^{\text{exp}} = (s_0, a_0, \dots, s_T)$ 
    Add tuples to  $\mathcal{B}_{\text{exp}}$ 
    Sample instruction  $i \sim p(i)$ 
    Sample  $z$  from either  $\mathcal{N}(f_\psi(\mathcal{T}), \rho^2 I)$  or  $g_\omega(\tau^{\text{exp}})$ 
    Roll out  $\epsilon$ -greedy execution policy  $\hat{Q}^{\text{exe}}(s_t, i, z, a_t; \theta)$  with  $r_t = R_{\mathcal{T}}(s_{t+1}, i)$ 
    Add tuples to  $\mathcal{B}_{\text{exe}}$ 
    Sample batches from  $\mathcal{B}_{\text{exp}}$ ; compute  $r_t^{\text{exp}} \leftarrow \|f_\psi(\mathcal{T}) - g_\omega(\tau_t^{\text{exp}})\|_2^2 - \|f_\psi(\mathcal{T}) - g_\omega(\tau_{t-1}^{\text{exp}})\|_2^2 - c$ 
    Optimize  $\phi$  with a DDQN update using  $(s_t, a_t, r_t^{\text{exp}}, s_{t+1})$ 
    Sample batches from  $\mathcal{B}_{\text{exe}}$ ; optimize  $\theta, \omega$  and  $\theta, \psi$  with DDQN updates
    Optimize  $\psi$  on  $\nabla_\psi \min(\|f_\psi(\mathcal{T})\|_2^2, K)$ 
    Optimize  $\omega$  on  $\nabla_\omega \sum_t \|f_\psi(\mathcal{T}) - g_\omega(\tau_t^{\text{exp}})\|_2^2$ 
    if trial  $\equiv 0 \pmod{\text{target frequency}}$  then
        Update target parameters  $\phi' \leftarrow \phi, \theta' \leftarrow \theta, \psi' \leftarrow \psi, \omega' \leftarrow \omega$ 
    end if
end for

```

---

**Relation to later methods.** DREAM establishes the pattern that both MetaCURE (Section 5.9.3.2) and the exploration component of CCM (Section 5.9.3.3) follow: a dedicated exploration policy trained on an information-theoretic reward, decoupled from the policy that exploits the resulting representation. Both later methods can be read as alternative choices of what mutual information to maximize, and how to bound it: MetaCURE uses an empowerment-style objective comparing task-aware and task-blind prediction error, while CCM uses an InfoNCE-based information-gain bound derived from its own contrastive encoder, rather than DREAM’s variational decoder  $q_\omega$ .

### A.3.1.2. MetaCURE: Meta Reinforcement Learning with Empowerment-Driven Exploration

Zhang et al. [102] sharpen DREAM’s motivation for a specific setting: sparse reward. Most meta-RL methods, including every belief-based method surveyed in Section 5.9.2, implicitly assume that adaptation data is reasonably easy to collect, so that the main difficulty lies in inferring the task from that data and acting on it well. In sparse-reward environments this assumption breaks down: the central challenge is no longer how to adapt given informative experience, but how to collect informative experience in the first place, when extrinsic reward is too rare to guide exploration on its own.

**Two-policy structure.** MetaCURE follows the same architectural pattern as DREAM in Section 5.9.3.1: an exploration policy  $\pi_e$ , responsible for collecting informative trajectories, and an exploitation policy  $\pi$ , responsible for maximizing environment reward given the resulting task representation. The meta-objective is

$$\max_{\pi, \pi_e} \mathbb{E}_{\kappa} \left[ R(\kappa, \pi(c_{\pi_e}^{\kappa})) \right],$$

where  $\kappa$  indexes the task and  $c_{\pi_e}^{\kappa}$  is the experience generated by the exploration policy for that task. Where the two methods differ is in how the exploration policy’s own objective is constructed: DREAM maximizes information about a learned latent  $z$  recovered via a trained decoder, while MetaCURE derives its exploration objective directly from an information-theoretic quantity, empowerment, expressed in terms of the true task index  $k$  rather than a learned intermediate variable.

**Empowerment-driven exploration.** The exploration policy’s objective is to maximize the mutual information between the collected experience and the task identity,

$$J^{\pi_e} = I(C_{0:H}; K) = \mathbb{E}_{p(C_{1:H}, k)} \left[ \log \frac{p(c_{0:H} | k)}{p(c_{0:H})} \right].$$

Expanding the trajectory probability autoregressively,  $p(c_{0:H}) = \prod_t p(a_t | c_{0:t}) p(r_t, s_{t+1} | c_{0:t}, a_t)$ , gives

$$I(C_{0:H}; K) = \mathbb{E} \left[ \sum_{t=0}^{T-1} \log \frac{p(a_t | c_{0:t}, k) p(r_t, s_{t+1} | k, c_{0:t}, a_t)}{p(a_t | c_{0:t}) p(r_t, s_{t+1} | c_{0:t}, a_t)} \right].$$

Because the exploration policy is not permitted to condition on the true task identity,  $\pi_e(a \mid c, k) = \pi_e(a \mid c)$ , the action-probability terms in the numerator and denominator are identical and cancel, leaving only the reward and transition terms. Applying the Markov property,  $p(r, s' \mid k, c, a) = p(r, s' \mid k, s, a)$ , the objective reduces to

$$J = I(C_{0:H}; K) = \mathbb{E} \left[ \sum_{t=0}^{T-1} \log \frac{p(r_t, s_{t+1} \mid k, s_t, a_t)}{p(r_t, s_{t+1} \mid c_{0:t}, a_t)} \right].$$

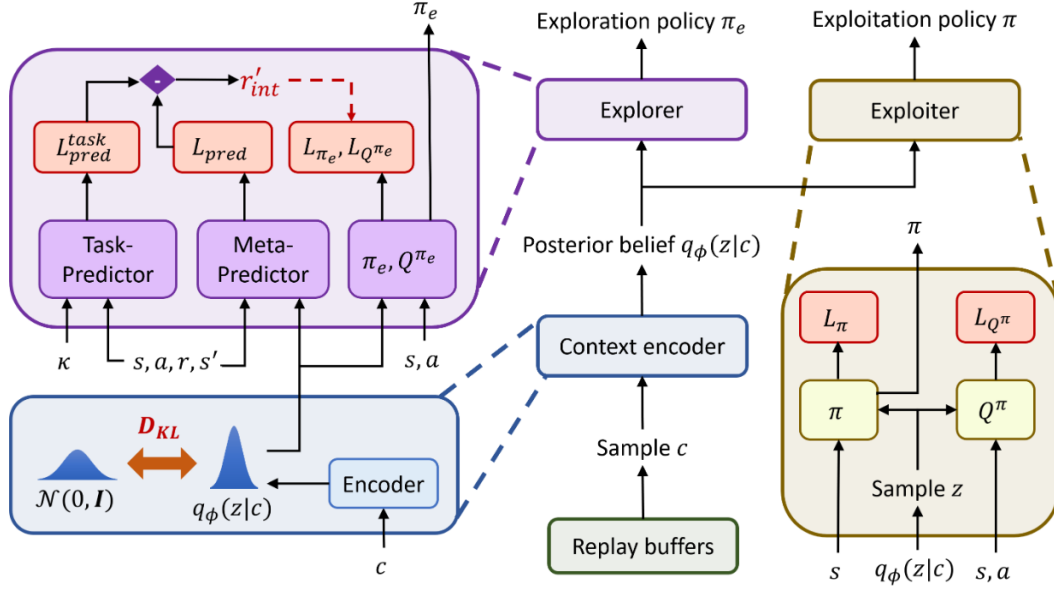
This ratio has a direct interpretation: the numerator is how predictable the transition is given the true task, and the denominator is how predictable it is given only the accumulated exploration context. If a transition is just as predictable without knowing the task as with it,  $p(r_t, s_{t+1} \mid k, s_t, a_t) \approx p(r_t, s_{t+1} \mid c_{0:t}, a_t)$ , the ratio approaches one and its log approaches zero, meaning that transition carried no information about the task at all.

**From intractable probabilities to a regression objective.** Neither term in this ratio can be computed directly, since the transition and reward distributions are unknown and the environment is treated as a black box. MetaCURE sidesteps this by converting probability into prediction loss, using the identity that negative log-likelihood is a valid, if implicit, way to estimate  $-\log p(x)$ , and treating each term as a supervised regression problem instead. The resulting intrinsic reward is

$$r'_{\text{int}} = -\log p(r, s' \mid c, a) + \log p(r, s' \mid k, s, a) = L_{\text{pred}} - L_{\text{pred}}^{\text{task}},$$

the difference between two prediction losses, computed by two separate networks shown in Figure A.29. The Meta-Predictor,  $(\hat{r}_t, \hat{s}_{t+1}) = f_{\text{Meta}}(c_{0:t}, a_t)$ , predicts the outcome of a transition using only the accumulated context, with loss  $L_{\text{pred}} = (r - \hat{r})^2 + \|s' - \hat{s}'\|^2$ , and asks how hard this transition is to predict without knowing the task. The Task-Predictor,  $(\tilde{r}_t, \tilde{s}_{t+1}) = f_{\text{Task}}(k, s_t, a_t)$ , predicts the same outcome given the true task identity, with loss  $L_{\text{pred}}^{\text{task}} = (r - \tilde{r})^2 + \|s' - \tilde{s}'\|^2$ , and asks how hard the transition is to predict if the task is already known. A large intrinsic reward means the Meta-Predictor struggled where the Task-Predictor did not, exactly the signature of a transition that is very informative about the task; a small or negative intrinsic reward means the transition was easy to predict even without knowing the task, and therefore carried little task-specific information.





**Figure A.29.** – MetaCURE architecture. The Task-Predictor and Meta-Predictor produce  $L_{\text{pred}}^{\text{task}}$  and  $L_{\text{pred}}$  from the true task index  $\kappa$  and from context  $(s, a, r, s')$  respectively; their difference gives the intrinsic reward  $r'_{\text{int}}$  that trains the exploration policy  $\pi_e$ . The context encoder produces the posterior belief  $q_\phi(z | c)$ , regularized toward  $\mathcal{N}(0, I)$ , which feeds both the Explorer and the Exploiter. The Exploiter trains  $\pi$  and  $Q^\pi$  using only environment reward, with no intrinsic signal. Adapted from Zhang et al. [102].

**Context encoder.** Alongside the exploration mechanism above, MetaCURE reuses the variational belief machinery of Section 5.9.2 for the exploitation side: a context encoder  $q_\phi(z | c)$ , shown at the bottom left of Figure A.29, is trained on the exploration experience  $c = \{(s, a, r, s')\}$  using a variational objective structurally close to PEARL’s,

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{z \sim q_\phi(z|c)} [\log p(Q^\pi(s, a, z))] - \beta D_{\text{KL}}(q_\phi(z | c) \| p(z)),$$

regularizing the posterior toward a standard Gaussian prior  $\mathcal{N}(0, I)$ , exactly as in Section 5.9.2.2. MetaCURE’s distinguishing contribution is entirely on the exploration side; its belief representation is not itself a new mechanism, but a direct reuse of the same information-bottleneck structure introduced earlier in this section, now driving both the exploration reward computation and the exploitation policy’s input.

**Explorer and exploiter.** As shown in Figure A.29, the exploration policy is trained on a combined reward,

$$r_{\text{exploration}} = r'_{\text{int}} + \lambda r,$$

adding the extrinsic environment reward  $r$  to the intrinsic empowerment signal, rather than discarding extrinsic reward entirely. This is a deliberate choice: even in a sparse-reward environment, extrinsic reward is still genuinely informative whenever it does occur, it simply cannot be relied upon as the sole training signal, so it is retained as a useful auxiliary component rather than an insufficient primary one. The exploitation policy, by contrast, is trained purely on extrinsic reward,  $\pi(a \mid s, z)$  optimizing  $r_{\text{env}}$  with no intrinsic term at all, since its role is purely to exploit the task representation already produced by exploration and inference, not to seek further information about the task. Both policies are trained with Soft Actor-Critic, chosen for being off-policy, sample-efficient, and naturally supporting stochastic policies, matching the same choice made by PEARL and ELUE in Section 5.9.2.

**Meta-training.** Algorithm 63 summarizes meta-training. Trajectories are first collected from both the exploration and exploitation policies for every sampled task and stored in per-task replay buffers. Network optimization then proceeds by, for each task, sampling a context batch for the encoder and a transition batch for the predictors, updating the Task-Predictor and Meta-Predictor by minimizing their respective losses, updating the context encoder via the ELBO objective above, computing the intrinsic reward as the difference between the two predictor losses, forming the combined exploration reward, and updating both policies' actors and critics with SAC.

---

**Algorithm 63** MetaCURE Meta-Training

---

**Require:** Meta-training task distribution  $p(\mathcal{K})$   
Sample meta-training tasks  $\{\kappa_m\}_{m=1}^M \sim p(\mathcal{K})$   
**for** each meta-training task **do**  
    Initialize replay buffer  $\mathcal{B}^{\kappa_m}$   
**end for**  
Initialize exploration policy  $\pi_e$ , exploitation policy  $\pi$ , context encoder  $q_\phi(z \mid c)$ , Task-Predictor, Meta-Predictor  
**while** training not converged **do** ▷ Collect trajectories  
    **for** each meta-training task **do**  
        Execute the adaptation procedure (Algorithm 64) on task  $\kappa_m$   
        Collect exploration and exploitation trajectories; store all transitions in  $\mathcal{B}^{\kappa_m}$   
    **end for** ▷ Network optimization  
    **for** each gradient update **do**  
        **for** each meta-training task **do**  
            Sample context batch  $b_{\text{enc}}^{\kappa_m} \sim \mathcal{B}^{\kappa_m}$  and transition batch  $b^{\kappa_m} \sim \mathcal{B}^{\kappa_m}$   
            Update Task-Predictor by minimizing  $L_{\text{pred}}^{\text{task}}$   
            Update Meta-Predictor by minimizing  $L_{\text{pred}}$   
            Update context encoder by maximizing  $\mathcal{L}_{\text{ELBO}}$   
            Compute intrinsic reward  $r'_{\text{int}} = L_{\text{pred}} - L_{\text{pred}}^{\text{task}}$   
            Compute exploration reward  $r_{\text{exploration}} = r'_{\text{int}} + \lambda r$   
            Update exploration critic and actor by minimizing  $L_{Q^{\pi_e}}$  and  $L_{\pi_e}$   
            Update exploitation critic and actor by minimizing  $L_{Q^\pi}$  and  $L_\pi$   
        **end for**  
    **end for**  
**end while**

---

**Adaptation.** At test time, Algorithm 64 splits interaction into two explicit phases, unlike PEARL and ELUE in Section 5.9.2, which interleave belief updates and action selection using a single policy throughout. During the exploration phase, the agent repeatedly computes the current posterior belief  $q_\phi(z \mid c)$ , selects actions using the exploration policy conditioned on that belief, and accumulates observed transitions into the context. Once exploration concludes, a task embedding  $z$  is sampled once from the final posterior, and the exploitation phase proceeds using the exploitation policy conditioned on this fixed  $z$ , with no further belief updates.

---

**Algorithm 64** MetaCURE Adaptation

---

**Require:** Meta-test task  $\kappa \sim p(\mathcal{K})$ , number of adaptation episodes  $E$

Initialize context  $c \leftarrow \emptyset$  ▷ Exploration phase

**for** each exploration episode **do**

**for** each environment step **do**

Compute posterior belief  $q_\phi(z \mid c)$

Select action  $a \sim \pi_e(a \mid s, q_\phi(z \mid c))$

Execute action and observe  $(r, s')$

Update context  $c \leftarrow c \cup \{(s, a, r, s')\}$

**end for**

**end for**

Sample latent task embedding  $z \sim q_\phi(z \mid c)$  ▷ Exploitation phase

**for** each environment step **do**

Select action  $a \sim \pi(a \mid s, z)$

Execute action

**end for**

---

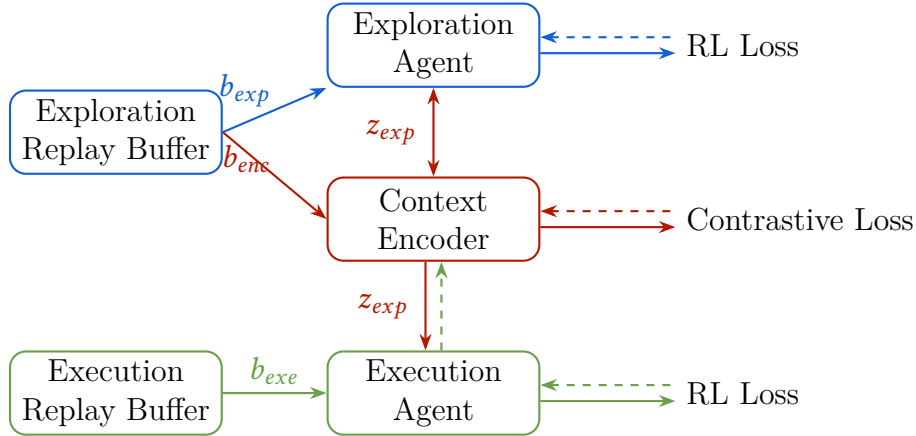
**Relation to earlier and later methods.** MetaCURE and DREAM share the same architectural commitment, decoupling a dedicated exploration policy from the policy that exploits the resulting task representation, but derive their exploration objectives from different sources. DREAM measures information recovery through a learned decoder  $q_\omega(z \mid \tau^{\text{exp}})$  predicting a compressed latent  $z$ , entirely independent of the exploitation policy’s own value estimates. MetaCURE instead measures information gain through the empowerment quantity  $I(C_{0:H}; K)$ , operationalized as the gap between a task-aware and a task-blind predictor of immediate transition outcomes, and additionally reuses the variational context-encoding machinery of Section 5.9.2 for its exploitation side, rather than building a separate encoder for that purpose as DREAM does. Section 5.9.3.3 turns to a third construction of the same underlying idea, CCM’s information-gain exploration, which derives its intrinsic reward from InfoNCE-based mutual information bounds tied directly to its own contrastive context encoder, rather than from either a learned decoder or a pair of task-aware and task-blind predictors.

### A.3.1.3. CCM: Information-Gain Exploration

Fu et al. [103] identify a weakness in PEARL that neither DREAM nor MetaCURE directly target: PEARL is weak not only in how it collects exploratory experience, but in how it encodes that experience once collected. An encoder trained only to support value-function and dynamics prediction, as PEARL’s is in Section 5.9.2.2, can learn a latent  $z$  that is largely indistinguishable across tasks, since accurately

predicting shared dynamics does not require discriminating between tasks at all; the encoder may capture irrelevant information and noise instead of task identity, giving a weak and uninformative gradient signal to whatever policy is conditioned on it. Because adaptation quality depends on two separate things, whether the encoder extracts genuinely task-specific information and whether the exploration policy collects trajectories that reveal that information at all, CCM proposes two separate fixes, a contrastive context encoder, covered in Section 5.9.4, and an information-gain exploration objective, covered here.

**Overall architecture.** CCM trains three modules jointly: a context encoder, an exploration policy, and an execution policy, connected as shown in Figure A.30. Two separate replay buffers feed the system: the exploration replay buffer supplies batches  $b_{\text{exp}}$  to the exploration agent and batches  $b_{\text{enc}}$  to the context encoder, while the execution replay buffer separately supplies batches  $b_{\text{exe}}$  to the execution agent. The context encoder is trained purely from  $b_{\text{enc}}$  using the contrastive loss described in Section 5.9.4, entirely independent of either policy’s own training signal. Its output, the latent context  $z_{\text{exp}}$ , is then fed forward to both agents: to the exploration agent, where it is used to compute the information-gain intrinsic reward described in this subsection, and to the execution agent, where it conditions the policy directly. Each agent is trained with its own ordinary RL loss, shown as the dashed arrows in Figure A.30, on top of whichever reward signal it receives, intrinsic-augmented reward for the exploration agent, environment reward alone for the execution agent.



**Figure A.30.** – Contextual control mechanism.

**Information gain.** Rather than maximizing environment reward directly, or maximizing a fixed empowerment quantity over an entire exploration episode as in Section 5.9.3.2, CCM’s exploration policy is rewarded for the incremental reduction in

task uncertainty produced by each individual transition. This is formalized as an information gain,

$$I(z \mid \tau_{1:i-1}; \tau_i) = H(z \mid \tau_{1:i-1}) - H(z \mid \tau_{1:i}) = I(z; \tau_{1:i}) - I(z; \tau_{1:i-1}),$$

the reduction in entropy of the task latent  $z$  that results from observing one additional transition  $\tau_i$ , given everything already observed. For this quantity to be meaningfully estimated from the encoder’s own output rather than from raw trajectories, the encoder must be context-sufficient: compressing a trajectory into an embedding must not discard information relevant to the task,

$$I(b_1, b_2) = I(e(b_1), b_2),$$

so that mutual information computed on the embeddings  $c = e(\tau)$  is the same as mutual information computed on the raw trajectories themselves,  $I(c_{\text{pos}}; c_{1:i}) = I(c_{\text{pos}}; c_{1:i-1})$ , where  $c_{\text{pos}}$  denotes the embedding of a positive trajectory patch, one drawn from the same task.

**Bounding information gain with InfoNCE.** Neither  $I(z; \tau_{1:i})$  nor  $I(z; \tau_{1:i-1})$  can be computed exactly, but both can be bounded using the InfoNCE construction reviewed in Appendix G. A standard InfoNCE bound gives a lower bound on the first quantity,

$$I(c_{\text{pos}}; c_{1:i}) > \log(W) - L_{\text{lower}}, \quad L_{\text{lower}} = -\mathbb{E} \left[ \log \frac{\exp(f(c_{1:i}, c_{\text{pos}}))}{\sum_j \exp(f(c_{1:i}, c_j))} \right],$$

where  $f$  is the same similarity function used by the contrastive encoder in Section 5.9.4, and  $W$  is the number of samples competing in the denominator. A structurally similar bound, but restricted to negative samples only in the denominator, gives an upper bound on the second quantity,

$$I(c_{\text{pos}}; c_{1:i-1}) \leq -L_{\text{upper}} + \log(W), \quad L_{\text{upper}} = -\mathbb{E} \left[ \log \frac{\exp(f(c_{1:i-1}, c_{\text{pos}}))}{\sum_{c_j \in C_{\text{neg}}} \exp(f(c_{1:i-1}, c_j))} \right],$$

with  $C_{\text{neg}}$  containing only negative samples, unlike  $L_{\text{lower}}$ ’s denominator, which sums over all samples including the positive. Subtracting the lower bound from the upper bound cancels the shared  $\log(W)$  term exactly, giving a clean estimate of the information gain itself,

$$I(z \mid \tau_{1:i-1}; \tau_i) = L_{\text{upper}} - L_{\text{lower}}.$$

This cancellation is the reason the two bounds are constructed with matching, but not identical, denominators:  $L_{\text{lower}}$ 's all-inclusive denominator and  $L_{\text{upper}}$ 's negatives-only denominator are what allow the two  $\log(W)$  terms to cancel cleanly when the bounds are subtracted, leaving a quantity computable directly from the encoder's own similarity scores.

**Intrinsic reward.** This information-gain estimate is used directly as an intrinsic reward for the exploration policy,

$$r_{\text{aux}} = L_{\text{upper}} - L_{\text{lower}},$$

combined with the environment reward to form the total exploration reward,  $r^e = r_{\text{env}} + \alpha r_{\text{aux}}$ , structurally identical to MetaCURE's combined exploration reward in Section 5.9.3.2, though derived from a contrastive bound rather than from a task-aware versus task-blind predictor gap. A large intrinsic reward indicates that the most recently collected transition substantially improved the quality of task inference; a near-zero reward indicates the transition added little the encoder did not already know.

**Interpretation of the two terms.** Expanding  $L_{\text{upper}} - L_{\text{lower}}$  reveals two distinct components,

$$L_{\text{upper}} - L_{\text{lower}} = \underbrace{\mathbb{E} [f(c_{1:i}, c_{\text{pos}}) - f(c_{1:i-1}, c_{\text{pos}})]}_{\text{increased similarity to the correct task}} - \underbrace{\mathbb{E} \left[ \log \frac{\sum_{c_j \in C} \exp(f(c_{1:i}, c_j))}{\sum_{c_j \in C_{\text{neg}}} \exp(f(c_{1:i-1}, c_j))} \right]}_{\text{regularization}}.$$

The first term rewards the new transition for making the trajectory embedding more similar to the correct task's representation than it was before observing that transition. The second term penalizes any increase in similarity that is not specific to the correct task, preventing the exploration policy from being rewarded for transitions that make the embedding more similar to every task's representation simultaneously, which would carry no task-discriminating information despite raising the raw similarity score.

**Training procedure.** As shown in Algorithm 65, training proceeds by sampling a task, rolling out the exploration policy, and storing the resulting trajectory in both an exploration buffer and an execution buffer. Positive and negative trajectory batches are sampled to compute the contrastive loss described in Section 5.9.4, updating the query encoder  $e_\phi$  and, via the momentum rule from Appendix G, the key encoder  $\bar{e}_\phi$ . No intrinsic reward is used when training the encoder itself; the contrastive loss is entirely self-supervised, independent of any reward signal. The information-gain quantities  $L_{\text{lower}}$  and  $L_{\text{upper}}$  are then computed from the encoder’s output on the current and previous trajectory prefixes, giving the intrinsic reward used to train the exploration policy with SAC. Separately, the execution policy is trained purely on environment reward, conditioned on the latent context computed from the execution buffer, which includes exploration trajectories but without any intrinsic reward attached to them.

**Advantages over PEARL.** Table A.17 summarizes CCM’s departures from PEARL across both axes it addresses. Where PEARL uses a single policy for both exploration and execution, CCM separates them; where PEARL’s encoder is trained through value-function and dynamics prediction, CCM’s is trained contrastively; where PEARL relies on Thompson sampling over its posterior for exploration, CCM optimizes an explicit information-gain reward; and where PEARL uses a single replay buffer, CCM separates exploration and execution data.

**Table A.17.** – CCM’s departures from PEARL.

| PEARL                          | CCM                                         |
|--------------------------------|---------------------------------------------|
| One policy                     | Separate exploration and execution policies |
| Value-function-trained encoder | Contrastive encoder                         |
| Thompson sampling exploration  | Information-gain exploration                |
| No intrinsic reward            | Contrastive intrinsic reward                |
| One replay buffer              | Two replay buffers                          |

**Relation to earlier methods and synthesis of Section 5.9.3.** DREAM, MetaCURE, and CCM all decouple a dedicated exploration policy from the policy that exploits the resulting task representation, and all reward exploration with some form of information-theoretic signal rather than task reward alone, but each derives that signal differently. DREAM measures how well a learned decoder can recover a task latent  $z$  from an exploration trajectory, entirely independent of the exploitation policy’s value estimates, and is the only one of the three with a formal consistency argument and sample-complexity bound. MetaCURE measures the gap between a task-aware and a task-blind predictor of immediate transition outcomes,



reusing PEARL-style variational belief machinery for its exploitation side. CCM measures per-transition information gain about a contrastively trained latent, using matched InfoNCE upper and lower bounds whose difference cancels a shared normalization term. All three demonstrate the same underlying lesson from a different angle: an exploration signal defined independently of the exploitation policy’s own gradients avoids the chicken-and-egg coupling that limits RL<sup>2</sup>-style and belief-based end-to-end methods, at the cost of additional architectural complexity, a second policy, a second replay buffer or auxiliary predictor network, and, for CCM and MetaCURE, no formal guarantee that the resulting exploration remains optimal in the way DREAM’s does.

Section 5.9.4 turns to the second half of CCM’s contribution, the contrastive context encoder itself, and to ESCP, a method that addresses a different weakness of PEARL’s encoder entirely: its slowness to adapt when the environment itself changes within an episode.

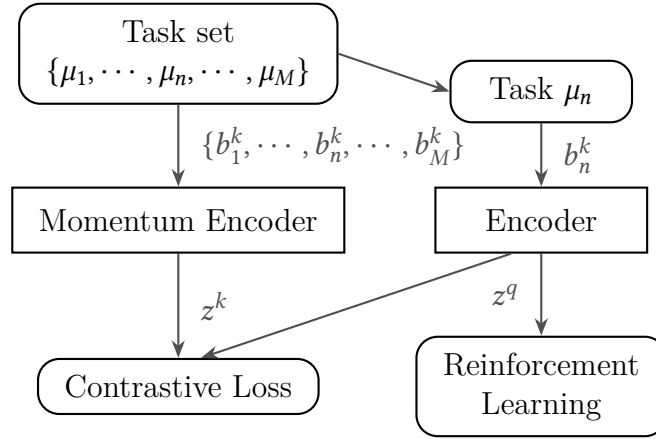
## A.3.2. Discriminative and Self-Supervised Representations

The methods surveyed in Section 5.9.2 all share a common architecture: an explicit generative or quasi-generative model, a KL divergence against a fixed prior, and, in VariBAD’s and ELUE’s case, a decoder reconstructing observed transitions. This subsection surveys two methods that abandon this apparatus entirely. Rather than fitting an approximate posterior to a generative model, these methods shape their task representation directly through a discriminative or self-supervised training signal, requiring only that the representation behave correctly with respect to some externally imposed structure, same-task consistency, or environment-specific separability, rather than that it correspond to a well-defined posterior over anything.

### A.3.2.1. CCM: Contrastive Context Encoder

The exploration side of CCM was introduced in Section 5.9.3.3; this subsection completes the method by describing the encoder itself, the second of the two fixes Fu et al. [103] propose for PEARL’s weaknesses. Where PEARL’s encoder in Section 5.9.2.2 is shaped indirectly, by whatever helps the critic predict value accurately, CCM’s encoder is shaped directly, by an objective stated purely in terms of task identity: embeddings of trajectories from the same task should be pulled together, and embeddings of trajectories from different tasks should be pushed apart. This is a direct application of the contrastive learning framework and the Momentum Contrast mechanism reviewed in Appendix G, with trajectory batches taking the place of augmented images.

**Positive and negative pairs.** Figure A.31 shows how training pairs are constructed. A task  $\mu_n$  is sampled from the task set  $\{\mu_1, \dots, \mu_M\}$ , and a set of trajectory batches  $b_n^k$  is drawn from it. Two such batches sampled from the same task,  $(b_n^q, b_n^k)$ , form a positive pair, while batches drawn from any other task in the set form negative pairs. Each batch in the positive pair is passed through a different encoder: the query batch  $b_n^q$  is passed through the ordinary, gradient-trained encoder,  $z_q = e_q(b_n^q)$ , and the key batch  $b_n^k$  is passed through the momentum encoder introduced in Appendix G,  $z_k = e_k(b_n^k)$ , updated only via the slow momentum rule and never directly by gradient descent.



**Figure A.31.** – Contrastive context learning in CCM. A task  $\mu_n$  is sampled from the task set, and trajectory batches  $b_n^k$  are drawn from it and passed through both the query encoder, producing  $z^q$ , and the momentum encoder, producing  $z^k$ . The pair  $(z^q, z^k)$  drives the contrastive loss, while  $z^q$  is separately consumed by reinforcement learning, that is, by the exploration and execution agents described in Section 5.9.3.3, with no gradient from the RL side flowing back into the encoder. Adapted from Fu et al. [103].

**Contrastive loss.** The encoder is trained with an InfoNCE loss, following the general form reviewed in Appendix G,

$$L_{\text{contrastive}} = \log \frac{\exp(f(z_q, z_{\text{pos}}))}{\sum_j \exp(f(z_q, z_j))},$$

where  $z_{\text{pos}}$  is the key embedding of the positive pair and the sum in the denominator ranges over the positive key together with every negative key drawn from other tasks. Rather than using a fixed similarity function such as a plain dot product, as

in the original MoCo formulation, CCM learns the similarity metric itself through a bilinear form,

$$f(z_q, z_k) = z_q^\top W z_k,$$

with  $W$  a learnable matrix, allowing the notion of similarity used to compare trajectory embeddings to be adapted during training rather than fixed in advance to plain cosine similarity or dot product.

**No coupling to the RL objective.** As Figure A.31 makes explicit,  $z^q$  is consumed by reinforcement learning, feeding both the exploration agent’s intrinsic reward computation and the execution agent’s policy, exactly as detailed in Section 5.9.3.3, but no gradient flows from either agent back into the encoder. The encoder’s only training signal is the contrastive loss above. This is the precise structural difference from PEARL that CCM’s authors identify as the source of PEARL’s weak, uninformative encoder gradient in Section 5.9.2.2: PEARL’s encoder parameters are updated using the critic loss directly, so what the encoder learns to represent is filtered entirely through whatever happens to help value prediction, whereas CCM’s encoder is updated using a loss stated purely in terms of task identity, with reinforcement learning appearing only as a downstream consumer of its output, never as a term in its training objective.

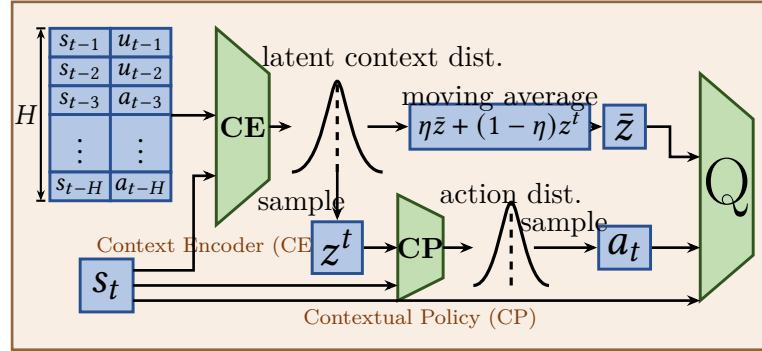
**Relation to Section 5.9.2.** This is the same qualitative shift that Section 5.9.2.3’s ELUE makes for a different reason: both methods remove the entanglement between representation learning and policy optimization, though ELUE does so by using a generative CVAE objective decoupled from the critic, while CCM does so by replacing the generative objective entirely with a discriminative, contrastive one. Section 5.9.4.2 turns to a second method, ESCP, which departs from Section 5.9.2’s variational framework in a related but distinct way, by abandoning the probabilistic posterior itself in favor of a deterministic embedding shaped by two purely statistical objectives.

### A.3.2.2. ESCP: Adapt to Environment Sudden Changes by Learning a Context-Sensitive Policy

Luo et al. [104] target an assumption implicit in every method surveyed so far in Section 5.9: that the environment remains fixed for the duration of an episode. Many real settings violate this. A self-driving car, for instance, can transition suddenly from a dry road to a wet or icy one mid-episode, changing the dynamics the policy

must contend with without any episode boundary to signal that a new context has begun. The standard context-based pipeline, history through a context encoder to a latent, through a policy to an action, struggles here for a structural reason: the encoder is built to converge toward a stable estimate of task identity over the course of an episode, and this same convergence behavior makes it slow to recognize that the environment has changed once it has already settled on an estimate for the old one.

**Architecture.** Figure A.32 shows the resulting architecture. A window of recent history, the last  $H$  state-action pairs  $(s_{t-1}, a_{t-1}), \dots, (s_{t-H}, a_{t-H})$ , is passed through a context encoder (CE) to produce a distribution over the latent context, from which a sample  $z^t$  is drawn. This sample is concatenated with the current state  $s_t$  and passed to a contextual policy (CP), which outputs an action distribution and samples an action  $a_t$ . In parallel, the sampled context  $z^t$  is combined with a stored running estimate  $\bar{z}$  through a moving average,  $\eta\bar{z} + (1 - \eta)z^t$ , producing a smoothed context  $\bar{z}$  that feeds the critic  $Q$ , so that the critic is trained against a more stable target than the raw, per-timestep context sample used by the policy.



**Figure A.32.** – ESCP architecture. A window of  $H$  past state-action pairs is passed through the context encoder (CE) to produce a latent context distribution, from which  $z^t$  is sampled. The contextual policy (CP) combines  $z^t$  with the current state  $s_t$  to produce an action distribution and sample  $a_t$ . A moving average of  $z^t$  against a stored estimate  $\bar{z}$  produces the smoothed context used by the critic  $Q$ . Adapted from Luo et al. [104].

**Variance minimization.** The first of ESCP’s two representation-learning objectives addresses noise in the encoder’s output. For a fixed environment, the latent context should be stable rather than fluctuating from one window of history to the next

purely due to sampling noise. This is enforced by minimizing the variance of the latent around its mean,

$$\mathcal{L} = \mathbb{E}[\|z - \mu\|^2] = \mathbb{E}[\|z - \mathbb{E}[z]\|^2] + \|\mathbb{E}[z] - u\|^2,$$

which decomposes into the variance of  $z$  around its own expectation and a term pulling that expectation toward a fixed reference  $u$ , together discouraging the encoder from producing noisy, high-variance estimates for what should be a consistent representation of the current environment.

**Determinant maximization.** The second objective addresses the complementary failure mode: different environments must produce genuinely separated latents, or the policy has no way to tell them apart. Given a similarity function between the latents produced for a set of tasks, taken here to be a Gaussian kernel,  $R(i, j) = \exp(-\alpha\|z_i - z_j\|^2)$ , this similarity can be assembled into a matrix  $R$  across all tasks under consideration. ESCP maximizes the determinant of this matrix,  $\mathcal{L} = \det(R)$ . The intuition is that a similarity matrix has a large determinant precisely when the vectors it compares are well separated from one another, close to orthogonal in the extreme case, and a small determinant when several of them collapse toward the same point, since near-duplicate rows or columns drive the determinant toward zero. Maximizing  $\det(R)$  therefore directly penalizes the encoder for representing two different environments with similar latents, addressing the opposite failure mode from variance minimization: variance minimization keeps a single environment’s representation tight, while determinant maximization keeps different environments’ representations apart.

**History truncation.** Because the GRU context encoder has only limited memory capacity, ESCP truncates the trajectory fed into it to a fixed-size window of the  $H$  most recent state-action pairs, rather than the full episode history, shown explicitly as the truncated window in Figure A.32. This is what allows the encoder to track a sudden environment change at all: if the encoder were conditioned on the entire episode history, evidence of a change partway through the episode would be diluted by a much larger volume of pre-change experience, and the encoder’s estimate would continue to reflect the old environment for longer than necessary. Restricting the window to recent history ensures that once the environment changes, old, no longer relevant experience is dropped from the encoder’s input relatively quickly, allowing the latent context to shift accordingly.

**Combined loss.** The two representation-learning terms are combined into a single context encoder loss,

$$L_{CE} = \lambda \sum_i \mathbb{E} [\|z_i - \bar{z}_i\|^2] - \log \det(R).$$

**Stop-gradient in the moving average.** A subtlety in how  $\bar{z}_i$  is maintained is important for training stability. The moving-average update itself is not treated as fully differentiable: the previous estimate  $\bar{z}_i$  is stop-graduated before being blended with the new batch estimate,

$$\bar{z}_i \leftarrow \eta \perp(\bar{z}_i) + (1 - \eta) \mathbb{E}_{z_i^t \sim \mathcal{B}_z} [z_i^t],$$

where  $\perp(\cdot)$  denotes the stop-gradient operator. This prevents gradients from the variance and determinant losses from flowing backward through the entire history of past moving-average updates, which would otherwise require differentiating through an unrolled chain of exponential averaging reaching arbitrarily far back in training; instead, at every update,  $\bar{z}_i$  is treated as a fixed target with respect to the current gradient step, and only the current batch’s contribution to it remains differentiable.

**Monte Carlo training objective.** In practice, the expectations in  $L_{CE}$  are estimated from a sampled batch of latent contexts  $\mathcal{B}_z$ , giving the Monte Carlo objective actually optimized during training,

$$\hat{\mathcal{L}}_{CE} = \lambda \sum_{i=0}^{M-1} \mathbb{E}_{z_i^t \sim \mathcal{B}_z} [\|z_i^t - \perp(\bar{z}_i)\|_2^2] - \log \det(R_{\{\bar{z}_i\}}),$$

where the variance term is computed against the stop-graduated moving-average estimate  $\perp(\bar{z}_i)$  rather than against a batch mean recomputed from scratch at every step, and the determinant term is computed on the similarity matrix built from the current moving-average estimates  $\{\bar{z}_i\}$  across all  $M$  training environments, rather than from the raw per-timestep samples  $z_i^t$ . This is the loss used directly in Algorithm 67.

**Training.** Training, summarized in Algorithm 67, proceeds by sampling a training environment, encoding the recent interaction history with the GRU to obtain a latent context, concatenating that context with the current state to select an action via the SAC actor, and storing the resulting transition in a replay buffer. From sampled minibatches, the moving-average context is updated using the stop-gradient rule above, the SAC actor and critic are trained using the sampled and moving-average contexts respectively, and the context encoder itself is trained using the Monte Carlo variance-plus-determinant objective  $\hat{\mathcal{L}}_{CE}$ .

---

**Algorithm 67** Training Process of ESCP

---

```

procedure TrainESCP( $\{p_0, \dots, p_{M-1}\}, g, \pi, Q, H, \{\bar{z}_0, \dots, \bar{z}_{M-1}\}, R$ )
  while training do
    Sample environment index  $i \sim \text{Uniform}(\{0, \dots, M-1\})$ 
    Initialize  $t \leftarrow 0$ 
    while episode not finished do
      Infer latent context  $z_i^t \sim g(\tau_{t-H:t}, s_t; \phi)$ 
      Sample action  $a_t \sim \pi(a_t \mid s_t, z_i^t; \theta)$ 
      Execute  $a_t$  in environment  $p_i$ ; observe  $s_{t+1}, r_t$ 
      Store  $(s_t, a_t, s_{t+1}, r_t, i)$  into replay buffer  $R$ 
      if network update is required then
        Sample minibatch  $B$  from  $R$ 
        Infer latent contexts  $z$  for  $B$  using context encoder  $g$ 
        Update moving-average contexts:  $\bar{z}_i \leftarrow \eta \perp(\bar{z}_i) + (1 - \eta) \mathbb{E}_{z_i^t \sim \mathcal{B}_z} [z_i^t]$ 
        Update actor parameters  $\theta$  and critic parameters  $\psi$  using SAC with
         $B, z$ , and  $\bar{z}$ 
        Compute context encoder loss  $\hat{\mathcal{L}}_{CE}(\phi) = \lambda \sum_{i=0}^{M-1} \mathbb{E}_{z_i^t \sim \mathcal{B}_z} [\|z_i^t - \perp(\bar{z}_i)\|_2^2] - \log \det(R_{\{\bar{z}_i\}})$ 
        Update encoder parameters  $\phi$  using Adam
      end if
       $t \leftarrow t + 1$ 
    end while
  end while
end procedure

```

---

**Why this works.** Table A.18 maps each of ESCP’s three mechanisms directly onto a specific failure mode of prior context-based methods.

**Table A.18.** – ESCP’s mechanisms mapped to the failure modes of prior context-based methods.

| Failure mode of previous methods                     | ESCP’s fix                                    |
|------------------------------------------------------|-----------------------------------------------|
| Encoder output is noisy                              | Variance minimization stabilizes it           |
| Different environments can collapse to similar codes | Determinant maximization separates them       |
| Old experiences dominate the RNN memory              | History truncation focuses on recent evidence |

**Departure from variational inference.** ESCP’s design is a deliberate departure from the variational machinery used throughout Section 5.9.2. Table A.19 makes this explicit by contrasting ESCP against a representative variational method such as PEARL.

**Table A.19.** – Variational inference (e.g. PEARL) compared to ESCP.

| Aspect                           | Variational Inference<br>(e.g. PEARL)                   | ESCP                                                               |
|----------------------------------|---------------------------------------------------------|--------------------------------------------------------------------|
| Goal                             | Approximate the posterior over the latent task variable | Learn a discriminative, stable environment representation          |
| Learns posterior $q_\phi(z   c)$ | Yes                                                     | No                                                                 |
| Uses prior $p(z)$                | Yes                                                     | No                                                                 |
| KL divergence                    | Yes                                                     | No                                                                 |
| ELBO optimization                | Yes                                                     | No                                                                 |
| Reparameterization trick         | Yes                                                     | Not required                                                       |
| Main loss                        | RL loss + KL regularization                             | Variance loss + determinant loss                                   |
| Latent meaning                   | Bayesian latent variable                                | Deterministic environment embedding (with fixed Gaussian sampling) |
| Training objective               | Variational inference                                   | Self-supervised representation learning                            |

Every method surveyed in Section 5.9.2 fits the left-hand column of this table: a posterior is fit against a prior using a KL-regularized objective, and the latent carries an explicit Bayesian interpretation as an approximate belief. ESCP fits none of these criteria. Its latent is not the output of a posterior in any generative-model sense; it is a deterministic embedding, shaped entirely by the variance and determinant objectives above, with Gaussian sampling retained only as a fixed mechanism for injecting



exploration noise rather than as part of an inference procedure. This places ESCP alongside CCM in Section 5.9.4.1 as a method that discards the Bayesian apparatus of Section 5.9.2 entirely, though the two differ in what statistical property they optimize for: CCM’s contrastive loss enforces task-discriminability through positive and negative sampling across a batch of tasks, while ESCP’s variance and determinant terms enforce, respectively, within-environment stability and between-environment separation directly, without requiring the batch-based positive/negative pair construction that CCM’s InfoNCE loss depends on.

**Closing Section 5.9.4.** CCM and ESCP address two different weaknesses of the variational, PEARL-style encoder using two different discriminative mechanisms: CCM addresses poor task discriminability with a contrastive objective operating across a distribution of tasks, while ESCP addresses slow adaptation to non-stationary, within-episode environment changes with a variance-and-separation objective operating on the encoder’s output directly, together with a bounded history window and a stop-gradient moving average for training stability. Both depart from the belief-based framework of Section 5.9.2 in the same direction, replacing an approximate Bayesian posterior with a representation shaped by a self-supervised statistical criterion, while retaining the same overall pipeline of encoding history into a latent that conditions a downstream policy. Section 5.9.5 turns to a more radical departure still, methods that dispense with a learned task representation of any kind, whether probabilistic or discriminative, and instead regress directly from experience to task-specific policy parameters or value estimates.

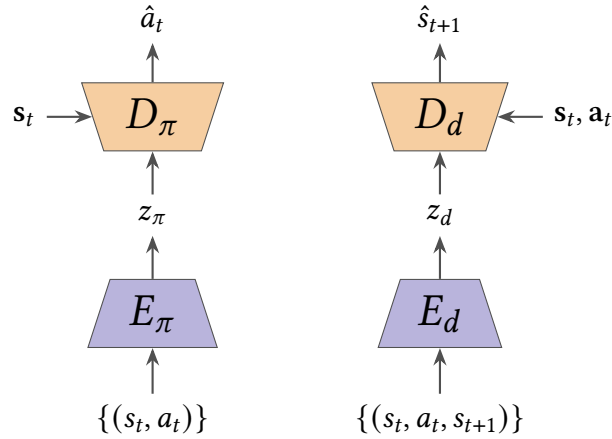
### A.3.3. Direct Parameter and Value Regression

Every method surveyed in Sections 5.9.2 through 5.9.4 builds some form of learned representation of the task, whether an approximate Bayesian posterior, a contrastive embedding, or a variance-and-separation-shaped latent, and conditions a policy on that representation. This subsection surveys a structurally different family, methods that skip the representation-and-condition pattern almost entirely and instead regress directly from experience to the quantity ultimately needed, whether task-specific policy parameters or an estimate of achievable return, using closed-form or supervised optimization rather than posterior inference of any kind.

### A.3.3.1. PD-VF: Fast Adaptation to New Environments via Policy-Dynamics Value Functions

Raileanu et al. [105] ask whether an agent can adapt after observing only a few transitions, without performing any gradient update at test time, and answer affirmatively through a mechanism unlike anything else surveyed in this section: rather than inferring a task belief and conditioning a policy on it, PD-VF learns two separate embeddings, a dynamics embedding characterizing the environment and a policy embedding characterizing a behavior, together with a value function that scores how well a given policy embedding would perform under a given dynamics embedding. Adaptation at test time is then a small optimization problem, solved in closed form, rather than a forward pass through a trained encoder.

**Policy-Dynamics Value Function.** The central object PD-VF learns is a function  $W(s, \pi, d) = \mathbb{E}[G_t \mid s, \pi, d]$ , predicting the expected trajectory return  $G_t$  achievable by policy  $\pi$  under environment dynamics  $d$ , starting from state  $s$ . Rather than representing  $\pi$  and  $d$  directly, both are represented by learned embeddings, a policy embedding  $z_\pi$  and a dynamics embedding  $z_d$ , built by two separate encoder-decoder pairs, shown in Figure A.33.



**Figure A.33.** – PD-VF’s self-supervised encoder-decoder pairs. The policy encoder  $E_\pi$  maps a sequence of state-action pairs  $\{(s_t, a_t)\}$  to a policy embedding  $z_\pi$ , decoded by  $D_\pi$  back into a predicted action  $\hat{a}_t$  given the current state  $s_t$ . The dynamics encoder  $E_d$  maps a sequence of transitions  $\{(s_t, a_t, s_{t+1})\}$  to a dynamics embedding  $z_d$ , decoded by  $D_d$  into a predicted next state  $\hat{s}_{t+1}$  given the current state and action. Adapted from Raileanu et al. [105].

**Dynamics embedding.** The dynamics encoder  $E_d$  takes a window of transitions,  $\{(s_t, a_t, s_{t+1})\}$ , and produces a dynamics embedding  $z_d$ . A corresponding decoder  $D_d$  is trained to predict the next state from the current state, action, and this embedding,  $\hat{s}_{t+1} = D_d(s_t, a_t, z_d)$ , with a standard regression loss,  $\mathcal{L} = \|s_{t+1} - \hat{s}_{t+1}\|^2$ . Training this pair to accurately reconstruct transitions forces  $z_d$  to capture whatever information about the environment’s dynamics is needed to predict how it responds to actions.

**Policy embedding.** The policy encoder  $E_\pi$  takes a window of state-action pairs,  $\{(s_t, a_t)\}$ , and produces a policy embedding  $z_\pi$ , with a decoder  $D_\pi$  trained to reconstruct the action taken given the current state and this embedding,  $\hat{a}_t = D_\pi(s_t, z_\pi)$ . Both embeddings are normalized to unit norm after encoding,  $z_d \leftarrow z_d / \|z_d\|_2$  and  $z_\pi \leftarrow z_\pi / \|z_\pi\|_2$ , a normalization that becomes important for the closed-form optimization described next.

**A quadratic value function.** Rather than allowing  $W$  to be an arbitrary function of  $z_\pi$  and  $z_d$ , PD-VF constrains it to be a quadratic form in the policy embedding,

$$\hat{G} = z_\pi^\top A(s_0, z_d) z_\pi,$$

where  $A(s_0, z_d) \in \mathbb{R}^{d \times d}$  is a matrix predicted by a neural network conditioned on the initial state and the dynamics embedding. This specific functional form is chosen deliberately, because it makes finding the best policy embedding for a given environment analytically solvable, without any gradient-based search at test time, unlike the general form  $z_\pi^* = \arg \max_{z_\pi} f(s, z_\pi, z_d)$  would require for an arbitrary  $f$ .

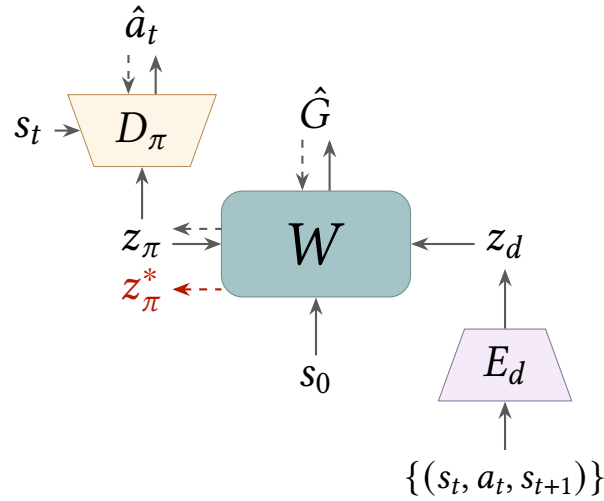
**Ensuring a well-posed optimization.** For this quadratic form to have a well-behaved maximum,  $A$  must be positive definite: if  $A$  had negative eigenvalues, the score  $z_\pi^\top A z_\pi$  could decrease indefinitely in some directions, or the resulting optimization could become ill-conditioned. PD-VF enforces this by construction rather than by penalty, predicting a lower-triangular matrix  $L$  and forming  $A = LL^\top$ , a Cholesky-style decomposition that guarantees  $A$  is positive semi-definite for any predicted  $L$ .

**Closed-form policy embedding search.** With  $A$  guaranteed positive definite, finding the best policy embedding for a given state and dynamics embedding reduces to

$$\max_{z_\pi} z_\pi^\top A z_\pi \quad \text{subject to} \quad \|z_\pi\| = 1,$$

the normalization constraint matching the unit-norm embeddings produced during self-supervised training. This is a standard constrained quadratic optimization problem whose solution is the eigenvector of  $A$  corresponding to its largest eigenvalue, equivalently obtained via the singular value decomposition  $A = USV^\top$ , taking  $z_\pi^* = U(:, 1)$ , the singular vector associated with the largest singular value. No gradient descent, sampling, or iterative search is required: given  $A$ , the optimal policy embedding is found by a single SVD.

**Decoding the optimized embedding.** Once  $z_\pi^*$  is found, it is passed through the same policy decoder  $D_\pi$  trained during self-supervised learning,  $\hat{a}_t = D_\pi(s_t, z_\pi^*)$ , to produce an executable policy. Figure A.34 shows how this fits together at test time: the dynamics encoder  $E_d$  processes a handful of observed transitions to produce  $z_d$ , the value function  $W$ , together with the initial state  $s_0$ , produces  $A$ , from which the closed-form optimization above yields  $z_\pi^*$ , shown by the red dashed path replacing the value function’s usual bidirectional interaction with  $z_\pi$ , and the policy decoder  $D_\pi$  turns this optimized embedding into actions.



**Figure A.34.** – PD-VF at adaptation time. The dynamics encoder  $E_d$  produces  $z_d$  from a handful of observed transitions, feeding the value function  $W$  together with the initial state  $s_0$ . Rather than searching over  $z_\pi$  via the value function’s ordinary gradient interaction (gray dashed arrow), the optimal embedding  $z_\pi^*$  (red dashed arrow) is obtained directly via SVD, and passed to the policy decoder  $D_\pi$  to produce actions. Adapted from Raileanu et al. [105].

**Two training phases.** Training proceeds in two stages, corresponding to the two panels of Figure A.33. The first, self-supervised, phase pretrains multiple PPO policies with different random seeds across the training environments, executes them to collect trajectories, computes policy and dynamics embeddings from these trajectories via  $E_\pi$  and  $E_d$ , and records the initial state  $s_0$  and the resulting Monte Carlo return  $G$  for each. The value function is then trained by regression,  $\hat{G} = W(s_0, z_\pi, z_d)$  with loss  $\mathcal{L} = (G - \hat{G})^2$ . This phase, however, has a structural blind spot: the value function only ever observes policy embeddings  $z_\pi$  produced by encoding real PPO policies, since those are the only policies that were ever executed to generate training data. It never observes the optimized embeddings  $z_\pi^*$  that the closed-form maximization will later produce, and which the value function will be asked to evaluate accurately at test time.

**Distribution shift and DAgger.** This mismatch is a genuine distribution-shift problem: the training set for  $W$  consists of pairs like  $\{(z_\pi^{(1)}, G), (z_\pi^{(2)}, G), \dots\}$ , one policy embedding per pretrained PPO policy, and when  $W$  is later queried at an unseen  $z_\pi^*$  found via SVD, it must extrapolate outside this training distribution, which is sometimes inaccurate. This is the same distribution-shift problem that Data Aggregation (DAgger) was originally designed to solve in imitation learning. In its general form, DAgger repeats the following cycle: adapt the current meta-policy  $\theta$  to each training task, obtaining an adapted policy  $\phi_i$ ; roll out the adapted policy to collect the states it actually visits; query an expert to label those visited states with the optimal action; and add this newly labeled data to the expert dataset,

$$\mathcal{D}_i^* \leftarrow \mathcal{D}_i^* \cup \{(s_t, \pi_i^*(s_t)) : s_t \sim \pi_{\phi_i}\}.$$

The underlying principle is that a model trained only on data generated by one distribution of behavior, here, states visited by an expert, or by a pretrained PPO policy, will perform poorly when later queried on data generated by a different distribution of behavior, here, states or embeddings produced by the model’s own, evolving policy, and the fix is to repeatedly generate fresh data from the model’s current behavior and fold it back into training, so that the training distribution comes to match the distribution the model will actually be evaluated under.

PD-VF instantiates this same cycle, with the roles reassigned to fit its own architecture: rather than adapting a meta-policy and querying an expert for action labels, PD-VF computes  $z_\pi^* = \arg \max_{z_\pi} W$  via the closed-form optimization above, decodes it into an executable policy, rolls that policy out to collect a fresh trajectory and its actual return, and adds this new  $(s_0, z_d, z_\pi^*, G)$  tuple to the training dataset, playing the role that a freshly labeled state-action pair plays in the general DAgger cycle above. Both the value function  $W$  and the policy decoder  $D_\pi$  are then retrained on

this aggregated dataset. Repeating this cycle progressively exposes the value function to the actual distribution of embeddings it will be queried at, closing the gap between what  $W$  was trained on and what it is asked to evaluate at test time, exactly as the general DAgger cycle closes the gap between an expert’s training distribution and a learned policy’s own visitation distribution.

**Algorithm.** Algorithm 68 summarizes the full procedure across five stages: reinforcement learning, which trains the pool of PPO policies used to generate initial training data; self-supervised learning, which trains the dynamics and policy encoder-decoder pairs; initial value function training, which fits  $W$  on embeddings from the pretrained PPO policies; data aggregation, which iteratively closes the distribution-shift gap described above; and evaluation, which shows the resulting test-time adaptation procedure applied to a genuinely new environment.

---

**Algorithm 68** Policy-Dynamics Value Functions (PD-VF)

---

**Require:** Training environments  $\mathcal{D}_{\text{train}}$ , embedding dimension  $d$ , inference horizon  $N_d$

**Ensure:** Policy encoder  $E_\pi$ , policy decoder  $D_\pi$ , dynamics encoder  $E_d$ , dynamics decoder  $D_d$ , value function  $W$

**procedure** ReinforcementLearningPhase

**for all** training environment  $d \in \mathcal{D}_{\text{train}}$  **do**

    Train multiple PPO policies with different random seeds; save checkpoints throughout training

**end for**

**for all** policy  $\pi_i$  **do**

**for all** training environment  $d_j$  **do**

      Execute  $\pi_i$  in  $d_j$ ; store trajectories  $\tau_{ij}$  and returns  $G_{ij}$

**end for**

**end for**

**end procedure**

**procedure** SelfSupervisedLearningPhase

**for all** trajectory  $\tau$  **do**

    Extract first  $N_d$  transitions  $\{(s_t, a_t, s_{t+1})\}$ ; compute  $z_d = E_d(\cdot)$

$\hat{s}_{t+1} = D_d(s_t, a_t, z_d)$ ; minimize  $L_d = \|s_{t+1} - \hat{s}_{t+1}\|_2^2$

    Extract state-action pairs  $\{(s_t, a_t)\}$ ; compute  $z_\pi = E_\pi(\cdot)$

$\hat{a}_t = D_\pi(s_t, z_\pi)$ ; minimize  $L_\pi = \|a_t - \hat{a}_t\|_2^2$

    Normalize  $z_d \leftarrow z_d / \|z_d\|_2$  and  $z_\pi \leftarrow z_\pi / \|z_\pi\|_2$

**end for**

**end procedure**

**procedure** InitialValueFunctionTraining

**for all** trajectory  $\tau$  **do**

    Compute  $z_d, z_\pi$ ; record  $s_0$  and Monte Carlo return  $G$

$L = A(s_0, z_d)$ ;  $A = LL^\top$ ;  $\hat{G} = z_\pi^\top A z_\pi$

    Minimize  $L_W = (G - \hat{G})^2$

**end for**

**end procedure**

**procedure** DataAggregation

**repeat**

**for all** training environment **do**

      Infer  $z_d$ ; compute  $L = A(s_0, z_d)$ ,  $A = LL^\top$

      Compute SVD  $A = USV^\top$ ;  $z_\pi^* = U(:, 1)$ ; normalize  $z_\pi^*$

      Execute decoded policy  $\hat{a}_t = D_\pi(s_t, z_\pi^*)$ ; collect trajectory and return  $G$

      Append  $(s_0, z_d, z_\pi^*, G)$  to the value-function dataset; append visited states to the decoder dataset

**end for**

    Retrain value function  $W$  and policy decoder  $D_\pi$  on the aggregated dataset

**until** convergence

**end procedure**

**procedure** Evaluation(new environment)

CXLIX

  Execute any pretrained PPO policy for  $N_d$  steps; collect transitions  $\{(s_t, a_t, s_{t+1})\}$

$z_d = E_d(\cdot)$ ;  $L = A(s_0, z_d)$ ;  $A = LL^\top$

  Compute SVD  $A = USV^\top$ ;  $z_\pi^* = U(:, 1)$

**while** episode not terminated **do**

**Relation to earlier methods.** PD-VF’s adaptation mechanism has no real counterpart among the methods surveyed in Sections 5.9.2 through 5.9.4: those methods all adapt via a forward pass through a trained encoder, producing a belief or embedding that a fixed policy then conditions on. PD-VF instead adapts by solving a small, closed-form optimization problem, using a learned value function as the objective and a single SVD as the solver, with no belief, no posterior, and no contrastive structure involved at any point. What it shares with the discriminative methods of Section 5.9.4 is only the absence of any variational machinery; what distinguishes it even from those is that no encoder output is ever fed directly to the policy at all, only the initial value-function optimization’s result is. Section 5.9.5.2 turns to FLAP, a second method in this family that also bypasses posterior inference, but regresses directly to task-specific policy parameters themselves rather than to a policy embedding scored by a learned value function.

#### A.3.3.2. FLAP: Linear Representations for Instant Meta-Reinforcement Learning Adaptation

Peng et al. [106] take the idea of regressing directly to a task-specific quantity, developed by PD-VF in Section 5.9.5.1 for a policy embedding scored by a learned value function, and apply it one level more directly: FLAP regresses straight to the task-specific policy weights themselves, skipping the intermediate embedding-and-search step entirely.

**Motivating critique of gradient-based adaptation.** FLAP is motivated by a direct critique of the policy parameter gradient methods of Section 5.6. Gradient-based adaptation, in the style of MAML, requires performing gradient updates at test time, which is computationally slow precisely when fast adaptation matters most; it often generalizes poorly to out-of-distribution tasks, since a small number of gradient steps starting from a shared initialization may not reach a good task-specific optimum for a genuinely novel task; and some variants additionally require reusing training buffers during adaptation, adding memory overhead. FLAP’s goal is an adaptation mechanism with none of these costs: no gradient steps, no iterative search, and no dependence on training data at test time.

**Linear structure assumption.** FLAP’s core assumption is that tasks drawn from a related distribution share a common low-dimensional, linear structure in how their policies are represented. Concretely, the policy is decomposed into a shared feature extractor,  $\phi : S \rightarrow \mathbb{R}^d$ , common to every task, and a task-specific linear output layer,  $(\mathbf{w}_i, b_i)$ , giving the policy for task  $i$ ,



$$\pi_i = \sigma(\langle \phi, \mathbf{w}_i \rangle + b_i).$$

During adaptation,  $\phi$  remains entirely fixed; only  $(\mathbf{w}_i, b_i)$  differs from task to task. This is a strong structural assumption, but one that, if it holds, reduces the entire adaptation problem to finding a single linear layer’s worth of parameters, a far smaller search space than adapting an entire network.

**Meta-training objective.** The shared features and the task-specific output layers are trained jointly across the training task distribution,

$$\{\phi, \{\mathbf{w}_i, b_i\}\} = \arg \max \frac{1}{n} \sum_{i=1}^n q_Y(\pi(\phi, \mathbf{w}_i, b_i), T_i),$$

with a standard off-policy critic trained alongside,

$$L_{\text{critic}} = \mathbb{E} \left[ Q_\psi(s, a) - (r + \gamma Q(s', a')) \right]^2.$$

At the end of meta-training, this produces a shared feature extractor  $\phi$  and, for every training task  $i$ , a genuine, RL-optimized pair of output-layer weights  $(\mathbf{w}_i, b_i)$ , exactly the target values the Adapter Network is trained to reproduce for new tasks.

**Adapter Network.** Rather than solving  $\mathbf{w}_{\text{new}}, b_{\text{new}} = \arg \max_{\mathbf{w}, b} q_Y(\pi(\phi, \mathbf{w}, b), T_{\text{new}})$  by gradient descent at test time, FLAP trains a separate network to predict the output-layer weights directly from a small number of observed transitions,

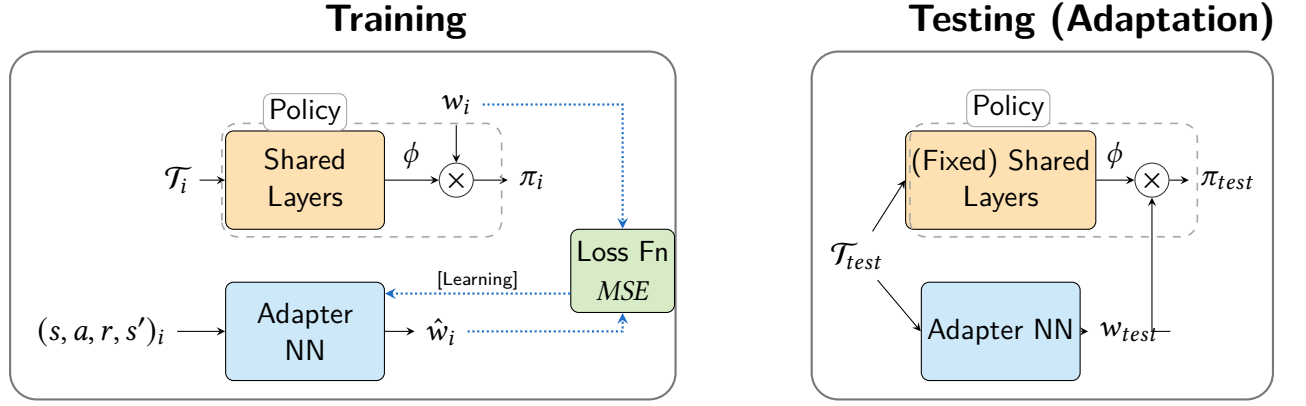
$$(s_t, \mathbf{a}_t, r_t, s_{t+1}) \longrightarrow (\hat{\mathbf{w}}_i, \hat{b}_i).$$

This Adapter Network is trained with a simple supervised regression loss, using the true, RL-optimized output-layer weights for each training task as the target,

$$L_{\text{adapter}} = \|\hat{\mathbf{w}}_i - \mathbf{w}_i\|^2.$$

Figure A.35 shows this training arrangement: the shared layers produce  $\phi$  from task  $T_i$ , combined with the true weights  $\mathbf{w}_i$  to produce the policy  $\pi_i$ ; separately, the Adapter NN takes a transition  $(s, \mathbf{a}, r, s')$  and predicts  $\hat{\mathbf{w}}_i$ ; both  $\mathbf{w}_i$  and  $\hat{\mathbf{w}}_i$  feed into the MSE loss function, whose gradient updates the Adapter NN, shown by the dotted learning arrow, while the shared layers and true weights are not themselves altered

by this loss. In effect, the Adapter Network is trained to distill what gradient-based optimization would have converged to, given enough data and enough steps, into a single forward pass computed from only a handful of transitions.



**Figure A.35.** – FLAP training and testing. Left (training): the shared layers produce  $\phi$  from task  $T_i$ , combined with the true, RL-optimized weights  $w_i$  to form the policy  $\pi_i$ . The Adapter NN separately predicts  $\hat{w}_i$  from a transition  $(s, a, r, s')_i$ , and both  $w_i$  and  $\hat{w}_i$  feed an MSE loss whose gradient trains only the Adapter NN. Right (testing): the shared layers are frozen; the Adapter NN predicts  $w_{\text{test}}$  directly from a transition observed on the test task  $T_{\text{test}}$ , combined with the frozen  $\phi$  to produce  $\pi_{\text{test}}$ , with no further training. Adapted from Peng et al. [106].

**Adaptation at test time.** Once the Adapter Network is trained, adaptation to a new task requires no gradient descent whatsoever,

$$(w, b) = \text{Adapter}(s, a, r, s'),$$

as shown on the right of Figure A.35: the shared layers are frozen entirely, and the Adapter NN produces  $w_{\text{test}}$  directly from a single observed transition on the test task, which is then combined with the fixed  $\phi$  to produce the adapted policy  $\pi_{\text{test}}$ . This is a more direct form of parameter-generation adaptation than PD-VF in Section 5.9.5.1: where PD-VF regresses to a policy embedding that must then be located via a closed-form optimization,  $\max_{z_\pi} z_\pi^\top A z_\pi$ , before being decoded into actions, FLAP’s adapter regresses straight to the usable output-layer weights themselves, with no intermediate search step at all. It is also the clearest instance in this thesis of the adaptation-via-parameters mechanism introduced in Section 5.7.1.1 as a contrast to RL<sup>2</sup>’s adaptation-via-input: Hypernetwork Meta-RL generates an entire

policy network from a task embedding via an end-to-end RL objective, while FLAP generates only a final linear layer, but does so through direct supervised distillation against ground-truth per-task weights, rather than through gradients flowing back from a downstream RL loss.

**Meta-training and meta-testing.** Algorithm 69 summarizes meta-training. For each training task, transitions are collected using the current policy and stored in a per-task replay buffer. Training then proceeds, for each task, by sampling a mini-batch, computing the actor and critic losses on the shared and task-specific policy parameters, and separately computing the adapter loss by flattening the true task-specific output-layer weights into a target vector and comparing it against the adapter’s prediction from the same mini-batch’s transitions. The critic, actor, and adapter parameters are each updated by their own gradient, independently. Algorithm 70 summarizes meta-testing: the shared layers are frozen, the output layer is initialized arbitrarily, for instance from any training task, and at every timestep the adapter predicts a new output layer directly from the most recently observed transition, replacing the policy’s output layer immediately, with no accumulation of gradient information required.

---

**Algorithm 69** FLAP Meta-Training

---

**Require:** Training task set  $\{T_i\}_{i=1}^N \sim \rho(T_{\text{train}})$

**for all** training tasks  $T_i$  **do**

Initialize replay buffer  $B_i$

**end for**

**while** training not finished **do**

**for all** training tasks  $T_i$  **do**

Collect transitions using policy  $\pi_\theta$ ; store in replay buffer  $B_i$

**end for**

**for**  $k = 1$  to training steps **do**

**for all** training tasks  $T_i$  **do**

Sample mini-batch  $b_i \sim B_i$

Compute actor loss  $L_{\text{actor}}^i(b_i)$  and critic loss  $L_{\text{critic}}^i(b_i)$

$\text{target}_i = \text{Flatten}(w_i, b_i)$

$(\hat{w}_i, \hat{b}_i) = \varphi(s, a, r, s')$ ; compute  $L_{\text{adapter}}^i = \|(\hat{w}_i, \hat{b}_i) - (w_i, b_i)\|^2$

**end for**

$\psi_Q \leftarrow \psi_Q - \alpha_Q \nabla_{\psi_Q} \sum_i L_{\text{critic}}^i$

$\theta_\pi \leftarrow \theta_\pi - \alpha_\pi \nabla_{\theta_\pi} \sum_i L_{\text{actor}}^i$

$\varphi \leftarrow \varphi - \alpha_\varphi \nabla_\varphi \sum_i L_{\text{adapter}}^i$

**end for**

**end while**

---

---

**Algorithm 70** FLAP Meta-Testing (Adaptation)

---

**Require:** Test task  $T \sim \rho(T_{\text{test}})$

Load shared policy parameters  $\theta$ ; freeze the shared layers

Load adapter network  $\varphi$

Initialize the output layer arbitrarily, e.g. from any training task

**for**  $t = 1$  to  $T$  **do**

    Sample action  $a_t \sim \pi_\theta(a_t | s_t)$

    Observe  $s_{t+1} \sim \rho(s_{t+1} | s_t, a_t)$ ; receive reward  $r_t = r(s_t, a_t)$

$(W, b) = \varphi(s_t, a_t, s_{t+1}, r_t)$

    Replace the output layer parameters with  $(W, b)$

**end for**

---

**Synthesis of Section 5.9.5.** PD-VF and FLAP both dispense with belief representation entirely and regress directly to what adaptation actually needs, but at different levels of directness. PD-VF regresses to a policy embedding  $z_\pi$  that must still be located by solving a closed-form optimization problem,  $\max_{z_\pi} z_\pi^\top A(s_0, z_d) z_\pi$ , using a separately learned quadratic value function as the objective, and only then decoded into actions; its two encoders, trained by plain MSE reconstruction with no variational structure, exist to build the space that this optimization searches over. FLAP dispenses with the search step entirely: its Adapter Network is trained by direct supervised regression against ground-truth, RL-optimized task weights, so that adaptation is a single forward pass producing immediately usable policy parameters, with no embedding, no value function, and no optimization problem to solve at test time at all. Both methods share the structural assumption that some low-dimensional, task-varying quantity, an embedding for PD-VF, a linear output layer for FLAP, is sufficient to capture everything adaptation requires, an assumption considerably stronger than anything required by the belief-based or discriminative methods of Sections 5.9.2 through 5.9.4, but one that, when it holds, yields adaptation mechanisms faster and structurally simpler than any other method surveyed in this section.

Section 5.9.6 turns to a final representation mechanism, BRUNO, which grounds task inference not in a learned embedding of any kind, but in the formal statistical theory of exchangeable stochastic processes.

### A.3.4. Exchangeable Models

Every context-based method surveyed in Section 5.9.2 must confront the same design question already flagged in PEARL (Section 5.9.2.2) and ELUE (Section 5.9.2.3): a context of collected transitions is a set, not a sequence, so processing it with a

recurrent network, as VariBAD does in Section 5.9.2.1, imposes an ordering bias that has no principled justification for a stationary task. PEARL and ELUE address this architecturally, through a product-of-Gaussians factorization and a Deep Sets construction respectively, both permutation-invariant by construction but justified primarily by the fact that they work. This section presents a method that addresses the same problem from a different direction entirely, by grounding permutation invariance in a formal statistical property of the data itself, rather than in an architectural choice.

#### A.3.4.1. BRUNO: Exchangeable Models in Meta-Reinforcement Learning

Korshunova et al. [107] ask whether the recurrent networks used throughout this chapter for task inference can be replaced with a model that is mathematically principled by construction, rather than merely convenient. Their observation is direct: for a stationary task, the order in which transitions were collected carries no information, only the transitions themselves do, so a recurrent encoder that processes context sequentially is imposing structure the underlying problem does not have. Their proposal is to replace  $\text{RNN}(\tau)$  with  $\text{BRUNO}(\tau)$ , a model that is exchangeable by construction rather than merely approximately invariant to permutation.

**Exchangeability.** A sequence of random variables  $x_1, \dots, x_n$  is exchangeable if its joint distribution is invariant under any permutation of its indices,

$$p(x_1, \dots, x_n) = p(x_{\pi(1)}, \dots, x_{\pi(n)}),$$

for any permutation  $\pi$ . The simplest nontrivial example of an exchangeable process is a jointly Gaussian one with constant variance and constant pairwise correlation,  $p(x_1, \dots, x_n) = \mathcal{N}_n(0, \Sigma)$  with  $\Sigma_{ii} = 1$  and  $\Sigma_{ij, i \neq j} = \rho$  for  $0 \leq \rho < 1$ : since every pair of variables shares the same correlation  $\rho$  regardless of their positions in the sequence, permuting the indices leaves the joint distribution unchanged. This simple example is not incidental; it is the exact structural template BRUNO’s Gaussian Process assumption reuses below.

**de Finetti’s theorem.** The theoretical foundation for the entire method is de Finetti’s representation theorem: every exchangeable process can be represented as conditionally independent and identically distributed, given a latent variable,

$$p(x_1, \dots, x_n) = \int \prod_{i=1}^n p(x_i \mid \theta) p(\theta) d\theta.$$

This is a substantively different justification for introducing a latent variable than any other method in this chapter offers. VariBAD, PEARL, and ELUE all introduce a latent task variable as an architectural bottleneck, motivated by the intuition that task-relevant information should compress into a low-dimensional code, but nothing in their construction guarantees that such a variable exists or that conditioning on it renders the transitions independent. De Finetti’s theorem provides exactly this guarantee for any process already known to be exchangeable: the existence of a latent variable  $\theta$  rendering the sequence conditionally i.i.d. is not an architectural choice, it is a theorem. Under this representation, the predictive distribution of a new observation given past ones follows directly from Bayesian updating,

$$p(x_n | x_{1:n-1}) = \int p(x_n | \theta) p(\theta | x_{1:n-1}) d\theta.$$

Applied to reinforcement learning, treating a full trajectory of transitions as exchangeable and  $\theta$  as the unknown task identity, this becomes

$$p(r, s' | s, a, \tau) = \int p(r', s' | s, a, \theta) p(\theta | \tau) d\theta,$$

directly recasting task inference as Bayesian model averaging over the task variable  $\theta$  given the trajectory  $\tau$  observed so far, exactly the same belief-based formulation introduced at the start of Section 5.9, but now with a formal argument for why maintaining a posterior over  $\theta$  is the right thing to do, rather than only a useful one.

**The BRUNO model.** BRUNO models a conditional exchangeable process  $p(y | x)$ , where  $x$  is a conditioning input and  $y$  is the output to be predicted. In the reinforcement learning setting,  $x$  is the current state-action pair and  $y$  is the resulting reward and next state, so that BRUNO directly models the transition distribution  $p(r, s' | s, a, \tau)$  conditioned on the context accumulated so far. The three main contributions of this construction are using an exchangeable model,  $\text{BRUNO}(\tau)$ , in place of a recurrent network,  $\text{RNN}(\tau)$ , for task inference; deriving a closed-form posterior  $p(\theta | \tau)$  from the resulting predictive distribution  $p(y_{n+1} | y_n)$ ; and integrating the resulting belief with Soft Actor-Critic for policy learning.

**Handling complex outputs with normalizing flows.** Rewards and next states are generally far from Gaussian, which would make the exchangeable Gaussian construction above inapplicable if applied directly to raw  $(r, s')$  values. BRUNO addresses this with a normalizing flow: rather than modeling  $y = (r, s')$  directly, an invertible

transformation  $f$  maps it into a latent space,  $\mathbf{z} = f(\mathbf{y})$ , chosen so that in this transformed space, a simple Gaussian Process assumption becomes tractable, exactly as in the exchangeability example above, with the latents assumed jointly Gaussian,

$$\mathbf{z}_1, \dots, \mathbf{z}_n \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad \Sigma_{ii} = v, \quad \Sigma_{ij, i \neq j} = \rho.$$

Because  $f$  is invertible, working with the latent  $\mathbf{z}$  loses no information relative to working with  $\mathbf{y}$  directly, while gaining the ability to apply the closed-form Gaussian machinery below.

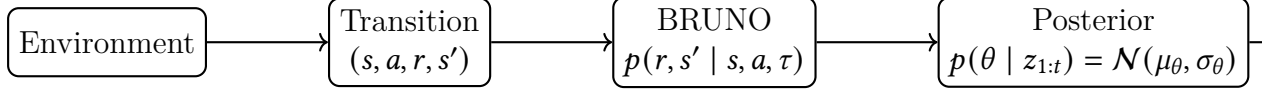
**Closed-form posterior.** This is the central technical payoff of the exchangeable construction. Under the Gaussian Process assumption above, the posterior over the task variable given the latents observed so far is itself Gaussian,  $p(\theta \mid \mathbf{z}_{1:t}) = \mathcal{N}(\mu_t, \sigma_t^2)$ , and can be updated recursively in closed form as each new observation arrives,

$$\mu_{t+1} = (1 - b_t)\mu_t + b_t \mathbf{z}_t, \quad \sigma_{t+1}^2 = (1 - b_t)(\sigma_t^2 - v + \rho), \quad b_t = \frac{\rho}{v + \rho(t - 1)}.$$

This is an exact Bayesian conjugate update, not an approximation. Every variational method surveyed in Section 5.9.2, VariBAD, PEARL, and ELUE, must approximate an intractable posterior using a learned inference network trained to maximize an ELBO, since the true posterior under their assumed generative model has no closed form. Here, because the process is constructed to be exchangeable with a Gaussian latent structure, the posterior update is exact and requires no learned inference network at all:  $\mu_t$  and  $\sigma_t^2$  are computed directly from the observed latents via the recursion above, with  $b_t$  playing the role of a precision-weighted learning rate that shrinks as more observations accumulate, analogous in spirit to the precision-weighted fusion PEARL uses in Section 5.9.2.2, but derived here from the exchangeability assumption rather than from an independently motivated Gaussian factorization.

**Architecture and training.** Figure A.36 describes the resulting pipeline conceptually: the environment produces a transition  $(s, \mathbf{a}, r, s')$ , which BRUNO models as  $p(r, s' \mid s, \mathbf{a}, \tau)$ ; this feeds the closed-form posterior update,  $p(\theta \mid \mathbf{z}_{1:t}) = \mathcal{N}(\mu_\theta, \sigma_\theta)$ , and the resulting belief conditions a Soft Actor-Critic policy,  $\pi(\mathbf{a} \mid s, \theta)$ . BRUNO is trained by maximizing the log-likelihood it assigns to observed transitions, equivalently minimizing the negative log-likelihood loss,

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log p(r_t, s'_t \mid s_t, a_t, \tau_{1:k}).$$



**Figure A.36.** – BRUNO pipeline. Transitions from the environment are modeled by BRUNO as  $p(r, s' \mid s, a, \tau)$ , which feeds the closed-form exchangeable posterior update over  $\theta$ , and the resulting belief conditions a Soft Actor-Critic policy.

**Posterior inference algorithm.** Algorithm 71 summarizes how the posterior is computed from a set of transitions. Each transition is split into an input  $x_i = (s_i, a_i)$  and an output  $y_i = (r_i, s'_i)$ , the output is passed through the invertible flow to obtain its latent representation  $z_i$ , and the posterior mean and variance are updated recursively according to the closed-form rule above, starting from the prior  $\mu_0 = 0$ ,  $\sigma_0^2 = \rho$ .

---

**Algorithm 71** BRUNO Posterior Inference

---

**Require:** Transition set  $\tau = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^T$ , invertible flow  $f_\phi$ , GP parameters  $v, \rho$

**Ensure:** Posterior  $p(\theta \mid \tau)$

  Compute inputs  $x_i = (s_i, a_i)$  and outputs  $y_i = (r_i, s'_i)$

**for**  $i = 1$  to  $T$  **do**

    Compute latent representation  $z_i = f_\phi(y_i, x_i)$

**end for**

  Initialize  $\mu_0 = 0$ ,  $\sigma_0^2 = \rho$

**for**  $t = 0$  to  $T - 1$  **do**

    Compute  $b_t = \frac{\rho}{v + \rho(t - 1)}$

    Update  $\mu_{t+1} = (1 - b_t)\mu_t + b_t z_t$

    Update  $\sigma_{t+1}^2 = (1 - b_t)(\sigma_t^2 - v + \rho)$

**end for**

**return**  $p(\theta \mid \tau) = \mathcal{N}(\mu_T, \sigma_T^2)$

---

**BrunoSAC: full meta-training loop.** Algorithm 72 embeds this posterior update directly into an SAC training loop. At every timestep, the current posterior mean and variance condition the policy, an action is taken and a transition observed, the transition is stored in a shared replay buffer, its latent representation is computed via



the flow, and the posterior is updated according to the same closed-form recursion. Periodically, BRUNO’s flow parameters  $\phi$  are updated by minimizing the negative log-likelihood loss above on batches of transition sequences grouped by task, while the actor and critic are updated using the standard SAC objectives on transitions sampled from the buffer.

---

**Algorithm 72** BrunoSAC

---

**Require:** Training tasks  $\{\mathcal{T}_i\}_{i=1}^N$ , BRUNO parameters  $\phi$ , actor parameters  $\psi$ , critic parameters  $\omega$ , replay buffer  $\mathcal{D}$

**while** not converged **do**

Sample a task  $\mathcal{T}$ ; initialize posterior  $\mu_0 = 0$ ,  $\sigma_0^2 = \rho$

**for**  $t = 1$  to  $H$  **do**

Observe state  $s_t$ ; sample action  $a_t \sim \pi_\psi(a \mid s_t, \mu_{t-1}, \sigma_{t-1}^2)$

Execute  $a_t$ ; observe reward  $r_t$  and next state  $s_{t+1}$

Store  $(s_t, a_t, r_t, s_{t+1}, \mathcal{T})$  in  $\mathcal{D}$

Compute  $x_t = (s_t, a_t)$ ,  $y_t = (r_t, s_{t+1})$ ,  $z_t = f_\phi(y_t, x_t)$

Compute  $b_t = \frac{\rho}{v + \rho(t-1)}$

Update  $\mu_t = (1 - b_t)\mu_{t-1} + b_t z_t$

Update  $\sigma_t^2 = (1 - b_t)(\sigma_{t-1}^2 - v + \rho)$

**end for**

Sample a batch of transition sequences from  $\mathcal{D}$  grouped by task

Update BRUNO by minimizing  $L = -\frac{1}{T} \sum_{t=1}^T \log p(r_t, s_{t+1} \mid s_t, a_t, \tau_{1:k})$

Sample a minibatch from  $\mathcal{D}$

Update critic parameters  $\omega$  using the SAC critic objective

Update actor parameters  $\psi$  using the SAC policy objective

Update target critics

**end while**

**return** Actor  $\pi_\psi$  and BRUNO model

---

**Relation to earlier methods.** BRUNO is the theoretical culmination of the permutation-invariance thread running through Section 5.9.2: PEARL’s product-of-Gaussians encoder and ELUE’s Deep Sets encoder both achieve permutation invariance by architectural construction, chosen because it works, while BRUNO achieves the same practical property, together with a closed-form, exact posterior update rather than a variationally approximated one, by starting from the formal assumption that the transition process is exchangeable and invoking de Finetti’s theorem to justify the existence and structure of a latent task variable in the first place. This closed-form exactness is BRUNO’s main advantage over every belief-based method in Section 5.9.2, none of which can guarantee their approximate posterior is correct, only

that it is optimized to be close to correct under an ELBO. The trade-off is the exchangeability assumption itself: it holds cleanly for stationary tasks where transition order truly carries no information, but is a poorer fit for the non-stationary, within-episode setting ESCP addresses in Section 5.9.4.2, where the order and recency of transitions is precisely the information that matters.

Section 5.9.7 turns from representation mechanisms entirely to a different question: given any of the belief or embedding representations surveyed across Sections 5.9.2 through 5.9.6, how can a practitioner verify what information that representation actually encodes, independently of whether it happens to support good task performance.

### A.3.5. Validating Learned Belief Representations

Every method surveyed in Sections 5.9.2 through 5.9.6 produces some internal representation, a posterior mean, a contrastive embedding, a deterministic latent, a Gaussian process belief, and argues, implicitly or explicitly, that this representation captures what matters about the task. None of these methods, on their own, provide direct evidence for that claim beyond downstream task performance: a representation might support a good policy simply because the policy has learned to work around its deficiencies, not because the representation itself is accurate or complete. This subsection covers a methodology, rather than a competing meta-RL method, for directly answering what a learned representation actually contains.

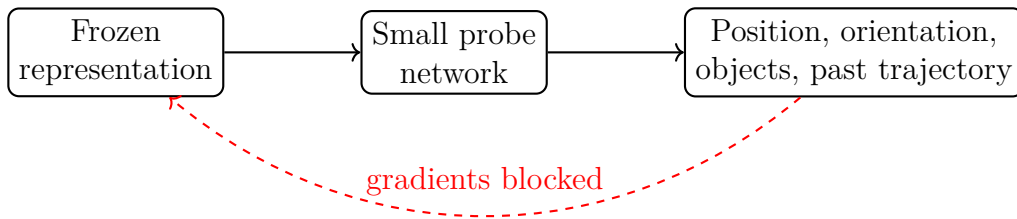
#### A.3.5.1. Neural Predictive Belief Representations

Guo et al. [108] ask whether an agent can learn an internal belief state from raw observations without any supervision telling it what to represent. More specifically: can a network trained only to predict future observations spontaneously learn something resembling the Bayesian belief state of a POMDP, an implicit representation of position, orientation, and uncertainty, without ever being given labels for any of these quantities?

**Learned belief in place of exact filtering.** As throughout this section, exact Bayesian filtering,  $b_t = P(x_t \mid h_t)$ , is intractable in general. This work replaces it with a belief computed by a neural network trained purely from a self-supervised prediction objective,  $b_t = f_\theta(h_t)$ , with no explicit generative model of reward or dynamics fit alongside it, as VariBAD does in Section 5.9.2.1, and no probabilistic posterior maintained in closed form, as BRUNO does in Section 5.9.6. Prior work typically justifies such a representation indirectly, by observing that it happens to

support high reward, and inferring from that success that the representation must be capturing something real. This is, in effect, a direct empirical test of the claim underlying the Approximate Information State framework of Section 5.9.1: that a representation trained only to be predictive, without an explicit correctness guarantee, can nonetheless end up encoding the information a well-behaved belief state should contain. Where AIS proves this must be true whenever the representation satisfies AP1 and AP2, this work asks the empirical question directly, and independently of task reward: for a representation trained by prediction alone, does it actually happen?

**Glass-box evaluation.** The paper’s core methodological contribution is a way of answering this question for any learned representation, not only its own, shown schematically in Figure A.37. The procedure is simple but decisive: train the representation using whatever self-supervised objective is under study, then freeze it entirely, and train a small probe network on top of the frozen representation to predict quantities the representation was never explicitly asked to encode, position, orientation, the location of objects in the scene, or aspects of the past trajectory. Gradients from the probe’s training loss are never allowed to flow back into the representation itself. If the probe can accurately recover these quantities from an otherwise frozen representation, that success is evidence the information was already present in the representation before the probe was ever trained, since the probe had no way to inject that information itself, it could only extract what was already there. This gives a direct, quantitative answer to a question that downstream task performance alone cannot answer: not merely whether the representation is useful for the policy, but specifically what it contains.



**Figure A.37.** – Glass-box evaluation. The learned representation is frozen; only the probe network is trained to predict position, orientation, object locations, and past trajectory from it. No gradient from the probe’s loss reaches the representation, so probe accuracy reflects only what the representation already encoded.

**Three self-supervised training objectives.** The paper considers three increasingly demanding self-supervised objectives for training the belief representation itself, presented here in order of increasing sophistication, and shown architecturally in Figure A.38.

Frame prediction, shown in Figure A.38(b), is the most direct approach: an observation  $o_t$  is encoded by a convolutional network into a feature  $z_t$ , combined with the previous belief and action through a GRU to produce the belief  $b_t$ , and a deconvolutional decoder predicts the next raw observation directly from  $b_t$  and the current action  $a_t$ ,  $\hat{o}_{t+1} = \text{DECONV}(b_t, a_t)$ . This requires the belief to carry enough information to reconstruct a full future observation, but reconstruction in raw observation space can force capacity toward irrelevant visual detail rather than toward the abstract, task-relevant structure a belief state should prioritize.

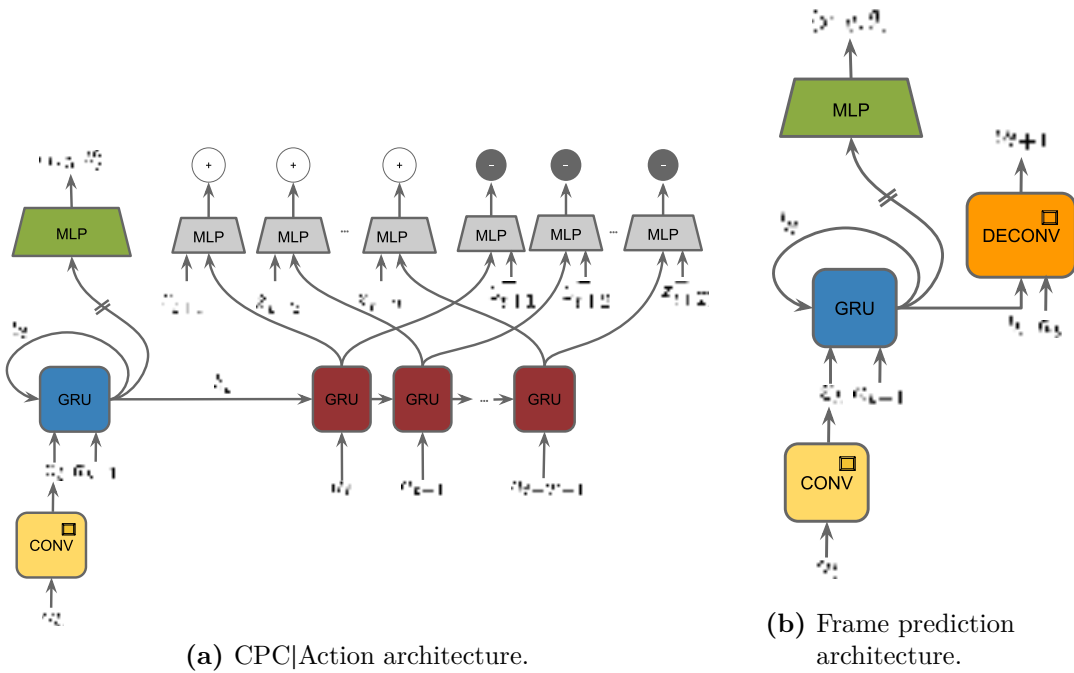
Contrastive Predictive Coding (CPC), already encountered in Section 5.7.3.2 as MRA’s auxiliary objective, reframes the prediction problem as classification rather than reconstruction. Instead of asking what the future observation is, the model is asked whether a given candidate observation is the true future observation drawn from the current trajectory, or a random one drawn from elsewhere, a classifier  $f(b_t, o_{t+k})$  trained to discriminate between the two. This side-steps the need to reconstruct irrelevant visual detail entirely, since the classifier only needs enough information to tell true futures from false ones, not to generate a future observation pixel by pixel. This objective has a precise information-theoretic justification: maximizing the binary classification objective is provably equivalent to maximizing the Jensen-Shannon divergence between the distribution of true future observations and the distribution of randomly drawn negatives,

$$\max_f J(f) = 2D_{\text{JS}}(\rho^+ \parallel \rho^-) - \log 4,$$

where  $\rho^+$  is the distribution of true future observations and  $\rho^-$  is the distribution of random, negative observations. This gives CPC a rigorous grounding as a divergence-maximizing objective, rather than an ad hoc heuristic, and is the same theoretical basis underlying the CPC auxiliary loss used by MRA in Section 5.7.3.2 and, in spirit, the world model reconstruction objective used by BIMRL.

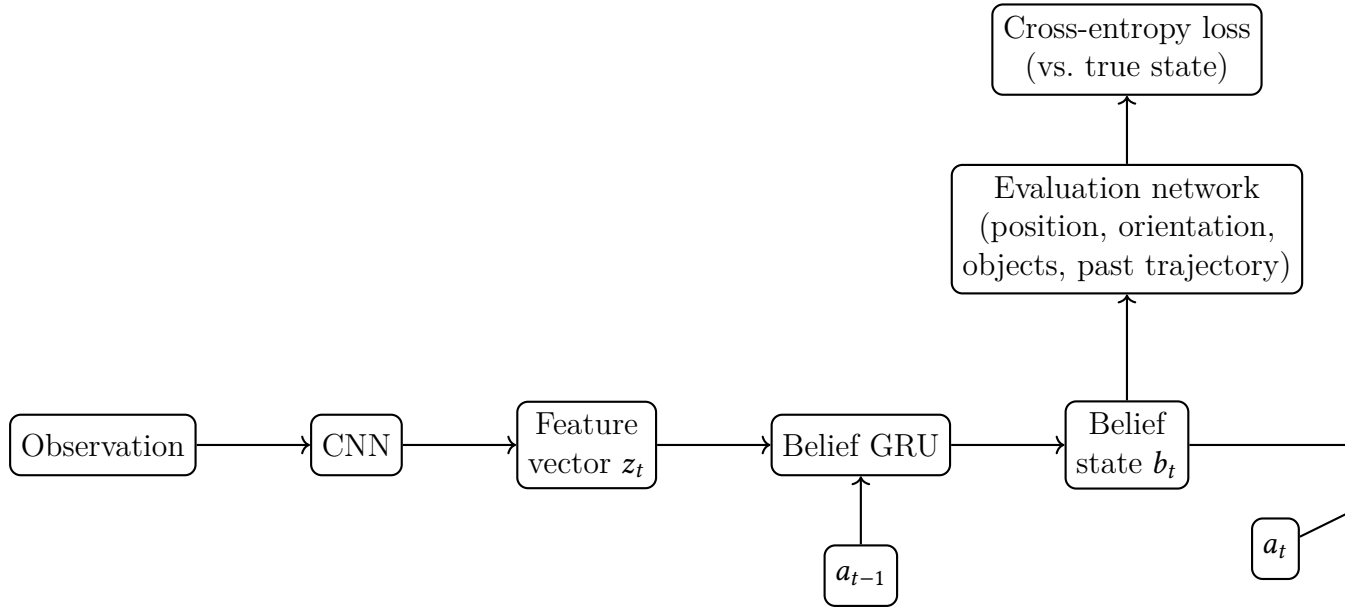
CPC|Action, shown in Figure A.38(a), extends this once more, conditioning the prediction not only on the belief but on a sequence of future actions as well, rather than assuming the agent will act in whatever way it happened to during data collection. As shown in the figure, the initial belief  $b_t$  is rolled forward through a chain of additional GRU steps driven by the true future actions  $a_t, a_{t+1}, \dots, a_{t+T-1}$ , producing action-conditioned beliefs that are compared, via small MLPs, against true

future embeddings  $z_{t+1}^+, \dots, z_{t+T}^+$  and against negative, randomly sampled embeddings  $z_{t+1}^-, \dots, z_{t+T}^-$ , with the MLP outputs trained to discriminate the two, shown as the  $\oplus$  and  $\ominus$  symbols at the top of the figure. This conditioning on future actions matters because a belief state intended to support planning must be able to answer counterfactual questions, what would I observe if I took this particular sequence of actions from here, not only passively predict what will happen given the actions actually taken.



**Figure A.38.** – Two self-supervised architectures for training the belief state  $b_t$ . (a) CPC|Action: an initial belief  $b_t$ , formed from a CNN-encoded observation  $z_t$  and the previous action  $a_{t-1}$ , is rolled forward through future actions  $a_t, \dots, a_{t+T-1}$  to produce action-conditioned beliefs, compared against true future embeddings  $z^+$  and random negative embeddings  $z^-$  via small MLP classifiers ( $\oplus$  /  $\ominus$ ). An evaluation MLP separately probes  $b_t$  for  $(x, y, \theta)$ , with gradients blocked (shown by the crossed arrow) from flowing back into the belief. (b) Frame prediction: the belief  $b_t$  and current action  $a_t$  feed a deconvolutional decoder that reconstructs the next raw observation  $o_{t+1}$  directly, with the same blocked-gradient evaluation probe shown above. Adapted from Guo et al. [108].

**Architecture and information flow.** Figure A.39 summarizes the overall information flow shared by both variants. Raw observations are passed through a CNN to produce a feature vector  $z_t$ , which, together with the previous action  $a_{t-1}$ , is fed into a recurrent belief network (a GRU) to produce the belief state  $b_t$ . This belief state feeds two downstream components: a future prediction model, trained with either the CPC or frame-prediction loss described above, together with the current or future action  $a_t$ ; and an evaluation network, trained separately with gradients blocked from flowing back into  $b_t$ , used to test whether position, orientation, object information, or past trajectory can be recovered from the frozen belief via cross-entropy loss against the true state, exactly the glass-box procedure of Figure A.37 applied concretely to this architecture’s own belief.



**Figure A.39.** – Overall information flow. A CNN encodes each observation into  $z_t$ , which the belief GRU combines with the previous action  $a_{t-1}$  to produce the belief state  $b_t$ . The belief drives a future prediction model trained with a CPC or frame-prediction loss, conditioned on the current or future action, while a separately trained evaluation network probes the frozen  $b_t$  against the true position, orientation, object locations, and past trajectory, without updating  $b_t$  itself.

**CPC|Action training.** Algorithm 73 summarizes training for the strongest of the three variants, matching the architecture in Figure A.38(a). For each sequence in

a sampled mini-batch, beliefs are computed recurrently across the trajectory. At each timestep  $t$ , a random future prediction horizon  $f_{\text{pred}}$  is sampled, the true future actions and the true future observation embedding at that horizon are extracted, and a negative example is sampled uniformly from elsewhere in the batch. The belief is advanced forward using the true future actions to produce an action-conditioned belief  $b^a$ , and a binary classifier  $h$  is trained to assign high probability to the true future embedding, given  $b^a$ , and low probability to the negative one, via sigmoid cross-entropy on each.

---

**Algorithm 73** CPC|Action Training

---

```

procedure TrainCPCAction( $f, F, g, h$ )
  while not converged do
     $\ell \leftarrow 0$ 
    Sample mini-batch  $B$  of size  $N$  from replay buffer
    Compute beliefs  $b_t = f(b_0, z_{1:t}, a_{1:t-1})$ 
    for  $j = 1$  to  $N$  do
      for  $t = 0$  to  $T$  do
        Sample  $f_{\text{pred}} \sim \text{Uniform}(1, F)$ 
         $a_{t:t+f_{\text{pred}}-1} \leftarrow$  future actions from time  $t$ 
         $z_{t+f_{\text{pred}}} \leftarrow$  future observation embedding
        Sample negative example  $z^-$  uniformly from  $B$ 
         $b^a = g(b_t, a_{t:t+f_{\text{pred}}-1})$ 
         $\ell^+ = \text{sigmoid\_cross\_entropy}(h(b^a, z_{t+f_{\text{pred}}}), 1)$ 
         $\ell^- = \text{sigmoid\_cross\_entropy}(h(b^a, z^-), 0)$ 
         $\ell \leftarrow \ell + \ell^+ + \ell^-$ 
      end for
    end for
     $\ell \leftarrow \ell / |B|$ 
    Update parameters of  $f$ ,  $g$ , and  $h$  by minimizing  $\ell$ 
  end while
end procedure

```

---

**Relation to the rest of Section 5.9.** The glass-box evaluation methodology introduced here is not tied to this paper’s own belief GRU; it is a general-purpose diagnostic applicable to any of the representations built throughout this section. VariBAD’s task variable  $m$  in Section 5.9.2.1, PEARL’s context variable  $z$  in Section 5.9.2.2, CCM’s contrastive embedding in Section 5.9.4.1, ESCP’s deterministic latent in Section 5.9.4.2, or BRUNO’s task variable  $\theta$  in Section 5.9.6 could each, in principle, be frozen and probed in exactly the way shown in Figure A.37, to determine directly what information each actually retains, independent of whatever

downstream task performance each achieves. Where the rest of Section 5.9 answers the question of how to build a task representation, this subsection answers a complementary question that applies to every one of those constructions: how to verify, after the fact, what such a representation actually learned to encode.

Section 5.9.8 closes this section with a synthesis across all of the mechanisms surveyed, from the theoretical grounding of Section 5.9.1 through the evaluation methodology introduced here, and situates task inference as a whole relative to the black-box methods of Section 5.7 and the policy parameter gradient methods of Section 5.6.

### **A.3.6. Synthesis and Comparison**

Section 5.9 has surveyed a substantially larger and more heterogeneous family of methods than either Section 5.6 or Section 5.7, unified only by the commitment stated in the section overview: task identity is made an explicit object of computation, rather than left implicit in an adapted parameter vector, as in Section 5.6, or in an unconstrained hidden state, as in Section 5.7. This closing subsection draws together the mechanisms surveyed across Sections 5.9.1 through 5.9.7, revisits the two design axes introduced at the start of the section, and situates task inference as a whole against the other two families in the taxonomy of Section 5.5.

**Revisiting the two design axes.** The section overview identified two largely independent questions that every method in this family answers, sometimes implicitly. The first is mechanistic: how is a representation of task identity actually computed from experience. The second concerns exploration: whether the same policy both explores and exploits, or whether these are decoupled into separately trained objectives. Table A.20 places every method surveyed in this section along both axes.



**Table A.20.** – Every Section 5.9 method placed along the two design axes introduced in the section overview.

| Method                  | Representation mechanism                                | Exploration             |
|-------------------------|---------------------------------------------------------|-------------------------|
| VariBAD (5.9.2.1)       | Variational posterior, ELBO with reconstruction         | Coupled (single policy) |
| PEARL (5.9.2.2)         | Variational posterior, product-of-Gaussians, no decoder | Coupled (single policy) |
| ELUE (5.9.2.3)          | Variational posterior, Deep Sets, off-policy embedding  | Coupled (single policy) |
| DREAM (5.9.3.1)         | Variational encoder over true task ID $\mu$             | Decoupled               |
| MetaCURE (5.9.3.2)      | Variational posterior (PEARL-style)                     | Decoupled               |
| CCM (5.9.3.3 / 5.9.4.1) | Contrastive (discriminative)                            | Decoupled               |
| ESCP (5.9.4.2)          | Discriminative, deterministic (variance + determinant)  | Coupled (single policy) |
| PD-VF (5.9.5.1)         | Direct regression to a value-scored embedding           | Coupled (single policy) |
| FLAP (5.9.5.2)          | Direct regression to policy parameters                  | Coupled (single policy) |
| BRUNO (5.9.6)           | Exchangeable process, closed-form Bayesian posterior    | Coupled (single policy) |

This table makes visible a pattern that individual subsections could only gesture at. The representation-mechanism axis spans a genuine spectrum of theoretical commitment, from BRUNO’s closed-form, provably exact posterior, through the approximate variational posteriors of Section 5.9.2, to the purely discriminative and directly regressive mechanisms of Sections 5.9.4 and 5.9.5, which offer no probabilistic interpretation at all. The exploration axis, by contrast, is dominated by coupled, single-policy designs; only DREAM, MetaCURE, and CCM commit to the decoupled alternative, and, as Section 5.9.3 established, this decoupling is a genuinely separable design choice that could in principle be combined with almost any representation mechanism in the left column, not only the ones it happens to have been paired with historically.

**What AIS and the glass-box methodology contribute to this table.** Two entries surveyed in this section, AIS (Section 5.9.1) and Neural Predictive Belief Representations (Section 5.9.7), do not appear in Table A.20 because they are not themselves representation mechanisms. AIS supplies the theoretical condition, approximate satisfaction of AP1 and AP2, that any row of the table should be checked against to understand why a given mechanism is a reasonable one to have built at all, and its error-propagation bound applies uniformly regardless of which mechanism produced the representation. The glass-box methodology supplies the empirical counterpart: a procedure for measuring, after training, what a given row’s representation actually encodes, independent of the downstream task performance that a method’s own reported results typically rely on as indirect evidence of representation quality. Together, these two entries frame every other method in this section as an answer to the same underlying question, how much should a learned representation be trusted, one theoretically and one empirically, rather than as a competing method in their own right.

**Recurring trade-offs within the family.** Several trade-offs recur across the methods surveyed, echoing patterns already familiar from Sections 5.6 and 5.7 but taking a different form here. Variational methods (Section 5.9.2) offer an explicit, interpretable belief and a principled uncertainty representation, but require approximating an intractable posterior, and, as ELUE’s critique of PEARL in Section 5.9.2.3 showed, can silently entangle representation learning with policy optimization even when off-policy training is the explicit design goal. Discriminative methods (Section 5.9.4) sidestep the approximation problem entirely by training the representation against a purely self-supervised criterion, at the cost of losing any principled notion of uncertainty over the task, ESCP’s deterministic embedding has no distributional interpretation at all, unlike a variational posterior’s mean and variance. Direct regression methods (Section 5.9.5) push furthest in this direction, dispensing with any intermediate representation-and-condition structure and instead regressing straight to what adaptation needs, policy weights, or a value-scored embedding, achieving the fastest test-time adaptation of any method surveyed in this section, entirely gradient-free and, for FLAP, requiring only a single forward pass, but relying on a strong linear or embedding-based structural assumption that may not hold for every task distribution. BRUNO occupies a unique position, offering the theoretical guarantees of the variational family without its approximation error, at the cost of an exchangeability assumption that, as noted in Section 5.9.6, is a poor match for the non-stationary, within-episode setting that motivates ESCP.

**Task inference relative to policy parameter gradient methods (Section 5.6).** Every method in this section can be read as answering the same question Sec-

tion 5.6 answers by a different route: what should change, or be inferred, when the agent encounters a new task. Section 5.6 commits to an explicit optimization procedure, gradient descent, with only its initialization, learning rate, or curvature meta-learned, giving a well-understood inner loop and generalization behavior that, as the MAML-versus-RL<sup>2</sup> comparison in Section 5.7.5 showed, often extends better outside the training distribution than black-box alternatives. Task inference methods replace this explicit optimization procedure with an explicit representation of task identity instead, trading the guarantee of a well-defined optimization trajectory for a different guarantee, in the best case, such as BRUNO, a provably correct posterior, or, in the AIS-grounded case, a bounded approximation error, without requiring any gradient computation during interaction with the task at all. Where Section 5.6 methods adapt by moving through parameter space via gradient steps, task inference methods adapt by moving through belief or embedding space via inference, whether variational, discriminative, or closed-form, and this difference in kind, not merely in implementation detail, is what separates the two families in the taxonomy of Section 5.5.

**Task inference relative to black-box methods (Section 5.7).** The relationship to Section 5.7 is more direct, since task inference methods were explicitly motivated throughout this section as a response to the entanglement problem diagnosed in Section 5.7.1.1: a black-box hidden state must simultaneously encode task identity, exploration history, and control-relevant state, with nothing in the architecture separating these roles. Every method surveyed in Section 5.9 addresses this by dedicating a component specifically to task inference, separated at the architectural level from the policy that consumes its output. This separation is not free. Table A.20’s discriminative and direct-regression methods, in particular, sacrifice some of the flexibility that made black-box methods attractive in the first place, PD-VF and FLAP both depend on a structural assumption, a quadratic value function or a linear output layer, that a sufficiently expressive black-box model would not need to assume in advance. Yet even the most theoretically committed methods in this section, BRUNO above all, retain the core practical advantage that motivated the entire family: no gradient computation is required during interaction with a task, adaptation is achieved through inference or, in Section 5.9.5’s case, direct regression, and, for BRUNO and the variational methods of Section 5.9.2, an explicit representation of task uncertainty is available to condition on, information a black-box hidden state carries only implicitly, if it carries it at all.

**Closing remark.** Across Section 5.6, Section 5.7, and Section 5.9, this chapter has surveyed three distinct answers to the same underlying problem, adapting quickly to a new task from limited experience, each committing to a different point on the

trade-off between structure and flexibility: an explicit optimization procedure with a fixed form, an unconstrained learned representation with no explicit structure at all, and an explicit representation of task identity with a structure that ranges, within this family alone, from fully Bayesian and provably correct to purely discriminative to a direct regression target. No family dominates the other two uniformly, and the choice between them, or, as several methods in this section demonstrate, between mechanisms combined from more than one family, is the trade-off this thesis returns to directly when evaluating these approaches empirically in the chapters that follow.

---

**Algorithm 47** Recursive DiCE for Higher-Order Gradient Estimation [86]

---

**Require:** Stochastic computation graph  $\mathcal{G}$ , parameters  $\theta$ , differentiation order  $n$

```

1: function MagicBox( $\mathcal{W}$ )
2:    $S \leftarrow \sum_{w \in \mathcal{W}} \log p(w; \theta)$ 
3:   return  $\exp(S - \perp(S))$ 
4: end function
5:
6: function DiCEObjective
7:    $\mathcal{L}_{\text{DiCE}} \leftarrow 0$ 
8:   for all cost nodes  $c \in \mathcal{C}$  do
9:     Determine stochastic ancestors  $\mathcal{W}_c$ 
10:     $\mathcal{L}_{\text{DiCE}} \leftarrow \mathcal{L}_{\text{DiCE}} + \text{MagicBox}(\mathcal{W}_c) c$ 
11:   end for
12:   return  $\mathcal{L}_{\text{DiCE}}$ 
13: end function
14:
15: function HigherOrderGradient( $\mathcal{L}, n$ )
16:   if  $n = 0$  then
17:     return  $\mathcal{L}$ 
18:   else
19:      $g \leftarrow \nabla_{\theta} \mathcal{L}$ 
20:     return HigherOrderGradient( $g, n - 1$ )
21:   end if
22: end function
23:
24:  $\mathcal{L}_{\text{DiCE}} \leftarrow \text{DiCEObjective}$ 
25:
26:  $g^{(n)} \leftarrow \text{HigherOrderGradient}(\mathcal{L}_{\text{DiCE}}, n)$ 
27: return  $g^{(n)}$ 

```

---

---

**Algorithm 48** Compute the Loaded DiCE Objective [87]

---

**Require:** Trajectory  $\{(s_t, a_t)\}_{t=0}^T$ , advantage estimates  $A(s_t, a_t)$ , discount factor  $\gamma$ , trace parameter  $\lambda$

- 1:  $J^\diamond \leftarrow 0$
- 2:  $w \leftarrow 0$
- 3: **for**  $t = 0$  to  $T$  **do**
- 4:    $w \leftarrow \lambda w + \log \pi(a_t | s_t)$
- 5:    $v \leftarrow w - \log \pi(a_t | s_t)$
- 6:    $\Delta_t \leftarrow \text{MagicBox}(w) - \text{MagicBox}(v)$
- 7:    $J^\diamond \leftarrow J^\diamond + \gamma^t \Delta_t A(s_t, a_t)$
- 8: **end for**
- 9: **return**  $J^\diamond$
- 10:
- 11: **function**  $\text{MagicBox}(x)$
- 12:   **return**  $\exp(x - \perp(x))$
- 13: **end function**

---

---

**Algorithm 49** Meta-Gradient Reinforcement Learning with Sampling Correction

---

**Require:** Task distribution  $p(\mathcal{T})$ , meta-parameters  $\eta$ , initial policy  $\theta_0$ , adaptation horizon  $K$ , truncation length  $L$ , sampling coefficient  $\lambda$

- 1: **for** each meta-update **do**
- 2:     Sample a batch of tasks  $\{\mathcal{T}^b\}_{b=1}^B \sim p(\mathcal{T})$
- 3:     Initialize the meta-gradient  $g_\eta \leftarrow 0$
- 4:     **for all** tasks  $\mathcal{T}^b$  **do**
- 5:         Initialize the policy  $\theta_0^{(b)} \leftarrow \theta_0$
- 6:         **for**  $i = 0, \dots, K - 1$  **do**
- 7:             Sample trajectories  $D_i^{(b)} \sim p(D \mid \theta_i^{(b)}, \mathcal{T}^b)$
- 8:             Compute the sampled policy gradient  
 $\hat{g}_i^{(b)} = \frac{1}{|D_i^{(b)}|} \sum_{\tau \in D_i^{(b)}} \sum_t \nabla_{\theta} \log \pi(a_t \mid s_t; \theta_i^{(b)}) R(\tau)$
- 9:             Update the policy  
 $\theta_{i+1}^{(b)} = \Psi(\eta, \theta_i^{(b)}, D_i^{(b)})$
- 10:         **end for**
- 11:         Select the truncation window  $\mathcal{W} = \{K - L, \dots, K\}$
- 12:         **for all**  $k \in \mathcal{W}$  **do**
- 13:             Sample evaluation trajectories  
 $D_k^{(b)} \sim p(D \mid \theta_k^{(b)}, \mathcal{T}^b)$
- 14:             Compute the return  
 $R_k^{(b)} = \sum_{\tau \in D_k^{(b)}} R(\tau)$
- 15:             Compute the direct term  
 $g_{\text{direct}} = \nabla_{\eta} \theta_k^{(b)} \nabla_{\theta_k^{(b)}} \log p(D_k^{(b)} \mid \theta_k^{(b)}) R_k^{(b)}$
- 16:             Initialize the sampling correction  
 $g_{\text{sc}} \leftarrow 0$
- 17:             **for**  $j = k - L, \dots, k - 1$  **do**
- 18:                  $g_{\text{sc}} \leftarrow g_{\text{sc}} + \nabla_{\eta} \theta_j^{(b)} \nabla_{\theta_j^{(b)}} \log p(D_j^{(b)} \mid \theta_j^{(b)})$
- 19:             **end for**
- 20:             Accumulate the meta-gradient  
 $g_\eta \leftarrow g_\eta + \lambda g_{\text{sc}} R_k^{(b)} + g_{\text{direct}}$
- 21:         **end for**
- 22:     **end for**
- 23:     Average over the task batch  
 $g_\eta \leftarrow \frac{1}{B} g_\eta$
- 24:     Update the meta-parameters  
 $\eta \leftarrow \eta + \alpha g_\eta$
- 25: **end for**

---

---

**Algorithm 50** Off-Policy Meta-Gradient Estimation Framework [69]

---

**Require:** Behavior policy  $\mu$ , target policy  $\pi_\theta$ , adaptation operator  $U$ , off-policy value estimator  $\widehat{V}^{\pi_\theta}$

- 1: Collect trajectories  $D \sim \mu$
  - 2: Estimate the target value using an off-policy estimator  
 $\widehat{V}^{\pi_\theta}(x_0)$
  - 3: Compute the policy gradient  
 $g = \nabla_\theta \widehat{V}^{\pi_\theta}(x_0)$
  - 4: Compute the Hessian  
 $H = \nabla_\theta^2 \widehat{V}^{\pi_\theta}(x_0)$
  - 5: Perform the adaptation step  
 $\theta' = \theta + U(\theta)$
  - 6: Compute the meta-gradient  
 $\nabla_\theta F(\theta) = (I + \eta H) \nabla_{\theta'} \widehat{V}^{\pi_{\theta'}}(x_0)$
  - 7: Update the policy parameters  
 $\theta \leftarrow \theta + \alpha \nabla_\theta F(\theta)$
- 

---

**Algorithm 51** Meta-Reinforcement Learning of Structured Exploration Strategies (MAESN)

---

**Require:** Task distribution  $p(\mathcal{T})$

**Require:** Policy parameters  $\theta$

**Require:** Task-specific latent parameters  $\{(\mu_i, \sigma_i)\}$

- 1: **while** not converged **do**
  - 2:   Sample a batch of tasks  $\mathcal{T}_i \sim p(\mathcal{T})$
  - 3:   **for all** task  $\tau_i$  **do**
  - 4:     Sample latent variable  $z \sim \mathcal{N}(\mu_i, \sigma_i)$
  - 5:     Collect trajectories using  $\pi_\theta(a|s, z)$
  - 6:     Update the latent parameters  
 $\mu_i \leftarrow \mu_i + \alpha_\mu \nabla_{\mu_i} J$   
 $\sigma_i \leftarrow \sigma_i + \alpha_\sigma \nabla_{\sigma_i} J$
  - 7:     Optionally update the policy parameters  
 $\theta' = \theta + \alpha_\theta \nabla_\theta J$
  - 8:   **end for**
  - 9:   Meta-update the policy and latent parameters by maximizing the meta-objective using TRPO
  - 10: **end while**
  - 11: **return**  $\theta, \{\mu_i, \sigma_i\}$
-



---

**Algorithm 52** Exploration-MAML (E-MAML)

---

**Require:** Task distribution  $p(\mathcal{T})$

**Require:** Initial policy parameters  $\theta$

**Require:** Inner learning rate  $\alpha$

**Require:** Meta learning rate  $\beta$

- 1: **while** not converged **do**
- 2:     Sample a batch of tasks  $\mathcal{T}_i \sim p(\mathcal{T})$
- 3:     **for all** task  $\tau_i$  **do**
- 4:         Collect exploration trajectories  $\bar{\tau} \sim \pi_\theta$
- 5:         Perform the inner policy update  
 $\theta' = U(\theta, \bar{\tau})$
- 6:         Collect meta trajectories  $\tau \sim \pi_{\theta'}$
- 7:     **end for**
- 8:     Update  $\theta$  using the exploration-aware meta-gradient

$$\nabla_\theta \mathcal{L}_{\text{meta}} = \underbrace{\nabla_\theta \log \pi_{U(\theta, \bar{\tau})}(\tau)}_{\text{adaptation}} + \underbrace{\nabla_\theta \log \pi_\theta(\bar{\tau})}_{\text{exploration}}$$

9: **end while**

---



---

**Algorithm 65** CCM: Contrastive Learning Augmented Context-Based Meta-RL

---

**Require:** Training task distribution  $p(\mu)$ , context encoder  $e_\phi$ , momentum encoder  $\bar{e}_\phi$ , exploration policy  $\pi_{\theta_e}$ , execution policy  $\pi_{\theta_x}$ , exploration buffer  $\mathcal{D}_{\text{exp}}$ , execution buffer  $\mathcal{D}_{\text{exe}}$

**Ensure:** Adapted execution policy  $\pi_{\theta_x}(a | s, z)$

**for** each training iteration **do**

    Sample task  $\mu \sim p(\mu)$ ; execute  $\pi_{\theta_e}$ ; collect trajectory  $\tau_{\text{exp}}$

    Store  $\tau_{\text{exp}}$  into  $\mathcal{D}_{\text{exp}}$  and  $\mathcal{D}_{\text{exe}}$

    Sample batches  $b_n^q, b_n^k$  from task  $\mu$  and negative batches  $\{b_j\}_{j \neq n}$  from other tasks

$z_q = e_\phi(b_n^q), \quad z_{\text{pos}} = \bar{e}_\phi(b_n^k), \quad z_j = \bar{e}_\phi(b_j)$

    Compute  $L_{\text{contrastive}} = -\log \frac{\exp(f(z_q, z_{\text{pos}}))}{\sum_j \exp(f(z_q, z_j))}$ ; update  $\phi$ ; update  $\bar{\phi} \leftarrow m\bar{\phi} + (1-m)\phi$

    Encode  $c_{1:i} = e_\phi(\tau_{1:i}), c_{1:i-1} = e_\phi(\tau_{1:i-1})$

    Compute  $L_{\text{lower}}, L_{\text{upper}}$ , and  $r_{\text{aux}} = L_{\text{upper}} - L_{\text{lower}}$

    Compute  $r_e = r_{\text{env}} + \alpha r_{\text{aux}}$ ; update  $\pi_{\theta_e}$  using SAC with reward  $r_e$

    Sample  $\tau_{\text{exe}} \sim \mathcal{D}_{\text{exe}}$ ; compute  $z = e_\phi(\tau_{\text{exe}})$ ; update  $\pi_{\theta_x}$  using SAC with reward

$r_{\text{env}}$

**end for**

---

---

**Algorithm 66** Meta Test

---

sample unseen task  $\mu_{\text{new}}$   
execute  $\pi_{\theta_e}$  to collect  $\tau$   
compute  $z = e_\phi(\tau)$   
execute  $\pi_{\theta_x}(a \mid s, z)$  with no parameter updates

---

---

## B. Contrastive Log-ratio Upper Bound (CLUB)

The Mutual Information Regularization (MIR3) algorithm estimates the mutual information between the observation history and the selected action using the Contrastive Log-ratio Upper Bound (CLUB) estimator [15]. CLUB provides a differentiable upper bound on mutual information that can be optimized efficiently using stochastic gradient descent, making it suitable for deep reinforcement learning.

Given two random variables  $X$  and  $Y$ , the CLUB estimator satisfies the following upper bound:

$$I(X; Y) \leq \underbrace{\mathbb{E}_{p(x,y)} [\log q_\phi(y|x)]}_{\text{Positive pairs}} - \underbrace{\mathbb{E}_{p(x)p(y)} [\log q_\phi(y|x)]}_{\text{Negative pairs}}, \quad (\text{B.1})$$

where  $q_\phi(y|x)$  is a neural network parameterized by  $\phi$  that approximates the conditional distribution  $p(y|x)$ .

The first expectation is computed over *positive pairs*, where  $(x, y)$  are sampled from the joint distribution  $p(x, y)$ . These pairs correspond to observations and actions that naturally occur together in the collected trajectories. The second expectation is computed over *negative pairs*, where  $x$  and  $y$  are sampled independently from the marginal distributions  $p(x)$  and  $p(y)$ . These mismatched pairs estimate how likely unrelated samples are under the learned conditional distribution.

In practice, CLUB is estimated using minibatches of training samples. Given a minibatch containing  $N$  paired samples  $\{(x_i, y_i)\}_{i=1}^N$ , the empirical estimator is

$$\hat{I}_{\text{CLUB}} = \frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N [\log q_\phi(y_i|x_i) - \log q_\phi(y_j|x_i)]. \quad (\text{B.2})$$

The first term measures the compatibility between matching observation-action pairs, while the second term evaluates mismatched pairs obtained by pairing each observation with the actions of other samples in the minibatch. Maximizing the difference between these two terms provides an estimate of the mutual information.

In MIR3, the CLUB estimator is used to estimate the mutual information between an agent’s observation history  $\mathbf{h}$  and its action  $\mathbf{a}$ . This estimate is incorporated into the regularized objective,

$$J(\pi) = \mathbb{E}[R] - \lambda I(\mathbf{h}; \mathbf{a}), \tag{B.3}$$

thereby encouraging the policy to retain only the information necessary for decision making while discarding redundant dependencies on teammate behaviors. This regularization improves the robustness of the learned policy without explicitly training against adversarial agents. The CLUB estimator is trained jointly with the policy network by learning a neural approximation of the conditional distribution  $q_\phi(\mathbf{y}|\mathbf{x})$ . As shown in Algorithm 74, the estimator first maximizes the conditional log-likelihood using positive observation-action pairs. It then estimates the mutual information upper bound by contrasting positive and negative pairs sampled from the minibatch. The estimated mutual information is finally minimized during policy optimization, encouraging the policy to discard redundant information while preserving task-relevant features .

---

**Algorithm 74** Mutual Information Minimization with CLUB

---

**Require:** Minibatch size  $N$ , conditional model  $q_\phi(y|x)$

**Ensure:** Estimated mutual information upper bound  $\hat{I}_{\text{CLUB}}$

- 1: **for** each training iteration **do**
- 2:     Sample minibatch  $\{(x_i, y_i)\}_{i=1}^N$  from the joint distribution  $p_\sigma(x, y)$
- 3:     Compute the conditional log-likelihood

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \log q_\phi(y_i|x_i)$$

- 4:     Update the conditional model  $q_\phi$  by maximizing  $\mathcal{L}(\theta)$
- 5:     **for**  $i = 1$  to  $N$  **do**
- 6:         **if** sampling is used **then**
- 7:             Uniformly sample  $k_i \in \{1, 2, \dots, N\}$
- 8:             Compute

$$U_i = \log q_\phi(y_i|x_i) - \log q_\phi(y_{k_i}|x_i)$$

- 9:         **else**
- 10:             Compute

$$U_i = \log q_\phi(y_i|x_i) - \frac{1}{N} \sum_{j=1}^N \log q_\phi(y_j|x_i)$$

- 11:         **end if**
- 12:     **end for**
- 13:     Estimate the CLUB upper bound

$$\hat{I}_{\text{CLUB}} = \frac{1}{N} \sum_{i=1}^N U_i$$

- 14:     Update the policy by minimizing  $\hat{I}_{\text{CLUB}}$
  - 15: **end for**
-



---

## C. Total Variation Distance

Total Variation Distance (TVD) is a statistical measure used to quantify the difference between two probability distributions. In reinforcement learning, it is commonly used to measure the discrepancy between the nominal and perturbed state distributions, transition models, or policies. A smaller TVD indicates that the two distributions are more similar, whereas a larger value indicates greater divergence.

Let  $(\Omega, \mathcal{F})$  be a measurable space, and let  $P$  and  $Q$  be two probability measures defined on this space. The Total Variation Distance between  $P$  and  $Q$  is defined as

$$d_{\text{TV}}(P, Q) = \sup_{A \in \mathcal{F}} |P(A) - Q(A)|, \quad (\text{C.1})$$

where the supremum is taken over all measurable sets  $A \in \mathcal{F}$ . Intuitively, this definition measures the largest possible difference between the probabilities assigned by the two distributions to the same event.

For discrete probability distributions, the Total Variation Distance has the equivalent form

$$d_{\text{TV}}(P, Q) = \frac{1}{2} \sum_x |P(x) - Q(x)|, \quad (\text{C.2})$$

where the summation is taken over all possible outcomes. For continuous probability distributions with probability density functions  $p(x)$  and  $q(x)$ , the distance becomes

$$d_{\text{TV}}(P, Q) = \frac{1}{2} \int |p(x) - q(x)| \, dx. \quad (\text{C.3})$$

These equivalent formulations are often easier to compute in practice than the original definition.

The Total Variation Distance is closely related to the Kullback-Leibler (KL) divergence. One of the most widely used relationships is Pinsker's inequality,

$$d_{\text{TV}}(P, Q) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(P \| Q)}, \quad (\text{C.4})$$

which provides an upper bound on the Total Variation Distance in terms of the KL divergence. Another useful bound is

$$d_{\text{TV}}(P, Q) \leq \sqrt{1 - e^{-D_{\text{KL}}(P \| Q)}}. \quad (\text{C.5})$$

These inequalities are frequently used in robust reinforcement learning to derive theoretical guarantees and relate different measures of distributional robustness.



---

## D. Exponential Family Distributions

Many probabilistic models used in machine learning belong to the exponential family of distributions. This family has attractive mathematical properties that simplify parameter estimation, Bayesian inference, and optimization. Since Bayesian reinforcement learning and several probabilistic models discussed in this thesis rely on these properties, this appendix briefly reviews the main concepts.

### D.1. Definition

A probability distribution belongs to the exponential family if it can be written as

$$p(x|\omega) = h(x) \exp(\phi(x)^\top \omega - A(\omega)), \quad (\text{D.1})$$

where

- $h(x)$  is the *base measure*,
- $\omega$  is the natural (canonical) parameter,
- $\phi(x)$  denotes the sufficient statistics,
- $A(\omega)$  is the log-partition function.

The log-partition function ensures that the distribution integrates to one and is defined as

$$A(\omega) = \log \int h(x) \exp(\phi(x)^\top \omega) dx. \quad (\text{D.2})$$

Many common probability distributions belong to this family, including the Gaussian, Bernoulli, Binomial, Poisson, Gamma, and Dirichlet distributions.

## D.2. Maximum Likelihood Estimation

Suppose that a dataset consists of  $n$  independent samples

$$\mathcal{D} = \{x_1, \dots, x_n\}.$$

The log-likelihood is

$$\ell(\omega) = \sum_{i=1}^n \log p(x_i|\omega). \quad (\text{D.3})$$

Taking the gradient with respect to the natural parameter gives

$$\nabla_{\omega} A(\omega) = \frac{1}{n} \sum_{i=1}^n \phi(x_i). \quad (\text{D.4})$$

The maximum likelihood estimate is therefore obtained by matching the expected sufficient statistics of the model with their empirical average.

An important identity is

$$\mathbb{E}_{p(x|\omega)}[\phi(x)] = \nabla_{\omega} A(\omega), \quad (\text{D.5})$$

which connects the model expectation directly to the gradient of the log-partition function.

## D.3. Conjugate Prior

One of the main advantages of exponential family distributions is the existence of conjugate priors. A conjugate prior has the form

$$p_F(\omega|\alpha, \nu) = \exp(\omega^{\top} \alpha - \nu A(\omega) - \log F(\alpha, \nu)), \quad (\text{D.6})$$

where  $\alpha$  and  $\nu$  are hyperparameters and

$$F(\alpha, \nu) = \int \exp(\omega^{\top} \alpha - \nu A(\omega)) d\omega \quad (\text{D.7})$$

is the normalization constant.

After observing a single sample  $x$ , the predictive distribution becomes

$$p(x) = h(x) \frac{F(\alpha + \phi(x), \nu + 1)}{F(\alpha, \nu)}. \quad (\text{D.8})$$

## D.4. Posterior Distribution

Given a dataset  $X = \{x_1, \dots, x_n\}$ , the posterior remains in the same exponential family:

$$p(\omega \mid X, \alpha, \nu) = p_F\left(\omega \mid \alpha + \sum_{i=1}^n \phi(x_i), \nu + n\right), \quad (\text{D.9})$$

or equivalently,

$$p(\omega \mid X, \alpha, \nu) = \frac{\exp[\omega^\top (\alpha + \sum_{i=1}^n \phi(x_i)) - (\nu + n)A(\omega)]}{F(\alpha + \sum_{i=1}^n \phi(x_i), \nu + n)}. \quad (\text{D.10})$$

The posterior mean satisfies the moment-matching condition

$$\nabla_\omega A(\omega^*) = \frac{\alpha + \sum_{i=1}^n \phi(x_i)}{\nu + n}. \quad (\text{D.11})$$

For a sufficiently large number of observations ( $n \gg \nu$ ),

$$\nabla_\omega A(\omega^*) \approx \frac{\alpha}{n} + \frac{1}{n} \sum_{i=1}^n \phi(x_i) = \mathbb{E}_{p(X)}[\phi(X)]. \quad (\text{D.12})$$

Therefore, the optimal natural parameter can be written as

$$\omega^* = \arg \min_{\omega} D_{\text{KL}}(p(X) \parallel p_\omega(X)), \quad (\text{D.13})$$

which states that the learned exponential-family distribution matches the empirical moments of the observed data.



---

## E. $n$ -Mode Tensor Product

Meta-Curvature represents the gradient of each network layer as an  $N$ -order tensor,

$$\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}.$$

Applying a matrix to one tensor mode requires first converting the tensor into a matrix. This operation is known as *mode- $n$  unfolding* (or matricization). The unfolded matrix is denoted by

$$X_{[n]},$$

where the mode- $n$  fibers are arranged as the rows of the matrix. An element of the tensor is mapped according to

$$X_{i_1, i_2, \dots, i_N} \rightarrow X_{[n]}(i_n, j),$$

with

$$j = 1 + \sum_{\substack{k=1 \\ k \neq n}}^N (i_k - 1) J_k,$$

where

$$J_k = \prod_{\substack{m=1 \\ m \neq n}}^{k-1} I_m.$$

After unfolding, the  $n$ -mode product of a tensor  $\mathcal{X}$  with a matrix  $M$  is computed as

$$\mathcal{Y} = \mathcal{X} \times_n M.$$

The computation consists of three steps:

1. Unfold the tensor along mode  $n$ ,

$$X_{[n]} \in \mathbb{R}^{I_n \times (I_1 \cdots I_{n-1} I_{n+1} \cdots I_N)}.$$

2. Multiply the unfolded tensor by the transformation matrix,

$$Y_{[n]} = M X_{[n]}.$$

3. Fold the resulting matrix back into a tensor to obtain

$$\mathcal{Y}.$$

In Meta-Curvature, the gradient tensor is transformed independently along the output, input, and filter dimensions using successive  $n$ -mode products,

$$\text{MC}(G) = G \times_3 M_f \times_2 M_i \times_1 M_o,$$

where  $M_o$ ,  $M_i$ , and  $M_f$  model dependencies between output channels, input channels, and filter parameters, respectively.

---

## F. Stochastic Computation Graphs

Many reinforcement learning and meta-learning algorithms optimize objectives that depend on stochastic sampling operations. Standard automatic differentiation computes derivatives through deterministic computations, but it cannot directly differentiate expectations over random variables. Stochastic computation graphs (SCGs) provide a unified framework for representing computation graphs that contain both deterministic and stochastic operations and for deriving unbiased gradient estimators. This appendix summarizes the notation and results required to understand the DiCE estimator presented in Chapter 7. The presentation follows Schulman et al. [109].

### F.1. Graph Definition

A stochastic computation graph is a directed acyclic graph composed of four types of nodes (Figure F.1):

- Input nodes  $\Theta$ , which represent parameters with respect to which gradients are computed.
- Deterministic nodes  $\mathcal{D}$ , which perform differentiable deterministic computations.
- Stochastic nodes  $\mathcal{S}$ , which represent random variables sampled from probability distributions.
- Cost nodes  $\mathcal{C}$ , which contribute to the objective function.

Two types of directed edges are considered.

- A stochastic edge  $v \prec w$  indicates that the distribution of  $w$  depends on  $v$ ,  $w \sim p(w \mid v)$ .
- A deterministic edge  $v \prec^D w$  indicates that  $w$  is computed deterministically from  $v$ .

The deterministic dependencies of a node  $v$  are defined as

$$\text{DEPS}_v = \{ w \in \Theta \mid w \prec^D v \}.$$

For every stochastic node  $v$ , the quantity  $\hat{Q}_v$

denotes the sum of all downstream costs influenced by that node, while  $\hat{v}$

denotes the sampled value of node  $v$ .

## F.2. Differentiability Requirements

For every parameter  $\theta \in \Theta$ , the graph must satisfy the following differentiability conditions.

- If a deterministic node  $w$  depends on another deterministic node  $v$ , then the Jacobian

$$\frac{\partial w}{\partial v}$$

must exist.

- If  $w$  is stochastic, then the probability distribution must be differentiable,

$$\frac{\partial}{\partial v} p(w \mid \text{PARENTS}_w).$$

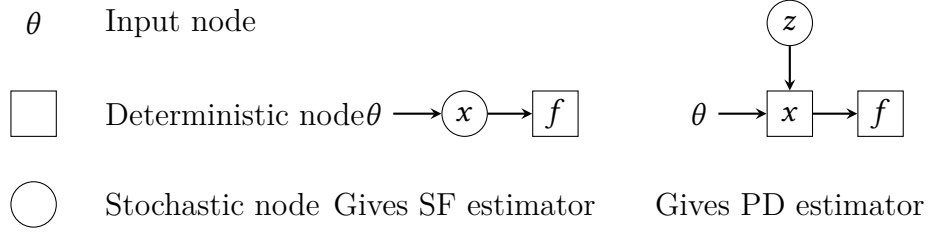
These conditions ensure that gradients can propagate through both deterministic computations and probability distributions.

### F.2.1. Gradient Estimation

The objective is to compute the gradient of the expected total cost with respect to an input parameter  $\theta$ ,

CXC





**Figure F.1.** – An example of computation graph

$$\begin{aligned}
 \frac{\partial}{\partial \theta} \mathbb{E} \left[ \sum_{c \in C} c \right] &= \mathbb{E} \left[ \sum_{\substack{w \in \mathcal{S}, \\ \theta \prec^D w}} \left( \frac{\partial}{\partial \theta} \log p(w \mid \text{DEPS}_w) \right) \hat{Q}_w + \sum_{\substack{c \in C, \\ \theta \prec^D c}} \frac{\partial}{\partial \theta} c(\text{DEPS}_c) \right] \\
 &= \mathbb{E} \left[ \sum_{c \in C} \hat{c} \sum_{\substack{w \prec c, \\ \theta \prec^D w}} \frac{\partial}{\partial \theta} \log p(w \mid \text{DEPS}_w) + \sum_{\substack{c \in C, \\ \theta \prec^D c}} \frac{\partial}{\partial \theta} c(\text{DEPS}_c) \right].
 \end{aligned}$$

This estimator is unbiased and combines the score-function estimator for stochastic nodes with standard backpropagation through deterministic nodes.



---

## G. Background: Contrastive Learning and Momentum Contrast

This appendix summarizes the contrastive learning machinery referenced by CCM, specifically the InfoNCE loss and the momentum-encoder mechanism used to stabilize it.

**Contrastive learning.** Contrastive learning learns a representation without labels by pulling representations of similar inputs together and pushing representations of dissimilar inputs apart, with similarity typically defined either through data augmentation, two augmented views of the same input are treated as similar, or, in a supervised variant, through shared class labels. The resulting embedding space is trained so that semantically related inputs lie close together and unrelated inputs lie far apart, without requiring any explicit label describing what makes them similar in the first place.

**InfoNCE.** The standard contrastive objective treats representation learning as a classification problem: given a query representation  $q$ , a positive key  $k^+$  known to correspond to the same underlying input as  $q$ , and a set of negative keys  $\{k^i\}$  that do not, the model is trained to assign high similarity to the true positive pair relative to every negative pair,

$$\mathcal{L}_q = -\log \frac{\exp(q \cdot k^+ / \tau)}{\sum_{i=0}^K \exp(q \cdot k^i / \tau)},$$

where  $q \cdot k$  denotes a similarity score, typically a dot product or cosine similarity, and  $\tau$  is a temperature parameter controlling how sharply the softmax distinguishes between similarity scores. This loss, known as InfoNCE, is the workhorse objective behind essentially every contrastive method referenced in this thesis, including CCM’s context encoder and its information-gain exploration bound: both are direct applications of this same query-positive-negatives structure, with  $q$  and  $k$  replaced by encoded trajectory batches rather than encoded images.

**The consistency problem.** A practical difficulty arises when the encoder producing the keys  $k^i$  is updated at every training step, as an ordinary neural network would be: representations of two very similar inputs encoded at different points in training, separated by many gradient updates, can end up far apart in the embedding space purely because the encoder itself changed in between, not because the inputs were actually dissimilar. This undermines the contrastive objective, since the negatives being compared against were not encoded under a consistent notion of similarity.

**Momentum Contrast (MoCo).** He et al. [110] address this with two separate encoders rather than one. A query encoder, with parameters  $\theta_q$ , is updated normally via gradient descent at every step. A key encoder, with parameters  $\theta_k$ , is not trained by gradient descent at all; instead, its parameters are updated as a slowly moving average of the query encoder’s parameters,

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q,$$

where  $m$  is a momentum coefficient close to one, so that  $\theta_k$  changes far more slowly than  $\theta_q$ . This keeps the representations used as keys reasonably consistent across nearby training steps, since the key encoder evolves gradually rather than jumping at every gradient update, addressing the consistency problem directly. Gradients are not backpropagated through the key encoder,  $k \leftarrow \text{stopgrad}(f_k(x_k; \theta_k))$ , since it is never trained directly, only updated by the momentum rule above.

Algorithm 75 summarizes the resulting training loop. Two augmented views of the same minibatch are passed through the query and key encoders respectively, the positive logit is computed as the similarity between the resulting query and key, negative logits are computed against a queue of previously encoded keys  $\mathcal{Q}$ , and the query encoder is trained with a standard cross-entropy loss over the resulting logits, equivalent to the InfoNCE loss above.

---

**Algorithm 75** Momentum Contrast (MoCo) Training

---

**Require:** Query encoder parameters  $\theta_q$ , key encoder parameters  $\theta_k$ , queue  $\mathcal{Q}$  of size  $K$ , momentum coefficient  $m$ , temperature  $\tau$   
Initialize  $\theta_k \leftarrow \theta_q$   
**for** each minibatch  $x = \{x_i\}_{i=1}^N$  **do**  
     $x_q \leftarrow \text{Aug}(x), \quad x_k \leftarrow \text{Aug}(x)$   
     $q \leftarrow f_q(x_q; \theta_q), \quad k \leftarrow f_k(x_k; \theta_k)$   
     $k \leftarrow \text{stopgrad}(k)$   
     $l_{\text{pos}} \leftarrow q^\top k, \quad l_{\text{neg}} \leftarrow q^\top \mathcal{Q}$   
     $\text{logits} \leftarrow \text{concat}(l_{\text{pos}}, l_{\text{neg}})$   
     $L \leftarrow \text{CrossEntropy}(\text{logits}/\tau, \mathbf{0}_N)$   
    Update  $\theta_q \leftarrow \theta_q - \eta \nabla_{\theta_q} L$   
    Update  $\theta_k \leftarrow m\theta_k + (1 - m)\theta_q$   
    Enqueue  $k$  into  $\mathcal{Q}$ ; dequeue the oldest keys to keep  $|\mathcal{Q}| = K$   
**end for**

---

**Relevance to CCM.** CCM’s contrastive context encoder is a direct application of this mechanism: the query encoder  $e_\phi$  plays the role of  $f_q$  above, encoding a batch of trajectories from the task currently being explored, while a momentum-updated key encoder  $\bar{e}_\phi$  encodes positive and negative trajectory batches from the same and different tasks respectively, exactly as in Algorithm 75, with trajectory batches taking the place of augmented images. CCM’s information-gain exploration bound reuses the same similarity function and InfoNCE structure introduced here, applied to the encoder’s own output at consecutive points in an exploration trajectory rather than to independently sampled positive and negative pairs.



---

# H. Use of Generative AI Tools

In accordance with the Declaration of Authorship, Table H.1 names every generative AI tool used in the preparation of this thesis and the purpose for which it was used. [FILL IN: verify this table matches your actual usage and your institution’s required disclosure format before submission; adjust or extend as needed.]

**Table H.1.** – Generative AI tools used in the preparation of this thesis.

| Tool                         | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude (Anthropic), Sonnet 5 | Drafting, restructuring, and editing thesis text under the author’s direction; converting source material into $\text{\LaTeX}$ ; technical formalization and verification of mathematical claims against cited source papers; identifying and fixing errors in derivations, cross-references, and citations; literature search and citation verification for newly added sources; document formatting and template migration. All generated content was reviewed, and in many cases directed and revised, by the author, who is responsible for the information and statements in this thesis. |





## List of Tables

|                                                                                                                                                                                                                                                                                    |         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| 2.1. Methods considered but excluded from each major taxonomy branch, against the selection criteria stated above. . . . .                                                                                                                                                         | 7       |
| 6.1. Comparison of different reinforcement learning paradigms. . . . .                                                                                                                                                                                                             | 84      |
| 6.2. Common types of non-stationary environments. . . . .                                                                                                                                                                                                                          | 89      |
| 7.1. Comparison between few-shot and many-shot Meta-RL. . . . .                                                                                                                                                                                                                    | 142     |
| 7.2. Adaptation via memory versus adaptation via generated parameters. . . . .                                                                                                                                                                                                     | 150     |
| 7.3. Integral Probability Metrics arising from different function classes $F$ . . . . .                                                                                                                                                                                            | 159     |
| 7.4. Choices for what the inner loop adapts when combining CTDE with policy parameter gradient methods. . . . .                                                                                                                                                                    | 174     |
| 8.1. Robust MARL and adaptive MARL compared on four axes. . . . .                                                                                                                                                                                                                  | 186     |
| 9.1. The curated exemplar methods referenced in Chapters 5–7 and Appendix A, one row per method, on the same five axes as Table 8.1. Every cell restates what that method’s own subsection, or Chapter 8’s comparison of it, already says; none is a new characterization. . . . . | 188     |
| A.1. Comparison of gradient transformations. . . . .                                                                                                                                                                                                                               | IX      |
| A.2. Optimization as a reinforcement learning problem. . . . .                                                                                                                                                                                                                     | XIX     |
| A.3. Unified interpretation of higher-order meta-gradient estimators based on off-policy value estimation. . . . .                                                                                                                                                                 | XLIV    |
| A.4. Bias–variance trade-off controlled by the Taylor expansion order. . . . .                                                                                                                                                                                                     | XLV     |
| A.5. Context vector computation in the MQN family. . . . .                                                                                                                                                                                                                         | LXV     |
| A.6. Best-suited scenarios for each variant of the MQN family. . . . .                                                                                                                                                                                                             | LXVI    |
| A.7. Main findings reported for the MRA agent. . . . .                                                                                                                                                                                                                             | LXXV    |
| A.8. Categories of limitations of the MRA agent. . . . .                                                                                                                                                                                                                           | LXXVI   |
| A.9. Interpretation of successive planning iterations in EPN. . . . .                                                                                                                                                                                                              | LXXXIII |
| A.10. Fast Weight Programmers compared to standard Transformer attention. . . . .                                                                                                                                                                                                  | XCI     |
| A.11. Interpretation of the components of a Self-Referential Weight Matrix update. . . . .                                                                                                                                                                                         | XCIII   |
| A.12. Learned components of MetODS and their role during adaptation. . . . .                                                                                                                                                                                                       | CIII    |
| A.13. Conceptual shift from standard reinforcement learning to learning a Hebbian update rule. . . . .                                                                                                                                                                             | CVI     |

|                                                                                                             |        |
|-------------------------------------------------------------------------------------------------------------|--------|
| A.14. Terms of the general Hebbian update rule. . . . .                                                     | CVI    |
| A.15. Policy parameter gradient adaptation (MAML) compared to black-box adaptation ( $RL^2$ ). . . . .      | CX     |
| A.16. Trade-off between end-to-end and decoupled exploration-execution training. . . . .                    | CXVIII |
| A.17. CCM's departures from PEARL. . . . .                                                                  | CXXXIV |
| A.18. ESCP's mechanisms mapped to the failure modes of prior context-based methods. . . . .                 | CXLII  |
| A.19. Variational inference (e.g. PEARL) compared to ESCP. . . . .                                          | CXLII  |
| A.20. Every Section 5.9 method placed along the two design axes introduced in the section overview. . . . . | CLXVII |
| H.1. Generative AI tools used in the preparation of this thesis. . . . .                                    | CXCVII |

## List of Figures

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 3.1. Taxonomy of the single-agent model-free algorithms covered in this chapter, restricted to methods that are directly extended to the multi-agent setting in Chapter 4 or reused as a component elsewhere in this thesis. . . . .                                                                                                                                                                                                                                                                                                                                                   | 10  |
| 5.1. Overview of the DrIGM training framework. Each agent learns a robust local action-value function, which is combined through a value factorization network to construct the robust joint action-value function. Robust Bellman targets are computed using the target network, and the network parameters are updated by minimizing the robust temporal-difference loss under the selected uncertainty model. Adapted from [42]. . . . .                                                                                                                                            | 63  |
| 5.2. Overview of the Adversarial Domain Randomization (ADR) framework. The Nature Player generates randomized environment parameters using an adversarial environment generator, while the MARL agents interact with the resulting environment. The actor-critic networks and the Nature Player are trained jointly in a minimax optimization framework, enabling the agents to learn policies that transfer more effectively from simulation to real-world environments. Adapted from [45]. . . . .                                                                                   | 67  |
| 7.1. Overview of the MAML framework, which learns an initialization $\theta$ for rapid adaptation to new tasks.[71] . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 146 |
| 7.2. VariBAD architecture. The encoder $q_\phi$ recurrently processes the current state, previous action, and reward, together with its own previous hidden state $h_{t-1}$ , to produce the belief $b_t = q_\phi(m \mid \tau_t)$ over the task variable $m$ . The decoder reconstructs the trajectory from a sampled $m$ , via a separate reward model $p_\theta^R$ and transition model $p_\theta^T$ , applied at every timestep $i$ . The policy $\pi_\psi$ is conditioned on the belief $b_t$ rather than on a single posterior sample. Adapted from Zintgraf et al. [76]. . . . . | 165 |

- 7.3. PEARL’s permutation-invariant context encoder. Each transition in the context is passed independently through the same network  $\phi$  to produce a per-transition Gaussian factor  $\Psi_\phi(\mathbf{z} \mid \mathbf{c}_n)$ . The factors are multiplied together to form the posterior  $q_\phi(\mathbf{z} \mid \mathbf{c})$ , making the encoder invariant to the order in which transitions are presented. Adapted from Rakelly et al. [77]. . . . . 169
- 7.4. Full PEARL training architecture. Context is sampled from the replay buffer via a separate strategy  $\mathcal{S}_C$  and passed through the encoder  $\phi$  to produce the posterior  $q_\phi(\mathbf{z} \mid \mathbf{c})$ , regularized toward the prior  $\mathcal{N}(0, I)$  via  $D_{\text{KL}}$ . A sample  $\mathbf{z}$  conditions both the critic  $Q_\theta$  and the actor  $\pi_\theta$ , trained with off-policy losses  $\mathcal{L}_{\text{critic}}$  and  $\mathcal{L}_{\text{actor}}$  on transitions drawn from the same buffer. Adapted from Rakelly et al. [77]. . . . . 170
- 7.5. Meta-CTDE overview. Training and test task trajectories, together with task-specific context gathered by a behavior policy, feed a Multi-Level Task Encoder producing a global representation  $\mathbf{z}$  and per-agent local representations  $\mathbf{z}^i$ . The global representation trains the centralized critic  $Q(s, \mathbf{a}, \mathbf{z})$  via  $\mathcal{L}_Q$ ; each local representation conditions a decentralized actor  $\pi^i(\mathbf{a} \mid \mathbf{o}^i, \mathbf{z}^i)$  via  $\mathcal{L}_\pi^i$ ; and  $\mathcal{L}_{\text{MG2L}}$  links the two representations. Black arrows show training-time data flow (both representations available); red arrows show test-time data flow (only the local representation is available). Adapted from Zhao et al. [78]. . . . . 176
- 7.6. Multi-Level Task Encoder detail. The transition encoder  $E_{\text{tran}}$  processes each transition  $\mathbf{c}_t^i$  into  $\mathbf{x}_t^i$ . The aggregation encoder combines the sequence  $\mathbf{x}_{t-1}^i, \mathbf{x}_t^i, \mathbf{x}_{t+1}^i, \dots$  across time using stacked Permutation-Invariance Attention blocks (inset), producing  $\mathbf{x}^i$ . The global inference module combines every agent’s  $\mathbf{x}^i$  into  $\mathbf{z}$ ; the local inference module produces each agent’s own  $\mathbf{z}^i$  from  $\mathbf{x}^i$  alone. Adapted from Zhao et al. [78]. . . . . 177
- 7.7. Full MG2L architecture, distinguishing data flow (black) from gradient flow (green). The Multi-Level Task Encoder (left) produces global and local task inferences, linked by the MG2L losses  $\mathcal{L}_G$  and  $\mathcal{L}_L$ . The resulting representations feed the centralized critic and decentralized actor (right), trained with the standard MAMRL losses  $\mathcal{L}_Q$  and  $\mathcal{L}_\pi$ . Adapted from Zhao et al. [78]. . . . . 180
- A.1. Overview of the CAVIA architecture. A low-dimensional context vector  $\phi$  is adapted for each task, while the shared parameters  $\theta$  remain fixed during the inner loop. Adapted from [82] . . . . . VI

- A.2. Tensor decomposition of the meta-curvature transformation. Instead of learning a full curvature matrix, Meta-Curvature factorizes the transformation into three learnable matrices,  $M_o$ ,  $M_i$ , and  $M_f$ , which model dependencies between output channels, input channels, and filter parameters, respectively. This decomposition enables an efficient approximation of the full gradient transformation while keeping the number of learnable parameters manageable. Adapted from [56]. . . . VIII
  
- A.3. WarpGrad architecture. Trainable warp layers are inserted between the original network layers to learn a shared parameter transformation. During adaptation, task-specific learners update the model parameters, while the shared warp layers transform the gradients through their Jacobians, producing a learned preconditioning matrix. Adapted from [57]. . . . . XII
  
- A.4. Geometric interpretation of WarpGrad. The warp transformation maps the original parameter space  $\theta$  to a warped space  $\gamma$ , where gradient descent follows the geometry induced by the learned transformation. The update is then projected back to the original parameter space through the Jacobian of the warp transformation. Adapted from [57]. . . . . XIII
  
- A.5. Training framework of the generalizable learned optimizer. During the inner loop, the policy observes optimization statistics and predicts relative modifications to the optimizer hyperparameters. The outer loop updates the policy using PPO according to the optimization reward. Adapted from [55]. . . . . XX
  
- A.6. figure illustrates how uncertainty increases with each adaptation step XXIII
  
- A.7. Comparison between the conventional surrogate loss and the DiCE objective. The surrogate loss computes the correct first-order policy gradient but loses the dependency of the sampled objective on the policy parameters during automatic differentiation, resulting in incorrect higher-order derivatives. DiCE preserves these dependencies by augmenting the computation graph with the MagicBox operator, enabling unbiased differentiation of arbitrary order. Adapted from [86].XXV
  
- A.8. DiCE objective for reinforcement learning. Each reward is weighted by a MagicBox operator that depends on all preceding stochastic actions influencing that reward. This construction preserves the correct higher-order dependencies, allowing automatic differentiation to compute unbiased higher-order policy gradients. Adapted from [86]. . XXVI

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| A.9. Examples of stochastic computation graphs encountered in reinforcement learning. Besides standard MDPs, latent variables and partial observability introduce additional stochastic dependencies that complicate higher-order gradient estimation. Loaded DiCE extends DiCE to support variance reduction techniques while preserving these dependencies. Adapted from [87]. . . . .                                                                                                                                                                                       | XXXII   |
| A.10. Stochastic computation graph for gradient-based meta-reinforcement learning. Starting from the initial policy $\theta^{(0)}$ , each adaptation step collects a trajectory $\tau^{(k)}$ , computes the corresponding policy gradient, and updates the policy parameters to $\theta^{(k+1)}$ . Consequently, the distribution of every trajectory depends on all previous policy updates, creating long-range stochastic dependencies between the meta-parameters, intermediate trajectories, and the final evaluation return $R(\tau^{\text{test}})$ . . . . .            | XXXVIII |
| A.11. Overview of MAESN, showing task-specific latent variables $z_i$ used to drive structured exploration during adaptation [65] . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                    | XLVII   |
| A.12. Computation graphs for PPO, MAML, and E-MAML. The E-MAML graph preserves the dependence between exploration trajectories and the post-adaptation objective, allowing the meta-gradient to optimize exploration behavior. [63] . . . . .                                                                                                                                                                                                                                                                                                                                  | L       |
| A.13. The two main components of the SNAIL architecture: (a) the Temporal Convolution (TC) block captures long-range temporal dependencies through dilated causal convolutions, while (b) the attention block selectively retrieves relevant information from past observations using causal self-attention. . . . .                                                                                                                                                                                                                                                           | LVI     |
| A.14. SNAIL architecture applied to supervised learning (left) and reinforcement learning (right). Orange nodes denote temporal convolution (TC) blocks, green nodes denote attention blocks, and red nodes denote the raw input sequence. Adapted from Mishra et al. [89]. . . .                                                                                                                                                                                                                                                                                              | LVI     |
| A.15. TrMRL architecture. Raw transitions are embedded into working memory vectors $\phi_t$ , processed by $L$ layers of a causal Transformer encoder with positional embedding, causal multi-head attention, and a position-wise MLP, producing refined representations $e_t$ . The policy head is conditioned on the final representation $e_t$ to produce the action $a_t$ . The attention maps below illustrate how each memory slot attends only to itself and earlier slots, with attention weight distributed unevenly across the window. Adapted from Melo [90]. . . . | LX      |

- A.16. Memory operations of the MQN family. (a) Write: the current input  $\mathbf{x}_t$  is projected into key and value memory blocks via  $W^{\text{key}}$  and  $W^{\text{val}}$ . (b) Read: the context vector  $\mathbf{h}_t$  is compared against the stored keys  $M_t^{\text{key}}$  via softmax attention to produce weights  $\mathbf{p}_t$ , which are used to combine the stored values  $M_t^{\text{val}}$  into the retrieved memory. Adapted from Oh et al. [91]. . . . . LXIX
- A.17. Architectural comparison of DQN, DRQN, MQN, RMQN, and FRMQN. DQN (a) stacks the last  $M$  frames as input with no separate memory module. DRQN (b) replaces frame-stacking with a recurrent context. MQN (c) adds an explicit memory module read by a memoryless context. RMQN (d) reads the same memory using a recurrent context. FRMQN (e) additionally feeds the retrieved memory back into the recurrent context, closing the feedback loop between reading and remembering. Adapted from Oh et al. [91]. . . . . LXX
- A.18. The MRA agent. (a) Working memory (an LSTM) and episodic memory (a key-value store) interact through explicit write and read operations: writing stores the current working memory state and input without gradient, and reading retrieves a nearest-neighbor-weighted combination of stored values, fed back into the working memory at the next timestep. (b) The CPC auxiliary loss: the working memory state  $\mathbf{h}_t$  at a given timestep is trained to predict the CNN+ResNet embeddings of future observations  $\mathbf{x}_{t+1}, \mathbf{x}_{t+2}, \mathbf{x}_{t+3}$ , shown by the dashed arrows, providing a reward-independent training signal. Adapted from Fortunato et al. [92]. . . . . LXXI
- A.19. Episodic recall across a sequence of tasks with repetition. As the agent moves through Task 1, Task 2, and Task 3, the reinstatement gate remains closed (no relevant memory to recall), and a new entry is written to the DND at the end of each task. When Task 1 recurs, the reinstatement gate opens, and the LSTM cell state is reinstated from the corresponding DND entry rather than being rebuilt from scratch. Adapted from Ritter et al. . . . . LXXVII
- A.20. The reinstatement mechanism inside a single LSTM cell. The standard forget-gate and input-gate pathway (gray) updates the cell state as usual. In parallel, the memory retrieved from  $\text{DND}_t$  is passed through a  $\tanh$  nonlinearity and gated by the reinstatement gate  $\mathbf{r}_t$  (pink), before being added into the cell state  $\mathbf{c}_t$ . The updated context key and cell state are written to the dictionary, producing  $\text{DND}_{t+1}$ . Adapted from Ritter et al. . . . . LXXVIII

- A.21. Episodic Planning Network architecture. Stored transitions form a graph over observations, the goal embedding is appended to every memory, and the planner module iteratively updates its belief representation  $B_i$  through repeated self-attention among memories, propagating goal-relevant information across the graph before the policy network selects an action. Adapted from Ritter et al. [93]. . . . . LXXXIV
- A.22. BIMRL architecture. The task inference module (GRU) produces a task embedding  $m$  from the transition  $(s, a, r)$ , feeding a hierarchical structure of environment learning, planning, and control modules, connected via  $h_1$ ,  $h_2$ , and  $h_3$ . The controller reads from and writes to a memory module combining explicit episodic memory and Hebbian memory via a consolidation process, and its output drives an actor-critic policy trained with PPO. Adapted from Rohani et al. [97]. . . . . XCV
- A.23. Read and write operations for BIMRL’s explicit episodic memory. The controller hidden state produces a value embedding, while the current state and task inference output jointly produce a key embedding for writing, and a query embedding for reading. Retrieval uses multi-head attention over the stored keys and values, together with each slot’s reference time, producing the retrieved information used by the controller. Adapted from Rohani et al. [97]. . . . . XCVII
- A.24. Multi-step recursive synaptic update in MetODS. Starting from  $v^{(0)}$  and the previous weight matrix  $W_t$ , each recursion step  $s$  reads from the current weights to produce  $v^{(s)}$  and writes a new self-correlation term into the weight matrix, shown as the graph structures  $W^{(1)}, W^{(2)}, \dots$ , growing denser with each recursion step. The accumulated activations and weights across all  $S$  steps produce  $v^{(S)}$  and  $W_{t+1}$ . Adapted from Chalvidal et al. [98]. . . . . CII
- A.25. The chicken-and-egg problem in end-to-end exploration-execution training. A good exploration trajectory  $\tau_{\text{good}}^{\text{exp}}$  can be penalized if the execution policy  $\pi_{\theta}^{\text{task}}$  fails to exploit it, while a bad trajectory  $\tau_{\text{bad}}^{\text{exp}}$  can be reinforced if it is followed by a lucky reward, since both  $\pi_{\phi}^{\text{exp}}$  and  $\pi_{\theta}^{\text{task}}$  are updated from the same reward signal  $\max_{\phi, \theta} R$ . Adapted from Liu et al. [101]. . . . . CXVII
- A.26. DREAM’s decoupled architecture. At train time, the problem ID module maximizes  $I(F_{\psi}(z | \mathcal{T}); \tau^{\text{exp}})$ , updating the exploration policy  $\pi_{\phi}^{\text{exp}}$  via  $\nabla_{\phi}$  without any gradient from the execution side. The execution policy  $\pi_{\theta}^{\text{exe}}$  is updated via  $\nabla_{\theta}$  to maximize return  $R$ , conditioned on  $z$ . At test time, only the exploration trajectory  $\tau^{\text{exp}}$ , not any gradient, passes from the exploration side to the execution side. Adapted from Liu et al. [101]. . . . . CXIX



- A.27. The information bottleneck compresses the full problem identity  $\mu$ , here specifying wall color, ingredients, and decorations, into a much smaller bottlenecked representation  $\mathbf{z}$ , which the exploitation policy  $\pi^{\text{task}}$  is conditioned on. Adapted from Liu et al. [101]. . . . . CXX
- A.28. The exploration policy  $\pi^{\text{exp}}$  produces an exploration episode  $\tau^{\text{exp}}$ , and is rewarded by the information recovery reward  $I(\mathbf{z}; \tau^{\text{exp}})$ , measuring how well  $\mathbf{z}$  can be recovered from the resulting trajectory, entirely independent of task reward. Adapted from Liu et al. [101]. . . . . CXXI
- A.29. MetaCURE architecture. The Task-Predictor and Meta-Predictor produce  $L_{\text{pred}}^{\text{task}}$  and  $L_{\text{pred}}$  from the true task index  $\kappa$  and from context  $(s, a, r, s')$  respectively; their difference gives the intrinsic reward  $r'_{\text{int}}$  that trains the exploration policy  $\pi_e$ . The context encoder produces the posterior belief  $q_\phi(\mathbf{z} | c)$ , regularized toward  $\mathcal{N}(0, I)$ , which feeds both the Explorer and the Exploiter. The Exploiter trains  $\pi$  and  $Q^\pi$  using only environment reward, with no intrinsic signal. Adapted from Zhang et al. [102]. . . . . CXXVII
- A.30. Contextual control mechanism. . . . . CXXXI
- A.31. Contrastive context learning in CCM. A task  $\mu_n$  is sampled from the task set, and trajectory batches  $b_n^k$  are drawn from it and passed through both the query encoder, producing  $\mathbf{z}^q$ , and the momentum encoder, producing  $\mathbf{z}^k$ . The pair  $(\mathbf{z}^q, \mathbf{z}^k)$  drives the contrastive loss, while  $\mathbf{z}^q$  is separately consumed by reinforcement learning, that is, by the exploration and execution agents described in Section 5.9.3.3, with no gradient from the RL side flowing back into the encoder. Adapted from Fu et al. [103]. . . . . CXXXVI
- A.32. ESCP architecture. A window of  $H$  past state-action pairs is passed through the context encoder (CE) to produce a latent context distribution, from which  $\mathbf{z}^t$  is sampled. The contextual policy (CP) combines  $\mathbf{z}^t$  with the current state  $s_t$  to produce an action distribution and sample  $a_t$ . A moving average of  $\mathbf{z}^t$  against a stored estimate  $\bar{\mathbf{z}}$  produces the smoothed context used by the critic  $Q$ . Adapted from Luo et al. [104]. . . . . CXXXVIII
- A.33. PD-VF's self-supervised encoder-decoder pairs. The policy encoder  $E_\pi$  maps a sequence of state-action pairs  $\{(s_t, a_t)\}$  to a policy embedding  $\mathbf{z}_\pi$ , decoded by  $D_\pi$  back into a predicted action  $\hat{a}_t$  given the current state  $s_t$ . The dynamics encoder  $E_d$  maps a sequence of transitions  $\{(s_t, a_t, s_{t+1})\}$  to a dynamics embedding  $\mathbf{z}_d$ , decoded by  $D_d$  into a predicted next state  $\hat{s}_{t+1}$  given the current state and action. Adapted from Raileanu et al. [105]. . . . . CXLIV

- A.34. PD-VF at adaptation time. The dynamics encoder  $E_d$  produces  $z_d$  from a handful of observed transitions, feeding the value function  $W$  together with the initial state  $s_0$ . Rather than searching over  $z_\pi$  via the value function's ordinary gradient interaction (gray dashed arrow), the optimal embedding  $z_\pi^*$  (red dashed arrow) is obtained directly via SVD, and passed to the policy decoder  $D_\pi$  to produce actions. Adapted from Raileanu et al. [105]. . . . . CXLVI
- A.35. FLAP training and testing. Left (training): the shared layers produce  $\phi$  from task  $T_i$ , combined with the true, RL-optimized weights  $w_i$  to form the policy  $\pi_i$ . The Adapter NN separately predicts  $\hat{w}_i$  from a transition  $(s, a, r, s')_i$ , and both  $w_i$  and  $\hat{w}_i$  feed an MSE loss whose gradient trains only the Adapter NN. Right (testing): the shared layers are frozen; the Adapter NN predicts  $w_{\text{test}}$  directly from a transition observed on the test task  $T_{\text{test}}$ , combined with the frozen  $\phi$  to produce  $\pi_{\text{test}}$ , with no further training. Adapted from Peng et al. [106]. . . . CLII
- A.36. BRUNO pipeline. Transitions from the environment are modeled by BRUNO as  $p(r, s' | s, a, \tau)$ , which feeds the closed-form exchangeable posterior update over  $\theta$ , and the resulting belief conditions a Soft Actor-Critic policy. . . . . CLVIII
- A.37. Glass-box evaluation. The learned representation is frozen; only the probe network is trained to predict position, orientation, object locations, and past trajectory from it. No gradient from the probe's loss reaches the representation, so probe accuracy reflects only what the representation already encoded. . . . . CLXI
- A.38. Two self-supervised architectures for training the belief state  $b_t$ . (a) CPC|Action: an initial belief  $b_t$ , formed from a CNN-encoded observation  $z_t$  and the previous action  $a_{t-1}$ , is rolled forward through future actions  $a_t, \dots, a_{t+T-1}$  to produce action-conditioned beliefs, compared against true future embeddings  $z^+$  and random negative embeddings  $z^-$  via small MLP classifiers ( $\oplus / \ominus$ ). An evaluation MLP separately probes  $b_t$  for  $(x, y, \theta)$ , with gradients blocked (shown by the crossed arrow) from flowing back into the belief. (b) Frame prediction: the belief  $b_t$  and current action  $a_t$  feed a deconvolutional decoder that reconstructs the next raw observation  $o_{t+1}$  directly, with the same blocked-gradient evaluation probe shown above. Adapted from Guo et al. [108]. . . . . CLXIII

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| A.39. Overall information flow. A CNN encodes each observation into $z_t$ , which the belief GRU combines with the previous action $a_{t-1}$ to produce the belief state $b_t$ . The belief drives a future prediction model trained with a CPC or frame-prediction loss, conditioned on the current or future action, while a separately trained evaluation network probes the frozen $b_t$ against the true position, orientation, object locations, and past trajectory, without updating $b_t$ itself. . . . . | CLXIV |
| F.1. An example of computation graph . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | CXCI  |

## Bibliography

- [1] P. Hernandez-Leal, M. Kaisers, T. Baarslag, and E. M. de Cote, “A survey of learning in multiagent environments: Dealing with non-stationarity,” *arXiv preprint arXiv:1707.09183*, 2017.
- [2] K. Zhang, Z. Yang, and T. Başar, “Multi-agent reinforcement learning: A selective overview of theories and algorithms,” in *Handbook on Reinforcement Learning and Control*, Springer, 2021, pp. 321–384.
- [3] J. Moos, K. Hansel, H. Abdulsamad, S. Stark, D. Clever, and J. Peters, “Robust reinforcement learning: A review of foundations and recent advances,” *Machine Learning and Knowledge Extraction*, vol. 4, no. 1, pp. 276–315, 2022.
- [4] C. Tessler, Y. Efroni, and S. Mannor, “Action robust reinforcement learning and applications in continuous control,” in *International Conference on Machine Learning*, PMLR, 2019, pp. 6215–6224.
- [5] F. Wu, L. Li, Z. Huang, Y. Vorobeychik, D. Zhao, and B. Li, “Crop: Certifying robust policies for reinforcement learning through functional smoothing,” in *International Conference on Learning Representations*, 2022.
- [6] G. N. Iyengar, “Robust dynamic programming,” *Mathematics of Operations Research*, vol. 30, no. 2, pp. 257–280, 2005.
- [7] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
- [8] A. Garivier and E. Moulines, “On upper-confidence bound policies for non-stationary bandit problems,” in *International Conference on Algorithmic Learning Theory*, Springer, 2011, pp. 174–188.
- [9] L. Li, R. Yang, and D. Luo, “Focal: Efficient fully-offline meta-reinforcement learning via distance metric learning and behavior regularization,” in *International Conference on Learning Representations*, 2021.
- [10] V. Mnih, K. Kavukcuoglu, D. Silver, *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [11] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3-4, pp. 229–256, 1992.

- [12] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016. [Online]. Available: <https://arxiv.org/abs/1506.02438>.
- [13] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *International conference on machine learning*, PMLR, 2015, pp. 1889–1897.
- [14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [15] V. Mnih, A. P. Badia, M. Mirza, *et al.*, “Asynchronous methods for deep reinforcement learning,” in *Proceedings of The 33rd International Conference on Machine Learning*, M. F. Balcan and K. Q. Weinberger, Eds., ser. Proceedings of Machine Learning Research, vol. 48, New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 1928–1937. [Online]. Available: <https://proceedings.mlr.press/v48/mniha16.html>.
- [16] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic policy gradient algorithms,” in *Proceedings of the 31st International Conference on Machine Learning*, E. P. Xing and T. Jebara, Eds., ser. Proceedings of Machine Learning Research, vol. 32, Beijing, China: PMLR, 22–24 Jun 2014, pp. 387–395. [Online]. Available: <https://proceedings.mlr.press/v32/silver14.html>.
- [17] T. P. Lillicrap, J. J. Hunt, A. Pritzel, *et al.*, “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2015.
- [18] S. Fujimoto, H. Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International conference on machine learning*, PMLR, 2018, pp. 1587–1596.
- [19] T. Haarnoja, A. Zhou, K. Hartikainen, *et al.*, “Soft actor-critic algorithms and applications,” *arXiv preprint arXiv:1812.05905*, 2018.
- [20] S. V. Albrecht, F. Christianos, and L. Schäfer, *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. [Online]. Available: <https://www.mar1-book.com>.
- [21] M. L. Littman, “Markov games as a framework for multi-agent reinforcement learning,” in *Machine learning proceedings 1994*, Elsevier, 1994, pp. 157–163.
- [22] J. Hu and M. P. Wellman, “Nash q-learning for general-sum stochastic games,” *Journal of machine learning research*, vol. 4, no. Nov, pp. 1039–1069, 2003.
- [23] A. Greenwald, K. Hall, R. Serrano, *et al.*, “Correlated q-learning,” in *ICML*, vol. 3, 2003, pp. 242–249.

- [24] R. J. Aumann, "Subjectivity and correlation in randomized strategies," *Journal of Mathematical Economics*, vol. 1, no. 1, pp. 67–96, 1974.
- [25] H. Moulin and J.-P. Vial, "Strategically zero-sum games: The class of games whose completely mixed equilibria cannot be improved upon," *International Journal of Game Theory*, vol. 7, no. 3-4, pp. 201–221, 1978.
- [26] S. Hart and A. Mas-Colell, "A general class of adaptive strategies," *Journal of Economic Theory*, vol. 98, no. 1, pp. 26–54, 2001.
- [27] P. Sunehag, G. Lever, A. Gruslys, *et al.*, "Value-decomposition networks for cooperative multi-agent learning," *arXiv preprint arXiv:1706.05296*, 2017.
- [28] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020.
- [29] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [30] C. Yu, A. Velu, E. Vinitsky, *et al.*, "The surprising effectiveness of ppo in cooperative multi-agent games," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [31] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, 2018.
- [32] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.
- [33] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.
- [34] J. Lin, K. Dzeparoska, S. Q. Zhang, A. Leon-Garcia, and N. Papernot, "On the robustness of cooperative multi-agent reinforcement learning," in *2020 IEEE Security and Privacy Workshops (SPW)*, 2020, pp. 62–68. doi: 10.1109/SPW50608.2020.00027.
- [35] R. Wiyatno and A. Xu, "Maximal jacobian-based saliency map attack," *arXiv preprint arXiv:1808.07945*, 2018.
- [36] S. Li, Y. Wu, X. Cui, H. Dong, F. Fang, and S. Russell, "Robust multi-agent reinforcement learning via minimax deep deterministic policy gradient," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, 2019, pp. 4213–4220.

- [37] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, “Robust adversarial reinforcement learning,” in *International conference on machine learning*, PMLR, 2017, pp. 2817–2826.
- [38] S. Li, R. Xu, J. Xiu, *et al.*, “Robust multi-agent reinforcement learning by mutual information regularization,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 10, pp. 18 118–18 132, 2025. doi: 10.1109/TNNLS.2025.3577259.
- [39] A. Bukharin, Y. Li, Y. Yu, *et al.*, “Robust multi-agent reinforcement learning via adversarial regularization: Theoretical foundation and stable algorithms,” *Advances in neural information processing systems*, vol. 36, pp. 68 121–68 133, 2023.
- [40] H. Zhang, H. Chen, C. Xiao, *et al.*, “Robust deep reinforcement learning against adversarial perturbations on state observations,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 21 024–21 037, 2020.
- [41] T. Oikarinen, W. Zhang, A. Megretski, L. Daniel, and T.-W. Weng, “Robust deep reinforcement learning through adversarial loss,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 26 156–26 167, 2021.
- [42] C. Qu, C. Yeh, K. Panaganti, E. Mazumdar, and A. Wierman, “Distributionally robust cooperative multi-agent reinforcement learning with value factorization,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=2T3L0pqI00>.
- [43] J. Du, Z. Kong, A. Sun, *et al.*, “Maddpg-based joint service placement and task offloading in mec empowered air-ground integrated networks,” *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 10 600–10 615, 2024. doi: 10.1109/JIOT.2023.3326820.
- [44] N. Tishby, F. C. Pereira, and W. Bialek, “The information bottleneck method,” *arXiv preprint physics/0004057*, 2000.
- [45] S. Chen, G. Liu, Z. Zhou, K. Zhang, and J. Wang, “Robust multi-agent reinforcement learning method based on adversarial domain randomization for real-world dual-uav cooperation,” *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 1615–1627, 2024. doi: 10.1109/TIV.2023.3307134.
- [46] O. Besbes, Y. Gur, and A. Zeevi, “Stochastic multi-armed-bandit problem with non-stationary rewards,” *Advances in neural information processing systems*, vol. 27, 2014.

- [47] D. Agudelo-España, S. Gomez-Gonzalez, S. Bauer, B. Schölkopf, and J. Peters, “Bayesian online prediction of change points,” in *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence (UAI)*, J. Peters and D. Sontag, Eds., ser. Proceedings of Machine Learning Research, vol. 124, PMLR, Mar. 2020, pp. 320–329. [Online]. Available: <https://proceedings.mlr.press/v124/agudelo-espana20a.html>.
- [48] P. Auer, T. Jaksch, and R. Ortner, “Near-optimal regret bounds for reinforcement learning,” *Advances in neural information processing systems*, vol. 21, 2008.
- [49] R. Alami, M. Mahfoud, and E. Moulines, “Restarted bayesian online change-point detection for non-stationary markov decision processes,” in *Conference on Lifelong Learning Agents*, PMLR, 2023, pp. 715–744.
- [50] B. C. Da Silva, E. W. Basso, A. L. Bazzan, and P. M. Engel, “Dealing with non-stationary environments using context detection,” in *Proceedings of the 23rd international conference on Machine learning*, 2006, pp. 217–224.
- [51] P. Gajane, R. Ortner, and P. Auer, “A sliding-window algorithm for markov decision processes with arbitrarily changing rewards and transitions,” *arXiv preprint arXiv:1805.10066*, 2018.
- [52] W. C. Cheung, D. Simchi-Levi, and R. Zhu, “Reinforcement learning for non-stationary markov decision processes: The blessing of (more) optimism,” in *International conference on machine learning*, PMLR, 2020, pp. 1843–1854.
- [53] A. Raghu, M. Raghu, S. Bengio, and O. Vinyals, “Rapid learning or feature reuse? towards understanding the effectiveness of maml,” *arXiv preprint arXiv:1909.09157*, 2019.
- [54] L. Zintgraf, K. Shiarli, V. Kurin, K. Hofmann, and S. Whiteson, “Fast context adaptation via meta-learning,” in *Proceedings of the 36th International Conference on Machine Learning*, K. Chaudhuri and R. Salakhutdinov, Eds., ser. Proceedings of Machine Learning Research, vol. 97, PMLR, Sep. 2019, pp. 7693–7702. [Online]. Available: <https://proceedings.mlr.press/v97/zintgraf19a.html>.
- [55] D. Almeida, C. Winter, J. Tang, and W. Zaremba, “A generalizable approach to learning optimizers,” *arXiv preprint arXiv:2106.00958*, 2021.
- [56] E. Park and J. B. Oliva, “Meta-curvature,” *Advances in neural information processing systems*, vol. 32, 2019.
- [57] S. Flennerhag, A. A. Rusu, R. Pascanu, F. Visin, H. Yin, and R. Hadsell, “Meta-learning with warped gradient descent,” *arXiv preprint arXiv:1909.00025*, 2019.



- [58] T. Kim, J. Yoon, O. Dia, S. Kim, Y. Bengio, and S. Ahn, “Bayesian model-agnostic meta-learning,” *arXiv preprint arXiv:1806.03836*, 2018.
- [59] Z. Wang, Y. Zhao, P. Yu, R. Zhang, and C. Chen, “Bayesian meta sampling for fast uncertainty adaptation,” in *International Conference on Learning Representations*, 2020.
- [60] A. Fallah, K. Georgiev, A. Mokhtari, and A. Ozdaglar, “On the convergence theory of debiased model-agnostic meta-reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 3096–3107, 2021.
- [61] H. Liu, R. Socher, and C. Xiong, “Taming maml: Efficient unbiased meta-reinforcement learning,” in *International conference on machine learning*, PMLR, 2019, pp. 4061–4071.
- [62] Y. Tang, “Biased gradient estimate with drastic variance reduction for meta reinforcement learning,” in *International Conference on Machine Learning*, PMLR, 2022, pp. 21 050–21 075.
- [63] B. C. Stadie, G. Yang, R. Houthoofd, *et al.*, “Some considerations on learning to explore via meta-reinforcement learning,” *arXiv preprint arXiv:1803.01118*, 2018.
- [64] J. Rothfuss, D. Lee, I. Clavera, T. Asfour, and P. Abbeel, “Promp: Proximal meta-policy search,” *arXiv preprint arXiv:1810.06784*, 2018.
- [65] A. Gupta, R. Mendonca, Y. Liu, P. Abbeel, and S. Levine, “Meta-reinforcement learning of structured exploration strategies,” *Advances in neural information processing systems*, vol. 31, 2018.
- [66] M. Al-Shedivat, T. Bansal, Y. Burda, I. Sutskever, I. Mordatch, and P. Abbeel, “Continuous adaptation via meta-learning in nonstationary and competitive environments,” *arXiv preprint arXiv:1710.03641*, 2017.
- [67] A. Ghadirzadeh, X. Chen, P. Poklukar, C. Finn, M. Björkman, and D. Kragic, “Bayesian meta-learning for few-shot policy adaptation across robotic platforms,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2021, pp. 1274–1280.
- [68] H. Zou, T. Ren, D. Yan, H. Su, and J. Zhu, “Learning task-distribution reward shaping with meta-learning,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, 2021, pp. 11 210–11 218.
- [69] Y. Tang, T. Kozuno, M. Rowland, R. Munos, and M. Valko, “Unifying gradient estimators for meta-reinforcement learning via off-policy evaluation,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 5303–5315, 2021.

- [70] R. Vuorio, J. A. Beck, G. Farquhar, J. N. Foerster, and S. Whiteson, “No DICE: An investigation of the bias-variance tradeoff in meta-gradients,” in *Deep RL Workshop NeurIPS 2021*, 2021. [Online]. Available: <https://openreview.net/forum?id=WB2dwqWZ6kU>.
- [71] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *International conference on machine learning*, PMLR, 2017, pp. 1126–1135.
- [72] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, “ $\text{RL}^2$ : Fast reinforcement learning via slow reinforcement learning,” *arXiv preprint arXiv:1611.02779*, 2016.
- [73] J. X. Wang, Z. Kurth-Nelson, D. Tirumala, *et al.*, “Learning to reinforcement learn,” *arXiv preprint arXiv:1611.05763*, 2016.
- [74] J. Beck, M. T. Jackson, R. Vuorio, and S. Whiteson, “Hypernetworks in meta-reinforcement learning,” in *Conference on robot learning*, PMLR, 2023, pp. 1478–1487.
- [75] J. Subramanian, A. Sinha, R. Seraj, and A. Mahajan, “Approximate information state for approximate planning and reinforcement learning in partially observed systems,” *Journal of Machine Learning Research*, vol. 23, no. 12, pp. 1–83, 2022.
- [76] L. Zintgraf, K. Shiarlis, M. Igl, *et al.*, “Varibad: A very good method for bayes-adaptive deep rl via meta-learning,” *arXiv preprint arXiv:1910.08348*, 2019.
- [77] K. Rakelly, A. Zhou, C. Finn, S. Levine, and D. Quillen, “Efficient off-policy meta-reinforcement learning via probabilistic context variables,” in *Proceedings of the 36th International Conference on Machine Learning*, K. Chaudhuri and R. Salakhutdinov, Eds., ser. Proceedings of Machine Learning Research, vol. 97, PMLR, Sep. 2019, pp. 5331–5340.
- [78] Z. Zhao, Y. Fu, J. Chai, Y. Zhu, and D. Zhao, “Meta learning task representation in multiagent reinforcement learning: From global inference to local inference,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 8, pp. 14 908–14 921, 2025. doi: 10.1109/TNNLS.2025.3540758.
- [79] Y. Wang and S. Zou, “Online robust reinforcement learning with model uncertainty,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021.
- [80] Z. U. Farhat, D. Ghosh, G. K. Atia, and Y. Wang, “Sample-efficient distributionally robust multi-agent reinforcement learning via online interaction,” *arXiv preprint arXiv:2508.02948*, 2025.

- [81] J. Oh, H. Yoo, C. Kim, and S.-Y. Yun, “Boil: Towards representation change for few-shot learning,” *arXiv preprint arXiv:2008.08882*, 2020.
- [82] S. Whiteson, “Fast context adaptation via meta- learning,” 2019.
- [83] Z. Li, F. Zhou, F. Chen, and H. Li, “Meta-sgd: Learning to learn quickly for few-shot learning,” *arXiv preprint arXiv:1707.09835*, 2017.
- [84] M. Andrychowicz, M. Denil, S. Gomez, *et al.*, “Learning to learn by gradient descent by gradient descent,” *Advances in neural information processing systems*, vol. 29, 2016.
- [85] T. Chen, X. Chen, W. Chen, *et al.*, “Learning to optimize: A primer and a benchmark,” *Journal of Machine Learning Research*, vol. 23, no. 189, pp. 1–59, 2022.
- [86] J. Foerster, G. Farquhar, M. Al-Shedivat, T. Rocktäschel, E. Xing, and S. Whiteson, “Dice: The infinitely differentiable monte carlo estimator,” in *International Conference on Machine Learning*, PMLR, 2018, pp. 1529–1538.
- [87] G. Farquhar, S. Whiteson, and J. Foerster, “Loaded dice: Trading off bias and variance in any-order score function gradient estimators for reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [88] N. Heess, J. J. Hunt, T. P. Lillicrap, and D. Silver, “Memory-based control with recurrent neural networks,” *arXiv preprint arXiv:1512.04455*, 2015.
- [89] N. Mishra, M. Rohaninejad, X. Chen, and P. Abbeel, “A simple neural attentive meta-learner,” *arXiv preprint arXiv:1707.03141*, 2017.
- [90] L. C. Melo, “Transformers are meta-reinforcement learners,” in *international conference on machine learning*, PMLR, 2022, pp. 15 340–15 359.
- [91] J. Oh, V. Chockalingam, H. Lee, *et al.*, “Control of memory, active perception, and action in minecraft,” in *International conference on machine learning*, PMLR, 2016, pp. 2790–2799.
- [92] M. Fortunato, M. Tan, R. Faulkner, *et al.*, “Generalization of reinforcement learners with working and episodic memory,” *Advances in neural information processing systems*, vol. 32, 2019.
- [93] S. Ritter, R. Faulkner, L. Sartran, A. Santoro, M. Botvinick, and D. Raposo, “Rapid task-solving in novel environments,” *arXiv preprint arXiv:2006.03662*, 2020.
- [94] T. Miconi, J. Clune, and K. O. Stanley, “Differentiable plasticity: Training plastic neural networks with backpropagation,” *arXiv preprint arXiv:1804.02464*, 2018.
- [95] T. Miconi, A. Rawal, J. Clune, and K. O. Stanley, “Backpropamine: Training self-modifying neural networks with differentiable neuromodulated plasticity,” *arXiv preprint arXiv:2002.10585*, 2020.

- [96] K. Irie, I. Schlag, R. Csordás, and J. Schmidhuber, “A modern self-referential weight matrix that learns to modify itself,” in *International conference on machine learning*, PMLR, 2022, pp. 9660–9677.
- [97] S. R. R. Rohani, S. Hedayatian, and M. S. Baghshah, “Bimrl: Brain inspired meta reinforcement learning,” in *2022 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, IEEE, 2022, pp. 9048–9053.
- [98] M. Chalvidal, T. Serre, and R. VanRullen, “Meta-reinforcement learning with self-modifying networks,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 7838–7851, 2022.
- [99] E. Najarro and S. Risi, “Meta-learning through hebbian plasticity in random networks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 20 719–20 731, 2020.
- [100] T. Imagawa, T. Hiraoka, and Y. Tsuruoka, “Off-policy meta-reinforcement learning with belief-based task inference,” *IEEE Access*, vol. 10, pp. 49 494–49 507, 2022. doi: 10.1109/ACCESS.2022.3170582.
- [101] E. Z. Liu, A. Raghunathan, P. Liang, and C. Finn, “Explore then execute: Adapting without rewards via factorized meta-reinforcement learning,” *arXiv preprint arXiv:2008.02790*, 2020.
- [102] J. Zhang, J. Wang, H. Hu, *et al.*, “Metacure: Meta reinforcement learning with empowerment-driven exploration,” in *Proceedings of the 38th International Conference on Machine Learning*, M. Meila and T. Zhang, Eds., ser. Proceedings of Machine Learning Research, vol. 139, PMLR, 18–24 Jul 2021, pp. 12 600–12 610. [Online]. Available: <https://proceedings.mlr.press/v139/zhang21w.html>.
- [103] H. Fu, H. Tang, J. Hao, *et al.*, “Towards effective context for meta-reinforcement learning: An approach based on contrastive learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 7457–7465.
- [104] F.-M. Luo, S. Jiang, Y. Yu, Z. Zhang, and Y.-F. Zhang, “Adapt to environment sudden changes by learning a context sensitive policy,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 36, 2022, pp. 7637–7646.
- [105] R. Raileanu, M. Goldstein, A. Szlam, and R. Fergus, “Fast adaptation to new environments via policy-dynamics value functions,” in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 7920–7931.
- [106] M. Peng, B. Zhu, and J. Jiao, “Linear representation meta-reinforcement learning for instant adaptation,” *arXiv preprint arXiv:2101.04750*, 2021.

- [107] I. Korshunova, J. Degraeve, J. Dambre, A. Gretton, and F. Huszar, “Exchangeable models in meta reinforcement learning,” in *4th Lifelong Machine Learning Workshop at ICML 2020*, 2020. [Online]. Available: <https://openreview.net/forum?id=TZFlzejFPT>.
- [108] Z. D. Guo, M. G. Azar, B. Piot, B. A. Pires, and R. Munos, “Neural predictive belief representations,” *arXiv preprint arXiv:1811.06407*, 2018.
- [109] J. Schulman, N. Heess, T. Weber, and P. Abbeel, “Gradient estimation using stochastic computation graphs,” *Advances in neural information processing systems*, vol. 28, 2015.
- [110] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 9726–9735. doi: 10.1109/CVPR42600.2020.00975.



# Declaration Of Authorship

I, Abdullah Alalwani, hereby affirm, that I have prepared the present work without the unauthorized help of third parties and without the use of any aids other than those specified. The data and concepts taken directly or indirectly from sources are marked with a reference to the source. This thesis has not been submitted to an examination board in the same or a similar form, or published in any other way, either at home or abroad. I have documented the use of all generative AI tools authorized for the production of this thesis in the appendix of this thesis in tabular form by naming the tool and the purpose for which it was used. All verbatim or paraphrased content from generative AI tools has been marked accordingly. I am aware that any attempt at undocumented use of generative AI tools will be considered an attempt to cheat under the examination regulations. I confirm that I have not adopted content generated using AI tools without reflection and that, as the author, I am responsible for the information and statements in this thesis.

Ilmenau, [FILL IN]

---

Abdullah Alalwani